Nucleus Networking

Version v1

Generated 2026-11-07 13:51:23 UTC

Documentation for Nucleus, a .NET networking engine with a Unity integration.

Contents

Client state 10
Controller retention 15
Reading who controls a system 17
Control, Client State and Prediction 19
Predicted members and prediction history 20
Reconciliation and replay 24
What control means 33
Assigning and releasing control 35
When prediction is worth its cost 36
Authenticating clients 38
Connections 41
Creating a CoreManager 44
Driving the loop yourself 46
Host mode 49
Core, Managers and the Tick Loop 51
Kicking and disconnecting a client 52
The manager map 54
The network loop steps 55
Object pacing 58
Keeping the world across a disconnect 60
Server, client and host roles 62
Kick and refusal notices 64
Session events 66
Shutting a session down 68
Threading and lifetime 69
Ticks, tick rate and time 70
Common errors and what causes them 72
Diagnosing desync 75
Frequently asked questions 77
Filing a bug report 78
Diagnostics and Troubleshooting 79
Reading serialization history (and the raw-packet test hook) 80
Round-trip time, jitter and packet loss 82
Logging 83
Network statistics 85
Securing a server 86
Technical limitations 93
Violation reference 94
Violations 99

Contents

What the server trusts 101
Core API 103
What area of interest is 104
Distance interest conditions 105
The host interest report 106
Interest Management and Scaling 107
How distances are measured: pairwise and the spatial grid 108
An object is missing, or will not go away 109
Registering interest conditions 111
Stopping versus culling 113
Supplying positions to the interest system 116
What scales and what does not 117
Writing an interest condition 119
Capturing a shot 122
How lag compensation works 124
Lag Compensation 126
Registering hitboxes 127
Judging a shot with rewound queries 130
The rewind window and the view clamp 134
Channels and What Is Guaranteed 136
Messaging and Remote Calls 137
When a Message or Call Reaches Its Handler 138
Messages and Calls That Do Not Arrive 139
Handler Registration and Scope 141
Judging and Refusing a Relayed Call 142
Seeing Your Own Call: RpcSelfDelivery 143
Who May Send a Call, and How Many 145
Choosing Who Receives a Call 147
Sending a Message to Many Peers 148
State, a Message, or a Call? 149
System RPCs 150
Typed Messages 152
Adapting your engine 154
How physics replication works 156
Physics Synchronization 157
Locally predicted bodies 158
NetworkPhysicsComponent 160
PhysicsConvergence 162
Replicate a physics body 164

Contents

Swept projection targets 167
Trajectory projection 169
The content bundle protocol 171
Networked Scenes, Content Bundles and Persistence 173
Moving and carrying objects between scenes 174
Opening and closing scene instances 176
Placing clients in scenes 178
Scene events and observers 180
Refusing a scene load 183
Registering a scene loader 184
How networked scenes work 185
Scene visibility: what a client in a scene can see 187
Scenes and bundles: things that look broken 189
Saving and loading a world 191
Choosing what a save keeps 193
Writing a world store 195
Reacting to changes: OnMembersChanged 198
Collection member reference 199
Sending your own data beside the members 203
Replicated State and Change Callbacks 206
What a member can hold, and how to keep one off the wire 207
NetworkMember 209
Networked collections 214
Retention, redundancy and recovery 216
How Often a Member Is Sent 218
How replicated state works 220
Letting clients set members 221
TransformComponent 223
Judging and arbitrating client writes 224
Your first networked component 226
Systems, Objects and Spawning 228
NetworkSystem reference 229
References between systems 232
Predicted spawn and despawn requests 233
Renting and finding systems in code 237
Spawn compensation 239
Binding spawns to your own engine 241
Starting and stopping systems 243
System groups 246

Contents

Parents, children and hierarchy 248
What a system is 249
Configuring a transport from launch arguments 250
Connecting across networks 252
Link states and connection events 254
Running a dedicated .NET server 257
Choosing a hosting topology 260
How transports work 262
Transports, Connectivity and Hosting 263
Packet transforms 264
The relay transport 266
Simulating latency, jitter and loss 269
The Synapse transport 271
The Transport contract 274
TransportManager 276
The Yak offline transport 278
Compression, Accuracy and Per-Type Registrations 279
Writing a Custom Serializer 280
Reaching Types You Cannot Annotate 283
Generator Errors and Warnings 284
Serialization, Codegen and the Wire 286
Networked Type Attributes 287
Wiring the Source Generator into a .NET Project 288
Type Identity on the Wire: Bundles and the Registry 289
Writer and Reader 291
Nucleus Networking Documentation 295
API type reference 296
How the benchmarks are run 298
CodeBoost reference 313
Compatibility 315
Component Index 316
Configuration reference 320
Editor menus and authoring tools 325
FishNet to Nucleus name map 328
Reference & Project Meta 330
Installing and updating Nucleus in Unity 331
Project assets and loaders 333
Reporting a bug 336
Versioning, upgrading and release notes 337

Contents

Violation reference 338
Add Nucleus to a .NET project 342
Start Here 343
Install Nucleus in Unity 344
If nothing replicates 345
Add a player 346
Replicate a value 348
Run two peers in one process 350
Terminology 352
Test with two editors 353
What Nucleus is 354
Where to go next 355
Your first .NET host 356
Set up a networked scene 359
Control in a Unity script 360
Keeping a player's objects across a disconnect 362
Smoothing a predicted object with GraphicalSmoother 363
Control, Client State and Prediction 365
Prediction settings on the Unity manager 366
Objects this peer does not control 368
Replaying a correction in Unity 369
What control means 375
Giving a client control of an object 377
Using prediction, from start to finish 378
When prediction is worth its cost 386
Sending client state from Unity 388
Host mode 393
Core, Managers and the Tick Loop 394
The manager map 395
The network loop steps 396
The Nucleus Core Manager component 398
Manager components reference 399
Play mode, domain reload and session lifetime 401
Reacting to session events in Unity 402
Starting and stopping a session in Unity 404
Tick rate and the loop in Unity 406
Common errors and what causes them 408
Developer tools in the editor 410
Diagnosing desync 414

Contents

Frequently asked questions 416
Filing a bug report 417
Handling violations in Unity 418
Diagnostics and Troubleshooting 420
Engine logging in the Unity console 421
Reading network statistics in Unity 422
Securing a server in Unity 425
Technical limitations 430
Violation reference 431
What the server trusts 436
Unity 438
What area of interest is 439
Making a host see what its players see 440
Interest Management and Scaling 442
An object is missing, or will not go away 443
Limit what a client receives by distance 445
Network Interest Object 446
Frozen or gone: what a stop does to the object 447
Unity Interest Manager 448
What scales and what does not 450
Clipping shots at walls 452
Firing and judging a shot 454
Worked example: the HitscanRewind demo 459
How lag compensation works 461
Lag Compensation 462
Making an object hittable 463
UnityLagCompensationManager 466
Registering Call Handlers Across a Component's Lifetime 469
Channels and What Is Guaranteed 471
Messaging and Remote Calls 472
When a Message or Call Reaches Its Handler 473
Typed Messages in Unity 474
Messages and Calls That Do Not Arrive 476
Sending a Remote Call From a Component 478
State, a Message, or a Call? 480
Driving a predicted body 481
How physics replication works 483
Physics Synchronization 484
Local physics scenes 485

Contents

NetworkNavMeshAgent 486
NetworkRigidbody 488
Physics execution modes 491
PhysicsSimulationDriver 492
Physics troubleshooting 494
Predicting a body with a validated behaviour 496
Replicate a rigidbody 502
NetworkTransform or NetworkRigidbody 503
Addressables-delivered scenes 504
Content bundles in Unity 505
Networked Scenes, Content Bundles and Persistence 507
The network scene manifest and scene references 508
Networked scenes in Unity 510
Objects authored into a scene 511
UnitySceneLoader component 512
UnitySceneManager component 514
Saving and loading a world from Unity 516
Replicated State and Change Callbacks 518
NetworkAnimator 519
NetworkTransform 522
Reacting to a replicated change in Unity 524
Replicating a value from a Unity script 526
How replicated state works 528
Several systems on one object 529
Systems, Objects and Spawning 531
NetworkSystemObject component 532
NucleusBehaviour: the script base class 535
Player objects 539
Client-predicted spawning in Unity 541
Prefab identity and the prefab collection 544
Pooling spawned objects 546
Carrying and re-parenting objects 548
Spawning and despawning objects 550
What a system is 553
Blitz Relay Transport component 554
Building a headless Unity server 556
Transports, Connectivity and Hosting 558
Hosting and joining a relay session 559
Simulating a bad connection in the editor 561

Contents

Synapse Transport component 564
Transport Manager component 566
Bundle Ids for Unity Assemblies 568
Code Generation in Unity 569
Generator Errors and Warnings 570
Serialization, Codegen and the Wire 572
Unity Types and System.Numerics 573

Client state

> **Using Unity?** See Sending client state from Unity.

Client state is what the controlling client sends the server every tick so the server can simulate the object: a move direction, a jump press, an aim angle. The server runs the same step from those values and replicates the result, so it only trusts what the player asked for, never where the player says it ended up.

Declare client state as Predict members

Client state is ordinary `NetworkMember<T0>` fields constructed with `role: MemberRole.Predict`, declared in the system's own `NetworkComponent` beside the members the simulation computes from them. There is no separate class to write and nothing to register.

```
using System.Numerics;
using Nucleus.Components;
using Nucleus.Connections;
using Nucleus.Managers.NetworkLoop;
using Nucleus.Systems;

public partial class MoverComponent : NetworkComponent, IMemberWriteValidator<Vector2>
{
    // Client state: written by the controller on its live tick, sent upstream every tick.
    public readonly NetworkMember<Vector2> MoveDirection = new(role: MemberRole.Predict);
    // A press that must not repeat when a missing tick is carried forward.
    public readonly NetworkMember<bool> JumpPressed = new(role: MemberRole.Predict, isCarr
iedForward: false);
    // A member the server computes from the client state.
    public readonly NetworkMember<Vector3> Position = new();

    // Set by the game from the device on the controlling client.
    public Vector2 IntendedDirection;

    private const float Speed = 5f;

    // Server: once per Predict member per client tick taken. Write the member and decline
 to keep your own value.
    public MemberWriteAction ValidateMemberWrite(NetworkMember<Vector2> networkMember, Vec
tor2 proposedValue, Connection writingClientConnection)
    {
        if (proposedValue.LengthSquared() <= 1f)
            return MemberWriteAction.Accept;

        MoveDirection.Value = Vector2.Normalize(proposedValue);

        return MemberWriteAction.Decline;
    }

    protected override void OnEarlyFixedUpdate(StepDelta stepDelta, SimulationContext simu
lationContext)
    {
        // A client watching a system it does not control: nothing to simulate.
        if (simulationContext.Type is SimulationType.None)
            return;

        // The controlling client's live tick: this becomes the tick's client state.
        if (simulationContext.Type is SimulationType.Forward && NetworkSystem.IsController
(ControllerType.Client))
            MoveDirection.Value = IntendedDirection;

        // The server's step for a tick whose client state did not arrive: carry the last
one forward, leaving the press at its default.
        if (stepDelta.IsClientStateMissing)
            TryGetLastState(isEveryMemberCarried: false);

        if (!CoreManager.TransportManager.IsServerStarted)
            return;

        Position.Value += new Vector3(MoveDirection.Value.X, 0f, MoveDirection.Value.Y) *
(Speed / CoreManager.NetworkLoopManager.TickRate);
    }
```

```
}
```

`Position` here is an ordinary member the server computes, so the controller sees its own movement a round trip late. To have the controller move at once and be corrected only when it is wrong, make the result `MemberRole.Validate` and let both peers run the step; see Predicted members and prediction history.

Write it in the fixed update on the controller's live tick

- Whoever controls the system writes its Predict members: the controlling client on its live tick (`SimulationType.Forward`), or the server for a system it controls.
- Write them inside the fixed update, from values the game gathered between ticks, so every pass of a tick reads the same numbers.
- A replay never writes them. It reads each past tick's value from the member's history.

A pure client sends its Predict members to the server every tick, stamped with its tick. A lost tick of client state never breaks the ticks after it.

A host playing its own object writes the members in the same process, so nothing travels upstream. Each tick it still puts their values to the same validator a remote controller faces, and a declined value is reported to it as a correction.

Validate client state on the server

Implement `IMemberWriteValidator<T0>` on the component, once per member type. The server calls it once per Predict member for each client tick it takes:

```
MemberWriteAction ValidateMemberWrite(NetworkMember<T0> networkMember, T0 proposedValue, C
onnection writingClientConnection);
```

- `MemberWriteAction.Accept` takes the client's value.
- `MemberWriteAction.Decline` keeps what the member already holds.
- To clamp rather than refuse, write the member inside the call and decline. The server keeps your value, simulates with it and forwards it.
- A validator that throws declines.

Several members of one type share one implementation, so compare the `networkMember` argument against your fields when they need different rules. The same interface judges a client's writes to ordinary members; see Validating client writes.

The server takes one tick of client state per server tick

The server takes one of the controller's ticks per server tick, released through the State Interpolation like any other state, so a burst that arrives together is spread over the ticks that follow. For each tick it runs the validator, then steps the system with its Predict members holding that tick's values.

The controller's `Connection` counts the ticks that went wrong:

```
public uint ClientStateDroppedCount { get; }
public uint ClientStateLateCount { get; }
public uint ClientStateReanchorCount { get; }
```

- **Dropped**: a tick the server could not take in time.
- **Late**: a tick that arrived after its tick was simulated.
- **Re-anchored** : a client that stalled and resumed, whose client state arrived further behind the ticks the server already simulated for it than the State Interpolation. The server re-bases onto the client's ticks

and sends a reconcile for each system the client controls.

The server's step is stamped with the server's own tick in `StepDelta.Tick`. The controller's tick it simulates is in `SimulationContext.RemoteTick`, for pairing and logging only. A client chooses its own tick numbers, so never time a cooldown or anything else by it. See The simulation context.

Carry client state over ticks that did not arrive

A system a remote client drives is simulated on every server tick, whether or not that tick's client state arrived: before the first one, and while the client's packets are lost. That lets the server act on the object in its own world, for a collision for instance. On such a step `StepDelta.IsClientStateMissing` and `SimulationContext.IsRemoteTickEstimated` are true, and every Predict member reads its default. To carry the previous tick's client state forward instead, call the component's:

```
public bool TryGetLastState(bool isEveryMemberCarried)
```

- For the rest of the step, the Predict members read their values as they stood at the end of the previous tick: the client's own when that tick's arrived, and otherwise whatever the server's step left in them. A value the server writes into a member is what the next tick carries.
- Pass `false` to keep members constructed with `isCarriedForward: false` at their defaults, so a jump press never fires twice. A press lost with its packet is lost, and the server corrects the jump the client showed.
- It returns `false`, leaving the defaults, before any client state has arrived.
- Nothing is carried unless you call it, so a game that does not carry is reconciled back to what the server simulated.

A tick runs as missing only once the State Interpolation says it should have arrived. Just after the client's states stall, or with a client clock running slower than the server's, the next client tick runs when it arrives instead, so those server ticks pass the object by.

Stop a carry that has gone on too long

This overload also reports the simulation context of the client state you are carrying:

```
public bool TryGetLastState(bool isEveryMemberCarried, out SimulationContext lastStateSimu
lationContext)
```

On a missing step, `lastStateSimulationContext` is the context of the newest step this component's client state arrived for. On the server its `RemoteTick` is the controller's tick that state was written on and `IsRemoteTickEstimated` is false, so the missing step's own `RemoteTick` minus it is how many ticks you are carrying it over:

```
protected override void OnEarlyFixedUpdate(StepDelta stepDelta, SimulationContext simulati
onContext)
{
    if (stepDelta.IsClientStateMissing && TryGetLastState(isEveryMemberCarried: false, out
 SimulationContext lastStateSimulationContext))
    {
        // Stop walking once the client has been silent for more than five of its ticks.
        if (simulationContext.RemoteTick - lastStateSimulationContext.RemoteTick > 5)
            MoveDirection.Value = Vector2.Zero;
    }

    // Simulate from MoveDirection as usual.
}
```

On a step that is not missing, it is the context of the step running now. Before any client state has arrived, or on a component with no Predict member, it is the default, whose `RemoteTick` is unset. Compare these ticks, but never time anything on the server by them.

Every observer receives the client state

The server forwards the Predict values it kept, after validation and its own writes, to every observer of the system, the controller included. Observers can drive presentation (footsteps, animation) from them, and an observer that predicts the system simulates from them (see Predicting another client's object).

On the controller, a server value for a past tick that differs from its own lands in that tick's history, so a later replay of that tick reads what the server simulated. This includes values the server writes from its own step.

React to changes with OnMembersChanged

`OnMembersChanged` reports Predict members on change, not every tick:

- `MemberChangeDirection.Read` on the server when it takes a client value that differs (`TryGetWritingClient` names the sender), and on an observer receiving a changed value.
- `MemberChangeDirection.Correction` on the controller when the server's value for one of its past ticks differs from its own. The server's value is already in the member's history at that tick when the callback runs.

The simulation reads the members every tick in its fixed update anyway, so use the callback for presentation or to react to a correction.

Only the controller may send client state

Client state is controller-only, whatever the system's write access allows. Client state from any other connection is refused and raises an `UnauthorizedMemberChangeViolation`.

Controller retention

> **Using Unity?** See Keeping a player's objects across a disconnect for the inspector fields and `NetworkPlayerDisconnectMode`.

Control is always given up the moment a connection leaves. Connection ids are recycled, so an object still naming a departed controller would hand its write permission and client state routing to the next connection given that id. `ControllerRetentionPolicy` only decides whether the departing connection is issued a token so its objects can be handed back later.

Choose `Release` or `Retain`

- `Release` (0), the default: the object is uncontrolled once its controller leaves, and nothing is remembered about who held it.
- `Retain` (1): the object is uncontrolled, and it joins the retention record the departing connection is issued a token for, so a returning player can be handed it back.

`SystemManager.ResolveControllerRetentionPolicy(NetworkSystem)` returns a system's policy, taking the first that is set:

> `NetworkSystem.ControllerRetentionPolicyOverride`.
>
> The override on the system's group.
>
> `SystemManager.DefaultControllerRetentionPolicy` (`Release` by default).

Systems and groups leave their override unset by default.

Save the token on the client

When a connection is first given control of something retained, the server delivers it a token once, reliably, over `ControllerRetentionTokenNotice`. On the client, `SystemManager.LocalControllerRetentionToken` holds it and `SystemManager.LocalControllerRetentionTokenReceived` fires when it arrives. Persist it then, somewhere that outlives the session.

`ControllerRetentionToken` is a readonly struct with `Id`, `Generation` and `IsValid` (true when `Id` is not the unset value). It is meaningful only to the peer that issued it and is safe to store beside an account record. A stale token fails to resolve instead of naming somebody else's record.

Reclaim the objects with the token

```
public bool TryReclaimController(Connection connection, ControllerRetentionToken controlle
rRetentionToken)
```

`SystemManager.TryReclaimController` hands the connection every object the record holds and ends the record. It returns `false` for a token that does not resolve: one whose record expired, was evicted or was already redeemed.

It does not check that the returning peer is the same player. Prove that first, through whatever channel your authentication already trusts, and store the token against a server-determined identity rather than in place of one. A server that accepts a token straight off the wire with no identity check behind it trusts whatever a client sends.

Three events report a record's lifetime

- `ControllerRetained`: a record opened for a departing connection. Raised only when at least one object was retained.
- `ControllerRetentionEnded`: a record stopped being redeemable. Carries the token and a `ControllerRetentionEndReason`.
- `ControllerRetentionAdopted`: a record standing after this peer adopted a dead host's world. It only means anything under host migration, which is planned but not yet built.

Only a reclaim returns the objects

`ControllerRetentionEndReason` says why a record ended:

- `Reclaimed` (0): a returning connection redeemed the token and was handed the objects.
- `Expired` (1): the record outlived `SystemManager.ControllerRetentionSeconds`.
- `Evicted` (2): the record was the oldest standing when a new one needed room under `SystemManager.MaximumRetainedControllerRecords`.
- `Emptied` (3): every object in the record left the world or was handed to another controller.

Every reason but `Reclaimed` leaves the objects uncontrolled and unrelated to one another. To act on a record's objects as a set, such as despawning a player's belongings when they don't return, do it in `ControllerRetentionEnded`, before the reason takes effect.

Limit records by time and by count

- `SystemManager.ControllerRetentionSeconds` (default `DefaultControllerRetentionSeconds`, 120) is how long a record stays redeemable. `SystemManager.UnlimitedControllerRetentionSeconds` (0) disables expiry, leaving the record count as the only limit.
- `SystemManager.MaximumRetainedControllerRecords` (default `DefaultMaximumRetainedControllerRecords`, 64) is how many records may stand at once. Past that, the oldest is evicted, so the most recent departure stays redeemable.

Retention does not migrate a host

Retention lets a peer promoted after a host dies rebuild retained records from the objects it adopted. Migrating the host itself is covered in Surviving the loss of a host.

Reading who controls a system

Using Unity? See Control in a Unity script.

Check whether the local peer controls a system

```
if (networkSystem.IsController(ControllerType.AnyController))
{
    // The local peer is the server (with no controlling client) or the controlling client
.
}
```

`ControllerType` is a flags enum:

Value	Meaning
`None`	No controller.
`Server`	The local peer is the server and no client controls the system.
`Client`	The local peer is the client that controls the system.
`AnyController`	`Server \

Combine flags to test either case at once, as `AnyController` does.

Read who the controller is

- `ControllingClient` is the controlling client's `Connection`, or `null` when the server controls the system. On a client it can briefly be `null` while `ControllerConnectionId` names someone, because a controller change can arrive before the client is told about that peer. It fills in once the peer is reported.
- `ControllerConnectionId` is the controlling client's connection Id, or `NetworkSystem.UnsetId` when the server controls it. This is the identity that replicates to clients.

Compare controllers by Id, never by reference

A `Connection` is pooled and reused once its link ends, so a reference kept from an earlier lookup can later name a different peer. To ask whether the local peer controls a system, use `IsController` or `ControllingClient`. To compare against a known peer, compare Ids:

```
bool controls = networkSystem.ControllerConnectionId == someConnection.Id;
```

`IsController` and `TryGetControlledSystems` already compare this way.

React to control changes

```
networkSystem.ControllerChanged += OnControllerChanged;

void OnControllerChanged(Connection previousControllerConnection, Connection currentContro
llerConnection)
{
    // ...
}
```

The handler type is `ControllerChangedHandler`. Either `Connection` is `null` when control sits with the

server, or with a peer client this peer holds no `Connection` for.

Find what a connection controls

```
if (coreManager.InterestManager.TryGetControlledSystems(connection, out IReadOnlyCollectio
n<NetworkSystem> controlledSystems))
{
    foreach (NetworkSystem controlledSystem in controlledSystems)
    {
        // ...
    }
}
```

The overload `TryGetControlledSystems(uint connectionId, out IReadOnlyCollection<NetworkSystem> controlledSystems)` takes an Id instead of a `Connection`.

It returns every system the connection controls, in no particular order. The core has no concept of a player, so deciding which controlled system is the player is up to the game. The collection is a live view, not a copy: copy it before doing anything that can reassign a controller while you walk it.

Fill a list instead in code that runs often

A `foreach` over the view allocates an enumerator on every walk. In code that runs every tick or for every pair, such as an interest condition, fill a list you keep:

```
private readonly List<NetworkSystem> _controlledSystems = new();

private void CheckControlledSystems(CoreManager coreManager, Connection connection)
{
    int count = coreManager.InterestManager.GetControlledSystems(connection, _controlledSy
stems);

    for (int i = 0; i < count; i++)
    {
        NetworkSystem controlledSystem = _controlledSystems[i];
        // ...
    }
}
```

`GetControlledSystems` clears the list, fills it with every system the connection controls and returns the count. It allocates nothing once the list has grown to fit. Because it is a copy, control may be reassigned while you walk it.

Control, Client State and Prediction

Pages in this category:

- Client state
- Reading who controls a system
- Predicted members and prediction history
- Reconciliation and replay
- What control means
- Assigning and releasing control
- Controller retention
- When prediction is worth its cost

Predicted members and prediction history

Using Unity? See Prediction settings on the Unity manager.

A member's role says what it is for

`NetworkMember<T0>` takes a `role` constructor argument, `MemberRole.Replicate` by default, readable afterwards as `NetworkMember<T0>.Role`:

```
public readonly NetworkMember<System.Numerics.Vector2> MoveDirection = new(role: MemberRol
e.Predict);
public readonly NetworkMember<System.Numerics.Vector3> Position = new(role: MemberRole.Val
idate, accuracy: 0.01f);
public readonly NetworkMember<ushort> Health = new();
```

Role	What it is	Who writes it	What the controller sends upstream
`Replicate`	An ordinary member. Inbound values apply as they arrive.	The server, or a client its write access allows.	The value, when it changes and write access allows it
`Predict`	Client state: a per-tick value the simulation reads.	Whoever controls the system, on its live tick.	The value, every tick (see [Client state](./client-state-components.md))
`Validate`	A predicted result, computed from the client state.	Every peer that simulates the system.	Nothing the server applies. The server checks the controller's result for each tick against its own and orders a correction when they differ

Roles are per member, and one component can mix all three. The usual predicted object has `Predict` members for what the player asked, `Validate` members for what the step computes from them, and `Replicate` members for what only the server decides. Nothing turns prediction on for you.

What a predicted role does depends on the peer:

- **The pure-client controller** writes its `Predict` members on its live tick and runs the step at once to compute its `Validate` members.
- **The server** takes the controller's `Predict` members, simulates the same tick and checks the controller's `Validate` result against its own, without ever applying it.
- **Any other client** applies the server's values as they arrive. During a reconcile, a member with a role other than `Replicate` is held at the server's start-tick values while the controlled systems replay. A client that sets `NetworkSystem.IsObserverPredictionEnabled` predicts the system as its controller does (see Predicting another client's object).
- **A host** applies values as they arrive. It is the server, so nothing it predicts is ever corrected.

The server checks the controller's result

The controller runs its live fixed updates and computes that tick's `Validate` results.

The server simulates the same tick of client state and compares its `Validate` results with the controller's. On a difference it counts `Connection.ResultMismatchCount`, orders a reconcile (counted in `ReconcileSentCount`) and raises `NetworkSystem.ResultMismatched`; see Validate results the server found different.

Results the controller computed before the correction arrived are passed over rather than ordering a reconcile each. If the controller has not applied the correction within a round trip plus `StateInterpolation`, the server sends the reconcile again (`ReconcileResentCount`), so a lost reconcile costs one round trip. A resend that reaches a controller which already applied the first does not order a second replay.

The controller never judges its own prediction: server values that arrive for a predicted member leave the prediction standing, and only a reconcile the server orders corrects it. So the check catches results that depend on values the client cannot see in time, such as a jump the server refuses because of health only the server drains.

Rules for `Validate` members:

- A value is only as precise as the member's `Accuracy`, the same on both peers. Choose an accuracy the simulation can tolerate.
- Any `SendInterval` works.
- For a value a replay cannot reproduce exactly, such as one a physics engine moves, construct the member with `resultTolerance`. A predicted value within that distance of the server's orders no reconcile. It applies only while the controller's `SystemManager.IsResultValueSendEnabled` is on, the default; with it off, the result is compared exactly. The Unity validated behaviours set it on their position, rotation and velocities.
- The system needs `Predict` members. Without client state the server has nothing to check, and its values simply overwrite the prediction.
- The result check runs after the client state is validated, so a tick your `IMemberWriteValidator<T0>` corrected produces a different result, and the controller is reconciled onto the server's outcome.

An observer that predicts compares for itself

The server checks only the controller. An observer predicting a system another client controls (`IsObserverPredictionEnabled`) compares the server's values for that system's `Validate` members itself, at the tick its own echo names, within each member's `Accuracy`:

- **A match** is discarded. The prediction stands and the server's value is not applied.
- **A mismatch** is recorded with the tick and a `MismatchCause`, and the system keeps predicting until the `Reconcile` step, where the engine restores and replays (see Reconciliation and replay).
- **A history too shallow** to reach the echoed tick is recorded as `MismatchCause.HistoryMissing`, and the system logs a warning once naming `SystemManager.PredictionHistoryTicks`.
- **A value the server sent without ordering a reconcile** (the first values received, or a resend after loss) is compared like any other, and is recorded as `MismatchCause.Resync` only when it differs.

A `Validate` member on `SendInterval.Normal` without projection is checked on every new tick the server confirms, against the server value the observer already holds when no new value arrived. A paced or projected member is checked only when a value arrives.

Set the prediction history depth before systems spawn

`SystemManager.PredictionHistoryTicks` sets how many ticks of history `Predict` and `Validate`

members keep, so a reconcile can restore any member to a past tick and a replay can read each past tick's client state. Cover the round trip plus `SystemManager.StateInterpolation`, with headroom: at 60 ticks per second a 250 ms round trip is 15 ticks, so with an interpolation of 2 use 24 or more.

- The default, `SystemManager.UnsetPredictionHistoryTicks` (0), gives the predicted members of every system a client holds, controlled or not, `SystemManager.DefaultPredictionHistoryTicks` (64 ticks, about a second at 60 ticks per second).
- The server never rolls back and keeps the members' own two ticks.
- When a reconcile starts at a tick older than the history holds, the server's value still lands there and the system replays from it, and a warning naming the shortfall is logged once per system.
- A member takes the history when it initializes, and a system grows its members to it when this client gains control, so set the value before systems spawn.
- A member served on an interval widens its own history further, independent of `PredictionHistoryTicks`, so a replay can still address the ticks between its serves.

Read past values

```
public bool TryGetValueByTick(uint tick, out T0 value)
public bool TryGetValueAtOrBeforeTick(uint tick, out T0 value)
public bool TryGetPreviousValueAndTickGap(out T0 previousValue, out uint tickGap)
public bool TryGetMismatchValues(out T0 predictedValue, out T0 serverValue)
public bool TryGetPreRewindValue(out T0 value)
```

- `TryGetValueByTick` returns the value written on exactly that tick, and false when the member was not written on it or its history no longer holds it. Use it to read a `Predict` member's value for a past tick.
- `TryGetValueAtOrBeforeTick` returns the last write at or before that tick, and is false only when every tick it holds is newer. The restore uses this lookup, so a predicted member need not be written every tick.
- `TryGetPreviousValueAndTickGap` returns the value before the live one with the tick span between them, for deriving a rate of change such as a velocity from position deltas. It returns false when the live value is not strictly newer than the previous one.
- `TryGetMismatchValues` returns this client's value and the server's at `ReplayReport.StartTick` when the latest reconcile found this member wrong, from the restore until the next tick advance.
- `TryGetPreRewindValue` returns the value the member held before the latest reconcile restored it, the value that was on screen, until the next tick advance.

Sending a Validate value upstream raises PredictedMemberChangeViolation

A controller must not send a `Validate` member's value upstream: its client state carries what the server needs, and the server simulates the outcome. If a client sends a value for a member it predicts, `NetworkSystem` raises a `PredictedMemberChangeViolation` with `ViolationAction.Ignore` and leaves the member untouched:

```
public struct PredictedMemberChangeViolation : IViolation
{
    public uint SystemId;
}
```

`SystemId` names the system whose predicted member the sender tried to change. Subscribe to

`ViolationManager.PredictedMemberChangeViolationDetected` to observe these without changing the outcome.

Reconciliation and replay

> **Using Unity?** See Replaying a correction in Unity.

The server orders a reconcile when it finds the controller's predicted result different from its own, when it re-anchors a stalled client, and when game code calls `SendReconcile`. The engine then restores the client's world to the server's values as of one tick, replays every tick since through the same fixed updates that produced them, and reports what changed. You write the simulation once; you do not write the replay.

What happens in one reconcile

Detect, on `LateStateUpdate`. A reconcile the server ordered is recorded against the echoed client state tick as `MismatchCause.ServerOrdered`. An observer predicting a system records its own compare the same way. From then on, the server values to restore are fixed, so remote calls and `LateStateUpdate` callbacks that run before the `Reconcile` step cannot move them.

Restore, at the head of the `Reconcile` step, before its callbacks. The start tick, the earliest tick any member recorded, is one tick for the whole world. Every predicted member of every system this client predicts takes its server value, its later history is cleared, and its value before the restore is kept as its pre-rewind value. A member the server has sent nothing for yet keeps its own history, rewound to the same tick. Systems this client does not control are held at their server values. `ReplayReport` lists the members whose restored value differs from what this client held for the start tick.

`ReconcileRequired` is raised with the start tick on each restored system, in system Id order.

Replay. `ReplayStarting` is raised on every system the replay covers. Then, for each tick from the start tick plus one to the live tick minus one, every restored system runs its early fixed update and then its late one as `SimulationType.Replayed`, in system Id order.

Report. Held systems return to their latest values. Each predicted member the replay moved away from its pre-rewind value is added to `ReplayReport.Corrections` and raises `OnMembersChanged` with `MemberChangeDirection.Correction`. Then `ReplayCompleted` is raised on every system that raised `ReplayStarting`.

Continue, on `EarlyFixedUpdate`. The live tick runs as `SimulationType.Forward` from the corrected state.

A reconcile covers the whole world, once per inbound pass. Members or systems that diverge in the same pass are restored to the same tick and replayed together, so one divergence costs one replay.

The engine replays your fixed updates

The engine replays a system's fixed updates when something simulates it: a component that overrides one of them, or a callback registered with `NetworkSystem.RegisterSimulationCallback`:

```
// On NetworkComponent:
protected virtual void OnEarlyFixedUpdate(StepDelta stepDelta, SimulationContext simulatio
nContext) { }
protected virtual void OnLateFixedUpdate(StepDelta stepDelta, SimulationContext simulation
Context) { }

// Or registered per system with NetworkSystem.RegisterSimulationCallback:
public interface INetworkSimulationCallback
{
    void OnSimulationStep(NetworkLoopSteps networkLoopStep, StepDelta stepDelta, Simulatio
nContext simulationContext);
}
```

Within a system, components run first in component order, then registered callbacks in registration order. `UnregisterSimulationCallback` removes a callback. A raw `INetworkLoopStepCallback` runs once per loop tick and is never replayed.

Peer	System	How the fixed updates run
Server or host	Driven by a remote client through client state	`Forward`, once per server tick on one of the client's ticks, after its client state is validated, or as a step missing its client state when that tick's did not arrive
Server or host	Controlled by the server, by the host's own client half, or by a client that sends no client state	`Forward`, once per tick
Controlling client	Its own system	`Forward` once per live tick, and `Replayed` once per tick a reconcile replays
Observing client with `IsObserverPredictionEnabled`	Another client's system	As the controlling client
Any other client	A system it does not control	`None`, once per loop tick; nothing is simulated

`NetworkSystem.IsSimulatedPerClientState` is true on the server for a system run on its client's ticks. A replayed step, and the server's step for a tick of a client's client state, is not a real frame, so its `Delta`, `FixedDelta` and `NormalizedDelta` are one tick interval and `TimeSinceLastFixedUpdate` is 0.

The simulation context

Each fixed update receives a `SimulationContext`, a read-only struct describing the step:

Field	Meaning
`Type`	`SimulationType.Forward` the first time a tick is simulated, `Replayed` for a past tick simulated again during a reconcile, `None` on a peer that does not simulate the system.

`RemoteTick`	The other end's tick the step pairs with. On the server, for a system a client drives, the controller's tick whose client state is being simulated. On a client, the server tick of the state it applied. `NetworkLoopManager.UnsetTick` when there is none: on the server for a system no client drives, and on a client before any state has arrived.
`IsRemoteTickEstimated`	True when `RemoteTick` was inferred rather than arriving with the data. On the server, for a client tick whose client state did not arrive, it is the tick the server assumed. On a client, for a tick no state was applied on (every replayed tick included), it is the newest server tick applied, carried on one tick per local tick.

Time everything by `StepDelta.Tick`, the local tick being simulated. On the server that is the server's own tick, even for a step that runs one of the client's ticks. On the server, `RemoteTick` is a number the client chose: use it to pair what the server did with what the client logged, never for a cooldown, a timeout or any other timing.

```
protected override void OnEarlyFixedUpdate(StepDelta stepDelta, SimulationContext simulati
onContext)
{
    if (simulationContext.Type is not SimulationType.Forward || !CoreManager.TransportMana
ger.IsServerStarted)
        return;

    // Pair the server's step with the client's tick in a log line; time anything by stepD
elta.Tick.
    if (stepDelta.IsClientStateMissing)
        System.Console.WriteLine($"Server tick {stepDelta.Tick} ran client tick {simulatio
nContext.RemoteTick} without its client state.");
}
```

Write the simulation once for every pass

```
using System.Numerics;
using Nucleus.Components;
using Nucleus.Managers.NetworkLoop;
using Nucleus.Systems;

public partial class MoverComponent : NetworkComponent
{
    // Client state: what the player asked for this tick.
    public readonly NetworkMember<Vector2> Direction = new(role: MemberRole.Predict);
    // The result the controller predicts and the server checks.
    public readonly NetworkMember<Vector3> Position = new(role: MemberRole.Validate, accur
acy: 0.01f, correctionSmoothingTicks: 6);

    // Set by the game from the device on the controlling client.
    public Vector2 IntendedDirection;

    private const float Speed = 5f;

    protected override void OnEarlyFixedUpdate(StepDelta stepDelta, SimulationContext simu
lationContext)
    {
        // A client watching a system it does not control: nothing to simulate.
        if (simulationContext.Type is SimulationType.None)
            return;

        // The controlling peer's live tick: this becomes the tick's client state. A repla
y reads the recorded client state instead.
        if (simulationContext.Type is SimulationType.Forward && NetworkSystem.IsController
(ControllerType.AnyController))
            Direction.Value = IntendedDirection;

        // The server's step for a tick whose client state never arrived: carry the last t
ick's instead of the defaults.
        if (stepDelta.IsClientStateMissing)
            TryGetLastState(isEveryMemberCarried: true);

        // The live tick, the server's run of each client tick, and every replayed tick al
l compute here.
        Position.Value += new Vector3(Direction.Value.X, 0f, Direction.Value.Y) * (Speed /
 CoreManager.NetworkLoopManager.TickRate);
    }
}
```

- Write the client state on the live tick only, before the step reads it. Every other pass reads the `Predict` member, which a replay points at the tick it replays.
- Step by the fixed tick, never by frame time, so every pass of a tick computes the same value.

Each kind of member behaves differently in a replayed tick

- `Validate` members write the replayed tick's value, and each replayed tick becomes the member's newest, so `Value` reads the last tick replayed and the next live write lands after it.
- `Predict` members read the client state recorded for the replayed tick.
- `Replicate` members keep their live value; writes to them land on the live value.
- Systems this client does not control read the server values that arrived with the start tick, unless the hold radius leaves them out.

- The core steps no physics itself. `SystemManager.ReplayEarlyFixedUpdated` is raised between the early and late fixed update of each replayed tick, so an integration can step its physics world there. The Unity integration does, so a body predicted through its `ValidatedRigidbody` replays.

`NetworkLoopManager.SimulationType` is the pass running now (`None` outside a simulate step), mirrored as `NetworkSystem.SimulationType`. `NetworkLoopManager.ReplayTick` is the replayed tick during a replay and `NetworkLoopManager.UnsetTick` otherwise. `SimulationTick` is the replayed tick during a replay and `Tick` otherwise; `Tick` stays the live tick throughout.
`NetworkSystem.IsPredicting` is true on the clients that replay a system (its controller, or an observer predicting it) while it has a predicted member.

The engine refuses side effects during a replay

A replay re-runs every tick since the start tick, so anything with an effect outside the simulation would happen again. While a replayed tick runs, these calls are refused, logging once per kind of call:

- `TrySendRpc` returns false, and `Connection.SendMessage<T0>` sends nothing.
- `EnsureStartPredictedSystem` and `EnsureStopPredictedSystem` refuse with `PredictedSpawnRefusal.Replaying`.
- Writes to collection members and reference members change nothing.
- `NetworkAnimator.SetTrigger`, `ResetTrigger`, `CrossFade` and `Play` do nothing.

Play one-shot effects (sounds, particles, an RPC to the server) on `SimulationType.Forward` only.

Read what the reconcile changed

```
public event ReplayStartingHandler? ReplayStarting;
public event ReplayCompletedHandler? ReplayCompleted;
public delegate void ReplayStartingHandler(ReplayReport replayReport);
public delegate void ReplayCompletedHandler(ReplayReport replayReport);

public ReplayReport ReplayReport { get; }
```

`NetworkSystem.ReplayReport` describes the latest reconcile on the system. One instance per system is reused by every reconcile, so read it while the reconcile it describes is current:

Member	Meaning
`StartTick`	The tick the world was restored to. The replay re-runs the ticks after it.
`EndTick`	The last tick the replay re-runs: the live tick less one.
`IsReplaySkipped`	True when the replay would have run longer than `SystemManager.MaximumReplayTicks`, so the server's values stand at `StartTick` without one.
`Mismatches`	The cause: each `PredictionMismatch` (`Member`, `MemberName`, `Cause`) whose value at `StartTick` differed from this client's. Empty on a system restored only because another one diverged.

`Corrections`	The effect: each `MemberCorrection` (`Member`, `MemberName`) whose present the replay moved. Filled before `ReplayCompleted`.

`MismatchCause` is `ValueDiffered` (the server's value differed), `HistoryMissing` (the history no longer reached the tick), `Resync` (the first values received, or a resend after loss, that differed) or `ServerOrdered` (a reconcile the server sent). A held system's report names the start tick and lists no mismatches.

Inside a handler, `NetworkMember<T0>.TryGetMismatchValues(out predictedValue, out serverValue)` gives both values at the start tick and `TryGetPreRewindValue(out value)` gives the value that was on screen, both until the next tick advance. A correction reaches components through `OnMembersChanged`, so a replay that took back a jump can cancel its effects:

```
public override void OnMembersChanged(ulong memberFlags, MemberChangeDirection memberChang
eDirection)
{
    if (memberChangeDirection != MemberChangeDirection.Correction)
        return;

    if ((memberFlags & (ulong)MovementComponentFlags.JumpCount) != 0 && JumpCount.TryGetPr
eRewindValue(out ushort shownJumpCount) && JumpCount.Value < shownJumpCount)
        CancelJumpEffects();
}
```

On the controlling client each system also counts `ReconcileCount`, `ReplayedTickCount` and `LastReplayTickCount` since it was rented, and `GetMismatchCount(MismatchCause)` per cause. All of them stay at zero while predictions agree.

Smooth a correction

Construct a member with `correctionSmoothingTicks` to have its `InterpolatedValue` glide from the pre-rewind value to the corrected one over that many presented ticks, while `Value` is correct at once. 0, the default, shows the correction at once.

```
public readonly NetworkMember<Vector3> Position = new(role: MemberRole.Validate, accuracy:
 0.01f, correctionSmoothingTicks: 6);
```

For a physics body, a convergence follower can hide the visual jump a rewind produces by interpolating toward the corrected pose; see PhysicsConvergence.

Limit how long a replay runs

```
coreManager.SystemManager.MaximumReplayTicks = 30;
```

A reconcile whose replay would run longer lands the server's values at the start tick without replaying, and its report sets `IsReplaySkipped`. The default, `SystemManager.UnsetMaximumReplayTicks` (0), sets no limit.

Systems this client does not control are held during a replay

A system controlled by the server or by another client takes its values from the server and is never compared or simulated on this client. It still takes part in a reconcile: from the restore, its predicted members read the server values that arrived with the start tick, and the controlled systems replay against it there. It raises `ReplayStarting` and `ReplayCompleted` with the replayed systems, so whatever shows it can be placed where the replay sees it and put back. Before `ReplayCompleted` it returns to its latest values, so a

replay leaves no net change on it.

Holding only the systems near a replay

In a crowded scene, set a radius so only systems near a restored system are held:

```
coreManager.SystemManager.ReplayHoldRadius = 20f;
```

Distances are read through the interest position reader (`InterestManager.SetPositionReader`). A system whose position cannot be read is always held. The rest keep their latest values and raise no replay events. The default, `SystemManager.UnsetReplayHoldRadius`, holds every system, and so does a build without distance interest.

Predicting another client's object

A client can predict a system another client controls instead of holding it. Opt in per system on that client:

```
observedSystem.IsObserverPredictionEnabled = true;
```

- Its fixed updates then run as `SimulationType.Forward` on every live tick, from the client state the server forwards, which lands on the local tick the server's echo names.
- On a live tick that client state has not arrived yet, so the step says `StepDelta.IsClientStateMissing`. Carry the last client state forward with `TryGetLastState`, as the server does.
- The observer compares the server's values for the `Validate` members with its prediction at its own echo, and a reconcile restores and replays the system with the ones the observer controls. Nothing new travels.
- The observer needs an echo of its own, so it must control a system that sends client state: one with `MemberRole.Predict` members. An observer that controls nothing like that gets no echo, and the server's values land on it directly.

Replay from ReconcileRequired yourself

`ReconcileRequired` is the low-level hook for a game that drives its own replay:

```
public delegate void ReconcileRequiredHandler(uint rewoundTick);

public event ReconcileRequiredHandler? ReconcileRequired;
```

- It is raised at the head of the `Reconcile` step, after the restore, on each restored system in system Id order.
- `rewoundTick` is the start tick. The server's value there already includes that tick's client state, so replay from the tick after it.
- It is `NetworkLoopManager.UnsetTick` when the correction was a snap with no replay target: a reconcile the server sent that landed with no echo of the processed client state tick.
- A server correction of the controller's client state alone does not raise it. A `Predict` value the server changed lands in that tick's history and raises `OnMembersChanged` with `MemberChangeDirection.Correction` instead.

When something on the system overrides a fixed update or registers a simulation callback, the engine replays it after `ReconcileRequired`, so do not replay it a second time. A system nothing simulates is left to you. Replay it on the `Reconcile` step (an `INetworkLoopStepCallback` returning `NetworkLoopSteps.Reconcile`), bracketing every past tick in tick order:

```
public void BeginReplayTick(uint tick);
public void EndReplayTick();
```

For each tick from `rewoundTick + 1` up to the tick before the current one, call `BeginReplayTick(tick)`, re-simulate, then call `EndReplayTick()`. Inside the bracket, `Validate` members write that tick's value, `Predict` members read its client state, and `NetworkLoopManager.SimulationType` reads `Replayed`, so the refusals above apply.

Apply the current tick's values normally.

A write to a predicted member outside the bracket lands on the live value and breaks the next result check. Systems held for the reconcile stay held until the `Reconcile` step's callbacks have run.

Force a correction with SendReconcile

```
public void SendReconcile(Connection connection);
```

`SendReconcile` pushes a system's complete current state to one connection, rather than waiting for the result check to find a difference. It is queued and sent to that connection at the end of the tick. Asking again for the same system and connection before then queues nothing more.

Only the server can call it. A client that tries is refused with a logged error, and nothing is queued.

Use it when a gameplay event invalidates the controller's prediction in a way the result check would not catch: a teleport, a respawn, or a member change the client could not have predicted from its own client state. The engine also sends one itself for a `Validate` result that differs and when it re-anchors a stalled client.

On arrival it is recorded as `MismatchCause.ServerOrdered` and restores and replays like any other reconcile. The controller's `Predict` values for the start tick are replaced with the ones the server applied after validation, raising `OnMembersChanged` with `MemberChangeDirection.Correction` when they differ from its own.

Validate results the server found different

When the server finds a controller's `MemberRole.Validate` result different from its own, it queues the correcting reconcile and raises `NetworkSystem.ResultMismatched` with a `ResultMismatchReport`:

- `Connection`: the controller.
- `Tick`: the server's tick the difference was found on.
- `RemoteTick`: the controller's tick it pairs with, for matching what the client logged, never for timing.
- `Members`: the members that differed, when `AreValuesSent` is true. Each answers `TryGetResultMismatchValues(out controllerValue, out serverValue)` until the tick advances.

The values are available while the controller's `SystemManager.IsResultValueSendEnabled` is set, the default. Turning it off in a release build saves bandwidth, and the report then shows only that a mismatch happened. No violation is raised, so what to do about the client is the game's call.

Read the report inside the handler and keep no reference to it after the handler returns:

```
using System.Numerics;
using Nucleus.Components;
using Nucleus.Systems;

public sealed class ResultMismatchLogger
{
    // Server: subscribe once the system has started.
    public void Watch(NetworkSystem networkSystem) => networkSystem.ResultMismatched += On
ResultMismatched;

    private void OnResultMismatched(ResultMismatchReport resultMismatchReport)
    {
        foreach (NetworkMemberBase networkMember in resultMismatchReport.Members)
        {
            if (networkMember is NetworkMember<Vector3> vectorMember && vectorMember.TryGe
tResultMismatchValues(out Vector3 controllerValue, out Vector3 serverValue))
                System.Console.WriteLine($"Client {resultMismatchReport.Connection.Id}, it
s tick {resultMismatchReport.RemoteTick}: {networkMember.MemberName} was {controllerValue}
, server had {serverValue}.");
        }
    }
}
```

What control means

Every `NetworkSystem` answers three separate questions: who is the server, who controls it, and who observes it. Mixing them up causes bugs that work on one machine and fail over the network.

Each system has a server, at most one controller and any number of observers

- **The server** is the peer where `NetworkSystem.IsServerStarted` is true. Its word on a system's state is final, whatever any client believes.
- **The controller** is the one connection, the server or a single client, that drives the system. At most one connection controls a system at a time, and if no client controls it, the server does. Check the local peer with `IsController`; see Reading who controls a system for every `ControllerType` value.
- **An observer** is any peer the system is replicated to. A system can have any number of observers, including none. The interest system decides who observes; see What area of interest is.

```
if (networkSystem.IsController(ControllerType.Client))
{
    // This peer is the client controlling this system.
}
```

Only the server assigns a controller

The server assigns control with `SetController`. A client that calls it is rejected: the server logs an error and nothing changes. Every observer is told who the controller is, and `IsController` reads that replicated identity, never anything a client asserts about itself. A controller change raises `ControllerChanged`.

Control decides who sends client state, who predicts and who sets members

These are controller-only, whatever the write-access setting:

- **Client state.** Only the controller's client state is accepted for the system.
- **Prediction.** The controlling client predicts the system from its own client state, and the server checks the result and corrects it when they differ. Any other client receives the server's state, and during a reconcile holds the system at the server's values while its own systems replay, unless it sets `NetworkSystem.IsObserverPredictionEnabled`.
- **Setting members.** The controller can always set members. The default access, `MemberWriteAccess.Controller`, allows the server or the controlling client. The Pro setting `MemberWriteAccess.AnyClient` widens this to other observers; see Letting clients set members.

Remote calls work the same way: `RpcSendAccess.Controller` is the default, so only whoever drives the object may ask it to do anything.

A connection must observe a system before anything else is checked

When a client sets a member or makes a remote call, the server checks, in order:

Does the connection observe the system? If not, it is refused.

Is it the system's controller (`ControllerConnectionId`)? If so, it is admitted.

Otherwise it is admitted only when the access in force is the any-client one (`MemberWriteAccess.AnyClient` or `RpcSendAccess.AnyClient`).

A client that does not observe the system is refused even when it is the controller or the access allows any client.

A host hides control bugs

A host runs the server and a client in one process. It is the server, it usually controls its own player object (so `IsController(ControllerType.Client)` is true), and interest never culls it, so it observes everything. One peer is all three roles at once, and every check above passes.

That hides bugs that appear as soon as a second machine joins: a missing observer registration, a write-access setting that was never exercised, or client state code that assumed the controller and the server were the same connection.

Assigning and releasing control

Using Unity? See Giving a client control of an object.

Assign control from the server

```
public void SetController(Connection controllingClientConnection)
```

- Pass a `Connection` to make that client the controller, or `null` to return control to the server.
- Only a peer with a started server may call it. A client that calls it gets a logged error and nothing happens.
- Calling it with the current controller, or with `null` when the server already controls the system, does nothing.

Every peer learns of the change through ordinary replication. There is no message to send.

The controller decides client state, prediction and write permission

Whether a connection holds `ControllerConnectionId` decides:

- **Client state routing**: whose client state the system applies.
- **Prediction**: whether this peer predicts the system locally.
- **Write permission**: the controller may always set members, whatever `MemberWriteAccess` says.

The server rejects a client write to a system the client does not control and raises an `UnauthorizedMemberChangeViolation`:

```
public struct UnauthorizedMemberChangeViolation : IViolation
{
    public uint SystemId;
}
```

It is raised with `ViolationAction.Ignore`, because the usual cause is a race (control or permission changed before the client's earlier write landed), not an attack. The write is dropped and the connection is not punished.

Retention decides what happens when the controller leaves

The server drops the departing connection's reference automatically. Whether the controlling identity is cleared at once or held for a returning connection to reclaim is set by Controller retention.

When prediction is worth its cost

Prediction only hides the controller's round trip

Without prediction, a client sends its client state and only sees the result once the server's outcome comes back. With it, the client applies its client state locally at once, and the server's outcome later confirms or corrects that guess.

That is the whole benefit. Prediction does not make an object more accurate, more responsive to other peers or cheaper to replicate, and it only matters to the peer driving the object. An object nobody controls has no round trip to hide, and most of a world (static geometry, background actors, anything only the server moves) is like that.

Prediction costs memory, bandwidth and replay time

You turn it on per member with the `role` constructor argument of `NetworkMember<T0>` (see NetworkMember): a result declared `MemberRole.Validate`, computed from client state declared `MemberRole.Predict`, is predicted by the controller and checked by the server. That costs:

- **A history, not just a live value.** Every client keeps `SystemManager.PredictionHistoryTicks` of history for a predicted member, or `SystemManager.DefaultPredictionHistoryTicks` (64) when that is unset, and a member served on a wider interval than the tick rate keeps more. An unpredicted member keeps only its current and previous values.
- **Upstream bandwidth and replays.** A matching result costs a little upstream bandwidth each tick, more while `SystemManager.IsResultValueSendEnabled` is on (the default, so the server can name what differed; turn it off to save that bandwidth). A result the server finds different orders a reconcile: every system the client predicts is restored to the server's values at the wrong tick, and the engine re-runs `OnEarlyFixedUpdate` and `OnLateFixedUpdate` as `SimulationType.Replayed` for every tick from there to the present. That is about a round trip plus the state interpolation in ticks, all simulated in one frame.
- **Game code that tolerates being re-run.** Anything that happens during a replayed tick happens again. A movement integration recomputes the same answer, but a side effect that cannot be undone or repeated safely belongs on `SimulationType.Forward` only. The engine refuses remote calls, messages, predicted spawns, collection and reference writes and `NetworkAnimator` triggers during a replay, but it cannot know about your own side effects: awarding score or writing to a static from a replayed tick happens again on every replay. A value that only feeds a display, such as a position feeding a camera or a resource bar, is safe: at worst the display corrects itself.

The core steps no physics itself. An integration steps its physics world on `SystemManager.ReplayEarlyFixedUpdated`, which the Unity integration does, and the Unity `ValidatedRigidbody` predicts a body that way.

Use a cheaper option when you can

Most objects need none of this. From cheapest:

- **An uncontrolled object with interpolation.** No controller, no history beyond the current and previous values, no result check, no replay. The receiver animates between the values it receives, paced by `SystemManager.StateInterpolation` (0 by default, capped at `MaximumStateInterpolation`). This is the default for anything nobody drives.
- **A controller-written value with no prediction.** The controller writes the member every tick, but its role

stays `MemberRole.Replicate`. The controller sees its own writes a round trip late like everyone else, with no rewind and no replay. This is cheaper than prediction and more responsive than a value only the server writes.

- **A purely local visual that is never replicated.** Particle effects, camera shake and UI feedback that only need to look right on the machine that triggered them are not a `NetworkMember<T0>` at all, cost nothing on the wire and need no reconciliation.

Choose prediction per member

`role` is set per member, not per project or per object, and nothing turns prediction on by default. A component can predict its position while leaving its health or ammo count as ordinary `MemberRole.Replicate`.

The Unity demo ValidatedJump declares the walk direction and jump press of `ValidatedJumpMovementComponent` as `MemberRole.Predict` client state and every movement result as `MemberRole.Validate`. In Unity, the validated behaviours, such as `ValidatedRigidbody` and `ValidatedTransform`, carry ready-made `MemberRole.Validate` members for a body, transform or character controller, and the BallRacer demo predicts a `Rigidbody` with one.

Predicted spawning is Pro, predicted motion is not

Predicted members, the client state that drives them and the reconcile replay are in every build. Predicted spawning, predicted despawn and spawn compensation are Pro-only and don't exist in a Free build.

Authenticating clients

Implement IClientAuthenticator

```
public interface IClientAuthenticator
{
    void OnClientConnecting(Connection connection);
}
```

`OnClientConnecting` is called once per connecting client, when its remote connection is established and before it is authenticated. The engine defines no credential of its own: run whatever handshake you want with ordinary `IMessage` types, then call `ServerManager.ApproveClient` or `ServerManager.DenyClient`.

The verdict can come any number of ticks later, after a backend round trip. Until it does, the connection is unauthenticated: it is delivered no message that requires authentication, and it observes nothing.

Call ApproveClient or DenyClient once you have a verdict

```
void ApproveClient(Connection connection, string identity = null);
void DenyClient(Connection connection);
void DenyClient(Connection connection, ulong timeoutMilliseconds);
void DenyClient<T0>(Connection connection, T0 notice) where T0 : IMessage, new();
void DenyClient<T0>(Connection connection, T0 notice, ulong timeoutMilliseconds) where T0
: IMessage, new();
```

- `ApproveClient` marks the connection authenticated and stamps `identity` onto `Connection.Identity`, the value that names this client across reconnects. Pass `null` to leave it unidentified. What the identity means is up to your code; the engine only carries it.
- `DenyClient(connection)` refuses and disconnects the client, telling it nothing. That is the right default for a failed authentication: a refused peer is owed no explanation, and one can help an attacker.
- `DenyClient(connection, notice)` first sends a message of your own on the reliable channel, so the client knows why. Its type is how the client tells a refusal from any other disconnect. Leave its `IsAuthenticationRequired` at the default; the refused client still receives it. A refusal is a kick, so the notice and the optional timeout behave as they do for kicking a client.

Calls for a connection that is already resolved, or gone, are ignored.

The default approves everyone by address

Until you install your own, `ServerManager` uses `AddressClientAuthenticator`:

```
public sealed class AddressClientAuthenticator : IClientAuthenticator
{
    public void OnClientConnecting(Connection connection) =>
        connection.CoreManager.ServerManager.ApproveClient(connection, connection.RemoteAd
dress);
}
```

It approves every client at once and identifies it by `Connection.RemoteAddress`, which cannot be forged: a peer spoofing its source address never receives the return traffic and never completes a connection. The port is not part of it, so a client that reconnects from a new port still matches.

An address is a location, not a player. Clients behind one NAT (housemates, an office, a carrier-grade NAT) share an address and resolve to the same identity, so code that restores a returning player's objects by

identity hands the second client the first one's. The default suits development, LAN and single-address deployments. Anything else needs your own authenticator.

Install your own before the server starts listening

```
void SetAuthenticator(IClientAuthenticator clientAuthenticator);
```

Call `ServerManager.SetAuthenticator` before the server starts listening. A client that connects earlier is resolved by whichever authenticator was in force at the time. Passing `null` restores `AddressClientAuthenticator`. In Unity there is no authenticator component; call it from a script once the managers exist.

Timeout

```
public ushort AuthenticationTimeoutSeconds = 30;
public const ushort UnsetAuthenticationTimeout = 0;
```

`ServerManager.AuthenticationTimeoutSeconds` (default 30) is how long a connecting client can stay unresolved, from when its remote connection is established, before the server disconnects it. Without it, a silent client or an authenticator that never reports would hold a connection slot open forever. It is checked once per tick, on `TickAdvance`. Set it to `UnsetAuthenticationTimeout` (0) to disable it.

The server and client raise authentication events

On the server, `ServerManager` raises:

- `ClientAuthenticated(Connection connection)`: after the client is marked authenticated and its `Identity` is stamped, so a handler can read both.
- `ClientAuthenticationDenied(Connection connection)`: for a client that was denied or timed out, never for one that merely dropped. It is raised before the client is disconnected.

On the client, `ClientManager` raises `LocalClientAuthenticated(Connection localClientConnection)` once this client's connection is authenticated. The client learns its server-assigned Id before any other state that might reference it.

Authenticate with a token

The client presents a token in a message. Because the client is not authenticated yet when it sends it, the message must opt out of the default requirement:

```
[NetworkType]
public struct TokenAuthentication : IMessage
{
    public bool IsAuthenticationRequired => false;

    public string Token;
}
```

`IMessage.IsAuthenticationRequired` defaults to `true`. A type that forgets the override is refused from every unauthenticated sender, including the client trying to authenticate with it.

```
public sealed class TokenClientAuthenticator : IClientAuthenticator
{
    public TokenClientAuthenticator(MessageManager messageManager)
    {
        messageManager.RegisterMessageHandler<TokenAuthentication>(OnTokenAuthentication);
    }

    public void OnClientConnecting(Connection connection)
    {
        // Nothing to do yet; wait for the client to present its token.
    }

    private void OnTokenAuthentication(in MessageContext messageContext, TokenAuthenticati
on tokenAuthentication)
    {
        Connection connection = messageContext.SenderConnection;

        if (TryResolveIdentity(tokenAuthentication.Token, out string identity))
            connection.CoreManager.ServerManager.ApproveClient(connection, identity);
        else
            connection.CoreManager.ServerManager.DenyClient(connection, new TokenRefusedNo
tice { Reason = "Invalid token." });
    }
}
```

The refusal notice is an ordinary message, and the client registers an ordinary handler for it to show the reason:

```
[NetworkType]
public struct TokenRefusedNotice : IMessage
{
    public string Reason;
}
```

The client sends `TokenAuthentication` on its own connection as soon as it connects. The token is a bearer credential: until the transport is encrypted it can be read off the wire and replayed. Make it:

- server-generated and unguessable;
- opaque about the identity it maps to;
- validated against server-held state, so an unrecognized value fails closed onto a fresh identity rather than an assumed one;
- expired or rotated on use.

Connections

A `Connection` represents one peer: a client from the server's point of view, or the server from a client's. It is pooled (`Connection : IPoolResettable`): rented when a peer arrives and returned when it leaves.

Do not store a `Connection` or its `Id` past the moment you read them. The next peer to connect can be handed the same object, and `Id` values are recycled too. To remember a peer for longer, hold a `ConnectionHandle`.

Read a connection's identity and role

Member	Meaning
`Id`	The connection's unique Id across all transports. `UnsetId` (`0`) means no Id is assigned.
`IsIdUnset`	`true` when `Id == UnsetId`.
`IdBySocket`	The Id assigned by the socket-level Transport. Unique within that transport, not across transports. Server sockets always read `ServerIdBySocket`.
`ServerIdBySocket`	Constant (`uint.MaxValue`) used for `IdBySocket` when the connection is a server socket.
`IsServer`	`true` when `IdBySocket == ServerIdBySocket`.
`IsClient`	`!IsServer`.
`IsConnected`	`true` when `LocalState == LocalConnectionState.Connected`.
`IsAuthenticated`	`true` when this side is a client that has authenticated, or when `IsServer`, or when `IsEmulated`.
`IsEmulated`	`true` for a connection that uses no socket, such as a fake or test peer.
`IsPeerStandIn`	`true` when this Connection names a peer the server told a client about, rather than a link this peer holds. A stand-in carries identity only; there is no socket behind it.
`IsHostLoopback`	`true` when this Connection is one half of a host's own loopback pair (the same peer at both ends).

Hold a ConnectionHandle to remember a peer

`ConnectionHandle` pairs the `Id` with a `Generation`, a counter bumped every time the instance is rented:

```
public readonly struct ConnectionHandle : IEquatable<ConnectionHandle>
{
    public readonly uint Id;
    public readonly uint Generation;

    public bool IsValid => Id != Connection.UnsetId;
}
```

Take it from `Connection.Handle` and resolve it later with `TransportManager.TryGetConnection`:

```
ConnectionHandle handle = connection.Handle;

// ... later, possibly after the peer disconnected and another peer connected ...

if (transportManager.TryGetConnection(handle, out Connection resolved))
{
    // resolved is still the same peer the handle was taken from.
}
```

If the connection was recycled or its Id reissued, resolution fails rather than naming whoever arrived next. A handle is a plain value type that allocates nothing, and it is local to the peer that took it.

Key players on Identity, not RemoteAddress

- `RemoteAddress` is where the connection came from, as the Transport reports it, without the port, so a reconnecting peer keeps the same value. It is `null` when the transport has no such notion, such as emulated or in-memory connections. The server observes it, so it cannot be forged, but several clients behind one NAT share an address. It identifies a location, not a player.
- `Identity` names a client across reconnects. The client authenticator stamps it when it approves the client; it is `null` until then, and stays `null` if the authenticator supplies none. Key anything that must recognize a returning player on `Identity`.

Find connections through the TransportManager

- `ActiveConnections`: the connections this peer knows. On the server, the remote clients. On a client, the peer stand-ins the server has reported, never the local client itself.
- `TryGetConnection(uint id, out Connection connection)`: this peer's own Connection for an Id.
- `TryGetConnection(ConnectionHandle handle, out Connection connection)`: resolves a handle, failing if the generation is stale.
- `TryGetLocalClientConnection(out Connection connection)`: this peer's own connection as a client.
- `TryGetServerConnection(out Connection connection)`: this peer's own server socket connection.

Read and react to connection state

Each `Connection` tracks two states:

- `LocalState` (`LocalConnectionState`): the connection this device created, your own client or the server socket when hosting.
- `RemoteState` (`RemoteConnectionState`): the connection as seen from the remote side. It has only `Disconnected` and `Connected`.

```
public enum LocalConnectionState : byte
{
    Disconnected = 0,
    Connecting = 1,
    Connected = 2,
    Disconnecting = 3,
    Error = 4,
    TimedOut = 5,
}
```

`Disconnected` is the zero value, so an unwritten state reads as down.

`ConnectionStateExtensions` adds two helpers on `LocalConnectionState`:

```
public static bool IsConnectedOrDisconnected(this LocalConnectionState localConnectionStat
e);
public static bool IsAnyDisconnected(this LocalConnectionState localConnectionState);
```

`IsConnectedOrDisconnected` is true only for `Connected` or `Disconnected`. `IsAnyDisconnected` is true for `Disconnecting`, `Disconnected`, `Error` and `TimedOut`.

`TransportManager` raises an event for each kind of change:

```
public event ConnectionStateChangedHandler ConnectionLocalStateChanged;
public event ConnectionStateChangedHandler ConnectionRemoteStateChanged;

public delegate void ConnectionStateChangedHandler(ConnectionStateChange connectionStateCh
ange);

public readonly struct ConnectionStateChange
{
    public readonly Invoker Invoker;
    public readonly Connection Connection;
    public readonly bool IsRemoteStateChange;
}
```

`Invoker` says which side raised it, `Connection` is the connection that changed, and `IsRemoteStateChange` says whether `RemoteState` or `LocalState` changed.

Log a connection with AsString

```
public static string AsString(this Connection? connection);
```

`ConnectionExtensions.AsString` reports the connection's `LocalState`, `Id`, `IdBySocket`, transport Id, `IsServer` and `IsAuthenticated`, and is safe to call on `null`.

For round-trip time and packet loss, see Round-trip time, jitter and packet loss.

Creating a CoreManager

Using Unity? See The Nucleus Core Manager component.

`CoreManager` is the root of a Nucleus session. Constructing one starts the session: when the constructor returns, every manager exists and the network loop is ticking. There is no separate `Initialize` or `Start` call.

```
private CoreManager _coreManager = new();
```

Pass the tick rate and loop provider to the constructor

```
public CoreManager(uint tickRate = NetworkLoopManager.DefaultTickRate, INetworkLoopStepPro
vider? networkLoopStepProvider = null)
```

- `tickRate` is the number of network ticks per second, fixed for the session. Values outside `NetworkLoopManager.MinimumTickRate` through `NetworkLoopManager.MaximumTickRate` are clamped into range and logged. It cannot change after construction.
- `networkLoopStepProvider` drives the network loop's steps. Leave it `null` to get the default provider, which steps the loop from the thread pool on a timer.

Supply your own `INetworkLoopStepProvider` here, not after construction. A provider passed to the constructor means the default never runs. Installed afterwards, the default has already been stepping the loop from the thread pool while your setup code ran, racing it. See Driving the loop yourself.

Reach every manager through its field

Every build has these eleven `readonly` fields:

Field	Type
`NetworkLoopManager`	`NetworkLoopManager`
`TransportManager`	`TransportManager`
`ServerManager`	`ServerManager`
`ClientManager`	`ClientManager`
`SystemManager`	`SystemManager`
`InterestManager`	`InterestManager`
`SceneManager`	`Scenes.SceneManager`
`PacketManager`	`PacketManager`
`MessageManager`	`MessageManager`
`RpcManager`	`RpcManager`
`ViolationManager`	`ViolationManager`

A Pro build adds two fields, which are not `readonly`:

Field	Type
`BundleManager`	`BundleManager`

`WorldPersistenceManager`	`WorldPersistenceManager`

In a Free build these two fields do not exist at all, so there is no null to check. `BundleManager` is for content delivered from outside the build (Addressables, a patcher, a CDN); without it a build can only spawn what shipped inside it. `WorldPersistenceManager` writes a world to disk and builds it back; without it a world lives only as long as the process holding it.

There is no engine-wide settings object. Each setting is a field on the manager it configures, documented on that manager's own page.

Use Instance only when there is one CoreManager

```
public static CoreManager Instance { get; private set; }
public static bool EnsureSetInstance(CoreManager coreManager)
```

- The first `CoreManager` constructed takes `Instance` and keeps it until it tears down. The slot then passes to the next live `CoreManager`, or becomes `null` if none remain. It never names a torn-down manager.
- `EnsureSetInstance` points the slot at a specific live `CoreManager`. It returns `false` and leaves the slot unchanged for `null` or an already deinitialized `CoreManager`.

You can run several `CoreManagers` in one process, such as a host pair or a two-peer test. Each owns its own complete set of managers, and none of their state is shared. Pass the one you mean explicitly instead of reading `Instance`, which can only name one of them.

Driving the loop yourself

Outside Unity, something has to supply the clock that steps the network loop; the types on this page are in `Nucleus.Managers.NetworkLoop`. If you pass no loop step provider to the `CoreManager` constructor, it uses `SystemNetworkLoopStepProvider`, a timer that steps the loop from a thread pool thread at any tick rate up to `NetworkLoopManager.MaximumTickRate` (128). That needs no code.

Write your own provider only when you have a clock the default cannot see, such as a dedicated server's own fixed-step loop, or a test that must step deterministically.

Implement INetworkLoopStepProvider

```
public interface INetworkLoopStepProvider
{
    public bool IsStarted { get; }

    public bool IsTickFrame { get; }

    public void Initialize(NetworkLoopManager networkLoopManager);

    public void Start();

    public void Stop();

    public void Return();
}
```

- `Initialize` hands the provider the `NetworkLoopManager` it will drive.
- `Start` and `Stop` bracket the period the provider may raise steps.
- `Return` tells the provider it is being let go and may pool itself. `Stop` is always called before `Return`.
- `IsTickFrame` says whether the frame being driven runs a tick. Left false, the frame's serialize steps send messages but write no state and raise no callbacks. A provider built on `NetworkLoopStepDriver` forwards `NetworkLoopStepDriver.IsTickFrame`. A provider that invokes the steps itself must set it true before the steps of a pass that ticks, and false otherwise.

Build it on NetworkLoopStepDriver

`NetworkLoopStepDriver` keeps the tick rate for you, as both shipped providers do. Call its `Initialize` once with the manager and the tick interval, then hand it each frame's elapsed time through `AdvanceEarly` and `AdvanceLate`:

```
public sealed class HostClockStepProvider : INetworkLoopStepProvider
{
    private readonly NetworkLoopStepDriver _driver = new();

    public bool IsStarted { get; private set; }

    public bool IsTickFrame => _driver.IsTickFrame;

    public void Initialize(NetworkLoopManager networkLoopManager)
    {
        float tickIntervalMilliseconds = 1000f / networkLoopManager.TickRate;

        _driver.Initialize(networkLoopManager, tickIntervalMilliseconds);
    }

    public void Start() => IsStarted = true;

    public void Stop() => IsStarted = false;

    public void Return() { }

    // Called once per frame from your host's own loop.
    public void OnHostFrame(float frameDeltaMilliseconds)
    {
        if (!IsStarted)
            return;

        _driver.AdvanceEarly(frameDeltaMilliseconds, isVariableUpdateAllowed: true);
        _driver.AdvanceLate(frameDeltaMilliseconds, isVariableUpdateAllowed: true);
    }
}
```

Call `AdvanceEarly` at the start of each frame and `AdvanceLate` at the end of the same frame, with the same frame delta and the same `isVariableUpdateAllowed`. A host with separate early and late phases calls one from each; a host with one frame callback calls both back to back, as above.

Pass your provider to the constructor

```
CoreManager coreManager = new(tickRate: 30, networkLoopStepProvider: new HostClockStepProv
ider());
```

Named in the constructor, your provider replaces the default before it ever starts. `NetworkLoopManager.UseNetworkLoopStepProvider(INetworkLoopStepProvider)` also swaps providers, stopping and returning the one it replaces, but by the time you can call it the default is already stepping the loop from a thread pool thread, racing the rest of your setup.

Step the loop by hand for tests and deterministic simulation

To control exactly when a tick happens, call `NetworkLoopManager.InvokeNetworkLoopStep(NetworkLoopSteps, StepDelta)` yourself.

Install a provider that never steps on its own and lets you set `IsTickFrame`:

```
public sealed class HandDrivenStepProvider : INetworkLoopStepProvider
{
    public bool IsStarted { get; private set; }

    // Set by the code driving the loop: true before the steps of a pass that ticks.
    public bool IsTickFrame { get; set; }

    public void Initialize(NetworkLoopManager networkLoopManager) { }

    public void Start() => IsStarted = true;

    public void Stop() => IsStarted = false;

    public void Return() { }
}
```

Set `IsTickFrame` to true and invoke all eleven steps in this order:

```
HandDrivenStepProvider stepProvider = new();
CoreManager coreManager = new(networkLoopStepProvider: stepProvider);

NetworkLoopManager networkLoopManager = coreManager.NetworkLoopManager;
StepDelta delta = new(0, 0, 0);

stepProvider.IsTickFrame = true;

networkLoopManager.InvokeNetworkLoopStep(NetworkLoopSteps.EarlyVariableUpdate, delta);
networkLoopManager.InvokeNetworkLoopStep(NetworkLoopSteps.EarlyStateUpdate, delta);
networkLoopManager.InvokeNetworkLoopStep(NetworkLoopSteps.LateStateUpdate, delta);
networkLoopManager.InvokeNetworkLoopStep(NetworkLoopSteps.Reconcile, delta);
networkLoopManager.InvokeNetworkLoopStep(NetworkLoopSteps.EarlyFixedUpdate, delta);
networkLoopManager.InvokeNetworkLoopStep(NetworkLoopSteps.LateFixedUpdate, delta);
networkLoopManager.InvokeNetworkLoopStep(NetworkLoopSteps.VariableUpdate, delta);
networkLoopManager.InvokeNetworkLoopStep(NetworkLoopSteps.EarlySerialize, delta);
networkLoopManager.InvokeNetworkLoopStep(NetworkLoopSteps.LateSerialize, delta);
networkLoopManager.InvokeNetworkLoopStep(NetworkLoopSteps.TickAdvance, delta);
networkLoopManager.InvokeNetworkLoopStep(NetworkLoopSteps.LateVariableUpdate, delta);
```

Do not reorder or skip steps: replicated state, messages and RPCs only work when every step runs, in this order. Only `TickAdvance` advances `Tick`.

Drive each CoreManager from one thread

Only one thread may drive a `NetworkLoopManager` at a time. A call to `InvokeNetworkLoopStep` from a second thread while a step is running on another is refused and logs an error. A re-entrant call from the thread already running the step runs normally.

Never hand-drive a `CoreManager` while a provider (the default timer or your own) is also stepping it. Pick one driver for its whole life.

Host mode

Using Unity? See Host mode.

A host is one process that starts a server and connects its own client to it. `TransportManager.IsServerStarted` and `IsClientStarted` are both true, and `IsHostStarted` is `IsServerStarted && IsClientStarted`. It uses the same `CoreManager` and transport as any other peer.

Start a host by connecting both invokers

```
await transport.ConnectAsync(Invoker.Server);
await transport.ConnectAsync(Invoker.Client);
```

A host holds one Connection for its own player

The server's roster entry for the host's own client is the client side's local `Connection` itself. So a call that names the host's own player by its local `Connection`, such as `RpcTarget.To` or `RpcTarget.ObserversExcept`, matches the roster entry the server holds.

Still compare players by `Connection.Id`, because a `Connection` is pooled and reused once its link ends:

```
bool isSelf = someConnection.Id == localClientConnection.Id;
```

Or resolve each side through the `TransportManager` rather than holding a reference across the two halves:

```
transportManager.TryGetLocalClientConnection(out Connection localClient);
transportManager.TryGetServerConnection(out Connection localServer);
```

- `TryGetLocalClientConnection` returns this peer's own client `Connection`. It is false on a pure server.
- `TryGetServerConnection` returns this peer's own server `Connection`. A controlling client uses it to find the upstream observer target.

Both halves of the host's own link are marked IsHostLoopback

`Connection.IsHostLoopback` marks a real, live link whose two ends are this same process. It is neither `IsEmulated` (no socket at all) nor `IsPeerStandIn` (a peer this client cannot reach directly). State sent across a loopback link is discarded on arrival, because the receiving process wrote it.

A custom `Transport` must implement `Transport.IsOwnClientConnection(Connection serverSideClientConnection)`, which answers whether a server-side client connection is this process's own client. A transport that cannot tell must answer false. A false answer only costs the host in-process delivery of its own traffic, but a wrong true answer would hand a stranger connecting over loopback this peer's own server half.

Loopback delivery is always Immediate

`TransportManager.HostLoopbackDelivery` controls when one half of a host sees what it addressed to the other. The only value a game may set is `HostLoopbackDelivery.Immediate`, the default: the receiving half's handlers run at the send site, before the send call returns, and the message is never serialized. Setting anything else logs an error and leaves it at `Immediate`:

```
// Refused: logs an error and leaves HostLoopbackDelivery at Immediate.
transportManager.HostLoopbackDelivery = HostLoopbackDelivery.Deferred;
```

Peer events name real connections on a host

On a client, `TransportManager.PeerConnectionDiscovered` and `PeerConnectionDropped` hand over a stand-in `Connection` (`IsPeerStandIn` true) for a peer the server reported. On a host they name the server's own live roster entry for the joining or leaving peer, so `IsPeerStandIn` is false. A host raises them itself as a peer joins or leaves. Neither is ever raised for the host's own client.

Never pair RpcTarget.To with Immediate self delivery on a host

Do not pair an addressed send (`RpcTarget.To(connection)`) with immediate self delivery (`RpcSelfDelivery.Immediate`) on a surface the host's own client also listens on. The host's handler runs once for the send and once more for the host's own copy, so the host runs every player's copy of a call meant for one. See RpcSelfDelivery.

Core, Managers and the Tick Loop

Pages in this category:

- Connections
- Server, client and host roles
- Session events
- Ticks, tick rate and time
- Host mode
- The network loop steps
- Threading and lifetime
- Creating a CoreManager
- Authenticating clients
- Kick and refusal notices
- Object pacing
- The manager map
- Driving the loop yourself
- Kicking and disconnecting a client
- Shutting a session down
- Keeping the world across a disconnect

Kicking and disconnecting a client

Kick a client from the server

```
coreManager.ServerManager.KickClient(connection);

// With an explicit policy:
coreManager.ServerManager.KickClient(connection, KickOptions.BlockOutgoing);

// Shorthand on the Connection, with the default policy:
connection.Kick();
```

`KickClient` ignores a null `connection`, a `connection` whose `Transport` is already null (it is back in the pool), and a `connection` that already has a kick pending. It does not check that this peer is a server.

KickOptions decides what stops before the kick runs

`KickOptions` is a `[Flags]` enum for what the server stops doing between marking the kick and running it:

Value	Effect
`None`	Keep serving the client normally until the kick executes.
`PurgeOutgoing`	Discard the state of every kind and the remote calls already queued to the client. Messages already queued are kept, which is how a reason sent before the kick still reaches the client.
`BlockOutgoing`	Queue nothing further to the client: no state, no despawns, no recovery, no remote calls and no messages.
`BlockIncoming`	Discard everything received from the client instead of processing it.
`Default`	`PurgeOutgoing

Telling the client why

Pass a notice, any message type of your own, to the kick. The engine sends it on the reliable channel, then marks the kick:

```
[NetworkType]
public struct KickedNotice : IMessage
{
    public string Reason;
}

coreManager.ServerManager.KickClient(connection, new KickedNotice { Reason = "Idle for too
 long." });

// The same call through the Connection shorthand:
connection.Kick(new KickedNotice { Reason = "Idle for too long." });
```

Always pass the notice to the kick rather than sending it yourself: a message sent after `Kick()` is silently dropped. The notice overloads also take a policy: `KickClient<T0>(connection, notice, kickOptions)`.

- The notice is sent only if this call actually kicks. A kick against a client that already has one pending is ignored, notice and all.
- On the client, handle the notice with an ordinary message handler. It is dispatched before the client sees its own disconnect; see Kick and refusal notices.
- The notice's type is how the client tells a kick from any other disconnect. Define one type per reason, or one type with a reason field.
- Leave the notice type's `IsAuthenticationRequired` at its default. Even a refused client reads a notice sent through `DenyClient` (see Authenticating clients).

The kick runs at the end of the frame, with a timeout

The policy applies at once. The disconnect runs once per frame at `LateSerialize`'s place in the loop, after that frame's messages are sent, so a notice has already gone out. A kick marked later in the frame (from a `TickAdvance` or `LateVariableUpdate` callback) runs on the next frame.

The transport then closes the link once the client has received what was already sent, so a client that receives its notice is normally gone almost at once. A client that never acknowledges is closed outright at the start of the first tick after the timeout, measured from the kick:

```
public ulong KickTimeoutMilliseconds = 500;
```

`ServerManager.KickTimeoutMilliseconds` is the timeout every kick uses unless you pass one. On Synapse, keep your own timeout above 250 ms: Synapse resends a lost reliable message every 250 ms, so a shorter timeout can close the link before a lost notice is resent. To give one kick a different timeout:

```
coreManager.ServerManager.KickClient(connection, KickOptions.Default, timeoutMilliseconds:
 2000);
coreManager.ServerManager.KickClient(connection, new KickedNotice { Reason = "Server resta
rting." }, KickOptions.Default, timeoutMilliseconds: 2000);
connection.Kick(timeoutMilliseconds: 2000);
connection.Kick(new KickedNotice { Reason = "Server restarting." }, timeoutMilliseconds: 2
000);
```

A marked kick cannot be stopped or loosened. A second kick against the same client is ignored, and the first policy and timeout stand. A client that leaves on its own before the kick runs is simply dropped from it.

Read the pending state on the Connection

- `IsKickPending`: true from the moment the kick is marked until the connection closes.
- `PendingKickOptions`: the policy that was applied.
- `IsUsable`: true only while there is no pending kick and the remote or local state is connected. This one check covers a kicked or already disconnected peer.

Violation kicks only run on a server

A violation that settles on the `Kick` action calls `ServerManager.KickClient` only when `IsServerStarted` is true. On a client it logs that the kick is not enforced instead. `KickClient` itself has no such check. See Violations.

The manager map

Using Unity? See The manager map for the Unity component that fronts each manager.

Each concern has one manager

`CoreManager` holds eleven managers as public fields, for the life of the session. Reach each one as a field, such as `coreManager.TransportManager`.

Manager	Owns
`NetworkLoopManager`	The tick loop: tick rate and the steps that drive a tick forward.
`TransportManager`	The transports a session sends and receives over.
`ServerManager`	Server-side session state.
`ClientManager`	Client-side session state.
`SystemManager`	Networked systems: spawning, ownership, and their lifecycle.
`InterestManager`	What each peer is allowed to see.
`SceneManager`	Scene load, stacking, and per-scene state.
`PacketManager`	Packet framing and dispatch.
`MessageManager`	Messages sent between peers.
`RpcManager`	Remote procedure calls.
`ViolationManager`	Detecting and acting on protocol violations.

A Pro build adds two more fields. In a Free build they are absent, not null.

Manager	Owns
`BundleManager`	Content delivered from outside the build (Addressables, a patcher, a CDN).
`WorldPersistenceManager`	Writing a world to disk and building it back.

Every setting lives on the manager that owns it

There is no single settings type. Tick rate, a transport's connection settings, interest rules and scene load behavior are each fields on the manager that owns them, so to change something, find its manager first.

None of the managers are static. Each belongs to one `CoreManager`, and a process can construct several `CoreManagers`, each with its own managers, such as a server and a client in one run or several test peers side by side.

The network loop steps

Using Unity? See The network loop steps.

Eleven steps run in a fixed order

Every frame walks the loop's steps in this order, and a frame that runs a tick walks all eleven:

`EarlyVariableUpdate`, `EarlyStateUpdate`, `LateStateUpdate`, `Reconcile`, `EarlyFixedUpdate`, `LateFixedUpdate`, `VariableUpdate`, `EarlySerialize`, `LateSerialize`, `TickAdvance`, `LateVariableUpdate`.

Each is one flag of the `NetworkLoopSteps` enum.

- **Variable steps run every frame.** `EarlyVariableUpdate`, `VariableUpdate` and `LateVariableUpdate` run on every frame, whatever the frame rate.
- **Tick steps run once per tick, on the frame that ticks.** `EarlyStateUpdate`, `LateStateUpdate`, `Reconcile`, `EarlySerialize`, `LateSerialize` and `TickAdvance` are where replicated state is applied, reconciled and written, and where the tick advances.
- **Fixed steps run once per elapsed tick interval.** `EarlyFixedUpdate` and `LateFixedUpdate` run on the frame that ticks, once for each tick interval that has passed, so after a long frame they can run twice on one frame. A frame still runs at most one tick.

A frame that does not reach the next tick runs only its variable steps, so do not assume a tick step or a fixed step fires every frame.

Some steps run engine work before or after your callback

- **`EarlyVariableUpdate`** receives packets before your callback, and dispatches received messages to their handlers after it. React to something a message delivered later than this step.
- **`LateStateUpdate`** catches `PresentedTick` up to `Tick`, applies received state, then dispatches received RPCs, all before your callback.
- **`EarlyFixedUpdate`**, in Pro, first advances a newly spawned object that is still catching up, so its step for the tick has happened before anything simulates against it.
- **`LateSerialize`** serializes changed state before your callback, then sends the frame's messages, RPCs and kicks after it. A write made in this callback is too late for this tick's state, but a message or call sent from it still leaves this frame.
- **`TickAdvance`**, after your callback, finishes stopping systems whose stop was pending, drops clients that did not authenticate in time, increments `Tick`, then starts systems whose start was deferred. So `LateVariableUpdate`, the frames between ticks and every message handler already read the tick their writes will ride.

The other steps run only your callbacks.

Messages, RPCs and kicks are sent on every frame, not only on frames that tick: a frame that does not tick still sends them at `LateSerialize`'s place, without raising a callback. Anything queued from a `TickAdvance` or `LateVariableUpdate` callback leaves on the next frame.

Read after the state applies and write before it serializes

- **Read replicated state on `LateStateUpdate` or later.** A read on `EarlyStateUpdate` or earlier can still see last tick's value.

- **Write on `EarlySerialize` to send this tick.** A write on `LateSerialize` or later rides the next tick.
- **Put per-frame code on a variable step,** such as `VariableUpdate`, and move things by `StepDelta.NormalizedDelta` rather than the raw frame delta, so each tick sees an even distance (see Ticks, tick rate and time).

Typed messages are dispatched on `EarlyVariableUpdate` and system RPCs on `LateStateUpdate`; see When a Message or Call Reaches Its Handler.

Hook into steps with INetworkLoopStepCallback

```
public interface INetworkLoopStepCallback
{
    public NetworkLoopSteps GetNetworkLoopSteps() => NetworkLoopSteps.None;

    public void OnNetworkLoopStep(NetworkLoopSteps networkLoopStep, StepDelta stepDelta);
}
```

`GetNetworkLoopSteps` declares the steps you want and can combine any number of flags. `OnNetworkLoopStep` is called once per declared step, each time with just that one flag and its `StepDelta`.

```
public class ScoreTracker : INetworkLoopStepCallback
{
    public NetworkLoopSteps GetNetworkLoopSteps() =>
        NetworkLoopSteps.LateStateUpdate | NetworkLoopSteps.EarlySerialize;

    public void OnNetworkLoopStep(NetworkLoopSteps networkLoopStep, StepDelta stepDelta)
    {
        if (networkLoopStep == NetworkLoopSteps.LateStateUpdate)
        {
            // Read applied state here.
        }
    }
}
```

Register it with the `NetworkLoopManager`, and unregister it when the object is torn down:

```
public void RegisterNetworkLoopStepCallbacks(INetworkLoopStepCallback networkLoopStepCallb
ack)
public void UnregisterNetworkLoopStepCallbacks(INetworkLoopStepCallback networkLoopStepCal
lback)

coreManager.NetworkLoopManager.RegisterNetworkLoopStepCallbacks(scoreTracker);
```

In Unity, `NucleusBehaviour` exposes each step as a virtual (`OnEarlyVariableUpdate` through `OnLateVariableUpdate`) and registers only for the steps your type overrides.

A callback registered during a step takes effect from the following step. A callback unregistered during a step is still invoked for the rest of that step.

To find a registered callback of a known type without keeping your own reference, pass a single step, not a combination:

```
public bool TryGetFirstNetworkLoopStepCallback<T0>(NetworkLoopSteps networkLoopStep, out T
0 networkLoopStepCallback)
    where T0 : class, INetworkLoopStepCallback
```

A callback that throws does not stop the step

The exception is caught and logged, and the step goes on to its remaining callbacks and engine work:

```
The loop callback [<CallbackType>] threw during [<Step>]; the step went on to its remainin
g callbacks: [<exception>].
```

Test a step without allocating

`NetworkLoopStepsExtensions.FastContains` checks one `NetworkLoopSteps` value for another without allocating:

```
if (networkLoopStep.FastContains(NetworkLoopSteps.EarlyFixedUpdate))
{
    // ...
}
```

Object pacing

> **Using Unity?** The budget is the **Maximum Object Kilobytes Per Tick** slider (1 to 255) under the **Object Pacing** header on the Unity Server Manager component. See Manager components reference.

Object pacing spreads a burst of new objects, such as a client joining a large world, a scene binding or a bundle releasing its withheld spawns, over several ordinary ticks instead of one huge one. A smaller tick is less likely to be lost, which matters most to a peer still joining, and the receiver's instantiation cost spreads over several frames.

This is a Pro feature. The Free edition does no pacing, ignores the settings below and has no **Object Pacing** header in Unity.

Set a budget in kilobytes per client per tick

```
coreManager.ServerManager.MaximumObjectKilobytesPerTick = 20;
```

`ServerManager.MaximumObjectKilobytesPerTick` is the most whole-object data, in kilobytes (1 kB is 1,024 bytes), that any one `Connection` is sent on a single tick. Whole objects are new objects (spawns) and objects a client is sent again whole after a loss. Ordinary updates to objects a client already holds are not paced. Whatever does not fit is sent on the following ticks, oldest first.

- `DefaultMaximumObjectKilobytesPerTick` is `50`.
- `MinimumObjectKilobytesPerTick` is `1`. A value below it is read as `1`.
- The widest budget is `255` kilobytes.
- `ServerManager.ObjectPacingEnabled` turns pacing on and off, and is on by default. Off, every whole object is sent on the tick it is owed, and objects still waiting are sent on the next tick.

Choose a budget a client can take in one tick

Keep the budget times the tick rate under what the slowest client's connection carries in a second: at 30 ticks a second, 8 kilobytes a tick is about 240 kilobytes a second. Raise it for a fast LAN, or for a world of many small objects that should arrive faster. Lower it for slow links.

If `ConnectionsAwaitingSpawnAdmissionCount` sits near zero outside bursts (a join, a scene binding, a bundle release), ordinary play fits the budget. A stall warning in the log means the world spawns faster than the budget lets a peer receive: raise the budget or spawn less at once.

Watch these counters

- `InterestManager.TotalSpawnsAdmitted` counts spawns sent to clients, once per connection a spawn was sent to.
- `InterestManager.TotalSpawnsDeferred` counts spawns that had to wait, once per connection when a system first waits, not again for each tick it waits.
- `InterestManager.ConnectionsAwaitingSpawnAdmissionCount` is how many connections are owed at least one waiting spawn.
- `Connection.SystemsAwaitingSpawnAdmissionCount` is one connection's backlog. It rises on a burst and falls each tick as the budget lets objects through.
- `InterestManager.SpawnAdmissionStallWarningTicks` (`90`) is how many ticks in a row a connection still owed spawns may receive none before the stall is logged.

Read `TotalSpawnsDeferred` against `TotalSpawnsAdmitted` as a ratio of how hard the budget is

working, not as a count of objects still waiting; use the two awaiting counts for that. A world built from `NetworkSystemGroups` reads high, because a grouped system always waits so its group can start whole, and is counted even when it starts on the next tick.

Pacing keeps these promises

- **A group is never left half started.** A `NetworkSystemGroup` starts as a whole, even when the budget has room for only some of its members, and a group wider than the whole budget still starts, on a tick of its own. Its objects count against the budget as they are sent, so a wide group can take more than one tick to arrive. In Unity the object is announced once every member has arrived.
- **An object wider than the whole budget still arrives.**
- **A peer's own controlled object never waits.**
- **Objects sent again after packet loss share the budget,** so recovering from loss never arrives as one burst either.
- **A host's own client and emulated peers are never paced.**
- **A receiver cannot tell a paced spawn** from one that arrived late for any other reason.

Keeping the world across a disconnect

Choose what a dropped client does with its world

`ClientManager.DisconnectResetMode` decides what happens to a client's world when its link to the server drops:

```
coreManager.ClientManager.DisconnectResetMode = DisconnectResetMode.RetainReceivedWorld;
```

- **`ClearReceivedWorld`** (the default) despawns every `NetworkSystem` the server sent and releases every networked scene it placed the client in.
- **`RetainReceivedWorld`** leaves every received `NetworkSystem` started and every networked scene loaded, as they stood when the link dropped.

Use `RetainReceivedWorld` only on a peer that will start its own server to carry on hosting the world it was shown. A retained world still carries the departed server's identifiers, so it is only usable once this peer adopts it. A client that reconnects elsewhere drops everything and rebuilds from the new server, whatever this setting says.

Honoring `RetainReceivedWorld` is Pro-only. The field exists in every edition, but a Free client always clears its world on disconnect.

Adopt a retained world before starting the server

Wait until the link has dropped.

Call `ServerManager.TryAdoptRetainedWorld()`. It returns `true` when a world was adopted and `false` when this peer holds nothing it was sent. It refuses to run once `TransportManager.IsServerStarted` is true.

Start the server.

If this peer's own player should keep playing, connect its client half afterwards. A pure-server successor holds the world but has no character in it.

```
coreManager.ServerManager.AuthorityAdopted += OnAuthorityAdopted;

if (coreManager.ServerManager.TryAdoptRetainedWorld())
    await transport.ConnectAsync(Invoker.Server);

void OnAuthorityAdopted(uint adoptedSystemCount, uint adoptedSceneCount)
{
    // Re-attach whatever this peer only did as a spectator while it was a client.
}
```

On success it raises `AuthorityAdopted(uint adoptedSystemCount, uint adoptedSceneCount)`, so the game can re-attach whatever it only does for objects it owns. Adoption keeps every identifier the world already has, so no object despawns and respawns.

Rebuild instead of reconciling on reconnect

A retained world normally reconciles against the new server: identifiers this peer already holds apply in place, and objects never blink. Set `SystemManager.RespawnWorldOnReconnectEnabled` to drop the world and rebuild it from the server's account instead, at the cost of a spawn per object, without carrying anything over that the server never heard of. With it set, `RetainReceivedWorld` no longer changes how reconnecting behaves.

Tell survivors where to regroup before the host drops

Once the old host drops there is no channel left to tell the other peers where the promoted host is, so tell them beforehand with `SessionDirectoryMessage`:

```
public struct SessionDirectoryMessage : IMessage
{
    public ulong SessionId;

    public const ulong NoSession = 0;
}
```

Send it over `Channel.Reliable` once a client authenticates, so every client that joins later gets it too:

```
coreManager.ServerManager.ClientAuthenticated += connection =>
{
    connection.SendMessage(Channel.Reliable, new SessionDirectoryMessage(sessionId));
};
```

`SessionId` names the session in whatever directory the game uses, or is `SessionDirectoryMessage.NoSession` when the host is not registered with one. Nucleus carries only the identifier; the directory, address and protocol are up to the game.

Treat `SessionId` as a secret shared only with the players. A directory cannot tell a peer that was told it from one that wasn't, so whoever holds it can generally take the session over.

Reconnecting the survivors to the promoted host is covered in Choosing a hosting topology and Relay transport.

Server, client and host roles

Using Unity? See Starting and stopping a session in Unity.

Invoker names the role an operation acts for

```
public enum Invoker : byte
{
    Client = 0,
    Server = 1,
}
```

`Invoker` is a parameter, not a status check: it does not assert that the role is started.

Start a role on a transport

Construct and configure the transport (see the transports pages), add it to the `TransportManager`, then connect each role you want:

```
await transportManager.TryAddTransportAsync(transport);

await transport.ConnectAsync(Invoker.Server);
await transport.ConnectAsync(Invoker.Client);
```

`ConnectAsync(Invoker.Server)` begins listening, and `ConnectAsync(Invoker.Client)` connects the local client to the server, using the transport's current configuration. Calling both starts a host.

Read which roles are started

`TransportManager` member	True when
`IsServerStarted`	The local server connection is connected.
`IsClientStarted`	The local client connection is connected **and authenticated**.
`IsHostStarted`	`IsServerStarted` and `IsClientStarted` are both true.

`IsClientStarted` waits for authentication, not just the socket, so gameplay code never sees a client as started before the server accepts it.

`NetworkSystem` has the same three members, so gameplay code can branch without reaching the transport layer:

```
if (networkSystem.IsServerStarted)
{
    // Authority-only logic.
}
```

Being the server and controlling a system are separate questions. Use `IsServerStarted` for the first and `NetworkSystem.IsController(ControllerType)` for the second; see What control means. The guards `EnsureIsController(ControllerType)` and `EnsureIsStarted(Invoker)` exist only on the Unity `NucleusBehaviourBase`.

Stop a role

```
await transport.DisconnectLocalConnectionAsync(Invoker.Client);

await transport.ShutdownAsync();
```

`DisconnectLocalConnectionAsync(Invoker)` disconnects the local connection for that role and closes its open sockets. `ShutdownAsync` stops all of a transport's local connections at once. Neither removes the transport from the `TransportManager`; it stays added, ready to `ConnectAsync` again.

Check the ConnectionStateChangeResult

```
public enum ConnectionStateChangeResult : byte
{
    Success,
    AlreadyInState,
    InvalidConnection,
    InvalidSocketState,
    UnspecifiedError,
}
```

`ConnectAsync` and `DisconnectLocalConnectionAsync` return it. `ShutdownAsync` does not. Code that did not make the call can react to a role starting or stopping through `TransportManager.ConnectionLocalStateChanged`; see Connections.

Kick and refusal notices

The engine attaches no reason to a kick or a refusal. To tell the client why, the server passes a notice, an ordinary message type of your own, to the kick or refusal, and the client receives it before it is disconnected.

Define one notice type per reason

The client tells a kick or refusal from any other disconnect by the notice's type. Give each reason you want to tell apart its own type, or carry the detail in a field:

```
[NetworkType]
public struct ServerFullNotice : IMessage
{
    public string Text;
}

[NetworkType]
public struct KickedNotice : IMessage
{
    public string Text;
}
```

Leave `IsAuthenticationRequired` at its default. A message from the server reaches a client that is not yet authenticated whatever that property says, so a refusal's notice still arrives.

Pass the notice to DenyClient or Kick

```
// Refusing a connecting client from an authenticator.
coreManager.ServerManager.DenyClient(connection, new ServerFullNotice { Text = "Server is
full." });

// Kicking a client already in the session.
connection.Kick(new KickedNotice { Text = "Idle for too long." });
```

The plain `DenyClient(connection)` and `Kick()` send nothing. See Kicking and disconnecting a client for the policy and timeout, and Authenticating clients for when to refuse.

Register the handlers on the client before connecting

Register a handler for each notice with `MessageManager.RegisterMessageHandler<T0>` before connecting, so a refusal during the handshake is not missed:

```
public sealed class DisconnectReasons
{
    public string LastReason { get; private set; }

    public DisconnectReasons(MessageManager messageManager)
    {
        messageManager.RegisterMessageHandler<ServerFullNotice>(OnServerFull);
        messageManager.RegisterMessageHandler<KickedNotice>(OnKicked);
    }

    private void OnServerFull(in MessageContext messageContext, ServerFullNotice notice) =
> LastReason = notice.Text;

    private void OnKicked(in MessageContext messageContext, KickedNotice notice) => LastRe
ason = notice.Text;
}
```

The notice reaches its handler before the client's own disconnect is raised, so a disconnect handler can read what the notice handler recorded, such as `LastReason` above. When no notice handler ran, the disconnect had another cause: the server sent no reason, the link timed out, or the server shut down.

Session events

Using Unity? See Reacting to session events in Unity.

Each event allows some work and asks you to defer the rest

Event	Safe here	Defer instead
`ServerManager.ClientAuthenticated`	Read `Connection.Identity`; do per-connection setup.	Anything that assumes the connection already observes something; that may not be true until a round trip later.
`ServerManager.ClientDisconnecting`	Look up what this connection controlled or owned; nothing has been released yet.	Anything that needs its controlled systems already gone; they are released right after this returns.
`ClientManager.LocalClientAuthenticated`	Read this peer's own `Connection.Id` for the first time.	Assuming the roster of other peers has arrived; it arrives separately.
`TransportManager.PeerConnectionStarted`	Store a reference to the peer's `Connection`.	
`TransportManager.PeerConnectionDropped`	Drop every stored reference to the peer's `Connection`.	Holding it past this handler.

ClientAuthenticated fires when the server admits a client

```
public event ClientAuthenticatedHandler? ClientAuthenticated;
public delegate void ClientAuthenticatedHandler(Connection connection);
```

Raised on the server from inside `ApproveClient`. The connection is already marked authenticated and its `Connection.Identity` is stamped, so a handler can read both.

- The client has not been told it was accepted yet, so nothing sent from here reaches it ahead of its own identity.
- Other clients have not been told about this connection yet.
- Interest may not have registered the connection as an observer yet. A connection whose `Connection.CanReceiveState` is already true, typically a host's own client, is registered at once. A remote client is registered shortly after it connects.

ClientDisconnecting fires while a dropped client's objects are still readable

```
public event ClientDisconnectingHandler? ClientDisconnecting;
public delegate void ClientDisconnectingHandler(Connection connection);
```

Raised on the server for a client that has dropped. Inside the handler you can still look up which systems the connection controlled; once it returns, they are released. Use this event rather than a lower-level transport disconnect event, whose handler order is not guaranteed.

LocalClientAuthenticated tells a peer who it is

```
public event LocalClientAuthenticatedHandler? LocalClientAuthenticated;
public delegate void LocalClientAuthenticatedHandler(Connection localClientConnection);
```

The local client connection's `Connection.Id` is `Connection.UnsetId` from socket connect until this fires. It is raised when the server's authentication response arrives, after the `Id` is set to the server-assigned value and the connection is marked authenticated. A host's own client half raises it the same way, so handlers need no special case for a host.

PeerConnectionDiscovered and PeerConnectionDropped describe other peers

```
public event PeerConnectionChangedHandler? PeerConnectionDiscovered;
public event PeerConnectionChangedHandler? PeerConnectionDropped;
public delegate void PeerConnectionChangedHandler(Connection connection);
```

These never fire for this peer's own session.

- On a client, `PeerConnectionDiscovered` fires when the server reports a new peer. The `Connection` is a stand-in, an identity with no live link behind it (`Connection.IsPeerStandIn` is true).
- On a host, it fires once `ApproveClient` admits the peer. The `Connection` is the server's own entry for that peer, so `IsPeerStandIn` is false. It never fires for the host's own client.

`PeerConnectionDropped` mirrors it: on a client when the server reports the peer gone, on a host when the peer leaves, in both cases while the entry is still held. Drop every reference to the `Connection` inside this handler.

A host raises server and client events in one call

- For the host's own client, `ClientAuthenticated` fires and then `LocalClientAuthenticated`, inside the same `ApproveClient` call rather than a round trip later.
- For a remote peer joining a host, `ClientAuthenticated` and then `PeerConnectionDiscovered` fire for that peer out of the same `ApproveClient` call.
- On departure, `ClientDisconnecting` fires before the connection's controlled systems are released, and `PeerConnectionDropped` fires for the same departure, never for the host's own client.

Shutting a session down

Using Unity? See Play mode, domain reload and session lifetime.

Call Deinitialize once when the session is done

```
coreManager.Deinitialize();
```

`CoreManager.Deinitialize()` stops the network loop, shuts every transport down and releases every manager the `CoreManager` owns. Call it from whatever code owns the session's lifetime.

A second call returns immediately, because `IsDeinitialized` is set before the teardown starts. So an explicit teardown followed by a destroy callback (a platform's `OnDestroy`, for example) is safe. You can still check first:

```
if (!coreManager.IsDeinitialized)
    coreManager.Deinitialize();
```

A torn-down `CoreManager` cannot be reused. To run another session, construct a new one.

Deinitialized is raised last

```
public event DeinitializedHandler? Deinitialized;
public delegate void DeinitializedHandler(CoreManager coreManager);
```

`Deinitialized` is raised after every manager has been released and after `Instance` has moved on to the next live `CoreManager` (or `null`). The `CoreManager` and its manager fields are still readable during the raise, so a consumer that keyed something per loop can find its key through `NetworkLoopManager` and drop its entry:

```
coreManager.Deinitialized += static tornDown =>
{
    // tornDown.NetworkLoopManager is still readable here.
};
```

Call ResetRegistry before each run when sessions run back to back

`CoreManager` keeps a static registry of every live manager, which `Instance` resolves from. A process that runs one session after another in the same run calls `CoreManager.ResetRegistry()` at the very start of the next run, before anything constructs a new `CoreManager`:

```
CoreManager.ResetRegistry();
```

It clears the registry and sets `Instance` to `null`, so the new run cannot inherit managers from the one before. It only drops references and tears nothing down, so a session that wants an orderly teardown still calls `Deinitialize()` while it is live.

Remove one transport without tearing the session down

```
await coreManager.TransportManager.TryRemoveTransport(transport);
```

`TryRemoveTransport` removes the transport from the manager, awaits `Transport.ShutdownAsync()` on it, and rescans the local server and client connections, while the rest of the session keeps running. It returns `false` if the transport was not in the list.

Threading and lifetime

Callbacks run on a thread pool thread by default

With no loop step provider, `CoreManager` uses `SystemNetworkLoopStepProvider`, which runs a `System.Timers.Timer` and so steps the loop on a thread pool thread, not the thread that constructed the `CoreManager`.

Every loop step callback (`INetworkLoopStepCallback.OnNetworkLoopStep`), message handler, RPC handler and `NetworkMember<T0>` changed callback runs inside a loop step, so with the default provider all of it runs on a pool thread. Code that touches a UI collection, a non-thread-safe cache or anything else not built for concurrent access must not assume it is on the constructing thread, unless you supplied your own provider.

The Unity integration drives the loop from Unity's main thread instead, so callbacks, handlers and scene and bundle continuations land there.

Name your provider in the constructor

```
public CoreManager(uint tickRate = NetworkLoopManager.DefaultTickRate, INetworkLoopStepPro
vider? networkLoopStepProvider = null)
```

With `null`, the constructor starts the default provider as its last step, so by the time it returns the loop is already running on a pool thread. Anything you register afterwards, such as `RegisterNetworkLoopStepCallbacks` calls or your own systems, races a loop that is already stepping; there is no guaranteed idle window. In a console host, pass your own `INetworkLoopStepProvider` to the constructor, so nothing steps until you say so. See Driving the loop yourself.

Only one thread may step a loop at a time

A call to `NetworkLoopManager.InvokeNetworkLoopStep` from a second thread while a step is running on another is refused and logs an error, and nothing registered on that step runs for it. A re-entrant call from inside a step, on the thread already running it, runs normally.

If you see this error, the usual cause is a `CoreManager` left on the default timer while something else, such as a manual step or a second timer, also drives it.

Each `NetworkLoopManager` has its own callback registrations. Two `CoreManagers` in one process, such as a host pair or a two-peer test, never dispatch each other's callbacks.

Debug builds catch cross-thread access

A Debug build of `Nucleus.dll` logs an error when Nucleus is called from the wrong thread while a step is running. Release builds do not check, so a threading mistake can go unnoticed there. If you are chasing a corrupted collection or a torn write, run the reproduction in Debug and check the log for these errors.

Loaders may complete on any thread

`ISceneLoader.LoadSceneAsync` and `UnloadSceneAsync` return `Task<bool>` and may await freely, such as a download, a disk read or an engine operation, without minding which thread the continuation resumes on. The loop never blocks on them, and the engine handles the outcome on the loop thread, so you never have to return to it yourself.

`IBundleLoader` (Pro-only) is the same for bundles: `LoadBundleAsync` and `UnloadBundleAsync` return `Task<bool>` and may complete on any thread.

Ticks, tick rate and time

Using Unity? See Tick rate and the loop in Unity.

Set the tick rate when you construct the CoreManager

`NetworkLoopManager.TickRate` is the number of ticks per second. It comes from the `tickRate` argument of the `CoreManager` constructor and has no setter:

```
public uint TickRate { get; }

CoreManager coreManager = new(tickRate: 60);
uint rate = coreManager.NetworkLoopManager.TickRate; // 60
```

- The default is `NetworkLoopManager.DefaultTickRate` (30).
- A rate outside `NetworkLoopManager.MinimumTickRate` (5) through `NetworkLoopManager.MaximumTickRate` (128) is clamped to the nearer bound and logged as a warning.
- The rate cannot change mid-session. A new rate means a new `CoreManager`.

Tick counts this peer's ticks

```
public uint Tick { get; private set; } = FirstTick;
```

`NetworkLoopManager.Tick` is the current local tick. It increments at the very end of the `TickAdvance` step, after its callbacks, so `LateVariableUpdate`, the frames between ticks and every message handler already read the tick their writes will ride.

It starts at `FirstTick` (1). `UnsetTick` is 0, and a live tick is never the unset value.

`Tick` is this peer's own counter. A client's `Tick` and the server's are independent counters that run at the same rate, not a shared clock. To estimate the server's tick, use `Connection.TryGetServerTickEstimate`:

```
public bool TryGetServerTickEstimate(uint localTick, out uint serverTickEstimate)
```

It derives the estimate from the newest state received from that connection, and returns `false` until at least one has arrived.

SubtickPercentage is how far through the tick you are

```
public float SubtickPercentage { get; private set; }
public void SetSubtickPercentage(float percentage)
```

`SubtickPercentage` is the fraction, 0 through 1, of the current tick that has elapsed. `SetSubtickPercentage` clamps into that range. `NetworkLoopStepDriver` sets it on every frame that runs a variable update, just before `VariableUpdate`, so any provider built on the driver (including the default and the Unity provider) keeps it current; do not set it yourself from such a provider. A provider that invokes the steps without the driver leaves it at zero, which is safe: interpolation then resolves to the previous value.

Every step callback receives a StepDelta

```
public struct StepDelta
{
    public long Delta;
    public long FixedDelta;
    public long TimeSinceLastFixedUpdate;
    public double NormalizedDelta;
}
```

All four are in milliseconds:

- `Delta`: time since the step last ran. Use it for variable-timing work.
- `FixedDelta`: the step's fixed delta when one applies, and zero otherwise. Use it for work that needs consistent timing, such as physics or anything keyed to the tick rate.
- `TimeSinceLastFixedUpdate`: time since the last fixed update completed. It does not change until that fixed update finishes.
- `NormalizedDelta`: the frame's delta adjusted so the frames between two ticks add up to exactly one tick interval. `NetworkLoopStepDriver` sets it, so it is zero when you invoke the steps yourself.

Move per-frame work by NormalizedDelta

Frames never line up with ticks, so an object moved by each frame's raw delta covers a different distance every tick, and uneven per-tick movement costs more bandwidth to replicate. `NormalizedDelta` keeps the object moving every frame but evens out the distance per tick, without losing time or changing its speed: time past one tick interval is carried onto the next frame. A single frame longer than a tick interval still gets its whole delta, plus anything carried.

Read it on `VariableUpdate` and convert it to seconds:

```
public void OnNetworkLoopStep(NetworkLoopSteps networkLoopStep, StepDelta stepDelta)
{
    float seconds = (float)(stepDelta.NormalizedDelta / 1000.0);
    position += velocity * seconds;
}
```

To run your own code on the loop, see The network loop steps.

Common errors and what causes them

Every line below is quoted exactly as the engine writes it.

Find your log line

- ["ran on thread [X] while a network loop step is executing on thread [Y]..."](#a-message-was-written-off-the-loop-thread): off-loop writer work
- ["touched the loop-owned collection [...] on thread [X] while a network loop step is executing on thread [Y]..."](#loop-owned-state-was-touched-off-the-loop-thread): off-loop collection access
- ["Step [X] was turned away on thread [...] because this manager is already executing a step on thread [...]..."](#two-threads-are-driving-one-manager): a second driver stepping the loop
- ["Violation [X] from Connection [...]"](#a-violation-was-raised-against-a-peer): a violation raised against a peer
- ["A Recovery entry from Connection [...] references unknown NetworkSystem Id [...]; that entry is skipped."](#recovery-named-something-the-receiver-cannot-use): recovery named a system the local world no longer has
- ["...acknowledges [an impossible tick]..." (InvalidStateAckViolation)](#recovery-named-something-the-receiver-cannot-use): an acknowledgment above the local tick
- "...retention window was exceeded..." (RetentionExceededViolation): an acknowledged tick fell out of the retention window
- ["A StatePacket subpacket from Connection [...] declares [X] body bits but only [Y] remain; discarding the rest of the tick."](#a-state-update-could-not-be-read): a state update that could not be read
- ["State subpacket [X] from Connection [...] left the reader at [...] bits but its framed boundary was [...]..."](#a-state-update-could-not-be-read): a state update that could not be read
- ["Connection [...] sent unrecognised PacketType [X]; the packet is discarded."](#a-peer-runs-a-different-build): version skew, not a kick

A message was written off the loop thread

`"[Caller<MessageType>] ran on thread [X] while a network loop step is executing on thread [Y]. This work must not run concurrently with the loop; it races the outbound WriterPool and can corrupt sent packets."`

Cause. Something wrote a message from a thread other than the one running a network loop step.

Fix. Move the work onto the loop. If it resumes after an `await`, route it through the loop rather than sending from wherever the continuation landed.

This check exists only in a `DEBUG` build. A Release build still has the race but logs nothing, so reproduce in Debug and confirm the fix in Release.

Loop-owned state was touched off the loop thread

`"[Caller] touched the loop-owned collection [Name] on thread [X] while a network loop step is executing on thread [Y]. This work must run on the network loop thread; touching the collection off the loop races the loop and can silently corrupt it."`

Cause. An async continuation resumed off the loop thread and touched state the loop owns. This can

corrupt it or drop an entry.

Fix. Switch back onto the loop before touching loop-owned state.

This check also exists only in a `DEBUG` build. In Release the race remains, unreported.

Two threads are driving one manager

`"Step [X] was turned away on thread [...] because this manager is already executing a step on thread [...]. One loop must drive one manager, and a second driver races every collection the loop owns. A manager whose owner never supplied a step provider, leaving the default one driving the loop from the thread pool, is the usual cause. At least [N] steps have been turned away since the previous report."`

Cause. Two threads are stepping the same manager's loop. Usually the owner never supplied its own `INetworkLoopStepProvider`, so the default background-timer provider drives the loop from the thread pool alongside whatever else steps it.

Fix. Make sure exactly one driver steps the manager: supply the owner's own step provider (a Unity integration's loop provider, or a harness driving ticks by hand).

The line is logged at most once per minute per manager. The count (`At least [N] steps have been turned away`) shows the real severity: a count of 1 differs greatly from one in the thousands.

A violation was raised against a peer

`"Violation [TypeName] from Connection [...]"`, sometimes followed by `"...settled on a kick, but this peer is not a server; the kick is not enforced."` or `"Disconnecting Connection [...] for violation [TypeName]."`

The first line is written when the violation carries a log entry. The second and third follow a `Kick`. Most types default to `Ignore` and write nothing, so register an `IViolationObserver` to see them. Look up `TypeName` in the Violation reference for its payload, its cause and what to check first.

The offending operation is always rejected. The action only decides what happens to the Connection. See Violations.

Recovery named something the receiver cannot use

`"A Recovery entry from Connection [...] references unknown NetworkSystem Id [X]; that entry is skipped."`

The sender believed the receiver still held a system the local world no longer has. The entry is skipped and the rest of the recovery continues.

`InvalidStateAckViolation`, `RetentionExceededViolation` and `RecoveryUnconfirmedViolation` default to `Ignore`. A single raise is ordinary for a peer on a bad link. Raises that keep recurring for the same Connection mean its latency exceeds the retention window, or something is wrong with how it acknowledges.

A state update could not be read

`"A StatePacket subpacket from Connection [...] declares [X] body bits but only [Y] remain; discarding the rest of the tick."`

`"State subpacket [Type] from Connection [...] left the reader at [...] bits but its framed boundary was [...] (declared [X] body bits, off by [Y]). The stream is misaligned; discarding the rest of the tick."`

The receiver could not read a state update the way the sender wrote it. This points at the two builds disagreeing about a type's serialized shape, not at game logic. The rest of the tick is discarded so nothing unreadable is applied.

A peer runs a different build

`"Connection [...] sent unrecognised PacketType [X]; the packet is discarded."`

An older or newer build sent a packet type this build does not know. The packet is discarded without a violation, since a version mismatch is not proof of a hostile peer.

Diagnosing desync

Work the layers in order

"It looks wrong on the other peer" is four different bugs with one symptom. Check them in this order, because each rules out the ones below it:

Is the object there at all? If the receiver never spawned it, or it vanished, this is interest: an `IInterestCondition` culled the pair (`InterestEffect.Spawn`, resolving to `InterestMembership.Unspawned`), or a real despawn happened.

Is it there but frozen? This is `InterestEffect.Stop`: the pair resolved to `InterestMembership.Stopped`, which keeps the receiver's object but stops sending it updates. It looks like a dead connection, but it is not.

Is it there and moving, but wrong? A member holds a value that does not match the sender. Either the value never changed on the receiver, or it changed to something incorrect.

Is it there and correct, but late? This is interpolation and the retention window, not a data bug.

Do not skip ahead. A value that "looks wrong" because the object was culled five seconds ago is not a serializer bug.

Split a wrong value into never arrived or arrived wrong

At step 3, decide which of these you have before touching a serializer:

- **The value never changes on the receiver.** This is observation or access. Is the Connection an observer of that `NetworkSystem`? Did an interest condition stop it (step 2)? Does the sender own or control the member it is changing? Usually the write never left the sender or never reached this receiver.
- **The value changes, but to the wrong thing.** This is serialization or prediction. Look at how the member is sent (`TransmissionMode.Divine` or `TransmissionMode.Interval`), or, for a controlled member, at prediction and reconciliation on the controlling side.

Read the violation stream first

Most desyncs that are protocol faults raise a violation before you notice anything visually. Register an `IViolationObserver` to hear every type, or subscribe to a per-type event such as `UnauthorizedMemberChangeViolationDetected`. See Violations.

The types worth knowing when chasing desync:

- `UnauthorizedMemberChangeViolation`: a peer changed a member of a `NetworkSystem` it does not control.
- `PredictedMemberChangeViolation`: a controller sent a value for a predicted member, which should only travel as client state.
- `EmptyCollectionDeltaViolation`: a collection change with no operations, which a compliant sender cannot produce. Usually a crafted packet or mismatched builds.
- `InvalidStateAckViolation`: a peer acknowledged a tick above anything sent to it.
- `RetentionExceededViolation`: a peer's acknowledged tick fell beyond the retention window, so it was re-sent the whole world. A single raise is ordinary heavy loss. Chronic raises mean the peer's latency exceeds `SystemManager.StateRetentionMilliseconds`, or it is withholding acknowledgments.

- `RecoveryUnconfirmedViolation`: recoveries were served to a Connection without it confirming one.
- `UnauthorizedSpawnViolation`: a client sent state naming a system id the server does not hold.

A matching raise points straight at the subsystem. If nothing is raised, the fault is in something violations do not police: an interest decision, ordinary loss inside the retention window, interpolation lag, or a prediction mismatch.

Reproduce receive bugs with two processes

Some bugs only appear on a client in a separate process, never on a host's own client. If a bug shows up between two processes but not on a host, test with a separate server and client.

Some faults are timing, not data

Each callback runs on a fixed step of the tick loop and only sees state applied before that step. You are looking at a step-ordering problem if:

- A value read in one callback is stale compared to the same value read in another on the same tick.
- A change does not take until the next tick, or takes twice.
- Symptoms vary with frame rate, or with several fixed updates landing inside one variable update.

Fix these by moving the callback to the right step, not by touching serialization. See The network loop steps.

Narrow it to a reproduction

Cut to one object and one member.

Set the member's `TransmissionMode` to `Interval`. If the bug disappears, it is in `Divine` transmission; if not, `Divine` was never the cause.

Add packet loss (a `NetworkConditions` with a `PacketLossChance`, passed to `TransportManager.SetNetworkConditions`) to see whether the symptom depends on loss.

Only then read a packet, now that you know which member, which ticks, and whether it never arrived or arrived wrong.

Frequently asked questions

Why did an object stop moving instead of disappearing

Interest stopped it rather than unspawning it. A stopped object stays on the receiver but is sent nothing further, so it looks frozen. An unspawned object is removed. See Stopping versus culling.

Why is nothing replicating at all

Check that the Nucleus source generator is added to the project as an analyzer, not as an ordinary assembly reference. Without it your networked types cannot be sent or read. See Wiring the source generator into a .NET project.

Why can I see my object but not drive it

Observing and controlling are separate. Interest decides whether a connection is sent an object's state. Control is assigned by the server. Being an observer never makes a client the controller. See What control means.

Why does it work in host mode and break with two processes

Some bugs only show up on a client in a separate process, never on a host's own client, so test with a second process. See Diagnosing desync.

Why does the host see my object differently than my client does

A host holds every object in the world, because it is the server. To find out what its own client would have received, turn on the host interest report. See The host interest report.

Why is my bandwidth number unchanged after I turned a feature off

The transport's byte and bit counters are running totals since the connection started. Turning a feature off changes how fast they climb, not their value. Sample them per second and compare the rates before and after. See Network statistics.

Why does this type not exist in my build

Pro-only code is left out of a Free build entirely, so a Pro-only type is a compile error in a Free project, not a runtime failure. Either your code uses Pro-only API in a Free build, or your project has the wrong edition's files.

Filing a bug report

Before you file

Reproduce it with the server and client in separate processes. A host runs both in one process, which can hide or create behavior a real split does not show, so a failure that only appears when hosting is usually not the bug it looks like.

Cut the repro down to one networked object, one member on it, and the smallest scene (Unity) or `Main` (plain host) that still fails.

Run it again with logging at `LoggerLevel.Information` and capture the output.

Include in the report

- The edition: Free or Pro.
- The DLLs you deployed, all from the same build. A mixed set, such as an old engine DLL next to a new source generator DLL, fails in ways that look like engine bugs.
- The Unity version, or for a plain host, the target framework you build against (the engine targets `netstandard2.1`).
- Any violations raised, with their context.
- The full log lines, not paraphrased or truncated.
- Counter readings before and after the failure, if it involves a count (packets sent or received, retries, spawned objects).
- The tick or sequence number where it fails, if you can find it.

Diagnostics and Troubleshooting

Pages in this category:

- Violation reference
- Violations
- What the server trusts
- Securing a server
- Logging
- Common errors and what causes them
- Network statistics
- Round-trip time, jitter and packet loss
- Technical limitations
- Diagnosing desync
- Frequently asked questions
- Filing a bug report
- Reading serialization history (and the raw-packet test hook)

Reading serialization history (and the raw-packet test hook)

When a client has the wrong value and the log does not explain why, ask two separate questions: what did the server decide to serialize, and, far more rarely, what actually left the socket. Different tools answer each.

Ask the server what it serialized

`SystemManager` keeps a rolling history of which `NetworkSystem` ids it serialized as changed or spawned on each tick, covering the last `SystemManager.SerializationHistoryTickCount` ticks. Anything older is gone.

- `IsSystemRecentlySerialized(uint systemId)`: whether this system was serialized within the window.
- `GetRecentlySerializedSystemIds(HashSet<uint> collectedSystemIds)`: every system id serialized within the window.
- `TryGetSerializedSystemIds(uint tick, HashSet<uint> collectedSystemIds)`: every system id serialized on one tick. Returns `false` if that tick has left the window.
- `TryGetSerializedSystemIdsAfterTick(uint afterTick, HashSet<uint> collectedSystemIds)`: every system id serialized on any tick after a baseline. Returns `false` if part of that range has left the window, so the results are incomplete rather than empty.

```
HashSet<uint> changedSystemIds = HashSetPool<uint>.Rent();

bool isComplete = coreManager.SystemManager.TryGetSerializedSystemIdsAfterTick(lastKnownGo
odTick, changedSystemIds);

foreach (uint systemId in changedSystemIds)
{
    if (coreManager.SystemManager.TryGetSystemReference(systemId, out NetworkSystem networ
kSystem))
        LoggingService.LogInformation($"Serialized: {networkSystem.AsString()}");
}

HashSetPool<uint>.Return(changedSystemIds);
```

If `isComplete` is `false`, treat the collected ids as a lower bound.

Turn ids into readable names

A bare system or connection id in a log line does not say which object or peer it was, or whether it was valid. Use these in every log line that names one:

- `ConnectionExtensions.AsString(this Connection connection)`: describes a `Connection`, or returns `"IsNull"` if it is null.
- `NetworkSystemExtensions.AsString(this NetworkSystem networkSystem)`: describes a `NetworkSystem`, or returns `"IsNull"` if it is null.

`TryGetSystemReference` turns a bare id back into its `NetworkSystem`.

The sender of a packet is the `Connection` the read path hands you, never an id read out of the payload. If the two disagree, the payload value is the anomaly to investigate.

Send a raw packet to test a transport

`TransportManager.RawPacketReceived` (`RawPacketReceivedHandler(Connection sendingConnection, Reader reader)`) fires only for `PacketType.Raw`, which exists only in a `DEBUG` build. The engine never sends one, so this is not a tap on gameplay traffic. To see what a transport delivers, send a raw packet yourself and subscribe to this event.

```
coreManager.TransportManager.RawPacketReceived += (sendingConnection, reader) =>
{
    // Read whatever fields you wrote into the raw packet, in the same order.
    uint id = reader.ReadUInt32();
    ArraySegment<byte> payload = reader.ReadArraySegmentWithCountAsBytes();
};
```

The `Reader` is reused the moment your handler returns. Copy the values you need inside the callback. Do not keep the `Reader` and read from it later.

Round-trip time, jitter and packet loss

Read the round trip time and jitter

- `Connection.RoundTripTimeMilliseconds` is the link's typical round trip time. A single slow reply does not swing it. It can read slightly higher than the true round trip, never lower.
- `Connection.RoundTripTimeDeviationMilliseconds` is the link's jitter: how much individual round trips vary around that figure.

Both read `UnsetRoundTripTimeMilliseconds` / `UnsetRoundTripTimeDeviationMilliseconds` (`uint.MaxValue`) until the link is measured. Zero is a valid reading on a loopback link, so check `Connection.IsRoundTripTimeUnset` rather than comparing against zero.

Each peer measures its own view of the link. A client holds the figure on its server stand-in `Connection`. A server holds it on each remote client's `Connection`.

Read packet loss

`Connection.PacketLossPercentage` is the share of recent probes that went unanswered, measured the way a ping tool does. It is round-trip loss, not one-way. It describes recent loss, so it falls back to 0% soon after loss stops. It reads 0% on a loopback link.

The server follows what each client reports

`Connection.ReportedRoundTripTimeMilliseconds` and `Connection.ReportedRoundTripTimeDeviationMilliseconds` are the figures a remote client stated for its own link. They are set on a server's remote client `Connection`s and unset on a client. The server measures each client once, when it authenticates, and from then on follows what the client reports.

A client that understates or fakes these figures only affects how well the server serves that one client. A server shows a client's reported packet loss as reported, and nothing the server decides depends on it.

A connection receives no state until its link is measured

`Connection.CanReceiveState` is `!IsRoundTripTimeUnset`. A peer receiving nothing before then is expected. The wait is about one round trip. A host's own loopback link does not wait.

Set how often the link is measured

`TransportManager.RoundTripTimeIntervalMilliseconds` sets how often a client measures its link to the server. It defaults to 1000 (`TransportManager.DefaultRoundTripTimeIntervalMilliseconds`). Zero stops repeated measurement, but the first measurement still happens.

Know when a link is first measured

`TransportManager.RoundTripTimeDiscovered` fires once per `Connection`, when its link is first measured, or immediately for a host's own loopback link. It does not fire for links measured before you subscribed, so also check connections that already exist when you subscribe.

A wrong figure shows up as traffic, not an error

A round trip time or deviation that reads too low shows up as more repair traffic than the link needs. One that reads too high shows up as too little repair traffic and slower recovery from loss.

Logging

Using Unity? See Engine logging in the Unity console.

A plain .NET host logs to the console through a `ConsoleLogger` from the start. A service has nothing reading its console, so replace the logger there.

Replace the logger before creating the CoreManager

Implement `ILogger` (see below).

Pass it to `LoggingService.UseLogger` before new `CoreManager()`. The constructor already logs, so lines raised before the swap go to the old logger.

```
LoggingService.UseLogger(new MyServiceLogger());
CoreManager coreManager = new();
```

`UseLogger` sets `LoggingService.Logger` and re-reads the logging level. Call it again to apply a level change in your own logger. You can set the logger and CodeBoost's application state in either order.

`LoggingService.LoggerSet` fires with the new `ILogger` whenever the logger changes:

```
LoggingService.LoggerSet += logger => Console.WriteLine($"Logger swapped to {logger.GetTyp
e().Name}");
```

Implement ILogger

```
public interface ILogger
{
    LoggerSetting GetLoggerSetting();
    bool DisableUnconditionalDevelopmentStacktrace();
    void LogInformation(string message);
    void LogWarning(string message);
    void LogError(string message);
}
```

- `LogInformation`, `LogWarning` and `LogError` receive the formatted message. Route each to your host's logging framework.
- `GetLoggerSetting` returns the `LoggerLevel` your logger runs at in each environment.
- `DisableUnconditionalDevelopmentStacktrace` returns `true` when your logger already captures its own stack traces, so the engine does not attach one in development environments.

Set the level for each environment

`LoggerLevel` is `Error`, `Warning`, `Information` (least to most verbose) or `Disabled`, which turns logging off. `LoggerSetting` holds one level per environment:

Field	Applies to
`Editor`	Running inside the editor
`DevelopmentBuilds`	Development builds outside the editor
`ReleaseBuilds`	Release builds

`ConsoleLogger` runs `Information` in the editor and development builds, and `Error` in release. In a plain .NET process, CodeBoost's default application state (`IdeApplicationState`) always reports the editor, in

Debug and Release builds alike. A .NET host therefore logs at `Information` by default, even in a Release build. To log less, register a logger whose `GetLoggerSetting` returns a lower `Editor` level.

Check the level before formatting a message, so the string is never built when the level is off:

```
if (LoggingService.IsInformationEnabled)
    LoggingService.LogInformation($"Tick {tick}: {expensiveDescription()}");
```

`LoggingService` also has `IsWarningEnabled` and `IsErrorEnabled`.

Choosing the level at runtime

When the level is decided at runtime, such as a warning the first time and an error once it keeps happening, pass it to `Logger.Log` and check it with `Logger.IsEnabled`:

```
LoggerLevel loggerLevel = desyncCount > 10 ? LoggerLevel.Error : LoggerLevel.Warning;

if (Logger.IsEnabled(loggerLevel))
    Logger.Log(typeof(MatchService), loggerLevel, $"Player {playerName} desynced {desyncCo
unt} times.");
```

`IsEnabled` is always false for `LoggerLevel.Disabled`, and `Log` with `Disabled` writes nothing, so `Disabled` also means "decided not to log".

When the decision and the write happen in different places, carry the line as a `LogEntry`, a level and a message together. `LogEntry.Information`, `LogEntry.Warning` and `LogEntry.Error` build one. The default `LogEntry` is unset (`IsSet` is false) and logs nothing. Violations use this so a handler can change or clear a line.

```
LogEntry logEntry = isLobbyFull ? LogEntry.Warning("Lobby is full; the join was refused.")
 : default;

// Later, wherever the line is written. An unset entry writes nothing.
Logger<MatchService>.Log(logEntry);
```

A `LogEntry` built from a constant string allocates nothing, which matters on a path that runs per packet.

Network statistics

Using Unity? See Reading network statistics in Unity.

`TransportManager` keeps four read-only `long` counters. The engine never resets them.

Counter	What it counts
`TotalBytesSent`	Packet bytes sent. Every redundant resend counts again.
`TotalBytesReceived`	Packet bytes received. Every redundant arrival counts again.
`TotalStatePacketPayloadBitsSent`	State payload bits sent, excluding headers and padding. Each serialized state payload counts once, however many peers it reached.
`TotalStatePacketPayloadBitsReceived`	State payload bits received, excluding headers and padding. Every redundant arrival counts again.

```
long bytesSent = coreManager.TransportManager.TotalBytesSent;
long bytesReceived = coreManager.TransportManager.TotalBytesReceived;
long stateBitsSent = coreManager.TransportManager.TotalStatePacketPayloadBitsSent;
long stateBitsReceived = coreManager.TransportManager.TotalStatePacketPayloadBitsReceived;
```

The two kinds of counter answer different questions

The byte counters answer "what did the link carry", including headers and every redundant copy. The state bit counters answer "what did replication cost". Sending one state payload to many peers, or resending over an unreliable link, moves the byte counters more than the bit counters. That gap is expected.

Sample twice to read a rate

The counters are totals for the manager's lifetime. Sample, wait, sample again and subtract:

```
long before = coreManager.TransportManager.TotalBytesSent;
// ... wait one second ...
long after = coreManager.TransportManager.TotalBytesSent;
long bytesPerSecond = after - before;
```

Divide a state bit difference by eight to get bytes.

No counter attributes cost to one object

The counters cover the whole manager. To measure one feature, read the counters with it on, then off, and compare. A test can assert on `TotalStatePacketPayloadBitsSent` and `TotalStatePacketPayloadBitsReceived` to catch a bandwidth regression.

Securing a server

> **Using Unity?** See Securing a server in Unity

This guide covers what the server already rejects and each setting and callback you can use to tighten it. What the server trusts explains the trust model, and Violations explains how violations are decided and reported.

Only a check on the server counts

Every check in this guide runs on the server. A client can run the same code to predict an answer or grey out a button, but a modified client can skip any check it holds.

The server already rejects a lot on its own

The server refuses all of the following without any code from you. The offending data is thrown away and never applied, whatever you later decide about the client.

- **A client must observe a system to touch it.** A write or remote call aimed at a system the client does not observe is refused, so a client cannot reach an object by guessing its id. See What the server trusts.
- **Only the controller may set a system's members unless you widen it.** A member value from a client that does not control the system is dropped and raises `UnauthorizedMemberChangeViolation`.
- **Only permitted clients may send a remote call.** A call from a client the system's `RpcSendAccess` does not admit is dropped and raises `RpcSendPermissionViolation`.
- **A client cannot pose as someone else.** The sender a handler is given is the connection the transport resolved for the packet, never an id the client wrote into it.
- **Data the server cannot read gets the client kicked in a release build.** The data is discarded and `MalformedDataViolation` is raised, which defaults to `Kick` in a release build. A debug build ignores it and logs the exception instead, so a serializer you are still writing does not keep kicking you.
- **An unauthenticated client may only send messages that opt out of authentication.** A message type requires authentication unless it sets `IsAuthenticationRequired` to `false`. A client that sends any other message before it is approved is kicked.
- **A client that never finishes authenticating is dropped.** `ServerManager.AuthenticationTimeoutSeconds` (30 by default) disconnects a client that stays unapproved that long.
- **Answers the server never asked for get the client kicked.** A client that sends a scene or bundle request, or reports a scene or bundle load the server never requested, raises one of the four protocol-direction violations, which default to `Kick`. See Violation reference.
- **A client cannot stall a scene load forever.** `SceneManager.LoadRequestTimeoutSeconds` (180 by default) gives up on a load a client never answers.
- **Remote calls are capped per drain.** `RpcManager.MaximumInboundRpcsPerConnectionPerDrain` (64 by default) drops calls past the cap and raises `RpcFloodViolation`.
- **Requests to repair lost state are capped.** A client may ask for at most four repairs per tick. The excess is refused with `StateLossReportFloodViolation`. A report sent to a peer that serves it no state is discarded with `UnexpectedStateLossReportViolation`.
- **Repair traffic is budgeted.** With `SystemManager.RecoveryBudgetEnabled` on (the default), a

client that keeps losing packets, or claims to, is not repaired past its budget. `RecoveryBudgetExhaustedViolation` tells you when a client keeps hitting it.

- **A lag-compensated shot cannot ask for an arbitrarily old world.** In Pro, a shooter's claimed view is clamped to `LagCompensationManager.MaximumRewindMilliseconds` (500 by default) and raises `LagCompensationViewClampedViolation`. See The rewind window and the view clamp.
- **Predicted spawns are refused until you allow them.** In Pro, no client can predict a spawn or despawn until the server registers a validator, and the rate is bounded by `SystemManager.PredictedSystemsPerSecond` (100 by default). See Predicted spawn and despawn requests.

The server does not encrypt traffic. For that, use the Pro packet transform layer. See Packet transforms.

Most violations are silent until you act on them

Most of the faults above neither kick nor log, because an honest client on a bad connection can raise many of them.

Default	Violations
`Kick`	`MalformedDataViolation` (release builds), `ClientLimitViolation`, the four scene and bundle protocol-direction violations, and in Pro `UnsolicitedLeaseAcknowledgementViolation`
`Ignore`	Everything else, including `UnauthorizedMemberChangeViolation`, `RpcSendPermissionViolation`, `RpcFloodViolation` and `StateLossReportFloodViolation`

Deciding which ignored violations deserve a kick in your game is the main security work. See Decide what each violation costs the client.

Limit how much each client can send

A flooding client costs the server CPU and memory even when every packet is rejected. Use the transport's limits first and the engine's limits as a fallback.

Configure limits on the transport first

Transport limits drop traffic before the engine reads it, and also protect the transport's own receive buffers.

The Synapse transport's limits are on `Synapse.ClientLimits`, a `SynapseClientLimits`. **They are enabled by default.** They apply only on a server. Set them before the server starts.

Field	Default	What it guards against
`Enabled`	`true`	Turns every limit below on or off. The socket's other checks, such as refusing a datagram wider than the transmission unit, stay on either way.

`MaximumPacketsPerSecond`	`500`	A client sending too many datagrams. The excess is dropped rather than kicked, because a datagram's source address can be forged.
`MaximumBytesPerSecond`	2 MiB	A client sending too many bytes. Enforced like the packet rate, over the same one-second window.
`MaximumOutOfOrderReliablePackets`	`64`	A client pinning server memory by never filling a gap in its reliable stream. One past the limit kicks and blacklists the client.
`MaximumReassembledPacketSize`	`0` (no limit)	A client declaring a huge split packet. It is checked on the first piece, and a client over it is kicked and blacklisted.
`MaximumSegments`	`0` (the protocol's limit of 255)	A packet split into too many pieces. A packet over it is dropped without a violation. It also caps what the server sends.
`MaximumConcurrentAssembliesPerConnection`	`16`	A client opening many split packets at once. One more evicts the oldest unfinished unreliable packet, or is dropped.

Set a single limit to `SynapseClientLimits.UnsetLimit` (`0`) to turn it off.

If an honest client loses input under load, for example because it controls many objects or the server runs a high tick rate, raise `MaximumPacketsPerSecond` first. Dropped datagrams show up as lost client state, not as a kick.

To see what the socket refused, subscribe to `Synapse.ViolationObserved`. It reports rate limits, oversized packets, security kicks and timeouts. A handler must not block or disconnect anyone. See Finding out why a connection dropped.

Use the engine limits as a fallback

Use `ServerManager.ClientLimits` for a transport without limits of its own, or as a second line behind one. The engine checks them on every packet a client sends, whatever the transport. They cannot protect the transport itself, but they stop every engine-side read and dispatch of a packet past a limit.

`ServerManager.ClientLimits` is a `ClientLimits` struct. **It is disabled by default.**

Field	Default	What it guards against
`Enabled`	`false`	Turns the engine limits on. While it is off, nothing is counted.

`MaximumPacketBytes`	`0` (no limit)	Oversized packets, including each piece of a large packet the engine splits. Set it at least one transmission unit above the largest packet an honest client sends, since the last piece of a split packet is usually shorter.
`MaximumPacketsPerSecond`	`500`	Too many packets in one second.
`MaximumBytesPerSecond`	2 MiB	Too many bytes in one second.

A packet past a limit is discarded unread and raises `ClientLimitViolation`, which **kicks by default**. A rate limit raises it once per one-second window and discards the rest of the client's packets in that window. A host's own client is exempt.

Copy the struct out, change it and assign it back. A change takes effect straight away.

```
ClientLimits clientLimits = coreManager.ServerManager.ClientLimits;
clientLimits.Enabled = true;
clientLimits.MaximumPacketBytes = 64 * 1024;
coreManager.ServerManager.ClientLimits = clientLimits;
```

Cap the number of connections

A server accepts any number of clients until you call `SetMaximumConnections(uint)` on the transport. For Synapse it sets `ServerConfiguration.MaximumConnections`, where `0` (the default) means unlimited. A client beyond the cap is refused during its handshake.

Authenticate every client before it can do anything

Until a client is approved it observes nothing and may only send messages that opt out of authentication. The shipped `AddressClientAuthenticator` approves everyone at once and identifies them by address, which suits development, not a public server.

- Implement `IClientAuthenticator`.
- Set it with `ServerManager.SetAuthenticator` before the server starts listening.
- Call `ApproveClient` or `DenyClient` once you have a verdict. Use `DenyClient(connection)` without a notice for a failed login, since an explanation can help an attacker.
- Keep `AuthenticationTimeoutSeconds` above zero so a client that never answers is dropped.

Keep the message types that skip authentication few and small. Anyone who can open a connection can send a message type with `IsAuthenticationRequired` set to `false`. Usually that is only the login message. Treat its contents as hostile, and remember that a token sent over an unencrypted transport can be read and replayed.

The server raises `ClientAuthenticated` and `ClientAuthenticationDenied` for logging or counting. See Authenticating clients.

Lock down who may set members and send calls

Every `NetworkSystem` has two access values. Both default to the safest setting.

Leave write access on the controller unless you need more

`MemberWriteAccess.Controller` is the default: only the server and the controlling client may set a

system's members. `MemberWriteAccess.AnyClient` (Pro) lets any observer set them. Only the server can change it, through `NetworkSystem.TrySetWriteAccess`. A Free build has no write access setting, so only the controller can write. See Letting clients set members.

Open `AnyClient` only where every observer genuinely needs to write, and give each such system a validator.

Leave send access on the controller unless you need more

`RpcSendAccess.Controller` is the default: only the server and the controlling client may send a system a remote call. `RpcSendAccess.AnyClient` lets any observer send one. Only the server can change it, through `NetworkSystem.TrySetRpcAccess`. See Who may send a call, and how many.

Open send access only for calls any nearby player should make, such as using a door or a shop, and treat every argument as hostile.

Validate every value a client is allowed to write

Access decides whether a client may write. To check that the value makes sense, implement `IMemberWriteValidator<T0>` on the `NetworkComponent` that owns the member:

```
public partial class InventoryComponent : NetworkComponent, IMemberWriteValidator<uint>
{
    private const uint SlotCount = 24;

    public readonly NetworkMember<uint> SelectedSlot = new();

    public MemberWriteAction ValidateMemberWrite(NetworkMember<uint> networkMember, uint p
roposedValue, Connection writingClientConnection) =>
        proposedValue < SlotCount ? MemberWriteAction.Accept : MemberWriteAction.Decline;
}
```

- The validator runs on the server only, before the value lands, so `networkMember.Value` is still the current value.
- `Decline` keeps the old value, and other observers never see the rejected one.
- A validator that throws declines the write.
- A declined write is not a violation. If a value can only come from a modified client, call `writingClientConnection.Kick()` from the validator.

Validate client state too. The per-tick input a controlling client sends, in members declared with `MemberRole.Predict`, goes through the same validator. Clamp move directions, check that a jump is allowed, and decline anything a real controller could not produce. See Judging and arbitrating client writes and Client state components.

Judge remote calls on the server

Check the sender and the arguments in the handler

A server handler receives an `RpcContext`. `RpcContext.SenderConnection` is the client that actually sent the call, so it is safe for permission checks and cooldowns. Check every argument against what the server knows: whether the sender is close enough to the door, owns the item, or has the money.

Refuse to relay a call that should not reach other clients

Every server handler returns an `RpcRelayAction`. Return `RpcRelayAction.Cancel` to keep a call on the server instead of passing it to the system's other observers. Any single `Cancel` wins, and a handler that throws counts as `Cancel`. Use this to moderate chat, emotes or anything else clients send each other. See

Judging and refusing a relayed call.

Tune the call cap to your game

`RpcManager.MaximumInboundRpcsPerConnectionPerDrain` (64 by default) bounds how many calls one client may deliver in a single drain. Lower it if your clients never send many calls at once. A public server should not set it to `UnsetMaximumInboundRpcsPerConnectionPerDrain` (`0`), which removes the cap.

Decide what each violation costs the client

Register deciding handlers in your server code, since a kick is only enforced on a peer that has started a server.

Kick for faults an honest client cannot produce

A handler returns `ViolationAction.Kick` or `ViolationAction.Ignore`, and can set `context.Log` to log a line of its own:

```
coreManager.ViolationManager.RegisterViolationHandler<RpcFloodViolation>((ref ViolationCon
text<RpcFloodViolation> context) =>
{
    context.Log = LogEntry.Warning($"Sent {context.Violation.ReceivedRpcCount} calls in on
e drain.");
    return ViolationAction.Kick;
});
```

There is one handler per type, and registering another replaces it. A throwing handler falls back to the default action.

Count before you kick for faults an honest client can reach

An honest client can raise `UnauthorizedMemberChangeViolation`, `RpcSendPermissionViolation`, `RetentionExceededViolation`, `StateLossReportFloodViolation` and several others once, through a race or a bad connection. Count raises per connection and kick past a threshold you choose:

```
private readonly Dictionary<uint, int> _permissionFaultCounts = new();

private ViolationAction OnRpcSendPermission(ref ViolationContext<RpcSendPermissionViolatio
n> context)
{
    uint connectionId = context.Connection.Id;
    _permissionFaultCounts.TryGetValue(connectionId, out int count);
    _permissionFaultCounts[connectionId] = ++count;

    return count > 20 ? ViolationAction.Kick : ViolationAction.Ignore;
}
```

Clear a connection's count when it disconnects, since connection ids are reused. The Violation reference lists what to check for each type before you assume an attack.

Watch every violation for auditing

To log every violation without changing any outcome, register an `IViolationObserver` with `ViolationManager.RegisterViolationObserver`. To watch a single type, subscribe to its event, such as `ViolationManager.ClientLimitViolationDetected`. Feed these into your own logs or anti-cheat reporting to spot patterns no single violation reveals. See Watching without deciding.

Kick cleanly when you remove a client

`ServerManager.KickClient` and the `Connection.Kick()` shorthand mark a client for removal. Under the default `KickOptions` the server stops sending to the client and stops believing anything it sends, and the link closes at the end of the frame. Pass a notice message to tell the client why; it is sent before the link closes. `ServerManager.KickTimeoutMilliseconds` (500 by default) closes the link outright if the client never acknowledges. See Kicking and disconnecting a client.

Technical limitations

These behaviors are not supported

Do not design a game that depends on either of these.

- **Send rate does not change with distance.** No shipped interest type slows an object's sends because it is far from a peer. Per-member send interval and transmission mode exist, but neither is driven by distance.
- **A replay does not roll back the whole physics world.** A reconcile's replay rewinds only the bodies a validated behaviour predicts and holds every other body still. See Predicting a body with a validated behaviour.

These features are planned but not built yet

- **Network profiler window:** an editor window attributing traffic to messages, objects and members. The counters it would read already exist.

Lag compensation (server-side rewind, lag-compensated queries and hitbox history) is built, as a Pro feature. See How lag compensation works.

Violation reference

A violation is raised through `ViolationManager` when a peer sends something the protocol does not permit. The offending operation is always rejected. The `ViolationAction` (`Ignore` or `Kick`) only decides what happens to the `Connection` that sent it. To change an action, log a violation or watch for one, see Violations.

There are twenty-two violation types. Three exist only in Pro.

Peer-behavior violations are ignored by default

These come from state, recovery, spawning and remote calls. All default to `Ignore`.

Type	Payload	Check first
`UnauthorizedMemberChangeViolation`	`SystemId`	Whether the sender controlled that `NetworkSystem` at the time. A control handoff racing a member change raises this too.
`PredictedMemberChangeViolation`	`SystemId`	Whether the member is really predicted. This usually means the member's transmission setup changed without the controller rebuilding.
`EmptyCollectionDeltaViolation`	`SystemId`, `MemberId`	Whether both builds share the same serializers. An honest sender cannot produce a collection change with no operations, so suspect a version mismatch before the sender's intent.
`InvalidStateAckViolation`	`AckedTick`	Whether the two peers' tick counters diverged (a restart, a save and load, a manual tick reset) before assuming a forged acknowledgment.
`RetentionExceededViolation`	`AckedTick`	The peer's latency against `SystemManager.StateRetentionMilliseconds`. A single raise is ordinary heavy loss.

`RecoveryBudgetExhaustedViolation`	`ConsecutiveSeconds`, `SecondsInLastMinute`	Raised at most once a second per Connection, for a second in which the peer was refused a repair because its `SystemManager.RecoveryBudgetFullWorlds` budget was spent. A short run is a burst of loss. A long `ConsecutiveSeconds` or a high `SecondsInLastMinute` is a peer on constant heavy loss, or one reporting losses it never had.
`StateLossReportFloodViolation`	`AllowedLossRepairCount`	Whether the peer just recovered from a stall, the only time an honest peer asks for several repairs at once. Repairs past the allowance are refused either way.
`UnexpectedStateLossReportViolation`	`Tick`	Whether a handover just changed which peer serves the sender its state. A report in flight across that change raises this honestly. The report is discarded and a warning is logged.
`RecoveryUnconfirmedViolation`	`UnconfirmedServeCount`	Whether the peer's connection is losing the recovery itself before assuming it is stalling on purpose.
`UnauthorizedSpawnViolation`	`SystemId`, `SystemBundleId`, `SystemLocalId`	A client re-sending a system the server just despawned does not raise this, so a raise means the id was never one the server despawned.
`RpcSendPermissionViolation`	`SystemId`	Whether permission was revoked while calls were in flight. An honest client raises this on that race alone.
`RpcFloodViolation`	`ReceivedRpcCount`, `AllowedRpcCount`	`RpcManager.MaximumInboundRpcsPerConne` against what the client honestly sends in one drain.

Unreadable data kicks the client in a release build

`MalformedDataViolation` fires on the server when reading data a client sent throws. The data is not applied.

- In a release build it defaults to `Kick` and logs `Sent data the server could not read.` as a warning.
- In a debug build (`#if DEBUG`) it is raised with `Ignore` and no log line, and the exception is logged instead.

- It is never raised on a client, for data with no known sender, for a server Connection, or for a host's own client. There the exception is logged and nobody is kicked.

Type	Payload	Check first
`MalformedDataViolation`	`Kind` (`State`, `FullBody`, `Recovery`, `ClientState`, `Message`, `Rpc`), `SystemId` (zero when not for a system)	Whether both builds share the same serializers and member layout for that `Kind`. A version mismatch looks exactly like a forged packet. Run a debug build to see the exception.

Crossing a client limit kicks the client

`ClientLimitViolation` fires on the server only, while `ServerManager.ClientLimits` has `Enabled` set, when a client crosses one of those limits. The packet is discarded unread. A size limit raises it once per refused packet. A rate limit raises it once per one-second window and discards the rest of that client's packets in the window. A host's own client is exempt. It defaults to `Kick` and logs a warning.

Type	Payload	Check first
`ClientLimitViolation`	`Kind` (`PacketBytes`, `SplitPacketBytes`, `PacketsPerSecond`, `BytesPerSecond`), `MeasuredValue`, `MaximumValue`, `Channel`	`MeasuredValue` against `MaximumValue` and what the game honestly sends at its busiest. A `PacketBytes` or `SplitPacketBytes` limit too close to the largest honest packet refuses honest split packets, so set `MaximumPacketBytes` at least one transmission unit above it.

Protocol-direction violations

These fire when a client sends a bundle or scene request, or reports a load or release the server never asked for. An unmodified client cannot produce any of the four, so they default to `Kick`.

Type	Payload	Check first
`UnexpectedBundleRequestViolation`	`BundleId`	Custom code calling the bundle-request path on the wrong side.
`UnsolicitedBundleReportViolation`	`BundleId`, `IsLoadReported`	Whether the server requested that bundle load, or recorded the client holding it before an unload report. A mismatched build id looks identical to a forged report.
`UnexpectedSceneRequestViolation`	`SceneHandle`, `IsLoadRequested`	Custom code calling the scene-request path on the wrong side.

`UnsolicitedSceneReportViolation`	`SceneHandle`, `IsLoadReported`		Whether the server requested that load or release. A handle never requested and never held, including the unset handle, lands here too.

Three violations exist only in Pro

These types, and their events on `ViolationManager`, do not exist in a Free build.

Type	Payload	Default	Check first
`PacketTransformRejectedViolation`	`ChannelId`, `PayloadByteCount`	`ViolationManager.DefaultAction` (`Ignore`), and always logs a warning	Whether the peer runs the same `IPacketTransform` configuration. Stray packets from anything else that reaches the port raise this too, so it is not evidence of an attack.
`UnauthorizedDespawnViolation`	`SystemId`	`Ignore`	Whether the client and server builds agree on that prefab's removal policy. Version skew raises this honestly.
`UnsolicitedLeaseAcknowledgementViolation`		`Kick`	Whether the client build matches the server's. An honest client acknowledges only the predicted spawn top-ups the server sent it.

A clamped lag compensation view is ignored by default

`LagCompensationViewClampedViolation` exists in every edition, but only lag compensation (Pro) raises it. A lag-compensated query raises it, at most once per query, when the shooter's view is older than the rewind window or newer than anything the server has recorded. The view is clamped into range and the shot is still judged. See The rewind window and the view clamp.

Type	Payload	Default	Check first

`LagCompensationViewClamped`	`RejectedTick`, `ClampedTick`	`Ignore`	The shooter's latency against `LagCompensationManager.Maximum` A single raise is a link that spiked. A steady stream is a very slow link, or a client asking to be judged against the oldest world it can get.

Judge a raise before you kick for it

An honest peer can raise the `Ignore` types through a race (a control handoff, a permission revocation, a recovery re-serve) or ordinary loss (`RetentionExceededViolation`, `RecoveryUnconfirmedViolation`, `RecoveryBudgetExhaustedViolation`, `StateLossReportFloodViolation`, `PacketTransformRejectedViolation`). Do not kick for a single raise of those.

An unmodified client never raises the four protocol-direction violations. If one fires, first check your own code for a request or report sent from the wrong side, then suspect a crafted client.

Violations

> **Using Unity?** See Handling violations in Unity

A violation is a protocol fault from a peer, such as setting members of a system it does not control, sending a malformed delta, or acknowledging a tick it was never sent. The offending operation is always rejected. `ViolationManager` only decides what happens to the `Connection` that sent it. The Violation reference lists every type and its default.

Most violations are silent by default

Most types are raised with `Ignore` and log nothing. A few, such as the scene-protocol faults, kick the offender. To see violations, register an `IViolationObserver`. To kick misbehaving clients, or to feed violations into your own audit system, register with `ViolationManager`.

Every violation passes through the same four stages

- **The deciding handler** for that type, if one is registered, settles the `ViolationAction`.
- **Global observers** (`IViolationObserver`) are told the settled action.
- **Per-type subscribers** for that type are told the settled action.
- **Enforcement** applies the action against the offending `Connection`.

Only the deciding handler can change the outcome. Observers and subscribers receive the context by read-only reference after the action is settled.

If the deciding handler throws, the violation falls back to its default action and log line. If an observer or subscriber throws, the rest still run and enforcement still happens. Each exception is logged as an error, so a client is still kicked when your code throws.

Decide the action with a handler

Register one deciding handler per type with `ViolationManager.RegisterViolationHandler<T0>(ViolationHandler<T0> violationHandler)`. Registering a second handler for the same type replaces the first. `TryUnregisterViolationHandler<T0>()` removes it.

```
coreManager.ViolationManager.RegisterViolationHandler<UnauthorizedMemberChangeViolation>((
ref ViolationContext<UnauthorizedMemberChangeViolation> context) =>
{
    context.Log = LogEntry.Error("Wrote state it does not control.");
    return ViolationAction.Kick;
});

public delegate ViolationAction ViolationHandler<T0>(ref ViolationContext<T0> violationCon
text)
    where T0 : struct, IViolation;
```

The handler receives a `ViolationContext<T0>`:

- `Connection`: the offending Connection, read-only.
- `Violation`: the typed violation payload, read-only.
- `Action`: seeded with the default for this raise. Return it unchanged to accept the default.
- `Log`: seeded by the raise site, if it set one. Leave it, replace it with your own `LogEntry`, or clear it with

`context.Log` = `default`.

Each type has its own default action

`ViolationAction` has two values:

- `Ignore`: no consequence. The Connection stays.
- `Kick`: disconnects the Connection.

`ViolationManager.DefaultAction` is `Ignore`. Each type is raised with its own default, which applies when no handler is registered. A registered handler can return either action.

Logging is set separately from the action

A violation raised with `ViolationManager.DefaultAction` logs a warning, "The offending operation was rejected." A violation raised with its own default logs only if the raise site or the handler set `context.Log`. A set `Log` is written as `Violation [<TypeName>] from Connection [<connection>]: <message>` at the level the `LogEntry` carries.

Watching without deciding

To audit every violation type without deciding any of them, implement `IViolationObserver` and register it with `ViolationManager.RegisterViolationObserver(IViolationObserver violationObserver)`. One registration hears every type. Remove it with `TryUnregisterViolationObserver(IViolationObserver violationObserver)`.

```
public sealed class ViolationAuditSink : IViolationObserver
{
    public void OnViolation<T0>(in ViolationContext<T0> violationContext) where T0 : struc
t, IViolation
    {
        // Log, count or alert. Cannot change violationContext.Action.
    }
}
```

To watch one type only, use `RegisterViolationDetectedHandler<T0>(ViolationDetectedHandler<T0> violationDetectedHandler)` or the matching per-type event, such as `UnauthorizedMemberChangeViolationDetected`. Both make the same subscription, and it costs nothing for types you did not subscribe to.

Only a peer running a server enforces a kick

`Kick` is enforced only on a peer that has started a server. A client that settles a violation on `Kick` against its own server Connection only logs it. On a server, the kick goes through `ServerManager.KickClient`, and the link closes at the end of the frame.

What the server trusts

Observation gates everything

Every member write and every remote call is first checked for whether the sending connection observes the system. A connection that does not observe it is rejected, whatever `MemberWriteAccess` or `RpcSendAccess` says, so a client cannot reach a system by guessing its id.

Next, the system's controller may always write to it or call it. Any other observer is admitted only when the access in force is `AnyClient`.

Access values are hints, not grants

The server re-checks every incoming write and call against its own copy of the access. What a client is told about access only saves an honest client from sending something the server would refuse. Forging or ignoring it gains nothing.

The sender is the connection, not the payload

An id a client writes into a payload is only a claim. The sender a handler is given is the `Connection` the transport resolved for that packet, such as `RpcContext.SenderConnection` for a call. A call the server relays to other observers still names its original sender, and a call arriving from a client is always attributed to that client, so a client cannot pose as another client.

Connections are not players

`Connection.RemoteAddress` is observed by the server, so it can be part of an identity check, but several clients behind one NAT share an address.

`Connection.Id` is assigned per session and reused once a connection is gone. To recognize a player across reconnects, key on `Connection.Identity`, which your authenticator sets on approval. Matching a returning `Identity` to what it owned before is up to your game.

Arbitration is not a security boundary

In Pro, `MemberWriteAccess.AnyClient` lets every observer set a system's members, and arbitration picks which of several simultaneous writes wins. The pick is arbitrary but stable, and a client that keeps reconnecting to change its connection id can farm a favorable result. Arbitration does not decide whether a write should have been allowed. Where winning matters, put your own arbiter in game code on top of the access check.

A Free build has no `MemberWriteAccess`. Only the controller may set a system's members, so there is nothing to arbitrate.

Nucleus does not encrypt traffic

To encrypt, compress, obfuscate or integrity-check packets, use the Pro packet transform, `TransportManager.PacketTransform`. See Packet transforms.

A shooter's view is bounded, not trusted

Lag compensation (Pro) judges a shot against the moment the client says its screen was showing. A view older than `MaximumRewindMilliseconds`, or newer than anything the server has recorded, is clamped into range, still judged, and raises `LagCompensationViewClampedViolation` (ignored by default). Within the window a client can name any moment, so the window is the limit on how far a shooter is favored.

The ray is the client's claim too. Clip it at walls on the server against the present-time scene. The queries test it only against hitboxes from the server's own history. See The rewind window and the view clamp.

Enforcement only happens on the server

Every check above runs on the server's copy of the system. A client can run the same check for prediction or UI, but a verdict reached on a client, such as a kick or a write it declines to send, enforces nothing.

Core API

Documentation for driving Nucleus directly from C#, with no game engine involved. The same subjects from the Unity integration live under Unity.

- Core, Managers and the Tick Loop
- Systems, Objects and Spawning
- Replicated State and Change Callbacks
- Control, Client State and Prediction
- Messaging and Remote Calls
- Interest Management and Scaling
- Networked Scenes, Content Bundles and Persistence
- Physics Synchronization
- Lag Compensation
- Transports, Connectivity and Hosting
- Serialization, Codegen and the Wire
- Diagnostics and Troubleshooting

What area of interest is

Using Unity? See What area of interest is and Unity Interest Manager.

Without any condition, every system replicates to every client

With no interest condition registered, Nucleus replicates every started `NetworkSystem` to every authenticated client, every tick. Interest conditions only restrict that default; nothing they return can give a connection more than it would get with no conditions at all. With no condition registered anywhere, interest evaluation costs almost nothing.

Conditions only restrict

An `IInterestCondition` returns an opinion for each effect it declared: `None` (no opinion), `Unmet` (veto the effect for this pair) or `Met` (no restriction). Only `Unmet` restricts. Conditions can be registered globally on the `InterestManager` or on a single system, and they stack: the most restrictive verdict per effect wins. See Writing an interest condition and Registering interest conditions.

Interest resolves one `(NetworkSystem, Connection)` pair at a time, and a changed verdict takes effect on the tick it is decided, so a system just culled for a connection does not serialize to it that tick. A condition must be a pure read: it must not start or stop systems, change observers, or block.

There are two effects

`InterestEffect` has two flags:

- **Spawn**: whether the system exists for the connection at all. While unmet, the system does not spawn there, and one already spawned is despawned.
- **Stop**: whether the system is still replicating to the connection. While unmet, the system stops for that connection but its object is kept, so it can stay visible without costing replication.

See Stopping versus culling.

Conditions are evaluated on a staggered cadence

`InterestManager.EvaluationCadenceTicks` sets how many ticks pass between evaluations of any one system's conditions. The default is 5, and 1 evaluates every system every tick. The work is spread across those ticks rather than done all at once.

Two cases skip the cadence and are evaluated on the tick they happen: a connection that just authenticated is evaluated against every started system, and a system that just started is evaluated against every authenticated connection.

Evaluation cost grows with systems times connections. See What scales and what does not.

A new connection waits until its link is measured

A connection is not registered as an observer, and so not evaluated, until its link is measured (`Connection.CanReceiveState`). That is typically one round trip after it authenticates, and immediate for a host's own client.

Distance interest conditions

Using Unity? See Limit what a client receives by distance.

Distance conditions are part of Nucleus Pro. A Free build has the interest system but not distance conditions.

Subclass DistanceInterestConditionBase

`DistanceInterestConditionBase` culls or stops a `NetworkSystem` by its distance from a connection's controlled objects. It derives from `InterestPositionConditionBase`, whose `TryGetNearestControlledDistance` finds the nearest controlled object. It declares `Capabilities => InterestEffect.Spawn | InterestEffect.Stop`, since both come from one measurement.

To write your own, subclass it and implement `TryReadSourcePosition`, returning the system's world-space position. Caching, finding the connection's controlled objects and picking the nearest are handled for you.

```
protected abstract bool TryReadSourcePosition(NetworkSystem networkSystem, out Vector3 pos
ition);
```

A ladder is built from a spawn cutoff and a stop cutoff

```
public DistanceInterestLadder(float stopDistance, float spawnDistance)
```

- `stopDistance`: past this, the system stops for the connection but keeps its object.
- `spawnDistance`: past this, the system is despawned.

`Resolve(float distance)` checks despawn before stop, so a distance past both cutoffs despawns.

`DistanceInterestLadder.UnsetDistance` is `0f`. A cutoff left at `0` is disabled, not "zero range": it never restricts anything.

Distance is measured from the connection's controlled objects

Distance is measured from each of the connection's controlled objects to the system being evaluated, and the nearest wins. A player driving both a character and a vehicle stays covered near either.

`InterestManager.MaximumControlledInterestObjects` caps how many controlled systems are measured. It defaults to `DefaultMaximumControlledInterestObjects` (`1`). Set it to `InterestManager.UnlimitedControlledInterestObjects` (`0`) to measure every controlled object.

```
public uint MaximumControlledInterestObjects = DefaultMaximumControlledInterestObjects;
```

A cap below the controlled count measures arbitrary objects, not the nearest. The cap counts `NetworkSystems`, not objects, so a controlled prefab made of two systems takes two slots. Set the cap to the total number of systems a player controls, or to `UnlimitedControlledInterestObjects`.

Unmeasurable pairs restrict nothing

When a distance cannot be measured, because the connection controls nothing yet or `TryReadSourcePosition` returns `false`, the condition restricts nothing. Return `false` for an object that has been destroyed or has not finished spawning. Culling on an unknown distance would empty the world for a player whose character has not finished spawning.

The host interest report

Using Unity? See Making a host see what its players see.

The report tells a host what its own client half would have received

A host holds every object in the world, so it cannot tell what its own client half would have been sent. Set `InterestManager.HostInterestEnabled` to resolve the host's own client against the registered conditions, as for a remote client.

The answer is only a report. Nothing is culled, no observer is dropped and no despawn is written for the host. Objects keep existing, replicating and being controllable as before.

It is off by default because it costs one condition pass per evaluated pair for the host connection. Turn it on in a host world that should see what its players see, and leave it off on a dedicated server. It can be switched during a session. Turning it off returns each system's membership to `InterestMembership.Streamed`, and raises that change, as the system is next resolved.

Read the report from each system

```
public InterestMembership HostInterestMembership { get; private set; }

public event HostInterestChangedHandler? HostInterestChanged;

public delegate void HostInterestChangedHandler(InterestMembership previousInterestMembers
hip, InterestMembership currentInterestMembership);
```

`HostInterestChanged` is raised only on a host, and only while `HostInterestEnabled` is set. It arrives on the network loop thread on the interest cadence, and a handler may do engine work. `HostInterestMembership` reads `InterestMembership.Streamed` on a client, on a dedicated server, and on a host with `HostInterestEnabled` cleared.

Hide on Unspawned, not Stopped

`InterestMembership.Unspawned` means a client would not have the object at all, so it is the one to hide on. `InterestMembership.Stopped` means a client would still see the object, just without updates, so hiding on `Stopped` shows the host less than its players see.

A system leaving the world, or the host's client half dropping its link, returns the membership to `Streamed` and raises that too, so anything you hid on `Unspawned` is always shown again.

The bundle gate abstains for a host, the scene gate does not

A host's own client already holds every content bundle the server loaded, so `BundleInterestCondition` abstains for any Connection with `IsHostLoopback` set, and for one on a local in-process transport such as Yak.

The scene condition does not abstain. A host is placed in scene instances like any other client, so the scene gate settles against its real scene records. Position and scene are what can genuinely differ for a host.

Interest Management and Scaling

Pages in this category:

- What area of interest is
- Distance interest conditions
- Registering interest conditions
- Writing an interest condition
- An object is missing, or will not go away
- Stopping versus culling
- What scales and what does not
- How distances are measured: pairwise and the spatial grid
- Supplying positions to the interest system
- The host interest report

How distances are measured: pairwise and the spatial grid

Choose a technique

`InterestManager.SetInterestTechnique(InterestTechnique interestTechnique, GridInterestAxes gridInterestAxes = GridInterestAxes.Xyz)` sets how every distance condition in a world measures distance. Both techniques reach identical verdicts; only the cost differs.

```
interestManager.SetInterestTechnique(InterestTechnique.SpatialGrid, GridInterestAxes.Xz);
```

The current choice reads back from `Technique` and `InterestAxes`.

- `InterestTechnique.Pairwise` (the default) measures each connection's nearest controlled object directly. Use it where most objects are in range of most players, which is most small and mid-sized worlds.
- `InterestTechnique.SpatialGrid` narrows candidates through the shared `SpatialInterestGrid`, then measures the survivors exactly against the same cutoffs. Use it in a world much wider than its cutoff. In a world where everything is in range of everything, it is slower than pairwise.

Pick the technique from the world's geometry, not from player count.

Restrict the axes for a flat world

`GridInterestAxes` chooses which axes a distance is measured across: `Xyz` (the default), `Xy`, `Xz` or `Yz`. It applies to both techniques. A top-down or side-on world can measure only in its plane, so a tower directly overhead no longer counts as a step away.

The technique is locked once a distance condition registers

`SetInterestTechnique` is refused, with a logged error, once any distance condition has registered in the world. Set the technique and the axes during setup, before anything with a distance rule spawns.

Query SpatialInterestGrid directly

`SpatialInterestGrid` is the index `InterestTechnique.SpatialGrid` uses. It knows nothing about transforms. It answers "which of these points lie within this radius of this point".

- `TryBeginPass(uint pass)` opens a pass. It returns `true` when the caller must insert this pass's points, so several callers can share one grid in a pass and only the first builds it.
- `Insert(uint id, Vector3 position)` adds one point for the current pass. A non-finite position is dropped.
- `CollectWithin(Vector3 center, float radius, HashSet<uint> results)` adds the id of every inserted point within `radius` of `center`. Results are exact.
- `IsIndexed(uint id)` reports whether an id added any point this pass, which tells "indexed and out of range" apart from "never indexed".
- `ConfigureAxes(GridInterestAxes gridInterestAxes)` chooses the plane, from the next rebuild.

`CellSize` reads the current cell size, which is never below `DefaultCellSize` (32 units).

An object is missing, or will not go away

A client receives objects from a scene it never loaded

An ungated spawn raises no error. It lands in the wrong scene on the client. The Unity inspector for `UnityInterestManager` warns about this when its rule list has no scene rule: "No rule gates scenes. A client will receive objects from scenes it never loaded, which puts them in the wrong scene rather than raising an error, and several open instances of one scene cannot be told apart by distance."

Fix: add the scene rule to the world's interest conditions, unless your own rule arbitrates scene membership.

Nothing is ever culled, however large the stop distance is

A distance condition reads positions through its own `TryReadSourcePosition` (the Unity conditions read the linked GameObject's transform), not through the reader set with `InterestManager.SetPositionReader`. A pair it cannot measure is never restricted. That happens when the connection controls nothing yet, or when `TryReadSourcePosition` returns `false` for the object or for every object the connection controls.

Fix: make sure each player controls the object it plays through, and that `TryReadSourcePosition` answers for both the evaluated object and the controlled ones.

An object disappears at range when it should only freeze, or the reverse

Past the despawn cutoff, `DistanceInterestLadder.Resolve` returns an unmet spawn. Past only the stop cutoff, it returns an unmet stop. Despawn always outranks stop. Whether an unmet stop removes the object depends on `NetworkSystem.DespawnWhenStoppedEnabled`: off, the object freezes and keeps its last state; on, it is treated as culled once stopped.

On a grouped object (several systems sharing one `NetworkSystemGroup`), the object leaves only once every member's stop is unmet and has `DespawnWhenStoppedEnabled` set. One member with the flag off keeps the whole object in place. An ungrouped system with the flag on despawns as soon as its stop is unmet.

Fix: check `DespawnWhenStoppedEnabled` on every system in the group, not just one.

Objects flicker in and out at a cull boundary

A distance cull has no hysteresis, so a player moving back and forth across a cutoff flips the verdict each evaluation. Distance is re-measured only every `EvaluationCadenceTicks`, so flicker only happens when a player can cross from just inside to just past the cutoff within one cadence.

Fix: widen the gap between the cutoff and how far a player can move in one `EvaluationCadenceTicks`, by raising the cutoff, lowering the cadence, or both. There is no setting to smooth the boundary itself.

A new client sees nothing for a second or two

A connection is not registered against any system until `Connection.CanReceiveState` is true, which happens once its link is measured: typically one round trip after it authenticates, and immediately for a host's own client. Until then it sees no objects.

This is expected and resolves itself. If it never resolves, check that the transport completes a round trip for that connection.

A join or scene load arrives as one huge tick and drops

`ServerManager.MaximumObjectKilobytesPerTick` (50 by default, where 1 kB is 1,024 bytes) caps how many kilobytes of new objects one client is sent per tick. Whatever does not fit is sent on the following ticks. Objects re-sent after packet loss share the same budget. Pacing cannot be switched off, so a join that still arrives as one tick too large for the link means the budget is too wide: lower it. One object wider than the whole budget is still sent, alone on its tick.

To check whether the budget is holding spawns back:

- `InterestManager.TotalSpawnsDeferred` counts spawns that have had to wait. It stays near zero in a world that never fills the budget.
- `Connection.SystemsAwaitingSpawnAdmissionCount` is how many spawns one connection is still owed.
- A warning is logged once a connection has been owed spawns for `InterestManager.SpawnAdmissionStallWarningTicks` (90) ticks without receiving one. It names the connection and the current `MaximumObjectKilobytesPerTick`.

If that warning fires, the world spawns faster than the budget lets that peer receive: raise `MaximumObjectKilobytesPerTick`, or spawn less at once. A `NetworkSystemGroup` wider than the budget is not a fault; its objects arrive over more than one tick.

Setting the technique or axes logs an error and does nothing

`InterestManager.SetInterestTechnique` is refused, with a logged error, once any distance condition has registered in that world, globally or per system.

Fix: call `SetInterestTechnique` during setup, before any system with a distance rule can spawn.

An authored distance rule stops objects but never despawns them

The inspector-authored Unity rule (`AuthoredDistanceInterestCondition`) only exposes a stop distance. It always leaves the despawn cutoff unset (`DistanceInterestLadder.UnsetDistance`), so an object authored this way only stops by distance, never culls.

Fix: to cull by distance, construct a `DistanceInterestCondition` in code. It takes `stopDistance` and `spawnDistance` separately.

Every distance lever on this page is Pro-only

Distance conditions, the spatial grid and object pacing are Pro-only, as are `AuthoredDistanceInterestCondition` and `DistanceInterestCondition` in Unity. The scene rule and the base interest system ship in Free. If one of these settings seems not to apply, check whether the build is Free.

Registering interest conditions

Using Unity? See Network Interest Object

Register a condition globally or on one system

`InterestManager.AddCondition` registers a condition for every `NetworkSystem` that starts from then on. Systems already started are not affected until they next start.

```
coreManager.InterestManager.AddCondition(interestCondition);
```

`NetworkSystem.AddInterestCondition` registers a condition on one system only, such as a per-prefab rule:

```
networkSystem.AddInterestCondition(interestCondition);
```

Both stack, and every condition can only restrict. `AddCondition` rejects a condition whose `Capabilities` is `InterestEffect.None` and logs an error. Register during setup; a condition registered mid-session reaches already-spawned objects only as they respawn.

Remove a condition

`InterestManager.TryRemoveCondition` removes a global condition and returns `true` if it was registered. `NetworkSystem.TryRemoveInterestCondition` does the same for a system's own condition.

```
bool wasRemoved = coreManager.InterestManager.TryRemoveCondition(interestCondition);
bool wasRemovedFromSystem = networkSystem.TryRemoveInterestCondition(interestCondition);
```

Systems already started keep a removed global condition until they next start. Removing the last condition of an effect leaves that effect's current resolution in place rather than restoring the default.

`InterestManager.HasConditions` is true while any condition is registered globally. It stays false in a world that uses only per-system conditions; that is not a fault.

The engine registers the scene gate by default

`InterestManager.DefaultConditions` lists the conditions the engine registered itself, in registration order. It holds the scene spawn gate (`SceneManager.SpawnGate`), registered globally so a client only sees the contents of scenes it has loaded. The bundle gate (`BundleManager.SpawnGate`, Pro) is not registered by default: attach it globally or per prefab wherever bundle gating should apply. `DefaultConditions` is empty when the engine registered none.

To arbitrate everything yourself, call `RemoveDefaultConditions()`. It removes only the engine's conditions and returns how many it removed. Systems already started keep them until they next start.

```
uint removedCount = coreManager.InterestManager.RemoveDefaultConditions();
```

Show an object only to its controller

`ControllerOnlyInterestCondition` shows a system only to the client that controls it, for objects such as an inventory, a private quest log or a hand of cards:

```
ControllerOnlyInterestCondition controllerOnlyInterestCondition = new();

inventorySystem.AddInterestCondition(controllerOnlyInterestCondition);
```

- It is a `Spawn` condition: other clients never hold the object, not even a stopped copy.
- When control moves, the next evaluation removes the object from the previous controller and spawns it for the new one.
- A group is answered as one object, so every member is shown to a client that controls any one of them.
- It holds no state, so one instance can be registered on any number of systems.

Register it per system, never globally. Registered with `AddCondition`, it hides every object no client controls, so every server-controlled object disappears from every client.

Force a re-resolve after a teleport

Conditions are evaluated every `InterestManager.EvaluationCadenceTicks` ticks (5 by default). See What area of interest is. Ordinary movement needs nothing more.

For a move the cadence cannot anticipate, such as a teleport, a scene change or a reparent, call `RefreshInterest(NetworkSystem)`. It re-resolves that system against every Connection at the start of the next pass:

```
coreManager.InterestManager.RefreshInterest(networkSystem);
```

For a change on the player's side, such as a teleported character, a newly controlled object or a scene move, call `RefreshInterest(Connection)`. It re-resolves that connection against every evaluated system:

```
coreManager.InterestManager.RefreshInterest(connection);
```

Stopping versus culling

Using Unity? See Frozen or gone: what a stop does to the object

Interest can end replication to a connection in two ways:

- **Cull** (`InterestEffect.Spawn`): while an unmet `Spawn` condition holds, the `NetworkSystem` does not spawn for the connection, and one already spawned is despawned like any ordinary despawn.
- **Stop** (`InterestEffect.Stop`): while an unmet `Stop` condition holds, the system stops replicating to the connection but the connection keeps its object. A distant object can stay visible at no replication cost. It is reclaimed only if a `Spawn` condition later culls it too.

Each pair resolves to one InterestMembership

`InterestMembership` is the resolved state of one `(NetworkSystem, Connection)` pair:

- `Streamed`: an ordinary observer, receiving updates. This is the default for a connection with no interest entry.
- `Stopped`: stopped by an unmet `Stop` condition. The connection holds no system, but its object was kept for a later re-entry.
- `Unspawned`: culled by an unmet `Spawn` condition. The connection holds neither the system nor its object.

Choose whether a stopped object is reclaimed

`NetworkSystem.DespawnWhenStoppedEnabled` decides what a stop costs the receiver:

- `false`: the stopped object stays on the receiver. Use this for something a player is looking at.
- `true`: the stopped object is reclaimed rather than left frozen. Use this for something a player has left behind.

For a grouped object, the object is reclaimed only once every member is stopped with this set. One member that is still streamed, or left to freeze, keeps the whole object. An ungrouped system's stop with this set is a despawn.

Override a stop with InterestStopResolving

Conditions can only restrict, so a rule authored for a whole object stops every system on it. `NetworkSystem.InterestStopResolving` lets a subscriber decline a stop no condition can undo, or impose one no condition asked for.

```
public delegate void InterestStopResolvingHandler(ref InterestStopContext interestStopCont
ext);
public event InterestStopResolvingHandler? InterestStopResolving;

public struct InterestStopContext
{
    public readonly NetworkSystem NetworkSystem;
    public readonly Connection Connection;
    public bool IsStopped;
}
```

- The context is seeded with what the conditions resolved and passed by reference through every subscriber in subscription order. The last subscriber to write settles the answer.

- A subscriber speaking for one system tests `NetworkSystem` first and otherwise leaves `IsStopped` alone.
- Where `DespawnWhenStoppedEnabled` is set, declining a stop keeps the whole object with that connection.
- It is raised once per pair on the interest cadence, on the network loop thread. Subscribers must be pure, cheap reads that neither start nor stop systems, change observers, nor block.
- It is never raised for an emulated connection or one on an in-memory transport. It is raised for a host's own client while `InterestManager.HostInterestEnabled` is set, but a stop settled there is only reported through `HostInterestMembership`, not acted on.
- Subscribing makes the interest pass evaluate the system even when it has no interest condition. Removing the last subscriber undoes that.

React when interest moves

Conditions and `InterestStopResolving` decide interest and must not act on it. To act when a pair actually moves, such as showing a "player left your view" marker or recording analytics, subscribe to an interest change event. Both events are raised on the authority only, on the network loop thread, after the tick's state has been sent. A handler may do engine work; a system it starts goes live with the next tick.

For one object, subscribe to `NetworkSystem.InterestChanged`, or override `OnInterestChanged` in a subclass:

```
public delegate void InterestChangedHandler(Connection connection, InterestMembership prev
iousInterestMembership, InterestMembership interestMembership);
public event InterestChangedHandler? InterestChanged;

networkSystem.InterestChanged += OnInterestChanged;

void OnInterestChanged(Connection connection, InterestMembership previousInterestMembershi
p, InterestMembership interestMembership)
{
    if (interestMembership is InterestMembership.Unspawned)
    {
        // This client no longer has the object at all.
    }
}
```

It is raised only for a move. An object that reaches a connection with no condition ever restricting it raises nothing, and neither does a system or connection leaving the world, since their own events cover that. It is never raised for a host's own client, which uses `HostInterestChanged` instead (see The host interest report).

For the whole world, subscribe to `InterestManager.InterestChanged`. It is raised once per tick that moved anything, with every move from that tick:

```
public delegate void InterestChangesHandler(IReadOnlyList<InterestChange> interestChanges)
;
public event InterestChangesHandler? InterestChanged;
```

```
coreManager.InterestManager.InterestChanged += OnInterestChanges;

void OnInterestChanges(IReadOnlyList<InterestChange> interestChanges)
{
    for (int i = 0; i < interestChanges.Count; i++)
    {
        InterestChange interestChange = interestChanges[i];

        if (interestChange.InterestMembership is InterestMembership.Streamed)
        {
            // interestChange.Connection gained interestChange.NetworkSystem.
        }
    }
}
```

- Each `InterestChange` carries the `NetworkSystem`, the `Connection`, the `PreviousInterestMembership` and the new `InterestMembership`.
- One call covers the whole tick, which suits a per-player visibility tracker.
- The list is cleared once your handler returns. Copy whatever must outlive the call.
- A move whose system has since left the world is still included, so check `NetworkSystem.IsStarted` where that matters.

To read where a pair stands now, call `NetworkSystem.GetInterestMembership(connection)` on the authority. A connection with no entry reads `Streamed`.

Supplying positions to the interest system

> **Using Unity?** The Unity integration registers a position reader for you. See Limit what a client receives by distance to configure the distance conditions.

The replay hold radius (`SystemManager.ReplayHoldRadius`) reads object positions through a position reader registered on the `InterestManager`. Core has no way to know where an object is, so a bare `CoreManager` registers none, and a replay then holds every system it cannot measure. Scene-based interest and distance conditions do not use this reader. A distance condition reads positions through its own `TryReadSourcePosition` (see Distance interest conditions).

Implement IInterestPositionReader

```
public interface IInterestPositionReader
{
    bool TryReadPosition(NetworkSystem networkSystem, out Vector3 position);
}
```

- `Vector3` is `System.Numerics.Vector3`. If your engine uses a different vector type, convert it in the reader.
- Return `false` for a system whose object has been destroyed or has not finished spawning. Never return a stale or default position.
- Write the raw read only. The manager reads each system once per interest pass and caches the result, failed reads included, so your reader needs no caching of its own.

A dictionary-backed reader suits a host with no scene graph to query:

```
public sealed class DictionaryInterestPositionReader : IInterestPositionReader
{
    private readonly Dictionary<NetworkSystem, Vector3> _positions = new();

    public void SetPosition(NetworkSystem networkSystem, Vector3 position) => _positions[n
etworkSystem] = position;

    public bool TryReadPosition(NetworkSystem networkSystem, out Vector3 position) => _pos
itions.TryGetValue(networkSystem, out position);
}
```

A system with no recorded position reads as unmeasurable, not as position zero.

Register the reader on the interest manager

```
public void SetPositionReader(IInterestPositionReader interestPositionReader);
```

Call `InterestManager.SetPositionReader` once during setup. Pass `null` to clear it when your integration shuts down.

What scales and what does not

Four levers cut replication cost

From largest saving to smallest:

Don't send it at all. An unmet `InterestEffect.Spawn` keeps a peer from ever holding an object it has no reason to see. Nothing is queued or sent.

Stop sending it. An unmet `InterestEffect.Stop` pauses replication to a peer that already holds the object, without removing it. The object still exists on the peer.

Send it less often. `SendInterval` paces a member's updates under `TransmissionMode.Interval`: `SendInterval.Normal` sends every changed tick, and a wider interval sends at most one update per interval. A member on `TransmissionMode.Divine` sets its own rate. A spawn, resync, recovery or reconcile always carries the current value.

Send it smaller. `TransmissionMode` and per-value compression shrink each write without removing any.

Find which of three bottlenecks you have

"Replication is too expensive" is usually one of three problems, and fixing the wrong one does nothing.

- **Per-client egress:** how many bytes one peer's connection pushes per second, as `TotalBytesSent` and the per-object and state-only bandwidth stat providers measure. Fix it with spawn and stop culling first, then send interval and compression.
- **Per-tick burst size:** how much lands in a single tick. A join, a scene load, a bundle release or a peer walking back into range can put many spawns into one tick, and a large tick is costly to lose. `ServerManager.MaximumObjectKilobytesPerTick` (default 50) caps how many kilobytes of new objects one peer is sent per tick, so a large join arrives over several ticks. It does not lower the join's total cost or steady-state egress. A higher `GlobalRedundancy` makes a burst more expensive, because the extra bandwidth applies to the whole burst.
- **The resolution pass:** the server CPU spent deciding who sees what. Every registered `IInterestCondition` is evaluated per (system, Connection) pair every `InterestManager.EvaluationCadenceTicks`, so the cost grows with systems times connections however few pairs end up replicating. A world with no condition registered skips it. Cadence and staggering are the only levers.

Read the counters before pulling a lever

The bandwidth counters (`TotalBytesSent`, `TotalBytesReceived`, `TotalStatePacketPayloadBitsSent`, `TotalStatePacketPayloadBitsReceived`) and the per-object and state-only stat provider rows show what a world costs. Every figure covers the whole peer, so a per-object number is an average: divide by object count, or use the per-object stat provider row, which does it for you. See Network statistics.

These scaling features do not exist

Do not design a world around any of these:

- **No bandwidth cap on ordinary traffic.** Object pacing caps new objects per tick, but nothing caps the rest of a connection's traffic, and nothing decides what goes first when a peer falls behind.

- **No per-object send priority.** Every eligible object is served the same way.
- **No broadphase hook for interest.** An integration cannot hand the interest pass a pre-narrowed set of candidates. Every world uses the per-pair pass.

Most levers are Pro-only

Distance and spatial-grid culling and object pacing (`MaximumObjectKilobytesPerTick`) are Pro-only. Free has the broadcast-to-everyone default and the interest system without those conditions.

Pro also compresses values more tightly than Free. This is fixed by the edition that built the peer and cannot be toggled per world.

Work out a budget for a small world

Take 50 objects, each with one `NetworkMember` sending every tick at `SendInterval.Normal`, replicated to 20 connections.

- **Egress:** each connection carries its own 50 objects' updates per tick, so per-connection egress grows with the number of objects it can see.
- **Resolution pass:** one distance condition means 50 ? 20 = 1,000 pair evaluations every `EvaluationCadenceTicks`. A cadence of 1 runs all 1,000 every tick. A cadence of 10 spreads them to about 100 per tick, at the cost of verdicts up to 10 ticks old.
- **Join burst:** a joining peer is owed all 50 spawns at once. Fifty small objects fit inside the default 50 kB budget, so pacing never engages.

This world does not need any lever pulled yet.

Writing an interest condition

An `IInterestCondition` decides one aspect of what a `Connection` receives for a `NetworkSystem`. For how conditions stack and when they run, see What area of interest is.

```
public interface IInterestCondition
{
    InterestEffect Capabilities { get; }

    InterestResult Evaluate(NetworkSystem networkSystem, Connection connection);
}
```

Declare every effect the condition can return

`Capabilities` states every effect the condition can ever return an opinion on. An effect left out is never asked, so an opinion on an undeclared effect is silently ignored. Declare only the effects the condition speaks to, since each declared effect costs an evaluation per pair. `InterestEffect.None` is rejected at registration.

`InterestEffect` is a `[Flags]` enum:

- `Spawn`: while unmet, the system does not exist for the connection, and one already spawned is despawned.
- `Stop`: while unmet, the system stops for the connection but its object is kept. See Stopping versus culling.

A spawn-only condition declares `InterestEffect.Spawn`. One that speaks to both declares `InterestEffect.Spawn | InterestEffect.Stop`.

Return an InterestResult

`Evaluate` returns an `InterestResult`, which can carry an opinion on each declared effect, so one measurement can veto both spawn and stop.

```
public readonly struct InterestResult
{
    public readonly InterestOpinion SpawnOpinion;
    public readonly InterestOpinion StopOpinion;

    public static InterestResult None => default;

    public InterestResult WithSpawn(bool isMet);
    public InterestResult WithStop(bool isMet);

    public static InterestResult Spawn(bool isMet);
    public static InterestResult Stop(bool isMet);
}
```

- `InterestResult.Spawn(isMet)`: a spawn-only verdict. An unmet one culls the system for the pair.
- `InterestResult.Stop(isMet)`: a stop-only verdict. An unmet one stops the system for the pair and keeps its object.
- `WithSpawn(isMet)` / `WithStop(isMet)`: set the other effect on a copy, for a condition that speaks to both.
- `InterestResult.None`: no opinion on any effect. Return this for a pair the condition cannot measure,

rather than guessing a restriction.

Each effect carries an `InterestOpinion`:

```
public enum InterestOpinion : byte
{
    None = 0,
    Unmet = 1,
    Met = 2,
}
```

Only `Unmet` restricts. `Met` and `None` resolve the same way. Any unmet `Spawn` opinion from any condition keeps the system away from the connection, and any unmet `Stop` opinion stops it, whatever the other conditions said.

Keep Evaluate a pure read

`Evaluate` runs on the network loop thread, on the manager's evaluation cadence. It must not start or stop systems, change observers, or block.

Start from one of these conditions

A condition that always approves spawn, the shape a non-distance rule (scene membership, team, bundle ownership) takes:

```
private sealed class AlwaysMetInterestCondition : IInterestCondition
{
    public InterestEffect Capabilities => InterestEffect.Spawn;

    public InterestResult Evaluate(NetworkSystem networkSystem, Connection connection) =>
InterestResult.Spawn(isMet: true);
}
```

A condition driven by a caller-supplied policy:

```
private sealed class PolicyInterestCondition : IInterestCondition
{
    public InterestEffect Capabilities { get; }
    private readonly Func<NetworkSystem, Connection, InterestResult> _evaluator;

    public PolicyInterestCondition(Func<NetworkSystem, Connection, InterestResult> evaluat
or, InterestEffect capabilities = InterestEffect.Spawn | InterestEffect.Stop)
    {
        _evaluator = evaluator;
        Capabilities = capabilities;
    }

    public InterestResult Evaluate(NetworkSystem networkSystem, Connection connection) =>
_evaluator(networkSystem, connection);
}

// Cull one connection from a system while every other connection keeps receiving it:
networkSystem.AddInterestCondition(new PolicyInterestCondition(
    (system, connection) => InterestResult.Spawn(connection != culledConnection)));
```

Register a condition on one system with `NetworkSystem.AddInterestCondition`, or globally with `InterestManager.AddCondition`. See Registering interest conditions.

Unity conditions authored on a prefab derive from

AuthoredInterestCondition

A condition authored on a prefab derives from `AuthoredInterestCondition`, a public abstract `[Serializable]` class, and builds the runtime `IInterestCondition` in its `CreateCondition` method. The add-rule menu finds subclasses automatically.

Capturing a shot

Using Unity? See Firing and judging a shot.

Capture on the frame the shot is fired

```
LagCompensatedShot lagCompensatedShot = coreManager.LagCompensationManager.CaptureShot(ori
gin, direction);
```

`CaptureShot(Vector3 origin, Vector3 direction)` returns a `LagCompensatedShot` with the origin and direction unchanged and the view that `CaptureView()` returns at that moment. The direction does not need to be of unit length.

Call it on the frame the player fires, not when the next tick's client state is built. Each rendered frame within a tick shows a different moment, so a view taken a frame later is not what the player aimed at.

Send the shot whole

Put the `LagCompensatedShot` itself in a message, a remote call argument or a client state member:

```
using Nucleus.Managers.LagCompensation;
using Nucleus.Managers.Messages;
using Nucleus.Serializers;

[NetworkType]
public struct ShotMessage : IMessage
{
    public LagCompensatedShot Shot;
}

if (coreManager.TransportManager.TryGetServerConnection(out Connection serverConnection))
    serverConnection.SendMessage(Channel.Reliable, new ShotMessage { Shot = lagCompensated
Shot });
```

Do not copy the origin and direction into plain `Vector3` fields. A plain `Vector3` is sent at lower precision, which can turn the direction enough to miss a headshot at range: roughly 0.04 metres off at 50 metres and 0.09 metres at 100 metres. A `LagCompensatedShot` keeps full precision. An infinity or NaN in either vector arrives as zero.

`LagCompensatedShot` is compared bit for bit, view included, so a member holding one sees each new shot as a change.

Pass the shot to a query on the server

The shot's `View`, `Origin` and `Direction` are the query's arguments:

```
LagCompensatedShot shot = shotMessage.Shot;

bool isHit = coreManager.LagCompensationManager.TryRaycast(messageContext.SenderConnection
, shot.View, shot.Origin, shot.Direction, maximumDistance, hitboxLayerMask, out LagCompens
atedHit lagCompensatedHit);
```

See Judging a shot with rewound queries for every query.

A 2D shot captures with a Z of zero

Capture with a Z of zero, then read the shot back through `Origin2D` and `Direction2D` for the 2D queries:

```
LagCompensatedShot lagCompensatedShot = coreManager.LagCompensationManager.CaptureShot(new
 Vector3(origin2D, 0f), new Vector3(direction2D, 0f));

bool isHit = coreManager.LagCompensationManager.TryRaycast2D(messageContext.SenderConnecti
on, shot.View, shot.Origin2D, shot.Direction2D, maximumDistance, hitboxLayerMask, out LagC
ompensatedHit lagCompensatedHit);
```

Capture a view alone

A shot that is not a ray, such as a melee swing whose volume the server builds, only needs the view. `CaptureView()` returns the `LagCompensationView` on screen now, and the server passes it to an overlap query. A `LagCompensationView` is a `[NetworkType]`, so it can ride a message, a remote call or a client state member directly. In client state it changes only on ticks that fire.

A server or host captures the present

On a server or host, `CaptureView` returns the newest recorded tick with a fraction of zero, so a host's own shots are judged against the present, which is what its player saw.

An empty view, `default`, has a `Tick` of `NetworkLoopManager.UnsetTick` and is judged as the present. A client captures an empty view before any state has arrived, and a free build always does.

How lag compensation works

> **Using Unity?** See Lag compensation in Unity.

A client aims at the past

A client draws the world a little behind the server, so by the time a shot reaches the server, a moving target has moved on. Judged against the server's present, a perfectly aimed shot misses. Lag compensation lets the server judge a shot against the world as the shooter saw it. It is a Pro feature; see What a free build does.

The server judges against its own hitbox history

`LagCompensationManager`, reached as `CoreManager.LagCompensationManager`, never moves objects, colliders or the physics engine. It keeps its own history of hitboxes:

Record. Each object that can be hit registers a hitbox root: a fixed list of hitbox shapes and an `IHitboxPoseSource` that reports where they are. The server records every root's poses once per tick and keeps them for `MaximumRewindMilliseconds`. See Registering hitboxes.

Capture. On the frame the shooter fires, it calls `CaptureShot`, which returns a `LagCompensatedShot`: the ray exactly as drawn, plus a `LagCompensationView` naming what was on screen. The shooter sends the shot to the server. See Capturing a shot.

Judge. The server passes the shot to a query such as `TryRaycast`, which rebuilds every hitbox as the shooter's screen showed it and tests the shot against them. See Judging a shot with rewound queries.

A query only reads the history, so the server can run any number of them at any time without disturbing the live simulation.

Only a server records. Registering on a client is refused without an error, so integration code can register on every peer. When this peer's server stops, every history is released.

A view is a tick and a blend

A `LagCompensationView` has a `Tick`, the older of the two server ticks the screen was blending between, and a `Fraction` toward the next tick, from 0 (exactly `Tick`) to 255 (exactly the next tick). On a server or host, the view is the newest recorded tick with a fraction of zero, because a server's screen shows the present.

A target whose position is sent with `TransmissionMode.Interval` is drawn behind by part of its interval. The server judges it where the shooter drew it, with no extra work from you.

A stopped target stays hittable for the rest of the window

A root whose system stops or returns to its pool stops recording, but its history stays readable until its newest tick leaves the rewind window. So a target killed a moment before a shot arrives can still be hit by it. The hit's `LagCompensatedHit.NetworkSystem` is then `null`, and `LagCompensatedHit.SystemId` still names it.

A pooled system handed out again, or restarted under the same Id, starts a fresh history and can never be found through the old one.

Lag compensation judges a shot against the past. Its counterpart, spawn compensation, brings a freshly spawned object up to the present.

What a free build does

Every name on these pages exists in both editions, so the same code compiles against either. In a free build:

- every `TryRegisterHitboxRoot` call is refused, with one logged error for the whole session, and nothing is recorded;
- `CaptureView` returns an empty view, and `CaptureShot` carries its ray exactly with an empty view;
- every query finds nothing.

Lag Compensation

Lag compensation is a Pro feature. Pages in this category:

- How lag compensation works
- Registering hitboxes
- Capturing a shot
- Judging a shot with rewound queries
- The rewind window and the view clamp

Registering hitboxes

Using Unity? See Making an object hittable, where `NetworkHitboxRoot` does all of this for you.

Register one hitbox root per object

Call `LagCompensationManager.TryRegisterHitboxRoot` with:

- the `NetworkSystem` the hitboxes belong to, one root per system;
- an `IHitboxPoseSource` that writes the hitboxes' world poses each recorded tick;
- a `HitboxDefinition` for each hitbox, its fixed shape in its own unscaled frame, in the order the pose source writes them.

```
HitboxDefinition[] hitboxDefinitions =
[
    new(HitboxShape.Capsule, radius: 0.3f, size: Vector3.Zero, height: 1.8f, layer: 8),
    new(HitboxShape.Sphere, radius: 0.15f, size: Vector3.Zero, height: 0f, layer: 9),
];

bool isRegistered = coreManager.LagCompensationManager.TryRegisterHitboxRoot(networkSystem
, soldierHitboxes, hitboxDefinitions);
```

The definitions are copied, so you may reuse the array. A root holds 1 to `LagCompensationManager.MaximumHitboxesPerRoot` (64) hitboxes.

`TryRegisterHitboxRoot` returns `false`:

- with a logged error, when the system or pose source is `null`, the hitbox count is out of range, or the system already has a root;
- without an error, when this peer is not running a server. Register once the server is running, for example when the system starts on the server;
- in a free build, logging once that lag compensation is not available.

The root's history length comes from `MaximumRewindMilliseconds` at registration; see The rewind window and the view clamp.

Describe each hitbox

Shape	Described by
`Sphere`	`Radius`
`Box`	`Size`, the full size on each axis
`Capsule`	`Radius` and `Height`, the full height including both rounded ends, along the hitbox's own up axis
`Circle`	`Radius`, in the XY plane
`Box2D`	`Size`, reading X and Y only, in the XY plane
`Capsule2D`	`Radius` and `Height`, along the hitbox's own up axis, in the XY plane

The constructor is `HitboxDefinition(HitboxShape shape, float radius, Vector3 size, float height, byte layer = 0)`; fields a shape does not use are ignored. `Layer` runs from 0 to 31, and a larger

value wraps into that range. A query's layer mask, one bit per layer, chooses which layers it can hit, so you can keep heads, bodies and shields on separate layers.

Scale is not part of the definition. It is recorded with every pose, so a hitbox that grows or shrinks is judged at its size on the tick asked about.

Write every hitbox's pose every tick

`IHitboxPoseSource` has one method:

```
public ulong WriteHitboxPoses(Span<HitboxPose> hitboxPoses);
```

It is called on the network loop thread, once per recorded tick, with one `HitboxPose` slot per hitbox in registration order. It must not allocate. For each hitbox, write:

- `Position`: the world position of the hitbox's center, including any offset from what it follows.
- `Rotation`: the world rotation. A capsule's length runs along its own up axis, so turn the rotation onto a capsule lying along another axis.
- `Scale`: the absolute world scale in the hitbox's own frame. For a capsule, Y is the scale along its length and X and Z the two across it.

Return a mask with bit `i` set when hitbox `i` is active this tick. An inactive hitbox is recorded but never hit on that tick. Still write every hitbox's pose every tick, active or not, because a shot judged between two ticks uses both ticks' poses.

```
using System;
using System.Numerics;
using Nucleus.Managers.LagCompensation;

public sealed class SoldierHitboxes : IHitboxPoseSource
{
    public Vector3 Position;
    public Quaternion Rotation = Quaternion.Identity;
    public bool IsDowned;

    public ulong WriteHitboxPoses(Span<HitboxPose> hitboxPoses)
    {
        // The body capsule, centered 0.9 m above the soldier's feet.
        hitboxPoses[0].Position = Position + Vector3.Transform(new Vector3(0f, 0.9f, 0f),
Rotation);
        hitboxPoses[0].Rotation = Rotation;
        hitboxPoses[0].Scale = Vector3.One;

        // The head sphere, 1.65 m up.
        hitboxPoses[1].Position = Position + Vector3.Transform(new Vector3(0f, 1.65f, 0f),
 Rotation);
        hitboxPoses[1].Rotation = Rotation;
        hitboxPoses[1].Scale = Vector3.One;

        // A downed soldier is recorded but cannot be hit.
        return IsDowned ? 0UL : 0b11UL;
    }
}
```

A source that throws is logged and loses only its own slot for that tick; every other root is still recorded.

2D hitboxes lie in the XY plane

A 2D game registers `Circle`, `Box2D` and `Capsule2D` hitboxes and judges them with the 2D queries. Write a 2D pose in the plane: a `Position` with a Z of zero and a `Rotation` about the Z axis only. Only the X and Y of `Scale` are read; for a 2D capsule, X is the scale across it. See 2D queries for how 2D and 3D hitboxes mix.

End a recording

The engine unregisters a system that stops or returns to its pool. Call `TryUnregisterHitboxRoot(networkSystem)` only to stop recording a system that keeps running; it returns `true` when the system had a root. Either way, what was recorded stays readable until it leaves the rewind window.

A grouped object registers once

Register one root for a grouped object, under one member, not one per member, or a single shot reports each hitbox once for every root that recorded it. When there is a choice, use the member carrying the object's `TransformComponent`, so a target sent on an interval is judged where it was drawn.

Judging a shot with rewound queries

Using Unity? See Firing and judging a shot.

Every query takes a shooter and a view

Each `LagCompensationManager` query rebuilds every registered hitbox where the shooter saw it and tests the shot against them. All of them start with the same two arguments:

- `shooterConnection`: the `Connection` that fired, normally `MessageContext.SenderConnection`. Its view is checked against what the server could have shown it, and hitboxes on any object it controls are skipped, so a shot never hits its own shooter. Pass `null` for a shot the server fires itself.
- `lagCompensationView`: the view captured when the shot was fired, normally `LagCompensatedShot.View`. An empty view, `default`, means the present.

```
private void OnShotReceived(in MessageContext messageContext, ShotMessage shotMessage)
{
    if (!_coreManager.TransportManager.IsServerStarted)
        return;

    LagCompensatedShot shot = shotMessage.Shot;

    if (_coreManager.LagCompensationManager.TryRaycast(messageContext.SenderConnection, sh
ot.View, shot.Origin, shot.Direction, MaximumDistance, HitboxLayerMask, out LagCompensated
Hit lagCompensatedHit))
    {
        // Apply damage to lagCompensatedHit.SystemId, scaled by which hitbox lagCompensat
edHit.HitboxIndex names.
    }
}
```

The layer mask has one bit per hitbox layer. A query never moves anything and allocates nothing, so it can run as often as you need.

Pick a 3D query

Query	Returns	Use it for
`TryRaycast`	the nearest hit	a hitscan shot
`RaycastAll`	every hit, nearest first	a piercing shot
`TrySphereCast`	the first hitbox a swept sphere touches	a thick shot that stops at the first thing it touches
`SphereCastAll`	every hitbox a swept sphere touches, nearest first	a thick piercing shot
`OverlapSphere`	every hitbox a sphere overlaps, nearest first	an explosion
`OverlapCapsule`	every hitbox a capsule overlaps, nearest first	a melee swing
`OverlapBox`	every hitbox a rotated box overlaps, nearest first	an area attack

Casts take their arguments in Unity's physics query order: origin, then radius for a sphere cast, then direction, then maximum distance. A direction does not need to be of unit length. A query with no direction, no reach or a negative radius finds nothing. Every geometric test is exact.

The `All` forms and the overlaps take a `List<LagCompensatedHit>`, clear it, fill it and return the count. Reuse one list so nothing is allocated:

```
private readonly List<LagCompensatedHit> _lagCompensatedHits = new();

int hitCount = lagCompensationManager.OverlapSphere(shooterConnection, view, blastCenter,
BlastRadius, HitboxLayerMask, _lagCompensatedHits);

for (int i = 0; i < hitCount; i++)
{
    float falloff = 1f - _lagCompensatedHits[i].Distance / BlastRadius;
    // Apply damage scaled by falloff to _lagCompensatedHits[i].SystemId.
}
```

Read what a hit reports

`LagCompensatedHit` carries:

- `NetworkSystem`: the system the hitbox belongs to, or `null` when that system has stopped since the judged tick.
- `SystemId`: the system's Id, readable even after it stops.
- `HitboxIndex`: the hitbox's index within its root, in registration order.
- `Layer`: the hitbox's layer.
- `Distance`, `Point` and `Normal`: the first contact.

The contact depends on the query:

- **Ray:** `Distance` is from the origin to the hit, and `Point` is where the ray hit.
- **Cast:** `Distance` is how far the swept shape's center travelled before touching the hitbox, and `Point` is where it first touched.
- **Overlap:** `Point` is the hitbox's nearest point to a sphere's center or a capsule's segment, or, for a box, the point of the hitbox inside it nearest its center. `Distance` is how far that point lies from the core, never more than the radius for a sphere or capsule, so an explosion can read falloff straight from it. When the core itself reaches the hitbox, `Distance` is zero and `Point` is the query's own origin: the sphere's or box's center, or where a capsule's segment, walked from its start, first enters the hitbox. Order those zero-distance capsule hits along the swing by the distance from the segment's start to each `Point`.

`Normal` is the hitbox's outward surface normal at the point, facing the query, and is never zero, so its negation always pushes the hitbox away from the query. At a box's edge or corner it is the direction there that faces the query.

An overlap's `Point` is a contact, not the hitbox's position. To find where the hitbox itself was, look it up through its root.

Ignore an object

Every query, 3D and 2D, has an overload that takes a `NetworkSystem ignoredNetworkSystem` right after the shooter. Pass the root of the object to ignore, or `null` to ignore nothing:

```
bool isHit = lagCompensationManager.TryRaycast(shooterConnection: null, turretSystem, view
, muzzle, aim, MaximumDistance, HitboxLayerMask, out LagCompensatedHit lagCompensatedHit);
```

The shooter's connection already skips what it controls, so use this overload when that does not cover it: a host firing for its own avatar with a `null` shooter, or a turret or NPC the server fires for.

- Naming any member of a grouped object skips the whole object, even after it despawned.
- Every object parented under the ignored one with `SystemManager.EnsureSetSystemParent`, at any depth, is skipped too, so a carried weapon or shield never blocks its owner's shot. Pass the root: naming a nested object leaves its parent hittable. Detaching an object ends the skip from that tick on.
- A nested object that despawned before the shot is judged is matched only by its own system and group, not through its old parent.

A ray can start inside a hitbox

`CanQueriesStartInHitboxes` is on by default, matching Unity's `Physics2D.queriesStartInColliders`. A ray whose origin is inside a hitbox, or a sphere or circle cast that starts touching one, hits it at distance zero, with its own origin as the point and a normal facing back along the query. That hit is the nearest one.

Turn it off and a ray or cast ignores every hitbox it starts inside or touching, and goes on to whatever lies beyond. This suits a muzzle that sits inside another object's hitbox. Overlaps are not affected. It is read on every query, so you can change it at any time.

```
coreManager.LagCompensationManager.CanQueriesStartInHitboxes = false;
```

2D queries

The 2D queries take `System.Numerics.Vector2` and run in the plane Z = 0, where `Circle`, `Box2D` and `Capsule2D` hitboxes lie:

Query	Mirrors
`TryRaycast2D`, `RaycastAll2D`	`TryRaycast`, `RaycastAll`
`TryCircleCast2D`, `CircleCastAll2D`	`TrySphereCast`, `SphereCastAll`, with arguments in `Physics2D.CircleCast`'s order
`OverlapCircle`	`OverlapSphere`
`OverlapCapsule2D`	`OverlapCapsule`
`OverlapBox2D`	`OverlapBox`, with the rectangle turned by `angleRadians` about Z, counterclockwise

`OverlapBox2D` takes radians; multiply a Unity angle in degrees by `Mathf.Deg2Rad` first. A hit on a 2D hitbox has a `Point` and `Normal` with a Z of zero.

A 2D query touches only 2D hitboxes, and a 3D query only 3D hitboxes, so a 2D game that calls `TryRaycast` finds nothing.

Set `Can2DQueriesHit3DHitboxes` to let 2D queries reach 3D hitboxes too. It is read on every query.

- A 2D ray and `OverlapBox2D` touch a 3D hitbox only where the plane Z = 0 cuts it. An `OverlapBox2D` hit's `Point` keeps a Z of zero, but its `Normal` is the 3D hitbox's own and can tilt out of the plane.
- A circle, a 2D capsule and a circle cast reach a 3D hitbox up to their radius either side of the plane.

Walls are not in the history

Only hitboxes are recorded, so a rewound query sees no walls. Clip the shot first:

1. Raycast the present-time world with your engine against level geometry, skipping the networked objects' own colliders. A target's present-time collider can stand nearer than its rewound hitbox and turn a clean hit into a miss.
2. Pass the distance to the nearest wall as the query's maximum distance.

The Unity integration ships this as `LagCompensationWalls`.

The rewind window and the view clamp

Using Unity? See UnityLagCompensationManager.

Set how far back a shot can be judged

`LagCompensationManager.MaximumRewindMilliseconds` is how far back poses are kept and a shot may be judged. It defaults to `LagCompensationManager.DefaultMaximumRewindMilliseconds`, 500 milliseconds. It is held to `MaximumRewindCeilingMilliseconds`, which also defaults to 500 milliseconds. A window set above the ceiling reads as the ceiling until the ceiling is raised, so the two can be set in either order.

```
coreManager.LagCompensationManager.MaximumRewindMilliseconds = 300;
```

Set it before objects spawn: each root's history length is fixed when the root registers. Queries read the current value every time.

An honest client's view stays inside the window unless its latency exceeds it. Size the window to the slowest link you are willing to favor the shooter for. A longer window lets a player on a slow link hit targets that have long since moved on, which the targets feel as being hit behind cover.

A view the server could not have shown is clamped

A view older than the window, or newer than anything recorded, is clamped and still judged: an old view at the oldest tick kept, a future view at the newest tick recorded, both with a fraction of zero.

Each clamp raises `LagCompensationViewClampedViolation` against the shooter's `Connection`, at most once per query, with:

- `ReportedTick`: the tick the shooter reported.
- `ClampedTick`: the tick the shot was judged at instead.

It defaults to `ViolationAction.Ignore`. A shot the server fires itself, with a `null` shooter, raises nothing.

Watch for clamps

Subscribe to `ViolationManager.LagCompensationViewClampedViolationDetected` to hear about each clamp without changing the outcome. It is raised on the network loop.

```
coreManager.ViolationManager.LagCompensationViewClampedViolationDetected += OnLagCompensat
ionViewClamped;

private void OnLagCompensationViewClamped(in ViolationContext<LagCompensationViewClampedVi
olation> violationContext)
{
    LagCompensationViewClampedViolation violation = violationContext.Violation;

    // Count clamps per violationContext.Connection; violation.ReportedTick and violation.
ClampedTick say how far out the view was.
}
```

A single clamp is normal on a link that spiked. A steady stream from one connection is either a very slow link or a client asking to be judged against the oldest world it can get. To act on it, for example by kicking, register a deciding handler with `RegisterViolationHandler<LagCompensationViewClampedViolation>`; see Violations.

The spawn compensation cap is set here too

`LagCompensationManager.MaximumSpawnCompensationMilliseconds` caps spawn compensation. It defaults to `DefaultMaximumSpawnCompensationMilliseconds` (150), and its setter holds it to at most `MaximumSpawnCompensationCeilingMilliseconds` (500). It exists in every edition, so code that sets it compiles against any build.

Channels and What Is Guaranteed

Choose a channel on every send

`Channel` has two values: `Reliable` (ordered and reliable) and `Unreliable`. Every message and call names one at the send:

```
connection.SendMessage(Channel.Reliable, myMessage);
mySystem.TrySendRpc(RpcTarget.Observers, Channel.Unreliable, myRpc);
```

The transport provides the guarantee. Nucleus adds no retries of its own, and messages and calls are never resent, so each one arrives once or not at all. If it must arrive, send it on `Channel.Reliable`.

A large unreliable send stays unreliable

A message or call larger than the transport's maximum transmission unit keeps its channel:

- A reliable one is split by the transport, which delivers every piece.
- An unreliable one is split into datagram-sized pieces and rebuilt on the receiver once every piece arrives. Losing any piece loses the whole payload, so the more pieces it needs, the more often it is lost. The receiver drops it after waiting five seconds for a missing piece.
- An unreliable payload needing more than 255 pieces (close to 300 KB at the default transmission unit) is dropped with an error. Send data that large reliably.

Late joiners never see past sends

A message or call that fired before a peer started observing is gone. If a late joiner needs the value, put it in a replicated member instead. See State, a Message, or a Call?.

Messaging and Remote Calls

Pages in this category:

- Choosing Who Receives a Call
- Channels and What Is Guaranteed
- System RPCs
- Typed Messages
- Seeing Your Own Call: RpcSelfDelivery
- Who May Send a Call, and How Many
- Handler Registration and Scope
- State, a Message, or a Call?
- When a Message or Call Reaches Its Handler
- Judging and Refusing a Relayed Call
- Sending a Message to Many Peers
- Messages and Calls That Do Not Arrive

When a Message or Call Reaches Its Handler

Outbound messages and calls leave once per frame

Sends queue, and the queue goes out once per frame at the `NetworkLoopSteps.LateSerialize` step, after its callbacks.

- Sent from a `LateSerialize` callback or any earlier step: leaves this frame.
- Sent from a `TickAdvance` or `LateVariableUpdate` callback: leaves next frame.

Messages and calls sent in the same frame leave together.

Inbound calls run after the tick's state applies

Inbound calls are dispatched on `NetworkLoopSteps.LateStateUpdate`, later than inbound messages. A call addressed to an object that spawned in the same frame finds the system already there.

Send order holds across message types

A peer's messages reach their handlers in the order they were sent, whatever mix of types they are.

A host delivers to itself immediately

When a host's server half and client half send to each other, the receiving half's handlers run at the send site, before the send call returns. This is `TransportManager.HostLoopbackDelivery` set to `HostLoopbackDelivery.Immediate`, the only value game code may set. Setting `HostLoopbackDelivery.Deferred` is refused with a logged reason. Calls always run at the send site on a host.

A handler that sends something back runs inside the original send, which is supported. One effect: if such a handler starts a spawn while the sending step is `LateSerialize` or `TickAdvance`, the spawn is deferred until the tick advances, like any other late spawn.

A handler that throws loses only its own payload

The exception is logged and that one message or call is dropped. A payload that fails to decode is dropped the same way. Every other message and call in the same frame still reaches its handler.

Messages and Calls That Do Not Arrive

The message hash could not be found for Type

`Connection.SendMessage<T0>` logs `The message hash could not be found for Type [...]` and sends nothing. A host sending to its own other half is refused the same way. There are two causes:

- **Another message type this peer already used has the same hash.** This is the usual cause; see the next section.
- **The type has no full name.** `The FullName for [...] could not be found.` is logged first. An ordinary concrete message type always has one.

Two types resolve to the same hash

A message or call type is identified by a hash of its full name, so no type needs registering before it is sent. If two different message types, or two different call types, produce the same hash, the first use of the second one logs an error naming both. From then on both types are refused for sending, for registering a handler and for receiving, and every send of either logs the error above.

To fix it, rename one of the two types or move it to another namespace. Registering a handler does not help. A message type can never collide with a call type.

A call reaches nobody

The receiver drops a call silently when it does not hold the addressed system: an interest cull, a despawn and a spawn that has not landed yet all look the same. A client's `RpcTarget.To` call is also dropped silently when the named connection has left or no longer observes the system.

So check interest first: does the receiver still hold the system, and does the recipient still observe it? A real fault shows up as a decode error or a violation.

The sender never sees its own call

`RpcSelfDelivery` defaults to `None`, so the sender's own handlers do not run. Pass `Immediate` or `OnDelivery`; see Seeing Your Own Call: RpcSelfDelivery.

A client's call is refused

On the client, `TrySendRpc` returns `false` without sending unless the client controls the system or `CanLocalClientSendRpc` is `true`. On the server, three checks can refuse the call, in this order:

`RpcAccess`. The sender must observe the system. `RpcSendAccess.AnyClient` then admits any client; otherwise only the controller is admitted. A host's own client half is held to the same rule. A refused call raises `RpcSendPermissionViolation` with the addressed system's `SystemId`.

A handler returns `RpcRelayAction.Cancel`. Any one `Cancel` stops the relay, and a handler that throws counts as `Cancel`. See Judging and Refusing a Relayed Call.

`MaximumInboundRpcsPerConnectionPerDrain` (default 64). Calls past the cap are dropped and `RpcFloodViolation` is raised once per drain. See Capping calls per drain.

A host runs calls addressed to other players

With `RpcSelfDelivery.Immediate` on an `RpcTarget.To` send, the host runs its handlers for every addressed call, including calls meant for other players. Nothing fires twice; check who the call was for. See

Seeing Your Own Call: RpcSelfDelivery.

The receiver never registered a handler

A message type with no handler on the receiver is skipped without an error or log. Make sure the receiving peer registered a handler; the sender needs none. A call type with no handler is not decoded, but the server still relays it.

Handler Registration and Scope

Using Unity? See Registering Call Handlers Across a Component's Lifetime.

Register through the system the call is for

```
public void RegisterRpcHandler<T0>(RpcManager.RpcReceivedHandler<T0> rpcReceivedHandler) w
here T0 : IRpc, new();

public void UnregisterRpcHandler<T0>(RpcManager.RpcReceivedHandler<T0> rpcReceivedHandler)
 where T0 : IRpc, new();
```

A handler registered through `NetworkSystem.RegisterRpcHandler` runs only for calls addressed to that instance, so one object's handler can never cancel the relay of another object's call.

You can register before the system spawns. From then on the handler follows the system:

- Removed when the system despawns.
- Registered again when the system respawns.
- Dropped for good when the instance returns to the pool.

Register with RpcManager to serve many systems

`RpcManager` has two overloads:

```
public void RegisterRpcHandler<T0>(RpcReceivedHandler<T0> rpcReceivedHandler) where T0 : I
Rpc, new();

public void RegisterRpcHandler<T0>(RpcReceivedHandler<T0> rpcReceivedHandler, IRpcScope rp
cScope) where T0 : IRpc, new();
```

The first is unscoped: the handler receives every call of that type in the world. The second runs the handler only for systems the scope accepts:

```
public interface IRpcScope
{
    bool IsRpcScopedTo(NetworkSystem networkSystem);
}
```

One scope may accept any number of systems. `NetworkSystem` itself is an `IRpcScope` that accepts only itself.

Handlers registered through a `NetworkSystem` cost the same however many objects handle that call type. Unscoped handlers, and handlers with any other `IRpcScope`, are each asked on every call of their type, so register through the system unless one handler really speaks for many systems.

Registering twice or inside a handler is safe

- Registering the same handler again does nothing.
- Unregistering removes by handler identity. A handler belongs to at most one scope, so this never removes another scope's handler.
- Registering or unregistering from inside a handler is allowed, including a handler unregistering itself. The change applies once the outermost dispatch finishes.

Judging and Refusing a Relayed Call

Return a verdict from every server handler

Every call handler returns an `RpcRelayAction`. On the server, the verdict decides whether the call goes on to the peers it was addressed to:

- `RpcRelayAction.Relay` (the default value, `0`) lets it travel.
- `RpcRelayAction.Cancel` keeps it on the server.

The verdict is applied on the server before anything is sent onward. Receiving peers are never told it.

Any one Cancel wins

When a call type has several handlers, every one of them runs, whatever the earlier ones returned. A single `Cancel` settles it, and a later `Relay` cannot undo it.

A handler that throws counts as `Cancel`. The handlers after it still receive the call, but the call does not travel on unjudged.

RpcContext.Disposition says what this peer is to the call

The handler's `RpcContext` (see System RPCs) carries an `RpcDisposition`, set locally on each peer:

- **`Recipient`**: the call was addressed to this peer, which acts on it. A server admitting a client's fan-out is a `Recipient` too, even though its verdict also decides whether the call travels on.
- **`Origin`**: this peer sent the call and is seeing its own copy, either as the server's local invoke or as an echo a client asked for.
- **`Routing`**: server only. The call is passing through on its way to the client it names, and was never addressed here. `RpcContext.TargetConnection` names that client; it is null for every other disposition.

On `Routing`, judge the call and return a verdict without applying its effect.

Cancel does not undo what handlers already did

Every handler, including the one that returns `Cancel`, has already run against the call. So a moderating handler can log or count the call it refuses:

```
private RpcRelayAction OnRpcReceived(in RpcContext rpcContext, MyRpc value)
{
    if (ShouldBlock(value))
    {
        // Still seen and logged, just not relayed.
        Logger.LogWarning(typeof(MySystem), $"Blocked call from {rpcContext.SenderConnecti
on?.AsString()}: {value}");
        return RpcRelayAction.Cancel;
    }

    return RpcRelayAction.Relay;
}
```

Seeing Your Own Call: RpcSelfDelivery

RpcSelfDelivery decides whether a sender sees its own call

`RpcSelfDelivery` is the last parameter of `NetworkSystem.TrySendRpc<T0>`. It controls only the sender's own view of the call; `RpcTarget` decides where the call goes.

```
public bool TrySendRpc<T0>(RpcTarget rpcTarget, Channel channel, T0 rpc, RpcSelfDelivery r
pcSelfDelivery = RpcSelfDelivery.None) where T0 : IRpc, new()
```

Value	The sender's own handlers run
`None` (default)	Never. Use it when the sender already applied the effect another way.
`Immediate`	At the send site, before the call goes anywhere. Feels instant, but runs even if the server later refuses the call.
`OnDelivery`	Once the call has been delivered, and not at all if it was refused. Use it when peers must agree on whether and when something happened.

On the server, `OnDelivery` behaves like `Immediate`, because the server is the delivery point. The same `TrySendRpc` call therefore reads the same on both peers.

OnDelivery with RpcTarget.Server is refused

A call to the server stops there, so there is nothing to come back. `TrySendRpc` returns `false` and logs an error ending with:

> RpcSelfDelivery.OnDelivery waits for a call to be passed on, and one addressed to the authority stops there.

Watch for these on a host

A host's client half and server half share one set of handlers per system.

- **`Immediate` with `RpcTarget.Server` is refused.** The call already runs the handlers locally, so `Immediate` would run them twice. `TrySendRpc` returns `false` and logs an error ending with:

> a host's call to its own authority is invoked locally already, so RpcSelfDelivery.Immediate on top of it would run the handlers twice.

- **`Immediate` with `RpcTarget.Observers` is safe.** The host's own client is served once.
- **`Immediate` with `RpcTarget.To` runs the host's handlers for every addressed send**, including calls meant for other players. The call still reaches only its recipient, but the host has already run it locally. A handler on a system the host's client listens to must check who the call was for before treating it as its own.

Tell your own copy apart

When `RpcContext.Disposition` is `RpcDisposition.Origin`, this peer sent the call and is seeing its own copy, either a local invoke on the server or the echo a client asked for with `OnDelivery`. Check it in any effect that must not fire for its own author. The other values are described in Judging and Refusing a

Relayed Call.

Who May Send a Call, and How Many

Choose who may call a system

Every `NetworkSystem` has an `RpcSendAccess`:

- `RpcSendAccess.Controller` (default): only the server, or the controlling client, may send the system a call.
- `RpcSendAccess.AnyClient`: the server and any client that currently observes the system may send. A client that stops observing the system loses the right with it.

Only the server can change it:

```
public bool TrySetRpcAccess(RpcSendAccess rpcSendAccess)
```

A client calling `TrySetRpcAccess` is rejected.

Read the current setting from `RpcAccess`. On a client, `CanLocalClientSendRpc` tells it whether it may send, as most recently declared by the server, so a well-behaved client never sends a call the server will refuse. It is always `false` on the server. The server checks every incoming call against its own `RpcAccess` and observer list, so a client cannot grant itself access.

`RpcSendAccess` is separate from `MemberWriteAccess`: a game can let observers call an object without letting them set its members.

Refused calls raise RpcSendPermissionViolation

A call sent without permission raises `RpcSendPermissionViolation`, at most once per drain:

```
public struct RpcSendPermissionViolation : IViolation
{
    public uint SystemId;
}
```

`SystemId` is the system the sender addressed. A well-behaved client can trigger it when permission is withdrawn while its calls are still in flight, so decide what it costs rather than always kicking. Subscribe through `ViolationManager.RpcSendPermissionViolationDetected`.

Capping calls per drain

`RpcManager` limits how many inbound calls one connection may deliver in a single drain:

```
public uint MaximumInboundRpcsPerConnectionPerDrain = DefaultMaximumInboundRpcsPerConnecti
onPerDrain;

public const uint DefaultMaximumInboundRpcsPerConnectionPerDrain = 64;

public const uint UnsetMaximumInboundRpcsPerConnectionPerDrain = 0;
```

The default is 64. Set it to `UnsetMaximumInboundRpcsPerConnectionPerDrain` to remove the limit.

Calls past the limit are dropped, and `RpcFloodViolation` is raised once per drain:

```
public struct RpcFloodViolation : IViolation
{
    public uint ReceivedRpcCount;
    public uint AllowedRpcCount;
}
```

`ReceivedRpcCount` is how many calls the sender delivered in the drain; `AllowedRpcCount` is the limit. Subscribe through `ViolationManager.RpcFloodViolationDetected`.

Choosing Who Receives a Call

Name the recipients with RpcTarget

Target	Reaches	Who may send it
`RpcTarget.Server`	The server, and nobody after it.	Client, including a host
`RpcTarget.Observers`	Every connection that observes the system.	Either peer
`RpcTarget.ObserversExcept(Connection excludedConnection, bool isControllerExcluded = false)`	Every observer except the named connection, and optionally except the system's controlling client.	Server
`RpcTarget.To(Connection connection)`	One named connection.	Either peer

Building a target checks nothing. Sending from the wrong peer, such as a client using `ObserversExcept`, is refused at the send and the reason is logged.

A client's call to other peers goes through the server

A client has no direct link to other clients, so the server handles each of its calls first:

- **`Observers`** from a client asks the server to fan the call out. The server admits it like any other client call, and a handler can still refuse it.
- **`To`** from a client names the recipient. The server runs its own handlers first, so it can refuse, then passes the call on. The recipient cannot tell it apart from a call the server addressed to it directly.
- **`Server`** is never forwarded. To reach other peers, address them with `Observers` or `To`.

Every client holds a `Connection` for every peer the server has reported, so `To` takes the connection itself, never an id.

A call from a client addressed the way only the server addresses calls is discarded and logged.

Sending a Message to Many Peers

Send to every authenticated client

`ServerManager.SendMessage<T0>` sends a message to every authenticated client, including a host's own client. A client still waiting to be authenticated is left out.

```
CoreManager.ServerManager.SendMessage(Channel.Reliable, new SomeMessage());
```

Clients have no fan-out; see The client direction.

Send to a chosen set

This overload sends to exactly the connections you pass, with no authentication filter:

```
CoreManager.ServerManager.SendMessage(Channel.Reliable, new SomeMessage(), myConnections);
```

`myConnections` is an `IReadOnlyCollection<Connection>`.

Send to everyone but one

`SendMessageExcept<T0>` sends to every authenticated client except one:

```
CoreManager.ServerManager.SendMessageExcept(Channel.Reliable, new SomeMessage(), excludedC
onnection);
```

The exclusion matches on `Connection.Id`, so any `Connection` for that player works, including one held from earlier. A null `excludedConnection` excludes nobody.

Send to a scene's occupants

`SendMessageToScene<T0>` sends to every active connection for which `connection.IsSceneLoaded(sceneHandle)` is true:

```
CoreManager.ServerManager.SendMessageToScene(Channel.Reliable, new SomeMessage(), sceneHan
dle);
```

These calls need a started server

On a peer that has not started a server, each of these calls does nothing and logs an error naming the method.

The client direction

A client only sends to the server it is connected to:

```
if (CoreManager.TransportManager.TryGetServerConnection(out Connection serverConnection))
    serverConnection.SendMessage(Channel.Reliable, new SomeMessage());
```

State, a Message, or a Call?

Pick by what a late joiner needs

- A **replicated member** holds a current value. The server writes it, and a peer that starts observing later sees the value as of the last write.
- A **message** goes to one peer and is forgotten.
- A **call** is addressed to a `NetworkSystem`. The receiver finds that system before any handler runs, and skips a call for a system it does not hold.

Nothing keeps a backlog of messages or calls. One that fired before a peer started observing is gone for that peer. So ask: does a peer that arrives later need to know this? If yes, it belongs in a replicated member. See How replicated state works.

Match common cases to a mechanism

- **A score.** Every peer, including one that joins mid-match, needs the current value. Replicated member.
- **A door's open state.** A peer that walks up five minutes after the door opened must see it open. Replicated member.
- **A chat line.** Nobody who arrives later needs it, and it is not about an object in the world. Message.
- **A hit effect.** It plays once on the object it happened to, and a later joiner does not need to see it. Call.
- **A server answer to one player's request.** Call, sent with `RpcTarget.To` rather than to every observer.

System RPCs

Using Unity? See Sending a Remote Call From a Component.

A system RPC is a remote call addressed to a `NetworkSystem`. The receiver finds the object before any handler runs, and skips a call for a system it does not hold.

Declare a call as a struct that implements IRpc

```
[NetworkType]
public struct DamageRpc : IRpc
{
    public int Amount;
}
```

`IRpc` is a marker interface. Every call requires an authenticated sender; there is no opt-out.

Send a call

```
bool sent = TrySendRpc(RpcTarget.Observers, Channel.Reliable, new DamageRpc { Amount = 10
});
```

- `RpcTarget` is who the call is for. See Choosing Who Receives a Call.
- `Channel` is kept whatever the call's size. See Channels and What Is Guaranteed.
- `RpcSelfDelivery` (optional, `None` by default) decides whether this peer sees its own call. See Seeing Your Own Call: RpcSelfDelivery.

A client cannot see the server's observers, so `true` from a client means the call was queued, not delivered.

Handle a call

Register the handler on the system so it runs only for that system's calls:

```
RegisterRpcHandler<DamageRpc>(OnDamageRpc);
// ...
UnregisterRpcHandler<DamageRpc>(OnDamageRpc);
```

A handler matches `RpcManager.RpcReceivedHandler<T0>`:

```
private RpcRelayAction OnDamageRpc(in RpcContext rpcContext, DamageRpc rpc)
{
    Health -= rpc.Amount;

    return RpcRelayAction.Relay;
}
```

Return `RpcRelayAction.Relay` to let the call travel on if it was going to, or `RpcRelayAction.Cancel` to keep it on this peer. Where nothing was going to travel on, the answer is ignored, so `Relay` is always safe.

Read the call's circumstances from RpcContext

Member	Meaning
`Channel`	The channel the call travelled on.

`NetworkSystem`	The system the call is addressed to, already resolved.
`SenderConnection`	The peer the call came from. Null only when neither role is running locally.
`TargetConnection`	The connection a routed call is bound for. Null unless `Disposition` is `Routing`.
`Disposition`	What this peer is to the call: `RpcDisposition.Recipient`, `RpcDisposition.Origin` or `RpcDisposition.Routing`. See [Judging and Refusing a Relayed Call](./rpc-relay-and-moderation.md).
`IsSenderServer`	Whether the server sent this call.

When `Disposition` is `RpcDisposition.Routing`, the call is for another peer: judge it and return without applying its effect.

Typed Messages

Using Unity? See Typed Messages in Unity.

A message is a payload sent to one peer. It is never routed onward and never echoed back to the sender.

Declare a message type

A message type implements `IMessage`:

```
public interface IMessage
{
    public bool IsAuthenticationRequired => true;
}
```

By default the sender must be authenticated. A message from an unauthenticated sender is refused, and the sender is kicked through `ServerManager.KickClient`.

Override `IsAuthenticationRequired` to `false` only for your authenticator's pre-authentication messages, such as the client's token, or the server refuses them. The engine ships no such types. Messages from the server reach a client that is not yet authenticated whatever the type says, so a refusal sent with `DenyClient` needs no override. The value is read once per type, when its handler is first registered.

Register a handler

```
public void MessageManager.RegisterMessageHandler<T0>(MessageReceivedHandler<T0> messageRe
ceivedHandler) where T0 : IMessage, new();
public void MessageManager.UnregisterMessageHandler<T0>(MessageReceivedHandler<T0> message
ReceivedHandler) where T0 : IMessage, new();

public delegate void MessageReceivedHandler<T0>(in MessageContext messageContext, T0 messa
ge) where T0 : IMessage, new();
```

The handler returns nothing, because a message is never relayed.

```
private void OnChatMessage(in MessageContext messageContext, ChatMessage message)
{
    // handle it
}

coreManager.MessageManager.RegisterMessageHandler<ChatMessage>(OnChatMessage);
```

Unregister the same delegate instance when you stop listening.

Send a message

```
public void Connection.SendMessage<T0>(Channel channel, T0 data) where T0 : IMessage, new(
);
```

This sends `data` to that one `Connection`. A host sending to its own other half is delivered in process. If the send logs a hash error and sends nothing, see Messages and Calls That Do Not Arrive.

Read MessageContext

```
public readonly struct MessageContext
{
    public readonly Channel Channel;
    public readonly Connection SenderConnection;
    public bool IsSenderServer => SenderConnection.IsServer;
}
```

- `Channel`: the channel the message travelled on.
- `SenderConnection`: the peer that sent it. Never null, unlike a call's `RpcContext.SenderConnection`.
- `IsSenderServer`: whether the server sent it. On a host, this is the only way to tell which half a message came from, because `TransportManager.IsServerStarted` says nothing about a particular message.

Adapting your engine

Implement IPhysicsBody

`IPhysicsBody` is everything the core reads from and writes to a body. Your adapter is also where your engine's vector and quaternion types convert to and from `System.Numerics` `Vector3` and `Quaternion`. Every member is required except the two default methods.

Member	Type	Contract
`Position`	`Vector3` (get/set)	World-space position.
`Rotation`	`Quaternion` (get/set)	World-space rotation.
`LinearVelocity`	`Vector3` (get/set)	Linear velocity.
`AngularVelocity`	`Vector3` (get/set)	Angular velocity, as an axis scaled by radians per second.
`IsSleeping`	`bool` (get/set)	Whether the body is sleeping; setting `true` puts it to sleep, `false` wakes it.
`IsKinematic`	`bool` (get)	True when the body is kinematic and does not respond to forces. `PhysicsConvergence.Step` skips a kinematic body.
`Gravity`	`Vector3` (get)	The gravitational acceleration actually acting on this body.
`Drag`	`float` (get)	The body's linear drag factor.
`Mass`	`float` (get)	The body's mass.
`AddForce(Vector3 force)`	method	Applies a continuous force to the body's center of mass for the current physics step.

Two getters cause real bugs if they are wrong:

- **`Gravity`** must return `Vector3.Zero` when the body has gravity disabled, not the world's gravity. Projection uses exactly this value, so returning world gravity makes a gravity-disabled body fall under projection while the engine holds it still.
- **`Drag`** must be your engine's own linear damping value, unconverted.

`CaptureSnapshot` and `ApplySnapshot` have default implementations built from the properties above. Override `ApplySnapshot` only if your engine needs extra bookkeeping when `Position`, `Rotation`, `LinearVelocity` and `AngularVelocity` are written.

The smallest working adapter is plain fields behind the interface, with the default snapshot methods and no sweep provider.

Implement ISweepProvider for swept targets

This is optional. Implement it only if your engine can sweep a shape along a segment against static

geometry:

```
bool TrySweep(IPhysicsBody physicsBody, Vector3 fromPosition, Vector3 toPosition, out Swee
pHit sweepHit);
```

Return `true` when the sweep hit blocking geometry, and fill `sweepHit` with:

- `Position`: the body-center position at which the sweep was blocked.
- `Normal`: the surface normal at the contact.
- `Fraction`: how far along the sweep the contact occurred, from zero to one.

Without an `ISweepProvider`, swept projection targets are unavailable and only the plain projection is used.

Use Unity's adapters as a reference

`UnityPhysicsBody` wraps a `Rigidbody`: `Gravity` returns `Vector3.Zero` when `Rigidbody.useGravity` is off, `Drag` maps to `Rigidbody.linearDamping`, and velocity writes are skipped on a kinematic body. Unity's sweep adapter is `UnitySweepProvider.Shared`, a static instance you can reference directly.

How physics replication works

The controller captures and every other peer follows

The peer that controls a body simulates it normally and, once per tick, captures its world position, world rotation and condition flags into ordinary members of a NetworkPhysicsComponent, along with the tick of the capture. A resting body sends no new updates.

Every other peer keeps simulating its own copy of the body through the same physics engine. Each physics step, a PhysicsConvergence steers that copy toward the server's state. Because every peer really simulates the body, contacts, stacks and collisions stay real everywhere.

Position and rotation use Divine or Interval transmission like any other member. Divine is Pro: a free build accepts `Divine` and replicates the member as `Interval` does.

The core never sees an engine type

The replicated members and snapshots use `System.Numerics` `Vector3` and `Quaternion`, so the core depends on no physics engine. An integration reads its engine's body into that shape and writes the result back; see Adapting your engine. Nucleus ships a Unity integration that does this for Unity's physics.

Some things are not built in

- **Replay-correct collision callbacks** are not provided. A converged body is never rewound. An integration can still step its physics world on each replayed tick through `SystemManager.ReplayEarlyFixedUpdated`; the Unity integration does, and rewinds only the bodies its validated behaviours predict (see Predicting a body with a validated behaviour).
- **Judging a shot against where targets were** is lag compensation (Pro), which keeps its own hitbox history and does not touch physics bodies.
- **Catching a freshly spawned projectile up through the latency it lost** is spawn compensation (Pro).

Physics Synchronization

Pages in this category:

- NetworkPhysicsComponent
- PhysicsConvergence
- How physics replication works
- Replicate a physics body
- Adapting your engine
- Trajectory projection
- Locally predicted bodies
- Swept projection targets

Locally predicted bodies

Using Unity? See Driving a predicted body.

Set `IsLocallyPredicted` on the `PhysicsConvergence` of a body this peer drives. The body then responds to local client state instantly and only trims the gap that opens against the server's snapshot, instead of trailing the server like a proxy.

What IsLocallyPredicted changes

`PhysicsConvergence.Step` still sets `physicsBody.LinearVelocity` from the projected snapshot every step, as for a proxy, so a velocity your code set before `Step` does not survive it. Three things differ:

	Proxy	Predicted body
Projection cap	`Settings.MaximumProjectionSeconds`	`PredictionMaximumProjectionSeconds`, larger by default; must cover a full round trip.
Teleport past `Settings.TeleportDistance`	Yes	Never. Any gap closes smoothly.
Correction	Past `Settings.PositionThreshold`, blends at `BlendPerTick` per `ConvergenceBlendMode`.	Only past `Settings.PositionThreshold` or `Settings.RotationThreshold`, closed through the body's motion at `PredictionCorrectionRate`.

Tune the correction with PredictionCorrectionRate

`PredictionCorrectionRate` (default `0.1f`) is the fraction of the position or rotation gap removed per tick once past threshold. A body that stays inside the thresholds is never nudged. Higher values pull the body back to the server harder and sooner; lower values correct more gently.

`0` disables correction. Since a predicted body never teleports either, nothing then pulls it back onto the server. Use `0` only to isolate prediction drift while debugging.

Cover the round trip with PredictionMaximumProjectionSeconds

`PredictionMaximumProjectionSeconds` (default `1f`) must be at least the longest round trip you expect. If it is shorter, the target lands short of where the body was predicted to be, and the correction drags the body backward.

Base ticksToProject on the round trip

For a predicted body, compute the `ticksToProject` argument of `Step` from `Connection.RoundTripTimeMilliseconds`, that peer's current round-trip estimate, rather than a fixed tick count. Projecting less than a full round trip makes the body respond less instantly to its own client state.

```
public uint RoundTripTimeMilliseconds { get; private set; }
```

Clear IsLocallyPredicted when control leaves

Set `IsLocallyPredicted` back to `false` as soon as control leaves the body. Otherwise it keeps skipping the teleport and proxy blend it needs once it follows the server again. `PhysicsConvergence.Reset()`

also clears it.

Client state, reconciling against confirmed server state and the replay window are covered on the control and prediction pages.

NetworkPhysicsComponent

`NetworkPhysicsComponent` replicates a physics body's position, rotation and condition flags through ordinary members. The controlling peer captures its body into the members each tick, and every other peer rebuilds a `PhysicsSnapshot` from them to follow. See Replicate a physics body for the full setup.

The component replicates four members

Member	Type	Settings
`WorldPosition`	`NetworkMember<Vector3>`	`CompressionLevel.Aggressive`, `TransmissionModeDefaults.Motion`
`WorldRotation`	`NetworkMember<Quaternion>`	`TransmissionModeDefaults.Motion`
`BodyFlags`	`NetworkMember<int>`	default; holds a `PhysicsBodyFlags` value
`CaptureTick`	`NetworkMember<int>`	default

`CaptureTick` is the tick of the capture, on the capturing peer's own tick counter: the server's for a server-controlled body, the controlling client's for a client-controlled one. It holds `NetworkLoopManager.UnsetTick` until a ticked capture runs.

BodyFlags holds PhysicsBodyFlags

Flag	Value	Meaning
`None`	`0`	No condition bits are set.
`Sleeping`	`1 << 0`	The body is sleeping.
`Kinematic`	`1 << 1`	The body is kinematic and does not respond to forces.
`Captured`	`1 << 4`	The flags were written by a real body capture.

Read the flags through properties

Property	Reads
`IsBodySleeping`	`BodyFlags` has `PhysicsBodyFlags.Sleeping` set.
`IsBodyKinematic`	`BodyFlags` has `PhysicsBodyFlags.Kinematic` set.
`HasReceivedState`	`BodyFlags` has `PhysicsBodyFlags.Captured` set.
`HasAdoptedState`	True once `TryAdoptInitialState` has placed a proxy body onto the first received state.

Use `HasReceivedState` to tell whether real state has arrived. Do not check the rotation: an unset rotation reads as identity, which looks the same as a captured identity rotation.

Capture, rebuild and adopt the body

```
public void CaptureBody(IPhysicsBody physicsBody, uint tick = Managers.NetworkLoop.Network
LoopManager.UnsetTick)
```

The controlling peer (the server, unless a client controls the body) calls this once per tick, before the tick's state is sent. It writes the body's state into the replicated members. A resting body sends no updates.

```
public PhysicsSnapshot BuildSnapshot()
```

Proxies call this to rebuild the latest server snapshot. A sleeping body, or one that has just appeared, has zero velocity in the snapshot.

```
public bool TryAdoptInitialState(IPhysicsBody physicsBody)
```

Places the body directly onto the first received state, once, so a newly spawned proxy starts at its real position instead of gliding there from the prefab pose. Call it each proxy step before convergence, and skip convergence on the step it returns `true`. It returns `false` until `HasReceivedState` is true, and after `HasAdoptedState` is true.

PhysicsSnapshot carries one body state

Remote state, local body state and projected targets all use this type.

Field	Type	Meaning
`WorldPosition`	`Vector3`	The world-space position of the body.
`WorldRotation`	`Quaternion`	The world-space rotation of the body.
`LinearVelocity`	`Vector3`	The linear velocity of the body.
`AngularVelocity`	`Vector3`	The angular velocity of the body, as an axis scaled by radians per second.
`IsSleeping`	`bool`	True when the body is sleeping; a sleeping snapshot projects to itself.

PhysicsConvergence

Using Unity? See NetworkRigidbody.

`PhysicsConvergence` follows the latest server snapshot for one proxy body, projected to the present, and blends or teleports a diverged pose back onto it.

Call Step before each physics step

```
public void Step(IPhysicsBody physicsBody, in PhysicsSnapshot remotePhysicsSnapshot, float
 tickDelta, float ticksToProject)
```

Call `Step` once per physics step for each proxy body, before your engine simulates.

- `remotePhysicsSnapshot`: the last snapshot received from the server.
- `tickDelta`: the duration of one tick.
- `ticksToProject`: how many ticks that snapshot lags local time.

`Step` writes the body's `LinearVelocity` and `AngularVelocity`, and on a blend or teleport its `Position` and `Rotation`. If `physicsBody.IsKinematic`, it returns without touching the body.

Tune it with PhysicsConvergenceSettings

`Settings` holds the tuning. `new PhysicsConvergenceSettings()` gives the recommended defaults below.

Field	Default	Meaning
`PositionThreshold`	`0.1`	Positional divergence in metres past which the body blends back toward the target.
`RotationThreshold`	`5`	Rotational divergence in degrees past which the body blends back toward the target.
`BlendPerTick`	`0.1`	Fraction of the remaining gap closed per tick while over threshold, from 0 (never) to 1 (onto the target in one tick).
`ResidualCloseRate`	`0.5`	How quickly, per second, an offset under `PositionThreshold` closes on a slow body, through its motion. Zero leaves the offset in place.
`ResidualCloseMaximumSpeed`	`0.5`	Body speed in metres per second above which an offset under threshold is left alone, so a bumped or moving body is not pulled against its motion.

`TeleportDistance`	`4`	Positional divergence in metres past which the whole pose snaps onto the target. Zero disables the teleport.
`MaximumProjectionSeconds`	`0.2`	Cap, in seconds, on how far a stale snapshot is projected before the blend takes over.
`GravityProjection`	`GravityProjection.Auto`	Whether gravity participates in the projection. See [Trajectory projection](./trajectory-projection.md).
`BlendMode`	`ConvergenceBlendMode.Position`	How an over-threshold divergence is closed.

Each step, `Step` measures the divergence and responds in one of three ways:

- **Over `TeleportDistance`:** position and rotation snap onto the target, and `HasTeleportedThisStep` is set. A locally predicted body never teleports.
- **Over a threshold:** position and rotation blend toward the target at `BlendPerTick`, per `BlendMode`.
- **Under threshold:** position is left alone, except that a body slower than `ResidualCloseMaximumSpeed` closes the offset through its motion at `ResidualCloseRate`.

Choose how a divergence closes with ConvergenceBlendMode

- `Position` moves the body straight toward the target each step. It can briefly push the body into geometry before it is pushed back out.
- `Velocity` closes the same fraction of the gap through the body's motion. It never disrupts contacts, so it never yanks a body out of a stack or through what it rests on.

Both close the gap at the same rate in free space.

Read the last step and reset

- `TargetError`: the positional error from the most recent step (target position minus body position).
- `HasTeleportedThisStep`: true when the most recent step snapped the pose. Use it to collapse a smoothed visual's interpolation so the jump renders instantly.
- `Reset()`: clears `TargetError`, `HasTeleportedThisStep` and `IsLocallyPredicted`, for pool reuse or respawn.

Replace the projection with TargetProvider

By default `Step` projects the snapshot along a plain ballistic path. Set `TargetProvider` to an `IProjectionTargetProvider` to compute the target another way, for example by sweeping against geometry. See Swept projection targets.

Predicted bodies use their own settings

`IsLocallyPredicted`, `PredictionMaximumProjectionSeconds` and `PredictionCorrectionRate` apply only to a body this peer drives with its own client state. See Locally predicted bodies.

Replicate a physics body

Using Unity? See Replicate a rigidbody.

The server captures its body into a `NetworkPhysicsComponent` after each world step. Every other peer rebuilds a snapshot from the component and steps a `PhysicsConvergence` toward it before its world step. Everything here is plain C# against `IPhysicsBody`; see Adapting your engine to wrap a real engine's body.

Get the component

On the server, rent a system with the component:

```
if (!NetworkSystemPool.TryRent<NetworkSystem, NetworkPhysicsComponent>(coreManager, out Ne
tworkSystem? networkSystem, canStartSystem: true))
    return;

networkSystem.TryGetComponent(out NetworkPhysicsComponent physicsComponent);
```

On a receiving peer, the system arrives over the network with nothing to look up by name. Declare a watch, and the pool hands you every system that carries a `NetworkPhysicsComponent` when it arrives:

```
NetworkSystemPool.Watch<NetworkSystem, NetworkPhysicsComponent>(
    coreManager,
    networkSystemAcquiredHandler: system =>
    {
        system.TryGetComponent(out NetworkPhysicsComponent physicsComponent);
        // physicsComponent is fully initialized here.
    });
```

Capture on the server after the world step

```
physicsComponent.CaptureBody(physicsBody, tick: currentTick);
```

This reads `Position`, `Rotation` and the sleeping and kinematic flags off `physicsBody`. Pass `tick` so receivers know the snapshot's exact age; without it, no capture tick is recorded.

Converge on every other peer before the world step

```
PhysicsSnapshot snapshot = physicsComponent.BuildSnapshot();
float ticksToProject = GetTicksToProject(physicsComponent);

convergence.Step(physicsBody, in snapshot, tickDelta, ticksToProject);
```

`ticksToProject` is the snapshot's age in ticks. Peers do not share a tick counter, so do not subtract `CaptureTick` from this peer's tick. Measure the age in the server's numbering with `Connection.TryGetServerTickEstimate` on the connection to the server:

```
private const float FallbackTicksToProject = 2f;

private float GetTicksToProject(NetworkPhysicsComponent physicsComponent)
{
    uint captureTick = (uint)physicsComponent.CaptureTick.Value;

    if (captureTick != NetworkLoopManager.UnsetTick
        && coreManager.TransportManager.TryGetServerConnection(out Connection serverConnec
tion)
        && serverConnection.TryGetServerTickEstimate(coreManager.NetworkLoopManager.Tick,
out uint serverTick)
        && serverTick > captureTick)
        return serverTick - captureTick;

    // No attributed capture or no estimate yet: project a short fixed distance.
    return FallbackTicksToProject;
}
```

This measures the age of a body the server captures, which is the setup on this page.

Adopt the first received state

A newly observed proxy sits at the pose it was created with, and converging from there would glide it visibly. Call `TryAdoptInitialState` before `Step` on every follower step, and skip `Step` when it returns `true`. It succeeds only once, after the first capture arrives.

Run both halves in the network loop

Subscribe to `EarlyFixedUpdate`, which runs before your world step, and `LateFixedUpdate`, which runs after it:

```
public class PhysicsDriver : INetworkLoopStepCallback
{
    public NetworkLoopSteps GetNetworkLoopSteps() => NetworkLoopSteps.EarlyFixedUpdate | N
etworkLoopSteps.LateFixedUpdate;

    public void OnNetworkLoopStep(NetworkLoopSteps networkLoopStep, StepDelta stepDelta)
    {
        if (networkLoopStep is NetworkLoopSteps.EarlyFixedUpdate)
        {
            // Follower: converge before the world steps, so the step integrates the corre
cted velocity.
            if (!isAuthority && !physicsComponent.TryAdoptInitialState(physicsBody))
            {
                PhysicsSnapshot snapshot = physicsComponent.BuildSnapshot();
                float ticksToProject = GetTicksToProject(physicsComponent);
                convergence.Step(physicsBody, in snapshot, tickDelta, ticksToProject);
            }

            WorldStep(); // your simulation
        }
        else if (networkLoopStep is NetworkLoopSteps.LateFixedUpdate)
        {
            // Authority: capture after the world has stepped, so the capture describes th
is tick's result.
            if (isAuthority)
                physicsComponent.CaptureBody(physicsBody, tick: currentTick);
        }
    }
}
```

Swept projection targets

Replace the projected target with IProjectionTargetProvider

By default `PhysicsConvergence.Step` projects the latest server snapshot forward with `TrajectoryProjector`, which ignores any geometry in the way. Set `PhysicsConvergence.TargetProvider` to an `IProjectionTargetProvider` and the follower steers onto whatever target it returns instead:

```
public interface IProjectionTargetProvider
{
    void ComputeTarget(IPhysicsBody physicsBody, in PhysicsSnapshot remotePhysicsSnapshot,
 float tickDelta,
        float ticksToProject, GravityProjection gravityProjection, out PhysicsSnapshot tar
getPhysicsSnapshot);
}
```

`TargetProvider` is `null` by default in the core API. In Unity, `NetworkRigidbody` always installs `SweptTargetProvider(UnitySweepProvider.Shared)` on every body it binds.

Stop a projected body at geometry with SweptTargetProvider

`SweptTargetProvider` projects the snapshot the ordinary way, then sweeps the body's shape from the old position to the projected one using an `ISweepProvider`. On a blocking hit, the target stops at the hit position, so it never lands past geometry in the way.

```
public sealed class SweptTargetProvider : IProjectionTargetProvider
{
    public float MinimumSweepDistance = 0.05f;
    public bool ReflectEnabled;
    public float Restitution = 0.6f;

    public SweptTargetProvider(ISweepProvider sweepProvider) { ... }
}
```

- **MinimumSweepDistance** (`0.05f`): projections shorter than this skip the sweep.
- **ReflectEnabled** (off): reflects the velocity at a stopped target about the contact normal, scaled by `Restitution`. Accurate only for simple bouncy shapes.
- **Restitution** (`0.6f`): the bounciness used when `ReflectEnabled` is on.

Install it on the follower:

```
convergence.TargetProvider = new SweptTargetProvider(mySweepProvider);
```

The sweep provider comes from your engine integration (see Adapting your engine). In Unity it is `UnitySweepProvider`, built on `Rigidbody.SweepTest`.

Each sweep is an extra collision query per physics step. Turn it on when a projected proxy can be carried through something it should hit: fast bodies, thin walls or floors, or any proxy that visibly tunnels through a surface. A slow body in open space gains nothing.

Write your own provider

Implement `IProjectionTargetProvider` directly when stop-at-first-contact is not what you want, for example to ignore certain layers. The provider must return a valid position and rotation, and the linear and

angular velocity that `PhysicsConvergence.Step` assigns straight onto the body.

Trajectory projection

Project a body forward with TrajectoryProjector

`TrajectoryProjector` is a static, engine-independent class that projects a body's position, velocity and rotation forward by any number of ticks in one call. The result equals stepping the body that many ticks with Euler integration, gravity and linear drag.

`TrajectoryProjector` does not cap `ticksAhead`. `PhysicsConvergence.Step` clamps its own `ticksToProject` to `Settings.MaximumProjectionSeconds`, or `PredictionMaximumProjectionSeconds` for a locally predicted body, before projecting. Call `TrajectoryProjector` directly with any `ticksAhead` you need, for example to lead a freshly spawned body or to show where a body will be.

```
public static Vector3 ProjectPosition(float tickDelta, float ticksAhead, Vector3 position,
 Vector3 velocity, Vector3 gravity, float drag)
```

Projects a position. `ticksAhead` can be fractional. A `drag` of zero or below projects without drag. Throws `ArgumentOutOfRangeException` if `ticksAhead` is negative.

```
public static Vector3 ProjectVelocity(float tickDelta, float ticksAhead, Vector3 velocity,
 Vector3 gravity, float drag)
```

Projects a linear velocity under the same gravity and drag, with the same negative-`ticksAhead` check.

```
public static Quaternion ProjectRotation(float tickDelta, float ticksAhead, Quaternion rot
ation, Vector3 angularVelocity)
```

Projects a rotation along a constant angular velocity, given as an axis scaled by radians per second. A zero angular velocity returns the input unchanged.

```
public static PhysicsSnapshot ProjectSnapshot(in PhysicsSnapshot physicsSnapshot, float ti
ckDelta, float ticksAhead, Vector3 gravity, float drag, GravityProjection gravityProjectio
n)
```

Projects a whole `PhysicsSnapshot`, the same as the three methods above with gravity resolved through `gravityProjection`. A sleeping snapshot is returned untouched.

```
public static Vector3 ExtractRotationVector(Quaternion fromRotation, Quaternion toRotation
)
```

The inverse of `ProjectRotation`: the shortest-arc rotation between two orientations, as a world-space axis-angle vector in radians. Divide it by the seconds it spans to get an angular velocity for `ProjectRotation`, or multiply it by a per-second gain to get a correcting angular velocity. A near-identity arc returns `Vector3.Zero`.

Choose a GravityProjection

```
public enum GravityProjection : byte
{
    Apply = 0,
    None = 1,
    Auto = 2,
}
```

- **Apply**: gravity is always applied.

- **None**: gravity is never applied.
- **Auto**: gravity is applied only when the snapshot is already descending fast enough to be a real fall. A slow or rising snapshot projects without gravity, so a resting body is not projected into the ground. It looks only at the snapshot's descent speed, not at any grounded flag or collision state.

On Unity's `NetworkRigidbody`, **Gravity Enabled** maps to `Auto` when checked and `None` when unchecked, and **Maximum Projection Time** sets `Settings.MaximumProjectionSeconds`.

The content bundle protocol

Using Unity? See Content bundles in Unity.

A content bundle is a `ushort` id for content that is not in the base build, such as prefabs a client may not have installed yet. Nucleus carries the id and the load protocol between peers; your loader does the loading. `Connection.BaseBuildBundleId` (0) is the content shipped in the build and is always loaded.

Register a loader

Implement `IBundleLoader`:

```
public interface IBundleLoader
{
    Task<bool> LoadBundleAsync(ushort bundleId);
    Task<bool> UnloadBundleAsync(ushort bundleId);
}
```

- `LoadBundleAsync` must register the bundle's prefabs before its task completes, because completion tells the server it may spawn from the bundle at once.
- `UnloadBundleAsync` releases the bundle.

Both may complete on any thread. Mapping a bundle id to an asset bundle, an Addressables entry or a download is up to you.

Each peer has one loader:

```
coreManager.BundleManager.SetBundleLoader(myLoader);
```

Gate spawns on content

`NetworkSystem.RequiredBundleId` is the bundle a spawn needs. It defaults to the bundle the prefab was built into (`PrefabBundleId`). Override it with `SetRequiredBundleId(ushort bundleId)`, and clear the override with `UnsetRequiredBundleId` (`ushort.MaxValue`).

`BundleManager.SpawnGate`, a `BundleInterestCondition`, withholds a spawn from a client that does not yet hold `RequiredBundleId`. The object spawns normally once the client confirms the bundle. Without the gate, a spawn for content a client lacks arrives anyway and fails at instantiation.

The gate is not registered for you. It is stateless, so add it globally or to any number of systems:

```
// Globally, for a world where most content hotloads:
coreManager.InterestManager.AddCondition(coreManager.BundleManager.SpawnGate);

// Or only on the prefabs that actually ship in a bundle:
networkSystem.AddInterestCondition(coreManager.BundleManager.SpawnGate);
```

Get the bundle loaded

With `BundleManager.AutomaticRequestOnBlockedSpawnEnabled` (default `true`), a spawn the gate blocks also asks that client to load the bundle, and the object appears once the client confirms. Set it to `false` to drive every load from your code; the gate still withholds spawns.

To ask a client yourself, on the server:

```
coreManager.BundleManager.RequestBundleLoad(connection, bundleId);
```

A repeat while a request for that client and bundle is outstanding is dropped.

To load or release a bundle on this peer itself, for example a server loading a zone before spawning from it, or a client preloading:

```
bool loaded = await coreManager.BundleManager.LoadBundleAsync(bundleId);
bool unloaded = await coreManager.BundleManager.UnloadBundleAsync(bundleId);
```

This runs through your `IBundleLoader`. On a client it is only a preload: the load is reported only if the server is already waiting on that bundle. Otherwise the server still asks when a spawn needs the bundle, and records the client's answer to that request. A bundle-backed scene load uses this call too.

Report outcomes from your own loading code

`LoadBundleAsync` and `UnloadBundleAsync` report to the server for you. Call `BundleManager.NotifyBundleLoaded(ushort bundleId)` and `NotifyBundleUnloaded(ushort bundleId)` only if your game loads outside an `IBundleLoader`. A load report is sent only in answer to a server request, and an unload report only for a bundle the server records the client holding.

On the server, `Connection.IsBundleLoaded(ushort bundleId)` reports whether a client holds a bundle.

Watch the protocol

```
coreManager.BundleManager.BundleLoadRequested += OnBundleLoadRequested;      // client: se
rver asked for a bundle
coreManager.BundleManager.BundleLoadStateChanged += OnBundleLoadStateChanged; // server: a
 client's bundles changed
```

Or implement `IBundleObserver` and register it with `RegisterBundleObserver`:

```
public interface IBundleObserver
{
    void OnBundleRequested(Connection connection, ushort bundleId);
    void OnBundleLoadStateChanged(Connection connection, ushort bundleId, bool isLoaded);
}
```

`OnBundleLoadStateChanged` fires only on an actual change, after the client's held bundles and interest are updated, so `Connection.IsBundleLoaded` inside it returns the new state. It mostly matters on the server, which tracks what each client holds.

Two violations catch a client acting on its own

- `UnsolicitedBundleReportViolation`: a load report for a bundle the server never asked for, or an unload report for one the server does not record the client holding. This catches a spoofed load report.
- `UnexpectedBundleRequestViolation`: a bundle load request, which only the server sends, arriving at the server.

Both default to a kick.

Most of the bundle API is Pro

`BundleManager`, `IBundleLoader`, `IBundleObserver`, `BundleInterestCondition` and `Connection.IsBundleLoaded` are Pro. In Free, `CoreManager` has no `BundleManager` member. The two violation types, `Connection.BaseBuildBundleId`, and `NetworkSystem.RequiredBundleId` with `SetRequiredBundleId` work in both editions.

Networked Scenes, Content Bundles and Persistence

Pages in this category:

- How networked scenes work
- Opening and closing scene instances
- Placing clients in scenes
- Registering a scene loader
- Choosing what a save keeps
- Scene events and observers
- Moving and carrying objects between scenes
- Refusing a scene load
- Saving and loading a world
- Scene visibility: what a client in a scene can see
- The content bundle protocol
- Scenes and bundles: things that look broken
- Writing a world store

Moving and carrying objects between scenes

Move a system to another scene

`SystemManager.EnsureMoveSystemToScene(NetworkSystem networkSystem, uint destinationSceneHandle)` moves a spawned object to another scene instance. It is server-only, and returns `false` without moving anything when:

- the system is not started and registered;
- it is a scene object or has no platform identity (only a dynamically spawned object can move);
- `destinationSceneHandle` is `NetworkSystem.UnsetSceneHandle`.

```
coreManager.SystemManager.EnsureMoveSystemToScene(asteroid, destinationSceneHandle);
```

A system in a group moves with its whole group. The move also asks the object's controlling client into the destination scene, so the client follows the object instead of losing it.

Read a system's scene

`NetworkSystem.SceneHandle` is the instance the system belongs to. `NetworkSystem.UnsetSceneHandle` (zero) means the boot scene every peer already holds.

`NetworkSystem.SceneChanged` fires when the handle changes: on the server as it moves the system, and on a client as it applies the move.

```
networkSystem.SceneChanged += (previousSceneHandle, currentSceneHandle) =>
{
    // previousSceneHandle: the scene the system was in before the move.
    // currentSceneHandle: the scene it is in now.
};
```

A refusal is not a carry failure

When the controlling client cannot follow the object, one of two things happened:

- **Carry failure**: the client's load of the destination scene failed. The object has lost the controller meant to follow it, and `SceneCarryFailureAction` decides what happens next.
- **Refusal**: the client's loader will not hold another scene at once. Place the client with `SceneReplaceMode.AllScenes` instead of loading the scene additively. The object is unaffected: it is not despawned or released from control.

Decide what happens on a carry failure

`SceneCarryFailureAction` values:

- `SurrenderControl` (default): return the object to server control and leave it where it is, live for whoever is in the scene.
- `Despawn`: despawn the object, and its whole group. Usually right for a player's avatar.
- `Retain`: change nothing, controller included. Your game resolves it, typically by moving the object back and asking the client into that scene.

The action is resolved once per carried object. Each layer below outranks the one before it:

`SceneManager.SceneCarryFailure`: the world-wide default, `SceneCarryFailureAction.SurrenderControl`.

`NetworkSystem.SceneCarryFailureOverride`: a per-object override. Null uses the manager's default.

A registered `ISceneCarryFailureResolver`: decides case by case.

```
public SceneCarryFailureAction? SceneCarryFailureOverride { get; set; }
```

Resolve case by case

```
public interface ISceneCarryFailureResolver
{
    SceneCarryFailureAction ResolveSceneCarryFailure(
        Connection connection,
        NetworkSystem networkSystem,
        uint sceneHandle,
        SceneCarryFailureAction defaultSceneCarryFailureAction);
}
```

`defaultSceneCarryFailureAction` is the answer without this resolver: the object's override, or the manager's default. Return it unchanged to leave a case alone. Several resolvers can be registered, and each receives the answer so far.

```
coreManager.SceneManager.RegisterSceneCarryFailureResolver(resolver);
coreManager.SceneManager.TryUnregisterSceneCarryFailureResolver(resolver);
```

In Unity, only root GameObjects follow a move

A root GameObject bound to its scene follows the move. A parented GameObject is refused and stays in its parent's scene. An object whose scene binding is not yet resolved waits until it resolves.

Opening and closing scene instances

Open an instance

`SceneManager.TryOpenScene(ushort sceneId, SceneScope sceneScope, out uint sceneHandle, SceneReplacePolicy sceneReplacePolicy = SceneReplacePolicy.Released)` opens a live instance of a scene on the server and returns its handle. `OpenSceneAsync(ushort sceneId, SceneScope sceneScope, SceneReplacePolicy sceneReplacePolicy = SceneReplacePolicy.Released)` does the same and completes once the scene has loaded.

```
if (sceneManager.TryOpenScene(myDungeonSceneId, SceneScope.Connections, out uint sceneHand
le))
{
    // sceneHandle identifies this instance.
}

uint sceneHandle = await sceneManager.OpenSceneAsync(myLobbySceneId, SceneScope.Global);
```

Every open makes a new, independent instance, even for a scene id already open. Handles are never reused.

Choose a scope

`SceneScope` cannot change after opening:

- `SceneScope.Connections`: opening places nobody. The game places clients later.
- `SceneScope.Global`: opening places every connected client, and a client that authenticates while the instance is open is placed as part of authentication.

`SceneManager.JoinPlacement` also places newly authenticated clients, and by default that includes `Connections` instances. See Placing clients in scenes.

Choose a replace policy

`SceneReplacePolicy`, chosen at open time, decides whether a client keeps this instance when a different placement later replaces what it holds:

- `SceneReplacePolicy.Released` (default): the replacing placement releases this instance along with everything else.
- `SceneReplacePolicy.Retained`: the client stays in this instance, and only the rest is released.

An explicit close or unload always takes effect, whatever the policy.

Read back what is open

- `OpenSceneHandles`: the handles of every open instance.
- `TryGetOpenScene(uint sceneHandle, out ushort sceneId)`: the scene id a handle belongs to.
- `TryGetSceneScope(uint sceneHandle, out SceneScope sceneScope)`: the scope it was opened with.
- `TryGetSceneReplacePolicy(uint sceneHandle, out SceneReplacePolicy sceneReplacePolicy)`: the replace policy it was opened with.

Close an instance

`EnsureCloseScene(uint sceneHandle)`, or the awaitable `CloseSceneAsync(uint sceneHandle)`, asks the occupants to leave and then closes the instance.

If an instance was unloaded without going through the manager, call `TryNotifySceneUnloadedUnexpectedly(uint sceneHandle)`. It despawns the instance's systems and removes its occupants, since nothing else will.

Placing clients in scenes

Using Unity? See UnitySceneManager component.

A client sees the objects in a scene instance only once it is placed in it. Only the server places clients.

Load a client into a scene

`TryRequestSceneLoad(Connection connection, uint sceneHandle, SceneReplaceMode sceneReplaceMode = SceneReplaceMode.None, uint systemId = NetworkSystem.UnsetId)` asks one client to load a scene instance. It returns `true` only when a request was sent: a scene the client already holds, or has already reported it cannot load, is not asked for again.

```
coreManager.SceneManager.TryRequestSceneLoad(connection, sceneHandle);
```

To place several clients, use `TryRequestSceneLoad(IReadOnlyList<Connection> connections, uint sceneHandle, SceneReplaceMode sceneReplaceMode = SceneReplaceMode.None)`. Each client is placed on its own, so one refused by a validator, already in the scene, or holding a recorded failure does not stop the rest. It returns how many requests were sent, which tells you when it did less than you asked.

```
uint requestedCount = coreManager.SceneManager.TryRequestSceneLoad(connections, sceneHandl
e, SceneReplaceMode.None);
```

Remove a client from a scene

`RequestSceneUnload(Connection connection, uint sceneHandle)` asks a client to release a scene instance it holds or is loading. The client stops receiving that scene's objects at once, not when it confirms.

The boot scene (`NetworkSystem.UnsetSceneHandle`) cannot be released this way.

Replace what a client holds

`SceneReplaceMode` sets what a load does to the scenes a client already holds:

- `None` (default): keeps everything and adds this scene.
- `AllScenes`: releases every server-opened scene the client holds or is loading, then loads this one. The target is validated first, so a refusal leaves the client where it was.
- `AllScenesAfterLoad`: loads this scene first, and releases the others only once the client confirms it and acknowledges the state sent with that confirmation. Use it when an object must move with the client: releasing first would destroy the client's copy of the old scene before the move reaches it.

Use `AllScenes` or `AllScenesAfterLoad` rather than finding and unloading every other scene yourself.

Choose where a new client lands

`SceneManager.JoinPlacement` (a `JoinScenePlacement`) decides what a client is placed into when it authenticates:

- `EveryOpenScene` (default): every open instance, whatever its scope. A `SceneScope.Connections` arena still gets every new client.
- `GlobalScenes`: only `SceneScope.Global` instances. Use this once your world uses private instances.
- `None`: nothing. The game places every client with `TryRequestSceneLoad`.

Check who is in a scene

- `Connection.IsSceneLoaded(uint sceneHandle)`: whether one client has confirmed holding the scene. Always `true` for `NetworkSystem.UnsetSceneHandle`.
- `SceneManager.CollectConnectionsInScene(uint sceneHandle, List<Connection> connections)`: adds every client that has confirmed the load to the list. Clients still loading, or whose load failed, are not added. For `NetworkSystem.UnsetSceneHandle` it adds every connected client.

Time out unanswered loads

`SceneManager.LoadRequestTimeoutSeconds` (default 180) is how long the server waits for a client to answer a load request. It catches a loader that never finishes: a stuck fetch, a hung load, or a client that stopped answering. Zero or less waits forever.

When it expires, the request is cancelled and recorded as a failure, the same as a failure the client reports. A late answer is ignored.

Leave automatic placement off

`SceneManager.AutomaticRequestOnBlockedSpawnEnabled` (default `false`): when `true`, a client is asked into a scene as soon as a spawn is withheld because the client is not in it. That pulls every client into every open scene, so leave it off unless every client belongs in every scene, and place clients with `TryRequestSceneLoad`. The bundle manager's flag of the same name defaults to `true`.

Scene events and observers

SceneManager raises seven events

Event	Delegate	Raised on	When
`SceneLoadRequested`	`SceneLoadRequestedHandler(uint sceneHandle, ushort sceneId, bool isLoadRequested)`	client	The server asks the client to load or release a scene, before the loader is called.
`SceneLoadStateChanged`	`SceneLoadStateChangedHandler(Connection connection, uint sceneHandle, bool isLoaded)`	server	A client's answer changes which scenes it holds.
`SceneLoaded`	`SceneLoadedHandler(uint sceneHandle, ushort sceneId)`	client	The client's `ISceneLoader` finishes a load the server asked for, after the outcome is reported. Not raised for the server's own copy, a host included: await `OpenSceneAsync` there, which returns the handle, or `NetworkSystem.UnsetSceneHandle` if the scene could not load.
`SceneUnloaded`	`SceneUnloadedHandler(uint sceneHandle, ushort sceneId)`	client	The client's `ISceneLoader` finishes a release the server asked for, after the report is sent. Not raised when the server closes its own copy: await `CloseSceneAsync` there.
`SceneLoadFailed`	`SceneLoadFailedHandler(uint sceneHandle, ushort sceneId)`	local peer	This peer's own load fails, after the failure is reported. Also raised on the server when `OpenSceneAsync` could not load the server's own copy.

`ClientSceneLoadFailed`	`ClientSceneLoadFailedHandler(Connection connection, uint sceneHandle)`	A client reports it could not load a scene, after the pending request is cancelled.
`SceneLoadProgressed`	`SceneLoadProgressedHandler(uint sceneHandle, ushort sceneId, float progress)`	This peer's own load advances, for a loading screen. Progress is never sent.

```
sceneManager.SceneLoadRequested += (sceneHandle, sceneId, isLoadRequested) =>
{
    // Show a loading screen, then load or release sceneId.
};

sceneManager.ClientSceneLoadFailed += (connection, sceneHandle) =>
{
    // Decide what happens to a client that could not fetch the scene.
};
```

`SceneLoadStateChanged` fires only when what a client holds actually changes, after interest has been updated.

Watch with an ISceneObserver

`ISceneObserver` receives the same occurrences as three of the events, without an event subscription per manager:

```
public interface ISceneObserver
{
    void OnSceneRequested(Connection connection, uint sceneHandle, bool isLoadRequested);
    void OnSceneLoadStateChanged(Connection connection, uint sceneHandle, bool isLoaded);
    void OnSceneLoadFailed(Connection connection, uint sceneHandle);
}
```

Register any number with `sceneManager.RegisterSceneObserver(observer)` and remove one with `TryUnregisterSceneObserver`. Registering the same observer twice is ignored; registering null logs an error. The events and the observer callbacks both fire for each occurrence.

Report from your own loading code

A registered `ISceneLoader` reports for you. If your game loads scenes itself, call these on the client:

- `NotifySceneLoaded(uint sceneHandle, uint systemId = NetworkSystem.UnsetId)`: the client now holds the instance.
- `NotifySceneUnloaded(uint sceneHandle)`: the client released the instance.
- `NotifySceneLoadFailed(uint sceneHandle, uint systemId = NetworkSystem.UnsetId)`: the client could not load the instance.
- `NotifySceneLoadProgress(uint sceneHandle, ushort sceneId, float progress)`: local progress only. Nothing is sent.

The three report methods are safe to call from any thread. Each is sent only in answer to a request the server made, so a load the server never asked for stays local.

Clear a stuck client

A reported failure is remembered, so the server stops asking a client that cannot fetch the content. Once the cause is fixed, clear it:

```
bool cleared = sceneManager.TryClearSceneLoadFailure(connection, sceneHandle);
```

`connection.HasSceneLoadFailed(sceneHandle)` tells you whether a failure is recorded.

A client reports one of four outcomes

`SceneLoadOutcome` for a scene the client was asked to load or release:

- **Loaded**: the client holds the instance and can be sent its objects.
- **Released** : the client released the instance.
- **Failed**: the load did not work. The content would not fetch, the scene would not resolve, or the load threw.
- **Refused**: the client's loader is already at `ISceneLoader.MaximumConcurrentScenes`. The loader was never called.

`Failed` and `Refused` both raise `ClientSceneLoadFailed` and `ISceneObserver.OnSceneLoadFailed`, and neither raises `SceneLoadStateChanged`. Only `Failed` triggers carry-failure handling for objects being carried into the scene (see Moving and carrying objects between scenes). Fix a refusal by moving the client with `SceneReplaceMode.AllScenes` instead of adding the scene.

Two violations catch a client acting on its own

- **UnsolicitedSceneReportViolation**: a client reports loading a scene it was never asked to load, or releasing one it was never asked to release. Carries `SceneHandle` and `IsLoadReported` (true for a claimed load, false for a claimed release).
- **UnexpectedSceneRequestViolation** : a client sends a scene load request, a message that only travels server to client. Carries `SceneHandle` and `IsLoadRequested`.

Both default to a kick.

Refusing a scene load

Implement and register a validator

`ISceneLoadValidator` lets the server refuse to place a client in a scene instance before the client is asked:

```
public interface ISceneLoadValidator
{
    bool CanLoadScene(Connection connection, uint sceneHandle, ushort sceneId);
}
```

- `connection`: the client the server is about to ask.
- `sceneHandle`: the live scene instance.
- `sceneId`: the scene asset that instance was opened from.

Return `true` to allow the request or `false` to refuse it. Only decide: do not open scenes, send messages or otherwise act inside it.

```
sceneManager.RegisterSceneLoadValidator(myValidator);

// later
sceneManager.TryUnregisterSceneLoadValidator(myValidator);
```

The first refusal wins

You can register any number. Before `TryRequestSceneLoad` asks a client into a scene, the server calls them in order and stops at the first that returns `false`. Validators after it are not called. With none registered, or none refusing, the request goes ahead.

The client is never told

A refused request is never sent, so the client cannot tell a refused scene from one that was never offered. That makes a validator the place for entitlement rules such as a party requirement, a ticket, a subscription or a ban.

Do not use a validator for capacity or visibility

- **Capacity**: how many scenes a peer can hold is `ISceneLoader.MaximumConcurrentScenes`. Do not count scenes in a validator.
- **Per-object visibility**: which objects in a scene a client sees is decided by interest. A validator only decides whether the client is placed in the scene at all.

Keep validators cheap

Every validator runs on every gated placement, for every client, until one refuses. Keep it to an in-memory check, or cache the result of any database or service lookup.

Registering a scene loader

Using Unity? See UnitySceneLoader component.

Nucleus carries only a scene id, never a path, build index or asset reference, and the core engine ships no `ISceneLoader`. Until you register one, scene requests do nothing. The Unity integration ships two, `NetworkSceneLoader` and `UnitySceneLoader`.

Implement ISceneLoader

```
public interface ISceneLoader
{
    uint MaximumConcurrentScenes { get; }

    Task<bool> LoadSceneAsync(uint sceneHandle, ushort sceneId);

    Task<bool> UnloadSceneAsync(uint sceneHandle);
}
```

- `LoadSceneAsync` loads one live instance of the scene `sceneId`, keyed under `sceneHandle`. Return `true` on success.
- `UnloadSceneAsync` releases the instance at `sceneHandle`. Return `true` on success.

Both may await anything, such as a download or an asset load, and may finish on any thread. The network loop never blocks on them.

How a scene id maps to content (a scene asset, an address, a bundle) is up to your loader.

Register it before any scene load

```
sceneManager.SetSceneLoader(mySceneLoader);
```

Each `SceneManager` has one loader. There is no default and no fallback.

Limit how many scenes a peer holds

`MaximumConcurrentScenes` is how many live scene instances this peer can hold at once. A loader that replaces the running world on each load can hold one; one that adds scenes alongside each other can hold any number. `SceneManager.UnlimitedConcurrentScenes` (0) means no limit.

A request over the limit never reaches `LoadSceneAsync`. It comes back as `SceneLoadOutcome.Refused` at once.

How networked scenes work

Three numbers identify a scene and its objects

Keep these apart. Mixing them up is the most common source of confusion.

- **Scene id** (`ushort`): which scene asset. The registered `ISceneLoader` turns an id into content on each peer. Nucleus never names a scene by a path or a name.
- **Scene handle** (`uint`): which live instance of that asset. Opening the same scene id twice gives two handles and two independent worlds.
- **Scene object id** (`uint`): which object inside a scene file. Editor tooling stamps it, and it is unique only within that one scene asset.

A scene object is named by its handle and its object id

`SceneObjectKey` pairs the scene handle with the scene object id. A `SceneObjectId` alone is ambiguous once two instances of the same scene asset are open, because both instances carry the same ids.

`NetworkSystem.SceneHandle` reports which instance an object belongs to. `NetworkSystem.UnsetSceneHandle` (zero) means it belongs to no server-opened scene: it is in the scene the peer booted into.

The server opens, places and closes

A client never decides. It loads a scene only when asked, releases one only when asked, and reports only what it was asked to report.

- Open instances: `SceneManager.TryOpenScene` and `OpenSceneAsync`.
- Place and remove clients: `TryRequestSceneLoad` and `RequestSceneUnload`.
- Close instances: `EnsureCloseScene` and `CloseSceneAsync`.

See Opening and closing scene instances and Placing clients in scenes.

Instances of one scene coexist

Opening a scene id again never reuses or merges state with an instance already open, and closing an instance removes only that one. Handles are never recycled, so a handle always names the same instance.

This is scene stacking: several instances of one asset open at once, sharing the same world coordinates and told apart only by handle. `UnitySceneLoader` loads each instance additively, so several copies of one Unity scene can be open in one process.

SceneScope sets who an instance is for

Scope is chosen when the instance opens and cannot change:

- `SceneScope.Connections`: nobody is placed until `TryRequestSceneLoad` asks. Use it for an arena, a dungeon or a match.
- `SceneScope.Global`: every connected client is placed at once, and every client that authenticates later is placed as it arrives. Use it for a single shared world or a lobby.

Scope is per instance, so one asset can run as a `Global` lobby while other instances run as private `Connections` matches.

Set it up in Unity

See UnitySceneLoader component and UnitySceneManager component.

Scene visibility:what a client in a scene can see

Clients see only the scenes they hold

`SceneManager.SpawnGate`, a `SceneInterestCondition`, is one of the engine's default interest conditions (listed in `InterestManager.DefaultConditions`), so it applies to every `NetworkSystem` with no setup. It withholds a system whose `SceneHandle` names a live scene instance from any client that does not hold that instance. A system with `NetworkSystem.UnsetSceneHandle` is seen by everyone, so a world that never opens a scene pays almost nothing for the gate.

To handle scene membership yourself, remove it with `InterestManager.RemoveDefaultConditions`.

The gate is also what keeps stacked scenes apart. Copies of one scene share the same coordinates, so without it a distance-based condition would match a player in one copy against objects in every other copy.

A leaving client is cut off at once

As soon as `SceneManager.RequestSceneUnload` asks a client to leave, that scene's objects stop being sent to it, even though `Connection.IsSceneLoaded` stays true until the client confirms.

A client's own object arrives as it enters

While a client's load of a scene is outstanding, the gate still lets through a spawned (not scene) object in that scene that the client controls, so the client does not wait for the load to see its own object. A client already holding the scene does not need this, and a client asked to leave never gets it.

A host's client is gated, a local peer is not

A host's own client is gated like any other connection. The result is reported through `NetworkSystem.HostInterestMembership` instead of acted on, so the host can hide an object its client would not have been sent without withholding anything from itself as the server.

The gate does not apply to a local peer: an emulated `Connection`, or a peer sharing an in-memory Transport (usually a test setup). A host's own client is not a local peer; it is the connection with `Connection.IsHostLoopback`.

Ask clients into scenes automatically

With `SceneManager.AutomaticRequestOnBlockedSpawnEnabled` set to `true` (default `false`), a withheld spawn also calls `SceneManager.TryRequestSceneLoad` for that client and scene, so the object spawns on the same tick the load is confirmed. Requests are deduplicated per client and scene, and stop once the client reports it cannot load the scene.

Find out why a client is missing an object

Check `NetworkSystem.SceneHandle` on the object. If it is not `NetworkSystem.UnsetSceneHandle`, the client must hold that exact instance.

Check `Connection.IsSceneLoaded(sceneHandle)` for the client. If it is false, the object is correctly withheld: place the client with `SceneManager.TryRequestSceneLoad`.

If the client was recently asked to leave the scene with `RequestSceneUnload`, everything in it is withheld at once. This is expected.

If it is the client's own controlled object arriving as the client enters the scene, confirm that the object is a spawned object rather than a scene object, that the scene load is outstanding for that client, and that the

client controls the object's group.

If none of the above applies, check whether `InterestManager.RemoveDefaultConditions` removed `SpawnGate`.

Scenes and bundles: things that look broken

A client connects and sees nothing

Check `SceneManager.JoinPlacement`. The default, `JoinScenePlacement.EveryOpenScene`, places a new client in every open scene with no `TryRequestSceneLoad` call. Under `JoinScenePlacement.None` nobody is placed automatically, and under `JoinScenePlacement.GlobalScenes` scenes opened `SceneScope.Connections` are skipped. A client that is never asked into a scene with `TryRequestSceneLoad` sees nothing in it.

Check when the scene opened. Join placement runs once, when the client authenticates, and covers only the scenes open then. A `SceneScope.Connections` scene opened later must be requested for clients already connected. A `SceneScope.Global` scene places every connected client when it opens.

Objects appear in the wrong copy of a stacked scene

Every copy of a stacked scene has the same name and path, so `GetSceneByName` and `GetSceneByPath` cannot tell them apart. Resolve the handle with `UnitySceneManager.TryGetScene`, and make that `Scene` active, or pass it to `MoveGameObjectToScene`, before spawning into it.

Handles are never reused, so the handle is not the cause. Look at the spawner: it instantiated while the wrong Unity scene was active, or moved the wrong copy.

A parented object shows the same symptom. Unity only moves root GameObjects between scenes, so an object with a parent is refused, stays in its parent's scene, and an error naming the parent is logged. Un-parent the object, or move its parent instead.

A load never completes

- **Server side**: `SceneManager.LoadRequestTimeoutSeconds` (default 180; zero or less waits forever) is how long the server waits for a client's answer. When it expires, the server logs a warning naming the scene handle and the timeout, treats the load as failed, and raises `ClientSceneLoadFailed`.
- **Client side**: a custom `ISceneLoader.LoadSceneAsync` must return `true` or `false`, and Nucleus reports the result for you. One that awaits something that never resolves hangs until the server's timeout. A game that loads scenes itself, without a registered loader, must report through `SceneManager.NotifySceneLoaded` or `NotifySceneLoadFailed`.

A load comes back as Refused rather than Failed

- `SceneLoadOutcome.Refused`: the client's `ISceneLoader.MaximumConcurrentScenes` was already reached, so the loader was never called. Asking again hits the same limit. Move the client with `SceneReplaceMode.AllScenes` so it releases what it holds first.
- `SceneLoadOutcome.Failed`: the loader was called and the load did not work. The content would not fetch, the scene would not resolve, or the load threw. Clear it with `SceneManager.TryClearSceneLoadFailure` and ask again.

A client is kicked during scene or bundle traffic

Four violations, all defaulting to a kick, catch a client trying to drive placement or content that only the server may drive. A well-behaved client cannot cause any of them, so one means a modified or malicious client, not a race.

- **`UnsolicitedSceneReportViolation`**: the client reported a scene load or release the server never asked for. Carries `SceneHandle` and `IsLoadReported` (true for a claimed load, false for a claimed release). A report about the unset scene handle counts too, because it is never requested.
- **`UnexpectedSceneRequestViolation`** : the client sent a scene load request, a message that only travels server to client. Carries `SceneHandle` and `IsLoadRequested`.
- **`UnsolicitedBundleReportViolation`**: the client reported a bundle load the server never asked for, or an unload of a bundle the server does not record it holding. Carries `BundleId` and `IsLoadReported`.
- **`UnexpectedBundleRequestViolation`** : the client sent the server a bundle load request, which only the server sends. Carries `BundleId`.

Prefabs silently never spawn for one client

`BundleInterestCondition` withholds a spawn from a client until it holds the object's bundle (`NetworkSystem.RequiredBundleId`). No error is logged; the object just does not exist there yet. It does not withhold scene objects, base-build content (`Connection.BaseBuildBundleId`), or a host's own local or loopback connection.

With `BundleManager.AutomaticRequestOnBlockedSpawnEnabled` on, the withheld spawn calls `BundleManager.RequestBundleLoad` for that client and bundle. With it off, nothing asks the client, and the spawn stays withheld until the client gets the bundle another way.

Check `Connection.IsBundleLoaded` for that client, and whether automatic requests are on.

A load brings a world back missing objects

The save is not corrupt. One of two lookups failed, and each failure is logged and skipped rather than aborting the load. Check the log for the scene id or system type it names:

- **A saved scene id the manifest no longer resolves.** `IWorldStore.OnSceneUnavailable` decides: drop that scene's objects and continue, or abandon the whole load (logged as "the world is empty rather than half built").
- **A saved object that no longer resolves.** A scene object whose `WorldSystemIdentity.PlatformId` is not found in the restored scene is dropped and logged as not found. A constructed object whose `WorldSystemIdentity.SystemTypeName` is no longer registered (`NetworkTypeRegistry.TryCreateSystemByName` fails) is dropped the same way.

A duplicated prefab or scene object keeps the original's id

In Unity, copying an object copies its stamped id, and two objects sharing an id collide.

Scene objects: saving the scene keeps the first occurrence of each `SceneObjectId` and gives any duplicate the next free id. Until you save, the duplicate keeps the copied id.

Prefabs: the id is stamped on the prefab asset, and saving does not change it. To renumber:

In the Nucleus window (**Tools > Nucleus**), select the **Build** tab and click **Rebuild** next to **Rebuild Network Prefab Collection**. The original keeps its id because the collection already lists it. The copy, or a prefab variant that inherited its base's id, gets the next free id.

Rebuild both the server and the client builds so every peer has the same ids.

A player build made before the rebuild fails and lists the clash. Registering two prefabs that share an id logs an error naming both.

Saving and loading a world

Using Unity? See Saving and loading a world from Unity.

World persistence is a Pro feature. `CoreManager.WorldPersistenceManager` saves the server's world to a store and builds it back.

Register a store

```
coreManager.WorldPersistenceManager.SetWorldStore(myWorldStore);
```

`SetWorldStore` replaces any store already registered, and `null` unregisters it. `SaveAsync` and `LoadAsync` fail at once when no store is registered. See Writing a world store.

Save and load only on the server

`SaveAsync` and `LoadAsync` are refused unless this peer's server is started. A client holds a copy of the server's world, not its own.

`IsBusy` is true while either runs. A second `SaveAsync` or `LoadAsync` called during one is refused.

Save the world

```
bool saved = await coreManager.WorldPersistenceManager.SaveAsync();
```

`SaveAsync` writes every open scene and every started system, then awaits the store's own save, which can take any number of ticks. A save always captures the whole world. There is no partial or incremental save.

Load without blocking the loop thread

```
Task<bool> loadTask = coreManager.WorldPersistenceManager.LoadAsync();

bool loaded = await loadTask;
```

`LoadAsync` finishes its work on the network loop thread. You can await it from any thread, the loop thread included, but:

- Never block the thread that drives the loop on the task (`.Result`, `.Wait()`).
- Never await it inside the code that would go on to drive the next tick.
- Keep driving the loop until the task completes.

Breaking any of these stops the loop, and the load deadlocks permanently.

Know what a save keeps

A save records each object, the type it was built from, its scene, what it belonged to, and its members' values.

It does not record observers, interest, access rosters, acknowledgment positions or controller assignments. These describe connected peers, and a saved world outlives its session.

Only systems whose type is registered with `NetworkTypeRegistry` are saved. A started system of an unregistered type is left out, and an error naming its id and type is logged; the log is the only record of what was skipped. The rest of the save goes ahead.

A load replaces the whole world

A load does not merge. It:

Despawns everything and closes every open scene. Connected clients see the despawns, then the spawns.

Reopens each saved scene. If one cannot be opened, the store's `OnSceneUnavailable` either drops that scene's objects and continues, or fails the whole load. A failed load leaves an empty world, because the old world is already gone.

Rebuilds the saved objects.

If no `ISceneLoader` is registered on `SceneManager`, a load logs a warning and drops every object in every saved scene, because none of those scenes can be opened.

Choosing what a save keeps

Keep a member out of saves with WorldStateIgnore

`WorldStateIgnoreAttribute` (`Nucleus.Managers.Persistence`) leaves a member out of a saved world without changing how it replicates. Put it on a `NetworkMemberBase` field to exclude that member, or on a `NetworkComponent` class to exclude every member it declares.

```
public partial class ScoreComponent : NetworkComponent
{
    public readonly NetworkMember<int> Persisted = new();

    [WorldStateIgnore]
    public readonly NetworkMember<int> Ignored = new();

    public readonly NetworkMember<int> AlsoPersisted = new();
}

[WorldStateIgnore]
public partial class SessionStatsComponent : NetworkComponent
{
    public readonly NetworkMember<int> Counter = new();
}
```

After a load, an excluded member holds the value its component was constructed with, not a saved value and not its value before the save. If a save from an earlier build still contains the member, that value is ignored.

The member replicates exactly as before, at no extra bandwidth.

The attribute compiles in Free but has no effect there, because saving worlds is Pro.

ReplicationIgnore is the opposite

`ReplicationIgnoreAttribute` (`Nucleus.Components`) keeps a member off the wire but still saves it. It takes the same two targets: a field for one member, the class for all of them.

Attribute	Opts out of	Member still replicates?	Member still saved?
`WorldStateIgnoreAttribute`	the save	yes	no
`ReplicationIgnoreAttribute`	the wire	no	yes, from whichever peer wrote it

A server promoted by adoption saves defaults

A peer promoted to server by adoption never received `ReplicationIgnore` values, so it holds the constructed defaults. Saving from it writes those defaults over what the store held. `WorldStateIgnore` members have no such problem, because no peer ever saves them.

Choose between them

- **A session counter or tick stamp**: observers should see it live, but it should not come back after a load. Use `WorldStateIgnore`.
- **A visual highlight or other presentation flag**: usually needs neither saving nor sending. Use either,

depending on whether other peers need to see it.

- **A cached local reference** (an object handle, a lookup recomputed on start): use `ReplicationIgnore` if it must never leave the peer that holds it, or `WorldStateIgnore` if it should replicate live but not survive a save.

`NetworkIgnore` is unrelated

`Nucleus.Serializers.NetworkIgnoreAttribute` leaves a field out when a struct or class of yours is sent. It has nothing to do with `NetworkComponent` members, replication or saving.

Writing a world store

`IWorldStore` decides where a saved world lives: a file, a database row, a save service or cloud storage. `JsonWorldStore` ships as the default and as a reference to read alongside your own.

Pro only: `IWorldStore`, `IWorldDocument`, `WorldSystemIdentity`, `WorldLoadFailureAction`, `JsonWorldStore` and `WorldDocument`. `IWorldStateWriter` and `IWorldStateReader` ship in both editions.

Register a store

The engine uses one store at a time:

```
CoreManager.WorldPersistenceManager.SetWorldStore(new JsonWorldStore(CoreManager, filePath
));
```

Implement IWorldStore

```
public interface IWorldStore
{
    Task<bool> SaveAsync(IWorldDocument worldDocument);
    IWorldDocument BeginSave();
    Task<IWorldDocument> LoadAsync();
    WorldLoadFailureAction OnSceneUnavailable(ushort sceneId, uint sceneHandle);
}
```

- `BeginSave`: return an empty document for the engine to write the world into. Nothing is committed until the write completes, so a failed save leaves the previous one intact.
- `SaveAsync(IWorldDocument)`: keep the finished document. Return `true` once it is safely stored.
- `LoadAsync`: return the document to read, or `null` when there is nothing to load.
- `OnSceneUnavailable(ushort sceneId, uint sceneHandle)`: called once for each saved scene instance this build cannot open, before any object from it is built. Return a `WorldLoadFailureAction`.

The async methods may complete on any thread. The engine does not block on them.

Implement IWorldDocument

A document is streamed, so neither saving nor loading holds a second copy of the world in memory. Scenes are written and read before objects, because an object names its scene.

```
public interface IWorldDocument
{
    void WriteScene(uint sceneHandle, ushort sceneId, SceneScope sceneScope);
    IWorldStateWriter BeginSystem(uint systemId, in WorldSystemIdentity worldSystemIdentit
y);
    void EndSystem();
    bool TryReadScene(out uint sceneHandle, out ushort sceneId, out SceneScope sceneScope)
;
    bool TryReadSystem(out uint systemId, out WorldSystemIdentity worldSystemIdentity, out
 IWorldStateReader worldStateReader);
}
```

Writing:

- `WriteScene` records one open scene instance.

- `BeginSystem` opens one object and returns the `IWorldStateWriter` its components are written through.
- `EndSystem` closes that object.

Reading:

- `TryReadScene` returns the next saved scene, or `false` when there are no more.
- `TryReadSystem` returns the next saved object, its `WorldSystemIdentity`, and the `IWorldStateReader` for its components. The reader only needs to stay valid until the next `TryReadSystem` call.

Store members by name

`IWorldStateWriter` and `IWorldStateReader` handle one object's component state, keyed by component type name and member name rather than position, so a save survives a component gaining or losing a member.

```
public interface IWorldStateWriter
{
    void BeginComponent(string componentTypeName);
    void Write(string memberName, NetworkMemberBase networkMember);
    void EndComponent();
}

public interface IWorldStateReader
{
    bool TryBeginComponent(string componentTypeName);
    bool TryRead(string memberName, NetworkMemberBase networkMember);
    void EndComponent();
}
```

Nucleus calls these, not your game code. Each `NetworkComponent` names its type, then writes each member in declaration order. `Write` passes the member itself: ask the member to write itself and store the result under its name, with no per-type cases.

On load, a member the save does not contain keeps its constructed default, and a saved name the component no longer has is skipped. There is no version or schema check. A member whose type changed but whose name stayed the same reads back silently wrong, so rename a member when its meaning changes.

WorldSystemIdentity describes each saved object

A readonly struct holding what a save needs beyond member values:

Field	Meaning
`SystemTypeName`	Full name of the `NetworkSystem` type to build, resolved through the type registry on load.
`PlatformId`	The prefab or scene-object identifier the object was started with.
`PrefabBundleId`	The content bundle the prefab was registered from, or the base build.

`IsSceneObject`	Whether the object is a scene object. A load binds it to the object the scene already placed instead of constructing one.
`SceneHandle`	The scene instance the object lives in.
`GroupId`	The group the object belongs to.
`ParentId`	The object this one is parented to.
`ComponentTypeNames`	Full names of the object's `NetworkComponent` types, in order, so a load knows which components to add. A scene object's components come from the scene, so for it this list is descriptive only.

Choose what a missing scene does

```
public enum WorldLoadFailureAction : byte
{
    Skip,
    Fail
}
```

Return one from `OnSceneUnavailable` when a saved scene cannot be opened in this build:

- `Skip` (default): drop that scene instance and every object saved in it, then load the rest. The safer answer when scenes are independent.
- `Fail`: abandon the load, and `LoadAsync` returns `false`. The old world does not come back: a load despawns every object and closes every scene before reading saved scenes. No saved object is built until every saved scene has been answered for, so the result is an empty world, not a half-built one. Saved scenes that reopened before the failure stay open and empty.

Start from JsonWorldStore

`JsonWorldStore` writes readable JSON (scenes, objects, components and member names as text), so saved worlds diff object by object and member by member.

```
public sealed class JsonWorldStore : IWorldStore
{
    public string FilePath { get; }
    public WorldLoadFailureAction MissingSceneAction { get; set; } = WorldLoadFailureActio
n.Skip;

    public JsonWorldStore(CoreManager coreManager, string filePath) { ... }
}
```

- `FilePath`: the file saved worlds are written to and read from.
- `MissingSceneAction` (default `Skip`): what `OnSceneUnavailable` returns, so you can choose `Skip` or `Fail` without writing your own store.

A save writes to a temporary file beside the target and moves it over only when complete, so an interrupted save leaves the previous world in place.

If your store needs an in-memory document to write into before sending it to your target, use `WorldDocument`, the `IWorldDocument`, `IWorldStateWriter` and `IWorldStateReader` implementation that `JsonWorldStore` returns from `BeginSave`.

Reacting to changes: OnMembersChanged

> **Using Unity?** See Reacting to a replicated change in Unity.

`NetworkComponent` has one callback for every member change, rather than an event per member. Override it to react without polling:

```
public virtual void OnMembersChanged(ulong memberFlags, MemberChangeDirection memberChange
Direction) { }
```

Test which members changed

`memberFlags` has one bit per member, in member-Id order. Cast it to the component's generated flags enum to test a member:

```
public override void OnMembersChanged(ulong memberFlags, MemberChangeDirection memberChang
eDirection)
{
    TransformComponentFlags flags = (TransformComponentFlags)memberFlags;
}
```

Check the direction

`memberChangeDirection` says where the change came from:

- `MemberChangeDirection.Write`: this peer changed the members. It runs before anything is sent this tick, and means the change was committed to this tick's send, not that anything left the process. It still runs when the only observer is culled by interest, or is withheld because the change came from that observer.
- `MemberChangeDirection.Read`: the members came from another peer. It runs once per tick, after everything received this tick has been applied and before the Reconcile step.

A host raises both in the same frame, `Write` first, without waiting for its own packet to come back. Code that should run only for a remote change must check for `Read`. `NetworkMemberBase.TryGetWritingClient` can only find a writer under `Read`.

Three flag cases catch people

- **Full apply**: a spawn, resync, reconcile or whole-component recovery reports `ulong.MaxValue`. Treat every member as changed.
- **Partial recovery**: a recovery can report only the members it repaired, not the full mask.
- **A set bit does not mean the value differs**: a member's bit can be set while its value matches what it held before. A predicted member whose prediction was confirmed still sets its bit.

Inherited members share bit positions

If a component's inheritance chain declares members on more than one layer, each layer's flags start at bit 0, so bits from different layers overlap in `memberFlags`. A single-layer component, the usual case, maps exactly onto its generated flags enum.

Collection member reference

Every collection member has these

All five derive from `NetworkCollectionMemberBase<TChange>` and share:

- `Values`: the live contents. `IReadOnlyList<T0>` for a list, `ReadOnlySpan<T0>` for an array, `IReadOnlyDictionary<T0, T1>` for a dictionary, `IReadOnlyCollection<T0>` for a hash set or queue.
- `PreviousValues`: the contents from the most recent tick, before the current one, on which the collection changed.
- `Count`: the number of items held.
- `Changes`: an `IReadOnlyList<TChange>` of this tick's operations.
- A constructor taking `int ringDepth = 2`, the number of snapshots kept. See Networked collections.
- Protected virtual `GetRingDepth()`, which returns the constructed ring depth, and `GetCurrentTick()`, which returns the network loop's tick, or an unset value when none is running. Override either only to change where a subclass gets those values.

NetworkListMember\<T0> is an ordered list

Items are addressed by index.

Member	Signature	Notes
`Add`	`void Add(T0 item)`	Appends to the end.
`AddRange`	`void AddRange(IReadOnlyList<T0> items)`	Appends each item in order.
`Insert`	`void Insert(int index, T0 item)`	Inserts at the index.
`Remove`	`bool Remove(T0 item)`	Removes the first occurrence; returns whether one was found.
`RemoveAt`	`void RemoveAt(int index)`	Removes the item at the index.
`Clear`	`void Clear()`	Empties the list.
`Contains`	`bool Contains(T0 item)`	
`IndexOf`	`int IndexOf(T0 item)`	Returns `-1` when not found.
indexer	`T0 this[int index] { get; set; }`	The setter records a `Set` operation. Writing the value the element already holds records nothing.
`GetEnumerator`	`List<T0>.Enumerator GetEnumerator()`	Struct enumerator over the live list.

```
NetworkListMember<int> scores = new();
scores.Add(10);
scores.AddRange(new List<int> { 20, 30 });
scores[0] = 15;
scores.RemoveAt(1);
```

`Changes` holds `NetworkListChange<T0>` entries with `Operation` (a `NetworkListOperation`: `Add`, `Insert`, `RemoveAt`, `Set` or `Clear`), `Index` and `Item`. `Index` is undefined for `Clear`. `Item` is the default of `T0` for `RemoveAt` and `Clear`.

NetworkArrayMember\<T0> is a fixed-length array

Items are addressed by index. It is the only collection with per-element interpolation.

Member	Signature	Notes
`Set`	`void Set(int index, T0 item)`	Replaces one element. Writing the value already held records nothing.
`Fill`	`void Fill(T0 item)`	Sets every element to the same value; length is unchanged.
`Resize`	`void Resize(int newLength)`	Grows or shrinks the array. New slots are default-filled; truncated slots are dropped. Throws `ArgumentOutOfRangeException` above 65,536, the largest length a receiver accepts.
`Clear`	`void Clear()`	Sets every element to the default of `T0`; length is unchanged.
`IndexOf`	`int IndexOf(T0 item)`	Returns `-1` when not found.
`Count` / `Length`	`int Count { get; }` / `int Length { get; }`	The same value.
`Values`	`ReadOnlySpan<T0> Values { get; }`	The live contents. Valid only until the next change.
`GetInterpolated`	`T0 GetInterpolated(int index)`	Animates the element at `index` from its value at the previous change toward its current value, for rendering between ticks. Returns the current value when the element type has no registered interpolator, or when the previous snapshot did not hold that index.
`GetEnumerator`	`Enumerator GetEnumerator()`	Struct enumerator over the live contents, up to the current length.

```
NetworkArrayMember<float> healthByRegion = new(ringDepth: 2);
healthByRegion.Resize(4);
healthByRegion.Set(0, 100f);
float rendered = healthByRegion.GetInterpolated(0);
```

`Changes` holds `NetworkArrayChange<T0>` entries with `Operation` (a `NetworkArrayOperation`: `Set`, `Fill`, `Resize` or `Clear`), `Index`, `Item` and `NewLength`. `Index` is undefined for `Fill`, `Resize` and `Clear`. `NewLength` is defined only for `Resize`. `Item` is the default of `T0` for `Resize` and `Clear`.

NetworkDictionaryMember\<T0, T1> maps keys to values

The key type is `T0` and the value type is `T1`.

Member	Signature	Notes
`Add`	`void Add(T0 key, T1 value)`	Throws when the key is already present.
`Remove`	`bool Remove(T0 key)`	Returns whether an entry was found and removed.
`Clear`	`void Clear()`	Empties the dictionary.
`ContainsKey`	`bool ContainsKey(T0 key)`	
`TryGetValue`	`bool TryGetValue(T0 key, out T1 value)`	
indexer	`T1 this[T0 key] { get; set; }`	The setter adds a key or overwrites one. Writing the value the key already holds records nothing.

```
NetworkDictionaryMember<int, string> playerNames = new();
playerNames.Add(1, "Alice");
playerNames[1] = "Alicia";
playerNames.TryGetValue(1, out string name);
```

`Changes` holds `NetworkDictionaryChange<T0, T1>` entries with `Operation` (a `NetworkDictionaryOperation`: `Set`, `Remove` or `Clear`), `Key` and `Value`. An add and an overwrite are both `Set`. `Key` is the default of `T0` for `Clear`. `Value` is the default of `T1` for `Remove` and `Clear`.

NetworkHashSetMember\<T0> holds unique values

The set is unordered.

Member	Signature	Notes
`Add`	`bool Add(T0 item)`	Returns false when the item is already present.
`Remove`	`bool Remove(T0 item)`	Returns whether the item was found and removed.
`Clear`	`void Clear()`	Empties the set.
`Contains`	`bool Contains(T0 item)`	

```
NetworkHashSetMember<int> activeQuestIds = new();
activeQuestIds.Add(7);
bool hasQuest = activeQuestIds.Contains(7);
activeQuestIds.Remove(7);
```

`Changes` holds `NetworkHashSetChange<T0>` entries with `Operation` (a `NetworkHashSetOperation` : `Add`, `Remove` or `Clear`) and `Item`. `Item` is the default of `T0` for `Clear`.

NetworkQueueMember\<T0> is a first-in-first-outqueue

Member	Signature	Notes
`Enqueue`	`void Enqueue(T0 item)`	Adds to the back.
`Dequeue`	`T0 Dequeue()`	Removes and returns the front item.
`TryDequeue`	`bool TryDequeue(out T0 item)`	Returns `false` with the default of `T0` when empty.
`Peek`	`T0 Peek()`	Returns the front item without removing it.
`TryPeek`	`bool TryPeek(out T0 item)`	Returns `false` with the default of `T0` when empty.
`Contains`	`bool Contains(T0 item)`	
`Clear`	`void Clear()`	Empties the queue.

```
NetworkQueueMember<string> pendingActions = new();
pendingActions.Enqueue("Attack");
if (pendingActions.TryDequeue(out string action))
{
    // action == "Attack"
}
```

`Changes` holds `NetworkQueueChange<T0>` entries with `Operation` (a `NetworkQueueOperation`: `Enqueue`, `Dequeue` or `Clear`) and `Item`: the item added for `Enqueue`, the item removed for `Dequeue`, and the default of `T0` for `Clear`.

Sending your own data beside the members

Override `WritePayload` and `ReadPayload` on a `NetworkComponent` to send data that is not a member value whenever a peer is told about the component as a whole.

For example, a cast bar's spell is a value, so it is a member. How far into the cast the caster is changes every tick, so as a member it would be paid for on every tick of every cast. A peer only needs it once, when it first learns about the object:

```
public partial class CastComponent : NetworkComponent
{
    public readonly NetworkMember<ushort> SpellIndex = new();

    private uint _castStartTick;
    private uint _elapsedCastTicks;

    protected override void WritePayload(Writer writer, PayloadReason reason, Invoker auth
or, ulong serializedMemberFlags)
    {
        // This serialization never carried the spell, so there is no cast here to describ
e.
        if ((serializedMemberFlags & (ulong)CastComponentFlags.SpellIndex) == 0)
            return;

        writer.WriteUInt32(CoreManager.NetworkLoopManager.Tick - _castStartTick);
    }

    protected override void ReadPayload(Reader reader, PayloadReason reason, Invoker autho
r, ulong serializedMemberFlags)
    {
        if ((serializedMemberFlags & (ulong)CastComponentFlags.SpellIndex) == 0)
            return;

        // SpellIndex.Value already holds what this serialization carried, so the bar can
be built from both.
        _elapsedCastTicks = reader.ReadUInt32();
    }
}
```

A client that spawns halfway through a cast now gets the spell from the member and the progress from the payload. So does a client that lost the tick the cast started on, because the repair for that tick carries the payload too.

When the payload runs

Only on these, never on an ordinary update:

- the spawn that builds the object on a peer;
- a resync, and an interest serve when an object comes back into view;
- a reconcile to a controlling client;
- a client's own write of a whole object upstream;
- the recovery that repairs a tick a peer lost.

A member that changes every tick costs no payload however long it changes. A component that overrides neither method costs nothing.

Members are applied first

By the time `ReadPayload` runs, every member of the component has been applied, so you can read them while parsing the payload. The example reads `SpellIndex.Value` this way.

Use the arguments to decide what to write

- `serializedMemberFlags`: which members this serialization carried, one bit per member Id, matching the generated `<Component>Flags` enum. It is `NetworkComponent.EverySerializedMember` when the whole component is carried (a spawn, resync, interest serve or reconcile). A recovery carries only the members the lost tick carried, so a payload about one member should test that member's bit and return when it is clear.
- `author`: an `Invoker` saying which side wrote the body: this peer when writing, the sender when reading. Each reason means the same thing whichever side wrote it.
- `reason`: why the whole component is being sent, one of three `PayloadReason` values:

Reason	When
`Serve`	The receiver builds the object, or rebuilds it from nothing it can trust. From the authority: a spawn, a resync or an interest serve. From a client: a predicted spawn.
`Reconcile`	The authority is correcting the client that controls the object.
`Recovery`	The authority is repairing a tick a peer lost. The only reason whose member flags can be narrower than the whole component.

A controlled object reconciles to its controller constantly, so a payload written on `Reconcile` is paid for on every tick of prediction, while one written on `Serve` is paid for once. Return early for reasons you have nothing to say for.

Read back every bit you write

Write four bytes and read two, and everything after the payload reads wrong. That shows up as wrong values or a whole tick discarded, often far from the component that caused it.

Both ends receive the same flags and reason. Decide what to write and what to read from those and from the member values only, never from local state the other end cannot see, such as a field only the server sets or something read from your own scene.

A read can happen on a body the engine then throws away, so applying a payload must be harmless.

When a payload throws

An exception in either method is caught and logged, and the whole body is dropped rather than sent half written.

- **Writing**: the body is not sent. Those peers are served again later, so a payload that keeps throwing keeps its object from ever reaching them.
- **Reading**: handled like a body that could not be read. A later repair replaces it, and a predicted spawn whose payload throws is refused.

Prefer a member when the data is a value

A payload has no change detection. It runs only alongside sends your members already cause, so a component whose members never change never writes one.

If the data is a value, a member is usually cheaper than it looks. A cast's start tick as a member changes once per cast, costs nothing in between, and every peer can subtract it from the current tick. Use a payload for something you compute rather than store, a variable-length list, or a snapshot of where an object stands right now.

A spawn payload is different

A spawn payload is declared per object rather than per component, sent only with the spawn and never with a repair, and read after the member values. Use it for what is true about an object when it spawns, and the component payload for state a component carries alongside its members.

Replicated State and Change Callbacks

Pages in this category:

- NetworkMember
- How replicated state works
- Networked collections
- Reacting to changes: OnMembersChanged
- Your first networked component
- Collection member reference
- TransformComponent
- What a member can hold, and how to keep one off the wire
- How Often a Member Is Sent
- Letting clients set members
- Retention, redundancy and recovery
- Judging and arbitrating client writes

What a member can hold, and how to keep one off the wire

Use a value type, a string or a pooled class

A `NetworkMember<T0>` value must be one of:

- a value type;
- `string`;
- a class implementing `IPoolResettable`, with automatic pooling.

A class-typed member must be assigned before it is first sent, because `null` cannot be sent.

```
private readonly NetworkMember<float> _health = new();
private readonly NetworkMember<MyPoolableClass> _payload = new(isAutomaticPoolingEnabled:
true);
```

Some types throw `NotImplementedException` when a member holding one sends a change, rather than replicating wrong: `Matrix4x4`, plain arrays, `NetworkSystemGroup`, `Channel` and `NetworkSystem`. For a type with no serializer, register one; see Writing a Custom Serializer.

Turn on automatic pooling for class values

With `isAutomaticPoolingEnabled: true`, a class-typed `IPoolResettable` value is returned to its pool when the member drops it from its history, and incoming instances are rented from `ResettableObjectPool<T0>` instead of allocated. Rent the instances you assign from `ResettableObjectPool<T0>`, and never keep or change one after assigning it.

If the type does not implement `IPoolResettable`, or has no pool returner registered (check `NetworkTypePoolReturner<T0>.IsRegistered`), an error is logged once at spawn.

Pool only flat objects. A nested class-typed field inside a pooled `T0` can end up shared between instances.

Keep a member off the wire with ReplicationIgnore

`[ReplicationIgnore]` keeps a member off the wire but still saves it. Put it on a `NetworkMemberBase` field to exclude that member, or on a `NetworkComponent` to exclude every member it declares.

```
[ReplicationIgnore]
private readonly NetworkListMember<ItemStack> _containerContents = new();

[ReplicationIgnore]
private readonly NetworkMember<bool> _isModerationFlagged = new();
```

Use it for data that must survive a restart without going to every observer, such as a container's contents or a moderation flag. What happens:

- **The member is never sent.** No send interval or transmission mode applies, and a tick where only ignored members changed sends nothing. `NetworkComponent.OnMembersChanged` still fires under `MemberChangeDirection.Write` on the peer that changed it, so server code can react normally.
- **Every peer but the server holds the constructed default.** Reading it on a client returns the type's default, not the server's value.
- **A peer promoted to server by adoption holds those defaults too.** Saving from it writes the defaults over what the store held.

For the opposite, keeping a member out of saves but still replicating it, see Choosing what a save keeps.

NetworkMember

`NetworkMember<T0>` is one replicated value on a `NetworkComponent`. Write access and `IsReplicated` live on its base type, `NetworkMemberBase`.

Read the value

Member	Description
`Value`	Gets or sets the current value. Changes are compared once per tick, not on every set, so assigning the same value costs nothing beyond the write itself.
`PreviousValue`	The value held immediately before the current one, or `default(T0)` if none has been committed.
`InterpolatedValue`	A smoothed value that animates from `PreviousValue` to `Value` across the member's send interval, then holds at `Value`. See below.

Read past values by tick

```
public bool TryGetValueByTick(uint tick, out T0 value);
```

Returns true when the member still holds a value written on that tick. A member is written only on ticks it changes, so a tick between two writes has no value of its own, and an old enough tick is no longer held. An unpredicted member keeps two values; a predicted one keeps its prediction history (see Predicted members and prediction history).

```
public bool TryGetValueAtOrBeforeTick(uint tick, out T0 value);
```

Returns the newest value written at or before the tick, so it answers for ticks between writes. Returns false only when every value the member still holds is newer.

```
public bool TryGetPreviousValueAndTickGap(out T0 previousValue, out uint tickGap);
```

Returns the value before the current one and the number of ticks between the two writes, for deriving a rate of change (a velocity from position changes, for example) instead of replicating it. Returns false, with `tickGap` set to `0`, unless both values have been written and the current one is strictly newer.

Construct a member

```
public NetworkMember(
    CompressionLevel deltaCompressionLevel = CompressionLevel.Tight,
    bool isAutomaticPoolingEnabled = false,
    MemberRole role = MemberRole.Replicate,
    float accuracy = float.NaN,
    float interpolationSnapThreshold = float.NaN,
    SendInterval sendInterval = SendInterval.Normal,
    TransmissionMode transmissionMode = TransmissionMode.Interval,
    bool isExtrapolationEnabled = true,
    byte correctionSmoothingTicks = 0,
    bool isCarriedForward = true,
    float resultTolerance = 0f);
```

Pass arguments by name at the field declaration, for example `NetworkMember<T> _x = new(role: MemberRole.Validate);`.

Argument	Construction-time only?	Meaning
`deltaCompressionLevel`	Yes	The compression level for this member's value. Defaults to `CompressionLevel.Tight`.
`isAutomaticPoolingEnabled`	Yes	Returns dropped `IPoolResettable` past values to their pool. When true, the member owns instances assigned to `Value`: rent them from `ResettableObjectPool<T0>` and never keep or change an instance after assigning it. See [What a member can hold](./member-value-types.md).
`role`	Yes	`MemberRole.Replicate` (default): an ordinary member; received values apply as they arrive. `MemberRole.Predict`: client state, a per-tick value the simulation reads (a move direction, a jump press), written by whoever controls the system and sent upstream by a controlling client every tick; see [Client state](../control/client-state-components.md). `MemberRole.Validate`: a predicted result that the controlling client computes ahead of the server; the server simulates the same tick, checks the client's result against its own without applying it, and orders a reconcile when they differ. Its value is only as precise as its `accuracy`. On a client that does not control the system, any role but `Replicate` is held at the server's values while a reconcile replays. See [Predicted members and prediction history](../control/prediction-history-and-predicte

`accuracy`	Construction default only	The accuracy the value is sent at. `float.NaN` (default) uses the type's default accuracy. A per-call accuracy passed to `Initialize` still overrides it. Coarser values save bandwidth on quantities that tolerate more error than position.
`interpolationSnapThreshold`	Yes	The distance beyond which `InterpolatedValue` snaps to `Value` instead of animating. `float.NaN` (default) never snaps. Only for numeric types with a registered tolerance comparer.
`sendInterval`	No, see `SetSendInterval`	How often a changed member sends while `TransmissionMode.Interval` is in force. `SendInterval.Normal` sends every tick.
`transmissionMode`	No, see `SetTransmissionMode`	`TransmissionMode.Interval` (default) sends changes paced to `sendInterval`. `TransmissionMode.Divine` (Pro) projects motion instead, for values that move predictably.
`isExtrapolationEnabled`	No, see `SetExtrapolationEnabled`	Whether receivers keep the value moving through the sender's silence, in either mode. `true` (default) favors staying live when an update is lost; `false` holds still until told and favors accuracy. See [How often a member is sent](./send-pacing.md).
`correctionSmoothingTicks`	Yes	On a client that predicts the member, how many ticks `InterpolatedValue` takes to glide from the value shown before a reconcile to the corrected one, while `Value` is already correct. `0` (default) shows the correction at once.

`isCarriedForward`	Yes	For a `MemberRole.Predict` member only. `false` keeps it at its default when `NetworkComponent.TryGetLastState(false)` carries the previous tick's client state across a tick whose client state did not arrive, for a value that must not repeat, such as a jump press. Ignored for other roles.
`resultTolerance`	Yes	For a predicted member only: how far its predicted value may be from the server's before it counts as a misprediction, for a value a replay cannot reproduce exactly, such as one physics moves. `0` (default) compares at `accuracy`. Applies only while `SystemManager.IsResultValueSendEnabled` is on (the default); with it off, results are compared exactly.

InterpolatedValue animates across the send interval

`InterpolatedValue` animates each arrival across the member's send interval: one tick for a member sent every tick, the whole span for a member paced to `Short` or `Long`.

When `PreviousValue` and `Value` are farther apart than `interpolationSnapThreshold`, `InterpolatedValue` returns `Value` directly, so a teleport is not swept across.

Change settings at runtime

Three settings can change after construction:

```
public void SetSendInterval(SendInterval sendInterval);
public void SetTransmissionMode(TransmissionMode transmissionMode);
public void SetExtrapolationEnabled(bool isExtrapolationEnabled);
```

`SetReplicates(bool isReplicated)` on `NetworkMemberBase` sets whether the member is sent at all. Do not call it: Nucleus calls it for a member marked `ReplicationIgnoreAttribute`, and calling it yourself makes this peer send something different from what its peers expect to read.

Find which client wrote a member

```
public bool TryGetWritingClient(out Connections.Connection connection);
```

Defined on `NetworkMemberBase`. Returns the client whose write landed on this member on the current tick. Call it from `NetworkComponent.OnMembersChanged` under `MemberChangeDirection.Read`. It returns false:

- under `MemberChangeDirection.Write`;
- on a later tick;

- for a server write, or a local write on either peer;
- after the writing client has disconnected.

Learn more about accuracy, pacing and write access

- Compression, Accuracy and Per-Type Registrations
- How often a member is sent
- Letting clients set members

Networked collections

Nucleus replicates five collection types

Each is a member type you declare on a `NetworkComponent`:

- `NetworkListMember<T0>`: an ordered, resizable list.
- `NetworkArrayMember<T0>`: a fixed-length array with `Length` and `Resize`.
- `NetworkDictionaryMember<T0, T1>`: a key/value map.
- `NetworkHashSetMember<T0>`: an unordered set of unique elements.
- `NetworkQueueMember<T0>`: a first-in-first-out queue.

Every method is listed in the Collection member reference.

Use a value type or `string` for elements and dictionary keys. Nothing enforces this, but an element held by reference can be changed without going through the collection, and the collection never sees that change.

Change a collection through its methods

Each collection has the usual methods: `Add`, `Insert`, `RemoveAt`, `Clear`, an indexer for a list, array or dictionary, `Enqueue` and `Dequeue` for a queue, `Add` and `Remove` for a hash set. Each change is recorded as an operation in that tick's `Changes`.

Writing the same slot twice in one tick, with `Set` or the indexer, replaces the pending operation instead of adding a second one. On a list, a `Set` at the index of the tick's most recent `Set`, `Add` or `Insert` folds into that operation.

Unchanged contents are not sent

A collection is sent only when its contents differ from the previous tick. Refilling a collection with the same contents every tick costs nothing. Elements compare like a scalar `NetworkMember<T0>`, so a floating-point element within the member's accuracy compares equal.

Read what changed with Changes

`OnMembersChanged` tells you which members changed this tick. A collection's `Changes` (`IReadOnlyList<TChange>`) tells you what happened inside it. It holds this tick's operations, whether applied locally or just received, and is cleared at the end of the tick. Read it inside `OnMembersChanged`.

When a whole collection is received at once, `Changes` is empty: read `Values` instead. That happens on a spawn, a recovery, and when a client writes a collection, in which case the server and other observers receive the whole collection.

Read previous contents

`PreviousValues` holds the contents from the most recent tick, before the current one, on which the collection changed. Every constructor takes `ringDepth`, the number of snapshots kept:

```
public NetworkListMember(int ringDepth = 2) : base(ringDepth)
```

2 (the default) is the minimum that gives a previous view; lower values are raised to 2. A deeper ring covers more history but keeps that many copies of the collection.

Collections allow only the controller to write

`NetworkSystem.TrySetWriteAccess` refuses to widen a system's access past `MemberWriteAccess.Controller` when the system holds any collection member. It returns `false` and logs an error saying the system holds a collection NetworkMember, so its write access cannot be widened beyond `Controller`.

Retention, redundancy and recovery

These `SystemManager` settings trade latency and bandwidth for tolerance to packet loss. Set `StateInterpolation`, `GlobalRedundancy`, `ClientRedundancy` and `StateRetentionMilliseconds` the same on both peers: `StateInterpolation` sets how large a buffer the receiver has, so a mismatch leaves one side planning for recovery the other cannot provide.

Buffer incoming state with StateInterpolation

`SystemManager.StateInterpolation` is how many ticks a receiver holds a state packet before applying it. `0` applies at once, `1` waits until the next tick, and so on. The maximum is `SystemManager.MaximumStateInterpolation` (7); higher values are clamped at runtime.

Each buffered tick adds a tick of delay, and gives late and resent packets more time to arrive. The server's redundant resends and recovery only work within this buffer. A client's upstream resends are the exception (see `ClientRedundancy`).

Resend state with GlobalRedundancy

`SystemManager.GlobalRedundancy` is how many past ticks of unreliable state every peer resends alongside the current one. At 2, each tick goes out three times, so a receiver can still apply a tick whose first copies were lost. On the server's sends, values above `StateInterpolation` are clamped, because those copies would arrive too late to apply.

Redundancy only helps on an unreliable channel. Call `SystemManager.CanUseRedundancy()` rather than reading `GlobalRedundancy`: it returns true only when the server's effective redundancy is above zero and the transport's default channel is unreliable.

```
if (systemManager.CanUseRedundancy())
{
    // Safe to plan around redundant resends on this connection.
}
```

Resend client writes with ClientRedundancy

`SystemManager.ClientRedundancy` (default 3) is how many extra copies a client adds, on top of `GlobalRedundancy`, to its predicted spawns and its own writes. With the defaults, a client sends each upstream tick four times.

It is not limited by `StateInterpolation`: a copy that arrives too late is discarded and costs only its upstream bytes, while one that arrives during a burst of loss is applied. A host's own client does not use it.

Redundancy is the first defence, not a guarantee. A predicted spawn whose copies are all lost is despawned locally. A client write that still goes missing is recovered (see below).

A tick missing some packets applies what arrived

A tick too large for one packet is sent as several. With `SystemManager.PartialStateApplyEnabled` on (the default), the receiver applies the packets that arrived, and only what was missing is sent again, starting as soon as the loss is noticed. With it off, the whole tick is discarded and recovered. That costs more under steady loss, and a large world can fall further behind with every attempt.

A tick is still discarded whole when a missing packet carried a change the rest relies on, such as a despawn, a controller change or a reparent.

The setting belongs to the receiver, so the peers do not need to match.

```
// Discard and recover any tick that loses a packet.
coreManager.SystemManager.PartialStateApplyEnabled = false;
```

A client's own writes always reach the server

Writes to a system a client controls, or has been allowed to write, are recovered however much upstream traffic is lost. Messages and remote calls are not: they may drop. Recovery costs nothing on a link that loses nothing, and `ClientRedundancy` keeps it rare on one that does.

Limits:

- A member the server has written since the client's lost write is not resent, so an older value never overrides the server's newer one.
- A client that may no longer write the system resends nothing for it.
- Nothing older than the retention window can be resent, so a link silent for longer than that loses what it was carrying.

Keep past state with StateRetentionMilliseconds

`SystemManager.StateRetentionMilliseconds` sets both how much past state is kept for recovery and how long a sent tick may go unacknowledged before recovery escalates. Raise it for high-latency players, so slow acknowledgments are not mistaken for loss.

- Default: `SystemManager.DefaultStateRetentionMilliseconds` (`1000`).
- Maximum: `SystemManager.MaximumStateRetentionMilliseconds` (`2000`). Higher values are capped.
- Changing it at runtime never reallocates.

A longer window gives struggling connections more time to recover, at the cost of a longer wait before a slow but healthy peer would be mistaken for a lossy one. A peer that falls behind by more than the window is sent the whole world it observes again.

Set them in Unity

`UnitySystemManager` has inspector fields for `GlobalRedundancy` (Global Redundancy), `ClientRedundancy` (Client Redundancy), `StateInterpolation` and `PartialStateApplyEnabled` (Partial State Apply). `StateRetentionMilliseconds` has no inspector field; set it in code.

Watch for recovery violations

A `RetentionExceededViolation` or `RecoveryUnconfirmedViolation` means the retention or recovery settings are too small for what a connection is experiencing. See Violation reference.

To check which systems were recently sent, see Reading serialization history.

How Often a Member Is Sent

Choose Interval or Divine

A `NetworkMember<T0>` uses one `TransmissionMode`, never both:

- `Interval` (default): sends changes paced to the member's `SendInterval`.
- `Divine` (Pro): for values that tend to move predictably, such as a moving transform. It can stay quiet where `Interval` would keep sending, and corrects only when needed. It ignores `SendInterval`.

Pick `Divine` for a value that moves smoothly, `Interval` otherwise.

Choose a send interval

`SendInterval` names a span rather than a tick count, so its meaning does not change with the tick rate:

- `Normal` (default): every tick the value changed.
- `Short`: at most every 250 ms.
- `Long`: at most once a second.

Choose `Short` for a value that needs to look live, and `Long` for one that only needs to be right before long.

`SendIntervalTicks.Resolve(SendInterval, uint)` converts a span to ticks for a tick rate. It rounds up, so a rate that does not divide the span evenly adds at most one tick of latency rather than sending faster than asked. `Normal` always resolves to one tick.

Pacing never loses a value. A member that changes several times within its interval sends its latest value on the next due tick, and a member that stops changing still sends its final value. A spawn, resync, recovery or reconcile always delivers the current value.

```
// Interval-paced, at most one update every 250ms.
NetworkMember<float> health = new(sendInterval: SendInterval.Short);

// Divine projection; sendInterval is ignored while this mode is set.
NetworkMember<Vector3> position = new(transmissionMode: TransmissionMode.Divine);
```

The interval applies the same way to every observer. No interval scales with distance or anything else.

Choose what silence means

`isExtrapolationEnabled` applies in both modes:

- `true` (default): receivers keep the value moving through the sender's silence. A lost update can cause a brief overshoot.
- `false`: receivers hold still until told. A lost update can cause a brief stall.

Start with `true`, and set it to `false` if a lost update visibly overshoots.

Under `Interval`, `true` lets a receiver render up to one send interval ahead of the newest value while it moves, and the sender sends one small extra update when the value stops so receivers settle exactly on it. That costs a tiny overhead on each update plus one update per stop. `false` costs nothing extra, and the receiver never renders past the newest value.

Receivers follow the sender's settings

`TransmissionMode` and `isExtrapolationEnabled` are construction defaults that you can override per instance at runtime. A receiver adopts the sender's mode and extrapolation setting, so the peers do not

need to be built with the same values. Set each once rather than changing it mid-stream.

Free builds send every changed tick

`Divine`, `Short` and `Long` are Pro. They compile in Free but change nothing there: `Divine` replicates as `Interval` does, and every span resolves to one tick, which is what `Normal` means. One prefab therefore loads in either edition without `#if` around its members.

How replicated state works

Members live in components, components live in systems

- A `NetworkSystem` holds up to 31 `NetworkComponents`. It tracks identity, lifecycle and observers.
- A `NetworkComponent` holds up to 64 members, one for each bit of the `ulong` `memberFlags` that `OnMembersChanged` reports. Nothing checks this limit: a component with more still builds but does not replicate correctly.

Put replicated values on components, not on your `NetworkSystem` subclass.

A separate limit applies to the structs and classes a member's value is made of: at 63 serialized fields and properties (`Constants.MaximumNetworkMemberCount`) the build reports `SERIALIZERS000`, so 62 is the most one can hold.

Late joiners need no extra code

A new observer receives every member's current value, then only changes. There is no separate late-joiner payload to write.

OnMembersChanged reports each tick's changes

`NetworkComponent.OnMembersChanged` runs up to twice per component per tick: once for what this peer changed (`MemberChangeDirection.Write`) and once for what it received (`MemberChangeDirection.Read`). See Reacting to changes: OnMembersChanged.

Only the controller writes by default

Under `MemberWriteAccess.Controller` (the default, value `0`), the server or the single controlling client sets a system's members. `MemberWriteAccess.AnyClient` lets other observers write too; it is Pro, and a Free build has no `MemberWriteAccess` type. See Letting clients set members.

Four settings decide what a change costs

- **Accuracy**: how finely a value's change is compared and sent.
- **Compression level**: how much the value is compressed before it is sent.
- **Transmission mode**: whether changes are interval-paced or projected.
- **Send interval**: how often a changed member may send.

See NetworkMember and How often a member is sent.

Use Nucleus terms

Do not substitute names from another engine's netcode:

- **System**, not object: `NetworkSystem` is the networked entity.
- **Component**, not behaviour: `NetworkComponent` holds members within a system.
- **Member**, not variable: `NetworkMemberBase` is one replicated value.
- **Controller**, not owner: the client allowed to set a system's members by default.

Letting clients set members

Choose who may write a system's members

Every `NetworkSystem` has a `MemberWriteAccess`:

```
public enum MemberWriteAccess : byte
{
    Controller = 0,
    AnyClient = 2,
}
```

- `Controller` (default): the server, or the single controlling client.
- `AnyClient`: the server, and any client that currently observes the system.

Write access adds writers; it does not change control. `ControllerConnectionId` stays exclusive and server-assigned, and client state, prediction and reconcile stay controller-only.

Set access on the server

```
public bool TrySetWriteAccess(MemberWriteAccess stateWriteAccess)
```

`TrySetWriteAccess` is server-only; a client call is rejected. It also refuses to widen a system that holds a collection member.

Read the current setting from `WriteAccess`. On a client, `CanLocalClientWriteMembers` says whether the server last told this client it may write. It is always `false` on the server.

The server checks every incoming write against its own `WriteAccess`. Nothing a client sends can widen what it may write.

A client that stops observing cannot write

Every writer, the controller included, must observe the system. Losing interest in a system removes write permission at the same moment.

Tune how long a contested write shows locally

So that a client does not see its own write bounce back as a correction, a losing or contested write keeps showing locally for a window before the server's value is accepted. Set the window per system:

```
public uint MemberWriteConvergenceDeadlineTicks = UnsetConvergenceDeadlineTicks;

public const uint UnsetConvergenceDeadlineTicks = uint.MaxValue;
```

Left at `UnsetConvergenceDeadlineTicks`, the window follows the connection's measured round-trip time. Set `NetworkSystem.MemberWriteConvergenceDeadlineTicks` when that feels wrong: a shared UI value and a contested physics object want different windows.

Write access beyond the controller is Pro

`MemberWriteAccess`, `NetworkSystem.WriteAccess`, `NetworkSystem.TrySetWriteAccess` and the convergence window are Pro. In Free, code naming them does not compile, and only the controller writes.

Even with Pro:

- A system holding a collection member cannot be widened past `Controller`.

- A client write on a system with more than one possible writer costs more bandwidth than a write from a single controller.

TransformComponent

> **Using Unity?** See NetworkTransform.

`TransformComponent` replicates an object's position, rotation and scale. It has no space of its own: the engine adapter that binds it decides whether the values are world or parent-local, through which subclass the system carries rather than a flag.

It replicates three members

Member	Type	Compression	Transmission mode
`Position`	`NetworkMember<Vector3>`	`CompressionLevel.Aggressive`	`TransmissionModeDefaults.Motion`
`Rotation`	`NetworkMember<Quaternion>`	default	`TransmissionModeDefaults.Motion`
`Scale`	`NetworkMember<Vector3>`	default	`TransmissionModeDefaults.Motion`

```
public readonly NetworkMember<Vector3> Position = new(CompressionLevel.Aggressive, transmi
ssionMode: TransmissionModeDefaults.Motion);
public readonly NetworkMember<Quaternion> Rotation = new(transmissionMode: TransmissionMod
eDefaults.Motion);
public readonly NetworkMember<Vector3> Scale = new(transmissionMode: TransmissionModeDefau
lts.Motion);
```

`Vector3` and `Quaternion` are `System.Numerics` types. An adapter converts to and from the engine's own types.

`TransmissionModeDefaults.Motion` is divine projection in Pro and `TransmissionMode.Interval` in Free, so framework components read correctly in both editions. Your own code can name a `TransmissionMode` directly in either edition: `TransmissionMode.Divine` compiles in Free and replicates as `Interval`.

Set all three members at once

These apply one choice to `Position`, `Rotation` and `Scale` together:

- `SetSendInterval(SendInterval sendInterval)`
- `SetTransmissionMode(TransmissionMode transmissionMode)`
- `SetExtrapolationEnabled(bool isExtrapolationEnabled)`

A receiver adopts the sender's transmission mode and extrapolation setting, so only the send interval must be the same on every peer.

Subclass it to bind a real transform

`TransformComponent` is a `partial class` for an engine adapter to subclass, binding the three members to a real transform and choosing their space. Both peers must carry the same subclass, and therefore the same space.

Judging and arbitrating client writes

Decline a value with IMemberWriteValidator

Control, and in Pro `MemberWriteAccess`, decides whether a client may write a member at all. To judge the value itself, implement `IMemberWriteValidator<T0>` on the `NetworkComponent` that owns the member and return a `MemberWriteAction` from `ValidateMemberWrite`:

```
public partial class SwitchBankComponent : NetworkComponent, IMemberWriteValidator<uint>
{
    private const uint SwitchCount = 3;

    public readonly NetworkMember<uint> ActiveSwitchIndex = new();

    public MemberWriteAction ValidateMemberWrite(NetworkMember<uint> networkMember, uint p
roposedValue, Connection writingClientConnection) =>
        proposedValue < SwitchCount ? MemberWriteAction.Accept : MemberWriteAction.Decline
;
}
```

- `MemberWriteAction.Accept` (the zero value): the value lands as if there were no validator.
- `MemberWriteAction.Decline`: the member keeps the value it holds.

Inside the call, `networkMember.Value` is still the current value, so you can compare it with `proposedValue`.

Implement it once per value type

Implement `IMemberWriteValidator<T0>` once for each member value type you want judged. A component with a `NetworkMember<uint>` and a `NetworkMember<bool>` implements it twice to judge both. A type you do not implement it for is not judged.

All members of one type share one implementation, so compare the member passed in against your own fields:

```
if (ReferenceEquals(networkMember, ActiveSwitchIndex))
{
    // ...
}
```

Only the server validates

`ValidateMemberWrite` runs only on the server, and never for the server's own writes.

A declined value is never relayed

Nothing else the client sent is undone. The declined value is not sent to other observers.

The client that wrote it keeps showing its own value for the convergence window (see Letting clients set members), then shows the server's value again. This costs no traffic.

Declining is not a violation and does not count against the client. Use violations only for something a client should never have been able to send.

The first write wins

When two clients write the same member on the same tick, the first write applied wins, with or without a

validator. Writes are applied in ascending `Connection.Id` order, so the lowest permitted `Connection.Id` wins.

`Connection.Id` is stable for the life of a connection but is not chosen for fairness and is not a security boundary. A client that reconnects gets a new `Id`, so it can reconnect into a better position. For anything that must survive a reconnect or resist that, arbitrate on `Connection.Identity`, which names a client across reconnects.

See which client's write landed

Call `TryGetWritingClient(out Connection connection)` on the member from `NetworkComponent.OnMembersChanged` under `MemberChangeDirection.Read`. See NetworkMember.

Your first networked component

Using Unity? See Replicating a value from a Unity script.

Declare the component

A networked component is a `partial class` that inherits `NetworkComponent` and declares `readonly NetworkMember<T0>` fields:

```
public partial class SimpleStateComponent : NetworkComponent
{
    public readonly NetworkMember<int> Counter = new();
}
```

`partial` is required. Nucleus generates the rest of the type, including a flags enum with one bit per member in member-Id order. Without `partial`, the component never replicates anything.

Add a field for each value. Each gets its own Id and its own bit in the flags enum.

Rent a system that carries it

A component lives on a `NetworkSystem`. `NetworkSystemPool.TryRent` takes the system type and one type argument per component, and returns a started system through its `out` argument. It returns `false` when no `CoreManager` was found or the start was refused:

```
if (!NetworkSystemPool.TryRent<NetworkSystem, SimpleStateComponent>(coreManager, out Netwo
rkSystem? system, canStartSystem: true))
    return;

system.TryGetComponent(out SimpleStateComponent component);
component.Counter.Value = 31337;
```

`canStartSystem` defaults to `true`. Pass `false` to start the system yourself later. A system rented this way has no engine object: it is plain C# on both peers.

Find it on the other peer

A system rented from code has `PlatformId` `NetworkSystem.UnsetPlatformId`, so no spawn handler hears about it (`INetworkSystemSpawnHandler.OnDynamicSystemSpawned` is not called). Use `NetworkSystemPool.Watch` instead. It hands your handler every system with the given system type and component types, as each starts locally or arrives:

```
NetworkSystemWatch watch = NetworkSystemPool.Watch<NetworkSystem, SimpleStateComponent>(
    coreManager,
    acquiredSystem =>
    {
        acquiredSystem.TryGetComponent(out SimpleStateComponent component);
        int value = component.Counter.Value;
    })!;
```

The match is on the components a system carries, not its exact type: any `NetworkSystem` or subclass carrying a `SimpleStateComponent` matches, whatever else it carries. A watch for a plain `NetworkSystem` with no component arguments matches every system.

Call `NetworkSystemPool.Unwatch(watch)` when you are done. `Watch` also takes an optional released handler, called when a matched system stops. For a system that arrived over the network, that means it is about to return to its pool, so drop any reference to it before the handler returns.

Read and write the value

`NetworkSystem.TryGetComponent<T0>` gets a component from a system on either peer:

```
if (system.TryGetComponent(out SimpleStateComponent component))
{
    component.Counter.Value = 42;   // write: only meaningful on the system's writer
    int current = component.Counter.Value;  // read: always safe
}
```

Setting `Value` marks the member changed this tick and queues it to send. On a peer that does not write the system, `Value` is the last value received.

Do not call the generated overrides

`NetworkComponent` declares `Write`, `WriteDelta`, `Read`, `ReadDelta`, `Initialize`, `Deinitialize` and `OnReturn` as `public virtual`. Nucleus implements the first four from your fields, and calls `Initialize` on rent and system start, `Deinitialize` on system stop, and `OnReturn` on return to the pool. Do not call any of them.

To react to a value changing, override `OnMembersChanged`. See Reacting to changes: OnMembersChanged.

Systems, Objects and Spawning

Pages in this category:

- NetworkSystem reference
- What a system is
- Renting and finding systems in code
- Starting and stopping systems
- Parents, children and hierarchy
- System groups
- Binding spawns to your own engine
- References between systems
- Spawn compensation
- Predicted spawn and despawn requests

NetworkSystem reference

Identify a system

Member	Type	Meaning
`Id`	`uint`	The network identity, unique across the network. `UnsetId` (`0`) until the system starts.
`PlatformId`	`uint`	The engine object this system links to: a scene object's identifier within its scene, or a dynamic prefab's local identifier within its shard. `UnsetPlatformId` (`0`) until assigned, and always for a system with no engine object.
`PrefabBundleId`	`ushort`	The content bundle owning this system's prefab. Meaningful only for a dynamically spawned prefab; zero is the base build.
`RequiredBundleId`	`ushort`	The bundle a client must hold before this system may spawn for it. Falls back to `PrefabBundleId` when no override is set. Server-side only; clients never receive it.
`SetRequiredBundleId(ushort)`	method	Overrides `RequiredBundleId` without changing `PrefabBundleId`.
`UnsetRequiredBundleId`	`const ushort`	`ushort.MaxValue`, **not** `0` like the other Unset constants, because zero is a valid requirement naming the base build.
`IsSceneObject`	`bool`	True when `PlatformId` names a scene object that already exists on every peer, rather than a prefab a receiver must instantiate.

`SceneHandle`	`uint`	The live scene instance this system belongs to. `UnsetSceneHandle` (`0`) when it belongs to the scene the peer booted into rather than one the server opened.
`GroupId`	`uint`	The `NetworkSystemGroup` this system belongs to. `UnsetGroupId` (`0`) when ungrouped.
`ParentId`	`uint`	The `Id` of the system this one is parented to, such as a crate riding a vehicle. `UnsetParentId` (an alias of `UnsetId`) at the root of its scene.

Track its lifecycle

`State` is a `NetworkSystemState`. `IsStarted` is true only while `State` is `Started`.

Value	Meaning
`Starting`	Accepted but not yet live. A start accepted late in a tick stays here until the next tick.
`Started`	Live and networked.
`Stopping`	A stop was accepted; it takes effect at the end of the tick.
`Stopped`	Not networked.

What the identity members hold during a transition:

- **Starting**: if the start was deferred to the next tick, `Id` is still `UnsetId` and `PlatformId`, `GroupId` and `SceneHandle` are not yet assigned. All are set together when the system reaches `Started`.
- **Stopping**: `Id`, `PlatformId`, `GroupId`, `SceneHandle` and `ParentId` keep their last live values until the end of the tick.

Check the local peer's role

These answer "is the server, client or host running here", not "do I control this object". For control, use `IsController(ControllerType)`.

Member	Meaning
`IsServerStarted`	The local peer runs the server.
`IsClientStarted`	The local peer runs an authenticated client.
`IsHostStarted`	The local peer runs a server and an authenticated local client. Both `IsServerStarted` and `IsClientStarted` are then true.

`CoreManager`	The `CoreManager` managing this system, which reaches every other manager.

Read its components

Member	Meaning
`ComponentCount`	The number of `NetworkComponent`s attached.
`TryGetComponent<T0>(out T0 component)`	The first attached component of type `T0`. Returns false when none is attached.
`TryGetComponent<T0>(byte componentId, out T0 component)`	The component at a component Id, when it is of type `T0`. The Id is the order the component was added in, the same on every peer.
`MaximumComponentCount`	The most components one system can hold: 31.
`GroupMemberCount`	The number of members in this system's group, or zero when ungrouped. On a client it is the count received when the group arrived.

What a system is shows how to walk every component of a repeated type.

Log a system

`NetworkSystemExtensions.AsString(this NetworkSystem)` returns a one-line summary of state, platform id, group id and id. It returns `"IsNull"` for a null reference, so it needs no null check.

```
Logger<MyType>.LogInformation($"System: [{system.AsString()}]");
```

Find the rest of its surface

Category	Covers
Control	`ControllerConnectionId` and who drives the system.
Write access	`MemberWriteAccess` and per-member write permissions.
Interest	Observer registration and what a connection is served.
RPCs	`IRpcScope`, `RpcSendAccess`, and remote calls scoped to a system.
Client state	`MemberRole.Predict` members, sent by the controlling client every tick.
Reconcile	`ReconcileRequired`, `ResultMismatched`, the simulation callbacks and correcting predicted state.

References between systems

A peer can lose sight of a system through interest and regain it later, so a plain object reference to another system goes stale. `NetworkSystemMember` and `NetworkConnectionMember` hold a reference that keeps working: they replicate an identifier and resolve it to a local instance whenever this peer can see the target.

Point at another system with NetworkSystemMember

Declare it as a field on a `NetworkComponent`, like a `NetworkMember<T0>`:

```
public readonly NetworkSystemMember Target = new();

public NetworkSystem Value { get; set; }
public uint TargetId { get; }

public void Initialize(NetworkComponent networkComponent, byte id);
```

- `Value` is the local instance, or `null` when the reference is empty or the target is not visible on this peer.
- `TargetId` is the target's Id, readable whether or not `Value` is resolved.
- `Initialize` is called for you, as for every other member.

Point at a peer with NetworkConnectionMember

`NetworkConnectionMember` has the same members, with `Value` typed as `Connection`:

```
public Connection Value { get; set; }
public uint TargetId { get; }

public void Initialize(NetworkComponent networkComponent, byte id);
```

Connection Ids are reused, so when the referenced peer leaves, `Value` returns to `null` rather than pointing at whoever gets that Id next. `TargetId` is kept, so the reference resolves again if the same peer reconnects with the same identifier.

References re-resolve on their own

Both members keep following their `TargetId` for as long as they exist. When the target leaves this peer's view, `Value` becomes `null`; when it returns, `Value` resolves again with no code on your part.

They are not value members: they have no interpolation, snap threshold or accuracy tolerance.

React when a reference resolves

A reference resolving or losing its target raises no event of its own. It is reported through the component's `OnMembersChanged` callback; see Reacting to changes: OnMembersChanged for how to tell it apart from a value write.

Predicted spawn and despawn requests

> **Using Unity?** See Client-predicted spawning in Unity.

Predicted spawning lets a client create or remove an object at once, without waiting a round trip for the server to agree. It is a Pro feature.

Turn it on for an object type

Nothing is predictable until both of these are true:

- The object's `NetworkSystem.PredictedSpawnPolicy` is not `None` (the default). Use `Spawn`, `Despawn` or `SpawnAndDespawn`, optionally combined with `AllowUndeclaredComposition`.
- The server has set `SystemManager.PredictedSpawnValidator` (for spawns) or `SystemManager.PredictedDespawnValidator` (for despawns). While either is `null`, every request of that kind is refused. Setting `PredictedSpawnValidator` also starts serving identifier leases to every authenticated client, now and later.

`PredictedSpawnPolicy` is not replicated; each peer reads its own copy. A prefab's policy is authored on the prefab, so it is the same everywhere. For a code-only object it is whatever that peer's code set. The server judges a despawn by its own copy, not the client's.

`AllowUndeclaredComposition` lets a client describe an object whose component composition no call site in this build declares. Without it, such a request is refused, because that is what a client fabricating its own systems looks like.

Ask from the client

```
public bool EnsureStartPredictedSystem(NetworkSystem networkSystem, uint platformId, ushor
t prefabBundleId, uint sceneHandle, out PredictedSpawnRefusal predictedSpawnRefusal);

public bool EnsureStopPredictedSystem(NetworkSystem networkSystem, out PredictedSpawnRefus
al predictedSpawnRefusal);
```

Both are on `SystemManager`. Both return `false` when the client's own framework refuses before anything is created, removed or sent (a host, an empty lease, a policy that does not allow it), report why in `predictedSpawnRefusal`, and raise `SystemManager.PredictedSpawnRefused`. This is separate from the server refusing a request it received.

Spawn

`EnsureStartPredictedSystem` creates the object locally and sends the server a request describing it.

- Set the object's opening values **before** the call. The request captures the components during the call; later writes reach the server only if the object also replicates them.
- `platformId` is the prefab's identifier within its bundle, or `NetworkSystem.UnsetPlatformId` for an object with no prefab.
- `prefabBundleId` is the content bundle the prefab was registered from.
- `sceneHandle` is the live scene instance, or `NetworkSystem.UnsetSceneHandle`.

Despawn

`EnsureStopPredictedSystem` is the only way a client removes a networked object. A client cannot stop a replicated system, and destroying its engine object leaves nothing to bring back on a refusal.

The call parks the object rather than removing it:

- `NetworkSystem.IsPredictedDespawnPending` becomes true and `PredictedDespawnPending` fires.
- `INetworkSystemSpawnHandler.OnPredictedDespawnPendingChanged(networkSystem, isPredictedDespawnPending)` is called with `true`, and with `false` if the server refuses. Its default body is empty; the Unity integration deactivates the GameObject.
- The system stays started and the server's state still applies, but the client cannot change it: member writes are ignored, `TrySendRpc` returns false, no client state is sent, and it is not simulated.

The server removes the object on the tick the client asked, so everything the client did before the call ran with the object present. If the server refuses, the object comes back, the pending flag clears, and `PredictedDespawnRejected` fires on the same instance.

When the server never answers

A single lost packet does not lose a request. If no answer arrives within two round trips (at least half a second, at most three seconds), the client gives up:

- A spawn is torn down, after `PredictedSpawnRejected` fires with `NotPermitted`.
- A despawn is undone, and `PredictedDespawnRejected` fires with `NotPermitted`.

If the server did admit a spawn but its answer arrived after the timeout, the client receives the server's object as an ordinary spawn, so both peers still hold the same object.

Judge requests on the server

A spawn request is judged by a delegate:

```
public delegate PredictedSpawnDecision PredictedSpawnValidatorHandler(in PredictedSpawnReq
uestContext predictedSpawnRequestContext);
```

`PredictedSpawnRequestContext` describes the request without building anything: `Connection`, `SystemType` (resolved against this build's own registry), `Components` (a `PredictedSpawnComponents` view of the declared composition), `SystemId`, `GroupId`, `PlatformId`, `SceneHandle`, `PrefabBundleId`, `SystemBundleId`, `SystemLocalId`, `IsDeclaredComposition` and `HasPrefab`.

Return a `PredictedSpawnDecision`:

```
public static PredictedSpawnDecision Permit(bool isRequesterController = false);
public static PredictedSpawnDecision Refuse();
```

`Permit(true)` gives the requesting client control of the object. The default is off, since the common predicted spawn is a projectile nobody controls.

A despawn request is judged the same way:

```
public delegate PredictedDespawnDecision PredictedDespawnValidatorHandler(in PredictedDesp
awnRequestContext predictedDespawnRequestContext);
```

`PredictedDespawnRequestContext` carries `Connection` and `System`, the server's own live object. Return `PredictedDespawnDecision.Permit()` or `Refuse()`.

Inspect opening values before deciding

Turn on `PredictedSpawnStateInspectionEnabled` to have the server build the object with its opening state before the validator runs. `PredictedSpawnRequestContext.PredictedSystem` is then that object, so the validator can check real values such as a projectile's speed or a spawn position. It is off by

default. A permitted spawn starts the instance that was inspected, so only a refused request costs an extra build.

Size the identifier lease

A client spends identifiers the server leased to it ahead of time. These are on `SystemManager`:

Member	Meaning
`PredictedSpawnLeaseSize`	How many objects a client can predict in a burst before waiting on the server. Default 8.
`PredictedSpawnSystemIdsPerSpawn`	How many `NetworkSystem` identifiers each leased spawn carries, so the most systems one predicted object can have. Default 8. An object needing more is refused as `PredictedSpawnRefusal.LeaseExhausted`.
`PredictedSpawnSystemLeaseSize`	Read-only: `PredictedSpawnLeaseSize * PredictedSpawnSystemIdsPerSpawn`.
`PredictedSpawnsPerSecond`	The sustained rate a client earns replacement spawns, or zero for no limit beyond the top-up cadence. Default 30.
`PredictedSpawnLeaseCount`	On a client, how many more objects it can predict now. Zero on the server.
`PredictedSpawnSystemLeaseCount`	On a client, how many more `NetworkSystem` identifiers it holds. Zero on the server.

- The lease size counts objects: a grouped prefab of five systems draws one, like a single system. Raise it for a weapon that fires in bursts. Raise `PredictedSpawnSystemIdsPerSpawn` for a prefab built from more systems.
- Spent spawns are replaced at most every 0.3 seconds, or once per round trip on a slower link. That alone allows about 26 spawns a second at the default lease of 8. `PredictedSpawnsPerSecond` holds a larger lease to a sustained rate; at the default of 30 it starts limiting once the lease is above about 9, so raising the lease raises the burst, not the rate. Lower it for objects that are expensive to build.
- An empty lease refuses locally with `PredictedSpawnRefusal.LeaseExhausted`, without contacting the server.

Handle the outcome on the client

`NetworkSystem` raises these on the predicting client:

- `PredictedSpawnConfirmed`: the server agrees the object exists. Raised before the server's state is applied.
- `PredictedSpawnRejected(PredictedSpawnRejectReason)`: the server refused the object. Raised just before the local copy is torn down.
- `PredictedDespawnPending`: the client asked to remove this object and is waiting.
- `PredictedDespawnRejected(PredictedDespawnRejectReason)`: the server refused the removal; the object comes back where the server has it.

Enum	Values
`PredictedSpawnRejectReason`	`NotPermitted` (the validator refused), `UnleasedId`, `UnknownType`, `SceneNotEntered`, `GroupMismatch`, `Malformed`, `NotPermittedByPolicy`, `UndeclaredComposition`
`PredictedDespawnRejectReason`	`NotPermitted`, `NotPermittedByPolicy`, `UnknownSystem`, `NotObserved`

`NetworkSystem.PredictedSpawnConfirmMode` sets what happens to the predicted state on confirmation:

- `ApplyAuthoritativeState` (default): the server's values apply in full, so the object lands where the server built it, and any difference shows.
- `KeepPredictedState`: the server's member values are discarded and the object keeps the path the client simulated, taking only identity, controller and placement. Use it for an object that must not visibly jump, such as a projectile the player is watching.

Renting and finding systems in code

Using Unity? See NucleusBehaviour: the script base class.

Rent a system

`NetworkSystemPool.TryRent<TSystem, TComponent0...>(coreManager, out networkSystem, canStartSystem)` gives you a `NetworkSystem` with no engine object behind it. Overloads take zero to thirty-one components (`TComponent0` through `TComponent30`). Each rents `TSystem`, attaches the named components, and starts the system unless you pass `canStartSystem: false`. It returns `false`, and logs the reason, when no `CoreManager` could be found or the start was refused.

```
if (!NetworkSystemPool.TryRent<NetworkSystem, MatchClockComponent>(coreManager, out Networ
kSystem networkSystem, canStartSystem: true))
    return; // no CoreManager instance could be resolved, or the start was refused

if (networkSystem.TryGetComponent(out MatchClockComponent matchClockComponent))
{
    matchClockComponent.Round.Value = 1;
    matchClockComponent.SecondsRemaining.Value = 10f;
}
```

`TSystem` is nearly always plain `NetworkSystem`; the component list is what tells one rent from another.

Rent a count decided at runtime

When the number of components comes from data, such as one per seat in a vehicle, rent one component type a runtime number of times:

```
public partial class SeatComponent : NetworkComponent
{
    public readonly NetworkMember<bool> IsOccupied = new();
}

byte seatCount = 4;

if (!NetworkSystemPool.TryRent<NetworkSystem, SeatComponent>(coreManager, seatCount, out N
etworkSystem vehicleSystem))
    return; // no CoreManager, a count outside 1 to NetworkSystem.MaximumComponentCount, o
r the start was refused

for (byte componentId = 0; componentId < seatCount; componentId++)
{
    if (vehicleSystem.TryGetComponent(componentId, out SeatComponent seatComponent))
        seatComponent.IsOccupied.Value = false;
}
```

The components take Ids `0` to `seatCount - 1`, the same on every peer. A count of zero, or more than `NetworkSystem.MaximumComponentCount` (31), is refused and logged.

Return a rented system

Give a rented system back rather than letting it fall out of scope:

- `NetworkSystemPool.Return<TSystem>(networkSystem)` returns it to the pool.
- `NetworkSystemPool.ReturnAndNullifyReference<TSystem>(ref networkSystem)` does

the same and sets your reference to null.

Do not call either on a started system. Stop it with `SystemManager.EnsureStopSystem(networkSystem, isPoolReturnRequestedOnDespawn: true)`, and it returns to the pool when the despawn completes. A receiver's copy returns to its pool automatically; never call `Return` on it.

Watch for a composition to find it on other peers

A rented system has `NetworkSystem.UnsetPlatformId`, so `INetworkSystemSpawnHandler` never sees it: there is no `TryBindSceneObject` or `OnDynamicSystemSpawned` call. Other peers have no identifier to look it up by.

Register a watch instead. `NetworkSystemPool.Watch<TSystem, TComponent0...>(coreManager, acquired, released)` subscribes to every system of that type and component composition on the peer that registers it, whether the system started there or arrived from the network. It returns a `NetworkSystemWatch`; pass it to `NetworkSystemPool.Unwatch(watch)` when you are done.

```
private NetworkSystemWatch _matchWatch;

private void OnMatchAcquired(NetworkSystem networkSystem) => _matchSystem = networkSystem;

private void OnMatchReleased(NetworkSystem networkSystem)
{
    if (_matchSystem == networkSystem)
        _matchSystem = null;
}

public void Start()
{
    _matchWatch = NetworkSystemPool.Watch<NetworkSystem, MatchClockComponent>(coreManager,
 OnMatchAcquired, OnMatchReleased);
}

public void Stop()
{
    NetworkSystemPool.Unwatch(_matchWatch);
    _matchWatch = null;
}
```

- `NetworkSystemAcquiredHandler` fires for every matching system.
- `NetworkSystemReleasedHandler` fires as an acquired system stops. Drop your reference there; a reference kept after the handler returns can end up naming a different spawn.
- Matching is by composition, not type, so name the components that set this system apart.
- Registering a watch replays it against every system already started, so it works before or after the system exists.

Use SystemManager.SystemStarted to see every system

`SystemManager.SystemStarted` and `SystemManager.SystemStopped` raise for every system that starts or stops on this peer, whatever its composition. Use them for logging, bookkeeping across all system types, or one subscriber that dispatches by type. For systems of a known composition, use a watch.

Spawn compensation

An object the server spawns reaches an observer a round trip late, after the spawn crosses the network and waits out that peer's interpolation. A projectile spawned moving appears at its origin instead of along the path it has already travelled. Spawn compensation closes the gap by giving the object extra simulation time right after it starts, most of it in the opening ticks. It is a Pro feature.

It only advances the new object's own simulation for a short window. It does not rewind the world or touch any other object, and it is not lag compensation, which judges a shot against where targets were.

Enable compensation once the object has started

Call `NetworkSystem.EnableSpawnCompensation` once the object has started:

```
EnableSpawnCompensation(SpawnCompensationScope.All);
```

`aggression` sets how strongly the time is front-loaded. It defaults to `SpawnCompensationCurve.DefaultAggression` (2); a value below `SpawnCompensationCurve.MinimumAggression` (1) clamps to 1, which spends the time at a constant rate. It changes how quickly the time is spent, never how much.

```
EnableSpawnCompensation(SpawnCompensationScope.All, aggression: 3f);
```

Calling it again restarts the schedule rather than adding to it. `CancelSpawnCompensation` abandons the rest of a catch-up, leaving the object where the schedule had reached:

```
CancelSpawnCompensation();
```

Each peer works out its own budget from its role and its measured link.

Choose a scope by who simulates the object

`SpawnCompensationScope` is a flags enum naming which peers catch up:

- `None`: nobody compensates.
- `Authority`: the server catches up through the time a client's predicted spawn request spent reaching it. The server's own spawns are never late, so this does nothing for them.
- `RemoteClients`: every client other than the one that predicted the spawn catches up through its own downstream latency.
- `All`: both. A remote client is owed what the server already spent plus its own delay, held to one cap.

The client that predicted the spawn never compensates, whatever the scope.

- An object the server simulates and replicates the pose of wants `Authority` alone. A client that fast-forwarded it would be pulled back by the next server update, which shows as a stutter.
- An object every peer simulates from its spawn parameters, such as an ordinary projectile, wants `All`.

Advance motion by the catch-up time

`IsSpawnCompensating` is true while a schedule still owes time. `SpawnCompensationSeconds` is the extra time this tick spends on the catch-up, or zero when nothing is compensating; read it beside the tick delta if the game simulates on its own step.

Or subscribe to `SpawnCompensationStepped`, raised once per tick while the schedule runs, before that tick's early fixed-update callbacks:

```
SpawnCompensationStepped += OnSpawnCompensationStepped;

void OnSpawnCompensationStepped(in SpawnCompensationDelta delta)
{
    transform.position += velocity * delta.TotalDelta;
}
```

`SpawnCompensationDelta` carries:

Field	Meaning
`FixedDelta`	The tick's ordinary fixed delta.
`CompensationDelta`	The extra time this step spends.
`TotalDelta`	Both added together. Advance motion by this so the handler needs no branch.
`BudgetSeconds`	The whole schedule.
`RemainingSeconds`	What is still owed after this step.
`StepIndex`, `StepCount`	Progress through the schedule, counting from one.

Cap how much time a peer makes up

The cap bounds one peer's whole catch-up across every hop, and so also how long it runs. In Unity it is **Max Spawn Compensation (ms)** on the Unity Lag Compensation Manager.

Member	Value
`LagCompensationManager.DefaultMaximumSpawnCompensationMilliseconds`	150, the default cap.
`LagCompensationManager.MaximumSpawnCompensationCeilingMilliseconds`	500, the highest the cap can be set.
`LagCompensationManager.MaximumSpawnCompensationMilliseconds`	The live cap. Its setter clamps to the ceiling.
`NetworkSystem.MaximumTransmittedSpawnCompensationTicks`	63, the most ticks of server-side compensation a remote client is told about. Larger amounts clamp to 63, independent of the millisecond cap.

```
CoreManager.LagCompensationManager.MaximumSpawnCompensationMilliseconds = 300;
```

Binding spawns to your own engine

Register a spawn handler

`SystemManager.SetSpawnHandler(INetworkSystemSpawnHandler)` registers the one object that links a spawned `NetworkSystem` to an object in your engine. Register it once, at bootstrap:

```
systemManager.SetSpawnHandler(mySpawnHandler);
```

Only spawns with a `PlatformId` reach the handler. A code-only system (`NetworkSystem.UnsetPlatformId`) never does; watch for it instead.

Bind scene objects and link spawned prefabs

`INetworkSystemSpawnHandler` has one member for each kind of networked object, and they differ in who builds the `NetworkSystem`.

Scene objects already exist locally. Return the system you built for the object from `TryBindSceneObject`:

```
bool TryBindSceneObject(in SceneObjectKey sceneObjectKey, out NetworkSystem networkSystem)
;
```

Return a system that is rented and initialized but not started; the framework assigns its identifiers and applies the incoming state. Returning `false` is an error: the scene was not loaded, or the scenes differ.

Dynamic prefabs are built and started by the framework, with state applied, before the handler is called. Instantiate the matching prefab and link it:

```
void OnDynamicSystemSpawned(NetworkSystem networkSystem, ushort prefabBundleId,
    uint prefabLocalId, uint groupId, uint sceneHandle);
```

`prefabBundleId` is the content bundle the prefab ships in (zero is the base build). `prefabLocalId` is the prefab's identifier within that bundle, and is also the spawn's `PlatformId`.

Look up scene objects by scene and id

A scene object's id is unique only inside its scene, and several live scenes can each hold the same id. `SceneObjectKey` carries both halves; use both to find the object:

```
public readonly struct SceneObjectKey : IEquatable<SceneObjectKey>
{
    public readonly uint SceneHandle;
    public readonly uint SceneObjectId;
}
```

`SceneHandle` is the live scene instance, or `NetworkSystem.UnsetSceneHandle` for the scene the peer booted into rather than one the server opened. `SceneObjectId` is the id stamped on the object.

Link a group's members to one instance

Systems that share a `groupId` other than `NetworkSystem.UnsetGroupId` link onto one engine instance. The first member to spawn instantiates the object; later members with the same `groupId` attach to it. Track the live instance per group id so later members can find it.

Handle moves, reparenting and despawns

- `OnSystemSceneChanged(NetworkSystem networkSystem, uint newSceneHandle)`: the

system moved to another server-opened scene. Move its object into that scene rather than rebuilding it. Must be idempotent.

- `OnSystemParentChanged(NetworkSystem childNetworkSystem, NetworkSystem parentNetworkSystem)`: re-seat the object under the new parent's object, or at the scene root when `parentNetworkSystem` is null. What the re-seat must preserve depends on the space the object's transform replicates in, which the handler reads from the object. Must be idempotent.
- `OnSystemDespawned(NetworkSystem networkSystem, bool isObjectKept)`: the system stopped, on the server once its despawn was sent, or on a receiver when the despawn applies.
  - `isObjectKept` true: an interest condition stopped the system for this peer alone and the server wants the object kept. Release the link, keep the object, and register it against `networkSystem`'s `Id` so a later re-entry spawn links onto it instead of building a second one.
  - `isObjectKept` false: release every link and destroy the object, unless the object's own destruction started the stop.
- `OnRetainedObjectReleased(uint systemId)`: the server has now culled or despawned a system whose object you kept. Reclaim the object kept under `systemId` as an ordinary despawn would. It is called for any despawn of an id this peer no longer routes, so it must do nothing when no object is kept under that id.

Handle the failures the framework cannot catch

- **A spawn for content not yet loaded.** `OnDynamicSystemSpawned` can name a `prefabBundleId`/`prefabLocalId` pair your registry has not loaded. Load content before spawns for it can arrive, and treat an unresolved pair as an error rather than dropping the system.
- **The wrong scene.** Put each built or moved object in the scene `sceneHandle` names, not the engine's default scene. Otherwise a world with several open scenes collapses every dynamic spawn into one.

Starting and stopping systems

Using Unity? See Spawning and despawning objects.

Start a system

`SystemManager.EnsureStartSystem` has two overloads. Use the plain one for a system with no engine object:

```
bool started = coreManager.SystemManager.EnsureStartSystem(networkSystem);
```

The system must already be initialized. Receivers build it with no engine object, so find it there through `SystemManager.SystemStarted` or a system watch.

Use the fuller overload when an integration binds the system to an object it already owns:

```
bool started = coreManager.SystemManager.EnsureStartSystem(
    networkSystem,
    platformId,
    isSceneObject: false,
    prefabBundleId: 0,
    sceneHandle: NetworkSystem.UnsetSceneHandle);
```

- `platformId` links the system to an engine object: a scene object's identifier within its scene, or a dynamic prefab's local identifier within its shard.
- `isSceneObject` is true when `platformId` names a scene object rather than a dynamically spawned prefab.
- `prefabBundleId` is the content bundle of a dynamically spawned prefab. Zero is the base build; scene objects carry none.
- `sceneHandle` is the live scene instance the system belongs to, so a receiver binds or instantiates it in the right scene. Leave it unset in a world that opens no scenes.

A start called late in a tick stays `Starting`, with its Id unset, until the next tick. The system is still usable by the caller in the meantime. A peer can also see the system a tick or two after it started; see Object pacing.

Stop a system

```
bool stopped = coreManager.SystemManager.EnsureStopSystem(networkSystem, isPoolReturnReque
stedOnDespawn: true);
```

An accepted stop moves the system to `Stopping` at once and despawns it on observers, including any that lose packets. At the end of the tick it moves to `Stopped`, its Id is released, and the spawn handler is notified. Keep no reference to a system that arrived from the server after that. A system you rented stays yours unless `isPoolReturnRequestedOnDespawn` returns it to the pool.

- Only the server stops a replicated system. A client's stop is refused. A peer with no started transport (offline) can stop its own systems.
- A stop called late in a tick takes effect on the next tick.
- Starting a system again while its stop is pending cancels the stop: the system stays `Started` with its identity intact. A spawn, despawn and spawn in the same tick resolves to the latest one.

Use the shorthands on the system

`NetworkSystemExtensions` offers both calls as instance methods that find the `CoreManager` from the

system. Both return false and log, rather than throw, when the system is null or has no `CoreManager`.

```
networkSystem.EnsureStart();
networkSystem.EnsureStop(isPoolReturnRequestedOnDespawn: true);
```

Observe the roster from your own code

```
coreManager.SystemManager.SystemStarted += networkSystem =>
{
    // Raised once a system has completed its start on this peer, whether this
    // peer created it or it arrived from the network.
};

coreManager.SystemManager.SystemStopped += networkSystem =>
{
    // Raised as a started system stops. Read what you need during the call,
    // and keep no reference to a system that arrived from the server.
};

int startedCount = coreManager.SystemManager.StartedSystemCount;

if (coreManager.SystemManager.TryGetSystemReference(id, out NetworkSystem networkSystem))
{
    // True either because the Id was found, or because id was the intentional
    // Unset value, in which case networkSystem comes back null.
}

coreManager.SystemManager.UnknownSystemDeltaReceived += (connection, systemId) =>
{
    // Received state named a system this peer doesn't have. That means a
    // missed spawn or a more serious desync.
};
```

To be handed only systems of one composition, register a system watch instead.

Move a started system between scene instances

```
coreManager.SystemManager.EnsureMoveSystemToScene(networkSystem, destinationSceneHandle);
```

This is the only call that moves a spawned object between scenes; nothing calls it for you. It is server-only, and only for a started, dynamically spawned system (not a scene object) with a platform identity. The system keeps its Id, controller, access and group. Peers no longer in the destination scene despawn it, and peers newly in it receive it as a spawn. See the scenes category for scene instances and handles.

Send one-time data with a spawn

A spawn payload is data attached to one object's spawn, such as a random seed, a loot table or a launch speed, that the object needs when it arrives but no member should keep replicating. Derive from `NetworkSpawnPayload` and declare an instance on the system in the same call stack as the rent:

```
public class CrateSpawnPayload : NetworkSpawnPayload
{
    public int LootTableIndex;

    public override void WriteSpawnPayload(Writer writer, NetworkSystem networkSystem) =>
writer.WriteInt32(LootTableIndex);

    public override void ReadSpawnPayload(Reader reader, NetworkSystem networkSystem) => L
ootTableIndex = reader.ReadInt32();
}

if (NetworkSystemPool.TryRent<NetworkSystem, CrateComponent>(coreManager, out NetworkSyste
m networkSystem))
    networkSystem.SetSpawnPayload(new CrateSpawnPayload { LootTableIndex = 3 });
```

- The payload arrives with every spawn of the object, including for a late joiner or a peer that regains interest, and never in the updates between.
- A receiver reads it into its own instance, available as `networkSystem.SpawnPayload`. Member values are already applied, so `ReadSpawnPayload` can use them to decide what to read.
- `ReadSpawnPayload` must read exactly what `WriteSpawnPayload` wrote. It can run more than once for one object, so whatever it applies must be safe to repeat.
- Declare it on the peer that creates the object: the server, or the client predicting a predicted spawn, whose payload the server passes on. A client cannot change a payload once the object exists.
- Every concrete payload type is registered for you. One without a public parameterless constructor is a build error.
- A system can carry a payload and nothing else: rent it with `NetworkSystemPool.TryRent<NetworkSystem>(coreManager, out NetworkSystem networkSystem)`.

System groups

Using Unity? See Several systems on one object.

A networked object is often several `NetworkSystems`: a body, an inventory, a health pool. A group marks them as one object, so a receiver waits for the whole set before treating it as arrived, and `Despawn()` stops every member together. Members share a `GroupId`, read from `NetworkSystem.GroupId`; an ungrouped system carries `NetworkSystem.UnsetGroupId`.

Build a group

`NetworkSystemGroup` is not replicated itself; it stamps a shared `GroupId` onto its members and spawns or despawns them together.

Initialize it before adding members. `Initialize` rents a fresh `GroupId`:

```
NetworkSystemGroup group = ...;
group.Initialize(coreManager);
```

A client predicting a spawn under a group id the server already leased to it uses `InitializeAdopted`, which stores that id instead of renting one:

```
group.InitializeAdopted(coreManager, groupId);
```

Add and remove members. Adding stamps the group's `GroupId` at once, whether or not the group has spawned.

```
group.AddSystem(bodySystem);
group.AddSystem(healthSystem);
group.RemoveSystem(healthSystem);
```

Spawn and despawn the group

`Spawn()` starts every member not already started. Call it again after adding members and only the new ones start, so some systems can join the object later:

```
group.Spawn();

// Later, once the player unlocks it:
group.AddSystem(inventorySystem);
group.Spawn();
```

`Spawn(networkSystem)` adds one system and starts it in one call:

```
group.Spawn(inventorySystem);
```

`Despawn()` stops every member. Stopping one member with `SystemManager.EnsureStopSystem` stops only that member; the rest stay spawned.

```
group.Despawn();
```

- A group always arrives as one object, even when it is wider than the per-tick budget, so a wide group can take more than one tick to arrive.
- Interest combines every member's despawn and stop votes, so one member out of range, or one member still wanted, can decide what happens to every member.

Read the member count

`NetworkSystem.GroupMemberCount` is the number of systems in the group, or zero when ungrouped. On the server it is the live count. On a receiver it is the count received with the first member, which tells it how many systems to expect for that `GroupId`.

```
uint expected = someSystem.GroupMemberCount;
```

Set controller retention for the whole group

`NetworkSystemGroup.ControllerRetentionPolicyOverride` sets what happens to every member's controller when the controller leaves. A member's own `ControllerRetentionPolicyOverride` wins if set, then the group's, then the engine default. Setting it on the group keeps one object's systems from disagreeing.

```
group.ControllerRetentionPolicyOverride = ControllerRetentionPolicy.Retain;
```

See Controller retention for what `Release` and `Retain` do.

`INetworkSystemGroup` is a marker interface with no members.

Parents, children and hierarchy

Using Unity? See Carrying and re-parenting objects.

A `NetworkSystem` can be attached to another so the two move together and replicate as a unit.

Set a parent on the server

`SystemManager.EnsureSetSystemParent` is the only way to change a started system's parent:

```
bool isParentChanged = CoreManager.SystemManager.EnsureSetSystemParent(crateSystem, carrie
rSystem);
```

Pass `null` to detach the system and return it to the root of its scene:

```
CoreManager.SystemManager.EnsureSetSystemParent(crateSystem, parentNetworkSystem: null);
```

It only succeeds on the server; a client's call is refused and logged. You cannot spawn an object directly into a parent; call `EnsureSetSystemParent` after the spawn.

Only dynamically spawned systems can be children

A scene object, or a system with no platform identity, cannot be a child. Any started system can be a parent. A parent is also refused when:

- it is the same system as the child,
- it is not started and registered,
- it belongs to a different `CoreManager`,
- it is in a different scene than the child (hierarchies cannot cross scenes), or
- attaching it would make the hierarchy circular or deeper than `SystemManager.MaximumParentDepth` (8).

Handle a parent that has not arrived yet

A new observer receives an object together with its current parent. A later change reaches every peer that holds the object, which re-seats it in place rather than spawning it again.

On a receiver, `ParentId` can name a system that has not arrived yet:

```
public uint ParentId { get; }
public bool IsParentResolved { get; }
public event ParentChangedHandler ParentChanged;
```

`IsParentResolved` is true once the system holds a live reference to the parent `ParentId` names. It is false for an unparented system and for one whose parent has not arrived. `ParentChanged` fires on the server as it reparents and on a client as it applies the change, and carries the Ids:

```
void OnParentChanged(uint previousParentId, uint currentParentId)
{
    // ...
}
```

Stopping a parent stops its children

Stopping a parented system stops its children too, deepest first. A child is never shown to a peer that cannot see its parent.

What a system is

A `NetworkSystem` is what Nucleus replicates. The system is the identity and lifecycle; the replicated data lives in the `NetworkComponent` instances attached to it. The NetworkSystem reference lists its identifiers, states and role reads.

A system holds up to 31 components

Read a component with `TryGetComponent<T0>` and count them with `ComponentCount`. The limit is `NetworkSystem.MaximumComponentCount`.

```
if (system.TryGetComponent<TransformComponent>(out TransformComponent transform))
{
    // transform is one of the components attached to this system.
}
```

Each component has a component Id: the order it was added in, starting at zero, and the same on every peer. `TryGetComponent<T0>(out T0)` returns the first component of a type. When a system carries one type several times, as a `NetworkTransform` replicating several parts does, reach each instance by its Id:

```
for (byte componentId = 0; componentId < system.ComponentCount; componentId++)
{
    if (system.TryGetComponent(componentId, out TransformComponent part))
    {
        // part is the component added at position componentId, the same one on every peer
.
    }
}
```

A system does not need an engine object

A purely logical system, such as matchmaking state or a game rule tracker, can start, replicate and stop without any GameObject. In an engine integration, one engine object can link to several systems, each with its own `Id` and lifecycle. In Unity, the component that links an engine object to its system is a separate bridge, not the system itself.

Configuring a transport from launch arguments

Apply launch arguments before connecting

Nothing applies launch arguments for you. Call `Transport.ApplyCommandLineArguments()` after adding the transport and **before** `ConnectAsync`; values changed after the connection starts do not affect it.

```
Transport transport = await coreManager.TransportManager.AddTransportAsync<Synapse>();
transport.ApplyCommandLineArguments();

await transport.ConnectAsync(invoker);
```

To apply arguments that did not come from this process's command line, such as a launcher's configuration, pass a `CommandLineArguments`:

```
CommandLineArguments launcherArguments = new(launcherProvidedArgs);
transport.ApplyCommandLineArguments(launcherArguments);
```

A flag that was not supplied leaves its setting as the build configured it. The call is safe when no arguments are passed.

Use the shipped flags

Flag	Honored by	Sets
`port` (`Transport.PortArgument`)	Every transport	`Configuration.Port`, read as a `ushort`
`address` (`Synapse.AddressArgument`)	`Synapse`	`RemoteHost`, the address the client dials. The server always binds every local address. A value that is not an IP address leaves `RemoteHost` unchanged and logs an error.

```
transport.ApplyCommandLineArguments(new(["Server.exe", "-port", "7777", "-address", "10.0.
0.5"]));
```

Add your own flags

Override `OnApplyCommandLineArguments` and call the base first so `port` still applies:

```
protected override void OnApplyCommandLineArguments(CommandLineArguments commandLineArgume
nts)
{
    base.OnApplyCommandLineArguments(commandLineArguments);

    if (commandLineArguments.TryGetValue(MyCustomArgument, out string value))
    {
        // apply value
    }
}
```

Argument formats the parser accepts

`CommandLineArguments` parses a `string[]` into flag and value pairs.

`CommandLineArguments.Process` parses `Environment.GetCommandLineArgs()` once and keeps the result for the life of the process.

- `-flag value`, `--flag value`, `-flag=value` and `--flag=value` all work.
- Flags are case-insensitive: `-PORT` and `-port` are the same.
- A dash followed by anything other than a letter is a value, so `-5` is read as a negative number.
- Argument zero, the executable path, is ignored.
- A flag with no value after it is recorded with an empty value, as a switch.

```
CommandLineArguments args = new(["Server.exe", "-port", "7777", "--address=10.0.0.5", "-he
adless", "-offset", "-5"]);
```

Read values

`TryGetValue`, `TryGetUInt16` and `TryGetUInt32` return false rather than throw:

```
if (commandLineArguments.TryGetUInt16("port", out ushort port))
    Configuration.Port = port;
```

- `TryGetValue` returns false for a flag never supplied or supplied with no value.
- `TryGetUInt16` and `TryGetUInt32` also return false when the value does not parse as that type.
- `Contains(flag)` reads a switch, where the flag's presence is the information:

```
if (commandLineArguments.Contains("headless"))
{
    // run without a display
}
```

Connecting across networks

Get each hop working in order

Each hop changes one thing, so when it breaks you know which layer to check.

Loopback, one process. Both peers run on `127.0.0.1`, the default **Remote Host** on the Unity `SynapseTransport` component and the default `RemoteHost` in C#. If this fails, the bug is in your game logic, not the network.

LAN, two machines. Point the client's Remote Host at the server machine's LAN IP, such as `192.168.x.x`. The server does not change. This tests that the port is open on the server and no firewall drops the traffic.

Open internet. The client needs the server's public IP or DNS name. This adds the router, and port forwarding, as a failure point.

Set the address on the client and the port on both

- Unity: set **Remote Host** on the `SynapseTransport` component to the server's address, and **Port** to the same value on both sides.
- C#: set `Synapse.RemoteHost` (a host name or IP address) before connecting the client, and `Configuration.Port` to the same value on both peers.

The server never reads Remote Host. By default it listens on every address the machine has (loopback, LAN and public) on its port. What matters is which address the client dials and whether traffic to it reaches the machine.

Check the failures in this order

Wrong port. The client's port does not match the server's. It is the cheapest check, so do it first.

Firewall. The server machine's firewall drops inbound packets to that port.

No port forward. On a home or office network, the router does not pass inbound internet traffic to the server's LAN address without a port forward or UPnP. This only affects the third hop.

Carrier-grade NAT. Some ISPs, and most mobile carriers, put the server behind a NAT you cannot forward through. If forwarding is set up correctly and still fails, this is usually why.

Tell which layer failed

`LocalConnectionState` cannot tell you: a Synapse client reports `Connected` once it sends its handshake, and every failure ends at `Disconnected`. Watch `ClientManager.LocalClientAuthenticated` (in Unity, on `UnityClientManager.NucleusClientManager`), which fires when the server accepts the client:

- **`Disconnected` without `LocalClientAuthenticated`**: the client's packets never reached the server, and it dropped after the idle timeout (`Configuration.ConnectedTimeoutSeconds`, or **Idle Timeout Seconds** on `SynapseTransport`, 15 by default). Check the address, port, firewall and forward. A server that refuses the client drops it at once instead, and `ClientManager.LastAuthenticationDenialReason` holds the reason.
- **`LocalClientAuthenticated`, then `Disconnected`**: the path worked at least once, then traffic stopped. Look for something intermittent, such as an expired NAT mapping, a selective firewall rule or a real network interruption, not the port or forward.

Use the relay when NAT cannot be beaten

If the port, firewall and forward are all right and it still fails over the internet, a peer is likely behind carrier-grade NAT. Use `RelayTransport`: one peer hosts a room and gets a `RoomCode`, and the others join with that code. No address, port or forwarding is needed. In Unity, see Hosting and joining a relay session.

Nucleus does not discover servers on the LAN or attempt NAT punchthrough. The address or room code always comes from you: typed in, read from a config file or fetched from your own service.

Link states and connection events

Invoker names which role acted

A host runs a server and a client in one process. Every state-change event and connect or disconnect result carries an `Invoker` saying which half it is about:

```
public enum Invoker : byte
{
    Client = 0,
    Server = 1,
}
```

Read a connection's local and remote state

`Connection.LocalState` is a `LocalConnectionState`, the state of a socket this peer owns: its own client connection, or on a server one of its client sockets.

```
public enum LocalConnectionState : byte
{
    Disconnected = 0,
    Connecting = 1,
    Connected = 2,
    Disconnecting = 3,
    Error = 4,
    TimedOut = 5,
}
```

A `Connection` nothing has touched reads `Disconnected`. The shipped transports never set `Error` or `TimedOut`: a failed connect and a later drop both end at `Disconnected`.

`Connection.RemoteState` is a `RemoteConnectionState`, what this peer believes about the other side:

```
public enum RemoteConnectionState : byte
{
    Disconnected,
    Connected,
}
```

`ConnectionStateExtensions` adds two checks on `LocalConnectionState`:

```
public static bool IsConnectedOrDisconnected(this LocalConnectionState localConnectionStat
e);
public static bool IsAnyDisconnected(this LocalConnectionState localConnectionState);
```

- `IsConnectedOrDisconnected`: true only for `Connected` or `Disconnected`, the two resting states.
- `IsAnyDisconnected`: true for anything but `Connecting` and `Connected`.

Subscribe to state changes

`TransportManager` raises two events, both carrying a `ConnectionStateChange`:

```
public event ConnectionStateChangedHandler? ConnectionLocalStateChanged;
public event ConnectionStateChangedHandler? ConnectionRemoteStateChanged;

public delegate void ConnectionStateChangedHandler(ConnectionStateChange connectionStateCh
ange);

public readonly struct ConnectionStateChange
{
    public readonly Invoker Invoker;
    public readonly Connection Connection;
    public readonly bool IsRemoteStateChange;
}
```

- `ConnectionLocalStateChanged` fires when `Connection.LocalState` changes.
- `ConnectionRemoteStateChanged` fires when `Connection.RemoteState` changes. On a server, this tells you a client arrived or left.
- `IsRemoteStateChange` tells a shared handler which event it is handling.

A local `Connected` does not mean the server accepted you. A `Synapse` client reaches `Connected` once it has sent its handshake, before the server answers. To know your client has joined, use `ClientManager.LocalClientAuthenticated`, which fires once the server accepts it and assigns its `Id`, or read `TransportManager.IsClientStarted`.

Check connect and disconnect results

These `Transport` calls return a result instead of throwing:

```
public abstract Task<ConnectionStateChangeResult> ConnectAsync(Invoker invoker);
public abstract Task<ConnectionStateChangeResult> DisconnectLocalConnectionAsync(Invoker i
nvoker);
public abstract Task<ConnectionStateChangeResult> DisconnectRemoteClientAsync(Connection c
onnection);

public enum ConnectionStateChangeResult : byte
{
    Success,
    AlreadyInState,
    InvalidConnection,
    InvalidSocketState,
    UnspecifiedError,
}
```

`AlreadyInState`, `InvalidConnection` and `InvalidSocketState` mean the call was redundant or the link was already gone. Log `UnspecifiedError`.

Track peers with PeerConnectionDiscovered and PeerConnectionDropped

```
public event PeerConnectionChangedHandler? PeerConnectionDiscovered;
public event PeerConnectionChangedHandler? PeerConnectionDropped;

public delegate void PeerConnectionChangedHandler(Connection connection);
```

These `TransportManager` events report peers the same way on a server, client or host. On a client, the `Connection` is a stand-in with no link behind it (`Connection.IsPeerStandIn`). `PeerConnectionDropped` fires while the `Connection` is still valid; clear anything that references it, and

keep no reference after the handler returns.

Store a handle, not a Connection

`Connection` objects are reused for later peers, so do not keep a reference. Read `LocalState`, `RemoteState` and `Transport` while you hold it, and store `Connection.Handle` (a `ConnectionHandle`) to remember the peer. A handle pairs the `Id` with a `Generation`, so a handle to a departed peer fails to resolve rather than naming whoever reused the `Id`:

```
public bool TryGetConnection(ConnectionHandle connectionHandle, out Connection connection)
```

Resolve it each time you need it, and do not cache the `Connection` it returns either.

Running a dedicated .NET server

> **Using Unity?** See Building a headless Unity server.

A dedicated server is a plain console application that references Nucleus. It needs no scene.

Reference the engine

There is no NuGet package. Reference `Nucleus.dll`, `CodeBoost.dll` and `SynapseSocket.dll` as a set from one release. A project that declares networked types also needs `Nucleus.CodeAnalysis.SourceGenerators.dll` from the same release as an `Analyzer` item, or those types cannot be sent. Adding Nucleus to a .NET project shows the project file.

`Nucleus.dll` targets `netstandard2.1`. Target a modern framework such as `net8.0` or `net9.0` for your own executable.

Write the program

```
using System.Diagnostics;
using Nucleus.Connections;
using Nucleus.Managers.Core;
using Nucleus.Managers.NetworkLoop;
using Nucleus.Managers.Transports;
using Nucleus.Transports.Synapse;

// Name your own provider here so the default thread-pool timer never starts.
MainThreadStepProvider loop = new();
CoreManager coreManager = new(networkLoopStepProvider: loop);
TransportManager transportManager = coreManager.TransportManager;

// Nothing steps the loop until this thread does, so this bring-up cannot race it.
Synapse synapse = (Synapse)await transportManager.AddTransportAsync<Synapse>();
synapse.Configuration.Port = 7777;

await synapse.ConnectAsync(Invoker.Server);

// Ctrl+C and a container stop (SIGTERM raises ProcessExit) both end the loop below.
CancellationTokenSource stopSource = new();
ManualResetEventSlim loopExited = new();
Console.CancelKeyPress += (_, eventArgs) =>
{
    eventArgs.Cancel = true;
    stopSource.Cancel();
};
AppDomain.CurrentDomain.ProcessExit += (_, _) =>
{
    stopSource.Cancel();
    loopExited.Wait(TimeSpan.FromSeconds(5));
};

// Step the network loop on this thread until a stop is requested.
Stopwatch frameClock = Stopwatch.StartNew();
while (!stopSource.IsCancellationRequested)
{
    float frameDeltaMilliseconds = (float)frameClock.Elapsed.TotalMilliseconds;
    frameClock.Restart();

    loop.Step(frameDeltaMilliseconds);
    Thread.Sleep(1);
}

await synapse.ShutdownAsync();
coreManager.Deinitialize();
loopExited.Set();

/// <summary>Steps the network loop only when called, on the calling thread.</summary>
sealed class MainThreadStepProvider : INetworkLoopStepProvider
{
    private readonly NetworkLoopStepDriver _driver = new();

    public bool IsStarted { get; private set; }

    public bool IsTickFrame => _driver.IsTickFrame;

    public void Initialize(NetworkLoopManager networkLoopManager) => _driver.Initialize(ne
```

```
tworkLoopManager, 1000f / networkLoopManager.TickRate);

    public void Start() => IsStarted = true;

    public void Stop() => IsStarted = false;

    public void Return() { }

    public void Step(float frameDeltaMilliseconds)
    {
        if (!IsStarted)
            return;

        _driver.AdvanceEarly(frameDeltaMilliseconds, isVariableUpdateAllowed: true);
        _driver.AdvanceLate(frameDeltaMilliseconds, isVariableUpdateAllowed: true);
    }
}
```

- Pass your own step provider. With none, `new CoreManager()` starts a default one on a thread-pool timer before the constructor returns, so adding the transport and connecting race a loop that is already running.
- Set `Configuration.Port` before `ConnectAsync(Invoker.Server)`, which starts listening.
- The `while` loop passes the time since its previous pass to the provider, whose `NetworkLoopStepDriver` decides when a tick runs. See Driving the loop yourself.

Callbacks run on the thread that steps the loop

With the provider above, every engine callback runs on `Main`'s thread inside `loop.Step`, so code in the `while` loop can share state with callbacks without locks. Only the two stop handlers run elsewhere, which is why they touch nothing but `stopSource` and `loopExited`.

The default provider runs every callback on a thread-pool thread instead. If you use it, treat state shared between `Main` and your handlers as cross-thread. See Threading and lifetime.

Shut down cleanly

Shut down the transports, then the manager. `Deinitialize()` is safe to call more than once.

```
await synapse.ShutdownAsync();
coreManager.Deinitialize();
```

In a container, run this on `SIGTERM` so clients get a clean disconnect instead of a dropped socket. The program above does: .NET raises `ProcessExit` on `SIGTERM`, the handler cancels `stopSource` and waits, and `Main` tears down on the thread that was stepping the loop. Do not call `ShutdownAsync` or `Deinitialize` from the handler itself, because `Main` would still be stepping the session.

Route engine logs

Nucleus logs through `CodeBoost.Logging.LoggingService`, which starts with a `ConsoleLogger`. To send logs elsewhere, implement `CodeBoost.Logging.ILogger` and call this before `new CoreManager()`:

```
LoggingService.UseLogger(myLogger);
```

Take the port from the command line

To set the port and bind address at launch, see Configuring a transport from launch arguments.

Choosing a hosting topology

Pick one of three shapes

A `CoreManager` can run its server half, its client half, or both. That choice is what makes a dedicated server, a player-hosted session or a relayed room.

- **Dedicated server.** A process runs only the server half, and every player connects as a client.
- **Host.** One player's process runs both halves, so that player is a client of their own server. It runs the same server and client code as every other shape; see `HostPairing` on The Synapse transport for how the halves connect.
- **Relayed room.** A host behind a `RelayTransport`. Host and joiners all connect out to a Blitz Relay server, which pairs them under a room code. No peer accepts inbound connections, so there is no port to forward and no NAT to fight.

	Dedicated server	Host	Relayed room
Who runs a machine	You, continuously	The hosting player	The relay operator, continuously
Reachability	Server needs an open, reachable port	Host needs an open, reachable port on the player's connection, which often fails behind carrier-grade NAT or a strict firewall	Neither peer needs to be reachable
Session lifetime	Independent of players	Ends when the host's process does	Ends with the host, unless paired with a directory handover
Latency	A fixed, usually central server	Good for the host, variable for everyone else	Adds a hop through the relay

The server is in charge in every shape

Every shape has exactly one server: one `CoreManager` running the same server logic. A host's server simply runs in a player's process, so decide whether you trust that player. A relay only forwards packets under a room code instead of a socket address; the host still authenticates and simulates as it would directly, and no peer gains anything by being relayed.

Keep a session alive after the host leaves

A plain host's session ends when the host's process does. For a relayed session to survive, pair it with a directory:

- `ISessionHost` is what a directory implementation uses to publish where a session is hosted (`HostedCredential`) and to join wherever the directory points (`JoinAsync`).
- `NewfarmHostMigration` polls the directory, so a session can hand its hosting role to another peer without clients being told a new address by hand. It is Pro, and it adopts the retained world when it promotes a peer, so the game state survives the handover.

This pairing is not engine-level host migration.

Bring your own services

Nucleus does not run matchmaking, a lobby service or a hosted relay. `RelayTransport` and `ISessionHost` connect to infrastructure you run or a service you choose. No shape limits which shipped transport you can use.

How transports work

A `Transport` only moves bytes between peers: it connects, disconnects, sends and receives datagrams, and reports whether a connection is its own client talking to its own server (`IsOwnClientConnection`). Replication, spawning and reconciliation work the same on every transport, so you can swap one for another without changing game code.

Pick a transport

Transport	What it is	When to use it
Synapse	Real UDP sockets.	Direct connections and dedicated servers, wherever the server has a reachable address and port.
Yak	Offline and in-process; nothing is bound.	Single-player or same-process testing where no socket should exist.
Blitz Relay	A third machine carries the traffic.	Peers that cannot reach each other directly, such as both behind NAT with no port forwarding. Costs relay overhead and depends on that machine.

A `CoreManager` can hold several transports at once. Add each with `TransportManager.AddTransportAsync` or `TryAddTransportAsync`, then connect with a separate `ConnectAsync` call. See TransportManager. In Unity, `UnityTransportManager` does this for you; see Transport Manager component.

Every send picks a channel

- `Channel.Reliable`: ordered and reliable.
- `Channel.Unreliable`: unreliable.

State, remote calls and messages default to `Channel.Unreliable`. A message or remote call too large for one packet still arrives whole: a reliable one in order, an unreliable one whole or not at all, on every transport including the relay.

The transmission unit limits the datagram, not the payload

`Configuration.MaximumTransmissionUnit` defaults to 1200 bytes. Each transport reports `DatagramOverheadBytes`, its per-datagram header, and that comes out of the 1200 before your data. The relay's overhead is larger than Synapse's, so a relayed session carries less per packet than a direct one at the same setting.

To build your own transport, see The Transport contract.

Transports, Connectivity and Hosting

Pages in this category:

- The Synapse transport
- TransportManager
- How transports work
- Running a dedicated .NET server
- Link states and connection events
- The Yak offline transport
- Choosing a hosting topology
- Configuring a transport from launch arguments
- Connecting across networks
- Packet transforms
- Simulating latency, jitter and loss
- The relay transport
- The Transport contract

Packet transforms

A packet transform rewrites the bytes of every packet to and from a transport, so you can add encryption, compression, obfuscation or an integrity check without changing any send or receive code. It is a Pro feature; Free builds have no `IPacketTransform` or the `TransportManager` hooks.

A transform sees everything the engine hands the transport. It does not see packets the transport sends on its own, such as its handshake or keep-alives.

Implement IPacketTransform

```
public interface IPacketTransform
{
    uint ReservedBytes { get; }

    bool TryTransform(PacketTransformDirection packetTransformDirection, Connection connec
tion, Channel channel, ReadOnlySpan<byte> source, Span<byte> destination, out int written
ength);
}
```

`TryTransform` runs once per packet, in each direction:

- `PacketTransformDirection.Outbound`: the packet is about to go to the transport.
- `PacketTransformDirection.Inbound`: the packet just arrived from the transport and has not been read.

Read `source`, write the result into `destination`, and set `writtenLength`. `destination` is always at least `source.Length` + `ReservedBytes` long. Keep it allocation-free; it runs for every packet.

- Return `true` when the rewrite succeeded.
- Return `false` to discard the packet, for example when an inbound integrity check fails. A discarded inbound packet raises `PacketTransformRejectedViolation`, carrying the `Channel` and its length before the transform ran.

`ReservedBytes` is the most the transform may add to an outbound packet. It comes out of the transport's `MaximumTransmissionUnit`; `TransportManager.GetMaximumTransmissionUnit` reports the reduced figure.

Install it on both peers before connecting

```
coreManager.TransportManager.PacketTransform = new MyPacketTransform();
```

Set `TransportManager.PacketTransform` before starting the transport. Both peers must run an equivalent transform before either connects; a mismatch is what `PacketTransformRejectedViolation` reports.

In Unity there is no component. Set it the same way before starting:

```
unityCoreManager.CoreManager.TransportManager.PacketTransform = new MyPacketTransform();
```

Key it on something both peers already hold

Use a build key, or something the transport itself established. Never use anything Nucleus negotiates: authentication travels through the transform too, so a key that only exists after authentication cannot unlock the packets that authenticate.

Do not ship ExampleXorPacketTransform

`ExampleXorPacketTransform` shows the length accounting and control flow a transform needs. It provides no security, because anyone reading the packet can undo it without a key. Use an authenticated cipher such as AES-GCM for real traffic.

The relay transport

Using Unity? See Blitz Relay Transport component.

`RelayTransport` connects peers through a Blitz Relay server. Neither peer listens: both connect out to the relay, one opens and holds a room, and the rest join with the room's code. No peer needs an inbound port, a public IP or a NAT hole.

`RelayTransport` and `RelaySessionHost` live in the `Nucleus.Integrations.BlitzRelay` assembly. The relay server and the newfarm directory server are separate services that you run or choose.

Configure it

Member	Purpose
`RelayEndPoint`	The relay to use. Defaults to loopback on `DefaultRelayPort` (7770).
`ConnectionKey`	The relay operator's key the relay admits peers with, not a player's key.
`RoomCode`	Set it before connecting a client, to name the room to join. Read it after connecting a server, to get the room the relay assigned.
`HostedRoomCode`	The room this peer is hosting, or empty when its hosting side is not up. For a peer that only joined, `RoomCode` is the joined room and this is empty.
`MaximumClients`	How many clients a room this peer creates holds. Defaults to `DefaultMaximumClients` (16). Used only when `ServerConfiguration.MaximumConnections` is not set, because a room must be given a size when it is made.
`RelayHandshakeTimeoutMilliseconds`	How long to wait for the relay to answer a handshake. Defaults to 10000.

`RelayTransport.DatagramOverheadBytes` includes the relay's own header, so a relayed session carries less per packet than a direct one.

When the relay closes the room

If the relay closes the host's link, for example because the relay shut down or dropped the room, the host's server stops on the next poll and disconnects every client. The usual server-stopped and client-disconnected callbacks fire, so you need no extra handling. To keep playing, open a new room or let `NewfarmHostMigration` do it.

Survive a lost host with NewfarmHostMigration

A room dies with the peer that made it, and its replacement gets a name nobody knew in advance. `NewfarmHostMigration` solves this with a newfarm directory: it registers the session, notices the host going, and either takes the session over or rejoins wherever the directory says it moved. This is the relay's

own handover, not engine-level host migration.

```
NewfarmHostMigration hostMigration = new(coreManager, sessionHost, directoryIpEndPoint);
```

Call it:

- `StartHostingAsync(uint timeoutMilliseconds = DefaultTimeoutMilliseconds)`: opens a session with the directory and hosts it on this peer.
- `JoinAsync(ulong sessionId, uint timeoutMilliseconds = DefaultTimeoutMilliseconds)`: finds a session by id and joins wherever it is. A join can end with this peer elected to host, which still reports success.
- `TrySurrenderHosting()`: hands the session to whoever the directory elects next; this peer stays as a client.
- `CloseSession()`: ends the session for everybody.
- `Poll()`: call once a frame or once a tick. It drives the directory link and the `ISessionHost`, and watches for the host going quiet.

Read:

- `State`: a `NewfarmMigrationState` describing what this peer is doing about the session.
- `SessionId`: the id the directory issued. The host gives it to clients so they can find their way back.

Handle:

- `Promoting`: raised on the peer chosen to take over, before it does. The session is not hosted again until the handler returns.
- `Promoted(string credential)`: this peer is now hosting, at the credential the service named.
- `Rejoined(string credential)`: a surviving peer has rejoined the session where it moved.
- `Abandoned(string reason)`: the session could not continue, because the directory refused it or no peer would host it.

NewfarmHostMigration needs Pro

A client kept with `DisconnectResetMode.RetainReceivedWorld` retains the world it had when its link dropped, and the promoted server adopts that world through `ServerManager.TryAdoptRetainedWorld`, which only Pro has. So `Nucleus.Integrations.Newfarm`, and `Nucleus.Integrations.BlitzRelay`, which references it, do not compile against a free `Nucleus.dll`.

Implement ISessionHost for another service

`NewfarmHostMigration` drives any session service through `ISessionHost`:

Member	Role
`AdapterTag`	Names the service. It is filed with every credential this publishes, so a receiving peer knows whether it can use it.
`HostedCredential`	What another peer needs to reach the session this peer hosts, or empty when not hosting.
`IsHostingLinkUp`	Whether this peer's hosting side is up.

`IsJoinedLinkUp`	Whether this peer's link to the current host is up. Independent of `IsHostingLinkUp`: a freshly promoted peer has one up and the other down.
`Poll()`	Drives the service, once per `NewfarmHostMigration.Poll`.
`StartHostingAsync(uint timeoutMilliseconds)`	Hosts the session on this peer, returning a `SessionHostResult`.
`JoinAsync(string credential, uint timeoutMilliseconds)`	Joins the session at the credential.
`StopHostingAsync()`	Stops hosting; `HostedCredential` becomes empty and `IsHostingLinkUp` false.
`LeaveAsync()`	Leaves a joined session; `IsJoinedLinkUp` becomes false.

`SessionHostResult` values:

- `Success`.
- `CredentialUnreachable`: the service answered but the room is not there or will not take this peer. The directory acts on this.
- `LocalFailure`: this peer could not try. It is never reported to the directory.

`RelaySessionHost` is the relay's implementation. Its `AdapterTag` is `"blitzrelay"`, `HostedCredential` reads `RelayTransport.HostedRoomCode`, and the four async calls connect and disconnect the transport's server and client sockets.

Simulating latency, jitter and loss

Using Unity? See Simulating a bad connection in the editor.

A loopback connection is instant and lossless. A transport can make it behave like a real connection so you can test bad-link behavior on demand.

Describe the connection with NetworkConditions

`NetworkConditions` is a struct in `Nucleus.Transports`. Its default simulates nothing.

Field	Meaning
`LatencyMilliseconds`	One-way delay added to every packet sent.
`JitterMilliseconds`	Extra delay on top of the latency: each packet gets a fresh random amount below this figure, so packets arrive at different times.
`PacketLossChance`	Chance, from 0 to 1, that a packet is dropped. `0.2f` drops one in five.
`OutOfOrderChance`	Chance, from 0 to 1, that a packet is held back until after packets sent later.
`DuplicateChance`	Chance, from 0 to 1, that a packet is also sent a second time.
`Seed`	Fixes the random choices, so a test repeats them for the same packets. `0` (default) varies them every run. A networked run can still differ, because how many packets are sent and when depends on timing. To replay a failing run, pick a seed, log it and set it.

```
using Nucleus.Transports;

NetworkConditions networkConditions = new()
{
    LatencyMilliseconds = 100,
    JitterMilliseconds = 40,
    PacketLossChance = 0.1f,
};
```

Apply the conditions

Add the transport.

Set the conditions. `Synapse` applies them at once, even on a link that is already connected, so you can also turn them on, change them or turn them off mid-session.

Connect.

```
// Every transport.
coreManager.TransportManager.SetNetworkConditions(networkConditions);

// Only Synapse transports.
coreManager.TransportManager.SetNetworkConditions<Synapse>(networkConditions);

// Pass null to go back to a clean connection.
coreManager.TransportManager.SetNetworkConditions(null);
```

If no transport of the named type has been added, the call logs a message and does nothing. You can also call `transport.SetNetworkConditions(networkConditions)` and `transport.GetNetworkConditions()` on one transport.

To read the conditions back, ask for one type or for every transport. The list for every transport is pooled, one entry per transport in `TransportManager.Transports` order; return it when done:

```
NetworkConditions? synapseConditions = coreManager.TransportManager.GetNetworkConditions<S
ynapse>();

List<NetworkConditions?> everyTransportsConditions = coreManager.TransportManager.GetNetwo
rkConditions();
// Read the entries here.
ListPool<NetworkConditions?>.Return(everyTransportsConditions);
```

Conditions belong to a transport. A test running a server `CoreManager` and a client `CoreManager` side by side sets each one separately.

Know what each transport supports

- A transport that cannot simulate logs a warning when you set conditions, and reads back null. One that can may support only some fields.
- `Synapse` supports everything except `DuplicateChance`, which it ignores. It holds an out-of-order packet back by up to 100 ms.
- `Synapse` sends its connection setup packets, the handshake and NAT traversal probes, with no simulated delay or loss. A connection always opens, so a successful connect proves nothing about the handshake on that link.

Loss applies to each direction separately

Each peer drops packets it sends. With `PacketLossChance = 0.2f` on both peers, a round trip survives less than 80% of the time, because either leg can be dropped.

Check the result

- Confirm the peer ends up holding exactly what the server holds, despite the drops.
- Read what the link actually measured from `Connection`, to confirm the conditions took effect:
  - `RoundTripTimeMilliseconds`: the smoothed round trip time.
  - `RoundTripTimeDeviationMilliseconds`: how far samples typically stray from it, which is the measured jitter.
  - `PacketLossPercentage`: the measured loss.

The Synapse transport

Using Unity? See Synapse Transport component.

Synapse is Nucleus's production UDP transport. Every setting is a plain C# property or field you set before connecting.

Add and connect it

```
Synapse synapse = (Synapse)await transportManager.AddTransportAsync<Synapse>();
synapse.RemoteHost = IPAddress.Loopback;

await synapse.ConnectAsync(Invoker.Server);
await synapse.ConnectAsync(Invoker.Client);
```

- `RemoteHost` is the address the client dials. It defaults to `IPAddress.Loopback`; set it before `ConnectAsync(Invoker.Client)` to reach another machine. The server always binds every local address, so `RemoteHost` does not affect it.
- A host calls `ConnectAsync` on the same `Synapse` instance twice, once with `Invoker.Server` and once with `Invoker.Client`.
- `SetPort(ushort)` sets both `Configuration.Port`, which the server binds, and `RemotePort`, which the client dials.

Configure it

`Synapse.Configuration` is the shared `Configuration` struct:

Field	Default	Notes
`Port`	`0`	The server's listen port and the port the client connects to. The Unity component defaults it to `7777`; the engine does not.
`MaximumTransmissionUnit`	`1200`	Larger packets are split.
`ConnectedTimeoutSeconds`	`15`	How long a connection waits without hearing from the other side before it times out. Never less than one second.
`ConnectingTimeoutSeconds`	`10`	How long a client waits for the server to answer its connect before returning to `Disconnected`. Zero uses `ConnectedTimeoutSeconds`. Client only.

Cap how many clients the server accepts with `synapse.SetMaximumConnections(uint)`. It sets `ServerConfiguration.MaximumConnections`, which is read-only to your code. `ServerConfiguration.UnsetMaximumConnections` (`0`, the default) accepts any number. The cap applies the next time the server starts. A client over the cap gets no answer, so it gives up as it would on a server that never replied.

Finding out why a connection dropped

A `Disconnected` connection carries no reason. Subscribe to `ViolationObserved` on the `Synapse` transport; it is raised for every violation the socket layer reports, just before the connection is reported `Disconnected`.

```
using SynapseSocket.Core.Events;

Synapse synapse = (Synapse)await coreManager.TransportManager.AddTransportAsync<Synapse>()
;
synapse.ViolationObserved += OnViolationObserved;

void OnViolationObserved(Connection? connection, in ViolationEventArgs violationEventArgs)
{
    if (violationEventArgs.Reason is ViolationReason.Timeout or ViolationReason.ReliableEx
hausted)
    {
        // Record why this connection is about to drop.
    }
}
```

- `Reason`: `Timeout` when nothing was heard within `ConnectedTimeoutSeconds`, `ReliableExhausted` when a reliable message ran out of resends, `PeerDisconnect` when the other side disconnected, and the security reasons when a peer sends something malformed, oversized or too fast.
- `Action` is the response already decided. The event cannot change it.
- `connection` is the remote client on a server (still registered) and the server on a client. It is null for a sender that never connected.

The event is raised on the network loop thread. A handler must not block, stop the transport or disconnect anyone.

Start and stop it

- `ConnectAsync(Invoker.Server)` starts listening for clients.
- `ConnectAsync(Invoker.Client)` connects the local client to `RemoteHost`.
- `DisconnectLocalConnectionAsync(Invoker)` disconnects the local server or client and closes its sockets.
- `DisconnectRemoteClientAsync(Connection)` disconnects one remote client from the server.
- `ShutdownAsync()` stops all local connections.

Choose how a host pairs with itself

`HostPairing` sets how a host's server and client halves connect. Set it before the client connects.

- `HostPairing.Endpoint` (default): the halves talk over the socket like any remote pair, so a host exercises the same path a remote client does.
- `HostPairing.Local`: the halves are paired in process and never exchange a datagram. It applies only when the client dials a loopback address with its own server already running.

`IsOwnClientConnection(Connection)` reports whether a connection is the host's own client half. `IsLocalTransport()` returns `false`, even under `HostPairing.Local`.

`Synapse.DatagramOverheadBytes` is the socket layer's largest per-datagram header, which comes out

of the transmission unit.

The Transport contract

`Transport` is the abstract base every connection method derives from, whether socket-backed like Synapse, offline like Yak, or your own. `Transport`, `SocketPairTransport<TServerSocket, TClientSocket>` and `CommonSocket` are public, so a custom transport can live in your own assembly. See Yak for a minimal offline transport and Synapse for a socket-backed one.

Implement the required members

`InitializeAsync(CoreManager coreManager, uint transportId)`: call base, which assigns `CoreManager` and `Id`, then do your own setup.

`ConnectAsync(Invoker invoker)`: a client invoker connects the local client to the server; a server invoker starts listening.

`OnSendPacket(OutgoingPacket outgoingPacket)`: transmit the packet. Never override `SendPacket`, which applies the packet transform and counts bytes before calling this.

`PollForIncomingPackets(Invoker invoker)`: required, but the engine never calls it. Poll inside `ReceivePackets` or from your own method.

`ReceivePackets(Invoker invoker, ref List<IncomingPacket> receivedPackets)`: called once per frame for each running half (the server while listening, the client while connecting or connected). Add every packet that arrived since the last call.

`TryGetConnection(Invoker invoker, out Connection connection)`: return the `Connection` for that invoker's socket.

`IsOwnClientConnection(Connection serverSideClientConnection)`: whether a connection the server holds is this transport's own client half. If the transport cannot tell, return `false`. That only costs a host in-process delivery of its own traffic; a wrong `true` hands a stranger the server's own half.

`DisconnectLocalConnectionAsync(Invoker invoker)`: disconnect a local connection and close its sockets.

`DisconnectRemoteClientAsync(Connection connection)`: disconnect a remote client from the server.

`ShutdownAsync()`: stop all local connections and clean up.

Override the virtuals you need

Virtual	Default	Override when
`DatagramOverheadBytes`	`0`	The transport adds its own header to every datagram. Return its size so packets still fit.
`OnApplyCommandLineArguments(ApplicationCommandLineArguments commandLineArguments)`	Applies `Port` to `Configuration.Port`.	The transport has its own endpoint settings. Call base, then read your flags.
`IsLocalTransport()`	Logs a warning and returns `false`.	The transport only runs locally and offline. Several security checks are disabled while it returns true.

`SetPort(ushort port)`	Logs a warning.	The transport has a port to set.

Derive from SocketPairTransport for a server socket plus a client socket

`SocketPairTransport<TServerSocket, TClientSocket>` implements the whole contract for a transport with one server socket and one client socket. Supply the two socket types, both deriving from `CommonSocket`, and only what is specific to your transport.

- Both sockets are rented and initialized in `InitializeAsync`. Override `OnSocketsInitialized()` to prepare the pair together once both are ready.
- Every other member routes to the socket for the calling `Invoker`. Reach them through `ServerSocket` and `ClientSocket`.
- In your sockets, override `CommonSocket`'s connect, disconnect, send and receive members, declaring each override `protected`.

Call back into the manager

A transport reports state through these `TransportManager` calls:

- `RentInitializedConnection(CommonSocket commonSocket)`: rents and initializes a `Connection` for a socket.
- `ChangeLocalConnectionState(Invoker invoker, LocalConnectionState nextLocalConnectionState, Connection? connection)`: changes a connection's local state and raises callbacks.
- `SetRemoteConnectionStateAsConnected(uint idBySocket, Connection connection)`: server-only; marks a connection remotely connected and raises callbacks.

To report a client leaving, the socket asks its transport to call its protected, server-only `SetRemoteConnectionStateAsDisconnected(Connection connection)`. Game code drops a client with `Connection.Kick()` instead.

Give each received payload exactly one owner

`CommonSocket` has two helpers that copy a received payload into a buffer rented from `ArrayPool<byte>.Shared`:

- `CopyIntoOwnedPayload(ArraySegment<byte> receivedPayload, out byte[] rentedArray)` copies a payload that is only lent for the receive callback, so the copy outlives it. The caller owns the rented array.
- `TryCopyIntoIncomingPacket(OutgoingPacket outgoingPacket, out IncomingPacket incomingPacket)` copies an `OutgoingPacket` into a new `IncomingPacket`. Offline transports such as `Yak` use it.

Whichever code queues the resulting `IncomingPacket` owns the array from then on. Any path that does not queue it must return the array to the pool itself. Two owners, or none, corrupt the pool.

TransportManager

Using Unity? See Transport Manager component.

`CoreManager.TransportManager` owns the added `Transport` instances, tracks every `Connection`, and answers role and peer queries.

Add a transport

`AddTransportAsync<T0>()` creates a `T0 : Transport` with its parameterless constructor, adds and initializes it, and returns it:

```
Transport transport = await coreManager.TransportManager.AddTransportAsync<MyTransport>();
```

When the transport needs constructor arguments or other setup, build it yourself and pass it to `TryAddTransportAsync(Transport transport)`. It returns `false`, without adding it again, if that reference is already in `Transports`:

```
MyTransport transport = new(customConfig);
bool added = await coreManager.TransportManager.TryAddTransportAsync(transport);
```

Remove a transport

- `TryRemoveTransport(Transport transport)` removes and shuts down one transport, returning `false` if it was not found.
- `RemoveTransports<T0>()` removes and shuts down every transport of type `T0`, returning them as a `List<Transport>`.

`Transports` (`IReadOnlyList<Transport>`) lists everything currently added.

Check the local role

- `IsServerStarted`: this peer's local server connection is `LocalConnectionState.Connected`.
- `IsClientStarted`: this peer's local client connection is `LocalConnectionState.Connected` **and** `IsAuthenticated`. A connected but unauthenticated client reads `false`.
- `IsHostStarted`: `IsServerStarted` && `IsClientStarted`.

Find a connection

- `TryGetConnection(uint id, out Connection connection)` finds a connection by id, including a host's own client connection. Returns `false` for `Connection.UnsetId` and for an id this peer was never told about.
- `TryGetConnection(ConnectionHandle connectionHandle, out Connection connection)` succeeds only while `connection.Generation` still matches `connectionHandle.Generation`. Ids and `Connection` instances are reused, so a bare id can later name a different peer. Store a handle to remember a peer longer than its link lasts.
- `TryGetServerConnection(out Connection connection)` returns this peer's server connection.
- `TryGetLocalClientConnection(out Connection connection)` returns this peer's client connection; a pure server has none and returns `false`.
- `ActiveConnections` holds every `Connection` this peer knows. On the server, these are the connected remote clients. On a client, they are stand-ins for peers the server reported, never the local client itself.

```
if (transportManager.TryGetConnection(storedHandle, out Connection connection))
{
    // connection is still the peer storedHandle was taken from.
}
```

Read the channel and transmission unit

- `DefaultChannel` is the `Channel` used when nothing more specific is asked for. Default `Channel.Unreliable`.
- `GetMaximumTransmissionUnit(Transport transport)` returns how many bytes of that transport's transmission unit the engine may fill, after any packet transform reservation. Ask here rather than reading the transport's configuration.

Leave HostLoopbackDelivery at Immediate

`HostLoopbackDelivery` controls how a host delivers a message or call its server half sends to its own client half, or the reverse. From game code it can only be `HostLoopbackDelivery.Immediate`: handlers run before the send call returns. Setting anything else is refused and logged.

`RentInitializedConnection`, `ChangeLocalConnectionState`, `SetRemoteConnectionStateAsConnected` and `SetRemoteConnectionStateAsDisconnected` are for transport implementations; see The Transport contract.

The Yak offline transport

`Yak` runs a server and client in the same process, on the same `CoreManager`, with no socket opened. Systems start, spawn and replicate as they would on a real transport. Use it for single-player, for tests and tools that need a real client-host pair without a socket, or to start a session offline and bring up a real transport once a player chooses to host.

Yak tells you nothing about what a real socket does: packet loss, latency, splitting large packets, or timing. Test those on Synapse.

Add and connect it

Connect both `Invoker.Server` and `Invoker.Client` on the same `Yak` instance:

```
Yak yak = new();

if (!await transportManager.TryAddTransportAsync(yak))
{
    // handle failure
}

_ = yak.ConnectAsync(Invoker.Server);
_ = yak.ConnectAsync(Invoker.Client);
```

No Unity component ships for Yak. In Unity, add it to the `CoreManager` from a script.

Peers on Yak count as local

`Yak.IsLocalTransport()` returns `true`. A connection counts as a local peer when it is emulated (`Connection.IsEmulated`), or when its transport is local and that transport's own client half is connected, which is always so on Yak. A local peer skips the interest and scene-completion checks a remote peer goes through, and in Pro it is never held back by object pacing.

Swap Yak for Synapse without restarting

A `CoreManager` that started offline on Yak can replace it with Synapse:

```
await coreManager.TransportManager.RemoveTransports<Yak>();

Synapse synapse = (Synapse)await coreManager.TransportManager.AddTransportAsync<Synapse>()
;
synapse.Configuration.Port = port;
synapse.RemoteHost = IPAddress.Loopback;

await synapse.ConnectAsync(Invoker.Server);
await synapse.ConnectAsync(Invoker.Client);
```

Started systems keep their state and ids. Another peer can then join over the real socket and receive the same world.

Compression, Accuracy and Per-Type Registrations

Choose a compression level

`CompressionLevel` has four values:

- **Aggressive**: the most compact for small changes. Less efficient on mid-range values. Use it for a quantity that mostly sits still and moves in small steps.
- **Tight**: balanced across the common range of change sizes.
- **Loose**: suited to values that can be large or jump around.
- **Uncompressed**: no compression.

`CompressionLevelExtensions.IsLooseOrUncompressed(this CompressionLevel)` returns true for `Loose` or `Uncompressed`.

The defaults differ by place:

Where	Default
`NetworkMember<T0>` constructor, `deltaCompressionLevel`	`CompressionLevel.Tight`
Every `Writer.Write*` method, `compressionLevel`	`CompressionLevel.Loose`, so `writer.WriteInt32(value)` is Loose unless you pass a level
`Writer.DeltaCompressionLevel`, the writer's fallback	`CompressionLevel.Tight`

Set accuracy per member

`NetworkMember<T0>`'s constructor also takes `accuracy`, `float.NaN` by default, which uses the type's default:

- `Writer.DefaultFloatingAccuracyAsSingle`: `1/1000`, the default for `float`, `double` and `decimal`.
- `Writer.MinimumFloatingAccuracyAsSingle`: `1/100000`, the finest accuracy allowed.
- `Writer.DefaultQuaternionAccuracyAsSingle`: `1/1000`, the default for `Quaternion` members.

Give a coarser accuracy, such as `0.01f`, to a value that tolerates more error, like a velocity. An accuracy passed to `Initialize` overrides the one given at construction.

Accuracy is also the change threshold: a change smaller than `accuracy` is not sent. A velocity with `0.01f` stops reporting movement smaller than that.

Choose a DeltaCheckMode

Every delta write method (`TryWriteDeltaInt32`, `TryWriteDeltaSingle`, `TryWriteDeltaBoolean` and so on) takes a `DeltaCheckMode`:

- **Checked**: writes only if the value changed.
- **Unchecked**: always writes, skipping the comparison. Use it when you already know the write is needed.

Writing a Custom Serializer

Write one only when you need a specific encoding

Every networked type the generator can reach is serialized for you. Write your own serializer for a packed encoding, a custom precision trade-off, or a type the generator cannot reach. A hand-written serializer always wins over a generated one for the same type, whichever is registered first.

If the type only needs to be visible to the generator, use a marker struct instead; see Reaching Types You Cannot Annotate.

Register with attributes

Mark static extension methods with these attributes from `Nucleus.Serializers`:

Attribute	Method shape
`[DefaultWriter]`	Extends `Writer`; takes the value and a `CompressionLevel`.
`[DefaultReader]`	Extends `Reader`; takes a `CompressionLevel` and returns the value.
`[DefaultDeltaWriter]`	Extends `Writer`; takes a previous value, a next value and a `DeltaCheckMode`, and returns `bool`.
`[DefaultDeltaReader]`	Extends `Reader`; takes a previous value and returns the next value.

```
[DefaultWriter]
public static void WriteMyType(this Writer writer, MyType value, CompressionLevel compress
ionLevel)
{
    // write value's fields through writer
}

[DefaultReader]
public static MyType ReadMyType(this Reader reader, CompressionLevel compressionLevel)
{
    // read fields back through reader and construct MyType
}

[DefaultDeltaWriter]
public static bool TryWriteDeltaMyType(this Writer writer, MyType previousValue, MyType ne
xtValue, DeltaCheckMode deltaCheckMode)
{
    // return true only if something was written
}

[DefaultDeltaReader]
public static MyType ReadDeltaMyType(this Reader reader, MyType previousValue)
{
    // mirror exactly what the delta writer chose to write
}
```

The generator checks each method's signature and registers it for you.

Register at runtime

The attributes call these methods, which you can also call yourself, for example from a plugin or code that runs early:

- `GenericFullWriter<T0>.UseWriteHandler(WriteHandler, bool isSerializerCustom)`
- `GenericFullReader<T0>.UseReadHandler(ReadHandler, bool isSerializerCustom)`
- `GenericDeltaWriter<T0>.UseWriteHandler(WriteHandler, bool isSerializerCustom)`
- `GenericDeltaReader<T0>.UseReadHandler(ReadHandler, bool isSerializerCustom)`

Each also has an overload taking `WriteWithAccuracyHandler` or `ReadWithAccuracyHandler`, which add a `float accuracy` parameter, for types whose encoding takes a precision hint.

```
// Nucleus.Serializers.GenericFullWriter<T0>
public delegate void WriteHandler(Writer writer, T0 value, CompressionLevel compressionLev
el);
public delegate void WriteWithAccuracyHandler(Writer writer, T0 value, CompressionLevel co
mpressionLevel, float accuracy);

// Nucleus.Serializers.GenericFullReader<T0>
public delegate T0 ReadHandler(Reader reader, CompressionLevel compressionLevel);
public delegate T0 ReadWithAccuracyHandler(Reader reader, CompressionLevel compressionLeve
l, float accuracy);

// Nucleus.Serializers.GenericDeltaWriter<T0>
public delegate bool WriteHandler(Writer writer, T0? previousValue, T0? nextValue, DeltaCh
eckMode deltaCheckMode);
public delegate bool WriteWithAccuracyHandler(Writer writer, T0? previousValue, T0? nextVa
lue, DeltaCheckMode deltaCheckMode, float accuracy);

// Nucleus.Serializers.GenericDeltaReader<T0>
public delegate T0 ReadHandler(Reader reader, T0 previousValue);
public delegate T0 ReadWithAccuracyHandler(Reader reader, T0 previousValue, float accuracy
);
```

Pass `isSerializerCustom: true`. Once registered, the handler is never replaced, neither by a generated serializer nor by a later registration of your own, so register each handler once.

```
GenericFullWriter<MyType>.UseWriteHandler(WriteMyType, isSerializerCustom: true);
GenericFullReader<MyType>.UseReadHandler(ReadMyType, isSerializerCustom: true);
GenericDeltaWriter<MyType>.UseWriteHandler(TryWriteDeltaMyType, isSerializerCustom: true);
GenericDeltaReader<MyType>.UseReadHandler(ReadDeltaMyType, isSerializerCustom: true);
```

Make the delta reader mirror the delta writer

A delta writer may write nothing when `DeltaCheckMode.Checked` finds no change. The reader must read exactly when the writer wrote and skip exactly when it did not. If they disagree, every later read in the stream returns wrong values, with no error.

Nullable types need no extra serializer

A member typed `MyType?` uses your serializer for `MyType`. `Writer.WriteNullable<T0>` and `TryWriteNullable<T0>` wrap the underlying type's registered handler for any `struct`. `NullableEncapsulatedChecker<T0>.GetIsNullable()` and `GetUnderlyingType()` tell you whether `T0` is `Nullable<>` and, if so, its underlying type.

Reuse the built-in helpers

- **`ByteArrayDifferenceSerializer`** : `TryWrite(Writer, byte[] previousValue, byte[] nextValue)` and `Read(Reader, byte[] previousValue, byte[] result)` write and read the difference between two same-length byte arrays. The length is not sent, so both peers must already know it. Use it inside a delta writer for any type that is a fixed-length byte buffer.
- **`DecimalPacked`** : a compact `decimal`. Build it with `DecimalPacked(decimal value, float accuracy)` and convert back with `AsDecimal(float accuracy)`.

Some types have no delta serializer by design

`Matrix4x4` (`Writer.TryWriteMatrix4X4`, `Reader.ReadDeltaMatrix4X4`), `Channel` (`Writer.TryWriteDeltaChannel`, `Reader.ReadDeltaChannel`), `NetworkSystemGroup` (`Writer.TryWriteDeltaNetworkSystemGroup`, `Reader.ReadDeltaNetworkSystemGroup`) and arrays through `Writer.TryWriteDeltaArrayAndCount<T0>` all throw. If your own type has no meaningful difference to send, throwing from its delta writer is correct.

Test it as a round trip

Write through a real `Writer`, initialized with `Initialize(coreManager: null)`.

Read back through a real `Reader` initialized from `writer.ToPackedBytes()`.

For a delta pair, write a run of values on one `Writer` and read the same run back on one `Reader`, checking each value in order. A single write and read will not catch a reader that falls out of step.

Never check only the writer's return value; writing successfully is not the same as writing the right bytes.

Reaching Types You Cannot Annotate

Use a marker struct for types in other assemblies

The generator only serializes a type that carries `[NetworkType]` or that it reaches through a member it already serializes. A type from an assembly you do not control, such as `System.Numerics.Vector2` or a third-party math type, cannot carry the attribute.

Declare a struct in your own assembly, mark it `[NetworkType]`, and give it one field per external type you need. Only the field types matter; the struct is never used at runtime.

```
[NetworkType]
public struct ExternalNetworkTypes
{
    public Vector4 Vector4;
    public Vector3 Vector3;
    public Vector2 Vector2;

    public Vector4Int Vector4Int;
    public Vector3Int32 Vector3Int32;
    public Vector2Int Vector2Int;

    public UnifiedColor UnifiedColor;
}
```

Without `[NetworkType]` the struct is ignored with no error. A field whose type is reachable some other way still gets a serializer, which can hide the mistake.

Nucleus already covers seven types

Nucleus ships the struct above as `ExternalNetworkTypes` in `Nucleus.Serializers`. It covers `Vector4`, `Vector3` and `Vector2` from `System.Numerics`, plus CodeBoost's `Vector4Int`, `Vector3Int32`, `Vector2Int` and `UnifiedColor`. You need your own marker only for other types.

SERIALIZERS001 usually means a missing marker

When a member's type has no serializer, the generator reports `SERIALIZERS001` naming the member; see Generator Errors and Warnings. For a plain data type from an assembly you do not control, add it to a marker struct. Write a custom serializer only when the type needs custom read and write logic.

Generator Errors and Warnings

Find diagnostics in your build output or the Unity console

The source generator runs during the normal C# compile. In a .NET project its diagnostics appear in the build output and your IDE's error list. In Unity, the generator DLL carries the `RoslynAnalyzer` label so Unity runs it, and its diagnostics appear in the Console with other compile errors.

Each diagnostic has an id, a severity and a message. The id tells you the stage; the message tells you what to fix.

Id	Severity	Stage
`NETWORKTYPEFINDER00`	Warning	Type discovery (walking your networked types and members)
`NETWORKTYPEFINDER01`	Error	Type discovery
`GENERATION000`	Error	Generation, invalid type
`GENERATION001`	Error	Generation, invalid Roslyn type
`GENERATION002`	Error	Generation, comparer or operation stage
`GENERATION003`	Warning	Generation, comparer or operation stage (declared, not currently raised)
`GENERATION004`	Warning	Two networked messages or two networked calls have the same hash
`SERIALIZERS000`	Error	Serializer generation
`SERIALIZERS001`	Warning	Serializer generation

`GENERATION004` does not stop the rest of the build. While the two types collide, the engine refuses to send, register or deliver either one. Rename one of them; see Messages and Calls That Do Not Arrive.

Fix the first diagnostic first

An early error stops later stages for that type, so one `NETWORKTYPEFINDER01` error often brings a long list of `SERIALIZERS001` warnings with it. Fix the top diagnostic for a type and rebuild before working through the rest.

SERIALIZERS001: a member has no serializer

This is the most common diagnostic. Its message looks like:

> `[Type.Member] Type [SomeType] for [Write/Read] SerializeMethod could not be found; this value will not be networked.`

Fix it one of two ways:

- If the type is in an assembly you do not control, add it to a marker struct. See Reaching Types You Cannot Annotate.

- If the type needs custom read and write logic, write a serializer. See Writing a Custom Serializer.

`UnityEngine` types such as `Vector3` and `Quaternion` do not raise this warning, but declare the `System.Numerics` equivalent anyway. See Unity Types and System.Numerics.

`SERIALIZERS000` is the error form: a required serializer is missing or malformed, for example `A [Write/Read] serializer could not be found for [Type].`, or a type exceeds the maximum networked member count. Fix it the same way.

NETWORKTYPEFINDER01: the type is declared in a way the generator cannot use

This covers unsupported type arguments and member types discovery does not recognize.

If instead you see ordinary C# compiler errors about a generated file, such as a missing `partial` modifier, check that your networked component or system, and every type that contains it, is declared `partial`, and check how it is nested.

Runtime problems are not generator problems

A generator diagnostic only means the generator could not produce code for your types. If the build is clean but replicated values are wrong at runtime, the cause is not the generator.

Serialization, Codegen and the Wire

Pages in this category:

- Networked Type Attributes
- Compression, Accuracy and Per-Type Registrations
- Type Identity on the Wire: Bundles and the Registry
- Wiring the Source Generator into a .NET Project
- Generator Errors and Warnings
- Writer and Reader
- Reaching Types You Cannot Annotate
- Writing a Custom Serializer

Networked Type Attributes

Most code needs none of these attributes. A type reached through a `NetworkMember<T0>` field is serialized automatically, and a member on a `NetworkComponent` is networked by default.

[NetworkType] makes the generator serialize a type it cannot otherwise find

Applies to a class, struct or enum. Use it only for a type nothing reaches through a `NetworkMember<T0>` field, such as one that is only boxed or reflected into. It also marks the structs described in Reaching Types You Cannot Annotate.

[NetworkIgnore] excludes a member from generation

Applies to a field or property. The generator skips the member entirely and produces no code for it.

[NetworkBundle(ushort)] sets an assembly's bundle id

```
[assembly: NetworkBundle(1)]
```

The main bundle defaults to `0` when the attribute is absent. Every other assembly with networked types declares its own unique id exactly once. See Type Identity on the Wire: Bundles and the Registry.

[ReplicationIgnore] keeps a value off the network

Applies to a `NetworkMemberBase` field, or to a whole `NetworkComponent` class to exclude every member it declares. Saving is unaffected; [[`WorldStateIgnore`]](../scenes/world-state-ignore.md) is the opposite, keeping a member out of a saved world while still replicating it.

- On every peer but the server, the value stays at its constructed default.
- A peer promoted to server by adoption never received these values. It holds the defaults, and saving from it writes those defaults over what the store held. Server-only values do not survive a handover.

Serializer attributes mark hand-written serializers

`[DefaultWriter]`, `[DefaultReader]`, `[DefaultDeltaWriter]` and `[DefaultDeltaReader]` derive from `SerializerAttribute`. See Writing a Custom Serializer.

Some attributes do not exist

`[NetworkTypeContainer]`, `[NetworkAccuracy(float)]`, `[NetworkSerializeMode(CompressionLevel)]` and `[NetworkExcludeCount]` do not exist in Nucleus. Accuracy and compression level are `NetworkMember<T0>` constructor arguments, for example new `NetworkMember<float>(CompressionLevel.Aggressive, accuracy: 0.01f)`.

Wiring the Source Generator into a .NET Project

Using Unity? See Code Generation in Unity.

Set up the project

Reference the engine DLLs `Nucleus.dll`, `CodeBoost.dll` and `SynapseSocket.dll` as ordinary `Reference` items. There is no NuGet package; Adding Nucleus to a .NET project shows the project file.

Add the generator as an `Analyzer` item, not a `Reference`:

```
 <ItemGroup>
     <Analyzer Include="..\lib\Nucleus\Nucleus.CodeAnalysis.SourceGenerators.dll" />
 </ItemGroup>
```

If the project's default C# version is older than 9, set `<LangVersion>9.0</LangVersion>` or newer. `Nucleus.dll` targets `netstandard2.1`, and `netstandard2.1` and .NET Core 3.x projects default to C# 8. Nothing else is needed on frameworks older than .NET 5.

If this project is not the main assembly, give it a bundle id: `[assembly: NetworkBundle(1)]`. See Type Identity on the Wire: Bundles and the Registry.

Take all four DLLs, the generator included, from one release. Mixing builds is not supported.

Without the analyzer reference, or without a reference to Nucleus itself, the project still compiles but the generator does not run and nothing replicates. There is no error.

See what the generator produced

```
<PropertyGroup>
    <EmitCompilerGeneratedFiles>true</EmitCompilerGeneratedFiles>
</PropertyGroup>
```

This also writes the generated code to `obj/<Configuration>/<TargetFramework>/generated/Nucleus.CodeAnalysis.SourceGenerators/`. Nothing is compiled twice, so there is nothing to exclude.

Neither the compiler nor `dotnet clean` deletes old files from that folder. To see only the current build's output, delete the `generated` folder and run `dotnet build --no-incremental`; an incremental build that finds nothing changed leaves the folder empty.

Type Identity on the Wire: Bundles and the Registry

Using Unity? See Bundle Ids for Unity Assemblies.

Each type is identified by a bundle id and a local id

Every spawned `NetworkSystem` or `NetworkComponent` is identified by a `(BundleId, LocalId)` pair, not its type name. The bundle id comes from `[assembly: NetworkBundle(n)]`, declared once per assembly; an assembly without it is bundle zero. The local id is assigned for you within the assembly. Only the pair has to be unique, so separately built content assemblies cannot clash with the main game as long as each declares its own bundle id.

A system's local id is a `byte` and a component's is a `ushort`, so one assembly can declare far fewer networked system types than component types.

Choose bundle ids counting up from zero

Use bundle zero for the main game, then one, two, three for its content assemblies. These values are reserved:

- `NetworkTypeRegistry.FrameworkBundleId` is `ushort.MaxValue`, for the framework's built-in types. The Unity integration uses the id one below it.
- `NetworkTypeRegistry.BaseNetworkSystemLocalId` is `0`, for the base `NetworkSystem` type, which is concrete, usable directly and registered under `FrameworkBundleId`. Your own systems' local ids start at one.

Look types up with NetworkTypeRegistry

`NetworkTypeRegistry` maps each type to its identity and back to a factory. Each assembly's types are registered automatically when it loads, through:

- `RegisterSystem<TSystem>(ushort bundleId, byte localId)`
- `RegisterComponent<TComponent>(ushort bundleId, ushort localId)`

Resolve a type to its identity and back:

- `TryGetSystemId(Type type, out ushort bundleId, out byte localId)`
- `TryGetComponentId(Type type, out ushort bundleId, out ushort localId)`
- `TryCreateSystem(ushort bundleId, byte localId, out NetworkSystem networkSystem)`
- `TryCreateComponent(ushort bundleId, ushort localId, out NetworkComponent networkComponent)`
- `TryGetSystemType(ushort bundleId, byte localId, out Type systemType)`
- `TryGetComponentType(ushort bundleId, ushort localId, out Type componentType)`, which, like `TryGetSystemType`, resolves the type without constructing anything.

An identity can name a different type in a different build, so use the by-name forms for anything that must name a type across builds:

- `TryGetSystemIdByName(string systemTypeName, out ushort bundleId, out byte localId)`

- `TryCreateSystemByName(string systemTypeName, out NetworkSystem networkSystem)`
- `TryGetSystemName(Type systemType, out string systemTypeName)`
- `TryGetComponentName(Type componentType, out string componentTypeName)`
- `TryCreateComponentByName(string componentTypeName, out NetworkComponent networkComponent)`

`TryReturnSystem(NetworkSystem networkSystem)` returns a system to the pool its factory rented it from, found by its runtime type. Use it when you hold a system by its base type, for example one rented ahead of a spawn that never used it.

```
if (NetworkTypeRegistry.TryGetSystemId(typeof(MySystem), out ushort bundleId, out byte loc
alId))
{
    // bundleId/localId are the identity of MySystem.
}

if (NetworkTypeRegistry.TryCreateSystem(bundleId, localId, out NetworkSystem system))
{
    // system was rented from MySystem's pool.
}
```

Declared compositions tell a receiver what the build could spawn

A composition is the set of component types named together at one spawn call site. Every distinct composition in the build is registered for you:

- `RegisterDeclaredComposition(params Type[] componentTypes)`
- `IsDeclaredComposition(List<uint> sortedComponentIds)`
- `GetComponentCompositionId(ushort bundleId, ushort localId)` returns the value that names a component in a composition check.

A receiver resolving a code-only spawn can refuse a composition the build never declares, since no legitimate call site could have produced it.

Writer and Reader

`Writer` and `Reader` are what custom serializers and payloads are written against. A `Reader` must read values in the same order, at the same compression level, as the `Writer` wrote them. Both are pooled: rent one, use it, return it.

Rent and return a Writer

Rent from `WriterPool`:

- `WriterPool.Rent()`: an uninitialized writer, for when there is no `CoreManager` yet.
- `WriterPool.Rent(CoreManager coreManager)`: the usual case.
- `WriterPool.RentByBitCapacity(uint bitCapacity, CoreManager coreManager)` and `RentByByteCapacity(uint byteCapacity, CoreManager coreManager)`: pre-sized, to avoid an early resize. Return these with `ReturnByCapacity`, not `Return`.
- `WriterPool.RentList()`, `RentList(int writerCount)` and `WriterPool.RentDictionary<T0>()`: pooled collections of writers, for building several payloads at once, such as one per observer.

Return to `WriterPool`:

- `Return(Writer? writer)`, or `ReturnAndNullifyReference(ref Writer? writer)` to also null your reference.
- `ReturnByCapacity(Writer? writer)` and `ReturnByCapacityAndNullifyReference(ref Writer? writer)` for capacity-rented writers.
- `Reset(List<Writer?> writers)`, `Return(List<Writer?> writers)` and `ReturnAndNullifyReference(ref List<Writer?> writers)` for a rented list; the last two also return the list.

Rent and return a Reader

`ReaderPool.Rent` has three forms:

- `ReaderPool.Rent(byte[] packedBytes, byte excessBits, CoreManager coreManager, Connection senderConnection)`: from a byte array plus the count of unused bits in its final byte.
- `ReaderPool.Rent(PackedBytes packedBytes, CoreManager coreManager, Connection senderConnection)`: from a `PackedBytes`.
- `ReaderPool.Rent(CoreManager coreManager, Connection senderConnection)`: an empty reader, for code that adds read spans itself.

Return with `ReaderPool.Return(Reader? reader)` or `ReaderPool.ReturnAndNullifyReference(ref Reader? reader)`. `Reset`, `Return` and `ReturnAndNullifyReference` also have overloads for `List<Reader?>` and `Dictionary<T0, Reader>`.

Write full values

Every full-value write takes a `CompressionLevel`, defaulting to `Loose`:

```
void WriteInt8(sbyte value, CompressionLevel compressionLevel = CompressionLevel.Loose);
void WriteUInt8(byte value, CompressionLevel compressionLevel = CompressionLevel.Loose);
void WriteInt16(short value, CompressionLevel compressionLevel = CompressionLevel.Loose);
void WriteUInt16(ushort value, CompressionLevel compressionLevel = CompressionLevel.Loose)
;
void WriteInt32(int value, CompressionLevel compressionLevel = CompressionLevel.Loose);
void WriteUInt32(uint value, CompressionLevel compressionLevel = CompressionLevel.Loose);
void WriteInt64(long value, CompressionLevel compressionLevel = CompressionLevel.Loose);
void WriteUInt64(ulong value, CompressionLevel compressionLevel = CompressionLevel.Loose);
void WriteBoolean(bool value, CompressionLevel compressionLevel = CompressionLevel.Loose);
void WriteChar(char value, CompressionLevel compressionLevel = CompressionLevel.Loose);
```

Floating-point writes add an `accuracy` argument, the precision compression may discard:

```
void WriteSingle(float value, CompressionLevel compressionLevel = CompressionLevel.Loose,
float accuracy = DefaultFloatingAccuracyAsSingle);
void WriteDouble(double value, CompressionLevel compressionLevel = CompressionLevel.Loose,
 float accuracy = DefaultFloatingAccuracyAsSingle);
void WriteDecimal(decimal value, CompressionLevel compressionLevel = CompressionLevel.Loos
e, float accuracy = DefaultFloatingAccuracyAsSingle);
```

Other full-value writes:

```
void WriteString(string value, CompressionLevel compressionLevel = CompressionLevel.Loose)
;
void WriteGuid(Guid value, CompressionLevel compressionLevel = CompressionLevel.Loose);
void WriteDateTime(DateTime dateTime, CompressionLevel compressionLevel = CompressionLevel
.Loose);
void WriteMatrix4X4(Matrix4x4 value, CompressionLevel compressionLevel = CompressionLevel.
Loose, float accuracy = DefaultFloatingAccuracyAsSingle);
void WriteQuaternion(Quaternion value, CompressionLevel compressionLevel = CompressionLeve
l.Loose, float accuracy = DefaultQuaternionAccuracyAsSingle);
```

Collections write their count with the values:

```
void WriteUInt8ArrayWithCountAsBytes(byte[] bytes, CompressionLevel compressionLevel = Com
pressionLevel.Uncompressed);
void WriteArraySegmentWithCountAsBytes(ArraySegment<byte> value, CompressionLevel compress
ionLevel = CompressionLevel.Uncompressed);
void WriteListWithCount<T0>(List<T0> value, CompressionLevel compressionLevel = Compressio
nLevel.Loose);
void WriteArrayWithCount<T0>(T0[] value, CompressionLevel compressionLevel = CompressionLe
vel.Loose);
void WriteDictionary<T0, T1>(Dictionary<T0, T1> value, CompressionLevel compressionLevel =
 CompressionLevel.Loose);
```

`WriteNullable<T0>` writes a nullable value, null included, and `Write<T0>` uses whichever writer is registered for `T0`:

```
void WriteNullable<T0>(T0? value, CompressionLevel compressionLevel = CompressionLevel.Loo
se) where T0 : struct;
void Write<T0>(T0 value, CompressionLevel compressionLevel = CompressionLevel.Loose);
```

Write deltas

A delta write writes a value relative to a previous one and returns whether it wrote anything. Under `DeltaCheckMode.Checked`, `false` means the value was unchanged; `DeltaCheckMode.Unchecked` always writes. Delta writes use `Writer.DeltaCompressionLevel` instead of a `CompressionLevel` argument.

```
bool TryWriteDeltaInt8(sbyte previousValue, sbyte nextValue, DeltaCheckMode deltaCheckMode
);
bool TryWriteDeltaUInt8(byte previousValue, byte nextValue, DeltaCheckMode deltaCheckMode)
;
bool TryWriteDeltaInt16(short previousValue, short nextValue, DeltaCheckMode deltaCheckMod
e);
bool TryWriteDeltaUInt16(ushort previousValue, ushort nextValue, DeltaCheckMode deltaCheck
Mode);
bool TryWriteDeltaInt32(int previousValue, int nextValue, DeltaCheckMode deltaCheckMode);
bool TryWriteDeltaUInt32(uint previousValue, uint nextValue, DeltaCheckMode deltaCheckMode
);
bool TryWriteDeltaInt64(long previousValue, long nextValue, DeltaCheckMode deltaCheckMode)
;
bool TryWriteDeltaUInt64(ulong previousValue, ulong nextValue, DeltaCheckMode deltaCheckMo
de);
bool TryWriteDeltaSingle(float previousValue, float nextValue, DeltaCheckMode deltaCheckMo
de, float accuracy = DefaultFloatingAccuracyAsSingle);
bool TryWriteDeltaDouble(double previousValue, double nextValue, DeltaCheckMode deltaCheck
Mode, float accuracy = DefaultFloatingAccuracyAsSingle);
bool TryWriteDeltaDecimal(decimal previousValue, decimal nextValue, DeltaCheckMode deltaCh
eckMode, float accuracy = DefaultFloatingAccuracyAsSingle);
bool TryWriteDeltaString(string previousValue, string nextValue, DeltaCheckMode deltaCheck
Mode);
bool TryWriteDeltaGuid(Guid previousValue, Guid nextValue, DeltaCheckMode deltaCheckMode);
bool TryWriteDeltaDateTime(DateTime previousValue, DateTime nextValue, DeltaCheckMode delt
aCheckMode);
bool TryWriteDeltaQuaternion(Quaternion previousValue, Quaternion nextValue, DeltaCheckMod
e deltaCheckMode, float accuracy = DefaultQuaternionAccuracyAsSingle);
bool TryWriteDelta<T0>(T0 previousValue, T0 nextValue, DeltaCheckMode deltaCheckMode);
```

`TryWriteDelta*` behaves the same whether the member uses `TransmissionMode.Divine` or `TransmissionMode.Interval`.

Write raw bits and move the position

```
void WriteBits(ulong value, uint bitCount);
void WriteBits(ulong value, uint bitCount, uint position);
void WriteBits(Writer otherWriter);
uint SkipBits(uint bitCount);
uint SetBitBufferIndex(uint value);
```

- `WriteBits(value, bitCount)` writes at the current position and advances it.
- `WriteBits(value, bitCount, position)` writes at an absolute position without moving the current one, for filling in a length or flag once it is known.
- `WriteBits(Writer otherWriter)` appends another writer's bits.
- `SkipBits(bitCount)` reserves space and returns the position it started from.
- `SetBitBufferIndex(value)` jumps to a position and returns the previous one.

Capacity grows only when you ask:

```
void CreateCapacityAsBits(uint bitCount);
void CreateCapacityAsBytes(uint byteCount);
void CreateCapacityForArray(int writeCount);
```

Get the bytes out

```
uint BitCount { get; }
uint BitIndex { get; }
uint BitCapacity { get; }
uint PackedByteCount { get; }
byte GetExcessBitCount();
ArraySegment<byte> ToPackedByteArraySegment(out byte excessBitCount);
PackedBytes ToPackedBytes();
```

- `BitCount`: bits written so far.
- `BitIndex`: that count's position within its final byte.
- `BitCapacity`: the buffer's current size in bits.
- `PackedByteCount`: how many bytes the written bits fill.
- `GetExcessBitCount()`: unused bits in the final byte.
- `ToPackedByteArraySegment` and `ToPackedBytes`: the written bytes, ready for a transport or a `Reader`.

`PackedBytes`, and its read-only interface `IReadOnlyPackedBytes`, carries the bytes and the excess-bit count. `IReadOnlyWriter` and `IReadOnlyReader` expose the counters and byte extraction without any write or read methods.

Never trust a Connection read from a payload

```
void WriteConnection(Connection connection, CompressionLevel compressionLevel = Compressio
nLevel.Loose);
Connection ReadConnection(CompressionLevel compressionLevel = CompressionLevel.Loose);
```

These send a `Connection` by id, and any sender can write any id. Never authorize an action on a `ReadConnection` result alone. The trustworthy sender is the context's sender connection, such as `RpcContext.SenderConnection`, which comes from the transport.

Read values back

`Reader` has a matching read for every write (`ReadInt8`, `ReadSingle`, `ReadString`, `ReadDelta<T0>` and so on).

- A full read takes the `CompressionLevel` the write used. Reads default to the same level as their writes (`Loose`, including `ReadString`), except `ReadQuaternion`, which has no default and must be given the level.
- A delta read takes the previous value and no `DeltaCheckMode`. It uses `Reader.DeltaCompressionLevel`, which must match the writer's `DeltaCompressionLevel`, or every later read goes wrong.

Passing the level explicitly on both sides is the clearest way to keep a pair in step.

```
CoreManager CoreManager { get; }
Connection Sender { get; }
uint BitsRemaining { get; }
bool HasBitsRemaining { get; }
CompressionLevel DeltaCompressionLevel { get; set; }
```

`Sender` is the connection passed to `ReaderPool.Rent`. `BitsRemaining` and `HasBitsRemaining` say whether there is more to read, and `HasReadSpan` (from `IReadOnlyReader`) says whether at least one read span has been added.

Nucleus Networking Documentation

Nucleus is a C#/.NET networking engine with an optional Unity integration. Pick a place to start:

- Start Here
- Unity
- Core API
- Reference & Project Meta

API type reference

This page covers the types you are expected to use directly. A generated reference listing every type is planned but not yet published. Using Unity? See the manager components reference for the components that front each core manager.

Everything is reached from CoreManager

Construct one `CoreManager` and it builds every manager. Everything else in the engine is reached from one of these fields, which are `readonly` and set in the constructor:

```
public readonly NetworkLoopManager NetworkLoopManager;
public readonly TransportManager TransportManager;
public readonly ServerManager ServerManager;
public readonly ClientManager ClientManager;
public readonly SystemManager SystemManager;
public readonly InterestManager InterestManager;
public readonly Scenes.SceneManager SceneManager;
public readonly PacketManager PacketManager;
public readonly MessageManager MessageManager;
public readonly RpcManager RpcManager;
public readonly ViolationManager ViolationManager;
```

A Pro build adds two fields that are not `readonly`:

```
public BundleManager BundleManager;
public WorldPersistenceManager WorldPersistenceManager;
```

In a Free build these two fields do not exist, so code that uses them fails to compile rather than failing at runtime. The same holds for every Pro-only type and member.

`CoreManager.Instance` names the oldest live `CoreManager` in the process, for a process running exactly one. With several managers in one process, such as a host pair or a two-peer test, pass the manager explicitly instead. `CoreManager.EnsureSetInstance(CoreManager)` points the slot at a manager you choose; it fails and logs if that manager is null or already deinitialized.

Find types by namespace

Namespace	What lives there
`Nucleus.Managers.Core`	`CoreManager` itself
`Nucleus.Managers.NetworkLoop`	The tick driver and its step providers
`Nucleus.Managers.Transports`	`TransportManager`, the active `Transport`
`Nucleus.Managers.Server`	`ServerManager`
`Nucleus.Managers.Client`	`ClientManager`
`Nucleus.Managers.Systems`	`SystemManager` and system lifecycle
`Nucleus.Managers.Interest`	`InterestManager`
`Nucleus.Managers.Scenes`	`SceneManager`
`Nucleus.Managers.Messages`	`MessageManager`
`Nucleus.Managers.Rpc`	`RpcManager`

`Nucleus.Managers.Violations`	`ViolationManager` and the shipped violation types
`Nucleus.Managers.Bundles`	`BundleManager` (Pro)
`Nucleus.Managers.Persistence`	`WorldPersistenceManager` (Pro)
`Nucleus.Systems`	`NetworkSystem` and the members you declare on one
`Nucleus.Components`	Engine-neutral replicated components, such as `TransformComponent`
`Nucleus.Connections`	`Connection` and peer-side state
`Nucleus.Serializers`	Reader/writer contracts and generated serializer plumbing
`Nucleus.Packets`	Wire packet types
`Nucleus.Physics`	Physics-adjacent components and adapters
`Nucleus.Transports`	Transport base types, plus the `Synapse`, `Yak`, and `Sockets` sub-namespaces for the shipped transport implementations

Managers live under `Nucleus.Managers.*`; the types they hand you live in the flatter namespaces.

Read the usage markers

These attributes, from the `CodeBoost.Types` namespace (see CodeBoost reference), say how a member is meant to be used. The compiler does not enforce them.

- `[ServerOnly]`: only meaningful, or only safe to call, from server-side code.
- `[ClientOnly]`: only meaningful, or only safe to call, from client-side code.
- `[InternalApi]`: mainly for internal use, and can change or move without warning. Its optional `Details` string says why.

Custom transports can live in your own assembly

The transport base types are public, and every member a transport implements is public or protected, so you can write a transport in your own game or package. The Transport contract lists what to implement.

How the benchmarks are run

Every framework runs a tick rate of 30. The server and the client are separate standalone builds. The results are on the Nucleus website.

Every object runs the same movement script

The bandwidth tests move their objects with this script, with linear motion on. Each object drifts in a straight line across a tilted belt while it spins, and reflects off the belt's rim. Each framework's own networked script runs it on the server, and the framework's transform component replicates it.

Each framework's spawner hands every object its place in the spawn order through `AssignNumber`, and that number decides how the object moves:

- Half of the objects multiply their speed range by 5.
- A quarter move like rigidbodies. Their speed and spin never settle: both glide from one end of their range to the other and back, 4 seconds each way, so they are always accelerating.
- The other three quarters move like character controllers. At each bounce they take a new speed, spin speed and spin direction at once, and hold them until the next bounce.

The two splits are interleaved, so half of the rigidbody quarter and half of the character controllers are fast. The same seed and numbers make the belt move the same way on every run.

The CPU results were measured with an earlier version of this script, in which every object orbits the center of the belt.

```
using UnityEngine;

public class BeltedAsteroid : MonoBehaviour
{
    public const float MinimumSpeedRangeMultiplier = 1f;
    public const float MaximumSpeedRangeMultiplier = 20f;

    [SerializeField]
    [Tooltip("Drift in a straight line and reflect off the belt's outer rim instead of orb
iting the center.")]
    private bool _isLinearMotion;
    [SerializeField]
    [Tooltip("The slowest self-spin a cube draws, in degrees per second.")]
    private float _minimumRotationDegreesPerSecond = 5f;
    [SerializeField]
    [Tooltip("The fastest self-spin a cube draws, in degrees per second.")]
    private float _maximumRotationDegreesPerSecond = 65f;
    [SerializeField]
    [Tooltip("The slowest ground speed a cube draws, in units per second: the drift speed
in linear mode, the tangential speed in orbit mode.")]
    private float _minimumMovementSpeed = 0.5f;
    [SerializeField]
    [Tooltip("The fastest ground speed a cube draws, in units per second: the drift speed
in linear mode, the tangential speed in orbit mode.")]
    private float _maximumMovementSpeed = 2f;
    [SerializeField]
    [Tooltip("The factor half of the asteroids multiply their movement speed range by, fro
m 1 to 20. Applied once as each asteroid wakes.")]
    private float _speedRangeMultiplier = 5f;
    [SerializeField]
    [Tooltip("How likely the ground speed is drawn near the maximum: 1 always picks the ma
x, 0.5 draws uniformly.")]
    private float _movementSpeedFullnessChance = 0.5f;
    [SerializeField]
    [Tooltip("How likely the self-spin speed is drawn near the maximum: 1 always picks the
 max, 0.5 draws uniformly.")]
    private float _rotationSpeedFullnessChance = 0.5f;
    [SerializeField]
    [Tooltip("The seed of the whole belt. The same seed draws the same courses for every a
steroid on every run; a different one draws a different belt.")]
    private uint _seed = 1u;

    private float _orbitRadius;
    private float _orbitHeight;
    private float _orbitAngle;
    private float _orbitAngularSpeed;
    private Vector3 _spinAxis;
    private SpeedRamp _rotationRamp;
    private Vector2 _linearPosition;
    private Vector2 _linearDirection;
    private SpeedRamp _linearSpeedRamp;
    private float _cubeMinimumMovementSpeed;
    private float _cubeMaximumMovementSpeed;
    private bool _isSpeedSnappedAtBounce;
    private bool _isMovementSweepRising;
    private bool _isSpinSweepRising;
    private AsteroidRandom _random;
```

```
    private static readonly Vector3 OrbitCenter = Vector3.zero;
    private static readonly Quaternion BeltTilt = Quaternion.Euler(27f, 0f, 14f);
    private const float BeltOuterRadius = 16f;
    private const float SpeedSweepDuration = 4f;
    private const float MinimumLinearStartRadiusFraction = 0.7f;
    private const float MaximumLinearStartRadiusFraction = 0.97f;

    public void AuthorityUpdate(float frameSeconds) => Advance(frameSeconds);

    public void AuthorityTick(float stepSeconds) => Advance(stepSeconds);

    public void AssignNumber(uint asteroidNumber) => SeedMotion(asteroidNumber);

    private void Awake()
    {
        SeedMotion((uint)transform.GetSiblingIndex());
    }

    private void SeedMotion(uint asteroidNumber)
    {
        _random = AsteroidRandom.ForAsteroid(_seed, asteroidNumber);

        float speedRangeMultiplier = asteroidNumber % 2u == 1u ? Mathf.Clamp(_speedRangeMu
ltiplier, MinimumSpeedRangeMultiplier, MaximumSpeedRangeMultiplier) : 1f;
        _cubeMinimumMovementSpeed = _minimumMovementSpeed * speedRangeMultiplier;
        _cubeMaximumMovementSpeed = _maximumMovementSpeed * speedRangeMultiplier;
        _isSpeedSnappedAtBounce = asteroidNumber / 2u % 4u != 0u;

        _orbitHeight = _random.NextRange(minimum: -1.25f, maximum: 1.25f);
        _rotationRamp.Reset(BiasedSpeed(ref _random, _minimumRotationDegreesPerSecond, _ma
ximumRotationDegreesPerSecond, _rotationSpeedFullnessChance));
        _spinAxis = _random.NextUnitVector();

        if (_isLinearMotion)
        {
            float startAngle = _random.NextRange(minimum: 0f, maximum: 2f * Mathf.PI);
            float startRadius = _random.NextRange(MinimumLinearStartRadiusFraction, Maximu
mLinearStartRadiusFraction) * BeltOuterRadius;
            Vector2 radial = new(Mathf.Cos(startAngle), Mathf.Sin(startAngle));
            _linearPosition = radial * startRadius;

            _linearDirection = new(-radial.y, radial.x);
            _linearSpeedRamp.Reset(BiasedSpeed(ref _random, _cubeMinimumMovementSpeed, _cu
beMaximumMovementSpeed, _movementSpeedFullnessChance));

            if (!_isSpeedSnappedAtBounce)
            {
                bool isSweepStartedAtMaximum = asteroidNumber / 8u % 2u == 1u;
                _linearSpeedRamp.Reset(isSweepStartedAtMaximum ? _cubeMaximumMovementSpeed
 : _cubeMinimumMovementSpeed);
                _rotationRamp.Reset(isSweepStartedAtMaximum ? _maximumRotationDegreesPerSe
cond : _minimumRotationDegreesPerSecond);
                _isMovementSweepRising = isSweepStartedAtMaximum;
                _isSpinSweepRising = isSweepStartedAtMaximum;
            }
        }
        else
```

```
        {
            _orbitRadius = _random.NextRange(minimum: 8f, maximum: BeltOuterRadius);
            _orbitAngle = _random.NextRange(minimum: 0f, maximum: 2f * Mathf.PI);

            _orbitAngularSpeed = BiasedSpeed(ref _random, _cubeMinimumMovementSpeed, _cube
MaximumMovementSpeed, _movementSpeedFullnessChance) / _orbitRadius;
        }

        transform.position = _isLinearMotion ? BeltWorldPosition(_linearPosition) : OrbitP
osition();
    }

    private void Advance(float deltaSeconds)
    {
        if (_isLinearMotion)
            DriveLinearMotion(deltaSeconds);
        else
            DriveOrbitMotion(deltaSeconds);

        _rotationRamp.Advance(deltaSeconds);

        if (_isLinearMotion && !_isSpeedSnappedAtBounce)
            ContinueSweep(ref _rotationRamp, ref _isSpinSweepRising, _minimumRotationDegre
esPerSecond, _maximumRotationDegreesPerSecond);

        transform.Rotate(_spinAxis * (_rotationRamp.Current * deltaSeconds), Space.Self);
    }

    private void DriveOrbitMotion(float deltaSeconds)
    {
        _orbitAngle += _orbitAngularSpeed * deltaSeconds;
        transform.position = OrbitPosition();
    }

    private void DriveLinearMotion(float deltaSeconds)
    {
        _linearSpeedRamp.Advance(deltaSeconds);

        if (!_isSpeedSnappedAtBounce)
            ContinueSweep(ref _linearSpeedRamp, ref _isMovementSweepRising, _cubeMinimumMo
vementSpeed, _cubeMaximumMovementSpeed);

        _linearPosition += _linearDirection * (_linearSpeedRamp.Current * deltaSeconds);

        float distance = _linearPosition.magnitude;

        if (distance > BeltOuterRadius)
        {
            Vector2 normal = _linearPosition / distance;
            _linearDirection = Vector2.Reflect(_linearDirection, normal);
            _linearPosition = normal * BeltOuterRadius;

            if (_isSpeedSnappedAtBounce)
            {
                _rotationRamp.Reset(BiasedSpeed(ref _random, _minimumRotationDegreesPerSec
ond, _maximumRotationDegreesPerSecond, _rotationSpeedFullnessChance));
                _spinAxis = _random.NextUnitVector();
                _linearSpeedRamp.Reset(BiasedSpeed(ref _random, _cubeMinimumMovementSpeed,
```

```
 _cubeMaximumMovementSpeed, _movementSpeedFullnessChance));
            }
        }

        transform.position = BeltWorldPosition(_linearPosition);
    }

    private Vector3 OrbitPosition() =>
        BeltWorldPosition(new(Mathf.Cos(_orbitAngle) * _orbitRadius, Mathf.Sin(_orbitAngle
) * _orbitRadius));

    private Vector3 BeltWorldPosition(Vector2 planePosition) =>
        OrbitCenter + BeltTilt * new Vector3(planePosition.x, _orbitHeight, planePosition.
y);

    private static float BiasedSpeed(ref AsteroidRandom asteroidRandom, float minimumSpeed
, float maximumSpeed, float fullnessChance)
    {
        if (fullnessChance >= 1f)
            return maximumSpeed;

        if (fullnessChance <= 0f)
            return minimumSpeed;

        float exponent = (1f - fullnessChance) / fullnessChance;
        float fraction = Mathf.Pow(asteroidRandom.NextSingle(), exponent);

        return Mathf.Lerp(minimumSpeed, maximumSpeed, fraction);
    }

    private static void ContinueSweep(ref SpeedRamp speedRamp, ref bool isSweepRising, flo
at minimumSpeed, float maximumSpeed)
    {
        if (!speedRamp.IsSettled)
            return;

        isSweepRising = !isSweepRising;
        speedRamp.SetTarget(isSweepRising ? maximumSpeed : minimumSpeed, SpeedSweepDuratio
n);
    }

    private struct SpeedRamp
    {
        public float Current { get; private set; }
        public bool IsSettled => _elapsedSeconds >= _durationSeconds;

        private float _startValue;
        private float _targetValue;
        private float _elapsedSeconds;
        private float _durationSeconds;

        public void Reset(float value)
        {
            _startValue = value;
            _targetValue = value;
            _elapsedSeconds = 0f;
            _durationSeconds = 0f;
            Current = value;
```

```
        }

        public void SetTarget(float target, float durationSeconds)
        {
            _startValue = Current;
            _targetValue = target;
            _elapsedSeconds = 0f;
            _durationSeconds = durationSeconds;
        }

        public void Advance(float deltaTime)
        {
            if (IsSettled)
            {
                Current = _targetValue;

                return;
            }

            _elapsedSeconds += deltaTime;
            Current = Mathf.Lerp(_startValue, _targetValue, Mathf.Clamp01(_elapsedSeconds
/ _durationSeconds));
        }
    }
}
```

```
using UnityEngine;

public struct AsteroidRandom
{
    private ulong _state;
    private ulong _increment;

    private const ulong Multiplier = 6364136223846793005UL;

    public AsteroidRandom(ulong initialState, ulong sequence)
    {
        _state = 0UL;
        _increment = (sequence << 1) | 1UL;

        NextUInt32();

        _state += initialState;

        NextUInt32();
    }

    public uint NextUInt32()
    {
        ulong previousState = _state;
        _state = unchecked((previousState * Multiplier) + _increment);

        uint xorShifted = (uint)(((previousState >> 18) ^ previousState) >> 27);
        int rotation = (int)(previousState >> 59);

        return (xorShifted >> rotation) | (xorShifted << ((-rotation) & 31));
    }

    public float NextSingle() => (NextUInt32() >> 8) * (1f / 16777216f);

    public float NextRange(float minimum, float maximum) => minimum + ((maximum - minimum)
 * NextSingle());

    public Vector3 NextUnitVector()
    {
        float height = (2f * NextSingle()) - 1f;
        float angle = 2f * Mathf.PI * NextSingle();
        float radius = Mathf.Sqrt(Mathf.Max(0f, 1f - (height * height)));

        return new(radius * Mathf.Cos(angle), radius * Mathf.Sin(angle), height);
    }

    public static AsteroidRandom ForAsteroid(uint seed, uint asteroidIndex) => new(Mix(((u
long)seed << 32) | asteroidIndex), asteroidIndex);

    private static ulong Mix(ulong value)
    {
        unchecked
        {
            value += 0x9E3779B97F4A7C15UL;
            value = (value ^ (value >> 30)) * 0xBF58476D1CE4E5B9UL;
            value = (value ^ (value >> 27)) * 0x94D049BB133111EBUL;
```

```
            return value ^ (value >> 31);
        }
    }
}
```

Bandwidth is counted by a relay on the client's machine

Measuring starts 10 seconds after the client holds all 200 objects and lasts 60 seconds. Positions and rotations are replicated at a precision of 0.001 and accurate to the tick. The server and one client run on the same machine, both capped at 90 frames per second, and a relay between them delays every packet by 75 ms each way, a 150 ms round trip. The relay counts every byte and packet the server sends. The bytes counted are divided by the 60 seconds, the 200 objects and the 30 ticks a second, which gives bytes per object, per tick.

Server CPU is measured by a monitor in the server process

The server runs on an AWS c7i.large, capped at 60 frames per second, with 200 objects and again with 2,000. Its clients run on a separate machine and connect 0.3 seconds apart, 50 of them and then 200. Measuring starts 2 minutes after the last client connects and lasts 10 seconds. The monitor below measures the CPU time of the whole server process and divides it by the frames the server ran. A framework that cannot hold 30 ticks a second is marked as failed.

Thank you to Kreyg for providing this CPU logger script.

```
using System;
using System.Diagnostics;
using System.Runtime.InteropServices;
using System.Text;
using UnityEngine;
using Debug = UnityEngine.Debug;

/// <summary>
/// Stack-agnostic CPU logger for headless runs. Warmup, then fixed windows, then everythi
ng is printed once at the end
/// (per-window lines, mean and median), so logging never runs during a measured window.
/// </summary>
/// <remarks>
/// Process CPU % is relative to ONE core: 100% means one full core busy. Measures whole-p
rocess CPU time, all threads.
/// On Windows builds the time comes from QueryProcessCycleTime, converted with a cycles-p
er-millisecond rate calibrated by a short
/// busy spin at start. GetProcessTimes only charges whole 15.6 ms clock ticks to whicheve
r thread is running at each tick, so a
/// light 60 fps process whose frames keep landing between ticks reads several times too l
ow; its figure is still logged for comparison.
/// Window results are stored in a preallocated array, so nothing allocates while measurin
g.
/// </remarks>
public class CpuLogger : MonoBehaviour
{
    [SerializeField]
    [Tooltip("Label printed with the results, e.g. 'Purr Off 300 showcase'.")]
    private string _label = "run";
    [SerializeField]
    [Tooltip("Seconds to wait before the first window, so spawning and JIT settle.")]
    private float _warmupSeconds = 5f;
    [SerializeField]
    [Tooltip("Length of each window, in seconds.")]
    private float _windowSeconds = 20f;
    [SerializeField]
    [Tooltip("How many windows to record. More windows make the median more robust to one-
off spikes.")]
    private int _windowCount = 1;
    [SerializeField]
    [Tooltip("Frame rate to lock to, so every stack runs the same number of frames. 0 leav
es it alone.")]
    private int _targetFrameRate = 60;
    [SerializeField]
    [Tooltip("Quit the application after printing the results.")]
    private bool _quitWhenDone = true;

    private struct WindowResult
    {
        public double WallSeconds;
        public double CpuMs;
        public double TickCpuMs;
        public int Frames;
        public int Gen0;
        public float WorstFrameMs;
        public double HeapMb;
    }
```

```
    private Process _process;
    private WindowResult[] _results;
    private double[] _sortBuffer;
    private float _startTime;
    private bool _isWarm;
    private bool _isDone;
    private int _windowIndex;

    private float _windowStartTime;
    private double _windowStartCpuMs;
    private double _windowStartTickCpuMs;
    private double _cyclesPerMs;
    private int _windowStartFrame;
    private int _windowStartGen0;
    private float _worstFrameMs;

#if UNITY_STANDALONE_WIN && !UNITY_EDITOR
    [DllImport("kernel32.dll")]
    private static extern IntPtr GetCurrentProcess();

    [DllImport("kernel32.dll", SetLastError = true)]
    private static extern bool GetProcessTimes(IntPtr process, out long creationTime, out
long exitTime, out long kernelTime, out long userTime);

    [DllImport("kernel32.dll", SetLastError = true)]
    private static extern bool QueryProcessCycleTime(IntPtr process, out ulong cycleTime);

    [DllImport("kernel32.dll")]
    private static extern IntPtr GetCurrentThread();

    [DllImport("kernel32.dll", SetLastError = true)]
    private static extern bool QueryThreadCycleTime(IntPtr thread, out ulong cycleTime);
#endif

#if UNITY_STANDALONE_LINUX && !UNITY_EDITOR
    // struct rusage on 64-bit Linux: ru_utime (sec, usec), ru_stime (sec, usec), then 14
more longs.
    private const int RusageSelf = 0;
    private readonly long[] _rusage = new long[18];

    [DllImport("libc", SetLastError = true)]
    private static extern int getrusage(int who, [Out] long[] usage);
#endif

    private void Start()
    {
        if (_targetFrameRate > 0 && Application.isBatchMode)
        {
            QualitySettings.vSyncCount = 0;
            Application.targetFrameRate = _targetFrameRate;
        }

        _process = Process.GetCurrentProcess();
        _results = new WindowResult[_windowCount];
        _sortBuffer = new double[_windowCount];
        _startTime = Time.realtimeSinceStartup;
#if UNITY_STANDALONE_WIN && !UNITY_EDITOR
```

```
        // Busy-spin this thread for 100 ms wall time, three times; the fastest rate seen
is the least preempted one.
        for (int attempt = 0; attempt < 3; attempt++)
        {
            QueryThreadCycleTime(GetCurrentThread(), out ulong startCycles);
            Stopwatch stopwatch = Stopwatch.StartNew();
            while (stopwatch.Elapsed.TotalMilliseconds < 100d)
            {
            }
            QueryThreadCycleTime(GetCurrentThread(), out ulong endCycles);
            _cyclesPerMs = Math.Max(_cyclesPerMs, (endCycles - startCycles) / stopwatch.El
apsed.TotalMilliseconds);
        }
        Debug.Log($"[CPU] cycle rate {_cyclesPerMs / 1000d:F1} MHz");
#endif
    }

    private void Update()
    {
        if (_isDone)
        {
            return;
        }

        float now = Time.realtimeSinceStartup;

        if (!_isWarm)
        {
            if (now - _startTime < _warmupSeconds)
            {
                return;
            }

            _isWarm = true;
            BeginWindow(now);
            return;
        }

        float frameMs = Time.unscaledDeltaTime * 1000f;
        if (frameMs > _worstFrameMs)
        {
            _worstFrameMs = frameMs;
        }

        if (now - _windowStartTime < _windowSeconds)
        {
            return;
        }

        EndWindow(now);
        _windowIndex++;

        if (_windowIndex < _windowCount)
        {
            BeginWindow(now);
            return;
        }
```

```
        _isDone = true;
        PrintResults();

        if (_quitWhenDone)
        {
            Application.Quit();
        }
    }

    /// <summary>
    /// Total CPU time this process has used, in milliseconds, across all threads.
    /// </summary>
    /// <remarks>On Windows builds this asks the OS directly, since Process.TotalProcessor
Time is coarse or zero under Unity.</remarks>
    private double GetProcessCpuMs()
    {
#if UNITY_STANDALONE_WIN && !UNITY_EDITOR
        if (_cyclesPerMs > 0d && QueryProcessCycleTime(GetCurrentProcess(), out ulong cycl
es))
        {
            return cycles / _cyclesPerMs;
        }
#endif
        return GetTickCpuMs();
    }

    /// <summary>
    /// Total CPU time from the OS's tick-sampled counters, as measured before cycle time;
 logged next to the cycle figure for comparison.
    /// </summary>
    private double GetTickCpuMs()
    {
#if UNITY_STANDALONE_WIN && !UNITY_EDITOR
        if (GetProcessTimes(GetCurrentProcess(), out long _, out long _, out long kernelTi
me, out long userTime))
        {
            // FILETIME units are 100 nanoseconds.
            return (kernelTime + userTime) / 10000d;
        }
#endif
#if UNITY_STANDALONE_LINUX && !UNITY_EDITOR
        if (getrusage(RusageSelf, _rusage) == 0)
        {
            // User + system time: seconds to ms, plus microseconds to ms.
            return (_rusage[0] + _rusage[2]) * 1000d + (_rusage[1] + _rusage[3]) / 1000d;
        }
#endif
        _process.Refresh();
        return _process.TotalProcessorTime.TotalMilliseconds;
    }

    private void BeginWindow(float now)
    {
        _windowStartTime = now;
        _windowStartCpuMs = GetProcessCpuMs();
        _windowStartTickCpuMs = GetTickCpuMs();
        _windowStartFrame = Time.frameCount;
        _windowStartGen0 = GC.CollectionCount(0);
```

```
        _worstFrameMs = 0f;
    }

    private void EndWindow(float now)
    {
        _results[_windowIndex] = new WindowResult
        {
            WallSeconds = now - _windowStartTime,
            CpuMs = GetProcessCpuMs() - _windowStartCpuMs,
            TickCpuMs = GetTickCpuMs() - _windowStartTickCpuMs,
            Frames = Time.frameCount - _windowStartFrame,
            Gen0 = GC.CollectionCount(0) - _windowStartGen0,
            WorstFrameMs = _worstFrameMs,
            HeapMb = GC.GetTotalMemory(forceFullCollection: false) / 1048576d,
        };
    }

    private void PrintResults()
    {
        StringBuilder builder = new();
        builder.AppendLine($"[CPU] {_label} results ({_windowCount} x {_windowSeconds:F0}s
, logged after the run):");

        double totalCpuMs = 0d;
        double totalTickCpuMs = 0d;
        double totalWallSeconds = 0d;
        int totalFrames = 0;
        int totalGen0 = 0;
        float overallWorstFrameMs = 0f;

        for (int i = 0; i < _windowCount; i++)
        {
            WindowResult result = _results[i];
            double cpuPercent = result.CpuMs / (result.WallSeconds * 10d);
            double msPerFrame = result.CpuMs / Math.Max(result.Frames, 1);

            builder.AppendLine($"[CPU] window {i + 1}: CPU {cpuPercent:F1}%, {msPerFrame:F
3} ms/frame, " +
                               $"{result.Frames / result.WallSeconds:F1} fps, worst frame
{result.WorstFrameMs:F1} ms, " +
                               $"GC gen0 {result.Gen0}, heap {result.HeapMb:F1} MB");

            totalCpuMs += result.CpuMs;
            totalTickCpuMs += result.TickCpuMs;
            totalWallSeconds += result.WallSeconds;
            totalFrames += result.Frames;
            totalGen0 += result.Gen0;

            if (result.WorstFrameMs > overallWorstFrameMs)
            {
                overallWorstFrameMs = result.WorstFrameMs;
            }

            _sortBuffer[i] = msPerFrame;
        }

        double medianMsPerFrame = Median(_sortBuffer);
```

```
        for (int i = 0; i < _windowCount; i++)
        {
            _sortBuffer[i] = _results[i].CpuMs / (_results[i].WallSeconds * 10d);
        }

        double medianCpuPercent = Median(_sortBuffer);

        builder.AppendLine($"[CPU] {_label} MEAN: CPU {totalCpuMs / (totalWallSeconds * 10
d):F1}%, " +
                           $"{totalCpuMs / Math.Max(totalFrames, 1):F3} ms/frame, {totalFr
ames / totalWallSeconds:F1} fps");
        builder.AppendLine($"[CPU] {_label} MEDIAN: CPU {medianCpuPercent:F1}%, {medianMsP
erFrame:F3} ms/frame");
        builder.AppendLine($"[CPU] {_label} TICK-SAMPLED MEAN: CPU {totalTickCpuMs / (tota
lWallSeconds * 10d):F1}%, {totalTickCpuMs / Math.Max(totalFrames, 1):F3} ms/frame");
        builder.Append($"[CPU] {_label} worst frame {overallWorstFrameMs:F1} ms, GC gen0 {
totalGen0}, " +
                       $"heap {_results[_windowCount - 1].HeapMb:F1} MB");

        Debug.Log(builder.ToString());
    }

    private static double Median(double[] values)
    {
        Array.Sort(values);

        int middle = values.Length / 2;

        if (values.Length % 2 == 0)
        {
            return (values[middle - 1] + values[middle]) / 2d;
        }

        return values[middle];
    }
}
```

Server garbage is counted by Unity's allocation counter

Measuring starts after a 1 minute warm-up and lasts 1 minute. The server spawns one object per tick from the framework's own pool. Each object writes 3 values on each of its next 2 ticks and is despawned on its third tick. The garbage Unity reports for every frame is added up and divided by the ticks measured. Unity only reports it in Development Builds, so both builds are Development Builds with script debugging off.

The script that spawns the objects and writes their values is different for each framework, so only the measuring part is shown:

```
// The garbage-measuring part of the benchmark probe. Each framework supplies CurrentTick(
), its own tick counter.

// GC Allocated In Frame is only reported by development players; release players leave it
 invalid.
private static Unity.Profiling.ProfilerRecorder _gcAllocatedRecorder;
private long _gcAllocatedBytes;
private uint _tickAtMeasureStart;

// At start-up:
_gcAllocatedRecorder = Unity.Profiling.ProfilerRecorder.StartNew(Unity.Profiling.ProfilerC
ategory.Memory, "GC Allocated In Frame");

// When the warm-up ends:
_gcAllocatedBytes = 0;
_tickAtMeasureStart = CurrentTick();

// Every frame of the measured window:
if (_gcAllocatedRecorder.Valid)
    _gcAllocatedBytes += _gcAllocatedRecorder.LastValue;

// When the window ends:
uint ticks = CurrentTick() - _tickAtMeasureStart;
double allocatedBytesPerTick = ticks == 0 ? 0d : (double)_gcAllocatedBytes / ticks;
```

Fusion runs with 100 objects

Fusion throttles beyond 100 objects. Its bandwidth is measured with 100 objects, since its bytes per object do not change with the count, and its CPU results are marked as failed.

CodeBoost reference

CodeBoost is a separate library, published under its own terms, that ships beside Nucleus (in the Unity package too) and provides logging, pooling and some low-allocation types. It is not part of the Nucleus API, but you are free to use it directly.

Replace the logger with LoggingService.UseLogger

`LoggingService.Logger` holds the active `ILogger`. It starts as a `ConsoleLogger`, so a plain .NET host gets console output with no setup. To replace it:

```
LoggingService.UseLogger(new MyServiceLogger());
```

`ILogger` declares `GetLoggerSetting()`, `DisableUnconditionalDevelopmentStacktrace()`, `LogInformation`, `LogWarning` and `LogError`.

- `LoggerSetting` holds three `LoggerLevel` values, `Editor`, `DevelopmentBuilds` and `ReleaseBuilds`, each one of `Disabled`, `Error`, `Warning` or `Information`. `LoggingService` uses the one for the current environment.
- Check `LoggingService.IsInformationEnabled`, `IsWarningEnabled` or `IsErrorEnabled` before formatting an interpolated log argument, so the string is not built when the level would drop it.
- For a level chosen at runtime, use `Logger.IsEnabled(LoggerLevel)` and `Logger.Log`, which takes the level as an argument. A `LogEntry` carries a level and a message together, so a line can be decided in one place and written in another. See Logging.
- `DisableUnconditionalDevelopmentStacktrace()` decides whether a development build adds a stack trace to every log call. `ConsoleLogger` returns `false`, so a development build traces its information and warning logs too. Errors always carry one.

Rent objects from the pools

- `ObjectPool<T0>` (constraint `new()`) rents and returns plain objects.
- `ResettableObjectPool<T0>` does the same for a type implementing `IPoolResettable`, calling `OnRent()` after renting and `OnReturn()` before storing it.
- `ListPool<T0>`, `HashSetPool<T0>`, `DictionaryPool<T0, T1>` and `QueuePool<T0>` pool the built-in collections.
- `Utf8EncodingPool` pools `UTF8Encoding` instances.

Each pool has `Rent()`, `Return(T0)` and `ReturnAndNullifyReference(ref T0)`.

```
List<int> list = ListPool<int>.Rent();
try
{
    // use list
}
finally
{
    ListPool<int>.Return(list);
}
```

Know the types Nucleus passes you

- `RingBuffer<T0>`: a fixed-size collection that overwrites its oldest entries once full.

- `BoostedQueue<T0>`: a queue used in place of `Queue<T0>` where Unity 2022's code generation cannot compile a `Queue<T0>` reference.
- `RentedArray<T0>`: an `IDisposable` wrapper around an `ArrayPool<T0>` rental; disposing it returns the array.
- `RoundRobinCursor<T0>`: walks a read-only list in wrap-around batches sized to sweep the whole list once over a configured time window.
- `Vector2Int`, `Vector3Int32`, `Vector3Int64`, `Vector4Int`: integer vectors.
- `QuaternionInt16`, `QuaternionInt32`: integer-component quaternions.
- `UnifiedColor`: an RGBA color stored as four bytes, with float accessors.

Mark pooled members so the build checks their reset

- `[PoolResettableMember]` marks a field or property that must be reset in `IPoolResettable.OnReturn()` and initialized in `OnRent()`.
- `[PoolResettableMethod]` marks a method that does that work for one or more such members outside `OnRent` and `OnReturn`.
- `[PoolDisposableMember]` marks a member that must be disposed rather than reset.

An analyzer in `CodeBoost.CodeAnalysis.Analyzers` fails the build when an `IPoolResettable` type leaves a `[PoolResettableMember]` out of its reset.

Check the environment and use the math helpers

The static `ApplicationState` class, an `IApplicationState`, answers `IsEditor()`, `IsDevelopment()`, `IsGuiBuild()`, `IsHeadlessBuild()`, `IsPlaying()` and `IsQuitting()`. `LoggingService` uses it to pick a `LoggerLevel`.

`MathCb`, in `CodeBoost.Mathematics`, holds floating-point and vector helper methods. `RoundingType` (`Down`, `Up`, `UpNonZero`, `ToEven`, `AwayFromZero`) selects the rounding rule for the integer divide extension methods.

Compatibility

Nucleus runs anywhere netstandard2.1 runs

The core targets `netstandard2.1`, with nullable reference types enabled and unsafe blocks allowed, and has no engine dependency. It runs on .NET 5 and later, Mono and Unity. On .NET Core 3.x, add to the consuming project:

- a `PackageReference` to `System.Runtime.CompilerServices.Unsafe` 5.0.0, which `CodeBoost.dll` depends on;
- `<LangVersion>9.0</LangVersion>`, because the generated code needs C# 9.

The Unity integration needs Unity 6.5 or newer

The Unity integration supports **Unity 6.5 (6000.5) and newer** and does not compile on older editors. Its files are wrapped in `#if UNITY_EDITOR || UNITY_6000_5_OR_NEWER`.

Integration and demo source is pinned to `LangVersion 9`, which Unity's compiler accepts. The prebuilt engine DLL is not subject to this.

Optional packages switch on extra code

Each of these compiles in only when its define is set. Without the package, the code is left out entirely.

Dependency	Define	Gates
`com.unity.addressables` 1.19+	`NUCLEUS_ADDRESSABLES`	Addressables-based asset loading
`com.unity.modules.ai`, the AI module, enabled in a new project	`NUCLEUS_NAVMESH`	`NetworkNavMeshAgent` and its inspector
`Unity.ResourceManager`	None	Referenced alongside Addressables
Relay transport	`BLITZ_RELAY`	The Blitz Relay transport (Pro only)

The demos need the Input System, uGUI 2.0 or newer (for TextMeshPro), AI Navigation, and the Vehicles and Terrain modules. Without them the demos are skipped, the project still compiles, and the editor warns once per session naming the packages to install. The demo materials use the Universal Render Pipeline (URP), so they draw pink on any other pipeline; the same warning says so.

Platform support depends on the transport

The core is platform-agnostic, so whether a platform works depends on the transport you use.

Component Index

Core managers

Component	Description
`UnityCoreManager`	Owns the process's single CoreManager and bootstraps every other `UnityManager` on its GameObject.
`UnityClientManager`	Inspector-side driver for the core client manager.
`UnityServerManager`	Inspector-side driver for the core server manager, including client limits and the per-tick object cap.
`UnityTransportManager`	Adds and configures the CoreManager's transport, and chooses whether the peer starts as a server, client, or neither.
`UnityNetworkLoopManager`	Carries the authored tick rate and gives the default physics world its driver.
`UnityPacketManager`	Inspector-side driver for the core packet manager.
`UnityMessageManager`	Inspector-side driver for the core message manager.
`UnitySystemManager`	Exposes the system manager's runtime knobs as serialized fields.
`UnityInterestManager`	Carries the world-wide interest rules and re-resolve interval.
`UnityPhysicsManager`	Inspector-side seat for the process-global physics execution mode and fixed-step alignment.
`UnitySceneManager`	Exposes the scene manager's placement policy and supplies a scene loader if none is placed.

State

Component	Description
`NetworkTransform`	Replicates a GameObject's position, rotation, and scale, in local or world space.
`NetworkAnimator`	Replicates an `Animator`'s parameters, layer weights, and speed.
`NetworkNavMeshAgent`	Replicates a `NavMeshAgent` as the path leg it is walking rather than as its raw pose.

Scenes

Component	Description

`NetworkSceneLoader`	Base behaviour for loading networked scenes; registers itself as the peer's scene loader.
`UnitySceneLoader`	Loads networked scenes additively through Unity's own scene loading.
`NetworkSceneManifest` (asset)	Maps scene identifiers to the scenes this peer loads and the content bundle each ships in.

Physics

Component	Description
`PhysicsSimulationDriver`	Aligns one scene's physics world with the network tick instead of Unity's own FixedUpdate.
`NetworkRigidbody`	Declares a rigidbody's replicated physics component and drives its sync, adopting, interpolating, or converging as needed.
`ReplayHeldBody`	Holds a `Rigidbody` or `Rigidbody2D` kinematic while a reconcile replays, for a body the replay does not rewind.

Prediction

Component	Description
`ValidatedTransform`	Keeps a transform's pose, scale, active state and layer in predicted members the server checks.
`ValidatedRigidbody`, `ValidatedRigidbody2D`	Keeps a body's pose, velocities and sleep state in predicted members, so its controlling client predicts it and a correction replays physics.
`ValidatedRigidbodyAdvanced`, `ValidatedRigidbody2DAdvanced`	Adds the body's kinematic flag, collision detection, constraints and center of mass.
`ValidatedCharacterController`	Keeps a `CharacterController`'s pose and enabled state in predicted members.
`ValidatedCharacterControllerAdvanced`	Adds the controller's height, center and radius.
`ValidatedConstantForce`, `ValidatedConstantForce2D`	Keeps a `ConstantForce`'s forces and torques in predicted members.
`ReconciledAnimator`	A `NetworkAnimator` for a predicted object, which also puts the parameters back before a replay.
`ValidatedAnimator`	Keeps an `Animator`'s parameters, layer weights and speed in predicted members the server checks, for parameters that decide gameplay.
`ValidatedAnimatorRootMotion`	Keeps the base layer's playback in predicted members, for root motion that decides where a character goes.

`GraphicalSmoother`	Glides a child graphic across each tick and each correction while the object stays at its true pose.

Lag compensation

Component	Description
`UnityLagCompensationManager`	Inspector-side seat for lag compensation (Pro): the rewind window, whether 2D queries reach 3D hitboxes, whether queries start in hitboxes, and the spawn compensation cap.
`NetworkHitboxRoot`	Gathers every `NetworkHitbox` on a networked object and registers them with the server, so shots can be judged against where they were; disabling it makes the object unhittable.
`NetworkHitbox`	One lag-compensated volume, a sphere, box or capsule or a 2D circle, box or capsule, copied from a collider beside it or typed in.

Transports

Component	Description
`NetworkTransport`	Base for an Inspector-configurable transport component.
`SynapseTransport`	Inspector-configurable component for the Synapse UDP transport: port, remote host, and timeouts.
`BlitzRelayTransport`	Inspector-configurable component for the Blitz Relay transport, which carries a session through a relay.

Bundles

Component	Description
`NetworkBundleLoader`	Base behaviour for loading content bundles; registers itself as the peer's bundle loader.
`UnityAssetBundleLoader`	Loads content bundles as Unity asset bundles and registers the prefabs they ship.
`NetworkBundleManifest` (asset)	Maps bundle identifiers to the asset bundle files this client loads them from.

Systems and spawning

Component	Description

`NetworkSystemObject`	Marks the root of a networked prefab or scene object; the single link between the GameObject and its NetworkSystem.
`NetworkPrefabCollection` (asset)	A serialized, enumerable container of networked prefabs shipped with a content unit.
`NetworkPlayerSpawner`	Spawns one prefab per authenticated client and hands that client control of it.
`NetworkInterestObject`	Authors an object's interest rules once, for every NetworkSystem it carries.
`NetworkHostVisibility`	Hides an object's renderers on a host while its own client half would not have been holding the object.
`NucleusBehaviour<TComponent0>`	Base for a script whose lifetime is one networked system; exposes the typed component it carries and this peer's role and control.

Developer tools

Component	Description
`NetworkCorrectionVisualizer`	Flashes a line from where a predicted position was shown to where a reconcile corrected it, on the client that predicts the object.
`NetworkInterestOverlay`	While selected on a server or host, draws a line to every networked object, colored by what the object's controller holds of it.

Configuration reference

Using Unity? See The Nucleus manager components for setting the same values from the inspector.

Every setting is a public field or property on the manager that owns it. Assign it after `new CoreManager()` returns. There is no engine-wide configuration object.

Two values cannot change after start-up

Every other setting can change at any time. These two cannot:

- **Tick rate.** `NetworkLoopManager.TickRate` is get-only, fixed by the `tickRate` argument passed when the loop starts. `DefaultTickRate` is 30. A rate below `MinimumTickRate` (5) or above `MaximumTickRate` (128) is clamped into range and logged.
- **`ServerConfiguration.MaximumConnections`** is read-only to your code. Change it with `transport.SetMaximumConnections(uint)`; Synapse applies the new cap the next time the server starts.

The step provider can change:
`NetworkLoopManager.UseNetworkLoopStepProvider(INetworkLoopStepProvider)` swaps it at runtime, stopping and returning the previous one. To use your own provider from the start, pass it to the `CoreManager` constructor instead, because after construction the default provider is already stepping the loop from the thread pool.

Only the tick rate must match between peers

Every other setting is local: a server and its clients can run different `StateInterpolation`, `EvaluationCadenceTicks`, timeout or retention values. A server and client with different `TickRate` values break replication and interpolation.

SystemManager

Setting	Default	Notes
`StateInterpolation`	0	Capped at `MaximumStateInterpolation` (7) when applied.
`GlobalRedundancy`	0	Only usable when the transport's default channel is unreliable. Capped at `StateInterpolation` on the server's sends.
`ClientRedundancy`	3	Extra resends a pure client adds to everything it sends upstream, on top of `GlobalRedundancy`. Not capped.
`StateRetentionMilliseconds`	1000	Capped at `MaximumStateRetentionMilliseconds` (2000).

`PartialStateApplyEnabled`	true	A tick that loses some of its packets applies the ones that arrived. Off, the whole tick is discarded and recovered. The receiver's choice alone.
`PredictionHistoryTicks`	`UnsetPredictionHistoryTicks` (0)	How many ticks of history `Predict` and `Validate` members keep. Unset, every client keeps `DefaultPredictionHistoryTicks` (64) and the server keeps two.
`MaximumReplayTicks`	`UnsetMaximumReplayTicks` (0)	Most ticks a reconcile replays; a longer one lands the server's values at its start tick without replaying. 0 sets no limit.
`ReplayHoldRadius`	`UnsetReplayHoldRadius` (0)	Client: only systems it does not control within this distance of a restored system are held during a replay. 0 holds every one.
`IsResultValueSendEnabled`	true	Read on the controller. On, a server that finds a `Validate` result different reports which members differed and both values through `NetworkSystem.ResultMismatched`, and each member's `resultTolerance` applies. Off saves bandwidth every tick; the report then shows only that a mismatch happened, and results are compared exactly.
`DefaultControllerRetentionPolicy`	`ControllerRetentionPolicy.Release`	A system's own override wins, then its group's, then this.
`ControllerRetentionSeconds`	120 (`DefaultControllerRetentionSeconds`)	`UnlimitedControllerRetentionSeconds` (0) removes the limit.
`MaximumRetainedControllerRecords`	64 (`DefaultMaximumRetainedControllerRecords`)	
`HostMigrationEnabled`	false	

InterestManager

Setting	Default	Notes
`EvaluationCadenceTicks`	5 (`DefaultEvaluationCadenceTicks`)	How often, in ticks, a system's interest is re-evaluated per connection.

`MaximumControlledInterestObjects`	1 (`DefaultMaximumControlledInterestObjects`)	`UnlimitedControlledInterestObjects` (0) measures against every controlled object.

SceneManager

Setting	Default	Notes
`JoinPlacement`	`JoinScenePlacement.EveryOpenScene`	Where a newly authenticated client is placed.
`LoadRequestTimeoutSeconds`	180	0 or less disables the timeout.
`AutomaticRequestOnBlockedSpawnEnabled`	false	
`SceneCarryFailure`	`SceneCarryFailureAction.SurrenderControl`	The world-wide answer. Override it per object with `NetworkSystem.SceneCarryFailureOverride`, or per case with a registered `ISceneCarryFailureResolver`.

RpcManager

Setting	Default	Notes
`MaximumInboundRpcsPerConnectionPerDrain`	64 (`DefaultMaximumInboundRpcsPerConnectionPerDrain`)	`UnsetMaximumInboundRpcsPerConnectionPerDrain` (0) removes the bound.

ServerManager

Setting	Default	Notes
`AuthenticationTimeoutSeconds`	30	A connecting client that never authenticates is disconnected after this. `UnsetAuthenticationTimeout` (0) disables the timeout.
`KickTimeoutMilliseconds`	500	How long a kicked or refused client's link may stay open after the kick before it is closed. Overloads taking `timeoutMilliseconds` override it for one kick.
`ObjectPacingEnabled`	`true`	Pro only. Off, every whole object is sent on the tick it is owed.

`MaximumObjectKilobytesPerTick`	50 (`DefaultMaximumObjectKilobytesPerTick`)	Pro only. The most kilobytes (1 kB is 1,024 bytes) of whole objects one client is sent per tick: new objects, and objects sent again whole after packet loss. The rest follow on later ticks. Ordinary updates to objects a client already holds are not paced. A value below 1 (`MinimumObjectKilobytesPerTick`) is read as 1, so this setting cannot switch pacing off.

ClientManager

Setting	Default	Notes
`DisconnectResetMode`	`DisconnectResetMode.ClearReceivedAtOnce`	What the client keeps of its received world on disconnect.

ViolationManager

Setting	Default	Notes
`DefaultAction`	`ViolationAction.Ignore`	A constant, used only for a violation type raised with no default of its own and no handler. `PacketTransformRejectedViolation` (Pro) is the only such type, and that raise always logs a warning. Every other type has its own default, mostly `Ignore`.

TransportManager

Setting	Default	Notes
`DefaultChannel`	`Channel.Unreliable`	
`HostLoopbackDelivery`	`HostLoopbackDelivery.Immediate`	Settable only to `Immediate`; any other value is refused and logged.
`RoundTripTimeIntervalMilliseconds`	1000 (`DefaultRoundTripTimeIntervalMilliseconds`)	0 disables round-trip time measurement.

BundleManager

Pro only.

Setting	Default	Notes

`AutomaticRequestOnBlockedSpawnEnabled`	`true`	A blocked spawn also asks the client for the bundle it needs, so content loads on demand.

Transports carry their own settings

`Transport.Configuration` and `Transport.ServerConfiguration` hold settings for a transport rather than a manager.

`Configuration`

Field	Default	Notes
`Port`	0	
`MaximumTransmissionUnit`	1200	Packets longer than this are split into multiple packets.
`ConnectingTimeoutSeconds`	10	How long a client waits for the server to answer its connect. Zero falls back to `ConnectedTimeoutSeconds`.
`ConnectedTimeoutSeconds`	15	Times out a connection that has received nothing within this window. The smallest value applied is one second.

`ServerConfiguration`

Field	Default	Notes
`MaximumConnections`	`UnsetMaximumConnections` (0)	0 accepts any number. Set it with `transport.SetMaximumConnections(uint)`; Synapse applies it at the next server start.

Editor menus and authoring tools

Open the Nucleus window from Tools > Nucleus

Tools > Nucleus is the only Nucleus menu entry. It opens the Nucleus window, or focuses it, on the tab it showed last. The banner names the edition (Pro or Free) and the peer's role: Host, Server, Client or Stopped while playing, and Edit mode outside play mode.

- **Traffic** shows the running session live: tick rate, bandwidth sent and received with a one-minute graph, connection and spawn counts, each connection's latency and packet loss, and simulated network conditions. Its buttons start and stop the server and the client. See Developer tools in the editor.
- **Setup** checks the open scene and offers a one-click fix for each failing row: **Create** a `UnityCoreManager`, **Add Synapse** as the transport, **Rebuild** the network prefab collection and the network scene manifest, and **Add** the open scene to the build. A **Validate Network Prefabs** button sits under the checklist. Once the scene has a `UnityCoreManager`, the tab also shows **Tick Rate**, which cannot change during play, and **Start On Play**, which decides how the session starts when play begins.
- **Build** has one row per build step, each with its own button: **Rebuild Network Prefab Collection** (**Rebuild**), **Rebuild Network Scene Manifest** (**Rebuild**), **Validate Network Prefabs** (**Validate**), and in Pro, **Build Content Bundles** (**Build**).
- **External Resources** opens the Nucleus website, this documentation, the demos guide (once the demos are imported) and the public tracker for bug reports and feature requests.

Log messages and dialogs name a build step by a path such as **Tools > Nucleus > Build > Rebuild Network Prefab Collection**. That means the step's button on the **Build** tab of the Nucleus window.

Run a build step from the Build tab

None of these steps runs at runtime, and each runs only when you click its button.

Rebuild Network Prefab Collection

Writes a `NetworkPrefabCollection` asset for every group of prefabs whose root carries a `NetworkSystemObject`, grouped by asset bundle. Prefabs with no asset bundle go into the base build's collection at `Assets/Resources/NetworkPrefabCollection.asset`.

Each prefab is stamped with a local prefab id and a bundle id. A prefab that already has a valid, unique id keeps it; only a prefab with no id or a duplicated one gets a new id, and each renumbering logs a warning naming the prefab, the prefab that already holds the id, and the new id.

An error is logged when two collections resolve to the same prefab bundle, because the second to load is refused at runtime. Give each content bundle assembly its own `[NetworkBundle(n)]`.

Validate Network Prefabs

Checks every prefab and logs each problem against the prefab, without changing anything: the identity problems a player build refuses (below), a `NetworkSystemObject` on a prefab's child, a networked component with no `NetworkSystemObject` at or above it, and a `NetworkHitboxRoot` that would record nothing. It loads every prefab, so it runs only when you ask. Developer tools in the editor lists each check.

Rebuild Network Scene Manifest

Records the scene id, path and content bundle of every scene enabled in Build Settings, and of scenes marked Addressable when the Addressables package is installed, in

`Assets/Resources/NetworkSceneManifest.asset`. A scene keeps its id across rebuilds; a new scene gets the next id above the highest recorded. A scene removed from the build keeps its entry, so its id is never reused.

Build Content Bundles

Pro only. Runs the prefab collection and scene manifest rebuilds first, then builds every asset bundle a `NetworkPrefabCollection` is assigned to into `Assets/StreamingAssets`, and records each bundle's id and file name in `NetworkBundleManifest`.

Some steps run on their own

- **Before every player build, a prefab check runs.** It fails the build when any networked prefab lacks the identity a rebuild would give it: no base collection, two collections sharing a prefab bundle, a prefab missing from its collection, a prefab with no id or a duplicated id, or a prefab stamped with a different bundle than its collection. The error lists every problem. Click **Rebuild** next to **Rebuild Network Prefab Collection** on the **Build** tab, then build every player you ship, server and client alike, again. The check never repairs prefabs itself.
- **Saving a scene assigns scene object ids.** A `NetworkSystemObject` whose `SceneObjectId` is unset or collides with another in the same scene gets the next free id; the first object holding a value keeps it.
- **A scene gets its id as soon as it enters the build.** The scene manifest updates when the Build Settings scene list changes, when a scene asset is imported, moved or deleted, when Addressables entries change, and right before a player build. It logs only when something changed.

Know when to re-run a step

Change	Step to run
Added, renamed, moved or duplicated a networked prefab	Rebuild Network Prefab Collection
Added or removed a scene from the build	Nothing. The scene manifest updates on its own; run Rebuild Network Scene Manifest by hand only if you want its log line confirming what mapped
Changed an asset bundle assignment (prefab or scene)	Rebuild Network Prefab Collection, then Build Content Bundles
Ready to ship the built asset bundles	Build Content Bundles (this also re-runs the prefab and scene rebuilds first)

Avoid two authoring traps

- **A duplicated prefab carries the original's id.** Until the next prefab collection rebuild, both share one prefab id. At runtime the first to register wins and the other logs an error, so spawning one can build the other, and player builds fail. Rebuild after every duplication. The rebuild keeps the id on the original, gives the copy a new one, and logs a warning.
- **A scene object id only has to be unique within its scene.** The same id in two different scenes is fine.

Scene reference fields show the resolved id

A `NetworkSceneReference` field is drawn as a scene-asset picker, with the scene id it resolves to shown

read-only beside it, looked up in the scene manifest each time.

The Nucleus inspectors add checks and readouts

- Every `NucleusBehaviour` shows its linked system, started state, id, state and control while playing.
- A `UnitySceneLoader` warns when no scene manifest is assigned, and a `UnityAssetBundleLoader` warns when no bundle manifest is assigned.
- A `NetworkCorrectionVisualizer` shows its flash count and the size of the last correction while playing.
- A `NetworkInterestOverlay` shows its options only while it is switched on, with a key to its colors.

FishNet to Nucleus name map

This map shows where FishNet and Nucleus concepts line up. It is not an alias list: write Nucleus names in Nucleus code. Some rows mark where the shapes differ, not just the names. There is no automated converter; port code by hand and write it in Nucleus's own shape.

Map objects and scripts

FishNet	Nucleus
`NetworkObject`	`NetworkSystem` in the core, `NetworkSystemObject` in the Unity integration
`NetworkBehaviour`	`NucleusBehaviour<T>` (Unity), layered over `MonoBehaviour` plus `NetworkSystemObject` plus `NetworkComponent`

A FishNet `NetworkObject` is one networked identity per `GameObject`. A Nucleus `NetworkSystem` is not tied to a `GameObject` in the core, and in Unity one `GameObject`'s `NetworkSystemObject` can link several `NetworkSystems`, each carrying its own `NetworkComponent`.

Map replicated state

FishNet	Nucleus
`SyncVar<T>`	`NetworkMember<T0>`
`SyncList<T>`	`NetworkListMember<T0>`
`SyncDictionary<TKey, TValue>`	`NetworkDictionaryMember<T0, T1>`
`SyncHashSet<T>`	`NetworkHashSetMember<T0>`
(no FishNet counterpart)	`NetworkArrayMember<T0>`
(no FishNet counterpart)	`NetworkQueueMember<T0>`
`OnChange` (per-member callback)	`NetworkComponent.OnMembersChanged(ulong memberFlags, MemberChangeDirection memberChangeDirection)`

`OnMembersChanged` fires once per component, with a flag mask of which members changed and a `MemberChangeDirection` (`Write` or `Read`) saying which side the change came from. There is no per-member changed event.

Map ownership and calls

FishNet	Nucleus
`Owner` / `IsOwner`	`ControllingClient` / `IsController(ControllerType.Client)`
`GiveOwnership(connection)`	`SetController(connection)`
`RemoveOwnership()`	`SetController(null)`

`[ServerRpc]` / `[ObserversRpc]` / `[TargetRpc]`	an `IRpc` struct sent with `TrySendRpc<T0>(RpcTarget, Channel, T0, RpcSelfDelivery)` and an `RpcTarget` (`RpcTarget.Server`, `RpcTarget.Observers`, `RpcTarget.ObserversExcept(...)`, `RpcTarget.To(connection)`)
`RequireOwnership`	`RpcSendAccess`
`IBroadcast`	`IMessage`

Nucleus has no attribute-driven RPCs. A remote call is a plain struct implementing `IRpc`, sent with `TrySendRpc`.

`ControllerType` also has `Server` and `AnyController`, for checking whether the local peer is the server, the controlling client, or either.

Map managers and infrastructure

FishNet	Nucleus
`TimeManager`	`NetworkLoopManager`
`InstanceFinder`	`CoreManager.Instance` (core) or `NucleusUnity.BoundCoreManager` (Unity)
`ObserverCondition`	`IInterestCondition`
`Authenticator`	`IClientAuthenticator`
`NetworkConnection`	`Connection`
`ClientId`	`Connection.Id`
kicking a client	`ServerManager.KickClient(connection)`

`CoreManager.Instance` is set when the first `CoreManager` is constructed, and `EnsureSetInstance` can point it at another. `NucleusUnity.BoundCoreManager` is the Unity equivalent. Each names one manager, so a process running several, such as a host pair or a two-peer test, should pass the `CoreManager` explicitly.

Some names are the same in both

- `Channel.Reliable`, `Channel.Unreliable`
- `IsHostStarted`, on a `NucleusBehaviour` as well as on `TransportManager` and `NetworkSystem`

`IsServerStarted` and `IsClientStarted` exist on `TransportManager` and `NetworkSystem`, but not on a `NucleusBehaviour`. Inside a script, use `IsStarted(Invoker.Server)` or `IsStarted(Invoker.Client)`.

Reference & Project Meta

These pages are for looking things up: type and component references, configuration, and compatibility.

- API type reference
- Compatibility
- How the benchmarks are run
- Component Index
- Configuration reference
- Editor menus and authoring tools
- Versioning, upgrading and release notes
- Installing and updating Nucleus in Unity
- FishNet to Nucleus name map
- Project assets and loaders
- CodeBoost reference

Installing and updating Nucleus in Unity

Driving the core API directly? See Adding Nucleus to a .NET project.

Install the package

Import the release's `.unitypackage` (**Assets > Import Package > Custom Package**), or copy its whole `Assets/Nucleus` folder into your project. These are the only supported methods: there is no UPM package, Asset Store listing or git-URL install.

The folder holds:

- `Nucleus.dll`, `CodeBoost.dll` and `SynapseSocket.dll`, each with a `.pdb` for debuggable stack traces.
- `Nucleus.CodeAnalysis.SourceGenerators.dll`, the source generator that makes networked types compile. It has no `.pdb`.
- `Nucleus.xml` and `SynapseSocket.xml`, which give your IDE hover docs for the public API.
- `LICENSE.txt`, and `VERSION.txt`, which names the release.
- An `Integrations` folder with four source trees, each its own assembly definition: `Unity`, `Blitz Relay`, `Newfarm Client` and `Newfarm Migration`.

The integration needs **Unity 6.5 (6000.5) or newer** and does not compile on older editors. An update never lowers that requirement.

Update without touching the .meta files

Leave Play mode. Replacing files during play reloads scripts and resets play mode.

Replace all four DLLs together with their `.pdb` and `.xml` files, plus `LICENSE.txt`, `VERSION.txt` and the `Integrations` source trees.

Keep every existing `.meta` file in the project.

Never replace the `.meta` files. `Nucleus.CodeAnalysis.SourceGenerators.dll.meta` carries the label that makes Unity run the generator:

```
labels:
- RoslynAnalyzer
```

If that `.meta` is overwritten or regenerated without the label, the project still compiles with no error, but nothing replicates. If replication silently stops after an update, open that `.meta` and confirm `RoslynAnalyzer` is listed under `labels`. The other `.meta` files keep asset GUIDs stable.

Take all four DLLs from one build. A mismatched set, such as a new `Nucleus.dll` beside an old `CodeBoost.dll`, still compiles but throws `MissingMethodException` when the mismatched call runs.

Optional pieces switch on with scripting defines

- `NUCLEUS_ADDRESSABLES` is set automatically when `com.unity.addressables` 1.19 or newer is installed.
- `NUCLEUS_NAVMESH` is set automatically while Unity's AI module, `com.unity.modules.ai`, is enabled, as it is in a new project. Without it, `NetworkNavMeshAgent` is left out and the rest still compiles.
- `BLITZ_RELAY` must be set in your project to compile the Blitz Relay transport.

The main integration assembly is `Nucleus.Integrations.Unity.asmdef`, with its editor code in

`Nucleus.Integrations.Unity.Editor.asmdef`. Every integration file is wrapped in:

```
#if UNITY_EDITOR || UNITY_6000_5_OR_NEWER
#define UNITY_ENGINE
#endif
```

so the same sources can sit in a non-Unity project without pulling in `UnityEngine`.

Identifying what you're running

`Assets/Nucleus/VERSION.txt` names the installed release. To check a built player, see Versioning, upgrading and release notes.

Project assets and loaders

A project needs a prefab collection and a scene manifest

A project with only base-build prefabs and scenes needs:

- One `NetworkPrefabCollection` at `Resources/NetworkPrefabCollection`. It registers automatically at startup. Without it nothing is registered, and a dynamic spawn fails to resolve its prefab on the receiving peer.
- One `NetworkSceneManifest` at `Resources/NetworkSceneManifest`, if it uses networked scenes.

The Build tab of the Nucleus window creates and rebuilds both. Everything else on this page is optional.

Register networked prefabs

NetworkPrefabCollection

A `ScriptableObject` (**Create > Nucleus > Network Prefab Collection**) listing the networked prefabs of one content shard. The base build has a default shard, and each asset bundle ships its own beside its prefabs.

- `_prefabs`, exposed as `Prefabs`: the prefabs in the shard.
- `_prefabBundleId`, exposed as `PrefabBundleId`: the bundle id the shard registers its prefabs under, taken from the shard assembly's `[NetworkBundle]` attribute. Zero is the base build's shard.
- `_bundleAssemblyName`, exposed as `BundleAssemblyName`: the simple name of the assembly that supplies that id; empty for the base build's shard.

`RegisterAll()` claims the shard's bundle id and registers every prefab; `UnregisterAll()` removes them and releases the claim when the shard's content unloads. Two loaded collections claiming the same bundle id is an error.

NetworkPrefabRegistry

The runtime lookup collections register into, keyed by `(BundleId, LocalId)`. A spawn resolves its prefab through `NetworkPrefabRegistry.TryGet`; a spawn whose identity has no entry cannot instantiate.

Recycle spawned instances with a prefab pool

Every dynamic spawn rents and returns its GameObject through an `INetworkPrefabPool`:

```
public interface INetworkPrefabPool
{
    GameObject Rent(GameObject prefabGameObject, ushort prefabBundleId, ushort prefabLocal
Id, Vector3 position, Quaternion rotation, Vector3 scale);
    void Return(GameObject pooledGameObject, ushort prefabBundleId, ushort prefabLocalId);
}
```

- **`DefaultNetworkPrefabPool`** (default): instantiates on rent and destroys on return.
- **`NetworkPrefabPool`** : deactivates a returned instance and reuses it on a later rent of the same prefab, taking the one parked longest. `ReturnCooldownSeconds` keeps a just-returned instance parked before it can be reused. `InstantiatedCount` and `ReusedCount` count new instances and reuses.

Assign the pool through `NucleusUnity.PrefabPool`; `null` restores `DefaultNetworkPrefabPool`:

```
NucleusUnity.PrefabPool = new NetworkPrefabPool { ReturnCooldownSeconds = 1f };
```

A reused instance does not run `Awake` again. Put per-life setup in the marker's `SystemLinked` and `SystemUnlinked` events, or in `OnEnable`.

Map and load networked scenes

NetworkSceneManifest

A `ScriptableObject` (**Create > Nucleus > Network Scene Manifest**) mapping the scene ids the server names to the scene assets a peer loads, and the content bundle each scene ships in. A project has one, loaded from `Resources/NetworkSceneManifest` by `NetworkSceneManifest.LoadProjectManifest()`, so nothing needs assigning in an inspector.

`_sceneEntries`, exposed as `Entries`, holds one `SceneEntry` per scene:

Field	Purpose
`SceneId`	The identifier the server names this scene by.
`SceneGuid`	The scene asset's GUID, which the identifier is keyed to.
`ScenePath`	The scene asset's path, handed to Unity's loader.
`RequiredBundleId`	The content bundle the scene ships in; the base build's identifier means it ships in the player.
`DeliveryMode`	How the peer gets the scene (`SceneDeliveryMode`), set by the build step.

NetworkSceneReference

A field type that names a networked scene by dragging in the scene asset. It stores only the scene's GUID (`_sceneGuid`), so renaming or moving the scene never changes which scene it names. `TryGetSceneId` resolves the id through the project's manifest when you use it, so a scene added later resolves once the manifest is rebuilt.

Go between a scene handle and a Scene

Use these methods on the `UnitySceneManager` component, not Unity's `SceneManager.GetSceneByName` or `GetSceneByPath`, which cannot tell apart several loaded instances of one scene:

```
public bool TryGetScene(uint sceneHandle, out Scene scene);
public bool TryGetSceneHandle(Scene scene, out uint sceneHandle);
```

`TryGetScene` returns the live Unity `Scene` for the server's scene instance handle, and false while the scene is still loading. `TryGetSceneHandle` is the reverse. A true result with `NetworkSystem.UnsetSceneHandle` means this peer loaded the scene itself and no instance will claim it.

NetworkSceneLoader and UnitySceneLoader

`NetworkSceneLoader` is the abstract `MonoBehaviour` base for loading networked scenes. It implements `ISceneLoader` and registers itself as the peer's scene loader on `Awake`; turn off `_automaticRegistrationEnabled` to call `Initialize` yourself. Derived classes implement `LoadSceneAsync` and `UnloadSceneAsync`.

`UnitySceneLoader` is the shipped loader. It resolves a scene through a manifest, loads the scene's content bundle first if it needs one, then loads the scene additively.

- `_sceneManifest`: the manifest to read. Left empty, it uses the project's manifest.
- `_localPhysicsEnabled`: loads each scene into its own physics world (`LocalPhysicsMode.Physics3D`) with a `PhysicsSimulationDriver` stepping it each tick, so a stacked scene's physics runs independently of the default world.

You do not need to place one: `UnitySceneManager` adds a `UnitySceneLoader` when no `NetworkSceneLoader` is in the loaded scenes. Add your own only to replace the loading.

Load content bundles

`NetworkBundleManifest` exists in every edition. `NetworkBundleLoader` and `UnityAssetBundleLoader` are Pro only and absent from a Free build, so a Free project has nothing built in that loads bundles. You need these only once prefabs or scenes ship in asset bundles rather than the base build.

NetworkBundleManifest

A `ScriptableObject` (**Create > Nucleus > Network Bundle Manifest**) mapping the bundle ids the server names to the asset bundle files a client loads.

- `_bundleEntries`, exposed as `Entries`: each `BundleEntry` pairs a `BundleId` with a `FileName`.
- `_rootDirectory`: the directory file names resolve against; empty uses `Application.streamingAssetsPath`.

NetworkBundleLoader and UnityAssetBundleLoader (Pro)

`NetworkBundleLoader` is the abstract `MonoBehaviour` base for loading content bundles. It implements `IBundleLoader` and registers itself on `Awake`, like `NetworkSceneLoader`. Derived classes implement `LoadBundleAsync` and `UnloadBundleAsync`.

`UnityAssetBundleLoader` is the shipped loader. Assign its `_bundleManifest`; without one, every load logs an error and fails. Once a bundle loads, it registers every `NetworkPrefabCollection` the bundle ships, so spawns waiting on that content can land.

Reporting a bug

Report bugs on the issue tracker. Its form asks for your version, edition, transport and topology. Attach a project that reproduces the fault: an issue without one is investigated at lower priority, and may be closed if it cannot be reproduced.

Keep the zipped project under 1 MB

Attachments over 1 MB are deleted, with no exceptions. Deleting the attachment does not close the issue, but it leaves the issue without a reproduction project. Do not attach an oversized project and explain the size instead.

Delete these folders, which Unity rebuilds when the project opens:

- `Library/`, usually most of the archive on its own.
- `Temp/` and `obj/`.
- `Build/`.
- `Logs/`. Paste the relevant log lines into the issue instead.
- `UserSettings/`.

Delete the `Assets/Nucleus` folder. **Do not include the Nucleus package**; give your version on the form.

Replace every third-party asset, such as anything from the Asset Store, with primitives and untextured materials.

Cut the project down to one scene showing one fault. If two things are wrong, open two issues.

Zip it. If it is still over 1 MB, something in `Assets/` is not needed for the bug; find it and remove it.

Do not send your whole game. Only you know which parts matter, so a full project is slower to investigate than none.

Say these things in the issue

- **The steps.** Which scene to open, what to press, and what to watch for.
- **The topology.** A host, or a dedicated server plus a client, and how many peers must be running. Many faults appear only as a host, or only without one.
- **How often it happens.** "Every time" or "roughly one run in four". An intermittent fault is still worth reporting.
- **Whether it also happens in a build**, or only in the editor.
- **Any simulated packet loss** you had switched on, with the percentage.

Versioning, upgrading and release notes

Check which build you are running

- In a project, `Assets/Nucleus/VERSION.txt` reads `Nucleus <version>`.
- A running player writes `Thank you for using Nucleus Networking <version>.` to its log at launch.

The DLLs carry no version number.

Run the same build on every peer

Peers do not exchange a protocol version when they connect, so nothing checks that both ends match. A client and server on different builds misread each other's packets or silently disagree about what a message means. If you see corruption or unexplained desync, confirm both ends run the same build first.

Replace removed members

Removed	Replacement
`IsAuthority`	`IsServerStarted` or `IsController`, depending on which you meant
`IsHostOwnClient`	removed, no direct replacement
`IsHostOwnServer`	removed, no direct replacement

A naming pass also renamed roughly 1,300 members. If a member you remember is not in the table and no longer resolves, search the current API for a similarly named member before assuming it is gone. If you still cannot find it, see the support page.

Violation reference

A violation is a struct implementing `IViolation`, raised through `ViolationManager` when a peer sends something the protocol does not permit. The offending operation is always rejected. The violation only decides the `ViolationAction` taken against the sending `Connection`: `Ignore` or `Kick`.

- Logging is separate from the action: `ViolationContext<T0>.Log` is a `LogEntry` the raise site may seed and a deciding handler may change or clear.
- Register a `ViolationHandler<T0>` to override a type's default action.
- Subscribe a `ViolationDetectedHandler<T0>`, or an `IViolationObserver`, to be told about a violation without changing the outcome.

There are twenty-two shipped violation types. Three exist only in Pro.

Read a violation you just saw

Most types default to `Ignore` and stay silent unless something sets `Log`, because an honest peer can reach them through an ordinary race (a control handoff, a revocation, a recovery re-send) or ordinary loss (`RetentionExceededViolation`, `RecoveryUnconfirmedViolation`, `PacketTransformRejectedViolation`). Do not kick for a single raise of those.

The four protocol-direction violations default to `Kick`, because an unmodified client never raises them. A raise means either a crafted client or your own code driving the request or report path from the wrong side. Check your own code first.

Peer-behavior violations default to Ignore

These come from state, spawning, remote calls and lag compensation.

Type	Payload	Check first
`UnauthorizedMemberChangeViolation`	`SystemId`	Whether the sender really controls that `NetworkSystem` right now; a control handoff racing a member change looks the same.
`PredictedMemberChangeViolation`	`SystemId`	Whether the member is really predicted. This usually means the member's transmission setup changed without the controller being rebuilt.
`EmptyCollectionDeltaViolation`	`SystemId`, `MemberId`	Whether the two builds' serializers differ; a compliant sender cannot produce an empty collection change.
`InvalidStateAckViolation`	`AckedTick`	Whether the two peers' tick counters really diverged (a restart, a save and load, a manual tick reset) before assuming a forgery.

`RetentionExceededViolation`	`AckedTick`, `ConsecutiveCount`	The peer's latency against `SystemManager.StateRetentionMilliseconds`. A single raise is ordinary heavy loss.
`RecoveryUnconfirmedViolation`	`UnconfirmedServeCount`	Whether packet loss is eating the peer's recovery itself before assuming it is stalling on purpose.
`RecoveryBudgetExhaustedViolation`	`ConsecutiveSeconds`, `SecondsInLastMinute`	The peer's packet loss. It is raised, at most once a second per `Connection`, for each second the server refused the peer some repair because it had already asked for its allowance. An honest peer gets through a burst of loss without it, so a long run means constant heavy loss, or a peer dropping packets or reporting losses on purpose.
`StateLossReportFloodViolation`	`AllowedLossRepairCount`	Whether the peer just came out of a stall, which lets an honest peer ask for several ticks of repair at once. Repairs past the allowance are refused either way.
`UnexpectedStateLossReportViolation`	`Tick`	Whether a handover changed which peer serves the sender while the report was in flight. The report is discarded, and this type logs a warning by default.
`UnauthorizedSpawnViolation`	`SystemId`, `SystemBundleId`, `SystemLocalId`	Whether that id was ever one the server despawned. A client re-sending a spawn the server just despawned raises nothing, so a raise means it was not.
`RpcSendPermissionViolation`	`SystemId`	Whether permission was revoked while calls were still in flight; an honest client reaches this on that race alone.
`RpcFloodViolation`	`ReceivedRpcCount`, `AllowedRpcCount`	`RpcManager.MaximumInboundRpcsPerConne against what the client really needs to send in one drain.

`LagCompensationViewClampedViolation`	`ReportedTick`, `ClampedTick`	The shooter's latency against `LagCompensationManager.MaximumRewindM The shot is still judged at the clamped tick, so only a steady stream from a client on a fast link suggests abuse. The type exists in both editions, but only Pro raises it.

Unreadable client data raises MalformedDataViolation

`MalformedDataViolation` fires on the server when reading data a client sent throws, and the data is not applied.

- In a release build it defaults to `Kick` and logs `Sent data the server could not read.` as a warning.
- In a debug build (`#if DEBUG`) it is `Ignore` with no log line, and the read site logs the exception instead.
- It is never raised on a client, for data with no known sender, for a server `Connection` (a host that dialled its own address reads from one as a client), or for a host's own client. There the read site logs the exception and nobody is kicked.

Type	Payload	Check first
`MalformedDataViolation`	`Kind` (`State`, `FullBody`, `Recovery`, `ClientState`, `Message`, `Rpc`), `SystemId` (zero when not for a system)	Whether both builds share the same serializers and members for that `Kind`; mismatched builds look exactly like a forged packet. Run a debug build to see the exception.

Exceeding a client limit raises ClientLimitViolation

`ClientLimitViolation` fires on the server when a client exceeds one of the limits on `ServerManager.ClientLimits`, which are off by default. The packet that crossed the limit is discarded unread. It defaults to `Kick` and logs a warning. A size limit raises it once per refused packet; a rate limit raises it once per one-second window and discards the rest of that client's packets in the window. A host's own client is exempt.

Type	Payload	Check first
`ClientLimitViolation`	`Kind` (`PacketBytes`, `SplitPacketBytes`, `PacketsPerSecond`, `BytesPerSecond`), `MeasuredValue`, `MaximumValue`, `Channel`	Whether the limit leaves room for what an honest client sends at its busiest. Set `ClientLimits.MaximumPacketBytes` at least one transmission unit above the largest packet a client honestly sends.

Protocol-direction violations default to Kick

These fire when a client answers about a bundle or scene it was never asked about.

Type	Payload	Check first

`UnexpectedBundleRequestViolation`	`BundleId`	Custom code calling the bundle-request path on the wrong side.
`UnsolicitedBundleReportViolation`	`BundleId`, `IsLoadReported`	Whether the server really requested that bundle load, or recorded the client holding it before an unload report. A mismatched build looks the same as a forged report.
`UnexpectedSceneRequestViolation`	`SceneHandle`, `IsLoadRequested`	Custom code calling the scene-request path on the wrong side.
`UnsolicitedSceneReportViolation`	`SceneHandle`, `IsLoadReported`	Whether the server really requested that load or release. A handle never requested or recorded held, including the unset handle, lands here too.

Three violations exist only in Pro

These types, and their events on `ViolationManager`, do not exist in a Free build.

Type	Payload	Default	Check first
`PacketTransformRejectedViolation`	`ChannelId`, `PayloadByteCount`	`ViolationManager.DefaultAction` (`Ignore`), always with a logged warning.	Whether the peer runs the same `IPacketTransform` configuration. Stray packets from anything else reach the port too, so this is not evidence of an attack.
`UnauthorizedDespawnViolation`	`SystemId`	`Ignore`	Whether the client and server builds agree on that prefab's removal policy; mismatched builds reach this honestly.
`UnsolicitedLeaseAcknowledgementViolation`		`Kick`	Whether the client build matches the server's. An honest client acknowledges only the predicted spawn top-ups the server sent it.

Add Nucleus to a .NET project

Using Unity? See Install Nucleus in Unity.

Nucleus ships as DLLs rather than a NuGet package. It targets `netstandard2.1`, so any .NET 5 or later project can use it.

Reference the DLLs

Copy these four DLLs from one release into a folder in your solution, such as `lib/Nucleus`: `Nucleus.dll`, `CodeBoost.dll`, `SynapseSocket.dll` and `Nucleus.CodeAnalysis.SourceGenerators.dll`.

Add this to your `.csproj`:

```
<ItemGroup>
    <Reference Include="Nucleus"><HintPath>..\lib\Nucleus\Nucleus.dll</HintPath></Referenc
e>
    <Reference Include="CodeBoost"><HintPath>..\lib\Nucleus\CodeBoost.dll</HintPath></Refe
rence>
    <Reference Include="SynapseSocket"><HintPath>..\lib\Nucleus\SynapseSocket.dll</HintPat
h></Reference>
    <Analyzer Include="..\lib\Nucleus\Nucleus.CodeAnalysis.SourceGenerators.dll" />
</ItemGroup>
```

The `Analyzer` line runs the Nucleus code generator over your project. Without it your project still compiles, but nothing replicates and no error appears. Add it to every project that declares networked types.

When you update, replace all four DLLs together. DLLs from different releases fail at runtime.

Give a second assembly its own bundle id

If networked types live in more than one of your assemblies, give each extra assembly its own id, in any one file:

```
[assembly: NetworkBundle(1)]
```

Your main assembly needs nothing, because it uses id 0.

Start Here

These pages take you from an empty project to two players moving on each other's screens. Read What Nucleus is, then follow the pages for how you use Nucleus.

Follow these pages if you use Unity

- Install Nucleus in Unity
- Set up a networked scene
- Add a player
- Test with two editors

If something does not show up on the other editor, see If nothing replicates.

Follow these pages if you use plain .NET

- Add Nucleus to a .NET project
- Your first .NET host
- Run two peers in one process
- Replicate a value

When you are done, see Where to go next. The Terminology page explains the words these pages use.

Install Nucleus in Unity

Using plain .NET? See Add Nucleus to a .NET project.

Nucleus needs Unity 6.5 (6000.5) or newer.

Import the package

In Unity, choose **Assets > Import Package > Custom Package**.

Select the Nucleus `.unitypackage` and click **Import**.

Everything is placed in `Assets/Nucleus`. When the import finishes, **Tools > Nucleus** appears in the menu bar.

Check that the code generator runs

Nucleus generates your networking code when Unity compiles. If the generator does not run, your project still compiles, but nothing replicates and no error appears.

In the Project window, open `Assets/Nucleus` and select `Nucleus.CodeAnalysis.SourceGenerators`.

At the bottom of the Inspector, check that the asset has the `RoslynAnalyzer` label.

If the label is missing, add it in the Inspector.

Update by importing the whole new package

When a new version comes out, import its `.unitypackage` over the old one. Never update only some of the DLLs in `Assets/Nucleus`, because DLLs from different versions fail at runtime.

To copy files by hand instead, or to see what each file is, read Installing and updating Nucleus in Unity.

If nothing replicates

If the second editor connects but does not show your objects, check these in order.

1. The session must be running in both editors

During Play, open **Tools > Nucleus** and select the **Traffic** tab. If it shows **Start** buttons instead of **Stop** buttons, the session never started. Set **Start On Play** to **Auto** in the Setup tab, or start the session from your own code with `StartHostAsync`, `StartServerAsync` or `StartClientAsync` on `UnityTransportManager`.

2. The code generator must be running

If the generator does not run, nothing replicates and no error appears. Check the label as described in Install Nucleus in Unity.

3. The prefab must be registered

If the client's Console shows this error, the client does not know the prefab:

```
No networked prefab is registered for identity [...]; spawned NetworkSystem Id [...] canno
t be instantiated.
```

In the Nucleus window's **Build** tab, click **Rebuild** next to **Rebuild Network Prefab Collection**. Then click **Validate** next to **Validate Network Prefabs**, which logs any other problem with your prefabs.

4. Your script must check the right role

Code that only runs when `IsController(ControllerType.Client)` is true does nothing on the server, and code that only runs on the server does nothing on a client. On a host, both the server and the client are running, which can hide a check that is wrong.

5. An object placed in a scene needs a networked scene

An object you place in a scene, rather than spawn, only appears on a client once the server has opened that scene through Nucleus and placed the client in it. See Networked scenes in Unity.

6. Interest rules must let the client see the object

Check the **World Rules** list on `UnityInterestManager` and the **Rules** list on the object's `NetworkInterestObject`, if it has one. Each rule can hide an object from a client. See What area of interest is.

Add a player

Using plain .NET? See Replicate a value.

This page gives every player a capsule that they move with WASD. Everyone else sees it move.

Make the player prefab

Choose **GameObject > 3D Object > Capsule** and name it Player.

Add the `NetworkSystemObject` component. It marks the object as networked.

Add the `NetworkTransform` component. It sends the object's position and rotation to everyone else.

Create a script named `PlayerMover` with the code below, and add it to Player.

Drag Player into the Project window to make it a prefab, then delete it from the scene.

```
using Nucleus.Integrations.Unity.Systems;
using Nucleus.Systems;
using UnityEngine;
using UnityEngine.InputSystem;

public class PlayerMover : NucleusBehaviour
{
    [SerializeField]
    private float _speed = 5f;

    private void Update()
    {
        // Only the player who controls this capsule moves it.
        if (!IsController(ControllerType.Client))
            return;

        Keyboard keyboard = Keyboard.current;

        if (keyboard == null)
            return;

        Vector3 direction = Vector3.zero;

        if (keyboard.wKey.isPressed)
            direction.z += 1f;
        if (keyboard.sKey.isPressed)
            direction.z -= 1f;
        if (keyboard.dKey.isPressed)
            direction.x += 1f;
        if (keyboard.aKey.isPressed)
            direction.x -= 1f;

        transform.position += direction.normalized * (_speed * Time.deltaTime);
    }
}
```

`NucleusBehaviour` is the base class for scripts on a networked object. `IsController(ControllerType.Client)` is true only on the computer of the player who controls this capsule, so each player moves only their own.

The script reads the keyboard through Unity's Input System package, which new Unity 6 projects include.

Spawn a player for everyone who joins

Create an empty GameObject named Player Spawner.

Add the `NetworkPlayerSpawner` component.

Drag the Player prefab into its **Player Prefab** field.

The spawner creates one Player for each player who joins, and gives that player control of it.

Register the prefab

Choose **Tools > Nucleus**, select the **Build** tab, and click **Rebuild** next to **Rebuild Network Prefab Collection**.

A prefab cannot be spawned over the network until you do this. Do it again whenever you add, remove or duplicate a networked prefab.

Press Play to check it

Press Play. A capsule appears, and WASD moves it.

Replicate a value

> **Using Unity?** See Add a player.

This page adds a score that the server changes and the client receives. It builds on the server and client from Run two peers in one process.

Declare the value

Put the values you want to replicate in a `NetworkComponent`, one `NetworkMember<T0>` field each:

```
using Nucleus.Components;

public partial class ScoreComponent : NetworkComponent
{
    public readonly NetworkMember<int> Score = new();
}
```

The class must be `partial`, because Nucleus generates the rest of it.

Create the object on the server

Once `server.TransportManager.IsServerStarted` is true, rent a network system that carries the component, and set the value:

```
using Nucleus.Systems;

if (NetworkSystemPool.TryRent<NetworkSystem, ScoreComponent>(server, out NetworkSystem? sc
oreSystem) && scoreSystem.TryGetComponent(out ScoreComponent score))
    score.Score.Value = 1;
```

Every value you write to `score.Score.Value` after this is sent to the client. When you no longer need the object, call `NetworkSystemPool.Return(scoreSystem)`.

Find the object on the client

Before the client connects, ask to be told when a system carrying `ScoreComponent` arrives:

```
NetworkSystemWatch? watch = NetworkSystemPool.Watch<NetworkSystem, ScoreComponent>(client,
 scoreSystem =>
{
    if (scoreSystem.TryGetComponent(out ScoreComponent score))
        Console.WriteLine($"The client found the score: {score.Score.Value}");
});
```

Call `NetworkSystemPool.Unwatch(watch)` when you no longer need it.

React when the value changes

Override `OnMembersChanged` in the component. `MemberChangeDirection.Read` means the change arrived from the network:

```
public partial class ScoreComponent : NetworkComponent
{
    public readonly NetworkMember<int> Score = new();

    public override void OnMembersChanged(ulong memberFlags, MemberChangeDirection memberC
hangeDirection)
    {
        if (memberChangeDirection == MemberChangeDirection.Read)
            Console.WriteLine($"Score is now {Score.Value}");
    }
}
```

Run the program and increase the score on the server every second. The client prints each new score.

Run two peers in one process

Using Unity? See Test with two editors.

The host on the previous page is one `CoreManager` playing both roles. To test a separate server and client, create two `CoreManagers` that talk over `Synapse`, the UDP transport, on this computer.

Create a server and a client

This program uses the `MainThreadStepProvider` from Your first .NET host:

```
using System.Diagnostics;
using Nucleus.Connections;
using Nucleus.Managers.Core;
using Nucleus.Transports.Synapse;

MainThreadStepProvider serverLoop = new();
MainThreadStepProvider clientLoop = new();

CoreManager server = new(networkLoopStepProvider: serverLoop);
Synapse serverSynapse = (Synapse)await server.TransportManager.AddTransportAsync<Synapse>(
);
serverSynapse.SetPort(7777);

CoreManager client = new(networkLoopStepProvider: clientLoop);
Synapse clientSynapse = (Synapse)await client.TransportManager.AddTransportAsync<Synapse>(
);
clientSynapse.SetPort(7777);

await serverSynapse.ConnectAsync(Invoker.Server);
await clientSynapse.ConnectAsync(Invoker.Client);

CancellationTokenSource stopSource = new();
Console.CancelKeyPress += (_, eventArgs) =>
{
    eventArgs.Cancel = true;
    stopSource.Cancel();
};

Stopwatch frameClock = Stopwatch.StartNew();
while (!stopSource.IsCancellationRequested)
{
    float frameDeltaMilliseconds = (float)frameClock.Elapsed.TotalMilliseconds;
    frameClock.Restart();

    serverLoop.Step(frameDeltaMilliseconds);
    clientLoop.Step(frameDeltaMilliseconds);
    Thread.Sleep(1);
}

client.Deinitialize();
server.Deinitialize();
```

`SetPort` sets the port the server listens on and the port the client connects to. Step both loops from the same thread.

Split them into two programs

To run the server and the client as separate programs, give each program only its own half. The client connects to `127.0.0.1` unless you set `RemoteHost` to the server's address:

```
clientSynapse.RemoteHost = "192.168.1.20";
```

Terminology

These are the words the Start Here pages use. Coming from FishNet? See the FishNet to Nucleus name map.

Peers

- A **peer** is one running copy of your game.
- The **server** is the peer that decides what is true.
- A **client** is a peer that connects to the server.
- A **host** is one peer that runs a server and a client together, so the person running it can play too.
- A **connection** (`Connection`) is the link to one peer.

Networked objects

- A **network system** (`NetworkSystem`) is one networked object. It is created and removed on every peer as a whole.
- A **network component** (`NetworkComponent`) is a class you write to list the values a network system replicates.
- A **network member** (`NetworkMember<T0>`) is one replicated value in a network component.
- To **replicate** a value is to send it from the server to the other peers so that every copy matches.
- To **spawn** an object is to create it on the server so that every client creates its own copy.

Control

- The **controller** of an object is the peer that drives it, which other engines call the owner. The server is the controller until it gives control to a client, as `NetworkPlayerSpawner` does with each player's object.
- `IsController` tells a script whether this peer is the controller.

Time

- A **tick** is one step of the network clock. Nucleus sends and applies updates once per tick.
- The **tick rate** is how many ticks run each second. It is 30 by default.

Visibility

- A client **observes** an object when it receives that object's updates.
- **Interest** is the set of rules that decides which objects each client observes.

Sending data

- A **message** is data you send to one connection with `Connection.SendMessage`.
- An **RPC** is a call you send on one network system with `NetworkSystem.TrySendRpc`, to the server, to every observer, or to one connection.
- `Channel.Reliable` data always arrives, in the order it was sent. `Channel.Unreliable` data can be lost.

Test with two editors

> **Using plain .NET?** See Run two peers in one process.

One editor only shows one player. To see a second player, run a second Unity editor on the same project with ParrelSync.

Open a second editor

1. Choose **Window > Package Manager**, click **+**, choose **Add package from git URL**, and enter `https://github.com/VeriorPies/ParrelSync.git?path=/ParrelSync`.
2. Choose **ParrelSync > Clones Manager**, click **Create new clone**, then click **Open in New Editor**.

The clone shares your project's Assets folder, so a change you save in one editor shows up in both.

Play in both editors

1. Press Play in the main editor. With **Start On Play** set to **Auto**, it becomes the host.
2. Press Play in the clone. It joins the host as a client.

Each editor shows two capsules. Click in one editor's Game view and press WASD: that capsule moves in both editors.

Stuck? See If nothing replicates.

What Nucleus is

Nucleus is a networking engine for server-authoritative multiplayer games. Its core is plain C# that runs on any .NET host. The Unity integration is a layer on top of that core, so the same networking code runs in a Unity game and in a dedicated server with no Unity installed.

Pick the pages for how you use Nucleus

- If you work in the Unity editor, follow the Unity pages in Start Here, then read the Unity section.
- If you call Nucleus from plain C#, such as a console app or a dedicated server, follow the .NET pages in Start Here, then read the Core API section.

Three ideas run through everything

Every networked object is a network system. A `NetworkSystem` holds one or more `NetworkComponents`, and each component declares its replicated values as `NetworkMember<T0>` fields:

```
public partial class ScoreComponent : NetworkComponent
{
    public readonly NetworkMember<int> Score = new();
}
```

The server decides what is true. The server sends each object's state to the clients. A client can only change an object the server lets it control, such as its own player.

Each client receives only the objects it can see. Interest rules decide which objects the server sends to each client. With no rules, every client receives every object.

Coming from FishNet? The FishNet to Nucleus name map lists the Nucleus name for each FishNet type.

Where to go next

Pick the page for what you want to build next.

If you use Unity

- To replicate your own values, read Replicating a value from a Unity script.
- To call a method on other peers, read Sending a remote call from a component.
- To spawn objects other than players, read Spawning and despawning objects.
- To decide who drives an object, read Control in a Unity script.
- To send each player only what is near them, read What area of interest is.
- To load levels or run several matches, read Networked scenes in Unity.
- To build a dedicated server, read Building a headless Unity server.

The Unity package also includes three tutorial games in `Assets/Nucleus/Integrations/Unity/Demos/Tutorials`. Each folder has a `README.html` that builds the game step by step.

If you use plain .NET

- To learn every option on a replicated value, read NetworkMember.
- To send messages, read Typed messages.
- To send remote calls, read Choosing who receives a call.
- To decide who drives an object, read Reading who controls a system.
- To create and remove objects, read Starting and stopping systems.
- To run a dedicated server, read Running a dedicated .NET server.

If something goes wrong

Read Common errors and what causes them, or report a bug.

Your first .NET host

Using Unity? See Set up a networked scene.

This page builds a console app that runs a server and a client in one process and sends a message from one to the other.

Add a loop provider

Nucleus does its work in a loop. Add this class to your project so the loop runs only when your code calls `Step`, on your own thread:

```
using Nucleus.Managers.NetworkLoop;

/// <summary>Steps the network loop only when called, on the calling thread.</summary>
sealed class MainThreadStepProvider : INetworkLoopStepProvider
{
    private readonly NetworkLoopStepDriver _driver = new();

    public bool IsStarted { get; private set; }

    public bool IsTickFrame => _driver.IsTickFrame;

    public void Initialize(NetworkLoopManager networkLoopManager) => _driver.Initialize(ne
tworkLoopManager, 1000f / networkLoopManager.TickRate);

    public void Start() => IsStarted = true;

    public void Stop() => IsStarted = false;

    public void Return() { }

    public void Step(float frameDeltaMilliseconds)
    {
        if (!IsStarted)
            return;

        _driver.AdvanceEarly(frameDeltaMilliseconds, isVariableUpdateAllowed: true);
        _driver.AdvanceLate(frameDeltaMilliseconds, isVariableUpdateAllowed: true);
    }
}
```

Write the program

The program below does five things in order:

It creates the session, a `CoreManager`, with the loop provider.

It adds `Yak`, a transport that connects a server and a client inside one process.

It registers a handler for a message, and sends that message when a client connects.

It starts the server and the client.

It steps the loop until you press Ctrl+C, then shuts the session down.

```
using System.Diagnostics;
using Nucleus.Connections;
using Nucleus.Managers.Core;
using Nucleus.Managers.Messages;
using Nucleus.Managers.Transports;
using Nucleus.Transports;
using Nucleus.Transports.Yak;

MainThreadStepProvider loop = new();
CoreManager coreManager = new(networkLoopStepProvider: loop);

TransportManager transportManager = coreManager.TransportManager;
Yak yak = new();
await transportManager.TryAddTransportAsync(yak);

coreManager.MessageManager.RegisterMessageHandler<HelloMessage>((in MessageContext context
, HelloMessage message) =>
{
    Console.WriteLine($"Received: {message.Text}");
});

transportManager.ConnectionRemoteStateChanged += change =>
{
    if (change.Connection.RemoteState is RemoteConnectionState.Connected && change.Connect
ion.IsClient)
        change.Connection.SendMessage(Channel.Reliable, new HelloMessage { Text = "hello f
rom the server" });
};

await yak.ConnectAsync(Invoker.Server);
await yak.ConnectAsync(Invoker.Client);

CancellationTokenSource stopSource = new();
Console.CancelKeyPress += (_, eventArgs) =>
{
    eventArgs.Cancel = true;
    stopSource.Cancel();
};

Stopwatch frameClock = Stopwatch.StartNew();
while (!stopSource.IsCancellationRequested)
{
    float frameDeltaMilliseconds = (float)frameClock.Elapsed.TotalMilliseconds;
    frameClock.Restart();

    loop.Step(frameDeltaMilliseconds);
    Thread.Sleep(1);
}

coreManager.Deinitialize();
```

The message is a struct in its own file:

```
using Nucleus.Managers.Messages;
using Nucleus.Serializers;

[NetworkType]
public struct HelloMessage : IMessage
{
    public string Text;
}
```

Run it, and the console prints `Received: hello from the server`.

Subscribe to events and register handlers before you call `ConnectAsync`. `Yak` connects inside `ConnectAsync`, so a handler added afterwards misses the connection.

Set up a networked scene

Using plain .NET? See Your first .NET host.

Every scene that uses Nucleus needs one `UnityCoreManager`. The Nucleus window adds it for you.

Add the core manager

Open your scene and save it.

Choose **Tools > Nucleus** and select the **Setup** tab.

Next to **A UnityCoreManager is in the scene**, click **Create**. This adds a GameObject named Nucleus that holds everything a session needs.

Click the button on every other row that shows a warning, until every row shows a tick.

Choose how the session starts

In the **Session** section of the Setup tab, set **Start On Play** to **Auto**. The first editor to press Play becomes the host, which runs a server and a client together so you can play too. A second editor finds the host already running and joins it as a client.

Leave **Tick Rate** at 30. It is how many times per second Nucleus sends updates, and 30 suits most games.

Press Play to check it

Press Play and select the **Traffic** tab of the Nucleus window. It shows **Stop Server** and **Stop Client** buttons, which means both halves of the host are running.

The session uses port 7777 on this computer. To play across computers, see Synapse Transport component.

Control in a Unity script

> **Driving the core API directly?** See Reading who controls a system.

A `NucleusBehaviour` runs on every peer that has the object, not just the controller. Check control before doing anything only the controller should do.

Check control with IsController

Inherit `NucleusBehaviour<TComponent0>`, or the non-generic `NucleusBehaviour` when another component, such as a `NetworkTransform`, declares the system and you only need the checks. Then call `IsController(ControllerType)`:

```
public class Health : NucleusBehaviour<HealthComponent>
{
    private void ApplyDamage(int amount)
    {
        if (!IsController(ControllerType.AnyController))
            return;

        Component.Health.Value -= amount;
    }
}
```

`ControllerType`	True when
`Server`	This peer is the server and no client controls the system.
`Client`	This peer is the client that controls the system.
`AnyController`	This peer is the controller either way. It combines `Server` and `Client`.

`IsController` returns false while no system is linked, so it is safe to call anytime.

Use the loud guards only for programming errors

`EnsureIsController(ControllerType)` and `EnsureIsStarted(Invoker)` return the same answer as `IsController` and `IsStarted`, but log a warning naming the calling member when it is false, once per calling member on each behaviour.

- Use `IsController` and `IsStarted` where a false answer is normal: input handling, RPC sends and per-frame or per-tick hooks all run on peers that do not control the object.
- Use the `Ensure` forms only where reaching the code without that control or role is a bug, such as a server-only method called only by other server code.

Check roles with IsStarted, not control

`IsStarted(Invoker)` reports whether this peer's server or client role is running. `Invoker` has only `Server` and `Client`; check a host with `IsHostStarted`.

A role is not control. On a host, `IsStarted(Invoker.Server)` is true for every object, including ones a client controls. Use `IsController` to ask whether this peer controls the object, and `IsStarted` or `IsHostStarted` for setup that depends only on the role.

React to control changing

Override `OnControllerChanged`. It fires on every peer when control of the linked system moves:

```
protected override void OnControllerChanged(Connection previousControllerConnection, Conne
ction currentControllerConnection)
{
    // previousControllerConnection and currentControllerConnection are null for the serve
r
}
```

It does not fire when a system links, even if this peer is already the controller. Read `IsController` in `OnSystemLinked` for the starting state.

Read NetworkSystem only while linked

`NetworkSystem` is null before the system links and after it unlinks. It is set when `OnSystemLinked` runs and still readable during `OnSystemUnlinked`. Reading it from `Awake` or an inspector callback reads it before the link.

Keeping a player's objects across a disconnect

> **Driving the core API directly?** See Controller retention.

By default, a disconnecting player's object stays in the world, uncontrolled, and nothing remembers who held it. With retention, the object is also held on a record and handed back to the same player if they reconnect before the record expires. An object whose record expires, is evicted or is redeemed by someone else is despawned.

These settings act only on the server. They do nothing on a client.

Set the project default on Unity System Manager

Under **Controller Retention** on the **Unity System Manager**:

Field	Default	Sets
On Controller Disconnect	`ControllerRetentionPolicy.Release`	`SystemManager.DefaultControllerRetentionPo the policy for every controlled object that does not override it.
Retention Seconds	120	`SystemManager.ControllerRetentionSeconds` how long a retained set stays redeemable. Zero never expires.
Maximum Retained Sets	64	`SystemManager.MaximumRetainedControllerR how many retained sets may stand at once before the oldest is evicted.

The last two are shown only while **On Controller Disconnect** is `Retain`.

Override it per prefab on NetworkSystemObject

Turn on **Override On Disconnect** on a prefab's `NetworkSystemObject`, then set its **On Controller Disconnect**, which defaults to `ControllerRetentionPolicy.Retain`. The override (`ControllerRetentionPolicyOverride`) covers every system the object carries.

Choose a Disconnect Mode on NetworkPlayerSpawner

Disconnect Mode (`NetworkPlayerDisconnectMode`) on `NetworkPlayerSpawner` decides what happens to the player objects it spawns, overriding the manager's default:

- **`Despawn`** (default): the object leaves with its player, and a returning player gets a new one.
- **`RetainForReturn`**: the object stays, uncontrolled, and is handed back through the spawner when the same `Connection.Identity` reconnects before the record lapses.

Use `RetainForReturn` only with an `IClientAuthenticator` that gives each player a real identity. The default `AddressClientAuthenticator` identifies a client by its address, and two players behind one NAT share an address, so the second can be handed the first player's object.

Read further

- Tokens, the retention events and reclaiming an object by hand: Controller retention.
- What happens to retained objects when the host itself is lost: Surviving the loss of a host.

Smoothing a predicted object with GraphicalSmoother

A predicted object moves once per tick, and a correction can move it again, so drawn as is it stutters and snaps. `GraphicalSmoother` (namespace `Nucleus.Integrations.Unity.Components`) smooths what is drawn without touching the simulation. It works for an object you place yourself each tick, a `ValidatedRigidbody` or `ValidatedCharacterController`, and anything else that moves once per tick. The `ValidatedJump` and `BallRacer` demos both use one.

Smooth a child graphic

Put the collider, the body and your scripts on the object, and the renderer on a child.

Add a `GraphicalSmoother` to the object and assign the child as its **Child Graphic**.

Each tick the child slides from where it was drawn to where the tick left the object, including across a correction. The object, its colliders and its hitboxes stay at the true pose, so physics, raycasts and lag compensation see exactly what the simulation did.

Adjust the slide in the inspector

Field	Default	Effect
Smoothing Duration Mode	**Fixed**	**Fixed** slides for **Duration** seconds, rounded up to whole ticks; the default of 0 is one tick. **Adaptive** slides for between **Minimum** (default 0) and **Maximum** (default 0.1 seconds), longer as the round trip grows toward **Maximum Latency (ms)** (default 200, at most 500).
Smoothed Properties	Position, Rotation	Which of position, rotation and scale are smoothed.
Child Graphic Enabled	On	Off smooths the object's own transform instead of a child.
Network Loop Callbacks Enabled	On	Turn off only to call `CapturePose`, `RestorePose` and `Smooth` yourself.

No slide lasts longer than half a second. A longer slide eases corrections out over more ticks, but the drawn object also trails steady movement by those extra ticks. The duration settings apply only with a child graphic.

Smoothing the object's own transform is safe for a `CharacterController` or a `Rigidbody`, because physics only steps inside a tick. Anything that reads the transform between ticks, a camera included, then reads the drawn pose, up to a tick behind the true one.

Follow the graphic with the camera in LateUpdate

Smoothing adds up to one tick of delay to what is drawn, and the slide is applied before your own `LateUpdate` scripts run. Follow the graphic from `LateUpdate`. From `Update`, on a frame that runs a tick, the camera reads the true pose, up to a tick ahead of what is drawn.

Smooth a member instead when you draw from Update

When you draw an object from a member's value in `Update` instead of placing it each tick, construct the member with `correctionSmoothingTicks` and draw its `InterpolatedValue`. `Value` is corrected at once, while `InterpolatedValue` eases from what was on screen. See Replaying a correction in Unity.

Control, Client State and Prediction

Pages in this category:

- Objects this peer does not control
- What control means
- Using prediction, from start to finish
- Sending client state from Unity
- Control in a Unity script
- Giving a client control of an object
- Prediction settings on the Unity manager
- When prediction is worth its cost
- Keeping a player's objects across a disconnect
- Replaying a correction in Unity
- Smoothing a predicted object with GraphicalSmoother

Prediction settings on the Unity manager

Driving the core API directly? See Predicted members and prediction history.

The prediction fields sit under **Prediction** on the Unity system manager. Set them in the inspector before you press Play.

Size the prediction history

Prediction History Ticks sets `SystemManager.PredictionHistoryTicks`: how many ticks of history `MemberRole.Predict` and `MemberRole.Validate` members keep, so a reconcile can restore a past tick and a replay can read each past tick's client state.

The default is 0 (`SystemManager.UnsetPredictionHistoryTicks`). Left at 0, every client keeps `SystemManager.DefaultPredictionHistoryTicks` (64 ticks, about a second at 60 ticks per second) for every object it holds, controlled or not, which covers most games. The server keeps two ticks.

Set a value only when the default does not fit. It must cover the worst round trip in ticks plus **State Interpolation**, with headroom: a 250 ms round trip at 60 ticks per second is 15 ticks, so with a State Interpolation of 2 use 24 or more.

- A history that falls short logs a warning once per system saying the server's value was older than its history holds, and every outcome on that system then reconciles with `MismatchCause.HistoryMissing`.
- Members take their history size when they initialize, so changing the value after objects have spawned does not resize existing members.
- A member on a paced send interval keeps a longer history of its own, whatever this field says.

Cap replay length from code

`SystemManager.MaximumReplayTicks` has no inspector field. Set it from code once the managers exist to cap how many ticks a reconcile replays. A longer reconcile lands the server's values at its start tick without replaying and sets `ReplayReport.IsReplaySkipped`. The default, 0, sets no limit.

Hold only nearby objects during a replay

During a replay, every object the client does not control is held at the replay's start tick. **Replay Hold Radius** (`SystemManager.ReplayHoldRadius`) limits that to objects within the radius of a restored object. An object whose position cannot be read is always held, and only Pro reads positions. The default, 0, holds every object, and a negative value is corrected to 0.

Send Result Values

Send Result Values (`SystemManager.IsResultValueSendEnabled`, on by default) is read from the controlling client. While it is on, a server that finds a client's `MemberRole.Validate` result different reports which members differed and both values through `NetworkSystem.ResultMismatched`, and a member's `resultTolerance` applies. Off saves bandwidth on every tick, but the report then shows only that a mismatch happened and on which tick, and results are compared exactly. Turn it off for a release build if you do not need the values.

Let observers predict an object

Tick **Predicted By Observers** under **Prediction** on a `NetworkSystemObject` to set `NetworkSystem.IsObserverPredictionEnabled` on every system it links, on every peer. Observing

clients then predict the object as its controller does.

- It only affects an observing client that itself controls an object sending client state.
- The object's simulation must carry missing client state with `TryGetLastState`.
- Each observing client pays a simulation per tick and a replay per reconcile for it.

See Predicting another client's object. For the replay that follows a correction, see Replaying a correction in Unity.

Objects this peer does not control

Most objects run on peers that do not control them: every client watching another player's character, and the server while a client controls an object. Every `NucleusBehaviour` hook, `OnControllerChanged` included, still runs on these peers every tick. Control only changes what your code inside the hook should do.

Skip controller-only work with IsController

```
protected override void OnEarlySerialize(StepDelta stepDelta)
{
    if (!IsController(ControllerType.AnyController))
        return;

    // Set gameplay members here.
}
```

Nothing stops a hook running on a non-controlling peer, so a script that does not check writes anyway, against members it does not own. See Control in a Unity script for `IsController`, `EnsureIsController` and `IsStarted`.

Do not move a proxy locally

With a `NetworkTransform`, a non-controlling peer follows the controller's position, rotation and, if enabled, scale, interpolating on `VariableUpdate`. Velocity is never sent. Anything that moves a proxy locally, such as a collision, a force or a script setting `transform.position`, is overwritten or fought on the next tick it receives.

- **Kinematic Management Enabled** (default on) holds an attached `Rigidbody` kinematic on every non-controlling peer, including the server while a client controls the object, so physics cannot push back against the pose. Turn it off only when other code owns the body's kinematic state.
- **Interpolate Scale Enabled** (default on) interpolates scale too. Off, the local scale keeps its current value.

See NetworkTransform for its other fields.

Raise State Interpolation for smoothness against jitter

State Interpolation on `UnitySystemManager` (`NucleusSystemManager.StateInterpolation`) is how many ticks received state waits before it is applied. The buffer lets a proxy interpolate between two real values instead of snapping to each one. It defaults to 1, the smallest the inspector allows. Raising it trades latency for smoothness against jitter and loss.

Within a tick, interpolation sweeps by `NetworkLoopManager.SubtickPercentage`, which updates every frame before `VariableUpdate`. To jump across a large discontinuity instead of sweeping, set a member's interpolation snap threshold; see NetworkMember.

Replaying a correction in Unity

Driving the core API directly? See Reconciliation and replay.

When a predicted member disagrees with the server, the engine restores the client's world to the server's values at a start tick and replays every tick since through your fixed updates. Put the simulation in `OnEarlyFixedUpdate` and the replay comes with it; there is no replay loop to write.

Simulate in OnEarlyFixedUpdate

Every `NucleusBehaviour` runs its two fixed-update virtuals once per simulated tick of the linked system:

```
protected virtual void OnEarlyFixedUpdate(StepDelta stepDelta, SimulationContext simulatio
nContext) { }
protected virtual void OnLateFixedUpdate(StepDelta stepDelta, SimulationContext simulation
Context) { }
```

`simulationContext.Type`	When
`Forward`	The first time a tick is simulated: on the controlling client once per live tick; on the server once per server tick, using one of the client's ticks for an object a client drives through client state.
`Replayed`	A past tick simulated again during a reconcile, on the controlling client or an observer predicting the object.
`None`	Nothing is simulated: a client watching an object it does not control. The call still comes once per tick.

- `stepDelta.Tick` is the local tick being simulated, the server's own tick on the server. A replayed step carries one tick's worth of every delta.
- `simulationContext.RemoteTick` is the other end's tick: on the server, the controller's tick whose client state is being simulated; on a client, the server tick of the state it applied. It is `NetworkLoopManager.UnsetTick` when there is none.
- `simulationContext.IsRemoteTickEstimated` is true when that tick was inferred rather than received, as on the server's step for a client tick whose client state did not arrive, and on every replayed tick.

Use `RemoteTick` only to pair a log line with what the other end did, never for timing: on the server it is a number the client chose. Time cooldowns and everything else by `stepDelta.Tick`.

With the client state as `MemberRole.Predict` members and the result as `MemberRole.Validate` members, the component looks like this:

```
using Nucleus.Components;
using NumericsVector2 = System.Numerics.Vector2;
using NumericsVector3 = System.Numerics.Vector3;

namespace MyGame
{
    public partial class PredictedMoverComponent : NetworkComponent
    {
        // Client state: written by the controller each tick.
        public readonly NetworkMember<NumericsVector2> Direction = new(role: MemberRole.Pr
edict);
        // Computed from client state by the server and, ahead of it, by the controlling c
lient.
        public readonly NetworkMember<NumericsVector3> Position = new(role: MemberRole.Val
idate, accuracy: 0.01f, correctionSmoothingTicks: 6);
    }
}
```

Write the step once and let every pass run it:

```
using Nucleus.Integrations.Unity.Serializers;
using Nucleus.Integrations.Unity.Systems;
using Nucleus.Managers.NetworkLoop;
using Nucleus.Systems;
using UnityEngine;

namespace MyGame
{
    public class PredictedMover : NucleusBehaviour<PredictedMoverComponent>
    {
        [SerializeField] private float _speed = 5f;

        private Vector2 _frameDirection;

        private void Update()
        {
            if (Component is null)
                return;

            // Replace with your project's input system.
            if (IsController(ControllerType.AnyController))
                _frameDirection = Vector2.ClampMagnitude(new Vector2(Input.GetAxisRaw("Hor
izontal"), Input.GetAxisRaw("Vertical")), 1f);

            transform.position = Component.Position.InterpolatedValue.ToUnity();
        }

        protected override void OnEarlyFixedUpdate(StepDelta stepDelta, SimulationContext
simulationContext)
        {
            if (simulationContext.Type is SimulationType.None)
                return; // a client watching an object it does not control: nothing to sim
ulate

            // Live tick on the controller: the keys become this tick's client state. A re
play re-reads the recorded client state instead.
            if (simulationContext.Type is SimulationType.Forward && IsController(Controlle
rType.AnyController))
                Component.Direction.Value = _frameDirection.ToNative();

            // The server's step for a tick whose client state never arrived: carry the la
st tick's instead of the defaults.
            if (stepDelta.IsClientStateMissing)
                Component.TryGetLastState(isEveryMemberCarried: true);

            // Runs for the live tick, on the server once per server tick on one of the cl
ient's ticks, and again for every replayed tick.
            Component.Position.Value = Step(Component.Position.Value.ToUnity(), Component.
Direction.Value.ToUnity()).ToNative();
        }

        private Vector3 Step(Vector3 position, Vector2 direction)
        {
            float seconds = 1f / CoreManager.NetworkLoopManager.TickRate;

            return position + new Vector3(direction.x, 0f, direction.y) * (_speed * second
s);
```

```
        }
    }
}
```

The server simulates the same tick from the controller's client state and orders a reconcile when its `Position` differs from the client's. While **Send Result Values** is on (the default), it also reports both values; see Send Result Values.

- **Run the same step on a replayed tick.** Read the `Predict` members and write the `Validate` members as usual, but never write the `Predict` members yourself, which is why the example sets `Direction` only on a `Forward` tick.
- **Give each `Validate` member an accuracy your simulation tolerates.** Its value is only as precise as its `accuracy`, on both peers.
- **Step by the fixed tick** (`1f / CoreManager.NetworkLoopManager.TickRate`), never `Time.deltaTime`, so every pass of a tick computes the same value.
- **Physics replays too.** It steps between the early and late fixed update of every replayed tick, so `CharacterController.Move` and the forces on a `ValidatedRigidbody` both replay. Bodies the replay does not rewind are held still; see Predicting a body with a validated behaviour.

The `ValidatedJump` demo under `Demos/Prediction` is a complete example: its movement lives in `ValidatedJumpMovementComponent.OnEarlyFixedUpdate`, and its behaviour only feeds it the keys and draws.

Play effects once

A replay re-runs every tick since the start tick, so play one-shot effects (sounds, particles, a jump animation) only on `SimulationType.Forward`. Outside the two fixed updates, the behaviour's `SimulationType` and `IsPredicting` (true on a client that replays this object) tell you whether a replay is running.

While a replayed tick runs, the engine refuses calls that would act twice, logging once per kind: `TrySendRpc` returns false, `Connection.SendMessage` sends nothing, predicted spawns report `PredictedSpawnRefusal.Replaying`, collection and reference writes change nothing, and `NetworkAnimator.SetTrigger`, `ResetTrigger`, `CrossFade` and `Play` do nothing.

Place the object around a replay

The engine does not move your Transform. Override the replay hooks to put the object at its member values, so replayed raycasts and `CharacterController.Move` calls start from the right place:

```
protected override void OnReplayStarting(ReplayReport replayReport)
{
    // Controlled: the restored start. Another player's object: its pose as of the start t
ick.
    transform.position = Component.Position.Value.ToUnity();
}

protected override void OnReplayCompleted(ReplayReport replayReport)
{
    // Controlled: the corrected present. Another player's object: back on its latest pose
.
    transform.position = Component.Position.Value.ToUnity();
}
```

They are called on a client for every object the replay covers: each one it controls, restored to `replayReport.StartTick` and then replayed, and each one it does not control, held at the server's

start-tick values and then returned to its latest values. Physics transforms are synced after each hook.

Read what was corrected

`replayReport.Mismatches` lists the predicted members that were wrong at the start tick, each with its `MemberName` and `Cause`, and `replayReport.Corrections` lists the members whose present the replay moved. `NetworkMember<T0>.TryGetMismatchValues` gives both values at the start tick, and `TryGetPreRewindValue` gives the value that was on screen, until the next tick advance:

```
protected override void OnReplayCompleted(ReplayReport replayReport)
{
    foreach (MemberCorrection memberCorrection in replayReport.Corrections)
        Debug.Log($"[{memberCorrection.MemberName}] corrected from tick {replayReport.Star
tTick}.");
}
```

To glide to a correction instead of jumping, construct the member with `correctionSmoothingTicks: 6` and render `InterpolatedValue`: it eases from the value that was on screen while `Value` is already correct. For an object you place on each tick instead, draw it through a `GraphicalSmoother`; see Smoothing a predicted object with GraphicalSmoother.

Hold a crowd, or predict other players

By default every object this client does not control is held at the start tick during a replay. Set **Replay Hold Radius** on the Unity system manager to hold only the ones near a restored object, or tick **Predicted By Observers** on a `NetworkSystemObject` to have observing clients predict and replay it instead of holding it. See Prediction settings on the Unity manager.

Replay from ReconcileRequired yourself

A game that must own the replay loop can subscribe to `NetworkSystem.ReconcileRequired` and replay in `OnReconcile`. Do this only for an object nothing simulates in `OnEarlyFixedUpdate` or `OnLateFixedUpdate`; otherwise the engine has already replayed it.

```
private uint _pendingRewoundTick = NetworkLoopManager.UnsetTick;

protected override void OnSystemLinked() => NetworkSystem.ReconcileRequired += OnReconcile
Required;

protected override void OnSystemUnlinked() => NetworkSystem.ReconcileRequired -= OnReconci
leRequired;

private void OnReconcileRequired(uint rewoundTick)
{
    if (rewoundTick != NetworkLoopManager.UnsetTick)
        _pendingRewoundTick = rewoundTick;
}

protected override void OnReconcile(StepDelta stepDelta)
{
    if (_pendingRewoundTick == NetworkLoopManager.UnsetTick)
        return;

    uint currentTick = CoreManager.NetworkLoopManager.Tick;

    for (uint tick = _pendingRewoundTick + 1; tick < currentTick; tick++)
    {
        NetworkSystem.BeginReplayTick(tick);

        // Re-run this tick's step: Predict members already read this tick's client state.

        NetworkSystem.EndReplayTick();
    }

    _pendingRewoundTick = NetworkLoopManager.UnsetTick;
}
```

- `ReconcileRequired` is raised on the `Reconcile` step after the restore, before `OnReconcile` runs.
- `rewoundTick` is the start tick. The server's value there already includes that tick's client state, so replay from the tick after it.
- `rewoundTick` is `NetworkLoopManager.UnsetTick` when a server `SendReconcile` arrived without the client state tick it processed. Skip the replay then.
- Bracket every past tick with `BeginReplayTick` and `EndReplayTick`, in tick order. A write to a predicted member outside the bracket lands on the live value and breaks the next comparison.
- A server correction of your client state alone does not raise `ReconcileRequired`. The component raises `OnMembersChanged` with `MemberChangeDirection.Correction` instead.

See also

- The restore, the report and every refusal are described in full in Reconciliation and replay.
- To predict a rigidbody, see Predicting a body with a validated behaviour. A body the server simulates while a client steers it converges instead; see Driving a predicted body.

What control means

Driving the core API directly? See What control means.

Every networked object answers three separate questions: who is the server, who controls it, and who observes it. Mixing them up causes bugs that work on one machine and fail over the network.

Each object has a server, at most one controller and any number of observers

- **The server** is the peer where `NetworkSystem.IsServerStarted` is true. Its word on an object's state is final, whatever any client believes.
- **The controller** is the one connection, the server or a single client, that drives the object. If no client controls it, the server does. Check the local peer with `IsController`; see Control in a Unity script.
- **An observer** is any peer the object is replicated to, decided by the interest system; see What area of interest is.

Only the server assigns a controller

The server assigns control with `SetController`; see Giving a client control of an object. A client that calls it is rejected: the server logs an error and nothing changes. `IsController` reads the controller the server replicated, never anything a client asserts about itself.

Control decides who sends client state, who predicts and who sets members

These are controller-only, whatever the write-access setting:

- **Client state.** Only the controller's client state is accepted.
- **Prediction.** The controlling client predicts the object from its own client state, and the server checks the result and corrects it when they differ. Other clients receive the server's state and hold the object still during a replay, unless **Predicted By Observers** (`NetworkSystem.IsObserverPredictionEnabled`) is ticked on its `NetworkSystemObject`.
- **Setting members.** The default access, `MemberWriteAccess.Controller`, allows the server or the controlling client. The Pro setting `MemberWriteAccess.AnyClient` widens this to other observers; see Letting clients set members.

Remote calls work the same way: `RpcSendAccess.Controller` is the default.

A connection must observe an object before anything else is checked

When a client sets a member or makes a remote call, the server checks, in order:

Does the connection observe the object? If not, it is refused.

Is it the object's controller? If so, it is admitted.

Otherwise it is admitted only when the access is `MemberWriteAccess.AnyClient` or `RpcSendAccess.AnyClient`.

Test control on a second machine, not only on a host

A host is the server, usually controls its own player object, and observes everything, so every check above passes there. Bugs such as a missing observer, an untested write-access setting or client state code that assumes the controller is the server only appear once a second machine joins.

Giving a client control of an object

Driving the core API directly? See Assigning and releasing control.

Give every player an object with NetworkPlayerSpawner

When every authenticated client needs one object of its own, add a `NetworkPlayerSpawner`. On the server it spawns **Player Prefab** for each client, on **Spawn Points** in turn, and gives the client control. It also sets `ControllerRetentionPolicyOverride` on every system of each object from its **Disconnect Mode**, overriding the manager's default retention policy. See Player objects for its fields, methods and events.

Assign control yourself with SetController

`SetController` belongs to the networked system, so call it through `NetworkSystem`. It only works on the server; a client's call is rejected and logged. Guard it with `IsStarted(Invoker.Server)` so it is skipped quietly on clients:

```
if (IsStarted(Invoker.Server))
    NetworkSystem.SetController(connection);
```

Passing `null` returns control to the server. For how repeated or conflicting calls resolve, see Assigning and releasing control.

Every peer sees OnControllerChanged

When control moves, `OnControllerChanged(previousControllerConnection, currentControllerConnection)` fires on every peer that has the system linked: the old controller, the new one and every other observer.

A `null` connection means the server, or a client this peer holds no `Connection` for. Control returning to the server reports `currentControllerConnection` as null, and an object spawned already under a client's control can report `previousControllerConnection` as null on a peer that cannot resolve who that was.

A controller-simulated object overwrites its Instantiate pose

`Instantiate(prefab, position, rotation)` places an ordinary controlled object correctly; `NetworkPlayerSpawner` does exactly this. But for an object whose pose the controller simulates, such as a `NetworkRigidbody`, the controller's simulation writes its own pose back onto the object within a tick of linking, replacing the pose `Instantiate` set.

On a host this often looks fine, because the host's own client is usually the peer asserting the pose. A second peer, starting from the true placement, shows the object snap.

Using prediction, from start to finish

> **Driving the core API directly?** See Predicted members and prediction history and Reconciliation and replay.

This guide builds a predicted player: a character that walks and jumps the instant its player presses a key, while the server stays in charge of what really happens. It recreates the `ValidatedJump` demo under `Demos/Prediction`; open the demo to see the finished code running.

> **The demo, and the player this guide builds, cause desyncs on purpose.** Only the server changes the jumper's health, and the client cannot see it fall in time, so a jump pressed as health drops past the threshold is shown on the client and refused by the server. This shows validation correcting the client. A real game would usually predict a drain the client can work out for itself.

Prediction hides the round trip from the player who is driving

With prediction, the client runs the movement code itself the moment a key is pressed, so the character moves at once instead of a round trip later. The server runs the same code with the same input and checks the client's answer. When they differ, the server sends its values, and Nucleus rewinds the client to the tick that went wrong and replays every tick since with the server's numbers.

Prediction only helps the player who controls the object; everyone else sees the server's values. Read When prediction is worth its cost before predicting anything other than a player's own character.

Every predicted member has one of three roles

Prediction is chosen per `NetworkMember<T0>`, with the `role` constructor argument:

Role	What it holds	Who writes it
`MemberRole.Predict`	What the player asked for this tick, such as a walk direction or a jump press. This is called client state.	The controlling client, on its live tick. The server receives it every tick.
`MemberRole.Validate`	What the movement code works out from the client state, such as a position or a vertical speed.	Every peer that runs the movement code. The client's value is checked by the server and never trusted.
`MemberRole.Replicate` (the default)	Anything only the server decides, such as health.	The server.

The usual predicted object uses all three. Nearly every prediction bug comes from a value in the wrong role.

Step 1: Declare the client state and the results in one component

Put the members and the movement code in a `NetworkComponent`. A component runs on every peer whether or not a scene object is linked to it, so the client and the server both run the code.

```
using CodeBoost.Types;
using Nucleus.Components;
using NumericsVector3 = System.Numerics.Vector3;

namespace MyGame
{
    public partial class JumperComponent : NetworkComponent
    {
        // Client state: what the player asked for this tick.
        // The walk direction: x in X, z in Y.
        public readonly NetworkMember<Vector2Int> MoveDirection = new(role: MemberRole.Pre
dict);
        public readonly NetworkMember<bool> IsJumpPressed = new(role: MemberRole.Predict,
isCarriedForward: false);

        // Results: what the movement code works out from the client state.
        public readonly NetworkMember<NumericsVector3> Position = new(role: MemberRole.Val
idate, accuracy: ValueAccuracy);
        public readonly NetworkMember<float> VerticalSpeed = new(role: MemberRole.Validate
, accuracy: ValueAccuracy);
        public readonly NetworkMember<bool> IsGrounded = new(role: MemberRole.Validate);

        // Only the server changes health, so it keeps the default role. This causes desyn
cs intentionally, to show prediction validation.
        public readonly NetworkMember<ushort> Health = new();

        public const float ValueAccuracy = 0.01f;
    }
}
```

Three choices in this declaration matter:

- **The walk direction is whole numbers.** It is a `CodeBoost.Types.Vector2Int` (not Unity's `Vector2Int`) whose axes are each -1, 0 or 1. A normalized float direction would be rounded on its way to the server, so the server would walk slightly differently and report a mismatch every tick. Send client state in a form that arrives exactly as written.
- **The jump press is never carried forward.** `IsJumpPressed` is true only on the tick jump was pressed, and `isCarriedForward: false` keeps it false on a tick whose client state was lost, so the server never jumps twice for one press. A press lost with its packet is simply lost, and the server corrects the jump the client showed.
- **Each float result has an accuracy.** A `Validate` member is only as precise as its `accuracy`, on both peers. Choose one your movement can tolerate; 0.01 is a centimetre for a position.

Step 2: Write the movement code once, in OnEarlyFixedUpdate

`OnEarlyFixedUpdate` runs once per simulated tick: the client's live tick, the server's copy of that tick, and every tick a correction replays. Write the movement there and every pass runs the same lines.

```
using Nucleus.Managers.NetworkLoop;
using Nucleus.Systems;

public partial class JumperComponent
{
    public const float MoveSpeed = 5f;
    public const float JumpSpeed = 7f;
    public const float Gravity = 20f;
    public const ushort JumpMinimumHealth = 50;

    protected override void OnEarlyFixedUpdate(StepDelta stepDelta, SimulationContext simu
lationContext)
    {
        // A client watching someone else's jumper has nothing to simulate.
        if (simulationContext.Type is SimulationType.None)
            return;

        // The server's step for a tick whose client state never arrived: keep walking wit
h the last tick's input, but never repeat a press.
        if (stepDelta.IsClientStateMissing)
            TryGetLastState(isEveryMemberCarried: false);

        if (IsJumpPressed.Value && IsGrounded.Value && Health.Value >= JumpMinimumHealth)
            VerticalSpeed.Value = JumpSpeed;

        Move(1f / CoreManager.NetworkLoopManager.TickRate);
    }

    private void Move(float deltaSeconds)
    {
        Vector2Int moveDirection = MoveDirection.Value;
        NumericsVector3 position = Position.Value;
        float diagonal = moveDirection.X != 0 && moveDirection.Y != 0 ? 1.41421354f : 1f;

        position.X += moveDirection.X / diagonal * MoveSpeed * deltaSeconds;
        position.Z += moveDirection.Y / diagonal * MoveSpeed * deltaSeconds;

        float verticalSpeed = VerticalSpeed.Value - Gravity * deltaSeconds;
        position.Y += verticalSpeed * deltaSeconds;

        bool isGrounded = position.Y <= 0f;

        if (isGrounded)
        {
            position.Y = 0f;
            verticalSpeed = 0f;
        }

        Position.Value = position;
        VerticalSpeed.Value = verticalSpeed;
        IsGrounded.Value = isGrounded;
    }
}
```

Follow these rules inside the movement code, or the client and the server will disagree:

- **Step by the tick, never by the frame.** Use `1f / CoreManager.NetworkLoopManager.TickRate`. `Time.deltaTime` differs on every machine and every replayed tick.

- **Read only members and constants.** Everything the step reads must be the same on both peers for that tick. A value from the scene, a static, `Random` or the current time is not, unless you make it so.
- **Keep the results in members.** The engine rewinds members, not your private fields, so a replay would start from the wrong field value.
- **Use plain arithmetic, a `CharacterController` or a validated behaviour.** A replay steps the whole physics world once per replayed tick, so a `Rigidbody` can be predicted too: add a `ValidatedRigidbody` and apply its forces in `OnEarlyFixedUpdate`. See Predicting a body with a validated behaviour.

Step 3: Read the keys in Update and write them on the live tick

Read the device in `Update`, since a key press belongs to a rendered frame. Hand it to the component, and write it into the `Predict` members only on the controlling peer's live tick (`SimulationType.Forward`). A replayed tick reads the client state recorded for it and must never overwrite it.

Add this to the component:

```
public Vector2Int IntendedMoveDirection;

private bool _isJumpPressPending;

public void PressJump() => _isJumpPressPending = true;

private void WriteClientState()
{
    MoveDirection.Value = IntendedMoveDirection;

    IsJumpPressed.Value = _isJumpPressPending;
    _isJumpPressPending = false;
}

public override void OnReturn()
{
    base.OnReturn();

    IntendedMoveDirection = default;
    _isJumpPressPending = false;
}
```

Call it near the top of `OnEarlyFixedUpdate`, after the missing-state check:

```
bool isLiveController = simulationContext.Type is SimulationType.Forward && NetworkSystem.
IsController(ControllerType.AnyController);

if (isLiveController)
    WriteClientState();
```

Then feed the keys from a `NucleusBehaviour` on the prefab:

```
using Nucleus.Integrations.Unity.Systems;
using Nucleus.Systems;
using UnityEngine;

namespace MyGame
{
    public class Jumper : NucleusBehaviour<JumperComponent>
    {
        private void Update()
        {
            if (Component is null || !IsController(ControllerType.AnyController))
                return;

            // Replace with your project's input system.
            Component.IntendedMoveDirection = new(Mathf.RoundToInt(Input.GetAxisRaw("Horiz
ontal")), Mathf.RoundToInt(Input.GetAxisRaw("Vertical")));

            if (Input.GetKeyDown(KeyCode.Space))
                Component.PressJump();
        }
    }
}
```

`ControllerType.AnyController` is true on the controlling client and on a host playing its own object, so the same script works in both. A host is the server, so its own object is never corrected.

Step 4: Let the server set up the object and check the input

Give the object its starting values on the server when it links:

```
protected override void OnSystemLinked()
{
    if (IsStarted(Invoker.Server))
    {
        Component.Position.Value = new(transform.position.x, 0f, transform.position.z);
        Component.Health.Value = 100;
    }
}
```

Client state comes from a player's machine, so treat it as untrusted. Implement `IMemberWriteValidator<T0>` on the component to check each value on the server before the step reads it. This one stops a modified client from walking faster than one step per axis:

```
public partial class JumperComponent : NetworkComponent, IMemberWriteValidator<Vector2Int>
{
    public MemberWriteAction ValidateMemberWrite(NetworkMember<Vector2Int> networkMember,
Vector2Int proposedValue, Connection writingClientConnection)
    {
        Vector2Int clampedValue = new(System.Math.Clamp(proposedValue.X, -1, 1), System.Ma
th.Clamp(proposedValue.Y, -1, 1));

        if (clampedValue.X == proposedValue.X && clampedValue.Y == proposedValue.Y)
            return MemberWriteAction.Accept;

        networkMember.Value = clampedValue;

        return MemberWriteAction.Decline;
    }
}
```

Writing the member and returning `Decline` keeps your corrected value. The server simulates with it, its result differs from the client's, and the client is corrected. You never need to validate `Validate` members: the server works out its own results and never uses the client's.

Finally, change health on the server only. In the demo the server drains health while the jumper stands in a red zone, which causes the intentional desyncs described at the top:

```
if (CoreManager.TransportManager.IsServerStarted)
    DrainOrRecoverHealth();
```

Step 5: Draw the object every frame, and smooth the corrections

The members change once per tick, but the screen draws many frames per tick. There are two ways to draw smoothly between ticks.

The first, which the demo uses, is to place the object each tick and let a `GraphicalSmoother` glide a child graphic between those places. Put the collider and the root on the object, the mesh on a child, add a `GraphicalSmoother`, and place the root in the behaviour's own `OnEarlyFixedUpdate`:

```
protected override void OnEarlyFixedUpdate(StepDelta stepDelta, SimulationContext simulati
onContext)
{
    System.Numerics.Vector3 position = Component.Position.Value;
    transform.position = new(position.X, position.Y, position.Z);
}
```

The component's step has already run for that tick, because a system runs its components before its behaviours. The smoother also glides across a correction, so a taken-back jump drops smoothly instead of snapping. See Smoothing a predicted object with GraphicalSmoother.

The second is to render `InterpolatedValue` from `Update` and construct the member with `correctionSmoothingTicks`:

```
public readonly NetworkMember<NumericsVector3> Position = new(role: MemberRole.Validate, a
ccuracy: ValueAccuracy, correctionSmoothingTicks: 6);

private void Update() => transform.position = Component.Position.InterpolatedValue.ToUnity
();
```

`Value` is corrected at once, so the simulation is always right, while `InterpolatedValue` eases from what was on screen over six ticks.

Step 6: Play effects once, on the live tick

A replay runs your step again for every tick since the one that went wrong, so anything the step does besides member writes happens again. Play sounds, particles and animation triggers only on the live tick:

```
if (IsJumpPressed.Value && IsGrounded.Value && Health.Value >= JumpMinimumHealth)
{
    VerticalSpeed.Value = JumpSpeed;

    if (isLiveController)
        PlayJumpSound();
}
```

During a replayed tick, remote calls, messages, predicted spawns, collection and reference writes and `NetworkAnimator` triggers do nothing, and each logs once. Your own side effects, such as adding score or writing a static, are not caught, so gate them yourself. `IsPredicting` and `SimulationType` on the behaviour tell code outside the fixed updates whether it is in a replay.

Step 7: Undo what the player saw when the server disagreed

A jump the client played and the server refused has already been seen. To react, for example to cancel a jump sound or show a hit marker as missed, listen for the correction. `OnMembersChanged` on the component reports `MemberChangeDirection.Correction` for the members a correction changed, and `TryGetPreRewindValue` gives the value that was on screen before it:

```
public readonly NetworkMember<ushort> JumpCount = new(role: MemberRole.Validate);

public override void OnMembersChanged(ulong memberFlags, MemberChangeDirection memberChang
eDirection)
{
    if (memberChangeDirection is not MemberChangeDirection.Correction || (memberFlags & (u
long)JumperComponentFlags.JumpCount) == 0)
        return;

    if (JumpCount.TryGetPreRewindValue(out ushort shownJumpCount) && JumpCount.Value < sho
wnJumpCount)
        CancelJumpEffects(shownJumpCount - JumpCount.Value);
}
```

`JumperComponentFlags` is generated for you from the component's members.

For a report of the whole correction, override `OnReplayCompleted` on the behaviour. `replayReport.StartTick` is the tick the replay started from, `replayReport.Mismatches` lists the members that were wrong there with a `MismatchCause`, and `replayReport.Corrections` lists the members whose present value the replay moved. See Replaying a correction in Unity.

Step 8: Make the scene ready for prediction

- **Test with a dedicated server and a separate client.** A host never predicts its own object, because it is the server. Run the server in one editor and the client in a second editor or a clone, or you will never see a correction.
- **Give the client control of its object.** Prediction runs only on the controller. `NetworkPlayerSpawner` hands each joining client its player; to do it yourself, see Giving a client control of an object.
- **Do not add a `NetworkTransform`.** The position is a `Validate` member this code replicates itself, and a `NetworkTransform` would fight it.

- **Check Prediction History Ticks on the system manager.** The default keeps 64 ticks, about a second at 60 ticks per second, which covers most games. Very high latency needs more; see Prediction settings on the Unity manager.
- **Turn off Send Result Values for a release build.** It lets the server name exactly what differed, which helps while you develop, but it costs bandwidth every tick.

A healthy predicted object is corrected rarely

Run the scene with some latency and watch the counters:

- `NetworkSystem.ReconcileCount` on the client counts the corrections it applied.
- `Connection.ResultMismatchCount` and `Connection.ReconcileSentCount` on the server count the results it found different and the corrections it ordered for that client.
- `NetworkSystem.ResultMismatched` on the server reports which members differed and both values, while **Send Result Values** is on.

With no packet loss and no server-only changes, the count should stay at zero while you walk and jump. A count that climbs every tick means the client and the server are not running the same code on the same numbers. Check these causes, most likely first:

The step reads something that is not a member. A Transform, a static, `Time.deltaTime`, `Random`, or a field that is not rewound.

Client state is rounded on the wire. A float input with no fixed step, so the server reads a slightly different value from the one the client wrote.

A result has no accuracy, or one too fine for the step. Give each float `Validate` member an accuracy.

Client state is written on a replayed tick. Only the `Forward` tick of the controller may write it.

The prediction history is too short. The log warns once per object that the server's value was older than the history holds, and the mismatch cause reads `MismatchCause.HistoryMissing`.

For a wider checklist, see Diagnosing desync.

Prediction goes further than one player

- **Predict other players' objects too.** Tick **Predicted By Observers** on a `NetworkSystemObject` so observing clients predict it as its controller does, which matters for players who collide or push each other. See Prediction settings on the Unity manager.
- **Hold only the nearby crowd during a replay.** Set **Replay Hold Radius** on the system manager so a replay holds only the objects near the ones it restores.
- **Spawn objects before the server answers.** A projectile can appear the instant it is fired and be confirmed or refused later. Predicted spawning is a Pro feature; see Client-predicted spawning in Unity and the two `PredictedProjectile` demos under `Demos/Prediction`.
- **Predict a physics body.** Add a `ValidatedRigidbody` and push the body from `OnEarlyFixedUpdate`; see Predicting a body with a validated behaviour. For a body the server simulates while a client steers it, see Driving a predicted body.
- **Keep client state flowing through loss.** `TryGetLastState`, carried members, and the late and dropped counts on `Connection` are covered in Sending client state from Unity.

When prediction is worth its cost

Driving the core API directly? See When prediction is worth its cost.

Prediction only hides the controller's round trip

With prediction, the controlling client applies its own input at once instead of waiting for the server's answer, and the server later confirms or corrects it. That is the whole benefit. It does not make an object more accurate, more responsive to other peers or cheaper to replicate. An object nobody controls has no round trip to hide, and most of a world (static geometry, background actors, anything only the server moves) is like that.

Prediction costs memory, bandwidth and replay time

You turn prediction on per member with the `role` argument of `NetworkMember<T0>`: client state declared `MemberRole.Predict` and results declared `MemberRole.Validate`. That costs:

- **History.** Every client keeps `SystemManager.PredictionHistoryTicks` of history for a predicted member, or `SystemManager.DefaultPredictionHistoryTicks` (64) when that is unset. A member sent on a slower interval than the tick rate keeps more.
- **Bandwidth and replays.** A matching result costs a little upstream bandwidth each tick, more while **Send Result Values** (`SystemManager.IsResultValueSendEnabled`) is on. A result the server finds different makes the client re-run `OnEarlyFixedUpdate` and `OnLateFixedUpdate` as `SimulationType.Replayed` for every tick since, about a round trip plus the state interpolation in ticks, all in one frame. Physics replays too, so a `ValidatedRigidbody` can be predicted; see Predicting a body with a validated behaviour.
- **Code that tolerates being re-run.** Anything a replayed tick does happens again. The engine refuses remote calls, messages, predicted spawns, collection and reference writes and `NetworkAnimator` triggers during a replay, but not your own side effects, such as awarding score or writing a static. Gate those on `SimulationType.Forward`, or keep the member that drives them unpredicted. A value that only feeds a display, such as a camera or a resource bar, is safe: at worst the display corrects itself.

Use a cheaper option when you can

Most objects need none of this:

- **An uncontrolled object with interpolation.** No history, no result check, no replay. The receiver animates between the values it receives, paced by **State Interpolation** (`SystemManager.StateInterpolation`, at most `MaximumStateInterpolation`). This is the default for anything nobody drives.
- **A controller-written value with no prediction.** The controller writes the member every tick, but its role stays `MemberRole.Replicate`, so the controller sees its own writes a round trip late, with no replay.
- **A purely local visual.** Particles, camera shake and UI feedback that only need to look right on one machine are not a `NetworkMember<T0>` at all, and cost nothing on the wire.

Choose prediction per member

`role` is set per member, and nothing is predicted by default. A component can predict its position while leaving health or ammo as `MemberRole.Replicate`.

The `ValidatedJump` demo under `Demos/Prediction` declares the walk direction and jump press as

`MemberRole.Predict` and every movement result as `MemberRole.Validate`. The validated behaviours, such as `ValidatedRigidbody` and `ValidatedTransform`, carry ready-made `Validate` members for a body, transform or character controller, and the `BallRacer` demo under `Demos/Tutorials` predicts a `Rigidbody` with one.

Predicted members and replays are in every build. Predicted spawning, predicted despawn and spawn compensation are Pro only.

Sending client state from Unity

> **Driving the core API directly?** See Client state.

Client state is what the controlling client's player asked for this tick, such as a move direction or a jump press. The client sends it upstream every tick, and the server simulates the object from it. This page sends a direction from a `NucleusBehaviour` and moves the object on the server.

Declare the client state in the component

Declare client state as `NetworkMember<T0>` fields with `role: MemberRole.Predict`, in the object's own `NetworkComponent` beside the members the server computes from them. There is nothing else to add or register.

```
using Nucleus.Components;
using Nucleus.Connections;
using NumericsVector2 = System.Numerics.Vector2;
using NumericsVector3 = System.Numerics.Vector3;

namespace MyGame
{
    public partial class MoverComponent : NetworkComponent, IMemberWriteValidator<Numerics
Vector2>, IMemberWriteValidator<NumericsVector3>
    {
        // Client state: written by the controller on its live tick, sent upstream every t
ick.
        public readonly NetworkMember<NumericsVector2> Direction = new(role: MemberRole.Pr
edict);
        // A member only the server computes from it.
        public readonly NetworkMember<NumericsVector3> Position = new();

        // Runs on the server for each tick of client state it takes. Write the member and
 decline to keep your own value instead.
        public MemberWriteAction ValidateMemberWrite(NetworkMember<NumericsVector2> networ
kMember, NumericsVector2 proposedValue, Connection writingClientConnection)
        {
            float length = proposedValue.Length();

            if (length <= 1f)
                return MemberWriteAction.Accept;

            Direction.Value = proposedValue / length;

            return MemberWriteAction.Decline;
        }

        // Only the server moves this object, so a client write to the position is decline
d.
        public MemberWriteAction ValidateMemberWrite(NetworkMember<NumericsVector3> networ
kMember, NumericsVector3 proposedValue, Connection writingClientConnection) => MemberWrite
Action.Decline;
    }
}
```

The validator runs on the server once per `Predict` member for every client tick it takes. Return `Accept` to take the value, `Decline` to keep what the member holds, or write the member and return `Decline` to keep

your own value, as the direction clamp does. A validator that throws declines.

Read the device in Update, write it on the tick

Read the keys in `Update` into a field, since a held key belongs to the rendered frame. Copy the field into the `Predict` member in `OnEarlyFixedUpdate`, only on the controller's live tick (`SimulationType.Forward`). A replay reads each past tick's value from the member's history and must never overwrite it.

```
using Nucleus.Connections;
using Nucleus.Integrations.Unity.Serializers;
using Nucleus.Integrations.Unity.Systems;
using Nucleus.Managers.NetworkLoop;
using Nucleus.Systems;
using UnityEngine;

namespace MyGame
{
    public class ServerMover : NucleusBehaviour<MoverComponent>
    {
        [SerializeField] private float _speed = 5f;

        private Vector2 _frameDirection;

        protected override void OnSystemLinked()
        {
            if (IsStarted(Invoker.Server))
                Component.Position.Value = transform.position.ToNative();
            else
                transform.position = Component.Position.Value.ToUnity();
        }

        protected override void OnSystemUnlinked() => _frameDirection = Vector2.zero;

        private void Update()
        {
            // Replace with your project's input system.
            if (IsController(ControllerType.Client))
                _frameDirection = Vector2.ClampMagnitude(new Vector2(Input.GetAxisRaw("Hor
izontal"), Input.GetAxisRaw("Vertical")), 1f);
        }

        protected override void OnEarlyFixedUpdate(StepDelta stepDelta, SimulationContext
simulationContext)
        {
            if (simulationContext.Type is SimulationType.None)
                return;

            // The controller's live tick: the keys become this tick's client state.
            if (simulationContext.Type is SimulationType.Forward && IsController(Controlle
rType.Client))
                Component.Direction.Value = _frameDirection.ToNative();

            if (!IsStarted(Invoker.Server))
                return;

            // A tick whose client state did not arrive: keep the last direction rather th
an stopping.
            if (stepDelta.IsClientStateMissing)
                Component.TryGetLastState(isEveryMemberCarried: true);

            Component.Position.Value = Step(Component.Position.Value.ToUnity(), Component.
Direction.Value.ToUnity()).ToNative();
        }

        protected override void OnVariableUpdate(StepDelta stepDelta) => transform.positio
```

```
n = Component.Position.InterpolatedValue.ToUnity();

        private Vector3 Step(Vector3 position, Vector2 direction)
        {
            float seconds = 1f / CoreManager.NetworkLoopManager.TickRate;

            return position + new Vector3(direction.x, 0f, direction.y) * (_speed * second
s);
        }
    }
}
```

- Do not put a `NetworkTransform` on the same object; this script replicates the position itself.
- Step by the fixed tick (`1f / CoreManager.NetworkLoopManager.TickRate`), never `Time.deltaTime`, so every peer integrates identically.

Handle client state that never arrived

The server steps the object on every server tick, one client tick at a time, through **State Interpolation**. On a host playing its own object nothing travels upstream, but each value still goes through the validator.

When a tick's client state never arrived, `stepDelta.IsClientStateMissing` is true and the `Predict` members read their defaults until `TryGetLastState(isEveryMemberCarried)` carries the previous tick's values.

- Declare a press as a bool with `isCarriedForward: false`, so `TryGetLastState(false)` leaves it at its default. A press lost with its packet is simply lost, and the server corrects the jump the client showed.
- `TryGetLastState(isEveryMemberCarried, out SimulationContext lastStateSimulationContext)` also reports which tick the carried state came from. On the server, `simulationContext.RemoteTick - lastStateSimulationContext.RemoteTick` is how many ticks it has been carried, for example to stop a character once the client has been silent too long.
- The controller's `Connection` counts late and dropped ticks as `ClientStateLateCount` and `ClientStateDroppedCount`.

The server's step runs on its own tick (`stepDelta.Tick`). Use `simulationContext.RemoteTick`, the client's tick being simulated, only for logging and pairing, never to time a cooldown: a client picks its own tick numbers.

React to client state

The server forwards the values it kept, after validation and its own writes, to every observer, the controller included. `OnMembersChanged` on the component reports them when they change:

- `MemberChangeDirection.Read`: on the server when it takes a client's changed value (`TryGetWritingClient` names the sender), and on an observer receiving a changed value. Drive presentation such as footsteps or animation from it.
- `MemberChangeDirection.Correction`: on the controller, when the server's value for one of its past ticks differs from its own, as when the validator clamped it. The server's value is already in the member's history for that tick.

```
public override void OnMembersChanged(ulong memberFlags, MemberChangeDirection memberChang
eDirection)
{
    if (memberChangeDirection is MemberChangeDirection.Correction && (memberFlags & (ulong
)MoverComponentFlags.Direction) != 0)
        UnityEngine.Debug.Log($"The server corrected the direction to {Direction.Value}.")
;
}
```

Put this override in `MoverComponent`. Use the callback for presentation or to react to a correction; the simulation reads the members every tick anyway.

Predict the result as well

Here only the server moves the object, so the controller sees its own movement a round trip late. To move it at once, declare `Position` with `role: MemberRole.Validate`, run the step on the controller too, and let the engine replay it when the server's result differs. See Replaying a correction in Unity.

Host mode

Driving the core API directly? See Host mode.

A host is one process that runs a server and its own client together. On a host, `TransportManager.IsServerStarted` and `IsClientStarted` are both true, and so is `IsHostStarted`.

Start a host

Set **Automatic Start Mode** on `UnityTransportManager` to `Host`, or call `StartHostAsync()`, which starts every transport as a server and then as a client:

```
await unityTransportManager.StartHostAsync();
```

Compare players by Connection.Id

The host's own client has one `Connection`, and the server's roster entry for that client is the same object. A call that names the host's player by its local `Connection`, such as `RpcTarget.To` or `RpcTarget.ObserversExcept`, therefore matches the roster entry.

Still compare players by `Connection.Id`, never by reference. A `Connection` is reused once its link ends, so a kept reference can end up naming a different player.

```
bool isSelf = someConnection.Id == localClientConnection.Id;
```

To get either side's own `Connection`, ask the `TransportManager`:

```
transportManager.TryGetLocalClientConnection(out Connection localClient);
transportManager.TryGetServerConnection(out Connection localServer);
```

`TryGetLocalClientConnection` returns false on a pure server. A controlling client uses `TryGetServerConnection` to find the upstream observer target.

Both ends of the host's own link report `Connection.IsHostLoopback` as true. It is a real, live link, so `IsEmulated` and `IsPeerStandIn` are false on it.

Leave HostLoopbackDelivery at Immediate

`TransportManager.HostLoopbackDelivery` controls when one half of a host sees what the other half addressed to it. `HostLoopbackDelivery.Immediate`, the default, runs the receiving handler at the send site, before the send call returns. It is the only value you can set: any other value logs an error and is ignored.

Peer events name live connections on a host

On a client, `TransportManager.PeerConnectionDiscovered` and `PeerConnectionDropped` hand over a stand-in `Connection` (`IsPeerStandIn` true). On a host they name the server's live roster entry for the peer, so `IsPeerStandIn` is false. Neither event is raised for the host's own client.

Never combine RpcTarget.To with immediate self delivery

Do not pair an addressed send (`RpcTarget.To(connection)`) with `RpcSelfDelivery.Immediate` on a surface the host's own client also listens on. The host runs the handler once for the send and once more for its own copy, so it runs every player's copy of a call meant for one. See Sending a remote call from a component.

Core, Managers and the Tick Loop

Pages in this category:

- The Nucleus Core Manager component
- Tick rate and the loop in Unity
- Host mode
- The network loop steps
- Starting and stopping a session in Unity
- Manager components reference
- The manager map
- Reacting to session events in Unity
- Play mode, domain reload and session lifetime

The manager map

Driving the core API directly? See The manager map.

Each concern has one manager

`CoreManager` holds eleven managers as public fields, such as `coreManager.TransportManager`. Every setting lives on the manager that owns its concern; there is no single settings object, so to change something, find its manager first.

Manager	Owns	Unity component
`NetworkLoopManager`	The tick loop: tick rate and the steps that drive a tick forward.	`UnityNetworkLoopManager`
`TransportManager`	The transports a session sends and receives over.	`UnityTransportManager`
`ServerManager`	Server-side session state.	`UnityServerManager`
`ClientManager`	Client-side session state.	`UnityClientManager`
`SystemManager`	Networked systems: spawning, ownership, and their lifecycle.	`UnitySystemManager`
`InterestManager`	What each peer is allowed to see.	`UnityInterestManager`
`SceneManager`	Scene load, stacking, and per-scene state.	`UnitySceneManager`
`PacketManager`	Packet framing and dispatch.	`UnityPacketManager`
`MessageManager`	Messages sent between peers.	`UnityMessageManager`
`RpcManager`	Remote procedure calls.	None
`ViolationManager`	Detecting and acting on protocol violations.	None

`BundleManager` (content delivered from outside the build, such as Addressables, a patcher or a CDN) and `WorldPersistenceManager` (writing a world to disk and building it back) exist only in Pro. In a Free build, `CoreManager` has no such fields at all.

`UnityPhysicsManager` is the one component that fronts no core manager. It holds physics execution settings; see Manager components reference.

Reach a manager from a script

- Through a component's `Nucleus*` property, for example `unityTransportManager.NucleusTransportManager`.
- Through `UnityCoreManager.CoreManager` or `NucleusUnity.BoundCoreManager`, then the field, for example `NucleusUnity.BoundCoreManager.RpcManager`. Use this for `RpcManager` and `ViolationManager`, which have no component.

The network loop steps

Driving the core API directly? See The network loop steps.

Every frame walks the loop's named steps in one fixed order. A `NucleusBehaviour` hooks a step by overriding its virtual, such as `OnLateStateUpdate`. Each step is one flag of the `NetworkLoopSteps` enum. To hook steps from a class that is not a `NucleusBehaviour`, implement `INetworkLoopStepCallback`, as the core API page shows.

The eleven steps run in this order

Step	Virtual	Runs	What to know
`EarlyVariableUpdate`	`OnEarlyVariableUpdate`	Every frame	Packets are received before your callback, and received messages reach their handlers after it.
`EarlyStateUpdate`	`OnEarlyStateUpdate`	Each tick	State received this tick is not applied yet, so a read can still see last tick's value.
`LateStateUpdate`	`OnLateStateUpdate`	Each tick	`PresentedTick` has caught up to `Tick`, and the state and RPCs received this tick are applied before your callback. Read replicated state here or later.
`Reconcile`	`OnReconcile`	Each tick	
`EarlyFixedUpdate`	`OnEarlyFixedUpdate`	Each elapsed tick interval	
`LateFixedUpdate`	`OnLateFixedUpdate`	Each elapsed tick interval	
`VariableUpdate`	`OnVariableUpdate`	Every frame	Put code that must run every frame here.
`EarlySerialize`	`OnEarlySerialize`	Each tick	Write the values you want sent this tick here.

`LateSerialize`	`OnLateSerialize`	Each tick	Changed state is written before your callback, so a member write here rides the next tick. Messages, RPCs and kicks, including any sent from this callback, leave after it.
`TickAdvance`	`OnTickAdvance`	Each tick	`Tick` increments after your callback, so `LateVariableUpdate`, the frames until the next tick and every message handler read the tick their writes will ride.
`LateVariableUpdate`	`OnLateVariableUpdate`	Every frame	

Do not expect a tick step every frame

- A frame that does not reach the next tick runs only the three variable steps.
- A frame runs at most one tick. The fixed steps run once for each tick interval that has passed, so after a long frame they can run twice on one frame.
- Anything moved every frame should move by `StepDelta.NormalizedDelta`, not the raw frame delta. See Tick rate and the loop in Unity.

Messages, RPCs and kicks are sent on every frame, not only on frames that tick: on a frame without a tick they leave at `LateSerialize`'s place in the order, with no callback raised. One queued from `OnTickAdvance` or `OnLateVariableUpdate` leaves on the next frame.

Typed messages reach their handlers on `EarlyVariableUpdate`, and system RPCs on `LateStateUpdate`. See When a message or call reaches its handler.

The Nucleus Core Manager component

Driving the core API directly? See Creating a CoreManager.

Every scene that uses Nucleus needs one `UnityCoreManager`. When the scene starts, it creates the session and adds any other Nucleus manager component the session needs.

Add it from the Nucleus window

Choose **Tools > Nucleus**, select the **Setup** tab, and click **Create** next to **A UnityCoreManager is in the scene**. You can also add the component to a GameObject yourself.

Change manager settings before you press Play

To change a manager's settings, add that manager component to the same GameObject, or to a child of it, and set its fields before you press Play. The tick rate on `UnityNetworkLoopManager` cannot change during play. The Unity manager components page lists every manager.

Reach the session from a script

The component's `CoreManager` property holds the session. A script anywhere in the scene can also reach it through `NucleusUnity.BoundCoreManager`. Every manager exists by the time your scripts' `Start` runs.

Destroying it ends the session

When the GameObject is destroyed, including when you leave Play mode, the session shuts down.

Manager components reference

Each manager component on the Nucleus GameObject sets up one core manager. For which manager owns what, see The manager map.

Set fields before you press Play

Each inspector field is copied into its core manager once, when the session starts. Editing a field during play changes the component but not the running session. To change a setting at runtime, set it on the component's `Nucleus*` property instead.

Each component exposes its core manager

Component	Core manager property
`UnityNetworkLoopManager`	`NucleusNetworkLoopManager` (`Nucleus.Managers.NetworkLoop.NetworkLoopManager`)
`UnityTransportManager`	`NucleusTransportManager` (`Nucleus.Managers.Transports.TransportManager`)
`UnityServerManager`	`NucleusServerManager` (`Nucleus.Managers.Server.ServerManager`)
`UnityClientManager`	`NucleusClientManager` (`Nucleus.Managers.Client.ClientManager`)
`UnitySystemManager`	`NucleusSystemManager` (`Nucleus.Managers.Systems.SystemManager`)
`UnityInterestManager`	`NucleusInterestManager` (`Nucleus.Managers.Interest.InterestManager`)
`UnitySceneManager`	`NucleusSceneManager` (`Nucleus.Managers.Scenes.SceneManager`)
`UnityPacketManager`	`NucleusPacketManager`
`UnityMessageManager`	`NucleusMessageManager`

`UnityClientManager`, `UnityPacketManager` and `UnityMessageManager` have no inspector fields. The settings on `UnitySystemManager` (redundancy, state interpolation, partial state apply, host migration, controller retention), `UnityInterestManager` (evaluation cadence, host interest, interest conditions) and `UnitySceneManager` (join placement, automatic scene requests, load timeout) are covered on their own pages.

Configure UnityServerManager

The **Client Limits** section is covered in Securing a server in Unity.

In Pro, the **Object Pacing** section limits how many kilobytes (1 kB is 1,024 bytes) of whole objects any one client is sent per tick: new objects, and objects sent again whole after packet loss. Ordinary updates to objects a client already holds are not limited. What does not fit is sent on the ticks after, so a join, a scene load or newly unlocked content arrives over several ticks instead of one.

Field	Default	Sets

Object Pacing Enabled	On	`ServerManager.ObjectPacingEnabled`. Off sends every whole object on the tick it is owed.
Maximum Object Kilobytes Per Tick	`50` (`ServerManager.DefaultMaximumObjectKilobytesPerTick`), range 1 to 255	`ServerManager.MaximumObjectKilobytesPerTick`. Hidden while pacing is off. Raise it for a fast LAN or a world of many small objects; lower it for slow links.

A Free build has neither field and sends every whole object on the tick it is owed. See Object pacing.

Configure UnityPhysicsManager

`UnityPhysicsManager` fronts no core manager. It holds `ExecutionMode` (a `PhysicsExecutionMode`) and a toggle that matches Unity's fixed delta time to the network tick rate. See Physics execution modes.

Write your own manager component

Derive from `UnityManager` and override `ManagersInstantiated(UnityCoreManager)`. It runs once, after the `CoreManager` and all its managers exist. Call `base.ManagersInstantiated(unityCoreManager)` first, then bind your core manager reference and push your inspector fields into it. The `UnityCoreManager` property holds the component that owns the session.

Play mode, domain reload and session lifetime

Driving the core API directly? See Shutting a session down.

Nucleus works with domain reload disabled

Nucleus resets its own static state, such as its `CoreManager` and its registered prefabs, at the start of every play session. It behaves the same with domain reload on or off (**Enter Play Mode Options**), and there is nothing to configure.

If your game keeps static fields of its own and you disable domain reload, reset them yourself, for example in a method marked `[RuntimeInitializeOnLoadMethod(RuntimeInitializeLoadType.SubsystemRegistration)]`.

End a session

A session ends when its `UnityCoreManager` is destroyed, including when play mode stops. To end one from code, call `CoreManager.Deinitialize()`. Calling it more than once is safe, and `CoreManager.IsDeinitialized` tells you whether it has run.

A deinitialized `CoreManager` cannot be started again. A new session always builds a new one.

Fix a session that leaked into the next one

A handler fires twice, or answers for an object that no longer exists. Something from the previous session is still subscribed. Check that its `UnityCoreManager` was really destroyed: a `DisallowMultipleComponent` conflict, a disabled GameObject or an abrupt stop can skip the teardown. With domain reload disabled, a static field in your own code that still holds a handler is another common cause.

`CoreManager.Instance` is not the manager you expect, or a manager keeps responding after play has stopped. The previous `CoreManager` was never torn down. Nucleus forgets it when the next session starts but cannot shut it down for you. Look in the previous session's log for an error thrown while its `UnityCoreManager` was starting or being destroyed.

Content that loaded in the first session does not load in the second. Registered prefabs are cleared at the start of every session. Check that your `NetworkPrefabCollection` is loaded in the new session.

Reacting to session events in Unity

Driving the core API directly? See Session events.

Four events report session changes

Event	Handler	Raised
`NucleusServerManager.ClientAuthenticated`	`Server.Manager.ClientAuthenticatedHandler(Connection connection)`	After a client has authenticated.
`NucleusServerManager.ClientDisconnecting`	`Server.Manager.ClientDisconnectingHandler(Connection connection)`	While the connection the departing client controlled are still readable through it.
`NucleusServerManager.ClientAuthenticationDenied`	`Server.Manager.AuthenticationDeniedHandler(Connection connection)`	For a client that timed out, before it disconnects.
`NucleusClientManager.LocalClientAuthenticated`	`Client.Manager.LocalClientAuthenticatedHandler(Connection localClientConnection)`	When the local client's own authentication completes.

Reach them through the `UnityServerManager` and `UnityClientManager` components on the Nucleus GameObject.

Subscribe and unsubscribe

Subscribe once the managers exist: in `Start` for an ordinary script, or in `ManagersInstantiated` for a `UnityManager` subclass. Never subscribe in `Awake`.

Unsubscribe every handler in `OnDestroy`. With domain reload disabled, a handler left subscribed survives into the next play session and fires against a `Connection` from a session that is gone. A `static` handler that never unsubscribes is the usual culprit.

```
public class SessionLogger : MonoBehaviour
{
    [SerializeField] private UnityServerManager _unityServerManager;

    private void Start()
    {
        _unityServerManager.NucleusServerManager.ClientAuthenticated += OnClientAuthentica
ted;
        _unityServerManager.NucleusServerManager.ClientDisconnecting += OnClientDisconnect
ing;
        _unityServerManager.NucleusServerManager.ClientAuthenticationDenied += OnClientAut
henticationDenied;
    }

    private void OnClientAuthenticated(Connection connection) { }
    private void OnClientDisconnecting(Connection connection) { }
    private void OnClientAuthenticationDenied(Connection connection) { }

    private void OnDestroy()
    {
        if (_unityServerManager == null)
            return;

        _unityServerManager.NucleusServerManager.ClientAuthenticated -= OnClientAuthentica
ted;
        _unityServerManager.NucleusServerManager.ClientDisconnecting -= OnClientDisconnect
ing;
        _unityServerManager.NucleusServerManager.ClientAuthenticationDenied -= OnClientAut
henticationDenied;
    }
}
```

A script with no reference to the components can reach the session through `NucleusUnity.BoundCoreManager`:

```
CoreManager coreManager = NucleusUnity.BoundCoreManager;
if (coreManager != null)
    coreManager.ServerManager.ClientAuthenticated += OnClientAuthenticated;
```

During shutdown, check `NucleusUnity.IsApplicationQuitting` before touching anything session-related. It turns true once `Application.quitting` fires and resets each session, so a teardown path can tell whether the transport is already coming down.

Scripts on a networked object use their own hooks

A `NucleusBehaviour` on a networked object does not need these events. It has its own hooks, `OnServerStarted`, `OnClientStarted` and `OnControllerChanged`; see NucleusBehaviour.

Starting and stopping a session in Unity

Driving the core API directly? See Server, client and host roles.

Start on Play with no code

Set **Automatic Start Mode** on `UnityTransportManager`, shown as **Start On Play** in the **Setup** tab of the Nucleus window (**Tools > Nucleus**). It is a `NetworkStartMode`:

- `None` (the default): nothing starts until a script starts it.
- `Server`: starts every transport as a server.
- `Client`: starts every transport as a client, connecting to its configured remote host.
- `Host`: starts every transport as both a server and a client in one process.
- `Auto`: starts as a client in a ParrelSync clone or when another process already holds the server port, and as a host otherwise.

To get two editors talking, set one to `Server` or `Host` and the other to `Client`, or set both to `Auto`, and press Play in each.

Start from code

Call the async methods on `UnityTransportManager` from `Start` or later, for example from a menu button:

```
public UnityTransportManager TransportManager;

public async void OnHostButtonPressed()
{
    await TransportManager.StartHostAsync();
}

public void OnStopButtonPressed()
{
    TransportManager.ShutdownAll();
}
```

`StartServerAsync()`, `StartClientAsync()` and `StartHostAsync()` each add the transports first. To add them without starting anything, call `EnsureAddedAsync()`; a second call does nothing.

The transports are the `NetworkTransport` components on the `UnityTransportManager`'s GameObject and its children, added in the order they are found. With none present, a `SynapseTransport` component is added. Configure each transport on its own component.

Stop a session

`ShutdownAll()` removes and shuts down every transport the manager added. The `CoreManager` stays until its `UnityCoreManager` is destroyed.

Unloading the scene shuts the transports down for you, so you do not need to call `ShutdownAll()` before a scene change.

Check what is running

`NucleusTransportManager` has three booleans:

- `IsServerStarted`: a server connection is established on any added transport.

- `IsClientStarted`: a client connection is established **and authenticated**. It stays false after a socket-level connect until the local client finishes authenticating.
- `IsHostStarted`: both are true.

```
if (unityTransportManager.NucleusTransportManager.IsClientStarted)
{
    // The local client is connected and authenticated.
}
```

Tick rate and the loop in Unity

Driving the core API directly? See Ticks, tick rate and time.

Set the tick rate before you press Play

Set **Tick Rate** on `UnityNetworkLoopManager`, or in the **Session** section of the **Setup** tab of the Nucleus window (**Tools > Nucleus**).

- Default: 30 (`NetworkLoopManager.DefaultTickRate`), which is also used when the scene has no `UnityNetworkLoopManager`.
- Range: 5 (`NetworkLoopManager.MinimumTickRate`) to 128 (`NetworkLoopManager.MaximumTickRate`).

The rate is fixed once the session starts, and the slider is disabled during play. To change it, stop play mode, change the value and start again.

Loop callbacks run on the main thread

Every network loop callback in Unity runs on Unity's main thread, so you can read and write transforms, components and GameObjects directly from one.

Time.timeScale does not affect networking

The loop runs on unscaled time. With `Time.timeScale = 0` the game keeps receiving, ticking and sending, so a paused client stays connected, and slow motion does not drift the tick away from other peers. If a pause should freeze networked gameplay, stop your own simulation in your callbacks, for example by skipping your movement code while paused.

Read the clock from a script

`UnityNetworkLoopManager.NucleusNetworkLoopManager` gives:

- `Tick`: the current local tick.
- `TickRate`: ticks per second, fixed for the session.
- `SubtickPercentage`: how far the loop is into the current tick, from 0 to 1. It updates every frame and is what render smoothing between ticks uses.

Move objects every frame by the normalized delta

If your authority moves an object every frame, move it in `OnVariableUpdate` by `stepDelta.NormalizedDelta`, not in `Update` by `Time.deltaTime`:

```
protected override void OnVariableUpdate(StepDelta stepDelta)
{
    if (!IsController(ControllerType.AnyController))
        return;

    float seconds = (float)(stepDelta.NormalizedDelta / 1000.0);
    transform.position += _velocity * seconds;
}
```

Frames never line up with ticks, so an object moved by `Time.deltaTime` covers a different distance each tick, and that unevenness costs bandwidth. `NormalizedDelta` adjusts each frame's delta so the frames between two ticks add up to exactly one tick interval. It is in milliseconds, like the other `StepDelta` fields.

See also

- The network loop steps: what runs at each step, and in what order.
- Driving the loop yourself: writing your own `INetworkLoopStepProvider` instead of the one Unity supplies.

Common errors and what causes them

Search this page for the text of a log line to find what produced it and what to change. Every line below is quoted verbatim from the engine.

A message was written off the loop thread

`"[Caller<MessageType>] ran on thread [X] while a network loop step is executing on thread [Y]. This work must not run concurrently with the loop; it races the outbound WriterPool and can corrupt sent packets."`

Cause. A message was written from a thread other than the one running the network loop.

Fix. Move the work onto the loop. If it resumes after an `await`, route it through the loop rather than sending from wherever the continuation landed.

This check exists only in `DEBUG` builds. A Release build still has the race but never logs it, so diagnose in Debug and confirm the fix in Release.

Loop-owned state was touched off the loop thread

`"[Caller] touched the loop-owned collection [Name] on thread [X] while a network loop step is executing on thread [Y]. This work must run on the network loop thread; touching the collection off the loop races the loop and can silently corrupt it."`

Cause. An async continuation resumed off the loop thread and touched state the loop owns.

Fix. Switch back onto the loop before touching loop-owned state.

This check is also `DEBUG`-only. In Release the race still happens, silently.

Two threads are driving one manager

`"Step [X] was turned away on thread [...] because this manager is already executing a step on thread [...]. One loop must drive one manager, and a second driver races every collection the loop owns. A manager whose owner never supplied a step provider, leaving the default one driving the loop from the thread pool, is the usual cause. At least [N] steps have been turned away since the previous report."`

Cause. Two threads are stepping the same manager's loop, usually because the owner never supplied its own `INetworkLoopStepProvider`, so the default provider steps the loop from the thread pool alongside whatever else steps it.

Fix. Make sure exactly one driver steps the manager by supplying the owner's own step provider (a Unity integration's loop provider, or a test harness driving ticks by hand).

The line is logged at most once per minute per manager. The count in `At least [N] steps have been turned away` includes every step turned away since the last report, so it tells you how severe the problem is.

A violation was logged

`"Violation [TypeName] from Connection [...]"`, sometimes followed by `"...settled on a kick, but this peer is not a server; the kick is not enforced."` or `"Disconnecting Connection [...] for violation [TypeName]."`

A violation writes one of these lines only when its `ViolationContext<T0>.Log` is set or it kicks. Most types default to `Ignore` and write nothing, so register an `IViolationObserver` to see them. Look up

`TypeName` in the Violation reference for its payload, its default action and what to check first.

A recovery named a system this peer no longer has

`"A Recovery entry from Connection [...] references unknown NetworkSystem Id [X]; that entry is skipped."`

The sender believed this peer still held a system it no longer has. The entry is skipped and the rest of the recovery continues.

`InvalidStateAckViolation`, `RetentionExceededViolation` and `RecoveryUnconfirmedViolation` default to `Ignore`, because a peer on a bad link raises them now and then. Raises that keep recurring for the same Connection mean its latency exceeds the retention window, or it is not acknowledging correctly.

A peer's state could not be read

`"A StatePacket subpacket from Connection [...] declares [X] body bits but only [Y] remain; discarding the rest of the tick."`

`"State subpacket [Type] from Connection [...] left the reader at [...] bits but its framed boundary was [...] (declared [X] body bits, off by [Y]). The stream is misaligned; discarding the rest of the tick."`

The receiver could not read a peer's state the way it was written, so the rest of the tick is discarded. A reader and writer disagree about a type's shape: look at the type's serializer, not at game logic.

A peer sent an unknown packet type

`"Connection [...] sent unrecognised PacketType [X]; the packet is discarded."`

The peer is running an older or newer build that sends a packet type this build does not know. The packet is discarded and the peer is not kicked.

Developer tools in the editor

Tool	Where it lives	What it answers
`NetworkCorrectionVisualizer`	Component, added under a networked object	Where and how far did prediction correct this object?
`NetworkInterestOverlay`	Component, added under a networked object	What is this object's controller being sent?
Traffic tab	**Tools > Nucleus**, then the **Traffic** tab	What is this session sending and receiving, and how healthy are its connections right now?
Validate Network Prefabs	**Tools > Nucleus**, then the **Build** tab	Will my networked prefabs spawn correctly?

None of them change the session, apart from the Traffic tab's control buttons and simulated conditions. Remove or switch off the two components before you ship.

See where prediction corrected an object

Add `NetworkCorrectionVisualizer` to a predicted object, on its `NetworkSystemObject` or a child of it. Without a `NetworkSystemObject` at or above it, it logs an error and disables itself. It watches every system linked to that object, including systems that link later.

For every correction larger than **Threshold**, it flashes a cross where the member was shown before the correction, a cross where the replay put it, and a line between them. The flash shows in the Scene view, and in the Game view while its Gizmos toggle is on. It only ever draws on a client that predicts the object.

Field	Default	What it does
Threshold	`0.05`	The largest correction ignored, in the member's own units (metres for a position). Set it to your prediction tolerance to see exactly the corrections the engine treats as real.
Flash Interval	`0`	The shortest time between two flashes, in seconds of unscaled time. Zero flashes on every correction past the threshold. Raise it when the lines turn into a smear.
Flash Duration	`0.5`	How long each flash stays drawn, in seconds.
Color	magenta	The color of the line and both crosses.
Marker Size	`0.15`	The half-size of the cross drawn at each end, in metres.

Log Enabled	off	Also log each flashed correction, naming the member, its size and the tick.

In play mode the inspector shows **Flashes**, the number of corrections drawn or logged so far, and **Last Correction (m)**, the size of the most recent position correction. Scripts and tests can read them as `FlashCount` and `LastCorrectionDistance`.

Only a `NetworkMember<System.Numerics.Vector3>` is drawn. A position replicated in a parent's local space is drawn where it would sit unparented. Corrections to other member types still count toward **Flashes** and are logged with **Log Enabled**, but nothing is drawn.

Read the lines

- **Short lines on every tick** mean the client predicts slightly differently from the server. Look for a value read from `Time.deltaTime` instead of the network step, a random number, or a physics setting that differs between builds.
- **Long lines now and then** usually match packet loss or a collision the client did not predict. Turn on **Log Enabled** to see the tick of each one.
- **No lines at all** while the object visibly snaps means the snap is not a prediction correction. Check smoothing and interpolation instead (see Proxies and smoothing).

Seeing what an object's controller is sent

Add `NetworkInterestOverlay` to a networked object, on its `NetworkSystemObject` or a child of it.

Switch on **Overlay Enabled**. It is off by default.

Enter play mode as the server or host and select the object.

For each client controlling a system on the object, it draws a line from this object to every other networked object, colored by what that client holds of it:

Color	Meaning
Green	Streamed: the client has the object and is sent its updates.
Yellow	Stopped: the client still has the object, but is no longer sent updates for it.
Faint red	Unspawned: the client does not have the object at all. Drawn only with **Unspawned Draw Enabled**.

An object with several systems takes the color of the most the client holds of any of them. When different clients control different systems on the selected object, each client's lines are raised by **Controller Spacing** so they stay apart. **Marker Radius** sets the size of the wire sphere drawn on each object a line reaches.

It draws nothing on a client, and nothing for an object no client controls.

To make the same check from code on the server, call `NetworkSystem.GetInterestMembership(connection)`, which returns the `InterestMembership` the system resolves toward that connection. To react when it changes, subscribe to the system's `InterestChanged` event instead of polling. Interest troubleshooting covers what to do next.

The Traffic tab

Open **Tools > Nucleus** and select the **Traffic** tab. Outside play mode it offers an **Enter Play Mode** button and the simulated network conditions. While a session runs, the window's banner shows this peer's role (Host, Server, Client or Stopped), and the tab shows these sections, refreshed twice a second:

- **Controls**: **Start Server** or **Stop Server**, **Start Client** or **Stop Client**, and **Start Host** while neither side runs. A client stopped here does not reconnect on its own.
- **Session**: average frame time, the tick rate in ticks per second and milliseconds per tick, reconciles, and tick drift (how far the measured tick rate strays from the configured one). Frame time turns amber when a frame lasts longer than a tick, and tick drift turns amber past five percent of the tick rate. A server under its rate is falling behind; a client away from it is having its clock adjusted to keep its input arriving on time.
- **Bandwidth**: bytes sent and received per second, total bytes sent and received, a graph of both rates over the last minute, and how much of each direction is replicated state.
- **World**: connections, started systems, networked objects in the open scenes, and spawns that interest management admitted and deferred, followed by the session's redundancy, state interpolation, state retention and partial apply settings.
- **Network conditions**: on a server or host, one row per client with the round trip latency, jitter and packet loss that client reported; on a client, the same figures for its link to the server. A dash means not measured yet. Under them, **Simulated conditions** has a **Simulate** toggle that delays, drops and reorders packets on every transport, presets from **Perfect** through **LAN**, **Broadband**, **Wi-Fi** and **Mobile** to **Awful**, and the **Latency (ms)**, **Jitter (ms)**, **Packet Loss** and **Out Of Order** values. These are settings of the scene's `UnityTransportManager`, so they save with the scene, and a change in play mode applies straight away.

A server or host shows **reconciles sent**, the corrections it sent its clients. A client shows **reconciles received**, counting only objects still in the scene. To see both, open the window in both editors (see Testing with two editors).

Validating your networked prefabs

Before a build, or when a spawn goes wrong in a way you cannot explain:

Open **Tools > Nucleus** and select the **Build** tab.

Click **Validate** next to **Validate Network Prefabs**.

Click each error in the Console to ping the prefab it names. A dialog at the end counts the problems.

The validator runs only when you click **Validate**, never on save or on entering play mode, and changes nothing in your project. It reports two kinds of problem.

Identity problems are the ones a player build refuses (see Prefab identity and the prefab collection): no base `NetworkPrefabCollection`, two collections resolving to the same prefab bundle, a networked prefab missing from its collection, a prefab with no prefab id, two prefabs sharing one id, or a prefab stamped with a different bundle than its collection. The build runs the same check. To fix any of them, click **Rebuild** next to **Rebuild Network Prefab Collection** on the same tab.

Structure problems are hierarchies a spawn would mishandle:

- A `NetworkSystemObject` on a child of the prefab. Only the root's marker is spawned, so remove the child's marker.
- A `NucleusBehaviour` or `NetworkTransform` with no `NetworkSystemObject` at or above it. It never replicates. Add a marker to the prefab's root.

- A `NetworkHitboxRoot` that would record nothing. The validator reports the errors its inspector shows; warnings and notes stay in the inspector.

Diagnosing desync

Work the layers in order

"It looks wrong on the other peer" can be four different bugs. Check them in this order, because each one rules out the ones below it:

Is the object there at all? If the receiver never spawned it, or it vanished, this is interest: an `IInterestCondition` culled the pair (`InterestEffect.Spawn`, resolving to `InterestMembership.Unspawned`), or the object really was despawned.

Is it there but frozen? This is `InterestEffect.Stop`: the pair resolved to `InterestMembership.Stopped`, so the receiver keeps the object but is sent no more updates. It looks like a dead connection, but it is not.

Is it there and moving, but wrong? A member holds a value that does not match the sender. See the next section.

Is it correct, but late? This is interpolation and the retention window, not a data bug.

Do not skip ahead. A value that looks wrong because the object was culled five seconds ago is not a serializer bug.

Tell a missing value from a wrong one

Split layer 3 again before you touch a serializer:

- **A value that never changes on the receiver** is an observation or access problem. Is the Connection an observer of that `NetworkSystem`? Did an interest condition stop it? Does the sender own or control the member it is changing? Usually the write never left the sender, or never reached this receiver.
- **A value that changes, but to the wrong thing**, is a serializer or prediction problem. Look at how the member is written (`TransmissionMode.Divine` or `TransmissionMode.Interval`) or, for a controlled member, at prediction and reconciliation on the controlling side.

Read the violation stream first

Most protocol faults raise a violation before you would notice anything on screen. Register an `IViolationObserver` to hear every type, or subscribe a per-type event such as `UnauthorizedMemberChangeViolationDetected` (see Handling violations in Unity).

The types that matter for desync are `UnauthorizedMemberChangeViolation`, `PredictedMemberChangeViolation`, `EmptyCollectionDeltaViolation`, `InvalidStateAckViolation`, `RetentionExceededViolation`, `RecoveryUnconfirmedViolation` and `UnauthorizedSpawnViolation`. The Violation reference says what each means and what to check first.

A matching raise names the subsystem directly. If nothing is raised, the cause is something violations do not police: an interest decision, ordinary loss inside the retention window, interpolation lag, or a prediction mismatch carried in a well-formed packet.

Reproduce with two processes, not only a host

Some bugs only show on a client in a separate process, never on a host's own client. Test with two separate processes (two clients, or a client against a dedicated server) as well as with a host.

Some faults are timing, not data

Some symptoms come from which network loop step a callback runs on, not from the data. A callback only sees what was applied before its step ran. Signs of a step-ordering problem:

- A value read in one callback is stale compared to the same value read in another callback on the same tick.
- A change does not take effect until the following tick, or takes effect twice.
- The symptom varies with frame rate, or with several fixed updates landing in one variable update.

Fix these by moving the callback to the right step, not by changing serialization.

Narrow it to a reproduction

Once you know the layer and which of the two failure modes it is, shrink the repro before reading a single packet:

Cut to one object and one member.

Set the member's `TransmissionMode` to `Interval`. If the bug disappears, it is in the `Divine` path.

Add packet loss deliberately (a `NetworkConditions` with a `PacketLossChance`, passed to `TransportManager.SetNetworkConditions`) to see whether the symptom depends on loss.

Only then read a packet, knowing which member, which tick range, and whether the value never arrived or arrived wrong.

Frequently asked questions

Why did an object stop moving instead of disappearing

Interest stopped it rather than unspawning it. A stopped object stays on the receiver but is sent no more updates, so it looks frozen. An unspawned object is removed. See Frozen or gone: what a stop does to the object.

Why is nothing replicating at all

Check that the Nucleus source generator runs as an analyzer, not just as a referenced assembly. Without it your networked types are never set up to replicate. See Code Generation in Unity.

Why can I see my object but not drive it

Observing and controlling are separate. Interest decides whether a connection is sent an object's state; the server separately assigns who controls it. Observing an object never makes you its controller. See What control means.

Why does it work in host mode and break with two processes

Some bugs only show on a client in a separate process, never on a host's own client, so test with a second process. See Diagnosing desync.

Why is my bandwidth number unchanged after I turned a feature off

The transport's byte and bit counters are running totals since the connection started, with no per-object or per-feature breakdown. Sample them once a second and compare the rates before and after the change. See Reading network statistics in Unity.

Why does this type not exist in my build

Pro-only types are left out of a Free build entirely, so referencing one is a compile error, not a runtime failure. Either your code uses Pro-only API in a Free project, or the project has the wrong edition's files.

Filing a bug report

Reproduce it before you file

Reproduce it with the server and client as separate processes. A host runs both in one process, which can hide or create behavior, so a repro that only fails in host mode is usually not the bug it looks like.

Cut the repro down to one networked object, one member on it, and the smallest scene (Unity) or the smallest `Main` (plain .NET host) that still fails.

Run it again with logging set to `LoggerLevel.Information` and capture the output.

Include these in the report

- The edition: Free or Pro.
- The DLL set you deployed, and confirmation that every DLL came from the same build. A mismatched set, such as an old engine DLL next to a new source-generator DLL, causes failures that look like engine bugs.
- The Unity version, or for a plain host, the target framework you build against (the engine targets `netstandard2.1`).
- Any violations raised, by name, with their context.
- The full log lines, not paraphrased or truncated.
- Counter readings before and after the failure, if it involves a count (packets sent or received, retries, spawned objects).
- The tick or sequence number where the failure happens, if you can identify it.

Handling violations in Unity

> **Driving the core API directly?** See Violations

There is no violation component and nothing to set in an inspector. Register handlers in C# on the `ViolationManager`, reached through `NucleusUnity.BoundCoreManager.ViolationManager`, with `RegisterViolationHandler<T0>`, `RegisterViolationDetectedHandler<T0>` or `RegisterViolationObserver`.

Registering at the right time

Register from `Start`, or from `ManagersInstantiated` on a `UnityManager`, not from `Awake`. The managers may not exist yet when your `Awake` runs.

```
private CoreManager _coreManager;
private ViolationManager.ViolationDetectedHandler<UnauthorizedMemberChangeViolation> _hand
ler;

private void Start()
{
    _coreManager = NucleusUnity.BoundCoreManager;
    _handler = OnUnauthorizedMemberChange;
    _coreManager.ViolationManager.UnauthorizedMemberChangeViolationDetected += _handler;
}

private void OnUnauthorizedMemberChange(in ViolationContext<UnauthorizedMemberChangeViolat
ion> violationContext)
{
    // inspect violationContext.Connection, violationContext.Violation, violationContext.A
ction
}
```

Unregistering in OnDestroy

Unregister every handler, observer and subscriber in `OnDestroy`:

```
private void OnDestroy()
{
    if (_coreManager is not null)
        _coreManager.ViolationManager.UnauthorizedMemberChangeViolationDetected -= _handle
r;
}
```

A registration is not tied to the scene or the GameObject that made it. It lives as long as the `ViolationManager`, which outlives a scene load and, with domain reload disabled, can outlive a play session. A handler you never unregister keeps a destroyed `MonoBehaviour` alive and is still called on the next raise. Unregister on objects a pool recycles too.

If a handler throws (usually a `MissingReferenceException` from a destroyed target), the exception is logged as an error and the violation falls back to its default action. A throwing observer does not stop the ones after it, and a throwing subscriber does not stop enforcement.

Only a server enforces a kick

A `ViolationAction.Kick` is carried out only on a peer that has a server started. On a client-only peer the kick is logged as not enforced and the connection is left alone. Put deciding handlers that return `Kick` on

the build that runs the server. Handlers that only observe or log can go anywhere.

See violations in the console

With no handler registered, most violations are `ViolationAction.Ignore` and write nothing. The scene and bundle protocol faults default to `Kick`. `PacketTransformRejectedViolation` (Pro) takes `ViolationManager.DefaultAction` (`Ignore`) and always logs a warning.

Otherwise, a line is written only when `violationContext.Log` is set. The raise site may set it, and the deciding handler may change or clear it. To see the silent ones, register an `IViolationObserver` or a per-type subscriber and log them yourself. A logged violation looks like this:

```
Violation [<ViolationTypeName>] from Connection [<connection>]: <message>
```

A `Kick` logs a similar line before disconnecting, and a `Kick` on a non-server peer logs that it was not enforced.

Find the GameObject behind a violation

Many violation payloads carry a `SystemId` (for example `UnauthorizedMemberChangeViolation.SystemId`) rather than a GameObject. Resolve the system, then its `NetworkSystemObject`:

```
if (_coreManager.SystemManager.TryGetSystemReference(violationContext.Violation.SystemId,
out NetworkSystem networkSystem)
    && networkSystem is not null
    && NetworkSystemObject.TryGetLinked(networkSystem, out NetworkSystemObject networkSyst
emObject))
{
    GameObject offendingGameObject = networkSystemObject.gameObject;
}
```

Diagnostics and Troubleshooting

Pages in this category:

- Violation reference
- What the server trusts
- Securing a server in Unity
- Common errors and what causes them
- Technical limitations
- Engine logging in the Unity console
- Handling violations in Unity
- Reading network statistics in Unity
- Developer tools in the editor
- Diagnosing desync
- Frequently asked questions
- Filing a bug report

Engine logging in the Unity console

Driving the core API directly? See Logging.

Every engine log line already shows up in the Unity console, with nothing to set up. Nucleus routes information, warnings and errors to `Debug.Log`, `Debug.LogWarning` and `Debug.LogError`, so each line clicks through to its call site and carries Unity's own stack trace as your Stack Trace Logging settings allow (**Project Settings > Player > Other Settings**). Each line starts with `[Type::Method]`, naming the type and method that logged it.

Set the log level

`LoggerSetting` field	Applies to	Default
`Editor`	The editor	Information
`DevelopmentBuilds`	Development players	Information
`ReleaseBuilds`	Release players	Error

To change the levels, register your own logger (below) whose `GetLoggerSetting` returns the levels you want.

Replace the logger

To send engine logs somewhere else, such as a file or a remote service, register your own `ILogger`:

```
LoggingService.UseLogger(new MyProjectLogger());
```

Nucleus registers its Unity logger before the first scene loads, so a call from any scene script replaces it. Your logger implements `GetLoggerSetting()`, `DisableUnconditionalDevelopmentStacktrace()` and the three log methods. It loses the Unity logger's click-through to the call site unless it provides its own.

Reading network statistics in Unity

Driving the core API directly? See Network statistics

The counters live on `TransportManager`, reached through `CoreManager.TransportManager`. Get the `CoreManager` from `NucleusUnity.BoundCoreManager` in any script, from the `CoreManager` property of `UnityCoreManager`, or from the `CoreManager` property a `NucleusBehaviour` already has.

Read a rate once a second

`TotalBytesSent` and `TotalBytesReceived` are running totals, so sample them once a second and subtract the previous sample:

```
using Nucleus.Connections;
using Nucleus.Integrations.Unity;
using Nucleus.Managers.Core;
using Nucleus.Managers.NetworkLoop;
using Nucleus.Managers.Transports;
using UnityEngine;

public class NetworkStatsReadout : MonoBehaviour, INetworkLoopStepCallback
{
    private CoreManager _coreManager;
    private long _lastBytesSent;
    private long _lastBytesReceived;
    private long _elapsedMilliseconds;

    private void OnEnable()
    {
        _coreManager = NucleusUnity.BoundCoreManager;
        _coreManager?.NetworkLoopManager.RegisterNetworkLoopStepCallbacks(this);
    }

    private void OnDisable() => _coreManager?.NetworkLoopManager.UnregisterNetworkLoopStep
Callbacks(this);

    public NetworkLoopSteps GetNetworkLoopSteps() => NetworkLoopSteps.LateVariableUpdate;

    public void OnNetworkLoopStep(NetworkLoopSteps networkLoopStep, StepDelta stepDelta)
    {
        _elapsedMilliseconds += stepDelta.Delta;

        if (_elapsedMilliseconds < 1000)
            return;

        _elapsedMilliseconds = 0;

        TransportManager transportManager = _coreManager.TransportManager;

        long bytesSent = transportManager.TotalBytesSent;
        long bytesReceived = transportManager.TotalBytesReceived;

        long sentPerSecond = bytesSent - _lastBytesSent;
        long receivedPerSecond = bytesReceived - _lastBytesReceived;

        _lastBytesSent = bytesSent;
        _lastBytesReceived = bytesReceived;

        Debug.Log($"Up: {sentPerSecond} B/s, Down: {receivedPerSecond} B/s");

        if (transportManager.IsServerStarted)
        {
            // A server holds one live link per client.
            foreach (Connection connection in transportManager.ActiveConnections)
                Debug.Log($"Client {connection.Id} RTT: {connection.RoundTripTimeMilliseco
nds} ms, Loss: {connection.PacketLossPercentage:F1}%");
        }
        else if (transportManager.IsClientStarted && transportManager.TryGetServerConnecti
on(out Connection serverConnection))
        {
```

```
            // A client's link figures live on the connection that stands for the server.
            Debug.Log($"Server RTT: {serverConnection.RoundTripTimeMilliseconds} ms, Loss:
 {serverConnection.PacketLossPercentage:F1}%");
        }
    }
}
```

- The readout registers itself with the network loop as a plain `MonoBehaviour`. A `NucleusBehaviour` only receives loop steps while a networked system is linked to it, and a readout for the whole process has none.
- `PacketLossPercentage` already runs from 0 to 100, so print it with `F1`, not `P1`.
- A client reads its own link from `TryGetServerConnection`. On a client, `ActiveConnections` holds stand-ins for the other players, which carry no link figures.
- `TransportManager.GetLocalConnections(Invoker.Server)` (or `Invoker.Client`) returns this peer's own connection for that role on each transport. On a client, the `Invoker.Server` entry is the same connection `TryGetServerConnection` returns.

Sample on a loop step, not Update

Read the counters from a network loop step callback (`LateVariableUpdate` above, or any other `NetworkLoopSteps` value). `Update` has no fixed order relative to the loop's own work and can read a counter partway through an update.

The demo HUD is sample code

The demos include a full diagnostics HUD, `NetworkDiagnosticsHud`, with providers for role, round-trip time, bandwidth, state-packet bandwidth and per-object bandwidth. It is part of the demos, not the integration your project links against, so if you copy it into your project you maintain the copy.

There is no per-object bandwidth counter. The demo's `NetworkObjectBandwidthStatisticProvider` estimates one by dividing bandwidth by `SystemManager.StartedSystemCount`, and you can do the same in your own readout.

Securing a server in Unity

> **Driving the core API directly?** See Securing a server

Securing a server lists what the server already rejects on its own, and applies to Unity unchanged. This page covers the settings you reach from an inspector or a Unity script. See also What the server trusts and Handling violations in Unity.

Only a check on the server counts

A modified client can skip any check it holds, so every check here must run on the server, for example behind `IsStarted(Invoker.Server)` on a `NucleusBehaviour`. A client may run the same code only to predict an answer or grey out a button.

Limit how much each client can send

Set limits on the transport wherever it supports them, and use the Unity Server Manager's limits for transports that do not. The transport drops traffic before the engine reads anything, which is cheaper and also protects the transport's own receive buffers.

Set the client limits on the Synapse Transport component

The **Client Limits** section of the `SynapseTransport` inspector is **enabled by default**. The limits apply only while the peer runs a server, and a change takes effect at the next server start.

Inspector field	Default	What it guards against
Client Limits Enabled	on	Turns every limit below on or off. The socket's other checks, such as refusing a datagram wider than the transmission unit, stay on either way.
Maximum Packets Per Second	`500`	A client sending too many datagrams. The excess is dropped rather than kicked, because the source address of a datagram can be forged.
Maximum Bytes Per Second	`2097152` (2 MiB)	A client sending too many bytes, enforced the same way as the packet rate.
Maximum Out Of Order Reliable Packets	`64`	A client pinning server memory by never filling a gap in its reliable stream. One past the limit kicks and blacklists the client.
Maximum Reassembled Packet Size	`0` (no limit)	A client declaring a huge split packet so the server sets memory aside to rebuild it. A client over it is kicked and blacklisted.

Maximum Segments	`0` (the protocol's limit of 255)	A packet split into too many pieces. A packet over it is dropped. It also caps what the server sends.
Maximum Concurrent Assemblies Per Connection	`16`	A client opening many split packets at once. One more evicts or drops a piece.

Zero turns a single limit off. The fields show only while Client Limits Enabled is ticked.

The packet rate is the tighter of the two rates by default. If an honest client loses input under load, for example because it controls many objects or the server runs a high tick rate, raise Maximum Packets Per Second first. Dropped datagrams show up as lost client state, not as a kick.

In the **Hosting** section, **Maximum Connections** defaults to `0`, which accepts any number of clients. Set it to what your server can carry.

To see what the socket refused, subscribe to `ViolationObserved` on the `Synapse` transport the component built. It reports rate limits, oversized packets, security kicks and timeouts. See Synapse and SynapseTransport component.

Turn on the Unity Server Manager's limits for transports without their own

The **Client Limits** section of the `UnityServerManager` inspector is **disabled by default**. These limits are checked on every packet a client sends, whatever the transport. Turn them on for a transport with no limits of its own, such as `BlitzRelayTransport`, or as a second line behind Synapse's.

Inspector field	Default	What it guards against
Client Limits Enabled	off	Turns the engine limits on. While it is off, nothing is counted.
Maximum Packet Bytes	`0` (no limit)	A client sending oversized packets. Keep it at least one transmission unit above the largest packet an honest client sends.
Maximum Packets Per Second	`500`	A client sending too many packets in one second.
Maximum Bytes Per Second	`2097152` (2 MiB)	A client sending too many bytes in one second.

A packet past a limit is discarded unread and raises `ClientLimitViolation`, which **kicks by default**. A host's own client is exempt. The fields show only while Client Limits Enabled is ticked.

To change the limits from a script, set `UnityServerManager.ClientLimits` at any time. Once the managers are built, the change applies straight away.

Set the rest of the server's defences from a script

These settings have no inspector field. Set them through `NucleusUnity.BoundCoreManager` from `Start`, not `Awake` (see Handling violations in Unity):

```
public class ServerSecurity : MonoBehaviour
{
    private void Start()
    {
        CoreManager coreManager = NucleusUnity.BoundCoreManager;

        coreManager.ServerManager.SetAuthenticator(new TokenClientAuthenticator(coreManage
r.MessageManager));
        coreManager.ServerManager.AuthenticationTimeoutSeconds = 15;
        coreManager.RpcManager.MaximumInboundRpcsPerConnectionPerDrain = 32;
        coreManager.ViolationManager.RegisterViolationHandler<RpcFloodViolation>(OnRpcFloo
d);
    }

    private ViolationAction OnRpcFlood(ref ViolationContext<RpcFloodViolation> context) =>
 ViolationAction.Kick;
}
```

`TokenClientAuthenticator` stands for your own type. Unregister handlers in `OnDestroy`, as Handling violations in Unity explains.

Setting	Default	What it guards against	More
`ServerManager.SetAuthenticator(...)`	`AcceptAllClientAuthenticator`, which approves everyone	Strangers joining a public server. Install your own before the server starts listening.	[Authenticating clients](../../core-api/core/authenticatio
`ServerManager.AuthenticationTimeoutSeconds`	`30`	A client holding a slot open without ever authenticating. Zero disables it.	[Authenticating clients](../../core-api/core/authenticatio
`ServerManager.KickTimeoutMilliseconds`	`500`	A kicked client that never acknowledges, keeping its link open.	[Kicking and disconnecting a client](../../core-api/core/kicking-and-di
`RpcManager.MaximumInboundRpcsPerConnectionPerDrain`	`64`	A client packing many remote calls together to cost the server work.	[Who may send a call, and how many](../../core-api/messaging/rpc-ser
`SystemManager.PredictedSpawnValidator` (Pro)	`SpawnValidator.Deny`	A client creating objects it should not.	[Client-predicted spawning in Unity](../systems/predicted-spawning.m

The one related setting with an inspector field is **Scene Load Timeout (s)** on the `UnitySceneManager` (180 by default). It gives up on a scene load a client never answers.

Lock down who may set members and send calls

By default only the server and the controlling client may set a `NetworkSystem`'s members (`MemberWriteAccess.Controller`) or send it remote calls (`RpcSendAccess.Controller`). Neither has an inspector field. Only the server can widen them, from a `NucleusBehaviour` through its `NetworkSystem` property:

```
if (IsStarted(Invoker.Server))
    NetworkSystem.TrySetRpcAccess(RpcSendAccess.AnyClient);
```

Widen access only where every nearby player needs it, such as a door anyone may open. `MemberWriteAccess.AnyClient` and `TrySetWriteAccess` are Pro. See Letting clients set members and Who may send a call, and how many.

Validate every value a client is allowed to write

Access decides whether a client may write, not whether the value makes sense. Implement `IMemberWriteValidator<T0>` on the `NetworkComponent` that owns the member and return `MemberWriteAction.Decline` for anything the game should not accept. It runs on the server only, and a validator that throws declines the write. This includes the per-tick input a controlling client sends. See Using prediction for an input example and Judging and arbitrating client writes for the full contract.

Check the sender and arguments of every remote call

A server handler can trust `RpcContext.SenderConnection`, which the transport resolved, and nothing else the client sent. Check each argument against what the server knows, and return `RpcRelayAction.Cancel` to stop a call reaching other clients. See Judging and refusing a relayed call.

Decide what each violation costs the client

Most violations are rejected silently and never kick. On the build that runs the server:

- Register a deciding handler that kicks for the faults an honest client cannot produce.
- For faults an honest client can reach through loss or a race, count raises per connection before kicking.
- Register an `IViolationObserver` that logs everything.

See Decide what each violation costs the client for worked handlers and the Violation reference for each type.

Leave server-only code out of client builds

Unity defines `UNITY_SERVER` when it builds for the Dedicated Server platform. Code wrapped in it is left out of client builds, so players cannot read your validation rules, cheat thresholds or loot tables from the game files. This hides your checks; it does not make them stronger. Wrap the server-only logic, such as a call handler and its registration:

```
public class ScoreBoard : NucleusBehaviour<ScoreComponent>
{
    protected override void OnSystemLinked()
    {
#if UNITY_SERVER || UNITY_EDITOR
        NetworkSystem.RegisterRpcHandler<AddScoreRpc>(OnAddScore);
#endif
    }

#if UNITY_SERVER || UNITY_EDITOR
    private RpcRelayAction OnAddScore(in RpcContext rpcContext, AddScoreRpc rpc)
    {
        if (rpc.Amount is < 1 or > 10)
            return RpcRelayAction.Cancel;

        Component.Score.Value += rpc.Amount;

        return RpcRelayAction.Relay;
    }
#endif
}
```

- **Keep everything networked the same in both builds.** Never wrap a `NucleusBehaviour` or `NetworkComponent` class, a member, or a `[NetworkType]` type, including a call or message only the server handles. Both builds must agree on every networked type, or the server reads the client's data as a forged packet and raises `MalformedDataViolation`.
- **Add `UNITY_EDITOR` to the condition** so the server code stays in the editor, where you play-test as the server whatever the build platform.
- **Use it only when every server is a dedicated server.** A build without the server code cannot host, so a game whose players host needs the code in every build that can host.

Check this list before you ship a server

- The Synapse Transport's Client Limits Enabled is ticked, with rates that suit your game.
- Any transport without its own limits is covered by the Unity Server Manager's client limits, with Maximum Packet Bytes set.
- Maximum Connections on the transport is above zero.
- Your own authenticator replaces `AddressClientAuthenticator`, and `AuthenticationTimeoutSeconds` is above zero.
- Only the login message sets `IsAuthenticationRequired` to `false`.
- Every object on `AnyClient` access is there on purpose.
- Every component with members a client can write, input included, implements `IMemberWriteValidator<T0>`.
- Every server call handler that relays between clients returns `Cancel` for anything you would not show other players.
- Deciding handlers and an `IViolationObserver` are registered on the server build.
- The server runs a release build of Nucleus. A debug build ignores `MalformedDataViolation` instead of kicking for it.

Technical limitations

These behaviors are not supported

Do not design a game that depends on either of these.

- **Send rate does not change with distance.** No shipped interest type slows an object's sends because it is far from a peer. Per-member send interval and transmission mode exist, but neither is driven by distance.
- **A replay does not roll back the whole physics world.** A reconcile's replay rewinds only the bodies a validated behaviour predicts and holds every other body still; see Predicting a body with a validated behaviour.

These features are planned but not built yet

- **Network profiler window**: an editor window attributing traffic to messages, objects and members.

Lag compensation (server-side rewind, lag-compensated queries and hitbox history) is already built, as a Pro feature. See How lag compensation works.

Violation reference

A violation is raised through `ViolationManager` when a peer sends something the protocol does not permit. The offending operation is always rejected. The violation only decides the `ViolationAction` taken against the sending `Connection`: `Ignore` or `Kick`. Logging is separate, set through `ViolationContext<T0>.Log`. To change or watch the outcome, see Handling violations in Unity.

There are twenty-two violation types.

Peer-behavior violations default to Ignore

Type	Payload	Check first
`UnauthorizedMemberChangeViolation`	`SystemId`	Whether the sender controls that `NetworkSystem` right now. A control handoff racing a member change looks the same.
`PredictedMemberChangeViolation`	`SystemId`	Whether the member is really predicted. A controller never sends a predicted member's value, so the member's transmission setup probably changed without the controller being rebuilt.
`EmptyCollectionDeltaViolation`	`SystemId`, `MemberId`	A compliant sender cannot produce a collection delta with no operations, so suspect two builds whose serializers differ before suspecting the sender.
`InvalidStateAckViolation`	`AckedTick`	Whether the peers' tick counters diverged (a restart, a save and load, a manual tick reset) before assuming a forged acknowledgment.
`RetentionExceededViolation`	`AckedTick`	The peer's latency against `SystemManager.StateRetentionMilliseconds`. A single raise is ordinary heavy loss.

`RecoveryBudgetExhaustedViolation`	`ConsecutiveSeconds`, `SecondsInLastMinute`	Both counts. It is raised at most once a second per Connection, for a second in which the server refused that peer a repair because its budget under `SystemManager.RecoveryBudgetFullWorlds` was spent. A short run is a burst of loss; a long `ConsecutiveSeconds` or a high `SecondsInLastMinute` is a peer on constant heavy loss, or one reporting losses it never had.
`StateLossReportFloodViolation`	`AllowedLossRepairCount`	Whether the peer just recovered from a stall, the only time an honest peer reaches this. Repairs past the allowance are refused either way.
`UnexpectedStateLossReportViolation`	`Tick`	Whether a handover just changed which peer serves the sender its state. A report already in flight across that change reaches this honestly. The report is discarded and a warning is logged.
`RecoveryUnconfirmedViolation`	`UnconfirmedServeCount`	Whether packet loss is eating the recovery itself before assuming the peer is stalling on purpose.
`UnauthorizedSpawnViolation`	`SystemId`, `SystemBundleId`, `SystemLocalId`	A client still sending state for an object the server just despawned does not raise this, so a raise means the id was never one the server despawned.
`RpcSendPermissionViolation`	`SystemId`	Whether permission was revoked while calls were in flight. A well-behaved client reaches this on that race alone.
`RpcFloodViolation`	`ReceivedRpcCount`, `AllowedRpcCount`	`RpcManager.MaximumInboundRpcsPerConne against what the client needs to send in one drain.

Unreadable client data raises MalformedDataViolation

`MalformedDataViolation` is raised on the server when reading data a client sent throws, so the data is not applied.

- In a release build it defaults to `Kick` and logs `Sent data the server could not read.` as a warning.
- In a debug build (`#if DEBUG`) it is `Ignore` with no line, and the exception is logged instead.

- It is never raised on a client, for data with no known sender, for a server Connection (a host that dialled its own address reads from one as a client), or for a host's own client. There the exception is logged and nobody is kicked.

Type	Payload	Check first
`MalformedDataViolation`	`Kind` (`State`, `FullBody`, `Recovery`, `ClientState`, `Message`, `Rpc`), `SystemId` (zero when not for a system)	Whether both builds share the same serializers and member layout for that `Kind`. A client and server built from different versions look exactly like a forged packet. Run a debug build to see the exception.

A client over its limits raises ClientLimitViolation

`ClientLimitViolation` is raised on the server only, while `ServerManager.ClientLimits` has `Enabled` set, when a client crosses a limit. The packet that crossed it is discarded unread. A size limit raises it once per refused packet. A rate limit raises it once per one-second window and silently discards the rest of that client's packets in the window. A host's own client is exempt. It defaults to `Kick` and logs a warning.

Type	Payload	Check first
`ClientLimitViolation`	`Kind` (`PacketBytes`, `SplitPacketBytes`, `PacketsPerSecond`, `BytesPerSecond`), `MeasuredValue`, `MaximumValue`, `Channel`	`MeasuredValue` against `MaximumValue` and what the game honestly sends at its busiest. Set `MaximumPacketBytes` at least one transmission unit above the largest honest packet, or honest split packets are refused.

Protocol-direction violations default to Kick

The server drives every bundle and scene load and release. Nothing in the engine sends these upstream, so an unmodified client never raises them. When one fires, first check your own code for a request or report sent from the wrong side.

Type	Payload	Check first
`UnexpectedBundleRequestViolation`	`BundleId`	Custom code calling the bundle-request path on the wrong side.
`UnsolicitedBundleReportViolation`	`BundleId`, `IsLoadReported`	Whether the server requested that bundle load, or recorded the client holding it before an unload report. A mismatched build id looks the same as a forged report.

`UnexpectedSceneRequestViolation`	`SceneHandle`, `IsLoadRequested`	Custom code calling the scene-request path on the wrong side.
`UnsolicitedSceneReportViolation`	`SceneHandle`, `IsLoadReported`	Whether the server requested that load or release. A handle never requested and never recorded as held, including the unset handle, lands here too.

Three violations exist only in Pro

`PacketTransformRejectedViolation`, `UnauthorizedDespawnViolation` and `UnsolicitedLeaseAcknowledgementViolation`, and their events on `ViolationManager`, do not exist in a Free build.

Type	Payload	Default	Check first
`PacketTransformRejectedViolation`	`Channel`, `PayloadByteCount`	`ViolationManager.DefaultAction` (`Ignore`), and always logs a warning	Whether the peer runs the same `IPacketTransform` configuration. Stray packets from something other than this game reach the port too, so this alone is not evidence of an attack.
`UnauthorizedDespawnViolation`	`SystemId`	`Ignore`	Whether the client and server builds agree on that prefab's removal policy. Version skew reaches this honestly.
`UnsolicitedLeaseAcknowledgementViolation`		`Kick`	Whether the client build matches the server's. An honest client acknowledges only the predicted spawn top-ups the server sent it.

Lag compensation raises LagCompensationViewClampedViolation

`LagCompensationViewClampedViolation` exists in every edition, but only lag compensation (Pro) raises it, at most once per query, when the shooter's view is older than the rewind window or newer than anything the server has recorded. The view is clamped into range and the shot is still judged. See UnityLagCompensationManager.

Type	Payload	Default	Check first

`LagCompensationViewClamped`	`RepeatedViolation`, `ClampedTick`	`Ignore`	The shooter's latency against `LagCompensationManager.Maximum` A single raise is a link that spiked; a steady stream is a very slow link, or a client asking to be judged against the oldest world it can get.

Do not kick for a single loss-related raise

Most types default to `Ignore` because an honest peer can reach them through a race (a control handoff, a revocation, a recovery re-serve) or through loss: `RetentionExceededViolation`, `RecoveryUnconfirmedViolation`, `RecoveryBudgetExhaustedViolation`, `StateLossReportFloodViolation` and `PacketTransformRejectedViolation`. Treat a single raise of these as noise.

What the server trusts

Every check on this page runs on the server's copy of the system. A client can reach the same verdict locally for prediction or UI, but a kick it decides or a write it declines to send is not enforcement.

Observation gates everything

Before any access check, the server asks whether the sending connection observes the system being addressed. A connection that does not is rejected, whatever the `MemberWriteAccess` or `RpcSendAccess`. A client therefore cannot reach a system by guessing its id: an id it was never told about, or has lost interest in, resolves to nothing.

Next, the system's controller may always set its members and call it, under any access. Any other observer is admitted only when the access is `AnyClient`.

Nothing travels as a grant

The server re-checks every incoming write and call against its own copy of the access, on every packet. The access value and the hint the server sends each observer only let an honest client avoid sends the server would refuse. A client that forges them or ignores them gains nothing.

The sender is the connection the transport resolved

An id a client puts in a payload is only a claim. A handler is given the `Connection` the transport resolved for the packet (`RpcContext.SenderConnection` for a call, and the same resolved connection for a member write), and the client cannot change it.

A call the server relays to other observers still carries the original sender's connection id. Only the server writes that id: a call arriving from a client is attributed to that client whatever originator id its packet names, so a client cannot pose as another client to a handler, to the relay's sender exclusion, or to `RpcContext.SenderConnection`.

Connections are not players

- `Connection.RemoteAddress` is where the connection came from, observed by the server, so it can be part of an identity check. It names a location, not a player: clients behind the same NAT share one address.
- `Connection.Id` is assigned per session and reused once a connection is gone.
- To recognize a player across a reconnect, key on `Connection.Identity`, the value your authenticator sets on approval. Matching a returning `Identity` to what it owned before is up to your game code.

Arbitration is not a security boundary

`MemberWriteAccess.AnyClient` (Pro) lets every observer set a system's members. A Free build has no `MemberWriteAccess`, so only the controller sets members and nothing needs arbitrating.

In Pro, when several observers write to a system at once, arbitration picks a winner that is arbitrary but stable. A client that keeps reconnecting to change its connection id can farm a favorable outcome. Arbitration only chooses between equally legitimate writes; it does not decide whether a write should be allowed. Where winning matters, put your own arbiter in game code on top of the access check.

Nucleus does not encrypt traffic

To encrypt, compress, obfuscate or add an integrity check to the bytes themselves, use the packet-transform layer, `TransportManager.PacketTransform` (Pro). It rewrites every packet's payload on both peers before anything else in the engine sees it.

A shooter's view is bounded, not trusted

Lag compensation (Pro) judges a shot against the moment the client says its screen was showing. A claimed view older than `MaximumRewindMilliseconds`, or newer than anything the server recorded, is clamped into range, still judged, and raises `LagCompensationViewClampedViolation` (ignored by default). Within the window a client can name any moment, so the window is the limit on how far a shooter is favored.

The ray is also the client's claim. Clip it at walls on the server against the present-time scene. The queries test it only against the hitboxes in the server's own history, never against anything the client reports about a target. See UnityLagCompensationManager.

Unity

Documentation for the Unity integration, organized by subject. Each category below favors the Unity component surface; the same subject from plain C# lives under Core API.

- Core, Managers and the Tick Loop
- Systems, Objects and Spawning
- Replicated State and Change Callbacks
- Control, Client State and Prediction
- Messaging and Remote Calls
- Interest Management and Scaling
- Networked Scenes, Content Bundles and Persistence
- Physics Synchronization
- Lag Compensation
- Transports, Connectivity and Hosting
- Serialization, Codegen and the Wire
- Diagnostics and Troubleshooting

What area of interest is

Driving the core API directly? See What area of interest is.

With no interest rule, every started `NetworkSystem` replicates to every authenticated client, every tick. Interest rules only restrict that default: no rule can send a client something it would not otherwise get. A world with no rules pays almost nothing for interest.

Rules stack, and the strictest one wins

Rules come from two places, and both stack:

- **World Rules** on the Unity Interest Manager apply to every system that starts.
- **Rules** on a prefab's Network Interest Object apply to that object only.

For each effect it declares, a rule answers `Met`, `Unmet` or `None` (no opinion). Only `Unmet` restricts, and the most restrictive answer across every rule wins.

Interest is decided for one object and one client at a time, and a changed verdict takes effect on the tick it is decided. A rule you write must only read: it must not start or stop systems, change observers, or block.

A rule can stop spawning, stop updates, or both

- **Spawn** (`InterestEffect.Spawn`): whether the object exists for the client at all. While unmet, the object never spawns there, and one already spawned is despawned like an ordinary despawn.
- **Stop** (`InterestEffect.Stop`): whether the object still receives updates. While unmet, the object stays on the client, frozen, and costs nothing to keep current. Only a later unmet Spawn removes it.

See Frozen or gone: what a stop does to the object for what each looks like on the client.

Evaluation runs on a cadence

Evaluation Cadence Ticks on the Unity Interest Manager sets how many ticks pass between evaluations of any one object. The default is 5, and 1 evaluates every object every tick. The work is spread across those ticks rather than done all at once.

A client that just authenticated, and a system that just started, are evaluated at once.

The work grows with objects times clients. See What scales and what does not.

Making a host see what its players see

Driving the core API directly? See The host interest report

A host is the server, so it holds every object whatever interest resolves for its players. Unless something acts on that, the person hosting sees every object in the level while their players are sent only some.

Turn on host hiding

On the `UnityInterestManager`, set **Host Interest Enabled** before the host starts. **Host Visibility Enabled** is already on, so this one switch turns hiding on.

- **Host Interest Enabled** resolves the host's own client against the interest rules, so the host learns which objects its client half would not have been sent. It is off by default because it costs the host one ordinary condition pass per evaluated pair, the same as every remote client.
- **Host Visibility Enabled** hides an object on the host while that report says its client half would not hold it. Only the renderers go dark: the object stays, keeps simulating and keeps replicating.

The hiding is done by a `NetworkHostVisibility` component, which is added to every networked object once the peer becomes a host, including objects already spawned. A client or a dedicated server never gets one.

Use NetworkHostVisibility from a script

Member	What it tells you
`IsVisibleToHostClient`	True while a host's client half would be holding this object, which is every moment outside a cull. Always true on a client, on a dedicated server, and on a host that resolves nothing for itself.
`HiddenRendererCount`	How many renderers this component currently has switched off. Zero on a peer that hides nothing, and on an object currently shown.
`RefreshRenderers()`	Gathers the renderers under this object again. Call it after adding or swapping renderers at runtime; a renderer created after this component's `Awake` is otherwise unknown to it.
`HostVisibilityChanged`	Raised when the object crosses into or out of what a host's client half would be holding, after the renderers have been switched. For a script that has more to hide than renderers.

A renderer the game had already disabled stays disabled. Only renderers this component switched off are switched back on.

Answer the report yourself

Clear **Host Visibility Enabled** to keep the report without the built-in hiding. Then handle `NetworkHostVisibility.HostVisibilityChanged` on a component you place yourself, or `NetworkSystem.HostInterestChanged` directly. Use this to dim an object instead of hiding it, or to hide more than its renderers.

Avoid these mistakes

- Both inspector switches are read once as the session starts, so changing either during play has no effect.
- `InterestManager.HostInterestEnabled` can change from code mid-session, but whether to hide is decided once, when the peer first becomes a host with its client linked. Turning it on after that gives the report with nothing hiding on it. Turning it off returns every object's membership to `InterestMembership.Streamed`, which shows it again. Code that sets it before the host starts must run after `UnityInterestManager` has applied its settings, or the inspector value overwrites it.
- A dedicated server has no client half. Leave both switches alone.
- Only `InterestMembership.Unspawned` hides an object. A system that is only stopped for a player keeps its object, frozen, so the host shows it too.

Interest Management and Scaling

Pages in this category:

- Network Interest Object
- Unity Interest Manager
- What area of interest is
- Limit what a client receives by distance
- An object is missing, or will not go away
- What scales and what does not
- Frozen or gone: what a stop does to the object
- Making a host see what its players see

An object is missing, or will not go away

Driving the core API directly? See An object is missing, or will not go away.

To see what interest resolved, add a `NetworkInterestOverlay` to the player object, switch it on, and select the object on the server or host while playing. A line goes to every other networked object, colored by whether the player is streamed it, stopped for it, or does not have it at all. See Developer tools in the editor.

Distance rules, the spatial grid and object pacing are Pro. The Scene rule and the base interest system ship in free. If a setting below seems not to apply, check whether the build is free.

A client receives objects from a scene it never loaded

No rule gates scenes, so the objects land in the wrong scene with no error anywhere. The Unity Interest Manager inspector warns "No rule gates scenes" in this case.

Fix: add the Scene rule to **World Rules** on the Unity Interest Manager, unless a rule of your own decides scene membership.

Nothing is ever culled, however large the stop distance is

A distance rule that cannot measure a pair never restricts it. Unity's distance rules read the linked GameObject's transform through `TryReadSourcePosition`. A pair cannot be measured when the client controls nothing yet, or when `TryReadSourcePosition` returns `false` for the object or for every object the client controls.

Fix: make sure each player controls the object it plays through. In a condition of your own, make sure `TryReadSourcePosition` answers for both the evaluated object and the controlled ones.

An object disappears at range when it should only freeze, or the reverse

Past the despawn distance the object is despawned. Past only the stop distance it stops, and despawn wins when both apply. Whether a stop removes the object is `NetworkSystem.DespawnWhenStoppedEnabled`: off, the object freezes with its last state; on, it is removed once stopped.

On an object with several systems sharing one `NetworkSystemGroup`, the object leaves only once every member has stopped with `DespawnWhenStoppedEnabled` set. One member with it off keeps the object in place. A system with no group despawns on its own stop.

Fix: check `DespawnWhenStoppedEnabled` on every system in the group, not just one.

A distance rule only stops, and never despawns

The inspector's Distance rule (`AuthoredDistanceInterestCondition`) has only a **Stop Distance** and no despawn distance, so it never despawns by distance.

Fix: to despawn by distance, construct a `DistanceInterestCondition` in code, which takes a `stopDistance` and a `spawnDistance` separately, and register it globally or per system.

Objects flicker in and out at a cull boundary

A distance cutoff has no hysteresis, so a player moving back and forth across it flips the verdict each evaluation. Only **Evaluation Cadence Ticks** limits this, because distance is measured once per that many ticks.

Fix: widen the gap between the cutoff and how far a player can move in one evaluation cadence, by raising the cutoff, lowering the cadence, or both. No setting smooths the boundary itself.

A new client sees nothing for a second or two

A client is not sent any object until its link is measured (`Connection.CanReceiveState`), typically one round trip after it authenticates, or at once for a host's own client. This is expected and resolves itself. If it never resolves, check that the transport completes a round trip for that connection.

A join or scene load arrives as one huge tick and drops

Object pacing limits how much of a join lands in one tick. Set it with the **Maximum Object Kilobytes Per Tick** slider under **Object Pacing** on the Unity Server Manager (`ServerManager.MaximumObjectKilobytesPerTick`). It is the most kilobytes (1 kB is 1,024 bytes) of new objects one client is sent per tick, 50 by default. What does not fit is sent on later ticks, and objects resent after packet loss share the same budget. Pacing cannot be switched off, so a join that still lands as one tick too large to arrive means the budget is wider than that client's link can take: lower it. One object wider than the whole budget is still sent, alone on its tick.

To check whether the budget is holding spawns back:

- `InterestManager.TotalSpawnsDeferred` counts spawns that have had to wait. A world that never fills the budget keeps it near zero.
- `Connection.SystemsAwaitingSpawnAdmissionCount` is how many spawns one connection is still owed.
- A warning is logged once a connection has been owed spawns for `InterestManager.SpawnAdmissionStallWarningTicks` (90) ticks without receiving one, naming the connection and the current budget.

If that warning fires, the world spawns faster than the budget lets that client receive: raise the budget, or spawn less at once. A `NetworkSystemGroup` wider than the budget is not a fault on its own; it arrives over more than one tick.

Setting the technique or axes logs an error and does nothing

The technique and axes are refused, with a logged error, once any distance rule has registered in the world, globally or per system.

Fix: set **Interest Technique** and **Interest Axes** on the Unity Interest Manager before play. From code, call `InterestManager.SetInterestTechnique` during setup, before any system carrying a distance rule can spawn.

Limit what a client receives by distance

Driving the core API directly? See Distance interest conditions

A Distance rule stops an object's systems replicating to a player once that player is too far away. Distance rules are Pro; a free build has none.

Add a distance rule

Add a **Distance** rule to one of these lists:

- **World Rules** on `UnityInterestManager`, to apply it to every object.
- **Rules** on a prefab's `NetworkInterestObject`, to apply it to that prefab only, on top of the world rules.

Set **Stop Distance**: the distance past which the object stops replicating to a player. At `0` the rule is off and costs nothing.

Distance is measured from the `NetworkSystem`s the client controls, not from a camera. A client that controls nothing measurable is never restricted by the rule.

Choose how distance is measured

Both settings are on `UnityInterestManager`. Set them before anything carrying a distance rule spawns; a change after that is refused.

- **Interest Axes**: `Xyz` measures the full volume, and each other option drops one axis. A top-down world uses **Xz**, so a player on a floor above an object is not treated as far from it.
- **Interest Technique**: **Pairwise** compares each object with each client's own objects, and is cheapest until there are many of both. **Spatial Grid** narrows to nearby objects first. Both give the same answers. Leave it on Pairwise until the cutoff excludes most pairs.

Use one distance rule per object

A `NetworkSystem` accepts only one rule that decides by distance. A second is refused, and logged, as the object spawns. The `NetworkInterestObject` inspector warns when its **Rules** list holds more than one.

Network Interest Object

> **Driving the core API directly?** See Registering interest conditions.

`NetworkInterestObject` sets interest rules for a whole prefab. Its rules apply to every `NetworkSystem` on the object, so the object is filtered as the one thing a player sees. It needs a `NetworkSystemObject` on the same GameObject. Only the server resolves interest, so on a client the component does nothing.

Add rules to the prefab

Add rules with the list's add button. Each rule only restricts, so an empty list filters nothing; it does not hide the object from everyone.

- **Controller Only** (`AuthoredControllerOnlyInterestCondition`) shows the object only to the client that controls it, for an inventory or other private state. It has no settings. Add it to each prefab that needs it, not to the Unity Interest Manager's **World Rules**: there it applies to every object, and an object no client controls is shown to no client. This is the only rule a free build offers here.
- **Distance** (Pro) stops replication past a distance. See Limit what a client receives by distance.

The Scene rule (`AuthoredSceneInterestCondition`) is world-only (`IsObjectScope` is `false`), so it is offered on the Unity Interest Manager instead.

To add your own rule, derive from `AuthoredInterestCondition`, mark the type `[Serializable]`, and build its `IInterestCondition` in `CreateCondition()`, which is called for every system on the object. It then appears in the list beside the shipped rules.

At most one rule on an object may decide by distance (`IsPositionCondition` is `true`). A second is refused as the object spawns, and the inspector warns while the list holds more than one.

Choose whether a stopped object is removed

Despawn When All Stopped is on by default: the object is removed from a player once every system on it has stopped for that player. Turn it off for something a player should keep seeing after it stops updating, such as a landmark or a corpse. Nothing on the component will then remove the object; something else has to. See Frozen or gone: what a stop does to the object.

Override a stop from a script

The component raises `NetworkSystem.InterestStopResolving` for its systems as its own `InterestStopResolving` event. A handler can waive a stop the rules resolved, or stop a player the rules did not. See Override a stop with InterestStopResolving.

Frozen or gone: what a stop does to the object

Driving the core API directly? See Stopping versus culling.

When a `NetworkSystem` stops for a player, its object either stays on that player's client, frozen where it was, or is despawned there like an ordinary despawn. A frozen object is cheap to bring back, because only its updates resume.

`NetworkSystem.DespawnWhenStoppedEnabled` chooses which. Rules and `InterestStopResolving` decide whether a system stops; this setting decides what the stop does.

Choose frozen or gone with Despawn When All Stopped

On `NetworkInterestObject`, the **Despawn When All Stopped** checkbox sets `DespawnWhenStoppedEnabled` on every `NetworkSystem` on the object. It is on by default.

On an object with several systems, the object is despawned for a player only once every system has stopped for that player. One system still replicating, because its rules have not stopped it or a handler waived the stop, keeps the whole object with that player.

Turn it off for something a player should keep seeing after it stops updating, such as a landmark or a corpse. Interest will then never remove the object, and the inspector warns about this. Remove it some other way, such as a lifetime timer or an explicit despawn.

Override a stop with InterestStopResolving

Rules can stop a system but never un-stop one. `NetworkInterestObject.InterestStopResolving` lets you waive a stop the rules resolved, or stop a pair the rules did not:

```
public event NetworkSystem.InterestStopResolvingHandler InterestStopResolving
```

It fires once per (system, connection) pair on the interest cadence, and one subscription covers every system on the object. Test `InterestStopContext.NetworkSystem` against the system you care about and leave other contexts alone:

```
private void OnInterestStopResolving(ref InterestStopContext context)
{
    if (context.NetworkSystem != _mySystem)
        return;

    context.IsStopped = false; // keep this member alive for the player
}
```

While **Despawn When All Stopped** is on, waiving a stop on one system keeps the whole object with that player.

Follow these rules in a handler:

- Keep it a cheap, pure read. It runs on the network loop thread, and must not start or stop systems, change observers, or block.
- Subscribe only where you need the decision. A subscriber alone makes the interest pass visit the object, even if the prefab has no rules.

Unity Interest Manager

`UnityInterestManager` holds the world-wide interest rules and settings for the core `InterestManager`. Its settings are applied when the session starts, so changing them during play has no effect.

In a free build the inspector shows only **World Rules**, **Evaluation Cadence Ticks** and the two host switches. The **Distance Measurement** section is Pro.

World Rules apply to every object

Each rule in this list applies to every `NetworkSystem` that starts, across the whole world. A new component starts with the Scene rule (`AuthoredSceneInterestCondition`).

Keep the Scene rule unless a rule of your own decides scene membership. Without one, the inspector warns, and a client receives objects from scenes it never loaded. They land in the wrong scene with no error, and several open copies of one scene cannot be told apart by distance.

Set the cadence and the host switches

Setting	Default	What it does
Evaluation Cadence Ticks	`5`	Ticks between interest evaluations of any one object. A joining client and a newly started object are resolved at once, off the cadence.
Host Interest Enabled	off	Resolves the host's own client against the rules, so the host learns which objects its client half would not have been sent. It costs the host one ordinary condition pass per evaluated pair, and culls nothing by itself.
Host Visibility Enabled	on	Hides an object on the host while that report says its client half would not hold it. Because this is already on, setting **Host Interest Enabled** is all it takes to turn hiding on. Clear it to keep the report and act on it yourself, for example through `NetworkSystem.HostInterestChanged`.

When hiding is on, a `NetworkHostVisibility` component is added to each networked object on a host only. See Making a host see what its players see.

Distance Measurement sets how distance is measured (Pro)

- **Interest Technique**: how every distance in the world is measured, pairwise or through a spatial grid.
- **Interest Axes**: which axes distances are measured across, the full volume or a plane that drops one

axis.

Set both before anything carrying a distance rule spawns. A change after a distance rule has registered is refused with a logged error. See Limit what a client receives by distance.

What scales and what does not

Driving the core API directly? See What scales and what does not.

Four ways to send less

From the largest saving to the smallest:

Don't send it. A condition that vetoes `InterestEffect.Spawn` means the client never holds the object, so nothing is sent at all.

Stop sending it. `InterestEffect.Stop` pauses updates to a client that keeps the object. It is cheaper than a fresh spawn later, but the object still exists on that client.

Send it less often. `SendInterval.Normal` sends on every tick the value changed; a wider `SendInterval` collects changes and sends at most once per interval. This applies only under `TransmissionMode.Interval`; a member on `TransmissionMode.Divine` sets its own rate. A spawn, resync, recovery or reconcile always carries the current value, whatever the interval.

Send it smaller. `TransmissionMode` and per-value compression make each send smaller without removing any.

Find which cost is too high

"Replication is too expensive" is usually one of three problems, and fixing the wrong one does nothing.

Cost	What it is	What lowers it
Per-client egress	Bytes one client's connection sends per second, all the time.	Spawn and stop culling first, then interval and compression. Pacing does not help.
Burst size	How much lands in one tick on a join, a scene load, a bundle release, or a client walking back into range. A large tick is costly to lose.	`ServerManager.MaximumObjectKilobytesPerT (default 50) spreads new objects over several ticks. It lowers neither the join's total cost nor steady egress, and a join that fits the budget never touches it.
Interest evaluation	Server CPU spent deciding who sees what. It grows with the number of objects, clients and rules.	**Evaluation Cadence Ticks** is the only setting.

Read the cost before you change anything

Use the bandwidth counters (`TotalBytesSent`, `TotalBytesReceived`, `TotalStatePacketPayloadBitsSent`, `TotalStatePacketPayloadBitsReceived`) and the per-object and state-only statistic rows. Every counter covers a whole peer's traffic, not one object's. A per-object figure is that total divided by the object count; the per-object statistic row does the division for you.

Don't plan around what Nucleus does not have

- No cap on ordinary traffic. Only new objects are paced, and nothing decides what is sent first when a client is behind.

- No per-object send priority. Every eligible object is served the same way.

Pro and free differ

Distance and spatial grid culling and object pacing (`MaximumObjectKilobytesPerTick`) are Pro. Pro also compresses values more tightly than free. Compression is fixed by the edition that built the peer, and cannot change per world.

Work out the cost of a small world

Take 50 objects, each with one `NetworkMember` sending a full-precision value every tick at `SendInterval.Normal`, and 20 clients.

- **Egress:** each client receives its own 50-object slice every tick, 1,000 sends per tick across all clients. Per-client egress grows with how many objects that client can see.
- **Evaluation:** at a cadence of 10, a client can see a change in range up to 10 ticks late.
- **Join burst:** a joining client is owed all 50 objects' starting state in one tick. That is far under 50 kB, so pacing never engages.

This world needs none of the levers yet.

Clipping shots at walls

Driving the core API directly? See Walls are not in the history.

Clip the shot before you judge it

Lag compensation records hitboxes, not level geometry, so a rewound query sees no walls. On the server, judge a shot in two steps:

Clip it against the present scene with `LagCompensationWalls.GetDistance`, which returns the distance to the nearest wall along the shot, or the full range when no wall is in reach.

Pass that distance as the range of `TryRaycast` or any other 3D query from the same origin. For 2D, pass `GetDistance2D`'s result to `TryRaycast2D` or any other 2D query.

```
float range = LagCompensationWalls.GetDistance(shot.Origin.ToUnity(), shot.Direction.ToUni
ty(), maximumRange, wallLayerMask);

bool isHit = lagCompensationManager.TryRaycast(shooterConnection, shot.View, shot.Origin,
shot.Direction, range, hitboxLayerMask, out LagCompensatedHit lagCompensatedHit);
```

Call it on the main thread, as with every Unity physics query. A message handler in a Unity session already runs there. It allocates nothing.

Don't clip with a plain Physics.Raycast

A plain `Physics.Raycast` also stops on the colliders of networked objects. An automatic `NetworkHitbox` needs a live collider beside it, so the raycast stops on the target's present-time collider. When the target runs toward the shooter, that collider stands nearer than its rewound hitbox, and a clean hit is judged a miss. A ray from a third-person camera stops on the shooter's own character controller almost at once.

`LagCompensationWalls` passes over a collider when its GameObject has a `NetworkHitbox`, or when it sits on or under a GameObject carrying a `NetworkHitboxRoot`, enabled or not. A disabled root's colliders are passed over too, so making an object unhittable never turns it into cover.

One limit follows: a wall whose near face lies inside a passed-over collider of a hitbox object is not seen.

Triggers never stop a shot

A trigger never stops a shot, whatever `Physics.queriesHitTriggers` and `Physics2D.queriesHitTriggers` say. A shooter standing inside a pickup or checkpoint volume still fires, and a trigger flush with a wall never carries a shot past it.

A direction it cannot follow returns zero

A direction with no length, or an infinite or NaN one, returns zero, which stops the shot where it starts. A client can never clear a shot through walls by sending a direction the check cannot follow. Any other direction need not be of unit length.

Use GetDistance2D for 2D

`GetDistance2D(Vector2 origin, Vector2 direction, float maximumDistance, LayerMask wallLayerMask)` does the same in the default 2D physics world. A 2D shot fired from the shooter's center starts inside its own collider, which is passed over like any other hitbox object's collider. A wall the shot itself starts inside stops it at distance zero while `Physics2D.queriesStartInColliders` is on, as an

ordinary 2D raycast would.

Pass the physics scene when scenes have their own

`GetDistance(origin, direction, maximumDistance, wallLayerMask)` looks in `Physics.defaultPhysicsScene`, which holds none of the walls of a scene loaded into its own local physics world. A networked scene loads that way when `UnitySceneLoader.LocalPhysicsEnabled` is on, which the scene manager's stacked-scene simulation setting turns on; see Local physics scenes. Where that can happen, pass the shooter's physics scene first:

```
float range = LagCompensationWalls.GetDistance(shooterScene.GetPhysicsScene(), shot.Origin
.ToUnity(), shot.Direction.ToUnity(), maximumRange, wallLayerMask);
```

This is always safe, because a scene without its own world returns the default one. `GetDistance2D` always uses the default 2D world, because Nucleus only gives a scene a local world for 3D physics.

Firing and judging a shot

Driving the core API directly? See Capturing a shot and Judging a shot with rewound queries.

Fire and judge from one script

The client captures a shot on the frame it fires and sends it. The server clips it at walls, judges it against the past, and applies the result. One script can hold both halves, because only the server runs the second:

```
using Nucleus.Connections;
using Nucleus.Integrations.Unity;
using Nucleus.Integrations.Unity.LagCompensation;
using Nucleus.Integrations.Unity.Serializers;
using Nucleus.Managers.Core;
using Nucleus.Managers.LagCompensation;
using Nucleus.Managers.Messages;
using Nucleus.Serializers;
using Nucleus.Systems;
using Nucleus.Transports;
using UnityEngine;

[NetworkType]
public struct ShotMessage : IMessage
{
    public LagCompensatedShot Shot;
}

public class Shooter : MonoBehaviour
{
    [SerializeField]
    private float _range = 100f;
    [SerializeField]
    private LayerMask _hitboxLayers = ~0;
    [SerializeField]
    private LayerMask _wallLayers = ~0;

    private CoreManager _coreManager;
    // Set by your own player code when this peer's avatar spawns, so a host's own shots s
kip it.
    private NetworkSystem _ownAvatarSystem;

    private void Start()
    {
        _coreManager = NucleusUnity.BoundCoreManager;
        _coreManager.MessageManager.RegisterMessageHandler<ShotMessage>(OnShotReceived);
    }

    private void OnDestroy()
    {
        _coreManager?.MessageManager.UnregisterMessageHandler<ShotMessage>(OnShotReceived)
;
    }

    // Call from your input code on the frame the player fires.
    public void Fire(Ray ray)
    {
        LagCompensatedShot shot = _coreManager.LagCompensationManager.CaptureShot(ray.orig
in.ToNative(), ray.direction.ToNative());

        // A host judges its own shot at once. Its view is the present, and a null shooter
 skips nothing, so it names its avatar.
        if (_coreManager.TransportManager.IsServerStarted)
        {
            Judge(shooterConnection: null, _ownAvatarSystem, shot);
            return;
        }
```

```
        if (_coreManager.TransportManager.TryGetServerConnection(out Connection serverConn
ection))
            serverConnection.SendMessage(Channel.Reliable, new ShotMessage { Shot = shot }
);
    }

    private void OnShotReceived(in MessageContext messageContext, ShotMessage shotMessage)
    {
        if (_coreManager.TransportManager.IsServerStarted)
            Judge(messageContext.SenderConnection, ignoredNetworkSystem: null, shotMessage
.Shot);
    }

    private void Judge(Connection shooterConnection, NetworkSystem ignoredNetworkSystem, L
agCompensatedShot shot)
    {
        // Walls are not in the history, so clip the shot against the present scene first.
        float range = LagCompensationWalls.GetDistance(shot.Origin.ToUnity(), shot.Directi
on.ToUnity(), _range, _wallLayers);

        if (!_coreManager.LagCompensationManager.TryRaycast(shooterConnection, ignoredNetw
orkSystem, shot.View, shot.Origin, shot.Direction, range, (uint)_hitboxLayers.value, out L
agCompensatedHit lagCompensatedHit))
            return;

        if (NetworkHitboxRoot.TryGetHitbox(lagCompensatedHit, out NetworkHitbox networkHit
box))
            Debug.Log($"Hit [{networkHitbox.name}] at [{lagCompensatedHit.Point.ToUnity()}
].");
    }
}
```

Capture on the frame the player fires

`CaptureShot(origin, direction)` takes `System.Numerics` vectors, so convert with `ToNative()` from `Nucleus.Integrations.Unity.Serializers`. It returns a `LagCompensatedShot` holding the ray unchanged and the view on screen, taken in the same call so the two always come from the same frame. The direction does not need to be of unit length.

Call it from the frame the click happens, never from code that builds the next tick's client state. Each rendered frame within a tick draws a different blend, and the view records which one was on screen.

Send the shot whole

Put the `LagCompensatedShot` itself in a message, as above, a remote call argument or a client state member. Never copy its origin and direction into plain `Vector3` fields: a `Vector3` is sent with less precision, which can turn a long shot enough to lose a headshot with no error. A `LagCompensatedShot` arrives exactly as captured; an infinity or NaN arrives as zero.

For a shot that is not a ray, such as a melee swing whose volume the server builds itself, send just the view: `CaptureView()` returns the `LagCompensationView` on screen, and the server passes it to an overlap query.

Judge on the server

Pass the shot's `View`, `Origin` and `Direction` straight into a query. Pass the sender's `Connection` as

the shooter: its own objects are skipped, and its view is checked against what the server could have shown it. Pass `null` for a shot the server fires itself.

The 3D queries are:

- `TryRaycast` and `RaycastAll` for rays.
- `TrySphereCast` and `SphereCastAll` for a thick shot swept along a ray.
- `OverlapSphere`, `OverlapCapsule` and `OverlapBox` for explosions, melee swings and area attacks.

The `All` forms and the overlaps fill a `List<LagCompensatedHit>` you pass in, nearest first. Reuse one list so nothing is allocated. The core API page describes each query and what its hit's `Distance`, `Point` and `Normal` mean.

A query's layer mask is a `uint` with one bit per layer; cast a `LayerMask`'s `value` to `uint`.

With **Queries Start In Hitboxes** on, the default, a ray or cast that starts inside or touching a hitbox hits it at distance zero, with its own origin as the point, which is right for a point-blank shot. To let the shot carry on instead, turn it off on the lag compensation manager.

Ignore an object

Every query has an overload taking a `NetworkSystem ignoredNetworkSystem` straight after the shooter, whose hitboxes it skips. Pass `null` to ignore nothing.

- Pass the root of the object to ignore, normally the shooter's own root object.
- A host firing for its own avatar judges with a `null` shooter, so it passes its avatar. A turret or NPC the server fires for passes itself.
- Everything under the named object is skipped too. Naming any member of a grouped object skips the whole object, and every object parented under it with `SystemManager.EnsureSetSystemParent`, such as a carried weapon or shield, is skipped at any depth. Naming a nested object skips only it and what is under it.

Read the hit

`LagCompensatedHit` carries the target's `NetworkSystem` (`null` when it has stopped since the judged tick), its `SystemId`, the `HitboxIndex` within its root, the hitbox's `Layer`, and the first contact's `Distance`, `Point` and `Normal`. `NetworkHitboxRoot.TryGetHitbox(lagCompensatedHit, out NetworkHitbox networkHitbox)` leads back to the hitbox's GameObject, where per-region data such as a headshot multiplier can live; see Making an object hittable.

Use the 2D queries in a 2D game

Capture with a Z of zero, and judge with the 2D queries, which take `System.Numerics.Vector2` and run in the plane Z = 0:

```
LagCompensatedShot shot = _coreManager.LagCompensationManager.CaptureShot(new(origin.ToNat
ive(), 0f), new(direction.ToNative(), 0f));

float range = LagCompensationWalls.GetDistance2D(shot.Origin2D.ToUnity(), shot.Direction2D
.ToUnity(), _range, _wallLayers);

bool isHit = _coreManager.LagCompensationManager.TryRaycast2D(shooterConnection, ignoredNe
tworkSystem, shot.View, shot.Origin2D, shot.Direction2D, range, (uint)_hitboxLayers.value,
 out LagCompensatedHit lagCompensatedHit);
```

- `TryRaycast2D` and `RaycastAll2D` mirror the 3D raycasts.

- `TryCircleCast2D` and `CircleCastAll2D` mirror the sphere casts, with their arguments in `Physics2D.CircleCast`'s order.
- `OverlapCircle`, `OverlapCapsule2D` and `OverlapBox2D` mirror the overlaps. `OverlapBox2D` takes its angle in radians, so multiply a Unity angle by `Mathf.Deg2Rad`.

A hit on a 2D hitbox has a `Point` and `Normal` with a Z of zero. A 3D query never touches a 2D hitbox, so `TryRaycast` in a 2D game finds nothing. A 2D query touches only 2D hitboxes unless **2D Queries Hit 3D Hitboxes** is ticked.

Worked example: the HitscanRewind demo

The `LagCompensation/HitscanRewind` demo shows the server judging a client's shot against the targets where that client saw them, not where they are when the shot arrives. The demo folder's `README.html` covers it step by step; this page reads its code as a template for your own game.

Run the demo

1. Open `Scenes/HitscanRewindDemo.unity` in two editors. A ParrelSync clone becomes the client.
2. Press Play in the main editor first, then in the client.
3. Click a moving target in the client's Game view.

Three blue targets, each a capsule body with a sphere head, sweep across the view fast enough that what the client sees is well behind where they really are. The transport adds 75 milliseconds to every packet each way, so two editors on one machine still see the effect. There are no walls, so every miss is a miss in time, not behind cover.

The targets copy their hitboxes from colliders

Each target prefab carries a `NetworkSystemObject` and a `NetworkHitboxRoot`. Its body and head each carry a `NetworkHitbox` left on `Automatic`, which copies the shape from the capsule and sphere colliders beside it. Nothing else is needed for the server to record them.

The colliders stay on the targets because the client raycasts them to find its own hit for the on-screen comparison. The server never touches them.

The message carries the shot whole

`HitscanShotMessage` carries the shot as a `LagCompensatedShot`, so its ray arrives exactly as captured. The other fields are demo-only, for the comparison:

```
[NetworkType]
public struct HitscanShotMessage : IMessage
{
    public bool IsAuthenticationRequired => true;

    public LagCompensatedShot Shot;
    public uint ShotNumber;
    public bool IsClientHit;
    public uint ClientTargetSystemId;
    public HitscanExactVector3 ClientHitPoint;
}
```

The client captures and sends

`HitscanShooter.Fire` runs on the frame the click happens. It captures the shot and sends it:

```
LagCompensatedShot lagCompensatedShot = _coreManager.LagCompensationManager.CaptureShot(ra
y.origin.ToNative(), ray.direction.ToNative());

serverConnection.SendMessage(Channel.Reliable, hitscanShotMessage);
```

On a server, the same method skips the message and judges the shot at once with a `null` shooter, against the present.

The server makes one query

`HitscanShooter.Judge` is the whole server side:

```
LagCompensationManager lagCompensationManager = _coreManager.LagCompensationManager;
LagCompensatedShot lagCompensatedShot = hitscanShotMessage.Shot;
uint layerMask = (uint)_hitboxLayers.value;

bool isRewoundHit = lagCompensationManager.TryRaycast(shooterConnection, lagCompensatedSho
t.View, lagCompensatedShot.Origin, lagCompensatedShot.Direction, _range, layerMask, out La
gCompensatedHit lagCompensatedHit);
bool isPresentHit = lagCompensationManager.TryRaycast(shooterConnection, default, lagCompe
nsatedShot.Origin, lagCompensatedShot.Direction, _range, layerMask, out _);
```

The first call is the one a game needs. It returns the nearest hit against the hitboxes where the shooter saw them, with its system, hitbox index, point and normal. The second passes an empty view, `default`, which means the present, only so the readout can count **Hits that only rewinding made**. A real game makes only the first call.

The hit's `SystemId` leads to the target, and the verdict goes back to the shooter in a reliable message.

Every hit shows how close the rebuild landed

Every hit leaves a grey copy of the target where the client saw it, a cyan sphere where the client's own raycast hit, and a yellow sphere where the server's rewound raycast hit. When both hit the same target, the server answers with the distance between the two points, usually a few hundredths of a metre.

The orbiting spheres show what a hitbox needs from replication

Two spheres between the first and second targets orbit and swap between a close and a far orbit together. The left one carries its own `NetworkTransform`. The right one is posed by a `NetworkAnimator` with **Precise Playback** on, so the client poses its animator at the playback position the server sent. Both land within 0.001 metres of where the server had them, so shots at either are judged against what the client saw.

With **Precise Playback** off, the client runs its own copy of the animation and lands a few hundredths of a metre from the server's pose: fine for looks, not for hitboxes. A hitbox is only as accurate as the replication that places what the player aims at.

Add these in a real game

- **Walls.** Clip the shot first with `LagCompensationWalls.GetDistance` and pass the result as the query's range.
- **Immediate feedback.** The demo's marker waits a round trip for the verdict. Play your own effect at once and let the verdict confirm the damage.
- **Per-region damage.** Read a component on the hit hitbox's GameObject through `NetworkHitboxRoot.TryGetHitbox`; see Making an object hittable.
- **A rewind window that fits.** The demo keeps the default 500 milliseconds; see UnityLagCompensationManager.

How lag compensation works

> **Driving the core API directly?** See How lag compensation works.

A client draws the world a little behind the server, so when a player fires at a moving target, the server has already moved it on. Lag compensation lets the server judge the shot against the world as the shooter saw it.

Lag compensation is Pro. In a free build every name below still exists, so the same scripts compile, but nothing is recorded and every query finds nothing.

The server keeps its own hitbox history

The server never rewinds transforms, colliders or Unity's physics. It records **hitboxes** and tests shots against that record:

Record. A `NetworkHitboxRoot` on a networked object registers every `NetworkHitbox` under it. Once per tick the server records where each hitbox was, after animation has posed the bones, so an animated rig is recorded as it was drawn. See Making an object hittable.

Capture. On the frame the player fires, the client calls `CaptureShot`, which returns a `LagCompensatedShot`: the ray exactly as drawn, and a view of what was on screen. The client sends it to the server.

Judge. The server passes the shot to a query such as `TryRaycast`, which rebuilds every hitbox where the shooter saw it and tests the shot against those shapes. See Firing and judging a shot.

A hitbox is not a collider, so `Physics.Raycast` never sees the past. Only the lag-compensated queries do. Walls are not recorded either; see Clipping shots at walls.

The server handles these cases for you

- **Blending.** A view between two ticks rebuilds each hitbox between both ticks' poses, as the shooter's screen blended them.
- **Interval targets.** A target whose `NetworkTransform` position is sent on an interval is drawn later by clients, and the server judges it that much earlier.
- **Grouped objects.** A grouped object's hitboxes are recorded once, so a shot hits each hitbox once.
- **Stopped targets.** A target that despawns stays hittable until its last tick leaves the rewind window, so a shot fired just before it went away still lands.
- **The shooter.** A shot never hits an object its shooter controls.
- **Hosts.** A host's view is the present, so its own shots are judged against the present.

Lag Compensation

Lag compensation is a Pro feature. Pages in this category:

- How lag compensation works
- Making an object hittable
- Firing and judging a shot
- Clipping shots at walls
- UnityLagCompensationManager
- Worked example: the HitscanRewind demo

Making an object hittable

Driving the core API directly? See Registering hitboxes.

Add the two components

Add a `NetworkHitboxRoot` beside the object's `NetworkSystemObject`. It gathers every `NetworkHitbox` on its GameObject and children. Without a `NetworkSystemObject` on the same GameObject, it logs an error and disables itself.

Add a `NetworkHitbox` to each part a shot should hit, such as a body, a head or a limb. Each is a single sphere, box or capsule, or in 2D a circle, 2D box or 2D capsule. It is not a collider, and the physics engine never sees it.

Put each hitbox's GameObject on the layer your queries filter by. A hitbox's layer is its GameObject's layer.

To do step 2 in one click, use **Add Hitboxes From Colliders** on the root; see Check what will be recorded.

Add every hitbox to the prefab. The root gathers its hitboxes once, when the object wakes, inactive ones included, so a hitbox added at runtime is never recorded. Only the server records; on a client the root does nothing.

Choose where a hitbox takes its shape from

Setup Mode (`HitboxSetupMode`) has two values:

- **`Automatic`**, the default, copies the shape from a collider on the same GameObject when the hitbox wakes. It looks for a `SphereCollider`, `BoxCollider`, `CapsuleCollider`, `CircleCollider2D`, `BoxCollider2D` or `CapsuleCollider2D`, in that order, trigger or not. It writes what it copied into its own settings, so switching to `Manual` starts from the collider's values.
- **`Manual`** uses the shape typed into the hitbox, for a part with no collider or a hitbox that should differ from its collider.

A `MeshCollider`, a `PolygonCollider2D`, or a `BoxCollider2D` with an edge radius cannot be tested exactly, so it is refused with an error rather than approximated. An automatic hitbox with nothing it can copy shows a warning in its inspector and is left out of the root.

A manual hitbox has these fields:

- **Shape** (`HitboxShape`).
- **Center**, in local space before scale.
- **Radius**, **Size** and **Height**, as the shape uses them. **Height** is the full height, both rounded ends included.
- **Direction**, for a capsule: a `HitboxAxis` of `X`, `Y` (the default) or `Z`. Its values match `CapsuleCollider.direction`, so a plain cast converts one to the other. A 2D capsule offers only `X` (horizontal) and `Y` (vertical).

Layers and per-region rules

A query's layer mask chooses which hitbox layers a shot can hit. For per-region rules such as a headshot multiplier, put your own component on the hitbox's GameObject and read it from the hit, rather than spending a layer on each region:

```
if (NetworkHitboxRoot.TryGetHitbox(lagCompensatedHit, out NetworkHitbox networkHitbox) &&
networkHitbox.TryGetComponent(out DamageZone damageZone))
    damage *= damageZone.Multiplier;
```

`NetworkHitboxRoot.TryGetHitbox(in LagCompensatedHit, out NetworkHitbox)` finds the hitbox a hit names. If the target's system has stopped since the judged tick, the hit is still reported but names no system, so `TryGetHitbox` returns `false`; `lagCompensatedHit.SystemId` still names the target. For an index you already hold, a root also has an instance `TryGetHitbox(int hitboxIndex, out NetworkHitbox)`.

Scale and offsets follow Unity's colliders

A 3D hitbox scales by `Transform.lossyScale` and places its **Center** through `Transform.TransformPoint`, as Unity's colliders do, so a hitbox and its collider agree at every depth. Scale is recorded every tick, so an object that grows or shrinks is judged at the size it had on the viewed tick. Neither a collider nor a hitbox represents exactly a non-uniformly scaled parent over a rotated child.

2D hitboxes lie in the plane Z = 0

A `Circle`, `Box2D` or `Capsule2D` hitbox is recorded in the plane Z = 0, whatever depth its object sits at, so sorting depth never moves it out of a shot. It follows its transform the way Unity's 2D colliders do:

- A half turn about Y, such as a sprite flip, mirrors it.
- A tilt about its own X or Y axis foreshortens a box or capsule.
- A circle stays round and takes the larger of its world X and Y scales.
- A tilt about any other axis makes the collider a parallelogram, which the hitbox follows as the rectangle along the flattened X axis.

2D hitboxes are judged only by the 2D queries, and 3D hitboxes only by the 3D ones, unless **2D Queries Hit 3D Hitboxes** is ticked on the lag compensation manager.

Check what will be recorded

A selected `NetworkHitbox` draws the exact shape the server will record, in orange, so you can compare it with the collider's outline. A selected `NetworkHitboxRoot` draws every hitbox it gathers, inactive ones included.

The `NetworkHitbox` inspector shows its setup mode, then either the shape fields or the shape copied from the collider. It warns when it finds no collider it can copy, and when no `NetworkHitboxRoot` is on its GameObject or above it, since such a hitbox is never recorded.

In edit mode, the `NetworkHitboxRoot` inspector offers:

- **Add Hitboxes From Colliders**, also in the root's context menu. It adds an automatic `NetworkHitbox` beside every solid collider under the root that has none yet, inactive GameObjects included. It skips triggers, which a shot passes through; colliders whose GameObject already has a hitbox, since a GameObject holds one; colliders no hitbox can copy; and any collider that is not the one an automatic hitbox on that GameObject would copy first. The Console lists what was added and what was skipped and why, and one undo takes back a whole click.
- **The hitbox table**, listing every hitbox in the order the root gathers them, with its GameObject, shape, layer, 2D or 3D, and whether it is usable. The # column is the `HitboxIndex` a hit reports. A hitbox with no usable shape takes no index, so every hitbox after it reports an index one lower than its place in the list. Click a name to find it, and double-click to select it.

- **Warnings** for a root that would record nothing: no `NetworkSystemObject` on its GameObject, no `NetworkHitbox` found, no hitbox with a usable shape, or more than 64 usable hitboxes, the most one root can hold. It also lists unusable hitboxes, and notes a root that mixes 2D and 3D hitboxes.

To log the same errors for every prefab, open the Nucleus window (**Tools > Nucleus**), select the **Build** tab, and click **Validate** next to **Validate Network Prefabs**.

While playing, the root inspector shows how many hitboxes were gathered and whether the server is recording the root (`IsRecording`).

Make an object unhittable for a while

A hitbox that is disabled, or whose GameObject is inactive, on a given tick is recorded but never hit on that tick. To make the whole object unhittable, for a downed, spawn-protected or ragdolled player, disable the root from code on the server:

```
networkHitboxRoot.enabled = false;
```

Neither component shows an enable checkbox in the inspector. A disabled root keeps recording, so a shot judged at a tick it was disabled on misses it however late the shot arrives. It is hittable again from the first tick recorded after `enabled = true`. Its colliders stay in the scene throughout, and `LagCompensationWalls` passes over them either way.

UnityLagCompensationManager

Driving the core API directly? See The rewind window and the view clamp.

`UnityLagCompensationManager` sets lag compensation and spawn compensation. Both are Pro; in a free build every value is stored and never read.

A scene without this component gets one automatically, so add it yourself only to change the defaults. Its values are applied when the session starts, and the fields are disabled during play. To change a value after that, set it from code on `NucleusLagCompensationManager`, which is the same object as `NucleusUnity.BoundCoreManager.LagCompensationManager`.

Set the rewind window and query rules

Inspector field	Engine setting	Default	What it does
Rewind Ceiling (ms)	`MaximumRewindCeilingMilliseconds`	500	The longest rewind window the server honours. **Max Rewind (ms)** is held to it. The slider runs to 2000.
Max Rewind (ms)	`MaximumRewindMilliseconds`	500	How far back hitbox poses are kept and a shot may be judged. Read when an object's hitboxes register, so it applies to objects spawned after it changes. The slider stops at **Rewind Ceiling (ms)**.
2D Queries Hit 3D Hitboxes	`Can2DQueriesHit3DHitboxes`	off	Lets 2D queries also hit 3D hitboxes, for a game that mixes the two. A 2D ray and `OverlapBox2D` touch a 3D hitbox where the plane Z = 0 cuts it, and a circle, a 2D capsule and a circle cast reach it as a ball or capsule of their radius lying in the plane. 3D queries never hit 2D hitboxes either way.

Queries Start In Hitboxes	`CanQueriesStartInHitboxes`	on	Lets a ray or cast hit a hitbox it starts inside, at distance zero, the way Unity's 2D queries hit a collider they start in. Turn it off when a muzzle can sit inside a hitbox the shot should pass through, such as another body part of a large creature; that hitbox is then ignored and the query goes on to what lies beyond. Overlaps are not affected.

Cap spawn compensation

Inspector field	Engine setting	Default	What it does
Max Spawn Compensation (ms)	`MaximumSpawnCompensationMilliseconds`	150	The most a freshly spawned object, such as a projectile, may be caught up through the latency it lost, counted across every hop it travelled. It also bounds how long a catch-up runs. The slider runs from 0 to the engine ceiling, 500, which the runtime clamps to.

A client watching somebody else's shot is owed the time the server already spent plus what the delivery cost it, and the two together are held to this one figure. On a local link the measured latency never asks for the whole of it, so raise it to watch the catch-up. See Spawn compensation.

The view clamp violation

A shooter whose view is older than **Max Rewind (ms)**, or newer than anything the server recorded, is reporting a moment the server could never have shown it. The query clamps the view into range and still judges the shot, so a player on a very slow link still gets results. It also raises `LagCompensationViewClampedViolation` against the shooter, carrying the `ReportedTick` and the `ClampedTick` the shot was judged at. The default action is `ViolationAction.Ignore`.

There is no inspector for violations. Subscribe from code in `Start`:

```
private CoreManager _coreManager;

private void Start()
{
    _coreManager = NucleusUnity.BoundCoreManager;
    _coreManager.ViolationManager.LagCompensationViewClampedViolationDetected += OnLagComp
ensationViewClamped;
}

private void OnDestroy()
{
    if (_coreManager is not null)
        _coreManager.ViolationManager.LagCompensationViewClampedViolationDetected -= OnLag
CompensationViewClamped;
}

private void OnLagCompensationViewClamped(in ViolationContext<LagCompensationViewClampedVi
olation> violationContext)
{
    LagCompensationViewClampedViolation violation = violationContext.Violation;

    Debug.Log($"Connection [{violationContext.Connection.Id}] reported tick [{violation.Re
portedTick}], judged at [{violation.ClampedTick}].");
}
```

A single clamp is ordinary on a link that spiked. A steady stream from one client is either a very slow link or a client asking to be judged against the oldest world it can get. To act on it, register a deciding handler; see Handling violations in Unity.

Registering Call Handlers Across a Component's Lifetime

Driving the core API directly? See Handler Registration and Scope.

Register call handlers through `NetworkSystem.RegisterRpcHandler<T0>`, not `RpcManager`, so a handler runs only for calls addressed to that system. The system then unregisters the handler when it despawns, registers it again if it respawns, and drops it when the instance returns to the pool, so `OnDestroy` never needs `UnregisterRpcHandler`.

Register in OnSystemLinked

In a script deriving from `NucleusBehaviour<TComponent0>`, register in `OnSystemLinked`:

```
public class Beacon : NucleusBehaviour<BeaconComponent>
{
    protected override void OnSystemLinked()
    {
        NetworkSystem.RegisterRpcHandler<BeaconStrikeRpc>(OnBeaconStruck);
    }

    protected override void OnSystemUnlinked()
    {
        // Reset per-life state.
    }
}
```

Do not register in `Awake`. `NetworkSystem` is null until the system links, and a pooled instance is reactivated without running `Awake` again. `OnSystemLinked` runs on the first spawn and on every respawn after a pool return.

Or subscribe to the marker's link events

A script that does not derive from `NucleusBehaviour` subscribes to the `NetworkSystemObject`'s events instead:

- `SystemsLinked` and `SystemsUnlinked` fire once for the object's whole set of systems. An ungrouped object fires on its first link; a grouped object fires when its linked count reaches the group's replicated member count. Use these for object-wide work.
- `SystemLinked` and `SystemUnlinked` fire once for each system as it links or unlinks, so several times on a `NetworkSystemGroup`.

If you subscribe to `SystemsLinked` after the object has already linked, your handler runs immediately. `SystemsUnlinked` does not do this: a late subscriber hears nothing until the next unlink.

```
private void Awake()
{
    _networkSystemObject = GetComponent<NetworkSystemObject>();

    // Runs OnSystemsLinked at once if the object already linked before this Awake ran.
    _networkSystemObject.SystemsLinked += OnSystemsLinked;
    _networkSystemObject.SystemsUnlinked += OnSystemsUnlinked;
}

private void OnSystemsLinked(NetworkSystemObject networkSystemObject)
{
    if (!networkSystemObject.TryGetFirstSystem(out NetworkSystem system))
        return;

    system.RegisterRpcHandler<BeaconStrikeRpc>(OnBeaconStruck);
}

private void OnSystemsUnlinked(NetworkSystemObject networkSystemObject)
{
    // Reset per-life state here; a pooled, reactivated object never runs Awake again.
}

private void OnDestroy()
{
    if (_networkSystemObject != null)
    {
        _networkSystemObject.SystemsLinked -= OnSystemsLinked;
        _networkSystemObject.SystemsUnlinked -= OnSystemsUnlinked;
    }
}
```

`TryGetFirstSystem` returns the object's primary system, the one to scope object-wide work against.

Unsubscribe from the marker events in `OnDestroy`, as above. This matters most for a component on a different GameObject that can outlive the marker.

Reach the manager from a plain MonoBehaviour

A script that is not a `NucleusBehaviour` gets the session through `NucleusUnity.BoundCoreManager`:

```
CoreManager coreManager = NucleusUnity.BoundCoreManager;

if (coreManager is null)
{
    Debug.LogError("No bound CoreManager; the scene needs a UnityCoreManager.");
    enabled = false;
    return;
}
```

It is `null` until the session starts and is cleared when it ends. It never carries a previous play session's manager into the next one, even with domain reload disabled.

Channels and What Is Guaranteed

Pick a channel for every send

Every message and call is sent on `Channel.Reliable` (ordered and reliable) or `Channel.Unreliable`, chosen at the send:

```
connection.SendMessage(Channel.Reliable, myMessage);
mySystem.TrySendRpc(RpcTarget.Observers, Channel.Unreliable, myRpc);
```

The transport alone provides reliability. `Reliable` guarantees order and delivery; `Unreliable` guarantees neither.

Calls and messages are not state

A call or a message arrives once or not at all. Nothing resends it, so the channel is your only reliability choice: if it must arrive, send it on `Channel.Reliable`. Sending on `Channel.Unreliable` means losing it now and then is fine.

One bad payload does not affect the others.

A large unreliable send stays unreliable

A message or call larger than the transport's maximum transmission unit keeps the channel you chose. A reliable one is always delivered in full.

A large unreliable payload arrives whole or not at all, and the larger it is, the more often it is lost. Send large data on `Channel.Reliable`.

Use a replicated member for anything a late joiner needs

Nothing records a message or call for a peer that starts observing afterward. If a value must be correct for a late joiner, put it in a replicated member. See State, a Message, or a Call? and How replicated state works.

Messaging and Remote Calls

Pages in this category:

- Channels and What Is Guaranteed
- Registering Call Handlers Across a Component's Lifetime
- Sending a Remote Call From a Component
- State, a Message, or a Call?
- When a Message or Call Reaches Its Handler
- Typed Messages in Unity
- Messages and Calls That Do Not Arrive

When a Message or Call Reaches Its Handler

Outbound messages and calls leave once per frame

Sends are queued and go out once per frame, at the `NetworkLoopSteps.LateSerialize` step, after its callbacks. A message or call sent from a `LateSerialize` callback or any earlier step leaves this frame. One sent from a `TickAdvance` or `LateVariableUpdate` callback leaves next frame. Messages and calls sent in the same frame leave together.

Inbound calls run on LateStateUpdate, after messages

Inbound calls are dispatched on `NetworkLoopSteps.LateStateUpdate`, after inbound messages and after the tick's state has applied. A call addressed to an object that spawned in the same tick finds its system already there.

Send order holds across types

A peer's messages reach their handlers in the order they were sent, whatever mix of message types they are.

Host loopback delivery is always Immediate

When one half of a host sends a message or call to its other half, `TransportManager.HostLoopbackDelivery` is `HostLoopbackDelivery.Immediate`: the receiving half's handlers run at the send site, before the send returns, on whatever loop step the sender was on. Setting `HostLoopbackDelivery.Deferred` from game code is refused with a logged reason.

A handler reached this way may send something back. If it starts a spawn while the sending step is `LateSerialize` or `TickAdvance`, after the tick's state was sent, the spawn waits until the tick advances, like any other late spawn.

A handler throw costs only its own payload

If a message or call handler throws, the exception is logged and that payload is dropped. Every other message or call in the same frame, from the same sender, still reaches its handler.

Typed Messages in Unity

Driving the core API directly? See Typed Messages.

Reach the message manager

`UnityMessageManager` has no inspector settings. Its `NucleusMessageManager` property is the core `MessageManager`. From any script, reach it through the bound session:

```
MessageManager messageManager = NucleusUnity.BoundCoreManager.MessageManager;
```

From a script on the manager's GameObject or a child of it, you can also get the component:

```
MessageManager messageManager = GetComponent<UnityMessageManager>().NucleusMessageManager;
```

Register and unregister a handler

A message handler is not tied to any GameObject. Nothing removes it when the object that registered it is destroyed, and a handler left registered keeps firing against a destroyed component. Pair `RegisterMessageHandler<T0>` with `UnregisterMessageHandler<T0>` yourself:

```
public class DamageAlertListener : MonoBehaviour
{
    private MessageManager _messageManager;

    private void Start()
    {
        _messageManager = NucleusUnity.BoundCoreManager.MessageManager;
        _messageManager.RegisterMessageHandler<DamageAlertMessage>(OnDamageAlert);
    }

    private void OnDestroy()
    {
        _messageManager?.UnregisterMessageHandler<DamageAlertMessage>(OnDamageAlert);
    }

    private void OnDamageAlert(in MessageContext messageContext, DamageAlertMessage messag
e)
    {
        // Handle the message.
    }
}
```

Cache the `MessageManager` in `Start` rather than resolving it in `OnDestroy`, because `NucleusUnity.BoundCoreManager` may already be null by then.

Send to the server from a client

`SendMessage<T0>` is a method on `Connection`. From a client, get the server connection with `TransportManager.TryGetServerConnection`:

```
if (NucleusUnity.BoundCoreManager.TransportManager.TryGetServerConnection(out Connection s
erverConnection))
    serverConnection.SendMessage(Channel.Reliable, new DamageAlertMessage());
```

To send from the server to one or many clients, see Sending a Message to Many Peers.

Keep the source generator running

A message type needs the Nucleus source generator to run over every assembly that declares or sends it, like any call payload. If `Nucleus.CodeAnalysis.SourceGenerators.dll` loses its `RoslynAnalyzer` label, the project still compiles but the message never sends. See Code Generation in Unity.

Messages and Calls That Do Not Arrive

The message hash could not be found for Type

`Connection.SendMessage<T0>` logs `The message hash could not be found for Type [...]` and returns without queuing anything when it cannot get a usable wire hash for the type. A type never needs registering before it is sent. The lookup fails in only two cases:

- **Another message type this peer has already used hashes to the same value.** This is the usual cause. The first time, a separate error names both types (see the next section). After that, every send of either type logs the error above.
- **The type has no full name.** That logs `The FullName for [...] could not be found.` first. An ordinary concrete message type always has one.

A host sending to its own other half is refused the same way.

To fix a collision, rename one of the two types or move it to another namespace. Registering a handler does not help, because the same lookup refuses the handler too.

Two types that share a wire hash are both refused

If two different message types, or two different call types, collide, neither can be sent or handled until you rename one of them.

A call for a system the receiver lacks is dropped silently

If the receiver does not hold the call's target system, the call is dropped with no log and no violation. An interest cull, a despawn and a spawn that has not arrived yet all look the same here.

A routed call (`RpcRoute.TargetRelay`) is also dropped silently if the connection it names has left or no longer observes the addressed system.

So when a call goes nowhere, check interest first: does the receiver still hold the system, and does the intended recipient still observe it? A data bug would produce a read error or a violation.

Nothing happens on the sender

`RpcSelfDelivery` defaults to `None`: the sender does not see its own call.

- `RpcSelfDelivery.Immediate` runs the sender's own handlers at the send site, before the call goes anywhere, even if the server later refuses it.
- `RpcSelfDelivery.OnDelivery` runs them only once the call is delivered, and not at all if it is refused. On the server's own send it behaves like `Immediate`.

Three gates can refuse a client's call

The server checks these in order:

`RpcAccess`. The sender must observe the system. Then `RpcSendAccess.AnyClient` admits any client, and otherwise only the system's controller is admitted. This applies to a host's own client half too. If the sender is not admitted, the server raises `RpcSendPermissionViolation` with the addressed `SystemId`. `CanLocalClientSendRpc` is a separate check on the sending side that lets a client queue a call at all when it is not the controller; it does not change the server's check.

A server handler returning `RpcRelayAction.Cancel`. The server runs every registered handler, and any single `Cancel` settles it. A handler that throws counts as `Cancel` and is logged, and the remaining

handlers still receive the call.

`MaximumInboundRpcsPerConnectionPerDrain` (default 64). Past this many calls from one connection in one drain, the server raises `RpcFloodViolation` once for that drain, with the received and allowed counts, and drops the rest of that drain's calls from the connection.

A host runs calls addressed to other players

A host never fires a call twice. What it does do: with `RpcSelfDelivery.Immediate` on an addressed send, the host's client half runs a copy of calls addressed to other players, not only those addressed to itself, because the sender's handlers run on the same process as the server that relays the call. Check the call's target, not whether it "happened twice".

A receiver with no handler skips the payload

A receiver with no handler for a message type skips that message silently. The sender needs no registration. If a message type seems to vanish, check that the receiving peer registered a handler for it.

Sending a Remote Call From a Component

Driving the core API directly? See System RPCs.

Send from the component's system

`TrySendRpc` is a method on `NetworkSystem`. A script deriving from `NucleusBehaviour<TComponent0>` has a `NetworkSystem` property, set while its required system is linked:

```
public class DemoScoreRpcSender : NucleusBehaviour<DemoScoreComponent>
{
    private void SendScoreRpc()
    {
        bool sent = NetworkSystem.TrySendRpc(RpcTarget.Server, Channel.Reliable, new AddSc
oreRpc { Amount = 1 });
    }
}
```

`Component`, the typed `NetworkComponent`, is for reading replicated values, not for sending calls.

Any other script gets a system from the object's `NetworkSystemObject`. `TryGetFirstSystem` returns the first-linked system, the one to send from when several systems share the object:

```
if (networkSystemObject.TryGetFirstSystem(out NetworkSystem networkSystem))
    networkSystem.TrySendRpc(RpcTarget.Server, Channel.Reliable, new AddScoreRpc { Amount
= 1 });
```

`TrySendRpc` returns `true` when the call was queued or invoked locally, not when it was delivered. `false` means nothing was sent, most often because this client lacks the access the target requires. Call it wherever the game decides to act: input handling, a UI callback, or an animation event.

The payload is a `[NetworkType]` struct implementing `IRpc`:

```
[NetworkType]
public struct AddScoreRpc : IRpc
{
    public int Amount;
}
```

If `Nucleus.CodeAnalysis.SourceGenerators.dll` is missing from `Assets/Nucleus` or has lost its `RoslynAnalyzer` label, the struct still builds but the call never sends. See Code Generation in Unity.

Guard the send

Use the `NucleusBehaviour` helpers instead of comparing connections or transport state yourself. Both return `false` before a system is linked, so they need no null check on `NetworkSystem`.

`IsController(ControllerType)` reports whether this peer controls the linked system. Input reaches every copy of an object, so `false` on the peers that do not control it is normal and logs nothing:

```
private void OnFireButton()
{
    if (!IsController(ControllerType.Client))
        return;

    NetworkSystem.TrySendRpc(RpcTarget.Server, Channel.Reliable, new AddScoreRpc { Amount
= 1 });
}
```

`IsStarted(Invoker)` reports whether a role is running, for code that cares about the role rather than control:

```
if (IsStarted(Invoker.Client))
    NetworkSystem.TrySendRpc(RpcTarget.Server, Channel.Reliable, new AddScoreRpc { Amount
= 1 });
```

Reach the RPC manager

There is no RPC manager component, and no call setting appears in an inspector. Targets, self-delivery and access are all set in C# on `RpcManager` or the system. Reach `RpcManager` as `CoreManager.RpcManager` inside a `NucleusBehaviour`, or `NucleusUnity.BoundCoreManager.RpcManager` anywhere else.

State, a Message, or a Call?

Each one carries something different

- A **replicated member** carries a current value. The server writes it, every observer's copy converges on it, and a late observer is brought up to date as soon as it starts observing.
- A **message** is sent to a peer's connection and forgotten. There is no acknowledgment and no record of it once the handler returns.
- A **call** is addressed to a `NetworkSystem`. The receiver resolves that system before any handler runs, and steps over a call for a system it does not have.

A late joiner sees state, never past messages or calls

A replicated member shows a new observer the value as of the last write. A message or call sent before a peer started observing is gone: neither `RpcManager` nor `MessageManager` keeps a backlog. If a peer needs to know something happened before it joined, keep that fact in a replicated member.

This usually decides it: does a peer that arrives later need to know this, or only the peers that saw it happen?

Choose by example

- **A score.** Every peer, including one that joins mid-match, needs the current value. Replicated member.
- **A door's open state.** A peer that walks up five minutes after the door opened needs to see it open. Replicated member.
- **A chat line.** Nobody who missed it needs it later, and it is not addressed to an object. Message.
- **A hit effect.** It plays once on the object it happened to, and a late joiner does not need it. Being addressed to a `NetworkSystem` makes it a call rather than a message.
- **A server answer to one player's request.** Addressed to that player's system, resolved once. A call sent with a routed `RpcRoute` rather than to every observer.

Driving a predicted body

> **Driving the core API directly?** See Locally predicted bodies.

This page covers a body the server controls and simulates while one client steers it, such as a player's own physics character. The steering client applies its own client state at once, while the server still owns where the body really is. For a body the client controls outright, use a validated behaviour instead; Predicting a body with a validated behaviour helps you choose.

Turn prediction on

On the peer steering the body, set `Convergence.IsLocallyPredicted` to `true`, and set it back to `false` the moment this peer stops driving the body:

```
networkRigidbody.Convergence.IsLocallyPredicted = true;
```

- The flag only has an effect on a peer that is not the body's controller. On the controller the body simulates freely and its result is sent.
- While it is set, the body only trims a position or rotation error once it passes its threshold, and never teleports.
- Every step still sets the body's velocity from its target. For your client state to show at once, assign `Convergence.TargetProvider` a provider whose target includes it, as the PhysicsBallMover demo does.
- A proxy left with the flag set has no client state of its own to trust.

Apply force on the network tick

Physics steps on the network tick, not on Unity's `FixedUpdate`, so force applied in `FixedUpdate` does nothing or fires at the wrong rate. Use one of these instead:

- Subscribe to `PhysicsSimulationDriver.WorldStepStarted`, raised before each world step:

```
driver.WorldStepStarted += (tick, stepDelta) =>
{
    rigidbody.AddForce(clientStateDirection * forceScale);
};
```

- Or override `OnEarlyFixedUpdate` on a `NucleusBehaviourBase`. It runs at the same point:

```
protected override void OnEarlyFixedUpdate(StepDelta stepDelta, SimulationContext simulati
onContext)
{
    rigidbody.AddForce(clientStateDirection * forceScale);
}
```

`stepDelta.FixedDelta` is the step's fixed timing. `SimulationContext` names the pass the step belongs to; see Replaying a correction in Unity.

Tune the correction

A predicted body ignores **Blend Per Tick** and uses **Prediction Correction Rate** (`Convergence.PredictionCorrectionRate`): the fraction of a position or rotation error closed per tick once past the threshold. Higher snaps back harder; lower corrects gently. `0` turns correction off entirely, and the body is then also exempt from teleporting, so a desync never closes. Leave some nonzero rate.

A healthy predicted body corrects rarely: only on a genuine divergence such as a hit, a collision the server resolved differently, or a dropped tick of client state. A good correction looks like a brief nudge that eases back in over a few ticks. If corrections look like a snap, the rate is too high, the threshold is too tight, or the divergence itself is worth chasing.

Choose how a remote-controlled proxy moves

A body a client controls outright (its `ControllerConnectionId` is set) is not predicted on other peers. It is a proxy driven by that client's state, and `RemoteExtrapolationEnabled` decides how it moves:

- `true` (the default): the proxy extrapolates forward from the last received state. It stays nearer present time but overshoots on abrupt changes: a stopping body slides on a little, then corrects. Prefer it for a fast body where responsiveness matters more than a rare overshoot.
- `false`: the proxy follows the received states as a kinematic body, always a step behind but smooth. Prefer it where overshoot looks worse than lag: dense contacts, tight geometry, or a body other players stand near.

Sending the client state itself is covered in Sending client state from Unity.

How physics replication works

Driving the core API directly? See How physics replication works.

Every peer keeps simulating the body

The peer that controls a body simulates it and sends the result each tick. Every other peer keeps simulating its own copy and steers it toward the server's latest state, carried forward to the present, instead of teleporting it there. Contacts, stacks and collisions involving the body stay real on every peer.

A replay rewinds only predicted bodies

When a client is corrected for an object it predicts, it replays every tick since the one that went wrong, and each replayed tick steps the whole physics world, 3D and 2D. Only a body predicted through a validated behaviour, such as `ValidatedRigidbody`, is put back to the server's values and stepped again. A `NetworkRigidbody` body is not rewound; its `ReplayHeldBody` holds it still for the replay. See NetworkRigidbody and Predicting a body with a validated behaviour.

Some things are not built

- Collision callbacks are not replay-correct.
- Judging a shot against where targets were is lag compensation (Pro). It keeps its own hitbox history and does not touch physics bodies.
- Catching a freshly spawned projectile up through the latency it lost is spawn compensation (Pro).

Physics Synchronization

Pages in this category:

- Physics execution modes
- NetworkRigidbody
- How physics replication works
- NetworkTransform or NetworkRigidbody
- Replicate a rigidbody
- Driving a predicted body
- Predicting a body with a validated behaviour
- NetworkNavMeshAgent
- PhysicsSimulationDriver
- Local physics scenes
- Physics troubleshooting

Local physics scenes

Give each scene its own physics world

With a local physics world per scene, a body only collides with bodies loaded into the same scene. Turn it on with either field:

- **Local Physics** on `UnitySceneLoader` (`UnitySceneLoader.LocalPhysicsEnabled`) loads each networked scene with `LocalPhysicsMode.Physics3D` instead of the shared world.
- **Auto Simulate Stacked Scenes** on `UnitySceneManager` sets the same field on the `UnitySceneLoader` the manager creates, so every stacked instance gets a local physics world.

Unity never steps a local physics scene on its own, so Nucleus adds a `PhysicsSimulationDriver` to each one as it loads and steps it on the tick. Bodies in a local physics scene still need manual interpolation.

Call NotifySceneChanged after moving a body

A `NetworkRigidbody` collides only within its own scene's physics world, so moving it to another scene changes what it can hit. `MoveGameObjectToScene` does not tell the body it moved:

Move the GameObject with `MoveGameObjectToScene`.

Call `NetworkRigidbody.NotifySceneChanged()` right after.

The body then simulates in its new scene from the next tick. Without the call, it keeps simulating in the scene it left. A spawn move calls it for you on every `NetworkRigidbody` under the moved system.

See also

- UnitySceneLoader component and UnitySceneManager component
- Opening and closing scene instances
- Moving and carrying objects between scenes

NetworkNavMeshAgent

NetworkNavMeshAgent sends the leg being walked, not the pose

`NetworkNavMeshAgent` replicates a `NavMeshAgent`. When a new leg begins, the controlling peer sends the point it is heading for, the point it set off from, and the agent's speed, angular speed and acceleration. Every other peer walks its own copy along that leg, turning toward its direction of travel, with its own `NavMeshAgent` switched off. An agent that holds its course sends nothing, however far it travels.

Use it instead of `NetworkTransform`, never alongside it: both would replicate the same motion.

Check the requirements

- Unity's AI module (`com.unity.modules.ai`) must be enabled, as it is in a new project. Without it the component is left out and the rest of the integration still compiles. The integration sets the `NUCLEUS_NAVMESH` define for you when the module is present.
- The controlling peer needs a baked NavMesh covering the agent's path, and so does any peer that may take control later. Followers need no NavMesh.
- The prefab's agent settings must match on every peer, since a follower walks from the replicated leg and its own local settings.

Set the inspector fields

Field	Default	Purpose
Point Change Tolerance	0.05	How far, in metres, the agent's steering target must move before it counts as a new leg. Too small a value sends a leg for every small nudge a path recalculation makes.
Resync Distance	0.5	How far, in metres, a follower may stand from a new leg's origin before it walks the gap off over the next moment. Below this it just retargets onto the new leg, so the error never shows.
Teleport Distance	6	How far, in metres, a follower may stand from a new leg's origin before it is placed there outright, such as after a warp or when it has fallen too far behind.
Rotation Enabled	on	Turns a follower toward its direction of travel at the replicated angular speed. Rotation is derived from the path, not sent.

Agent Management Enabled	on	Disables the attached `NavMeshAgent` on every peer except the controller, and enables it there. A follower's own agent must not run, or it fights the replicated leg.
Send Interval	`SendInterval.Normal`	Normal sends a leg on the tick it begins. A larger interval delays a leg by up to that interval.

A received spawn is always placed where the sender stood when it sent the spawn, moving at the sender's velocity, whatever these distances are.

Drive it from code

These act on the attached `NavMeshAgent` and work only on the peer that controls the object (`IsController(ControllerType.AnyController)`). A call from a follower is refused.

```
public bool SetDestination(Vector3 destination);
public void Stop();
public void Resume();
public bool Warp(Vector3 position);
```

The path is found on the controlling peer only, so other peers never learn the destination. A `Warp` reaches followers through the next leg, whose origin is the warped-to position, and follows the Resync Distance and Teleport Distance rules above.

Read where the agent is walking

```
public Vector3 NextPoint { get; }
public Vector3 PreviousPoint { get; }
public float FollowerSpeed { get; }
```

- `NextPoint` and `PreviousPoint` are the current leg's endpoints in world space.
- `FollowerSpeed` is the speed a follower is walking at right now. It ramps toward the replicated speed and drops through a turn. It reads zero on the controller, which walks its real agent.

NetworkRigidbody

Driving the core API directly? See PhysicsConvergence.

`NetworkRigidbody` keeps one `Rigidbody` in agreement across every peer. To set one up, see Replicate a rigidbody.

Read it from a script

Member	Type	Notes
`Convergence`	`PhysicsConvergence` (read-only)	The follower that steers the body. The inspector fields below write into `Convergence.Settings`.
`SmoothedVisual`	`Transform`	Optional child that holds the body's renderer, drawn smoothly between physics steps. `NetworkRigidbody` writes its world pose every frame, so no other script or animation may write that transform.
`RemoteExtrapolationEnabled`	`bool` (default `true`)	On, a remote-controlled proxy extrapolates forward from its last received state: more current, but it overshoots on sudden client state changes. Off, it interpolates between received states: smooth, never overshoots, always a step behind.
`Body`	`UnityPhysicsBody` (read-only)	The adapter around the attached `Rigidbody`.
`PhysicsComponent`	`NetworkPhysicsComponent` (read-only)	The replicated component this body sends from or follows.
`NetworkSystem`	`NetworkSystem` (read-only)	The system this body belongs to. Its controller is the peer whose simulation counts.

Set when a body corrects

A proxy is a body this peer only follows. A predicted body is one whose `Convergence.IsLocallyPredicted` is set; see Driving a predicted body.

Field	What it does
Position Threshold (metres)	The position error past which the body blends back toward where it should be. Below it, the body keeps its small offset unless the residual fields close it. Applies to proxies and predicted bodies.

Rotation Threshold (degrees)	The rotation error past which the body blends back. Applies to proxies and predicted bodies.
Blend Per Tick	The fraction of the remaining gap a proxy closes per tick while over a threshold. A predicted body ignores it.
Prediction Correction Rate (default `0.1`)	The fraction of the gap a predicted body closes per tick once past Position Threshold. Higher snaps back harder and sooner; lower lets the prediction ride looser.
Blend Mode	`ConvergenceBlendMode.Position` moves the body straight toward where it should be, so a correction into geometry can briefly push it in before it is pushed back out. `ConvergenceBlendMode.Velocity` corrects through the body's own motion, so contacts, stacks and other bodies are never disrupted. The two look the same in open space.
Residual Close Rate (per second)	How quickly an error below Position Threshold is closed. Higher closes a slow body's drift faster, lower is gentler at first contact, and `0` leaves the offset in place.
Residual Close Maximum Speed	The body speed above which a small error is left alone, so the close never fights a fresh contact.
Teleport Distance (metres)	Proxies only. The error past which the whole pose snaps onto the target instead of gliding; the smoothed visual jumps too. `0` disables it. A predicted body never teleports.
Maximum Projection Time (seconds)	Proxies only. How far a proxy may lead its last received state.
Gravity Enabled (default on)	On, a falling body is carried down under gravity while resting or supported bodies are left alone. Off, gravity is ignored.
Visual Smoothing (seconds, default `0`)	Smooths how `SmoothedVisual` moves, hiding the small jumps corrections cause. It needs a **Smoothed Visual** and changes only what is drawn, never the body or what it sends. Larger values smooth more but trail further; `0` turns it off.

A predicted body's counterpart to Maximum Projection Time is `Convergence.PredictionMaximumProjectionSeconds`, which has no inspector field. The PhysicsConvergence page covers every setting in full.

Keep the body still during a replay

Replay Hold Enabled (on by default) adds a `ReplayHeldBody` to the object when it has none. A

`NetworkRigidbody` body is never rewound, so while a correction replays, `ReplayHeldBody` makes it kinematic. Afterwards the body is dynamic again with its velocities, sleep state and collision detection mode as they were. A continuous collision mode is speculative for the length of the replay, since a kinematic body cannot use it.

- A held body keeps its colliders and stands where it is now, not where it was on the replayed tick. A predicted body that pushed it in the past meets something that does not give.
- Switching to kinematic and back resets the body's contacts, so a body resting on another may settle slightly differently after a replay.

Turn the field off only when you hold the body some other way. Add a `ReplayHeldBody` yourself to any other dynamic `Rigidbody` or `Rigidbody2D` a replay must not move; in play mode its inspector shows whether a replay is holding it. A body predicted through a `ValidatedRigidbody` needs none; see Predicting a body with a validated behaviour.

Call NotifySceneChanged after a scene move

Call `NotifySceneChanged()` after you move the body's GameObject to another scene, or the body keeps simulating in the scene it left. Spawn moves call it for you.

`PreSimulate(float stepDelta)` and `PostSimulate(uint tick)` run the body's work before and after each world step. The `PhysicsSimulationDriver` calls them; you will not normally call them yourself.

Physics execution modes

`UnityPhysicsManager` decides who steps Unity's physics: Nucleus once per network tick, or Unity on its own fixed step. The choice applies to the whole process and is read once when the session starts. Both fields are disabled in the inspector during play; stop play to change them.

Choose the mode in the inspector

Field	Default	Purpose
Execution Mode (`PhysicsExecutionMode`)	`NucleusPerTick`	Who steps physics. The read-only `ExecutionMode` property holds the active mode.
Match Fixed Delta Time To Tick Rate	off	Under `UnityRun` only, sets `Time.fixedDeltaTime` to one over the tick rate so Unity's fixed step matches the tick. The previous value is restored when the manager is destroyed. Ignored under `NucleusPerTick`.

NucleusPerTick steps every world on the tick

Nucleus sets `UnityEngine.Physics.simulationMode` to `SimulationMode.Script` and steps every physics world once per network tick. Projected physics on the default world requires this mode. The network loop runs on unscaled time, so physics keeps stepping at the tick rate while `Time.timeScale` is zero or slowed.

UnityRun leaves the default world to Unity

The default world stays on Unity's own fixed step and carries no projected physics at all. `Rigidbody.Interpolate` works on it again. Projected physics still works in the local-physics worlds of stacked scenes, each stepped by its own `PhysicsSimulationDriver`.

Which scene steps which world is set per scene by the driver; see PhysicsSimulationDriver.

Check which mode is live

- `UnityEngine.Physics.simulationMode` reads `Script` under `NucleusPerTick` and stays `FixedUpdate` under `UnityRun`.
- `Time.fixedDeltaTime` equals one over the tick rate only under `UnityRun` with the match field on.

PhysicsSimulationDriver

PhysicsSimulationDriver steps one scene's physics on the network tick

A scene with a `PhysicsSimulationDriver` steps its bodies on `NetworkLoopSteps.EarlyFixedUpdate` and its physics world on `NetworkLoopSteps.LateFixedUpdate`, instead of on Unity's `FixedUpdate`. You rarely add one: under the default execution mode Nucleus adds a driver for the default physics world, and a local-physics scene gets its own when it loads.

- Each scene is stepped by exactly one driver.
- Scenes that share a physics world (every scene not loaded with a local physics world) are stepped once per tick between them, however many drivers point at that world.
- A driver needs a `UnityCoreManager` in the scene. See Physics troubleshooting for the error when it has none.

Subscribe to the driver's events

Event	Signature	Fires
`WorldStepStarted`	`WorldStepStartedHandler(uint tick, float stepDelta)`	Before each world step. Apply per-step forces from client state here.
`BodyRegistered`	`BodyRegisteredHandler(NetworkRigidbody networkRigidbody)`	When a body joins the driver's step set.
`BodyDeregistered`	`BodyDeregisteredHandler(NetworkRigidbody networkRigidbody)`	When a body leaves the driver's step set.

```
driver.WorldStepStarted += (tick, stepDelta) =>
{
    // Apply per-step forces from client state here.
};
```

To follow the live body set, subscribe to `BodyRegistered` and `BodyDeregistered`, then read `Bodies` to pick up the bodies already present; otherwise you miss every body registered before you subscribed.

Read the driver's state

- `CurrentTick` (`uint`): the tick being simulated.
- `SubtickFraction` (`float`): how far the render frame has progressed through the current tick, for placing smoothed visuals between the last two stepped poses.
- `Bodies` (`IReadOnlyList<NetworkRigidbody>`): the bodies this driver steps, in registration order.
- `TryGetSnapshotLocalTick(uint captureTick, out uint snapshotLocalTick)`: maps a received capture tick into local ticks. Returns `false` when the capture tick carried no attribution or nothing mapped.

Find the driver for a scene

`ForScene(Scene scene)` returns the driver stepping a scene's physics, or `null` when nothing steps it and the scene runs on Unity's own physics timing. Call it once and keep the result rather than calling it every frame; a later deregistration then reaches the same driver.

```
PhysicsSimulationDriver driver = PhysicsSimulationDriver.ForScene(gameObject.scene);
```

When the driver may not exist yet, for example because it is added at runtime, use `RegisterForScene` and `UnregisterForScene`. The callback fires at once if a driver already steps the scene.

```
PhysicsSimulationDriver.RegisterForScene(gameObject.scene, OnDriverReady);

void OnDriverReady(PhysicsSimulationDriver driver) { /* ... */ }

// later
PhysicsSimulationDriver.UnregisterForScene(gameObject.scene, OnDriverReady);
```

The callback type is `DriverReadyHandler(PhysicsSimulationDriver physicsSimulationDriver)`.

Register and deregister bodies

`Register(NetworkRigidbody networkRigidbody)` and `Deregister(NetworkRigidbody networkRigidbody)` add a body to the step set and remove it. Calling `Register` twice for one body is harmless.

Physics troubleshooting

Nothing steps at all

A `UnityPhysicsManager` is always present, so a missing manager is never the cause.

- **Enabling a driver throws "No CoreManager is bound, so there is no physics to register with."** A `PhysicsSimulationDriver` needs a bound `CoreManager` when it is enabled. Add a `UnityCoreManager` to the scene.
- **A scene has its own local physics world and no driver.** Nothing steps that world at all. Add a `PhysicsSimulationDriver` to the scene.
- **The execution mode is `PhysicsExecutionMode.UnityRun`.** The default physics world is then stepped by Unity's own `FixedUpdate`, not the network tick. This is expected; see Physics execution modes.

A body jitters or trails

- **Visual Smoothing is set but no Smoothed Visual is assigned.** Visual Smoothing has nothing to act on. Assign a child of the body that holds its renderer, such as a child named `Visual`, and let no other script or animation write that child's transform.
- **Blend Mode is Position on a body in contact.** On a body resting in a stack or on the ground, `ConvergenceBlendMode.Position` fights the contacts and shows as jitter. Set **Blend Mode** to `ConvergenceBlendMode.Velocity`, which corrects through the body's own motion.
- **Residual Close Maximum Speed is too low.** A moving body stays above it, so its small error is never closed and shows as a small, persistent trail. Raise it above the body's ordinary travel speed.

The small-error close (**Residual Close Rate**) pauses above **Residual Close Maximum Speed** on purpose, so it never fights a body that was just bumped. It resumes once the body slows down.

A body snaps

The body's whole pose snaps onto the target when its error passes **Teleport Distance**. That is expected on a respawn, on a proxy that lost and then recovered its received state, and on any other large miss. If bodies snap on ordinary corrections, raise **Teleport Distance**, or set it to `0` to turn teleporting off.

A body steps the wrong world after a scene move

A scene move, such as `MoveGameObjectToScene`, does not tell `NetworkRigidbody` that the body moved, so it keeps simulating in the scene it left. Call `NetworkRigidbody.NotifySceneChanged()` right after the move.

A pooled or spawned instance often lands in the active scene rather than the scene it belongs to. It still finds the driver that steps its physics world, because a body looks up the driver for its own scene first and then for its physics world. If a body is stepped by the wrong driver, check which scene it was actually instantiated or moved into, and call `NotifySceneChanged()` if it moved after `OnEnable`.

A second driver on a scene does nothing

A second driver on a scene that already has one logs *"Scene [...] is already stepped by the driver on [...]; the one on [...] does nothing. A scene is stepped by exactly one driver."* The first driver keeps stepping the scene. Remove the second one.

NetworkTransform and NetworkRigidbody fight

Both move the same transform from replicated state. Use one or the other for an object's pose; see NetworkTransform or NetworkRigidbody.

Read bandwidth and round-trip time

To judge whether a correction is worth its cost, see Reading network statistics in Unity for bandwidth and Round-trip time, jitter and packet loss for the link figures.

Predicting a body with a validated behaviour

> **Driving the core API directly?** See Predicted members and prediction history and Reconciliation and replay.

A validated behaviour lets the client that controls an object predict a Unity `Rigidbody`, transform or `CharacterController` from its own client state, while the server runs the same client state and checks the result. When the two disagree, the client is corrected and replays every tick since, physics included. You add the behaviour and write only the client state and the forces.

The `BallRacer` demo under `Demos/Tutorials` is the worked example: a ball a client steers left and right, with a local cheat key the server catches.

Choose a validated behaviour when the client controls the body

- **A validated behaviour** suits a body the client controls outright, such as a player's own ball or vehicle. The server never takes the client's result, so it resists cheating with no extra code. Each correction costs a replay of the physics world, and physics is not exactly the same on every machine, so a rare correction is normal.
- **`NetworkRigidbody` with `Convergence.IsLocallyPredicted`** suits a body the server controls and simulates while one client steers it. The client never replays; its body eases toward the server's state over a few ticks. See Driving a predicted body.

Choose a validated behaviour for competitive play where the player's own body must match the server's. Choose convergence when you would rather never pay for a replay and a softer correction is fine.

Add one validated behaviour per Unity component

Each validated behaviour keeps the state of one Unity component in `MemberRole.Validate` members and declares a predicted system of its own for them, so your script does not list it in its composition.

The behaviours live in the `Nucleus.Integrations.Unity.Prediction` namespace:

Behaviour	What it keeps	Notes
`ValidatedTransform`	Position, rotation, scale, active state and layer.	**Transform Space** is World (the default) or Local; Local suits an object carried by a parent that is replicated itself. **Active State Apply Enabled** (off by default) applies the active state to the GameObject.
`ValidatedRigidbody`	Everything `ValidatedTransform` keeps, plus the velocities and whether the body is asleep.	Add `ValidatedRigidbodyAdvanced` beside it to also keep kinematic, collision detection, constraints and center of mass.
`ValidatedRigidbody2D`	The same for a `Rigidbody2D`.	`ValidatedRigidbody2DAdvanced` is the 2D version of the advanced settings.

`ValidatedCharacterController`	Everything `ValidatedTransform` keeps, plus whether the controller is enabled.	Add `ValidatedCharacterControllerAdvanced` for height, center and radius. A replay does not restore the controller's velocity, grounded flag or collision flags, so keep your own velocity and grounded state as members.
`ValidatedConstantForce`, `ValidatedConstantForce2D`	The force, relative force, torque and relative torque of a `ConstantForce`.	Add it beside the pushed body's `ValidatedRigidbody` or `ValidatedRigidbody2D`.
`ReconciledAnimator`	Every animator parameter and layer weight, on this peer only.	It is a `NetworkAnimator` that also puts back the parameters of the restored tick before a replay, so a mispredicted trigger or bool does not stick. Use it instead of `NetworkAnimator` on a predicted object. A client-authoritative object keeps the plain `NetworkAnimator`.
`ValidatedAnimator`	Up to 16 float, 16 int, 64 bool and 16 trigger parameters, 8 layer weights and the speed.	The server checks them, so use it when a parameter decides gameplay. Set parameters on the animator as usual and fire triggers through `ValidatedAnimator.SetTrigger`. Use it instead of `ReconciledAnimator` or `NetworkAnimator`, not beside them.
`ValidatedAnimatorRootMotion`	The base layer's playback.	It needs a `ReconciledAnimator` or a `ValidatedAnimator`. Use it when root motion decides where a character goes. It is the only one that declares no system of its own: list `ValidatedAnimatorRootMotionComponent` in the composition of the behaviour that sets the animator's parameters. It costs a full animator step for every replayed tick.

- Add only one of `ValidatedTransform`, `ValidatedRigidbody` or `ValidatedCharacterController` to an object. The body and controller versions already keep the transform's values.

- Never put a validated behaviour on an object that also has a `NetworkRigidbody` or a `NetworkTransform`, because both would write the same body.

Write the client state and push the body

Put the client state in a component of `MemberRole.Predict` members:

```
using Nucleus.Components;

namespace MyGame
{
    public partial class SteerComponent : NetworkComponent
    {
        // -1 left, 0 straight, 1 right. Whole steps, so the server simulates exactly the
steer the client did.
        public readonly NetworkMember<sbyte> Steer = new(role: MemberRole.Predict);
    }
}
```

Push the body from `OnEarlyFixedUpdate`. Physics steps after every early fixed update, live or replayed, and the `ValidatedRigidbody` captures the result on the late one:

```
using Nucleus.Integrations.Unity.Systems;
using Nucleus.Managers.NetworkLoop;
using Nucleus.Systems;
using UnityEngine;
using UnityEngine.InputSystem;

namespace MyGame
{
    // The prefab also carries a NetworkSystemObject and a ValidatedRigidbody. This script
 declares only the client state.
    [RequireComponent(typeof(Rigidbody))]
    public class BallSteering : NucleusBehaviour<SteerComponent>
    {
        [SerializeField] private float _steerAcceleration = 20f;
        [SerializeField] private float _sidewaysGrip = 3f;

        private Rigidbody _rigidbody;
        private sbyte _intendedSteer;

        protected override void Awake()
        {
            _rigidbody = GetComponent<Rigidbody>();

            base.Awake();
        }

        protected override void OnEarlyFixedUpdate(StepDelta stepDelta, SimulationContext
simulationContext)
        {
            // A client watching a ball it does not predict has nothing to simulate.
            if (simulationContext.Type is SimulationType.None)
                return;

            // The server's step for a tick whose client state never arrived: keep steerin
g the way the client last asked.
            if (stepDelta.IsClientStateMissing)
                Component.TryGetLastState(isEveryMemberCarried: true);

            // Only the controller's live tick writes client state. A replayed tick reads
the steer recorded for it.
            if (simulationContext.Type is SimulationType.Forward && IsController(Controlle
rType.AnyController))
                Component.Steer.Value = _intendedSteer;

            Vector3 velocity = _rigidbody.linearVelocity;
            float sidewaysAcceleration = Component.Steer.Value * _steerAcceleration - velo
city.x * _sidewaysGrip;
            _rigidbody.AddForce(new Vector3(sidewaysAcceleration, 0f, 0f), ForceMode.Accel
eration);
        }

        private void Update()
        {
            if (Component is null || !IsController(ControllerType.AnyController))
                return;

            // Replace with your project's input system.
```

```
            Keyboard keyboard = Keyboard.current;

            if (keyboard is null)
                return;

            _intendedSteer = (sbyte)((keyboard.dKey.isPressed ? 1 : 0) - (keyboard.aKey.is
Pressed ? 1 : 0));
        }
    }
}
```

Give the client control of the object, for example with `NetworkPlayerSpawner`.

Test with a dedicated server and a separate client. A host is the server, so its own ball is never corrected.

Follow the same rules as any predicted step:

- **Apply forces only in `OnEarlyFixedUpdate`.** Physics steps on the network tick, so a force applied in `FixedUpdate` runs at the wrong rate and is never replayed.
- **Read only members, the body and constants.** The step must compute the same thing on the client and the server for the same tick.
- **Play sounds and effects only on `SimulationType.Forward`.** A replay runs the step again for every tick since the one that went wrong.

Hold every other body still during a replay

A replay needs the default **NucleusPerTick** execution mode (see Physics execution modes). Every dynamic body that is not being replayed must be held still during a replay:

- A validated body that is not being replayed holds itself.
- A `NetworkRigidbody` body adds a `ReplayHeldBody` by default. See NetworkRigidbody.
- Any other dynamic body a replay must not move needs a `ReplayHeldBody` of its own.

A held body is not rewound, so a predicted body that pushed it in the past can meet it in a different place.

Set how often results are sent

Every validated behaviour has a **Send Interval** field. Set it before play; it is read when the behaviour links.

- **Normal** (default) sends every tick. It is the best choice for competitive games.
- **Short** sends less often, saving bandwidth and CPU at the cost of slightly more corrections.
- **Long** sends least often, for casual or large-scale games.

Short and Long only pace in Pro. A paced interval still sends on the tick the object's client state changes. If another script on the object has client state that never moves the body, such as a weapon's fire button, list it under **Ignored Behaviours** so its changes no longer send the body's results early.

Let other clients predict the body too

Tick **Predicted By Observers** on the object's `NetworkSystemObject` and observing clients predict it as its controller does, the validated system included. Use it for bodies that players collide with or push, including a server-owned body nobody controls, such as a ball every player can hit. The simulation must call `TryGetLastState` when `stepDelta.IsClientStateMissing` is true, since an observer never has the controller's client state in time. See Prediction settings on the Unity manager.

Draw the body through a GraphicalSmoother

The body moves once per tick, and a correction moves it again. Put the renderer on a child, add a `GraphicalSmoother` to the body and assign the child as its **Child Graphic**, so the drawn ball glides between ticks and across corrections while the body and its colliders stay exact. See Smoothing a predicted object with GraphicalSmoother.

Expect a rare correction

The validated behaviours give their position, rotation and velocities a `resultTolerance`: a predicted value that close to the server's does not count as wrong. The server can only measure that distance while the controller's **Send Result Values** is on, which is the default; with it off, results are compared exactly and corrections are more frequent.

Watch `NetworkSystem.ReconcileCount` on the client and `Connection.ResultMismatchCount` on the server while you play with some latency. A correction now and then is normal. A count that climbs every tick means the client and the server step differently; see Diagnosing desync.

Replicate a rigidbody

Driving the core API directly? See Replicate a physics body.

Add NetworkRigidbody

Add `NetworkRigidbody` to the GameObject. Unity also adds a `Rigidbody` and a `NetworkSystemObject` if the object lacks them.

Optional: put the renderer on a child, such as one named `Visual`, and assign it to **Smoothed Visual** so the body draws smoothly between physics steps. Set **Visual Smoothing** if corrections show; leave it at `0` on a body that must not trail, such as a fast-falling object. See NetworkRigidbody.

Do not add a `NetworkTransform` to the same object. See NetworkTransform or NetworkRigidbody.

Under the default execution mode, `PhysicsExecutionMode.NucleusPerTick`, you never rent a system or add a physics driver yourself. Under `PhysicsExecutionMode.UnityRun` the default world is left to Unity and projected physics is unavailable there; see Physics execution modes.

Only the controlling peer simulates freely

The peer where `NetworkSystem.IsController(ControllerType.AnyController)` is true steps the `Rigidbody` normally and sends the result each tick.

On every other peer, the body appears where the server has it, then follows the server's body while it keeps simulating and colliding. The exception is a body a remote client controls with `RemoteExtrapolationEnabled` off, which follows the received states as a kinematic body; see Driving a predicted body.

Check it across two editors

Run two editors.

On the controlling peer, push the `Rigidbody` or let it fall.

On the second editor, the same `Rigidbody` converges toward that motion tick by tick rather than teleporting.

Until the first state arrives, the second editor's copy sits at its prefab pose.

NetworkTransform or NetworkRigidbody

Choose by what a proxy has to do

`NetworkTransform` makes every other peer follow the pose: it applies the received position, rotation and scale one tick behind, and the proxy has no physical motion of its own. `NetworkRigidbody` keeps every peer simulating: each peer's `Rigidbody` steps with real collision response while it is steered toward the server's state.

Use `NetworkTransform` when a proxy only has to look right. It is cheaper and never fights local physics. Use `NetworkRigidbody` when a proxy has to behave right, so other bodies can bump it, land on it or collide with it on every peer.

Object	Use
Player capsules, doors	`NetworkTransform`. Proxies don't need to be bumpable, and following the pose is cheap and smooth.
Thrown and bumped bodies, such as a grenade or a kicked prop	`NetworkRigidbody`, so they bounce off geometry the same way for everyone watching.
A body its controlling client predicts	Neither. Use a `ValidatedRigidbody`; see [Predicting a body with a validated behaviour](./predicting-a-body-with-a-validated-behaviour.md).
Debris nobody steers	`NetworkRigidbody`, or no replication at all if it is not gameplay-relevant.
Agents that walk a path	Neither. Use [NetworkNavMeshAgent](./network-nav-mesh-agent.md), which sends the leg being walked.

Never put both on one object

`NetworkTransform`'s **Kinematic Management** (on by default) holds an attached `Rigidbody` kinematic on every peer that isn't the controller, including the server while a client controls the object. `NetworkRigidbody` never moves a kinematic body, so on every proxy the body sits where `NetworkTransform` last put it and never collides correctly. Replicate an object's pose through one component only.

Compare setup and sending cost

`NetworkTransform` needs no physics setup. `NetworkRigidbody` needs a `Rigidbody`, and Nucleus adds the physics driver it runs under; see Replicate a rigidbody.

Both send through the same per-member `TransmissionMode` choice, `Divine` or `Interval`, with `SendInterval` under `Interval`. `Divine` can go quiet for stretches where `Interval` would still send. It is Pro: a free build sends a `Divine` member as `Interval`. Pro also compresses values more tightly than Free. See How Often a Member Is Sent.

Addressables-delivered scenes

Each `NetworkSceneManifest.SceneEntry` in the network scene manifest has a `SceneDeliveryMode`, taken from the project:

- `SceneDeliveryMode.BuildSettings`: the scene ships in the player and is opened by path. Every scene not marked Addressable uses this.
- `SceneDeliveryMode.Addressables`: the scene is opened through Addressables, which fetches whatever carries it.

Nothing else about a scene's setup differs. An Addressables scene loads additively like any other, and the loader's **Local Physics** setting applies the same way. Loading a scene through Addressables works in any edition; only fetching content from an asset bundle needs Pro.

Deliver a scene through Addressables

Install `com.unity.addressables` 1.19 or newer. The integration then sets the `NUCLEUS_ADDRESSABLES` define for you; without it, the Addressables path in `UnitySceneLoader` is compiled out.

Mark the scene Addressable, in whatever group and address your project uses. Keep the group's **Internal Asset Naming Mode** at Full Path (see below).

Rebuild the manifest. It rebuilds itself when Addressables entries change; to run it yourself, open the Nucleus window (**Tools > Nucleus**), select the **Build** tab and click **Rebuild** next to **Rebuild Network Scene Manifest**. The scene's entry records `SceneDeliveryMode.Addressables`.

Do not put scenes in asset bundles

The manifest rebuild always gives a scene the base build's bundle id (`Connection.BaseBuildBundleId`), so a scene entry's `RequiredBundleId` never points at an asset bundle. If a networked scene has any asset bundle name, including the same one as a `NetworkPrefabCollection`, the rebuild logs an error and maps the scene as base build anyway, so it still loads from the player. Scenes that are neither in the build nor Addressable are not networked and are ignored. Use Addressables to stream a scene's content ahead of the load.

A failed Addressables load does not throw

A failed Addressables load does not throw or fault the task: an unresolvable key returns a handle that is already complete with a failed status. `UnitySceneLoader` checks that status and logs an error telling you to check that the scene is marked Addressable and its content is built. A loader subclass that calls Addressables itself must check the status the same way, or it will treat a failed load as a success.

The loader also checks that the loaded scene's path matches the entry's `ScenePath`. A group set to GUID or Dynamic naming fails this check, with an error telling you to set the group's Internal Asset Naming Mode to Full Path.

Content bundles in Unity

> **Driving the core API directly?** See The content bundle protocol.

Content bundles ship networked prefabs outside the base build, as asset bundles loaded into a running game. The server withholds spawns of a bundle's prefabs from a client until that client has loaded the bundle.

Content bundles need Pro. A Free build has no `CoreManager.BundleManager`, and the runtime loaders (`NetworkBundleLoader`, `UnityAssetBundleLoader`) exist only to register with it.

Shard your prefabs

A shard is one `NetworkPrefabCollection` asset. Unity's asset bundle assignment decides what it holds: the collection assigned to asset bundle `x` owns every networked prefab assigned to `x`, and the collection assigned to no bundle owns the prefabs assigned to none, which is the base build.

Each collection has:

- `BundleAssemblyName`: the simple name of the assembly whose `[assembly: NetworkBundle(n)]` attribute supplies the bundle id. Empty means the base build, bundle `0`.
- `PrefabBundleId`: the bundle id this shard's prefabs register under. It is stamped from `BundleAssemblyName` when the collection is rebuilt, so never set it yourself.

Give each separately built assembly that ships its own content its own bundle id:

```
[assembly: NetworkBundle(1)]
```

The main assembly is bundle `0` without the attribute. A named assembly that is not loaded, or carries no `NetworkBundleAttribute`, also falls back to bundle `0` and logs an editor error, so do not rely on that fallback.

Build the shards

In the Nucleus window (**Tools > Nucleus**), select the **Build** tab, then:

Click **Rebuild** next to **Rebuild Network Prefab Collection**. It groups every prefab carrying a `NetworkSystemObject` by its asset bundle, and stamps each prefab's `PrefabId` and its shard's `PrefabBundleId`. Ids are kept across rebuilds, so a bundle built before a change still resolves on a client built after it.

Click **Build** next to **Build Content Bundles**. It rebuilds the collections, builds every asset bundle into `Assets/StreamingAssets`, and fills in the `NetworkBundleManifest` from what it built.

If the project has no `NetworkBundleManifest`, the build creates one and reports where. If it finds more than one, it changes none of them and warns, so keep exactly one.

The bundle manifest maps bundle ids to files

`NetworkBundleManifest` has:

- `Entries`: a list of `BundleEntry { BundleId, FileName }`. **Build Content Bundles** writes them, replacing entries by id, so an entry you keep for content loaded through another pipeline survives.
- `RootDirectory`: the directory file names resolve against. When unset it is `Application.streamingAssetsPath`, which is `Assets/StreamingAssets` in the editor and the shipped copy in a build.

- `TryGetBundlePath(ushort bundleId, out string path)`: the full file path for a bundle id.

Load bundles at runtime

Add a `UnityAssetBundleLoader` component to a GameObject in your bootstrap scene.

Set its **Manifest** field to your `NetworkBundleManifest` asset.

With **Auto Register** on (the default), the loader registers with the integration's `CoreManager` in `Awake`, so your own code can request a bundle from `Start`. Turn it off and call `Initialize(CoreManager)` yourself if your game has more than one `CoreManager` or creates one later.

`UnityAssetBundleLoader.LoadBundleAsync(ushort bundleId)` loads the bundle and registers every `NetworkPrefabCollection` inside it before its task completes. To load bundles from a patcher or download cache instead of `StreamingAssets`, override its `protected virtual TryGetBundlePath`.

Override a prefab's required bundle

Use the override when a prefab needs a different bundle than the one that ships it, such as content the editor scan could not stamp, or a prefab that ships in the base build while the assets it needs do not. On the `NetworkSystemObject`:

- **Override Required Bundle**: turns the override on. When off, the required bundle is the prefab's own `PrefabBundleId`.
- **Required Bundle Id**: the bundle a client must hold before the object spawns for it. Shown only while the override is on.

`NetworkSystemObject.RequiredBundleId` returns the override when it is on, otherwise `PrefabBundleId`. The prefab's identity, `PrefabBundleId` paired with `PrefabId`, does not change.

Networked Scenes, Content Bundles and Persistence

Pages in this category:

- The network scene manifest and scene references
- UnitySceneLoader component
- UnitySceneManager component
- How networked scenes work
- Networked scenes in Unity
- Content bundles in Unity
- Objects authored into a scene
- Scenes and bundles: things that look broken
- Addressables-delivered scenes
- Saving and loading a world from Unity

The network scene manifest and scene references

The server names a scene by a `ushort` scene id, never by its path or name. `NetworkSceneManifest`, a `ScriptableObject` in `Nucleus.Integrations.Unity.Scenes`, maps each id to a scene asset on every peer. A project has exactly one, named `NetworkSceneManifest` and kept under a `Resources` folder. It is found by that name, not assigned in an inspector.

Scene ids are assigned for you

You never assign scene ids by hand. The editor rebuilds the manifest when:

- the Build Settings scene list changes,
- a scene asset is imported, moved or deleted (saving a scene re-imports it),
- the project's Addressables entries change,
- a player build starts.

To rebuild it yourself, open the Nucleus window (**Tools > Nucleus**), select the **Build** tab and click **Rebuild** next to **Rebuild Network Scene Manifest**.

The rebuild covers scenes in Build Settings and scenes the project has made Addressable. Each entry's delivery mode follows the project, so making a scene Addressable is the whole change needed to deliver it through Addressables (see Addressables-delivered scenes).

A scene's id is keyed to its GUID, so it survives a rename or move and is kept across rebuilds. A rebuild only adds ids for new scenes, and it keeps the entry of a scene that has left the build, so that id is never given to another scene.

Look up a scene from code

`NetworkSceneManifest` exposes:

- `Entries`: every networked scene this peer knows, as an `IReadOnlyList<SceneEntry>`.
- `UnsetSceneId`: the constant `0`, which no scene uses, so a failed lookup can be told apart from the first scene.
- `ResourceName`: the constant `"NetworkSceneManifest"`, the file name the manifest must have under a `Resources` folder.
- `TryGetScene(ushort sceneId, out SceneEntry sceneEntry)`: finds the entry for an id.
- `TryGetSceneId(string scenePath, out ushort sceneId)`: finds the id for a scene's asset path.
- `TryGetSceneIdByGuid(string sceneGuid, out ushort sceneId)`: finds the id for a scene's GUID. Prefer it, since the id is keyed to the GUID.
- `LoadProjectManifest()`: a static method that loads the project's manifest with `Resources.Load<NetworkSceneManifest>(ResourceName)`. It does not cache the result.

```
NetworkSceneManifest manifest = NetworkSceneManifest.LoadProjectManifest();

if (manifest != null && manifest.TryGetScene(sceneId, out NetworkSceneManifest.SceneEntry
entry))
{
    // entry.ScenePath is what you hand to Unity's scene loader.
}
```

Each `SceneEntry` holds:

Field	Purpose
`SceneId`	The id the server names this scene by. Assigned once and never renumbered.
`SceneGuid`	The scene asset's GUID, which the id is keyed to.
`ScenePath`	The scene asset's path, which the loader hands to Unity.
`RequiredBundleId`	The content bundle the scene ships in, or zero when it ships in the player build. A scene in a bundle cannot load until that bundle is held.
`DeliveryMode`	`SceneDeliveryMode.BuildSettings` or `SceneDeliveryMode.Addressables`: how the loader opens the scene.

Each peer's own manifest decides how a scene is delivered, not the server. A build can point at different content, or ship a renamed scene, without changing the id the server uses.

Name a scene in a field with NetworkSceneReference

For a serialized field that names a scene, use `NetworkSceneReference`. It stores only the scene asset's GUID (`SceneGuid`), and offers `HasScene` (true once a scene is assigned) and `TryGetSceneId(out ushort sceneId)`.

```
[SerializeField]
private NetworkSceneReference _lobbyScene;

if (_lobbyScene.TryGetSceneId(out ushort sceneId))
{
    // Tell the server to open sceneId.
}
```

In the inspector it is a scene-asset picker, with the scene id it currently resolves to shown beside it. `TryGetSceneId` looks the id up in the project's manifest each time it is called. Renaming or moving the scene never breaks the field, a scene added to the manifest later resolves after the next rebuild, and a scene that stopped being networked resolves to `UnsetSceneId` instead of opening the wrong scene.

Networked scenes in Unity

Driving the core API directly? See Opening and closing scene instances.

Add the manager and loader

`UnityCoreManager` adds a `UnitySceneManager` with default settings when neither its GameObject nor any child has one. That `UnitySceneManager` adds a `UnitySceneLoader` when no `NetworkSceneLoader` exists in the loaded scenes, inactive objects included. For a first scene, the defaults are enough.

To change the scene manager's settings, add `UnitySceneManager` yourself to the `UnityCoreManager`'s GameObject or to a child of it. Keep that child active: a `UnitySceneManager` on an inactive child is never set up. A loader you place yourself is used instead of the one it would add. The settings are on UnitySceneManager component.

Give the scene an id

Add the scene to Build Settings. The network scene manifest rebuilds itself and gives the scene an id.

If it has not, open the Nucleus window (**Tools > Nucleus**), select the **Build** tab and click **Rebuild** next to **Rebuild Network Scene Manifest**.

Leave the loader's **Manifest** field empty, and it uses the project's manifest. Assign one only if your project deliberately keeps more than one manifest.

Open an instance and send a client to it

`UnitySceneManager.NucleusSceneManager` is the core scene manager:

```
ushort sceneId; // the id the manifest gave your scene
if (unitySceneManager.NucleusSceneManager.TryOpenScene(sceneId, SceneScope.Connections, ou
t uint sceneHandle))
{
    unitySceneManager.NucleusSceneManager.TryRequestSceneLoad(connection, sceneHandle);
}
```

`TryOpenScene` only opens the instance on the server, and a `SceneScope.Connections` instance places nobody. Nothing loads on a client until you call `TryRequestSceneLoad(connection, sceneHandle)`. If you skip it, the scene never appears on the client and no error is logged.

Join placement only covers new joins

Join Placement (`JoinPlacement` on the core manager) decides which of the scenes already open a newly authenticated client is placed in. It does nothing for scenes opened later:

- A `SceneScope.Global` scene opened later still reaches every client, because opening it places every connected client.
- A `SceneScope.Connections` scene opened after a client joined is never offered to that client, even under the default `EveryOpenScene`. Call `TryRequestSceneLoad` for each connection you want in it.

Objects authored into a scene

A scene object is a `NetworkSystemObject` placed in a scene asset instead of spawned from a prefab. It already exists on every peer that has the scene loaded, so spawning it binds the network system to that GameObject instead of creating one.

Saving the scene stamps its id

`NetworkSystemObject.SceneObjectId` names the object within its scene, and `IsSceneObject` is `true` whenever that id is non-zero. You never set the id: saving a scene keeps every id already in use and gives any unset or duplicated id the next free value. The field is hidden in the inspector.

The id only has to be unique within one scene file. Peers identify an authored object by `SceneObjectKey(SceneHandle, SceneObjectId)`, and each loaded instance of a scene has its own `SceneHandle`. Two different scenes, or two stacked copies of one scene, can both hold an object with id `1` without colliding.

A scene object is bound, not instantiated

	Spawned prefab	Scene object
`NetworkSystem.IsSceneObject`	`false`	`true`
`NetworkSystem.PlatformId`	The prefab to instantiate	The scene identifier
On the receiver	Instantiated fresh	Bound to the GameObject already in the loaded scene

Save the scene after duplicating a scene object

Copying a stamped object copies its `SceneObjectId`. Until the scene is saved, both copies share the id, and a spawn can bind to the wrong one. Save the scene: the object seen first keeps the id and the other gets a free value. Because the field is hidden, saving is the only way to fix it.

UnitySceneLoader component

Driving the core API directly? See Registering a scene loader.

`UnitySceneLoader` is the Unity `ISceneLoader`. It looks up each scene id the server sends in the network scene manifest, loads the scene additively, and reports back so withheld spawns can land. Add it to a GameObject that can reach the integration's `CoreManager`, and it registers itself.

Set the inspector fields

Label	Default	What it does
Auto Register	on	Registers with the integration's `CoreManager` in `Awake`, so your own code can load scenes from `Start`. Turn it off to call `Initialize(CoreManager)` yourself.
Manifest	empty	Maps scene ids to scene assets and the content bundle each ships in. Leave it empty to use the project's manifest from Resources.
Local Physics	off	Loads each scene into its own local physics world (`LocalPhysicsMode.Physics3D`) and has Nucleus step it every tick. Turn it on so stacked copies of one scene do not collide with each other. Unity never steps a local physics scene by itself. Bodies in such a scene need manual interpolation.

The `Manifest` property returns the manifest in use, resolving the project's manifest when none is assigned.

Write your own loader

`NetworkSceneLoader` is the abstract `MonoBehaviour` base that implements `ISceneLoader`, and `UnitySceneLoader` derives from it. It defines:

- `LoadSceneAsync(uint sceneHandle, ushort sceneId)` and `UnloadSceneAsync(uint sceneHandle)`: abstract and asynchronous. They may take as long as needed.
- `MaximumConcurrentScenes`: a `virtual uint` that defaults to `SceneManager.UnlimitedConcurrentScenes`. Override it only if your loader cannot hold every scene the server asks for at once.
- **Auto Register** and `Initialize(CoreManager)`, as above.

To change one part of `UnitySceneLoader`, subclass it and override one of these hooks:

- `TryGetScene(ushort sceneId, out NetworkSceneManifest.SceneEntry sceneEntry)`:

looks up a scene id. Override it to read entries from somewhere other than a `NetworkSceneManifest`.

- `EnsureSceneContentAsync(ushort requiredBundleId)`: makes sure the content bundle a scene ships in is available before the scene loads.
- `LoadSceneCoreAsync(uint sceneHandle, ushort sceneId)`: the additive load, or `LoadAddressableSceneCoreAsync` when the entry is Addressable.
- `LoadAddressableSceneCoreAsync(uint sceneHandle, ushort sceneId, NetworkSceneManifest.SceneEntry sceneEntry, Task<bool> bindingTask)`: the Addressables load, through `Addressables.LoadSceneAsync`. It is compiled only under `NUCLEUS_ADDRESSABLES`.
- `UnloadSceneCoreAsync(uint sceneHandle)`: the unload.
- `ResolveManifest()`: returns the assigned manifest, or `NetworkSceneManifest.LoadProjectManifest()` when none is assigned.

`ResolveManifest` is `protected`; the others are `protected virtual`.

Turn on Addressables support

Install `com.unity.addressables` 1.19 or newer, and the integration sets the `NUCLEUS_ADDRESSABLES` define for you, which turns on the Addressables load path. See Addressables-delivered scenes.

UnitySceneManager component

Driving the core API directly? See Placing clients in scenes.

`UnitySceneManager` sets up the core `SceneManager` from the inspector and gives the peer a scene loader if the project has none. `UnityCoreManager` adds one with default settings if you have not.

Set the fields

The fields are applied once, when the managers are created.

Inspector label	Default	What it does
Join Placement	`JoinScenePlacement.EveryOpenScene`	Sets `SceneManager.JoinPlacement`.
Auto Request On Blocked Spawn	off	Sets `SceneManager.AutomaticRequestOnBlockedS
Scene Loader Enabled	on	Adds a `UnitySceneLoader` to this GameObject when no `NetworkSceneLoader` exists in the loaded scenes, active or inactive. A loader the project placed itself always wins.
Auto Simulate Stacked Scenes	off	Sets `UnitySceneLoader.LocalPhysicsEnabled` on the loader this component adds. A loader the project placed itself keeps its own value.
Scene Load Timeout (s)	`180`	Sets `SceneManager.LoadRequestTimeoutSeconds` how long the server waits for a client to answer a scene load request before treating the load as failed. Zero or less waits indefinitely.

Reach the core SceneManager

`NucleusSceneManager` holds the core `Nucleus.Managers.Scenes.SceneManager`. It is set when the managers are created.

Translate between a scene handle and a Unity scene

Stacked copies of one scene asset share a name and path, so Unity's `GetSceneByName` and `GetSceneByPath` return an arbitrary copy. Only the scene handle tells them apart. Use these methods instead:

```
public bool TryGetScene(uint sceneHandle, out Scene scene)
public bool TryGetSceneHandle(Scene scene, out uint sceneHandle)
```

`TryGetScene` returns the live Unity scene for a handle. It returns false while the scene is still loading, and

on a peer the instance was never opened for.

`TryGetSceneHandle` returns the handle for a live scene, with three outcomes:

- False: the scene is still loading, and its objects are running `Awake` now.
- True with `NetworkSystem.UnsetSceneHandle`: this peer loaded the scene itself, and no instance will ever claim it.
- True with any other value: the handle of the instance.

Saving and loading a world from Unity

Driving the core API directly? See Saving and loading a world.

There is no Unity persistence component. Call `CoreManager.WorldPersistenceManager` from a `MonoBehaviour` you write. World persistence needs Pro, and it runs only on the server: `SaveAsync` and `LoadAsync` are refused unless `TransportManager.IsServerStarted` is true.

Register a store

`JsonWorldStore`, which ships with Nucleus, writes the world to a JSON file. Give it the `CoreManager` and a path under `Application.persistentDataPath`, which is writable on every platform Unity targets.

```
UnityCoreManager unityCoreManager = GetComponent<UnityCoreManager>();
CoreManager coreManager = unityCoreManager.CoreManager;
WorldPersistenceManager worldPersistenceManager = coreManager.WorldPersistenceManager;

string filePath = Path.Combine(Application.persistentDataPath, "world.json");
JsonWorldStore jsonWorldStore = new(coreManager, filePath);

worldPersistenceManager.SetWorldStore(jsonWorldStore);
```

`JsonWorldStore` writes to a temporary file and moves it over the target only when the write finishes, so a crash mid-save leaves the previous world intact.

Save the world

```
bool saved = await worldPersistenceManager.SaveAsync();
```

Load the world without blocking the main thread

A load can only finish while Unity keeps driving the network loop from `Update` on the main thread. Never block the main thread on the load task (`.Wait()`, or `.Result` before it completes): the load deadlocks. Await it from an `async` method, or start it and poll it from `Update`.

```
public class WorldLoader : MonoBehaviour
{
    public void BeginLoad() => _ = LoadWorldAsync();

    private async Task LoadWorldAsync()
    {
        bool loaded = await GetComponent<UnityCoreManager>().CoreManager
            .WorldPersistenceManager.LoadAsync();

        // handle the result
    }
}
```

```
public class WorldLoader : MonoBehaviour
{
    private Task<bool> _loadTask;

    public void BeginLoad()
    {
        _loadTask = GetComponent<UnityCoreManager>().CoreManager
            .WorldPersistenceManager.LoadAsync();
    }

    private void Update()
    {
        if (_loadTask is { IsCompleted: true })
        {
            bool loaded = _loadTask.Result;
            _loadTask = null;
            // handle the result
        }
    }
}
```

A load replaces the whole world

A load despawns every object, closes every open scene, reopens the saved scenes through the registered `ISceneLoader` under their saved handles, then rebuilds the saved objects. Every connected client sees a world of despawns followed by a world of spawns, so consider putting players on a loading scene while it runs.

- A scene object is bound to the object already in its reopened scene, matched by `PlatformId`.
- Any other object is constructed from its saved system type, then looked up in the prefab registry by its `PrefabId` and prefab bundle id. A prefab missing from the collection that shipped that bundle cannot come back.
- A scene the loader cannot open is the most common failure. The store's `WorldLoadFailureAction` decides whether that scene's objects are dropped or the whole load fails. A failed load leaves an empty world, not the old one.

Replicated State and Change Callbacks

Pages in this category:

- NetworkTransform
- How replicated state works
- Replicating a value from a Unity script
- Reacting to a replicated change in Unity
- NetworkAnimator

NetworkAnimator

`NetworkAnimator` replicates an `Animator`. The controller's float, int, bool and trigger parameters, layer weights and playback speed are sent, and every other peer runs its own state machine from them.

Each layer's current state and phase are sent as well, as a correction. A peer whose state machine branched differently is pulled back onto the server's state, and a peer that starts watching the object, through a spawn, a late join, an interest change or a resync, opens every layer at the authority's current phase.

Add it to an animated object

Add `NetworkAnimator` to the GameObject with the `Animator`. It requires a `NetworkSystemObject`.

Field	Default	Effect
Animator	empty	The `Animator` to replicate. Empty uses the one on this GameObject.
State Correction Interval	`Disabled`	How often each layer's phase is refreshed between transitions. See [Choose a state correction interval](#choose-a-state-correction-interval).
Precise Playback Enabled	off	Followers play the authority's exact pose. See [Turn on precise playback to line up hitboxes](#turn-on-precise-playback-to-line-up-
Root Motion Management Enabled	on	Clears `Animator.applyRootMotion` on every peer that does not drive the object, so root motion cannot fight the replicated pose, and restores it on the peer that does. Turn it off when other code owns the flag, or when the object does not replicate a transform.

Choose a state correction interval

State Correction Interval (`StateCorrectionInterval`) bounds how far a follower already in the session can drift from the authority's phase during a long loop. A follower whose state machine took a different branch is corrected on the tick it happens, whatever this is set to.

- `Disabled` (default): the phase updates only on a transition, so a follower in a long loop runs on its own clock until the next one. Costs nothing.
- `Always`: refreshed every tick, so a follower's phase tracks the server's exactly. The most expensive.
- `Short`: refreshed at most every 250 ms.
- `Long`: refreshed at most once a second, for a loop a joiner only needs to open roughly in the right place.

`Short` and `Long` mean the same spans as the `SendInterval` values of the same names. Every value

works in a Free build.

Turn on precise playback to line up hitboxes

Turn on **Precise Playback Enabled** for an animated part whose pose must match the authority's, such as a limb carrying hitboxes that a lag-compensated shot is judged against. A follower then stops running its own state machine and places each layer at the authority's playback position, including through transitions. The pose is applied at once, so a raycast in the same frame sees it.

- It costs bandwidth for every layer on every tick, and more during a transition. A state that does not loop is held at its end, so a finished clip stops being sent.
- Animation events do not fire on a follower. Fire game logic from replicated state or a remote call instead.

Drive the animator through NetworkAnimator

```
public float Speed { get; set; }

public void SetFloat(string parameterName, float value);
public void SetFloat(int parameterHash, float value);

public void SetInteger(string parameterName, int value);
public void SetInteger(int parameterHash, int value);

public void SetBool(string parameterName, bool isParameterTrue);
public void SetBool(int parameterHash, bool isParameterTrue);

public void SetLayerWeight(int layerIndex, float weight);

public void SetTrigger(string parameterName);
public void SetTrigger(int parameterHash);

public void ResetTrigger(string parameterName);
public void ResetTrigger(int parameterHash);

public void Play(int stateNameHash, int layerIndex = 0, float normalizedTime = 0f);
public void CrossFade(int stateNameHash, float normalizedTransitionDuration, int layerInde
x = 0);
```

- Every write does nothing on a peer that does not drive the object, so the same script can run on every peer.
- `Speed` is `Animator.speed`, the whole animator's playback rate, not a parameter named "Speed".
- `Play` and `CrossFade` drive the controller's animator. Followers are moved onto the destination state at the server's phase rather than blending into it, so only the controller plays the fade.
- Only the first 64 bool parameters replicate. An animator with more logs an error with the count; move the extra bools to int parameters or a second animator.

Fire triggers only through SetTrigger

Never call `Animator.SetTrigger` directly on a replicated animator: Unity can consume the trigger before it is sent. `NetworkAnimator.SetTrigger` fires the local animator and replicates the fire. `ResetTrigger` clears the trigger locally and also takes back a fire not yet sent this tick; a reset on a later tick is local only.

A peer that joins after triggers fired does not replay them. Its animator starts clean.

React to a parameter landing

To run code when a replicated parameter or layer state changes on an observing peer, use `OnMembersChanged` on the replicated component, `UnityAnimatorComponent`, instead of polling the `Animator`. Its members are `FloatParameters`, `IntParameters`, `BoolParameters`, `TriggerPulses`, `LayerWeights`, `Speed`, `LayerStateHashes` and `LayerNormalizedTimes`, plus `LayerPlaybackPositions`, `LayerNextStateHashes`, `LayerNextPlaybackPositions` and `LayerTransitionProgress`, which stay empty unless Precise Playback is on.

NetworkTransform

Driving the core API directly? See TransformComponent.

`NetworkTransform` replicates a GameObject's position, rotation and scale from the controller to every other peer. Add it to a networked object, on the GameObject that carries the `NetworkSystemObject` or on a child of it. It belongs to the nearest `NetworkSystemObject` at or above it; with none there it never gets a system. To catch that, open the Nucleus window (**Tools > Nucleus**), select the **Build** tab and click **Validate** next to **Validate Network Prefabs**.

The controller's pose is captured on the `EarlySerialize` step, just before the tick is sent. Every other peer follows that pose smoothly, slightly behind.

Configure it in the inspector

Field	Default	Effect
Transforms	empty	The transforms to replicate, all on one system. Empty replicates the component's own GameObject. See [Replicate several parts with one NetworkTransform](#replicate-several-parts-wit
Transform Space	`Local`	Which space the pose replicates in. See [Choose the transform space](#choose-the-transform-space).
Interpolate Scale Enabled	`true`	Interpolates the replicated scale on non-controllers. Off, local scale is left as it is.
Kinematic Management Enabled	`true`	Holds an attached `Rigidbody` kinematic on every peer that is not the controller, including the server while a client controls the system, so local physics cannot fight the replicated pose. The controller's body is returned to simulation. Turn it off when other code owns the body's kinematic state.
Send Interval	`Normal`	Paces replication while Transmission Mode is `Interval`. No effect under `Divine`.
Transmission Mode	`TransmissionModeDefaults.Motion`	Applied to position, rotation and scale. Resolves to `Divine` where a projector is registered, `Interval` otherwise.

Extrapolation Enabled	`true`	Keeps the object moving on other peers through the sender's silence, under either mode. A lost update is then covered by motion but can briefly overshoot; turn it off if that is visible, and the object holds still until told instead. Under `Interval` it costs a little extra bandwidth. See [How often a member is sent](../../core-api/state/send-pacing.md).

Transforms and Transform Space cannot change during play.

Choose the transform space

The space is fixed for the object's life.

- **`TransformSpace.Local`** (default) uses a `UnityLocalTransformComponent`, which replicates `Transform.localPosition` and `Transform.localRotation`. An object that stays put inside a moving parent sends nothing. The parent must be replicated too, so both peers measure the offset from the same frame. On a reparent the object keeps its local offset inside the new parent (`IsWorldPoseKeptOnReparent` is `false`).
- **`TransformSpace.World`** uses a `UnityWorldTransformComponent`, which replicates `Transform.position` and `Transform.rotation`. Use it when the object can hang from something that is not replicated, such as a client-local rig. A reparent moves nothing (`IsWorldPoseKeptOnReparent` is `true`), but a carried object sends its pose every tick it rides, even while still relative to its carrier.

Replicate several parts with one NetworkTransform

For an object with several moving pieces, such as a turret and its barrel, list the pieces in **Transforms** on one `NetworkTransform`. They replicate on one system with one controller and one spawn, share the component's settings, and moving one piece sends only that piece. When the parts share settings, this costs less to send than one `NetworkTransform` per part.

- Keep the list on the prefab, not edited per instance: entry *i* is matched to the same entry on every peer by its position in the list.
- Every entry must sit at or below the object's `NetworkSystemObject`. Null entries are skipped.
- A list longer than `NetworkSystem.MaximumComponentCount` (31) is refused with an error.

To give a part its own settings, put a separate `NetworkTransform` directly on the child. It joins the nearest `NetworkSystemObject` above it. Do not add a second `NetworkSystemObject` to the child: only a prefab's root one is spawned.

Use NetworkRigidbody for a simulated body

`NetworkTransform` makes non-controllers follow a received pose; it does not run local physics on them. For a rigidbody that should simulate locally on every peer and converge toward the server's state, use `NetworkRigidbody`.

Reacting to a replicated change in Unity

Driving the core API directly? See Reacting to changes: OnMembersChanged.

`NetworkComponent.OnMembersChanged` is called once per tick per direction when any of the component's members changed, so a script never has to poll a value.

```
public virtual void OnMembersChanged(ulong memberFlags, MemberChangeDirection memberChange
Direction) { }
```

Forward OnMembersChanged to your MonoBehaviour

A MonoBehaviour cannot inherit from a `NetworkComponent`, and the component can be rented and returned independently of the scene object watching it. So override `OnMembersChanged` on the component and raise an event:

```
public partial class CoinComponent : NetworkComponent
{
    public event MembersChangedHandler MembersChanged;
    public delegate void MembersChangedHandler(ulong memberFlags, MemberChangeDirection me
mberChangeDirection);

    public override void OnMembersChanged(ulong memberFlags, MemberChangeDirection memberC
hangeDirection)
        => MembersChanged?.Invoke(memberFlags, memberChangeDirection);
}
```

Subscribe from the MonoBehaviour when it rents the system, and unsubscribe when it is destroyed:

```
_coinComponent.MembersChanged += OnCoinMembersChanged;
// ...
_coinComponent.MembersChanged -= OnCoinMembersChanged;
```

Find which member changed

Every `NetworkComponent` gets a generated `[Flags] enum <ComponentName>Flags : ulong`, one value per member in declaration order. Cast `memberFlags` to it and test the bits you need:

```
private void OnCoinMembersChanged(ulong memberFlags, MemberChangeDirection memberChangeDir
ection)
{
    CoinComponentFlags changed = (CoinComponentFlags)memberFlags;

    if ((changed & CoinComponentFlags.Collector) != 0)
    {
        // React to the collector reference changing.
    }
}
```

A set flag means the member was applied, not that its value differs: a predicted member's confirmed outcome still sets its flag.

A spawn, a resync, a reconcile or a whole-component recovery reports `memberFlags` as `ulong.MaxValue` under `Read`: treat every member as changed. A late-joining client's first call looks like this. A member-granular recovery reports only the members it repaired.

Usually react only to MemberChangeDirection.Read

- `Write`: this peer changed the members itself. It is raised before the tick is sent, even for a system with no observers.
- `Read`: the members were applied from received state. It is raised once everything received for the tick is applied, before the Reconcile step. `NetworkMemberBase.TryGetWritingClient` only resolves under `Read`.

A host raises both directions in the same frame for its own change, `Write` first. A handler that ignores the direction reacts twice on a host and once everywhere else, so guard one-sided reactions, such as a cooldown or a "you scored" label:

```
if (memberChangeDirection is not MemberChangeDirection.Read)
    return;
```

A value recomputed the same way on every peer from the replicated state does not need the guard.

Do per-tick work in a step virtual, not Update

To read state the apply just landed, or run something every tick at that point, override a step virtual on your `NucleusBehaviourBase` script instead of polling in `Update`, which has no fixed relationship to the network loop:

```
protected override void OnLateStateUpdate(StepDelta stepDelta)
{
    // Runs once replicated state has been applied for this tick.
}
```

`OnEarlyStateUpdate` runs just before replicated state is applied, `OnLateStateUpdate` once it has been, and `OnReconcile` after state has been set back for a reconcile.

Replicating a value from a Unity script

Driving the core API directly? See Your first networked component.

Declare the component

A networked component is a plain `partial` class, not a `MonoBehaviour`. Declare its values as `NetworkMember<T0>` fields; the source generator writes the rest, so never hand-write serialization.

```
public partial class HillTimerComponent : NetworkComponent
{
    public readonly NetworkMember<float> SecondsRemaining = new();
}
```

Attach it with NucleusBehaviour

A `MonoBehaviour` adopts the component by inheriting `NucleusBehaviour<TComponent0>`:

```
public class HillTimer : NucleusBehaviour<HillTimerComponent>
{
}
```

The GameObject needs a `NetworkSystemObject`. Several `NucleusBehaviour` scripts on one GameObject, each with a different component, share that one `NetworkSystemObject` and link their systems into one `NetworkSystemGroup`.

Use Component only once the system links

`Component` is null until the system links, so writing to it from `Awake` throws a `NullReferenceException`. Use it from the hooks raised once a system is linked: `OnSystemLinked`, `OnServerStarted` and `OnClientStarted`, and their counterparts `OnSystemUnlinked`, `OnServerStopped` and `OnClientStopped`.

- `OnSystemLinked` fires once, when the system arrives, on every peer.
- `OnServerStarted` and `OnClientStarted` fire once each when that role comes up while a system is linked, immediately if the role was already up.

Because every peer links the system, check the role before a server-only write:

```
protected override void OnSystemLinked()
{
    if (!IsController(ControllerType.Server))
        return;

    Component.SecondsRemaining.Value = _survivalSeconds;
}
```

Reading a replicated value needs no guard: once linked, every peer's `Component` holds the latest value.

Guard writes with IsController

`IsController(ControllerType)` reports whether this peer controls the linked system, and `IsStarted(Invoker)` whether `Invoker.Server` or `Invoker.Client` is up. Both return false while no system is linked, so they are safe to call anywhere.

Input handlers, RPC sends and per-tick writes run on peers that do not control the object as a matter of course, so use these quiet forms there:

```
protected override void OnEarlyFixedUpdate(StepDelta stepDelta, SimulationContext simulati
onContext)
{
    if (!IsController(ControllerType.Server))
        return;

    Component.SecondsRemaining.Value = Mathf.Max(Component.SecondsRemaining.Value - _secon
dsPerTick, 0f);
}
```

Use `EnsureIsController(ControllerType)` and `EnsureIsStarted(Invoker)` only where reaching the code without that control or role is a bug, such as a server-only method called only by other server code. They return the same answer but also log a warning naming the calling member, once per calling member on each behaviour.

For previous-value access, interpolation and the constructor options that control how a member replicates, see NetworkMember.

How replicated state works

Driving the core API directly? See How replicated state works.

Systems hold components, and components hold members

- A `NetworkSystem` is the networked entity. It holds up to 31 `NetworkComponents` (`NetworkSystem.MaximumComponentCount`).
- A `NetworkComponent` holds the values that replicate, as `NetworkMembers`: at most 64, one for each bit of the `memberFlags` that `OnMembersChanged` reports. The generator does not check this limit, so a component that declares more still builds but does not replicate correctly.
- A struct or class used as a member's value can hold at most 62 serialized fields and properties. At 63 (`Constants.MaximumNetworkMemberCount`) the generator reports `SERIALIZERS000`.

Late joiners need nothing extra

A member holds the value as of the last write. When a new observer starts watching a system, it receives every member's current value, so you never write a separate late-joiner payload.

OnMembersChanged reports writes and reads

Each tick, `NetworkComponent.OnMembersChanged(ulong memberFlags, MemberChangeDirection memberChangeDirection)` is raised up to twice per component: `Write` for what this peer changed, `Read` for what it received. A host raises both for its own change, so code that only cares about a remote change, including anything calling `NetworkMemberBase.TryGetWritingClient`, must check for `Read`. See Reacting to a replicated change in Unity.

Only the controller writes by default

`MemberWriteAccess.Controller`, the default, lets the server or the single controlling client set a system's members. Letting other observers write too (`MemberWriteAccess.AnyClient`) is a Pro option; see Letting clients set members. A Free build has no `MemberWriteAccess` type, and only the controller writes.

Four settings decide what a change costs

- **Accuracy**: how precisely a change is compared and sent.
- **Compression level**: how tightly the value is compressed.
- **Transmission mode**: how changes are sent across ticks, such as `Interval` or `Divine`.
- **Send interval**: how often a changed member may send. See How often a member is sent.

Use Nucleus terms

- **System**, not object: `NetworkSystem`.
- **Component**, not behaviour: `NetworkComponent`.
- **Member**, not variable: `NetworkMemberBase`.
- **Controller**, not owner: the client permitted to set a system's members by default.

Several systems on one object

Driving the core API directly? See System groups.

Split an object so its parts replicate separately

One `NetworkSystemObject` can link several `NetworkSystem` instances. Each replicates on its own terms: interest can stop one, such as the transform, for a distant observer, while another whose changes matter at any range keeps replicating. The GameObject stays until every system has left it, so it does not flicker while its parts come and go.

All systems on one marker share one `GroupId`, and a receiver hosts them all on a single GameObject instead of instantiating the prefab again.

Require a second system from a second script

Each script on the object declares its own need with `NetworkSystemObjectPool.RequireSystem` in its own `Awake`. `NetworkAnimator`, for example, does this:

```
NetworkSystemObjectPool.RequireSystem<NetworkSystem, UnityAnimatorComponent>(this, OnSyste
mAcquired, OnSystemReleased);
```

Each requirement is served once, from a system the marker already links or from a fresh rent, whether the script's `Awake` runs before or after that system links. You never retry or guard against a double rent.

`TryRent<TSystem, TComponent0>` is different: each call rents and links another system, so calling it twice for the same need gives you two systems. Use it only when a driver wants its system back at once, at a moment of its own choosing.

Run object-wide work once every system has linked

`SystemsLinked` fires once the marker's whole set is linked, and at once for a handler added after that. Register remote-call handlers or other object-wide work there:

```
void Awake()
{
    GetComponent<NetworkSystemObject>().SystemsLinked += OnSystemsLinked;
}

void OnSystemsLinked(NetworkSystemObject networkSystemObject)
{
    if (networkSystemObject.TryGetFirstSystem(out NetworkSystem primary))
    {
        // Every system the group carries is present now. Register remote-call
        // handlers or other object-wide work against the primary system here.
    }
}
```

NetworkSystemObject component lists `Systems`, `LinkedSystemCount`, `System` and the other events.

Despawn one system or all of them

- `NetworkSystemObject.Despawn()` stops every linked system, so they leave together.
- `SystemManager.EnsureStopSystem` stops only that one system. The others stay linked and the GameObject stays.

On a receiver, the GameObject is reclaimed only when its last system leaves: a scene object is destroyed,

and a dynamic prefab goes back to the configured `INetworkPrefabPool`. A reused instance gets a fresh `GroupId` on its next spawn.

When interest stops a system for one peer but keeps the object, the GameObject is kept, and the spawn that arrives when the peer re-enters interest links onto the same GameObject.

Systems, Objects and Spawning

Pages in this category:

- NetworkSystemObject component
- NucleusBehaviour: the script base class
- What a system is
- Player objects
- Prefab identity and the prefab collection
- Spawning and despawning objects
- Carrying and re-parenting objects
- Pooling spawned objects
- Several systems on one object
- Client-predicted spawning in Unity

NetworkSystemObject component

NetworkSystemObject links a GameObject to its systems

Put one `NetworkSystemObject` on the root of each networked prefab or scene object. It links the GameObject to one or more `NetworkSystem` instances. An object normally links one system; it links several when a `NetworkSystemGroup` hosts more than one system on it (see Several systems on one object).

Set it up in the inspector

Section	Field	Purpose
Identity	**Scene Object Id**, or **Prefab Id** and **Prefab Bundle Id**	Read-only, stamped by the editor. A scene object shows its scene object id; a prefab shows its prefab id and bundle id. See [Prefab identity and the prefab collection](./prefab-ids-and-collections.md).
Predicted Spawning	**Predicted Spawn Policy** (default `None`)	What a client may do to this object before the server agrees: create it, remove it, both or neither. The server still decides. It is applied to each system as it links (`NetworkSystem.PredictedSpawnPolicy`). See [Client-predicted spawning in Unity](./predicted-spawning.md).
Prediction	**Predicted By Observers**	Observing clients predict this object as its controller does. See [Predicting a body with a validated behaviour](../physics/predicting-a-body-with-a-v
Content Bundle	**Override Required Bundle**	Off, the content bundle a client must hold before this object may spawn for it is read from the prefab. On, **Required Bundle Id** is used instead.
Content Bundle	**Required Bundle Id**	The bundle to require when the override is on. `0` means the object needs no content, whichever bundle it ships in.

Controller Retention	**Override On Disconnect**	Off, the object uses the Unity System Manager's default for what happens when its controller disconnects. On, **On Controller Disconnect** decides it for this object.
Controller Retention	**On Controller Disconnect** (default `Retain`)	Control is surrendered when the controller disconnects either way; this decides only whether the object is held so a returning player can be handed it back. The value is applied when the object's group is rented, not read continuously.
Spawn Payload	**Payload** and **Auto Transform Payload**	One-time data sent with the spawn. See [Spawning and despawning objects](./spawning-objects.md).

A duplicated prefab carries the same **Prefab Id** as its source. Rebuilding the prefab collection gives the copy a new id; see Prefab identity and the prefab collection.

Read it from a script

Member	Type	Meaning
`System`	`NetworkSystem`	The first system linked to this object, or `null`. If it leaves while others remain, the next-oldest takes its place.
`Systems`	`IReadOnlyCollection<NetworkSystem>`	Every linked system, in the order each linked.
`LinkedSystemCount`	`int`	How many systems are linked.
`TryGetFirstSystem(out NetworkSystem)`	`bool`	Resolves `System`. Use it for object-wide work, such as registering remote-call handlers.
`PrefabId`	`ushort`	The prefab identifier stamped by the prefab collection build, or zero when unset.
`PrefabBundleId`	`ushort`	The content bundle whose collection stamped this prefab.
`RequiredBundleId`	`ushort`	**Required Bundle Id** when **Override Required Bundle** is on, otherwise `PrefabBundleId`.

`SceneObjectId`	`uint`	This object's identifier within its scene, or zero when it is not a scene object.
`IsSceneObject`	`bool`	True when `SceneObjectId` is not zero.
`PredictedSpawnPolicy`	`Nucleus.Systems.PredictedSpawnPolicy`	This policy authored on the prefab.

`NetworkSystemObject.TryGetLinked(NetworkSystem, out NetworkSystemObject)` goes the other way: it finds the marker linked to a system, or returns false when none is.

Remote-call handlers belong on a `NetworkSystem`, which implements `IRpcScope`, never on the marker. Resolve the system through `System`, `Systems` or `TryGetFirstSystem`.

Subscribe to its events

- **`SystemLinked(NetworkSystem)`** : raised each time a system links.
- **`SystemUnlinked(NetworkSystem)`** : raised when a linked system is cleared on despawn.
- **`SystemsLinked(NetworkSystemObject)`** : raised once the object's whole set of systems has linked. An ungrouped object raises it on its first link; a grouped object raises it when its linked count reaches the group's member count. It is raised again each time a complete set is assembled, including after the group grows. A handler added after the set is complete is called at once.
- **`SystemsUnlinked(NetworkSystemObject)`** : raised once the last system unlinks.

Despawn the object

`Despawn()` stops every system linked to this object on the server and replicates the despawn to observers. A call with no linked system does nothing. Use it instead of destroying the GameObject when the instance should go back to the prefab pool; destroying the GameObject also despawns it network-wide. See Spawning and despawning objects.

NucleusBehaviour: the script base class

Driving the core API directly? See Renting and finding systems in code.

Inherit `NucleusBehaviour<TComponent0>` for a gameplay script that belongs to one networked system. It requires a `NetworkSystem` carrying `TComponent0` for you and exposes that component as `Component` once the system links.

Pick the base class

Base class	Use it when
`NucleusBehaviour<TComponent0>`	The script needs one component on a plain `NetworkSystem`.
`NucleusBehaviour<TSystem, TComponent0, ...>`	The script needs a system type other than `NetworkSystem`, or more than one component. Overloads go up to 31 component types (`TComponent0` through `TComponent30`).
`NucleusBehaviour` (non-generic)	The script needs only the role and control checks and the hooks, and another component on the object, such as a `NetworkTransform` or `NetworkRigidbody`, already declares the system.

`NucleusBehaviourBase` holds the shared members below; scripts do not inherit it directly.

The non-generic `NucleusBehaviour` declares nothing and adopts whichever system the object's `NetworkSystemObject` links. If nothing else on the object declares a system, it waits for the object's whole life. If the system it adopted stops while the object keeps another, it adopts the remaining one at the end of that frame; a whole-object despawn unlinks it and adopts nothing.

If a script needs a system it did not declare, rent or find it through the core API rather than adding a type parameter you don't otherwise need.

Read the linked system

Property	Type	Meaning
`NetworkSystem`	`NetworkSystem`	The linked system, or null before linking and after unlinking.
`CoreManager`	`CoreManager`	The bound manager, or null when none was bound.
`IsHostStarted`	`bool`	True when this peer runs both server and client, for a linked system.
`Component`	`TComponent0`	The required component, or null before `OnSystemLinked` and after `OnSystemUnlinked`.

Check role and control

- `IsController(ControllerType controllerType)` returns `NetworkSystem.IsController(controllerType)`, or false while no system is linked.
- `IsStarted(Invoker invoker)` returns whether this peer's server or client role is started, for a linked system.

`EnsureIsController(ControllerType controllerType)` and `EnsureIsStarted(Invoker invoker)` return the same answers, and log a warning naming the calling member when the answer is false. Each warns once per calling member and argument.

Use the `Ensure` form only where reaching the code without that role or control is a programming error, such as a server-only method that only server code calls. Use the plain form for checks that are expected to fail on some peers, such as input handling, RPC sends and per-tick callbacks:

```
protected override void OnEarlySerialize(StepDelta stepDelta)
{
    if (!IsController(ControllerType.AnyController))
        return;

    // Safe to set controlled members here.
}
```

Override the lifecycle hooks

Override these `protected virtual` methods:

Hook	Called
`OnSystemLinked()`	Once `NetworkSystem` is linked.
`OnSystemUnlinked()`	As `NetworkSystem` unlinks, including when the behaviour is destroyed while linked. The system is still readable during the call. Reset a pooled instance's per-life state here.
`OnServerStarted()`	Once this peer's server role starts, with a system linked.
`OnServerStopped()`	Once this peer's server role stops while a system is linked.
`OnClientStarted()`	Once this peer's client role starts, with a system linked.
`OnClientStopped()`	Once this peer's client role stops while a system is linked.
`OnControllerChanged(Connection previousControllerConnection, Connection currentControllerConnection)`	On every peer when control of the linked system moves.

When a system links while a role is already running, `OnServerStarted` or `OnClientStarted` is raised at once, so a script never misses a role that started before it linked.

Call the base Awake, OnEnable, OnDisable and OnDestroy

The base class needs its own `Awake`, `OnEnable`, `OnDisable` and `OnDestroy`. If your script needs one of them, override it and call the base method:

```
protected override void Awake()
{
    base.Awake();

    _renderer = GetComponent<Renderer>();
}
```

Call the base method first, unless `OnSystemLinked` reads something you set up: `base.Awake()` can link a system that already exists and raise `OnSystemLinked` on the spot, so cache what that hook reads before the call. Setup that needs the system belongs in `OnSystemLinked`.

The build reports both mistakes as errors, since either would leave the behaviour unlinked with nothing logged:

Id	Reported when
`NUCLEUS001`	A class deriving, directly or indirectly, from `NucleusBehaviourBase` declares `Awake`, `OnEnable`, `OnDisable` or `OnDestroy` without `override`.
`NUCLEUS002`	An override of one of those four methods never calls the base method.

The lifecycle hooks and per-step methods are empty in the base class, so their overrides may skip the base call.

Run code on a loop step

Override any of these `protected virtual void On...(StepDelta stepDelta)` methods, listed in loop order:

```
OnEarlyVariableUpdate
OnEarlyStateUpdate
OnLateStateUpdate
OnReconcile
OnEarlyFixedUpdate
OnLateFixedUpdate
OnVariableUpdate
OnEarlySerialize
OnLateSerialize
OnTickAdvance
OnLateVariableUpdate
```

- `OnEarlyVariableUpdate`, `OnVariableUpdate` and `OnLateVariableUpdate` run every frame.
- `OnEarlyStateUpdate`, `OnLateStateUpdate`, `OnReconcile`, `OnEarlySerialize`, `OnLateSerialize` and `OnTickAdvance` run once per tick, on the frame that ticks.
- `OnEarlyFixedUpdate` and `OnLateFixedUpdate` run once per elapsed tick interval, so they can run twice in one frame after a long frame. They also take a `SimulationContext` naming the pass, and on a client that predicts the object they run again for every tick a reconcile replays (see Replaying a correction in Unity).

See The network loop steps for what each step means.

To move an object every frame, override `OnVariableUpdate` and move it by `stepDelta.NormalizedDelta` rather than moving it in `Update` by `Time.deltaTime`. The normalized

delta keeps the distance moved each tick even, which costs far less to replicate. See Tick rate and the loop in Unity.

Delay the start with CanStartSystem

`protected virtual bool CanStartSystem => true` decides whether the server may start the required system as soon as it is rented. Override it to return false for an object that must set its opening values before the spawn is announced, then start the system yourself. It is read once, when the system is declared, and ignored on a receiving client.

Player objects

NetworkPlayerSpawner gives every client one object of its own

On the server, `NetworkPlayerSpawner` spawns one prefab for each authenticated client, gives that client control of it, and remembers which object belongs to which connection. A player object is an ordinary networked object that one client controls; replication, interest, client state, write permission and controller retention all work as for any controlled object.

Only the server spawns. The spawner also gives an object to every client that authenticated before it existed, and does so again whenever this peer's server role starts. Losing the server role clears every connection-to-object record the spawner holds.

Set it up in the inspector

- **Player Prefab** (`GameObject`): the networked prefab spawned for each client. It needs a `NetworkSystemObject`; without one, the spawner logs an error and disables itself.
- **Spawn Points** (`Transform[]`): the poses new objects are placed on, taken in turn and wrapping round. Leave it empty to spawn every player on the spawner's own transform.
- **Disconnect Mode** (`NetworkPlayerDisconnectMode`, default `Despawn`): what happens to a player's object when that player leaves.
  - `Despawn`: the object leaves with the player, and a returning player always gets a new one.
  - `RetainForReturn`: the object stays in the world, uncontrolled, and is handed back to the same player if they return before the retention record lapses. The spawner marks every system on the object `ControllerRetentionPolicy.Retain`. An object whose record expires, is evicted or empties is despawned then.

`RetainForReturn` matches a returning player by `Connection.Identity`, so it is only as trustworthy as that identity. The default `AddressClientAuthenticator` identifies a client by its address, which two players behind one NAT can share, so the second player could be handed the first player's object. Use an `IClientAuthenticator` that issues a real per-player identity with this mode.

Find a player's object from code

```
public bool IsServerStarted { get; }
public int PlayerObjectCount { get; }

public bool EnsurePlayerObject(Connection connection);

public bool TryGetPlayerObject(uint connectionId, out NetworkSystemObject playerNetworkSys
temObject);
public bool TryGetPlayerObject(Connection connection, out NetworkSystemObject playerNetwor
kSystemObject);
```

- `EnsurePlayerObject` gives a connection its object. Calling it again for the same connection never spawns a second one: it first uses this spawner's record, then an object the connection already controls (after a world adoption), then a retention record redeemed by identity, and only then spawns.
- `TryGetPlayerObject` looks up by `Connection.Id` or by the `Connection`. It only answers for objects this spawner created or took over.
- `PlayerObjectCount` is how many clients this spawner holds an object for.

React when a player gets or loses an object

```
public event PlayerObjectSpawnedHandler PlayerObjectSpawned;
public event PlayerObjectReclaimedHandler PlayerObjectReclaimed;
public event PlayerObjectDespawnedHandler PlayerObjectDespawned;
```

- **`PlayerObjectSpawned(Connection connection, NetworkSystemObject playerNetworkSystemObject)`** : raised on the server when a client is given a new object, after it has been made the controller.
- **`PlayerObjectReclaimed(Connection connection, NetworkSystemObject playerNetworkSystemObject)`** : raised when a client is given an object that already existed, either redeemed from a retention record or already under its control after this peer adopted a world. The object keeps the state it had, so anything you reset for a fresh player must also run here and be safe on a used object.
- **`PlayerObjectDespawned(NetworkSystemObject playerNetworkSystemObject)`** : raised when a player object leaves the world, because its player left under `Despawn` or its retention record lapsed.

Write your own spawner

Your own spawner needs two things:

On `ClientAuthenticated`, spawn your prefab, get its `NetworkSystemObject`, and give the client control:

```
private void OnClientAuthenticated(Connection connection)
{
    // spawn your prefab, get its NetworkSystemObject, then:
    playerNetworkSystem.SetController(connection);
}
```

At startup, do the same for every connection that is already authenticated. `ClientAuthenticated` never fires for a client that authenticated before you subscribed, and an emulated or in-memory connection is authenticated as it connects without raising the event at all.

Client-predicted spawning in Unity

> **Driving the core API directly?** See Predicted spawn and despawn requests.

Predicted spawning is a Pro feature. A client creates a prefab instance the moment it decides to, instead of waiting a round trip for the server. The server still decides whether the object survives, and the client finds out which. The server's side, including its validators and every refusal reason, is on Predicted spawn and despawn requests.

Mark the prefab

Set **Predicted Spawn Policy** on the prefab's `NetworkSystemObject`. The values are flags and can combine:

- `None` (default): only the server creates and removes the object.
- `Spawn`: a client may create the object at once and ask the server afterwards.
- `Despawn`: a client may remove the object at once and ask the server afterwards, even an object the server created.
- `SpawnAndDespawn`: both of the above.
- `AllowUndeclaredComposition`: permits a predicted spawn whose component composition no call site in this build declares. Without it, such a spawn is refused.

Every peer reads the policy from its own copy of the prefab; it is never sent. The server judges a request against its own copy, never against what the client claims.

Ask for a predicted spawn

In the common case, the normal pool rent rents the object and asks in one step. When the object's opening values matter, ask in three steps:

Rent the object's system unstarted (`canStartSystem: false`).

Write everything the server needs to know about the object's birth, such as a shot's origin and velocity.

Call `NetworkSystemObjectPool.EnsureStartPredicted`. It sends a snapshot of the object as it is at that moment.

```
GameObject projectileObject = Instantiate(_projectilePrefab, _muzzle.position, Quaternion.
LookRotation(direction));
NetworkSystemObject marker = projectileObject.GetComponent<NetworkSystemObject>();

// Rented unstarted, so the shot can be written into the object before the request that ca
rries it is sent.
if (!NetworkSystemObjectPool.TryRent<NetworkSystem, PredictedProjectileComponent>(marker,
out NetworkSystem projectileSystem, canStartSystem: false))
{
    Destroy(projectileObject);
    return;
}

if (projectileSystem.TryGetComponent(out PredictedProjectileComponent projectileComponent)
)
{
    projectileComponent.Origin.Value = _muzzle.position.ToNative();
    projectileComponent.Velocity.Value = (direction * _projectileSpeed).ToNative();
}

if (NetworkSystemObjectPool.EnsureStartPredicted(marker, projectileSystem, out PredictedSp
awnRefusal predictedSpawnRefusal))
{
    // Started and sent. Wait for PredictedSpawnConfirmed or PredictedSpawnRejected.
    return;
}

// Refused locally, before anything was sent to the server.
Destroy(projectileObject);
```

A request is sent only when the system is unstarted and grouped, as a `canStartSystem: false` rent gives you, and the policy includes `Spawn`. A `false` return with a `PredictedSpawnRefusal` means this client declined before sending anything, for example because its lease of group identifiers from the server has run out. It is not the server's answer.

Watch the outcome

The server's answer arrives on the system the marker linked:

- `PredictedSpawnConfirmed`: raised when the server's snapshot for the object arrives. It fires before that snapshot is applied, so a handler still reads the values the client predicted.
- `PredictedSpawnRejected(PredictedSpawnRejectReason reason)`: raised when the server refuses the object, just before the local copy is torn down.
- `IsPredictedSpawnOrigin`: true on the client that created the object. It stays true after confirmation.
- `IsPredictedSpawnPending`: true while the spawn is still waiting on the server.

```
projectileSystem.PredictedSpawnConfirmed += OnShotConfirmed;
projectileSystem.PredictedSpawnRejected += reason => OnShotRejected(projectileObject, reas
on);
```

A rejected object appears and then vanishes a round trip later. Handle `PredictedSpawnRejected` with a small effect, a sound or a HUD message so it does not just pop out of existence. The demo project's predicted-projectile gun destroys a shot rejected for firing too fast with a log line explaining why.

If no answer arrives, the client despawns its copy and raises `PredictedSpawnRejected` with `NotPermitted`. It waits two round trips, but never less than half a second or more than three seconds. A

lost request is resent on the following ticks, so one lost packet does not lose it. If the server did admit the spawn and its answer arrives after the client gave up, the client receives the server's object as an ordinary spawn, so both peers still hold the same object.

Predict a despawn

With the `Despawn` flag, ask with `coreManager.SystemManager.EnsureStopPredictedSystem(networkSystemObject.System, out PredictedSpawnRefusal refusal)`.

- `PredictedDespawnPending` is raised when the request goes out. The object is parked, not despawned: the GameObject is deactivated, so its renderers and colliders leave the world and its behaviours stop. Member writes from any script are ignored, and it sends no client state or remote calls. It keeps receiving the server's state.
- `IsPredictedDespawnPending` is true until the server answers.
- If the server agrees, it removes the object on the tick the request rode with, and the GameObject returns to the prefab pool like any despawned object.
- `PredictedDespawnRejected(PredictedDespawnRejectReason reason)` is raised when the server refuses, after the same GameObject is reactivated where the server has it.
- If the request is lost, the client brings the GameObject back after at most three seconds and raises `PredictedDespawnRejected` with `NotPermitted`.

Never destroy a networked GameObject on a client. A destroyed object cannot come back if the server refuses, and the stop its destruction triggers is refused on a client with an error, leaving the system registered with nothing linked to it.

Prefab identity and the prefab collection

Rebuild the prefab collection after changing networked prefabs

You never assign prefab identity by hand. In the Nucleus window (**Tools > Nucleus**), select the **Build** tab and click **Rebuild** next to **Rebuild Network Prefab Collection** whenever you:

- add a networked prefab,
- delete a networked prefab, or
- change a prefab's asset bundle assignment.

The rebuild finds every prefab with a `NetworkSystemObject`, groups them by the asset bundle Unity has them assigned to, and writes one `NetworkPrefabCollection` per group. The collection with no asset bundle (the one under `Resources`) is the base build's; a collection assigned to asset bundle `x` holds the prefabs assigned to `x`.

The rebuild stamps each prefab's `NetworkSystemObject` with a local prefab id and its collection's bundle id. This `(bundle id, local id)` pair is the prefab's identity on every peer. Every machine that rebuilds from the same assets gets the same result, and a rebuild never renumbers a prefab that already has a unique id, so adding a prefab never changes the id of one that already shipped.

Fix a duplicated prefab id

Duplicating a prefab in the Project window copies its stamped id, so the copy has the same identity as the original and a spawn of one can build the other.

A player build fails while any networked prefab has an identity a rebuild would change, and the build error lists each problem:

- a missing base collection,
- two collections resolving to the same prefab bundle,
- a prefab missing from its collection,
- a prefab with no id, or with an id another prefab holds,
- a prefab stamped with a different bundle than its collection.

To find these without building, click **Validate** next to **Validate Network Prefabs** on the **Build** tab (see Developer tools in the editor).

To fix them:

Click **Rebuild** next to **Rebuild Network Prefab Collection**. The prefab the collection already lists keeps the shared id, and the copy gets a new one. A warning names the renumbered prefab, the prefab that held the id, and the new id.

Rebuild both the server and the client builds so every peer agrees.

If a clash reaches runtime, `NetworkPrefabRegistry.Register` keeps the first prefab registered under the identity and logs an error naming both prefabs.

Register a collection when its content loads

`NetworkPrefabCollection` is a `ScriptableObject` listing the networked prefabs one content shard ships. The rebuild writes these for you; you can also create one from **Assets > Create > Nucleus > Network Prefab Collection**.

- **Prefabs**: the networked prefabs this shard ships.
- **PrefabBundleId**: the bundle id this shard registers its prefabs under. Zero is the base build's shard.
- **BundleAssemblyName**: the name of the assembly whose `NetworkBundle` attribute supplies the bundle id. Empty means the base build's shard.

Call `RegisterAll()` when the shard's content loads, and `UnregisterAll()` when it unloads. A prefab with an unset id logs an error and is skipped. The base build ships bundle zero; every other shard needs a unique `NetworkBundle` id on its assembly so separately built content never collides.

Look up prefabs in the registry

`NetworkPrefabRegistry` maps a prefab's identity to the prefab a client instantiates. Collections fill it as they load and empty it as they unload.

- `EnsureClaimBundle(bundleId, collection)` claims a bundle id for a collection before its prefabs register. Claiming a bundle id another loaded collection owns is an error.
- `Register(bundleId, localId, prefab)` adds a prefab under its identity.
- `TryGet(bundleId, localId, out prefab)` finds the prefab registered for an identity.
- `TryGetPredictedSpawnPolicy(bundleId, localId, out predictedSpawnPolicy)` finds what a client may do to that prefab ahead of the server.
- `ReleaseBundle(bundleId, collection)` releases a bundle id when its collection unloads.

Pooling spawned objects

Install a pool to reuse spawned objects

By default every dynamic spawn instantiates a GameObject and every despawn destroys one. For objects that come and go often, such as projectiles or hit effects, assign `NetworkPrefabPool` to recycle them instead:

```
NucleusUnity.PrefabPool = new NetworkPrefabPool { ReturnCooldownSeconds = 1f };
```

The assignment works before or after the integration starts, and lasts until you change it. Assign `null` to go back to `DefaultNetworkPrefabPool`, which instantiates on rent and destroys on return.

Watch and tune NetworkPrefabPool

`NetworkPrefabPool` deactivates a returned instance and keeps it under an inactive pool root, keyed by prefab identity. A later rent for the same prefab reactivates the longest-parked instance, and instantiates only when none is eligible.

- `InstantiatedCount`: how many instances the pool has had to instantiate because nothing was parked. It settles at your peak number of instances alive at once.
- `ReusedCount`: how many rents reused a parked instance. It keeps climbing after `InstantiatedCount` settles.
- `ReturnCooldownSeconds`: how long a returned instance stays parked before it may be reused. `0` reuses it on the next rent and keeps the peak lowest. A longer cooldown keeps more instances out of use, so a burst instantiates more and `InstantiatedCount` settles higher.

Reset per-life state through SystemLinked, not Awake

A reused instance does not run `Awake` again. What carries over between lives:

- Its `NetworkSystemObject` keeps its `SystemLinked` and `SystemUnlinked` subscriptions and the `RequireSystem` declarations its scripts made. They clear only when the GameObject is destroyed, so the next life is served without declaring anything again.
- The `NetworkSystem` does not carry over. It despawns with the old life, and so do the remote-call handlers registered on it. Register them again against the next system through `SystemLinked`.

Reset per-life state in `SystemLinked` and `SystemUnlinked`, or in `OnSystemLinked` and `OnSystemUnlinked` on a `NucleusBehaviour`.

Write your own pool

Implement `INetworkPrefabPool`. Nucleus calls `Rent` when a dynamic spawn needs an instance and `Return` when a despawn releases one:

```
GameObject Rent(GameObject prefabGameObject, ushort prefabBundleId, ushort prefabLocalId,
Vector3 position, Quaternion rotation, Vector3 scale);

void Return(GameObject pooledGameObject, ushort prefabBundleId, ushort prefabLocalId);
```

- `prefabBundleId` and `prefabLocalId` are the prefab's identity, the same values a `NetworkSystemObject` exposes as `PrefabBundleId` and `PrefabId`. Key your pool on them: instances of different prefabs are never interchangeable.
- `Rent` must apply `position`, `rotation` and `scale` before the instance runs any of its own code, so a

script reading its transform in `Awake` or `OnEnable` sees the spawn pose rather than the prefab's.

Carrying and re-parenting objects

Driving the core API directly? See Parents, children and hierarchy.

Parent one object to another on the server

Call `EnsureSetSystemParent` on the server with the child's and the parent's `NetworkSystem`, not their GameObjects:

```
CoreManager.SystemManager.EnsureSetSystemParent(crateSystem, carrierSystem);
```

Pass `null` as the parent to detach the object to the root of its scene:

```
CoreManager.SystemManager.EnsureSetSystemParent(crateSystem, parentNetworkSystem: null);
```

It returns `true` when the parent changed, and `false` when the caller is not the server, either system is not started, or the pair is not allowed (see the limits below).

Every peer, the server included, moves the child's GameObject under the new parent's transform for you, so never call `transform.SetParent` yourself. The server also refreshes the object's interest.

Place the child where you want it

`NetworkTransform`'s **Transform Space** decides what the ride costs:

- **Local** (default): the child sends its offset inside the parent, which stops changing while it rides, so it costs nothing to send. Only the carrier's own transform is sent each tick.
- **World**: the child sends its scene pose and keeps paying for full pose replication while it rides.

Under Local, a re-parent carries the child's local values through instead of holding its world pose, so it lands wherever those values put it inside the new parent. To land it on a specific spot, such as a deck slot or a socket, set its world pose right after the call:

```
if (!CoreManager.SystemManager.EnsureSetSystemParent(crateSystem, carrierSystem))
    return;

crate.transform.SetPositionAndRotation(carrierTransform.TransformPoint(slotPosition), carr
ierTransform.rotation);
```

When detaching, read the world pose before the call and write it back afterwards, or the object jumps to where its local offset puts it from the scene root.

React to a parent change

`NetworkSystem` exposes the link:

- `ParentId`: the server's `Id` for the current parent, or unset when there is none.
- `HasParent`: true whenever `ParentId` is set, even if the parent has not arrived on this peer yet.
- `Parent`: the parent's `NetworkSystem`, or `null` when unparented or not yet arrived.
- `IsParentResolved`: true once `Parent` is not null.
- `ParentChanged`: a `NetworkSystem.ParentChangedHandler(uint previousParentId, uint currentParentId)` event, raised on the server as it makes the change and on each client as it applies it.

`Parent` may still be null inside the callback, so read `ParentId` or `HasParent` there:

```
private void OnSystemLinked(NetworkSystem system)
{
    _crateSystem = system;
    _crateSystem.ParentChanged += OnParentChanged;
}

private void OnParentChanged(uint previousParentId, uint currentParentId)
{
    // React to _crateSystem.HasParent here; _crateSystem.Parent may not be resolved yet.
}
```

Design around the hierarchy limits

- **Only a dynamically spawned prefab can be a child.** A scene object or a system with no GameObject cannot, though either can be a parent.
- **Hierarchies nest at most 8 deep**, counted from a root.
- **Despawning a parent despawns its children**, deepest first.

The core page lists every check the server makes before it accepts a parent.

Spawning and despawning objects

Driving the core API directly? See Starting and stopping systems.

Declare what an object needs with RequireSystem

From a component on a networked object, call `NetworkSystemObjectPool.RequireSystem<TSystem, TComponent0...>` once. It hands you the system as soon as one exists: at once if the object already holds a matching system, rented on the spot if this peer creates it, or when it arrives from the server. It also waits while the transport is still starting.

```
NetworkSystemObjectPool.RequireSystem<NetworkSystem, ColorPulseComponent>(
    this,
    OnSystemAcquired,
    OnSystemReleased,
    canStartSystem: true);

private void OnSystemAcquired(NetworkSystem networkSystem)
{
    // Read/write the linked system's components here.
}

private void OnSystemReleased(NetworkSystem networkSystem)
{
    // Reset per-life state; the system has unlinked.
}
```

The handlers are `NetworkSystemAcquiredHandler` and `NetworkSystemReleasedHandler`. Pass `null` for `OnSystemReleased` when there is nothing to reset. A `NucleusBehaviour` makes this call for you; see NucleusBehaviour: the script base class.

Rent directly with TryRent

`NetworkSystemObjectPool.TryRent<TSystem, TComponent0...>(contextComponent, out networkSystem, canStartSystem = true)` returns the system at once. Use it only when a script needs the system at a moment of its own choosing, such as a driver creating an object's system. Called too early, it fails while the transport is still starting and misses a system that already linked.

```
if (!NetworkSystemObjectPool.TryRent<NetworkSystem, UnityLocalTransformComponent>(this, ou
t NetworkSystem system))
    return;
```

It returns `true` once a system is linked to the object, and `false` on an error, which it logs. One call is correct on every peer:

- **On the server**, it returns a started system, which replicates to observers.
- **While a received spawn is being instantiated**, it returns the system built from that spawn.
- **On a client, for a scene object**, it returns a local, unstarted system that the server's spawn binds when it arrives.

Know the rent rules

- The rent finds the `NetworkSystemObject` on the context component or one of its parents.
- A second rent against the same marker gives a second system. The two share one `GroupId`, so a

receiver hosts both on one GameObject; see Several systems on one object.

- On a receiving client, a rent in a freshly instantiated prefab's `Awake` must ask for the same `TSystem` and component types the server spawned, or it logs an error and returns `false`.
- `TryRent` and `RequireSystem` take up to 31 component types (`TComponent0` through `TComponent30`), or none for a system that carries only a spawn payload.
- When the number of components comes from data, both take a count: `TryRent<TSystem, TComponent0>(this, componentCount, out networkSystem)` and `RequireSystem<TSystem, TComponent0>(this, componentCount, onAcquired)` give one system carrying `TComponent0` that many times. A system matches only when it holds exactly that many, all of that type. Reach each with `NetworkSystem.TryGetComponent(componentId, out component)`. This is how `NetworkTransform` puts every part it lists on one system.

A scene object needs no separate spawn call

An object already in a loaded scene is resolved by the same `TryRent` or `RequireSystem` call that spawns a dynamic prefab. It is built locally and bound once the server's spawn arrives.

Despawn on the server

```
_networkSystemObject.Despawn();
```

`NetworkSystemObject.Despawn()` stops every system linked to the object, replicates the despawn to observers, and returns the systems to their pool once the stop completes. A call with no linked system does nothing. Only the server may despawn a replicated system; a client's call is refused.

On receivers, a dynamic prefab's GameObject returns to the prefab pool, and a scene object's GameObject is destroyed.

A received spawn arrives already in place

A received spawn is instantiated already in place, never at the origin first:

- If its system carries a `UnityTransformComponentBase`, its `ReplicatedPosition`, `ReplicatedRotation` and `LocalScale` give the pose.
- If it carries no transform component, the prefab's own transform is used, unless the marker's **Auto Transform Payload** is on, in which case it arrives at the server's parent and pose.

A body whose position its controller drives, such as a `NetworkRigidbody` another peer simulates, is the exception. The controlling peer builds its copy from the prefab and asserts the prefab's pose back within a tick, overwriting a spawn point the server chose. Derive or publish the spawn point so the controlling peer can place itself.

Send one-time data with the spawn

Data an object needs only when it arrives, such as a random seed or a loot table, goes in a spawn payload rather than a member. Pick a payload type from **Payload** in the marker's **Spawn Payload** section, or declare one from code with `NetworkSystem.SetSpawnPayload`. Starting and stopping systems explains how payloads work and who may declare them.

An object that needs only a payload can require a system with no components: `NetworkSystemObjectPool.RequireSystem<NetworkSystem>(this, OnSystemAcquired)`.

On a client, a scene object is bound after it has already linked. Read its payload in the payload's own `ReadSpawnPayload`, or from `SystemManager.SystemStarted` or a system watch, not in

`OnSystemLinked`.

What a system is

> **Driving the core API directly?** See What a system is and the NetworkSystem reference, which list a system's identifiers, states and role reads.

A `NetworkSystem` is what Nucleus replicates. The system is the identity and lifecycle; the replicated data lives in the `NetworkComponent` instances attached to it, up to 31 per system (`NetworkSystem.MaximumComponentCount`).

A GameObject links to its systems through NetworkSystemObject

In Unity, the NetworkSystemObject component links a GameObject to its systems. It is a bridge, not the system itself. One GameObject can link several systems, each with its own `Id` and lifecycle. A system does not need a GameObject: a purely logical system, such as matchmaking state or a game rule tracker, can start, replicate and stop without one.

Read a system's components

`TryGetComponent<T0>` returns the first component of a type, and `ComponentCount` says how many are attached:

```
if (system.TryGetComponent<TransformComponent>(out TransformComponent transform))
{
    // transform is one of the components attached to this system.
}

int attached = system.ComponentCount;
```

Each component has a component Id: the order it was added in, starting at zero, and the same on every peer. When a system carries one type several times, as a `NetworkTransform` replicating several parts does, reach each instance by its Id:

```
for (byte componentId = 0; componentId < system.ComponentCount; componentId++)
{
    if (system.TryGetComponent(componentId, out TransformComponent part))
    {
        // part is the component added at position componentId, the same one on every peer
.
    }
}
```

Check the state before reading data

`IsStarted` is true only while `NetworkSystem.State` is `Started`. A start accepted late in a tick stays `Starting`, with its `Id` unset, until the next tick.

```
if (system.IsStarted)
{
    // safe to read replicated component data here.
}
```

`IsServerStarted`, `IsClientStarted` and `IsHostStarted` answer whether the server, client or host is running on this peer, not whether this peer controls the object. For control, use `IsController(ControllerType)`.

Blitz Relay Transport component

Driving the core API directly? See The relay transport.

`BlitzRelayTransport` carries a session through a relay, so two peers behind routers or NAT can reach each other without either one listening on a port. Add it to the `UnityCoreManager`'s GameObject, and `UnityTransportManager` adds it to the `CoreManager` with the inspector's settings.

Before you use it:

- It needs Pro. The relay integration does not compile against a Free `Nucleus.dll`.
- Add the `BLITZ_RELAY` scripting define. The component and its editor compile only behind it.
- Run a Blitz Relay service at the address and port below, and a directory service too if Host Migration is on. This component dials them; it does not start them. See Hosting and joining a relay session.

Point it at the relay

Field	Default	Purpose
Address	`127.0.0.1`	Address of the Blitz Relay carrying this session.
Port	`7770` (`RelayTransport.DefaultRelayPort`)	Port the relay listens on.
Connection Key	empty	Key the relay admits peers with. Set by whoever runs the relay, never by a player.

Set up the room

Field	Default	Purpose
Room Code	empty	Room to join. Leave empty to host, or when joining by session id instead.
Maximum Clients	`16` (`RelayTransport.DefaultMaximumClients`)	How many clients a room created by this peer holds. The relay refuses a room without a real number, so this cannot mean unlimited.
Transmission Unit	`1200`	Largest datagram to build before it is split. Nucleus already leaves room for the relay's routing, so it needs no adjustment for the relay.

Survive losing the host

Field	Default	Purpose

Host Migration	`true`	Registers the session with a directory so it survives losing whoever is hosting it.
Directory Address	`127.0.0.1`	Address of the directory the session registers with. Shown only while Host Migration is on.
Directory Port	`47778`	Port the directory listens on. Shown only while Host Migration is on.

Turning Host Migration on sets `ClientManager.DisconnectResetMode` to `RetainReceivedWorld`, so a client keeps its received world when its link drops instead of being reset to an empty one.

This is the only host migration Nucleus has: a client keeps its world across a host loss, and a peer can rejoin a moved session by session id instead of a room code. Nothing beyond this page and Hosting and joining a relay session exists yet.

Read the room and session while playing

While the game plays, the inspector shows two read-only fields: **Current Room**, the room this peer is hosting or has joined, and **Session Id**, in hex. Session Id stays blank unless Host Migration is on and a session has registered with a directory.

Building a headless Unity server

Driving the core API directly? See Running a dedicated .NET server.

Set up the server scene

Build with Unity's Dedicated Server target.

Put a `UnityCoreManager` in the scene that loads on launch. It adds a `UnityTransportManager` if the scene has none.

Set **Automatic Start Mode** on the `UnityTransportManager` to `Server`, or leave it at `None` and start the server from your own script if it reads launch arguments (below).

Apply launch arguments before the server starts

Nothing in the Unity integration reads the process's launch arguments for you. Call `ApplyCommandLineArguments()` on each transport from your own script, before the transport connects.

Do not combine that script with **Automatic Start Mode** set to `Server`. `UnityTransportManager` starts from its own `Start`, and Unity does not order two components' `Start` calls, so the server can come up before your arguments are applied. Leave **Automatic Start Mode** at `None` and start the server after applying them:

```
using Nucleus.Transports;

public class ServerBootstrap : MonoBehaviour
{
    private async void Start()
    {
        UnityTransportManager transportManager = GetComponent<UnityTransportManager>();

        await transportManager.EnsureAddedAsync();

        foreach (Transport transport in transportManager.Transports)
            transport.ApplyCommandLineArguments();

        await transportManager.StartServerAsync();
    }
}
```

`StartServerAsync` calls `EnsureAddedAsync` itself; the explicit call is there so `Transports` is filled before the loop.

`ApplyCommandLineArguments()` with no arguments reads the process's command line (`CommandLineArguments.Process`). The port flag is `-port <value>`, and any flag your transport recognizes works as `-flag value` or `-flag=value`.

Read logs from the player log file

A headless server has no window, and `NetworkDiagnosticsHud` is demo code that does not ship. Nucleus logs through `UnityEngine.Debug`, so its output lands in the standalone player's log file.

Shut the transports down early if you need to

`UnityTransportManager.ShutdownAll()` removes and shuts down every transport the manager added. It already runs when the component is destroyed, so call it yourself only to shut down earlier, such as

from a signal handler that also tears down other state.

Skip Unity if the server does not need it

If the server does not need Unity's engine loop, physics or other engine features, a plain .NET host avoids the editor, the Dedicated Server build and this bootstrap. See Running a dedicated .NET server.

Transports, Connectivity and Hosting

Pages in this category:

- Synapse Transport component
- Transport Manager component
- How transports work
- Blitz Relay Transport component
- Choosing a hosting topology
- Building a headless Unity server
- Connecting across networks
- Simulating a bad connection in the editor
- Hosting and joining a relay session

Hosting and joining a relay session

Driving the core API directly? See The relay transport.

Run the relay and the directory

Nucleus does not include either service, so run both yourself:

- **The relay**, listening on UDP 7770, with a connection key that matches **Connection Key** on `BlitzRelayTransport`.
- **The newfarm directory**, listening on UDP 47778, which resolves session ids and elections.

Their addresses and ports are fields on `BlitzRelayTransport`, defaulting to `127.0.0.1` and the ports above.

Host a session

Add `BlitzRelayTransport` to the `UnityCoreManager`'s GameObject.

Call `StartHostingSessionAsync()`. It opens a session, hosts it and registers the room the relay names.

When it returns `true`, give joining players the `RoomCode` or the `SessionId`.

```
bool isHosting = await blitzRelayTransport.StartHostingSessionAsync();

string roomCode = blitzRelayTransport.RoomCode;
ulong sessionId = blitzRelayTransport.SessionId;
```

`RoomCode` reads `RelayTransport.HostedRoomCode` on the core transport once one exists, and the inspector's **Room Code** otherwise. `SessionId` reads `SessionDirectoryUnset` (`0`) when this peer holds no session, which is the case while `IsHostMigrationEnabled` is off or before hosting starts.

Join a session

- **By room code:** set `RoomCode` before connecting. The peer joins that room and nothing else.
- **By session id:** call `JoinSessionAsync(sessionId)`. It finds the session and joins whatever room it is in now.

```
bool isJoined = await blitzRelayTransport.JoinSessionAsync(sessionId);
```

Hand out a session id if the host can change

A room code names one room on the relay. Every handover moves the session to a new room, so a room code goes stale when the host changes. A session id is registered with the newfarm directory and survives a handover, because a rejoining peer asks the directory, not the relay.

Give players a room code only for a session that will never change host. Otherwise give them the session id.

Hand hosting to another peer

To stop hosting without ending the match, call `TrySurrenderHostingSession()`. The session passes to whoever the directory elects next, and play does not stop.

```
bool isSurrendered = blitzRelayTransport.TrySurrenderHostingSession();
```

The component runs the handover itself through `Migration`, a `NewfarmHostMigration` it drives from its own `Update`. Once your game starts hosting or joins, election, adoption and finding the session's new

location need nothing more from it. The elected peer adopts the world it was receiving and hosts it. Other peers rejoin the new host's room through the session id, the same way they joined.

Keep Host Migration on

`IsHostMigrationEnabled` (on by default) is what creates `Migration`. With it off, `StartHostingSessionAsync`, `JoinSessionAsync` and `TrySurrenderHostingSession` fail and log an error telling you to drive the `CoreManager` directly.

Turning it on also sets `ClientManager.DisconnectResetMode` to `RetainReceivedWorld`, so a client keeps the world it received when its link to the server drops. The elected peer needs that world to adopt it. `RetainReceivedWorld` can be set in a Free build, but adopting the retained world needs Pro, so the Blitz Relay and Newfarm Migration assemblies do not compile against a Free `Nucleus.dll`.

`SystemManager.RespawnWorldOnReconnectEnabled` changes only what happens when a client reconnects. The world is still kept when the link drops, and an elected peer still adopts it, but a peer that rejoins drops what it kept and rebuilds it from the new server instead of being relinked to it. That costs a spawn per object, and never keeps anything the new server does not know about.

Simulating a bad connection in the editor

> **Driving the core API directly?** See Simulating latency, jitter and loss.

Play mode runs over loopback, which is instant and lossless. To make it behave like a real connection, have the transports simulate network conditions. What each field means, including `Seed`, is on Simulating latency, jitter and loss.

Set the conditions in the inspector

1. Select the `UnityTransportManager`.
2. Tick the checkbox on the **Simulated Network Conditions** section header.
3. Fill in **Latency Milliseconds**, **Jitter Milliseconds**, **Packet Loss Chance**, **Out Of Order Chance** and **Duplicate Chance**. Synapse ignores **Duplicate Chance**, and a transport that supports none of them logs a warning.

The conditions are set on every transport when the transports are added, just before they start, and cleared when the box is unticked. This works with any **Automatic Start Mode**.

Synapse sends its connection setup packets without simulated delay or loss, so a connection always opens however bad the conditions are. A connect that succeeds does not show that the handshake copes with that link.

Set the conditions from code

`SetNetworkConditions` on `UnityTransportManager` sets every transport, and `SetNetworkConditions<Synapse>` sets only the Synapse ones. Pass null to go back to a clean connection. Conditions set from code replace the inspector's.

The transports must exist before they take conditions. To have the conditions from the first packet:

1. Set **Automatic Start Mode** on `UnityTransportManager` to `None`.
2. Add the transports with `EnsureAddedAsync`.
3. Set the conditions.
4. Start the network.

```
using Nucleus.Integrations.Unity.Managers.Transports;
using Nucleus.Transports;
using UnityEngine;

public class ConnectionSimulator : MonoBehaviour
{
    [SerializeField]
    private UnityTransportManager _unityTransportManager;

    private async void Start()
    {
        await _unityTransportManager.EnsureAddedAsync();

        _unityTransportManager.SetNetworkConditions(new NetworkConditions
        {
            LatencyMilliseconds = 100,
            JitterMilliseconds = 40,
            PacketLossChance = 0.05f,
        });

        await _unityTransportManager.StartHostAsync();
    }
}
```

Use `StartServerAsync` or `StartClientAsync` in place of `StartHostAsync` for the other roles.

To read what is set, call `GetNetworkConditions<Synapse>()`, or `GetNetworkConditions()` for a pooled list with one entry per transport, which you return to `ListPool` when done.

Change the conditions while playing

A Synapse transport applies new values at once, even on a link that is already connected, so you can turn conditions on, change them or turn them off while playing.

Each editor sets its own conditions

Conditions belong to the transports in one editor. In a ParrelSync pair each editor is its own process, so setting conditions in the host editor does nothing to the clone, and you can make only the clone's connection bad.

Each peer applies loss to the packets it sends. A request and its reply each roll separately, so a round trip survives only when both directions do.

Check the effect on the Connection

While playing, read the measured values from `Connection`:

- `Connection.RoundTripTimeMilliseconds`: the smoothed round trip time.
- `Connection.RoundTripTimeDeviationMilliseconds`: the jitter, meaning how far samples stray from the smoothed average.
- `Connection.PacketLossPercentage`: the measured loss.

They are measured on the live connection, not echoed from your settings, so they show whether the conditions are taking effect.

Test at more than one setting

Set jitter as well as latency: a steady 100 ms and a link swinging between 20 ms and 180 ms average the

same but feel nothing alike, and some bugs only show when timing varies. Try loss of 1 to 5% for typical broadband, and 10 to 20% to stress recovery.

The simulation delays, drops and reorders packets inside the process. It does not model a slow upload beside a fast download, or a real path that silently drops packets over some size, so also test over an actual network.

Synapse Transport component

Driving the core API directly? See The Synapse transport.

`SynapseTransport` configures Synapse, the built-in UDP transport, from the inspector. Add it to the `UnityCoreManager`'s GameObject. If no transport component is present, `UnityTransportManager` adds one with default settings.

Set where clients connect

Under **Connection**:

- **Port** (`ushort`, default `7777`): the port the server listens on and the client connects to.
- **Remote Host** (`string`, default `"127.0.0.1"`): the host name or IP address a client connects to. The server ignores it.
- **Bind Address** (`string`, default empty): the local address the server and client bind to, an IP address or `localhost`. Empty binds every interface.
- **Maximum Transmission Unit** (`ushort`, default `1200`): the largest packet built before it is split.

Set the timeouts

Under **Timeouts**:

- **Idle Timeout Seconds** (`ushort`, default `15`): how long a connected peer waits without hearing from the other side before it drops the link. The least applied is one second.
- **Connecting Timeout Seconds** (`ushort`, default `10`): how long a client waits for the server to answer its connect before giving up and returning to `Disconnected`. Zero uses **Idle Timeout Seconds**.
- **Host Resolve Timeout Seconds** (`ushort`, default `5`): how long a client waits for a host name in **Remote Host** to resolve before it stops connecting. Unused for an IP address.
- **Resolved Address Cache Seconds** (`ushort`, default `60`): how long a resolved host name is reused before it is looked up again. Zero looks it up on every connect.

Set the hosting options

Under **Hosting**, **Maximum Connections** (`uint`, default `0`) is the most clients the server accepts at once. Zero accepts any number.

Host Pairing (default `HostPairing.Local`) sets how the host's server reaches its own client half:

- `HostPairing.Local`: the two halves are paired in process and never exchange a datagram, so the host's own client is connected at once. The host's own half then tests nothing about the socket; a remote client of the same host still uses it.
- `HostPairing.Endpoint`: the two halves talk over the socket like any other client. This is the default of the core `Synapse` transport.

Pairing only engages for a loopback address. With **Remote Host** pointing at another machine, it changes nothing.

Limit what each client may send

Under **Client Limits**, the server holds each client's datagrams to limits that are on by default. See Securing a server in Unity for every field and its default.

Set values from a script

`Port` (`ushort`), `RemoteHost` (`string`) and `HostPairing` are properties over the inspector fields.

Transport Manager component

Driving the core API directly? See TransportManager.

`UnityTransportManager` adds your transport components to the core `TransportManager` and starts them. `UnityCoreManager` adds one if you have not.

Set the fields

Field	Default	Description
Default Channel	`Channel.Unreliable`	The channel a send takes when it names none.
Automatic Start Mode	`None`	What to start as when play begins: `None`, `Server`, `Client`, `Host` or `Auto`. `Auto` starts as a client in a ParrelSync clone or when another process already holds the server port, and as a host otherwise. The core manager has no equivalent.
Simulated Network Conditions	off	Latency, jitter and loss to simulate on every transport. See [Simulating a bad connection in the editor](./simulating-bad-connections-unity.md).

Transport components are found for you

`UnityTransportManager` adds every `NetworkTransport` component on its GameObject and its children to the core manager, in the order Unity finds them. If there are none, it adds a `SynapseTransport`. Each `NetworkTransport` is an inspector wrapper that adds and configures one core `Transport`.

Start and stop from a script

```
UnityTransportManager transportManager = GetComponent<UnityTransportManager>();

await transportManager.StartServerAsync();
await transportManager.StartClientAsync();
await transportManager.StartHostAsync();

// Adds every discovered transport to the core manager without connecting.
// StartServerAsync/StartClientAsync/StartHostAsync call this for you.
await transportManager.EnsureAddedAsync();

// Removes and shuts down every transport this component added.
transportManager.ShutdownAll();

IReadOnlyList<Transport> transports = transportManager.Transports;
```

`EnsureAddedAsync` adds transports only once; later calls return at once. `ShutdownAll` also runs when the component is destroyed.

For anything else, use the core manager through `NucleusTransportManager`.

Bundle Ids for Unity Assemblies

Driving the core API directly? See Type Identity on the Wire: Bundles and the Registry.

Declare the bundle

Two separately compiled assemblies that both declare no bundle collide, because both default to bundle zero and each numbers its own types from the start. If you build a content shard as its own asmdef, give it its own bundle id before you ship it alongside anything else:

Add a C# file inside the shard's asmdef. It must be that asmdef, not a shared assembly the shard references, because the prefab collection takes its bundle id from the assembly it names. Putting the attribute in a shared assembly pulls every prefab in it into one bundle.

Declare the bundle at assembly scope:

```
using Nucleus.Serializers;

[assembly: NetworkBundle(ContentBundleShard.BundleId)]
```

The gated-prefabs content shard demo does this, with a class that only holds the id:

```
public static class ContentBundleShard
{
    public const ushort BundleId = 1;
}
```

Assign shard ids from 1 upward: 1, 2, 3.

These bundle ids are already taken

- `NetworkTypeRegistry.FrameworkBundleId` (`ushort.MaxValue`) is the framework's own.
- `FrameworkBundleId - 1` is the `Nucleus.Integrations.Unity` assembly's.
- `FrameworkBundleId - 2` is reserved by Nucleus.
- Zero is the default for any Unity assembly that declares no `[assembly: NetworkBundle(...)]`.

The prefab collection's bundle id is filled in for you

`NetworkPrefabCollection` has its own `PrefabBundleId` and `BundleAssemblyName` fields. They feed `NetworkPrefabRegistry`, which resolves which prefab a spawn names, not `NetworkTypeRegistry`, which the `[assembly: NetworkBundle(...)]` attribute feeds. The editor fills them in from the collection's own assembly; do not set them yourself.

A collision logs an error and drops the type

If two types land on the same `(bundleId, localId)` pair, `NetworkTypeRegistry` refuses the second one and logs an error naming the bundle and local id. That type is missing from the registry, so spawning or replicating it fails. If a component or system never spawns and the console shows that error, check for a missing or duplicated `[assembly: NetworkBundle(...)]` first.

Code Generation in Unity

Driving the core API directly? See Wiring the Source Generator into a .NET Project.

Keep the four DLLs in Assets/Nucleus

`Assets/Nucleus` needs these four files side by side:

- `Nucleus.dll`, the engine
- `CodeBoost.dll`, which the engine depends on
- `SynapseSocket.dll`, the transport
- `Nucleus.CodeAnalysis.SourceGenerators.dll`, the source generator

Unity runs the generator only while its `.meta` file carries the `RoslynAnalyzer` label:

```
labels:
- RoslynAnalyzer
```

When you update the generator, replace the DLL and leave its `.meta` alone. Replacing the `.meta` too, or deleting and re-adding the DLL, drops the label, and Unity silently compiles without the generator.

Reference Nucleus from your component's assembly

The generator skips any assembly that does not reference `Nucleus.dll`.

- **Scripts with no asmdef** compile into `Assembly-CSharp`, which picks up the reference automatically.
- **Scripts under your own `.asmdef`** need `Nucleus.Integrations.Unity` (or another assembly that references `Nucleus.dll`) in that asmdef's `references` array.

Write C# 9 in Unity scripts

Unity compiles with `LangVersion` 9, so your components and anything else Unity compiles are limited to C# 9: namespaces need braces (no `namespace Foo;`), and there are no collection expressions (`[1, 2, 3]`).

If a component replicates nothing, check the generator

If the generator did not run over your assembly, a `partial NetworkComponent` still compiles but is never registered and never replicates. There is no error or warning. Check that the generator's `.meta` still carries `RoslynAnalyzer`, and that the component's assembly references `Nucleus.dll`.

Generator errors appear in the Unity console with the same ids the .NET build uses. See Generator Errors and Warnings.

Generator Errors and Warnings

The source generator reports in the Unity console alongside ordinary compile errors, or in a plain .NET project's build output. Each diagnostic has an id of the form `FAMILY###` and a message. The id tells you which stage raised it; the message tells you what to fix, so search for the message text you see.

Each id family is one stage

Id	Severity	Stage
`NETWORKTYPEFINDER00`	Warning	Type discovery
`NETWORKTYPEFINDER01`	Error	Type discovery
`GENERATION000`	Error	Generation: invalid type
`GENERATION001`	Error	Generation: invalid Roslyn type
`GENERATION002`	Error	Generation: comparer or operation stage
`GENERATION003`	Warning	Generation: comparer or operation stage (declared, not currently raised)
`GENERATION004`	Warning	Generation: two networked messages or calls hash to the same wire id
`SERIALIZERS000`	Error	Serializer generation
`SERIALIZERS001`	Warning	Serializer generation

`NETWORKTYPEFINDER*` fires first, while the generator decides which types and members are networked. `GENERATION*` and `SERIALIZERS*` fire afterward, while it generates code for them.

`GENERATION004` does not stop the rest of the assembly from generating. While two types collide, the engine sends neither, registers neither and hands neither to a handler. Rename one of them.

The commonest cause is a member with no serializer

`SERIALIZERS001` is by far the most common, with a message like:

> `[Type.Member] Type [SomeType] for [Write/Read] SerializeMethod could not be found; this value will not be networked.`

The generator found no serializer for that member's type. Fix it one of two ways:

- **Marker struct.** If the type lives in an assembly you do not control (a `System.Numerics` type, a third-party math type), add it as a field on a marker struct carrying `[NetworkType]`. See Reaching Types You Cannot Annotate.
- **Hand-written serializer.** If the type needs custom read and write logic. See Writing a Custom Serializer.

A `UnityEngine` type such as `Vector3` or `Quaternion` does not raise this warning, because it builds as a plain struct. Declare the `System.Numerics` equivalent anyway; see Unity Types and System.Numerics.

`SERIALIZERS000` is the error-level version: a required serializer is missing or malformed so badly the generator cannot continue, for example `A [Write/Read] serializer could not be found for`

`[Type].` or a type over the maximum networked member count. Fix it the same way.

The next most common is a badly declared type

`NETWORKTYPEFINDER01` covers structural problems in how a networked type is declared, such as unsupported type arguments or a member type discovery does not recognize.

A networked component or system that is not `partial`, or is nested in a way the generator cannot reach, fails with an ordinary C# compiler error about a generated file instead of a Nucleus id. Check that the type and every type containing it is `partial`, and check its nesting.

Read the top diagnostic, not the longest list

An early error stops later stages for that type, so one `NETWORKTYPEFINDER01` error often brings a long tail of `SERIALIZERS001` warnings. Fix the first diagnostic for a type and rebuild before working through the rest.

A generator diagnostic is a compile-time problem, never a runtime one

If the build is clean but replicated values are wrong, systems never converge, or packets look malformed at runtime, the problem is not in the generator. See Diagnosing desync.

Serialization, Codegen and the Wire

Pages in this category:

- Code Generation in Unity
- Generator Errors and Warnings
- Bundle Ids for Unity Assemblies
- Unity Types and System.Numerics

Unity Types and System.Numerics

Declare members in System.Numerics, not UnityEngine

Declare a networked member as `NetworkMember<System.Numerics.Vector3>`, never `NetworkMember<UnityEngine.Vector3>`. Nucleus applies its vector and rotation handling, such as interpolation and Divine projection, only to the `System.Numerics` types. A UnityEngine member still builds and replicates without any diagnostic, but as a plain struct with none of that handling, and a `UnityEngine.Quaternion` member sends more than it needs to.

Alias the `System.Numerics` type at the top of the file to avoid clashing with `UnityEngine.Vector3`, as the shipped `UnityNavMeshAgentComponent` does:

```
using NumericsVector3 = System.Numerics.Vector3;

public partial class UnityNavMeshAgentComponent : NetworkComponent
{
    public readonly NetworkMember<NumericsVector3> NextPoint = new(CompressionLevel.Aggres
sive);
    public readonly NetworkMember<NumericsVector3> PreviousPoint = new(CompressionLevel.Ag
gressive);
}
```

Convert at the Unity boundary

`UnityConversionExtensions` converts with `ToUnity()` (native to Unity) and `ToNative()` (Unity to native), for `Vector2`, `Vector3`, `Vector4`, `Quaternion`, `Plane` and `Matrix4x4`. Convert only where you read a member into a `Transform` or other Unity API, or write one back:

```
public Vector3 ReplicatedNextPoint
{
    get => NextPoint.Value.ToUnity();
    set => NextPoint.Value = value.ToNative();
}
```

Keep the `NetworkComponent` and its `NetworkMember<T0>` fields native, and the `MonoBehaviour`, `Transform` and other Unity APIs in Unity types.

These types need no attribute

These already have serializers and work in a `NetworkMember<T>` with no attribute of your own:

- The `System.Numerics` vectors `Vector2`, `Vector3` and `Vector4`
- The CodeBoost types `Vector2Int`, `Vector3Int32`, `Vector4Int` and `UnifiedColor`
- Any struct you declare made entirely of serializable fields, as long as a `NetworkMember<T>` reaches it

A member type with no serializer is reported at build time as `SERIALIZERS000` or `SERIALIZERS001` (see Generator Errors and Warnings). For a third-party type you cannot put `[NetworkType]` on, see Reaching Types You Cannot Annotate.